Portolan

Générateur de schémas de baies informatiques

Documentation technique

Version 0.1.0

Table des matières

I Conception **12**

1 Introduction **13**

1.1 Objet et lecteurs . . . 13
1.2 Organisation du document . . . 13
1.3 Documents de référence . . . 13
1.4 Lire une entrée de la référence . . . 14
1.5 Tenir la documentation à jour . . . 14
1.6 Conventions du code . . . 15

2 Architecture **16**

2.1 Vue d'ensemble . . . 16
2.2 Technologies . . . 17
2.3 Organisation des dossiers . . . 17
2.4 Couches et dépendances . . . 18
2.5 Flux de données . . . 19
2.6 Principes transverses . . . 19
2.7 Le build . . . 20

3 Modèle de données **21**

3.1 Principes . . . 21
3.2 Vue d'ensemble d'un projet . . . 22
3.3 Fabriques . . . 26
3.4 Index et résolution des références . . . 26
3.5 Validation et invariants . . . 27
3.6 Fonctions annexes . . . 28
3.7 Format de fichier et version . . . 29
3.8 Ajouter un champ au modèle . . . 29
3.9 Tests . . . 30

4 Règles métier **31**

4.1 Principes communs . . . 31
4.2 Placement dans une armoire . . . 32
4.3 Géométrie d'une armoire . . . 34
4.4 Placer à la souris et au clavier . . . 35

4.5 Liaisons . . . 36
4.6 Anomalies . . . 37
4.7 Ports ajoutés à un équipement . . . 37
4.8 Disposition des pages . . . 38
4.9 Limites connues . . . 38
4.10 Ajouter ou modifier une règle . . . 39

5 Pages, vues, cartouche et onglet Schéma 40
5.1 Repères et unités . . . 40
5.2 Format de page et zone de dessin . . . 41
5.3 Vues . . . 41
5.4 Cartouche (PAG-09, PAG-10) . . . 43
5.5 Onglet Schéma (PAG-07, PAG-08) . . . 43
5.6 Sélection au rectangle (ERG-02) . . . 45
5.7 Recherche globale (ERG-05) . . . 45
5.8 Éditeur de machine (EDM-03 à EDM-07) . . . 45
5.9 Bibliothèques dans le modèle . . . 46
5.10 Modifier ce domaine . . . 47
5.11 Points d'attention . . . 47

6 Bibliothèques, équipements intégrés et démarrage 48
6.1 Équipements, machines et paramètres . . . 48
6.2 La bibliothèque par défaut . . . 52
6.3 La page des bibliothèques de l'équipe . . . 52
6.4 Démarrage de l'application . . . 53
6.5 Internationalisation et textes . . . 55
6.6 Comment faire . . . 55
6.7 Limites connues . . . 56

7 Routage des câbles 57
7.1 Vue d'ensemble . . . 57
7.2 Dans la baie : goulottes, couloirs et pistes . . . 58
7.3 Hors de la baie . . . 62
7.4 Renvois (ROU-08) . . . 63
7.5 Câbles masqués (PAG-15) . . . 63
7.6 Mode repères (ROU-09) . . . 63
7.7 Retouche manuelle et points rattachés (ROU-06, ROU-07, ROU-16) . . . 64
7.8 Faisceaux et repères de câbles (ROU-10, ROU-11) . . . 65
7.9 Croisements et ponts (ROU-12, ROU-15) . . . 66
7.10 Orchestration d'une page . . . 67
7.11 Invariants . . . 67
7.12 Performances . . . 68
7.13 Tests et captures . . . 68

7.14 Modifier une règle . . . 68
7.15 Limites connues . . . 69

8 Rendu SVG **70**
8.1 Principes . . . 70
8.2 Repères et unités . . . 71
8.3 Arbre des composants d'une page . . . 71
8.4 L'armoire . . . 72
8.5 La face d'un équipement . . . 73
8.6 Les ports . . . 74
8.7 La couche des câbles . . . 75
8.8 Éléments du schéma . . . 76
8.9 Cartouche, légende, révisions . . . 76
8.10 Planche d'étiquettes . . . 76
8.11 Images et texte : les deux contextes . . . 77
8.12 Ce que partagent l'écran et les exports . . . 77
8.13 Modifier un dessin . . . 77
8.14 Jeux d'essai . . . 78
8.15 Limites connues . . . 79

9 Texte : polices, mesure et découpage **80**
9.1 Les exigences et leurs conséquences . . . 80
9.2 La police embarquée . . . 81
9.3 Mesurer un texte . . . 81
9.4 Découper un texte en lignes . . . 82
9.5 Les polices dans les exports . . . 83
9.6 Lire et alléger une police TrueType . . . 84
9.7 Pourquoi ce dossier ne dépend ni de React ni du DOM . . . 85
9.8 Tests et captures . . . 85
9.9 Modifier le texte . . . 85

10 Fichiers, stockage et import **87**
10.1 Vue d'ensemble des flux . . . 87
10.2 Le format de fichier . . . 88
10.3 Lecture défensive d'une archive . . . 90
10.4 Schémas JSON publiés . . . 91
10.5 Versions du format et migrations . . . 92
10.6 Stockage dans le navigateur . . . 93
10.7 Ouvrir et enregistrer des fichiers . . . 94
10.8 Images importées . . . 95
10.9 Import tableur simplifié . . . 95
10.10 Tests . . . 98

11 Exports **99**
11.1 Principe : un seul dessin, plusieurs formats . . . 99
11.2 Contrat commun . . . 100
11.3 Dessin d'une page ou d'une vue (SVG) . . . 101
11.4 PNG . . . 103
11.5 PDF . . . 103
11.6 Tableur . . . 105
11.7 Étiquettes de câbles . . . 106
11.8 Limites connues . . . 107
11.9 Faire évoluer les exports . . . 107

12 État, actions et commandes **110**
12.1 Vue d'ensemble . . . 110
12.2 L'éditeur et l'historique . . . 111
12.3 Les actions . . . 112
12.4 Le projet d'exemple . . . 114
12.5 Le registre des commandes . . . 115
12.6 Limites connues et conseils . . . 117

13 Interface utilisateur **119**
13.1 Démarrage et contextes React . . . 119
13.2 Arbre des composants . . . 120
13.3 L'état de l'interface . . . 121
13.4 Commandes, menus et raccourcis . . . 123
13.5 La zone de dessin . . . 123
13.6 Flux d'interaction . . . 124
13.7 Panneau des propriétés et champs . . . 125
13.8 Dialogues . . . 126
13.9 Textes et internationalisation . . . 126
13.10 Accessibilité et tests de bout en bout . . . 127
13.11 Limites connues et conseils . . . 127

14 Tests **128**
14.1 Vue d'ensemble . . . 128
14.2 Tests unitaires (Vitest) . . . 128
14.3 Tests de bout en bout (Playwright) . . . 130
14.4 Règle : ne jamais affaiblir un test . . . 132
14.5 Définition de fin . . . 132

15 Intégration continue et publication **134**
15.1 Principe : un dépôt privé, un site public non référencé . . . 134
15.2 Intégration continue : `ci.yml` . . . 135
15.3 Publication : `publish.yml` . . . 135
15.4 Construction du site : `npm run site` . . . 136
15.5 Contrôle du site : `npm run check:site` . . . 138
15.6 Hors ligne : service worker et cache . . . 138
15.7 Politique de sécurité du contenu (CSP) . . . 139

16 Outils du dépôt **140**
16.1 Scripts npm et outils . . . 140
16.2 `tools/build-doc` . . . 141
16.3 `tools/build-example-library` . . . 141
16.4 `tools/build-json-schema` . . . 142
16.5 `tools/build-site` . . . 142
16.6 `tools/build-tech-doc` . . . 142
16.7 `tools/check-doc` . . . 143
16.8 `tools/check-neutral` . . . 143
16.9 `tools/check-site` . . . 143
16.10 `tools/check-tech-doc` . . . 144
16.11 `tools/json-schema` . . . 144
16.12 `tools/libavoid-no-eval` . . . 145
16.13 `tools/manual` . . . 145
16.14 `tools/manual-commands` . . . 146
16.15 `tools/neutral` . . . 146
16.16 `tools/publish-libraries` . . . 147
16.17 `tools/site` . . . 147
16.18 `tools/tech-doc` . . . 148
16.19 `tools/tech-doc-files` . . . 149
16.20 Autres fichiers de `tools/` . . . 149

II Référence du code **150**

17 Lire la référence **151**

18 Référence : modèle de base **152**
18.1 `model/equipment` . . . 152
18.2 `model/factories` . . . 155
18.3 `model/ids` . . . 161
18.4 `model/index` . . . 161
18.5 `model/lookup` . . . 161
18.6 `model/portStates` . . . 165

18.7 model/projectTemplate . . . 166
18.8 model/purge . . . 167
18.9 model/schema . . . 168
18.10 model/validate . . . 185

19 Référence : règles métier du modèle 188
19.1 model/placement . . . 188
19.2 model/rackGeometry . . . 196
19.3 model/dropTarget . . . 204
19.4 model/depthInfo . . . 207
19.5 model/links . . . 208
19.6 model/anomalies . . . 214
19.7 model/instancePorts . . . 215
19.8 model/arrange . . . 217
19.9 model/autoLayout . . . 220

20 Référence : pages, vues, cartouche, Schéma, recherche et bibliothèques 224
20.1 model/annotations . . . 224
20.2 model/libraryEdit . . . 232
20.3 model/libraryTransfer . . . 234
20.4 model/libraryTree . . . 239
20.5 model/modelEditor . . . 241
20.6 model/pageGeometry . . . 246
20.7 model/pageLayout . . . 252
20.8 model/search . . . 255
20.9 model/titleBlockLayout . . . 257

21 Référence : bibliothèques, équipements intégrés et démarrage 261
21.1 app-version . . . 261
21.2 app/pwa . . . 261
21.3 app/session . . . 262
21.4 app/site . . . 265
21.5 help/documents . . . 266
21.6 help/markdown . . . 267
21.7 i18n/index . . . 268
21.8 libraries/catalogue . . . 270
21.9 libraries/crypto . . . 271
21.10 libraries/page . . . 273
21.11 library/defaultModels . . . 273
21.12 library/defaultTypes . . . 273
21.13 library/equipment/change . . . 275
21.14 library/equipment/generators . . . 280
21.15 library/equipment/index . . . 287

21.16 library/equipment/params . . . 287
21.17 library/index . . . 298
21.18 main . . . 299
21.19 test-setup . . . 299

22 Référence : routage **300**
22.1 routing/attach . . . 300
22.2 routing/avoid . . . 303
22.3 routing/bridges . . . 304
22.4 routing/bundles . . . 307
22.5 routing/external . . . 310
22.6 routing/hidden . . . 311
22.7 routing/lanes . . . 312
22.8 routing/manual . . . 315
22.9 routing/markers . . . 318
22.10 routing/outside . . . 320
22.11 routing/page . . . 321
22.12 routing/straight . . . 324
22.13 routing/trays . . . 325

23 Référence : texte **329**
23.1 text/fontFiles . . . 329
23.2 text/fonts . . . 331
23.3 text/index . . . 332
23.4 text/layout . . . 332
23.5 text/measure . . . 334
23.6 text/sfnt . . . 335
23.7 text/unicodeRange . . . 336

24 Référence : rendu SVG **338**
24.1 render/AnnotationsLayer . . . 338
24.2 render/DeviceFace . . . 339
24.3 render/ExternalDrawing . . . 340
24.4 render/LabelSheet . . . 341
24.5 render/LinksLayer . . . 341
24.6 render/PageView . . . 343
24.7 render/PortShape . . . 345
24.8 render/RackContents . . . 346
24.9 render/RackView . . . 348
24.10 render/StandardDecoration . . . 349
24.11 render/TextBlock . . . 350
24.12 render/TextMeasureContext . . . 351
24.13 render/TitleBlock . . . 352

24.14 render/images . . . 353
24.15 render/logo . . . 354
24.16 render/portShapes . . . 357

25 Référence : jeux d'essai **360**
25.1 testing/fixtures . . . 360
25.2 testing/fonts . . . 363
25.3 testing/generatedRack . . . 364
25.4 testing/rangeProject . . . 365
25.5 testing/referenceProject . . . 365
25.6 testing/routingChecks . . . 366

26 Référence : fichiers, stockage et import **368**
26.1 import/read . . . 368
26.2 import/rules . . . 372
26.3 import/template . . . 376
26.4 io/archive . . . 377
26.5 io/draft . . . 383
26.6 io/files . . . 385
26.7 io/hash . . . 388
26.8 io/images . . . 388
26.9 io/index . . . 391
26.10 io/migrations . . . 391
26.11 io/storage . . . 393
26.12 io/svg . . . 394

27 Référence : exports **396**
27.1 export/appFonts . . . 396
27.2 export/csv . . . 396
27.3 export/encoding . . . 398
27.4 export/fileNames . . . 399
27.5 export/fonts . . . 400
27.6 export/imageExport . . . 401
27.7 export/labels . . . 402
27.8 export/labelsExport . . . 406
27.9 export/pdfExport . . . 408
27.10 export/pdfImages . . . 410
27.11 export/png . . . 412
27.12 export/spreadsheet . . . 414
27.13 export/svgExport . . . 416
27.14 export/tables . . . 419
27.15 export/types . . . 422

28 Référence : état et commandes **423**

28.1 commands/annotation . . . 423
28.2 commands/context . . . 425
28.3 commands/device . . . 425
28.4 commands/edit . . . 427
28.5 commands/export . . . 427
28.6 commands/file . . . 427
28.7 commands/help . . . 428
28.8 commands/import . . . 429
28.9 commands/index . . . 429
28.10 commands/inspect . . . 429
28.11 commands/library . . . 430
28.12 commands/libraryFiles . . . 430
28.13 commands/link . . . 431
28.14 commands/page . . . 431
28.15 commands/project . . . 432
28.16 commands/rack . . . 432
28.17 commands/registry . . . 433
28.18 commands/route . . . 436
28.19 commands/selectionOps . . . 437
28.20 commands/shortcuts . . . 438
28.21 commands/view . . . 439
28.22 store/actions . . . 440
28.23 store/annotationActions . . . 445
28.24 store/deviceActions . . . 448
28.25 store/editor . . . 450
28.26 store/exampleProject . . . 453
28.27 store/importActions . . . 454
28.28 store/index . . . 454
28.29 store/libraryActions . . . 454
28.30 store/machineActions . . . 460
28.31 store/pageActions . . . 463
28.32 store/portActions . . . 467
28.33 store/projectActions . . . 468
28.34 store/rackActions . . . 470
28.35 store/react . . . 472
28.36 store/routeActions . . . 473
28.37 store/routingActions . . . 474
28.38 store/selectors . . . 475

29 Référence : interface utilisateur **478**

29.1 ui/App . . . 478
29.2 ui/CommandButton . . . 479
29.3 ui/ContextMenu . . . 479
29.4 ui/DraftDialog . . . 479
29.5 ui/ErrorDialog . . . 480
29.6 ui/MenuBar . . . 480
29.7 ui/Messages . . . 481
29.8 ui/Modal . . . 481
29.9 ui/PageTabs . . . 482
29.10 ui/Toolbar . . . 482
29.11 ui/UpdateBanner . . . 482
29.12 ui/menus . . . 482
29.13 ui/useCommandShortcuts . . . 483
29.14 ui/state . . . 484
29.15 ui/hooks . . . 489
29.16 ui/placementFlow . . . 491
29.17 ui/linkFlow . . . 492
29.18 ui/annotationFlow . . . 492
29.19 ui/imageImport . . . 494
29.20 ui/canvas/Canvas . . . 495
29.21 ui/canvas/coords . . . 496
29.22 ui/canvas/Overlay . . . 497
29.23 ui/canvas/Highlights . . . 499
29.24 ui/canvas/RouteHandles . . . 500
29.25 ui/canvas/PortTooltip . . . 501
29.26 ui/canvas/DepthTooltip . . . 501
29.27 ui/panels/LeftPanel . . . 502
29.28 ui/panels/tabIcons . . . 502
29.29 ui/panels/LibraryPanel . . . 503
29.30 ui/panels/ExplorerPanel . . . 504
29.31 ui/panels/ChecksPanel . . . 504
29.32 ui/panels/SchemaPanel . . . 505
29.33 ui/properties/PropertiesPanel . . . 505
29.34 ui/properties/PageProperties . . . 505
29.35 ui/properties/ViewProperties . . . 506
29.36 ui/properties/RackProperties . . . 506
29.37 ui/properties/DeviceProperties . . . 506
29.38 ui/properties/InstancePorts . . . 507
29.39 ui/properties/LinkProperties . . . 507
29.40 ui/properties/AnnotationProperties . . . 508

29.41 ui/fields 509
29.42 ui/dialogs/DialogHost 511
29.43 ui/dialogs/ProjectDialogs 511
29.44 ui/dialogs/RackDialogs 513
29.45 ui/dialogs/PlaceDeviceDialog 513
29.46 ui/dialogs/LinkDialog 513
29.47 ui/dialogs/LinksTableDialog 514
29.48 ui/dialogs/LinkVisibilityDialog 514
29.49 ui/dialogs/RoutingDialog 514
29.50 ui/dialogs/SearchDialog 515
29.51 ui/dialogs/PreferencesDialog 515
29.52 ui/dialogs/HelpDialogs 516
29.53 ui/dialogs/LibraryDialogs 517
29.54 ui/dialogs/ImportModelsDialog 517
29.55 ui/dialogs/ExportLibraryDialog 518
29.56 ui/dialogs/ImportSpreadsheetDialog 518
29.57 ui/exports/download 519
29.58 ui/exports/ExportPartsFields 519
29.59 ui/exports/ExportImageDialog 519
29.60 ui/exports/ExportPdfDialog 520
29.61 ui/exports/ExportSpreadsheetDialog 520
29.62 ui/exports/ExportLabelsDialog 520
29.63 ui/library/LibrarySheet 521
29.64 ui/machine/paramFields 521
29.65 ui/machine/MachineParamsForm 523
29.66 ui/machine/MachineEditorDialog 524
29.67 ui/modelEditor/RackPreview 524
29.68 ui/modelEditor/FaceEditor 524
29.69 ui/modelEditor/PortGridForm 525
29.70 ui/modelEditor/PortsTable 525

Index **526**

Première partie

Conception

Chapitre 1

Introduction

1.1 Objet et lecteurs

Ce document est la **documentation technique** de Portolan, générateur de schémas de baies informatiques fonctionnant entièrement dans le navigateur (exigence DOC-10 du cahier des charges). Il s'adresse aux développeurs et aux agents qui maintiennent ou font évoluer l'application : il doit permettre de comprendre une partie du code avant de la modifier, sans avoir à tout relire.

La documentation *utilisateur* (manuel complet, prise en main rapide) décrit ce que fait l'application ; celle-ci décrit *comment* elle le fait. Les trois documents sont rédigés en LaTeX, compilés en PDF, livrés avec l'application (menu Aide) et publiés en accès libre sur le site.

1.2 Organisation du document

Première partie, Conception. Un chapitre par domaine : architecture, modèle de données, règles métier, pages et vues, bibliothèques, routage des câbles, rendu, texte, fichiers et import, exports, état et commandes, interface, tests, publication et outils du dépôt. Chaque chapitre explique les principes, les algorithmes, les invariants et les exigences du cahier des charges qu'il met en œuvre, puis donne des conseils pour faire évoluer le domaine.

Deuxième partie, Référence du code. Chaque module de `src/` (un fichier), puis **chaque déclaration exportée** : fonction, constante, type, interface, classe ou composant React. Les outils de `tools/` sont décrits au chapitre « Outils du dépôt ».

Un index alphabétique des modules et des déclarations termine le document ; dans le PDF, chaque nom en police à chasse fixe soulignée par un lien renvoie à sa description.

1.3 Documents de référence

- `docs/cahier-des-charges.md` : le besoin. **Il fait foi.** Les identifiants d'exigences cités ici (`PLA-03`, `ROU-14`…) en viennent.
- `docs/plan-developpement.md` : les lots, leurs dépendances et la définition de fin commune (§ 1.3).

- `docs/decisions.md` : le journal des décisions techniques, lot par lot ; ce document en reprend l'essentiel, le journal garde l'historique et les alternatives écartées.
- `CLAUDE.md` et `README.md` : règles de travail et prise en main du dépôt.

1.4 Lire une entrée de la référence

Chaque déclaration exportée a une entrée de cette forme :

- en tête, son **nom**, sa **nature** (fonction, constante, variable, type, interface, classe) et sa **signature**. La signature est *extraite du code* à chaque compilation (`tools/tech-doc.ts`) : elle est toujours exacte. Pour une interface, elle liste les champs ; pour un type ou une constante longue, elle est tronquée (« … ») ;
- puis une description **rédigée** : rôle, paramètres, résultat, cas d'erreur, et un exemple quand il est utile. Pour une interface, le sens de chaque champ et ses unités.

Les unités du modèle sont le **millimètre** pour les dimensions physiques et la mise en page sur le papier, le **U** (44,45 mm) pour les hauteurs dans les armoires, le pixel pour l'écran et les images matricielles.

1.5 Tenir la documentation à jour

La documentation technique fait partie de la **définition de fin** de chaque lot (plan, § 1.3) : un lot qui ajoute, renomme ou supprime une déclaration exportée, un module ou un outil met à jour ce document dans le même commit.

1.5.1 Le contrôle `npm run check:tech-doc`

Exécuté en intégration continue, il échoue si :

- une déclaration exportée de `src/` n'a pas sa section `\fonction{module.nom}`, ou si une section ne correspond à aucune déclaration (code supprimé ou renommé) ;
- une section est en double, n'est pas placée sous la section `\module{…}` de son fichier, ou n'a pas de description rédigée (au moins une phrase) ;
- un fichier de `src/` ou de `tools/` n'a pas sa section `\module{id}`, ou une telle section ne correspond plus à aucun fichier ;
- un renvoi `\voir{module.nom}` désigne une déclaration qui n'existe pas, ou un renvoi `\voirmodule{id}` un module qui n'existe pas ;
- le PDF ne compile pas, ou compile en perdant du contenu (caractère absent des polices, tableau plus grand que sa page).

L'identifiant d'un module est le chemin de son fichier relatif à `src/`, sans extension (`model/placement`, `library/equipment/params`) ; pour un outil, `tools/` suivi du nom du fichier (`tools/site`). Une déclaration est désignée par `module.nom` (`model/placement.canPlace`). Sont « exportées » les déclarations qui portent `export` dans leur fichier, ou qu'un `export {…}` du même fichier rend publiques ; une réexportation (`export … from`) est documentée là où la déclaration est écrite. Les fichiers de test (`*.test.ts`) et de déclarations (`*.d.ts`) ne sont pas concernés.

1.5.2 Écrire une entrée

Les macros sont définies dans `docs/technique/macros.tex` :

```
\module{model/placement}
Règles de placement des équipements dans les armoires (PLA-01 à PLA-08)…

\fonction{model/placement.canPlace}
Indique si … Utilisée par …
\begin{parametres}
  \item[project] le projet ;
  \item[rackId] l'armoire visée.
\end{parametres}
\resultat{…}
\erreurs{Aucune : un placement impossible est signalé par le résultat.}
```

Dans le corps du document, le caractère _ est ordinaire : les identifiants s'écrivent tels quels. Les autres caractères spéciaux de LaTeX (`& % $ # { }`) se protègent par une contre-oblique, sauf dans l'environnement `exemple`, qui reproduit le code tel quel.

1.5.3 Compiler

Commande	Effet
`npm run doc:tech`	Compile `docs/technique/build/portolan-technique.pdf`.
`npm run check:tech-doc`	Contrôle de couverture, puis compilation.
`… check:tech-doc -- --only=routing/ --no-pdf`	Contrôle limité à des modules ou à un dossier (identifiant terminé par `/`), sans compiler.
`… doc:tech -- --only=reference/rendu --job=essai`	Compile seulement ces chapitres (`\includeonly`) dans `build/essai/`, sans gêner une autre compilation.

Le PDF est ensuite intégré au build de l'application (`manual/portolan-technique.pdf`) par `npm run build` : il est ouvert par le menu Aide › Documentation technique, mis en cache pour le hors ligne, publié sur le site et inclus dans l'archive téléchargeable (voir le chapitre « Intégration continue et publication »). La CI le joint aussi à ses résultats (artefact `documentation-technique`).

1.6 Conventions du code

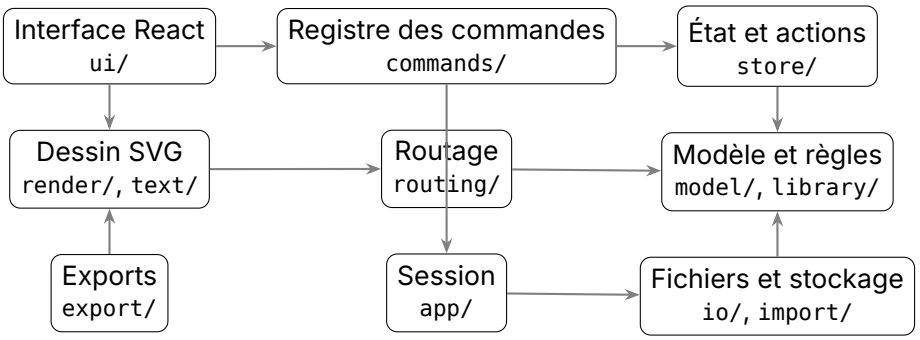
- Code, identifiants et commentaires en anglais ; textes de l'interface et documentation en français. Aucun texte d'interface écrit en dur : tout passe par `src/i18n/fr.json` (CG-05).
- TypeScript strict (`noUncheckedIndexedAccess`, `verbatimModuleSyntax`).
- Tests unitaires à côté du code (`x.test.ts`) ; tests de bout en bout dans `tests/`.
- Neutralité (CG-08) : aucun nom de marque ou d'entreprise dans `src/`, `manual/`, `site/`, `public/` ni dans cette documentation (`npm run check:neutral`).

Chapitre 2

Architecture

2.1 Vue d'ensemble

Portolan est une **application web statique** : un fichier `index.html`, des scripts, une police, un module WebAssembly et des PDF, servis tels quels par n'importe quel serveur web (chemins relatifs, NF-11). Il n'y a **aucun serveur applicatif** : les projets sont lus et écrits sur le poste (fichiers `.rkdg`, stockage du navigateur), rien ne quitte le navigateur et aucune ressource n'est chargée depuis un autre site (CG-02, politique de sécurité stricte). Une fois ouverte, l'application fonctionne hors ligne grâce à un service worker (CG-03).



Une action de l'utilisateur (menu, bouton, raccourci, geste sur le dessin) passe par une **commande** du registre, qui appelle une **action** du magasin d'état ; l'action modifie le **projet** (le modèle) en enregistrant de quoi l'annuler. Le dessin est une **fonction du projet** : les composants SVG relisent le projet, le routage calcule les tracés des câbles, le texte est mesuré et découpé à l'avance. Les exports réutilisent exactement les mêmes composants de dessin.

2.2 Technologies

Bibliothèque	Rôle
React 19	Interface et dessin SVG (composants fonctionnels).
Zustand (magasin « vanilla ») et Immer	État de l'éditeur, modifications par correctifs (*patches*) pour annuler et rétablir.
Zod	Schémas du modèle : validation des fichiers ouverts, types TypeScript déduits, schémas JSON publiés (`public/schemas/`).
i18next	Textes de l'interface (`src/i18n/fr.json`).
libavoid-js (WebAssembly)	Tracé orthogonal des câbles hors des armoires.
jsPDF et svg2pdf.js	Export PDF vectoriel.
ExcelJS	Export tableur XLSX ; import tableur.
fflate	Archives ZIP (`.rkdg`, `.rkdglib`, archive du site).
idb	Accès à IndexedDB (brouillon de secours, préférences).
DOMPurify	Nettoyage des images SVG importées.
Inter (Fontsource, TrueType)	Police unique de l'écran et des exports.
Vite, vite-plugin-pwa	Build, service worker et manifeste.
Vitest, Playwright	Tests unitaires ; tests de bout en bout et captures.

Les modules lourds (export PDF, tableur, libavoid) sont chargés à la demande (`import()` dynamique) puis mis en cache pour le hors ligne.

2.3 Organisation des dossiers

Dossier	Contenu
`src/model/`	Modèle de données (schémas Zod), identifiants, fabriques, recherches, validation ; règles métier : placement, profondeur, liaisons et chemins de bout en bout, anomalies, géométrie des armoires et des pages, annotations, mise en page du cartouche, édition et transfert de bibliothèques.
`src/library/`	Catalogue des équipements intégrés (paramètres, générateurs de machines), bibliothèque et types par défaut.
`src/routing/`	Tracé des câbles : goulottes, couloirs, hors baie (libavoid), faisceaux, ponts, repères, retouche, points rattachés aux vues.
`src/text/`	Polices embarquées, mesure et découpage du texte.
`src/render/`	Composants SVG du dessin, communs à l'écran et aux exports.
`src/export/`	Exports SVG, PNG, PDF, tableur, CSV, étiquettes.
`src/io/`	Format `.rkdg` / `.rkdglib`, images, stockage du navigateur, brouillon, ouverture et enregistrement des fichiers.
`src/import/`	Import tableur simplifié (modèle, lecture, règles).
`src/store/`	Magasin d'état, actions annulables par domaine, sélecteurs, projet d'exemple.
`src/commands/`	Registre des commandes (DOC-09) et raccourcis.
`src/ui/`	Interface React : menus, barre d'outils, panneaux, zone de dessin, boîtes de dialogue, état de l'interface.

Dossier	Contenu
`src/app/`	Session (ouvrir, enregistrer, brouillon), hors ligne et mises à jour, liens du site.
`src/libraries/`	Chiffrement et page des bibliothèques de l'équipe (site).
`src/help/`	Notes de version embarquées et leur lecteur Markdown.
`src/i18n/`	Textes de l'interface.
`src/testing/`	Jeux d'essai partagés par les tests unitaires, le banc de rendu et les tests de bout en bout.
`tools/`	Contrôles et scripts du dépôt (neutralité, documentations, site, publication, schémas JSON).
`tests/`	`e2e/` (application, captures de référence), `build/` (build servi), `site/` (site construit), `manual/` (captures du manuel), `harness/` (banc de rendu).
`manual/`	Documentation utilisateur (LaTeX) et ses captures.
`docs/`	Cahier des charges, plan, décisions, publication, et cette documentation (`docs/technique/`).
`site/`	Fichiers propres au site public (page des bibliothèques, `robots.txt`).
`libraries/`	Sources des bibliothèques de marques : seul endroit où les marques sont admises, hors de l'application.
`public/`	Fichiers copiés tels quels dans le build (icônes, schémas JSON).

2.4 Couches et dépendances

2.4.1 Le cœur ne dépend pas de React

Le modèle (`model/`), les règles métier, le catalogue (`library/`), le routage (`routing/`), le format de fichier (`io/`), l'import (`import/`) et le texte (`text/`) sont écrits en TypeScript pur : ils ne dépendent ni de React ni du DOM (sauf, pour `io/`, des API de fichiers du navigateur, isolées dans quelques fonctions) et sont testés unitairement sous Node (CLAUDE.md, règle 6). Le magasin d'état (`store/`) est lui aussi indépendant de React : les composants y accèdent par les crochets de `store/react`.

2.4.2 Dépendances observées entre dossiers

Dossier	Importe
`model`	`i18n`, `library`, `io` (types d'images)
`library`	`model`, `i18n`
`routing`	`model`, `library`, `i18n`
`io`	`model`, `store` (brouillon), `i18n`
`import`	`model`, `library`, `export`, `i18n`
`store`	`model`, `library`, `routing`, `import`, `io`, `i18n`
`render`	`model`, `routing`, `text`, `store`, `library`, `io`, `export`, `i18n`
`export`	`render`, `routing`, `model`, `text`, `store`, `io`, `i18n`
`commands`	`store`, `model`, `app`, `ui` (état de l'interface), `io`, `import`, `routing`, `library`, `i18n`
`ui`	tous les dossiers ci-dessus

`i18n` ne dépend que d'i18next (pas de `react-i18next`) : le cœur peut produire des messages

traduits sans React. Une nouvelle dépendance doit respecter cette orientation ; en particulier, rien dans `model/`, `routing/`, `text/`, `io/`, `import/` ni `library/` n'importe `react`.

2.5 Flux de données

2.5.1 Démarrage

`src/main.tsx` (`main`) charge la police Inter, enregistre le module WebAssembly de libavoid, attend les polices (le texte n'est mesuré qu'avec la police embarquée, NF-07b), crée le mesureur de texte, le magasin d'état (`createEditor`) avec le projet de démarrage, l'état de l'interface (`createUi`), ouvre le stockage du navigateur (préférences, brouillon), crée la session (`createSession`) puis monte `App` dans les contextes React. En développement, il expose `window.__portolan` pour les tests de bout en bout.

2.5.2 Une modification

1. Un élément d'interface exécute une commande du registre (`commands/registry`) : la commande vérifie qu'elle est disponible (`isEnabled`) puis appelle une ou plusieurs actions.
2. L'action (`store/*Actions.ts`) appelle `editor.apply(label, recipe)` : la recette modifie un brouillon Immer du projet ; les correctifs et leurs inverses sont empilés dans l'historique (100 niveaux, PRJ-07). Annuler et rétablir rejouent ces correctifs.
3. Les composants abonnés au magasin se redessinent. Le routage est recalculé à partir du projet (résultats mis en cache par page).
4. La session marque le projet modifié et écrit le brouillon de secours dans IndexedDB.

2.5.3 Un export

Les exports construisent hors écran le même arbre SVG que la zone de dessin (rendu React côté serveur dans un magasin dédié), avec le texte déjà découpé en lignes, puis le convertissent : sérialisation (SVG), rastérisation (PNG), conversion vectorielle (PDF), ou lisent directement le modèle (tableur, étiquettes). Un test compare le SVG exporté au dessin de l'écran.

2.6 Principes transverses

Un seul dessin. Écran et exports partagent `render/` ; une correction de dessin vaut partout.

Registre des commandes. Toute commande est déclarée dans `src/commands` avant d'être branchée sur un menu, un bouton ou un raccourci, et documentée dans le manuel utilisateur (`\commande{id}`, contrôle `npm run check:doc`, DOC-09).

Aucun texte en dur. Les textes passent par `fr.json` ; une règle ESLint l'impose dans les composants (CG-05).

Neutralité. Aucun nom de marque dans le cœur, le manuel, le site ni cette documentation (CG-08, `npm run check:neutral`).

Aucun chargement externe. Politique de sécurité du contenu stricte au build (`default-src 'self'`, pas d'`unsafe-eval`) ; la colle JavaScript de libavoid est réécrite au build pour ne pas créer de fonction à l'exécution (`tools/libavoid-no-eval`).

Format avant la version 1. Les fichiers d'un format plus ancien sont refusés avec un message clair (FMT-04, décision du 05/11/2026) ; à partir de la version 1, une chaîne de migrations les ouvrira.

Décisions consignées. Tout choix technique non évident est consigné dans `docs/decisions.md`, avec sa raison et les alternatives écartées.

2.7 Le build

`vite.config.ts` assemble l'application :

- greffon `portolan-csp` : injecte la politique de sécurité dans `index.html` (build seulement : le serveur de développement en a besoin d'une plus souple pour le rechargement à chaud) ;
- greffon `portolan-libavoid-no-eval` : réécrit la colle JavaScript de libavoid-js ;
- greffon `portolan-manual` : copie dans le build les deux documents de l'utilisateur (`manual/build/portolan.pdf`, `manual/build/prise-en-main-rapide.pdf`) et cette documentation (`docs/technique/build/portolan-technique.pdf`) sous `manual/`, s'ils ont été compilés (`npm run doc`, `npm run doc:tech`) ;
- vite-plugin-pwa : manifeste, icônes et service worker qui met en cache toute l'application, ses PDF compris, à la première visite ;
- alias `inter-ttf/` (police TrueType pour le PDF) et `libavoid-wasm` (module WebAssembly copié dans le build) ;
- `base: './'` : chemins relatifs, le build fonctionne depuis n'importe quel sous-dossier ; cartes de sources générées mais jamais référencées ni publiées.

La configuration des tests unitaires (Vitest) est dans le même fichier : seuils de couverture de 70 % sur le cœur, environnement Node, police et modules transformés comme dans l'application.

Chapitre 3

Modèle de données

Ce chapitre décrit le cœur de Portolan : la structure d'un projet, telle qu'elle est tenue en mémoire et enregistrée dans le `project.json` d'une archive `.rkdg` (cahier des charges, § 6 et § 8). Tout le reste de l'application (règles de placement, routage, rendu, exports, interface) lit ce modèle et le transforme par des fonctions pures ; aucun de ces modules ne dépend de React (règle 6 de `CLAUDE.md`).

Les modules décrits ici sont réunis sous `src/model/` :

Module	Rôle
`model/schema`	schémas Zod de toutes les entités et types TypeScript qui en sont déduits
`model/ids`	identifiants (UUID aléatoires, identifiants fixes des données intégrées)
`model/factories`	création d'entités neuves, toujours valides
`model/equipment`	catégories des équipements intégrés et prédicats sur les machines
`model/lookup`	index par identifiant, résolution des références de ports
`model/validate`	validation d'un projet : structure puis références croisées
`model/portStates`	ports libres et occupés d'une page (PRT-03)
`model/projectTemplate`	révisions et nouveau projet à partir d'un existant
`model/purge`	purge de la bibliothèque (BIB-05)
`model/index`	point d'entrée qui réexporte le modèle

Les règles métier qui s'appuient sur ce modèle (placement, liaisons, anomalies, géométrie des armoires et des pages, cartouche) sont décrites dans les chapitres suivants ; le format d'archive et les migrations, dans le chapitre consacré aux fichiers.

3.1 Principes

Le schéma Zod est la source unique. Chaque entité est décrite une seule fois, par un schéma Zod de `model/schema` ; le type TypeScript correspondant en est déduit par `z.infer` (`Project` est `z.infer<typeof ProjectSchema>`). La validation à l'ouverture d'un fichier, le typage du code et les schémas JSON publiés (FMT-02, `public/schemas/`) ne peuvent donc pas diverger. Les messages d'erreur de Zod sont en français (`z.config(z.locales.fr())`), les messages propres à Portolan passent par `src/i18n/fr.json` (clés `validation.*`).

Unités. Les dimensions sont en millimètres (armoires, profondeurs, pages, annotations), les hauteurs d'équipement en U, les positions en U toujours comptées depuis le bas de l'armoire (`uBottom`, à partir de 1), quel que soit le sens de numérotation affiché (`uNumbering` n'est qu'un choix d'affichage). La position et la taille d'un port sont des proportions de sa face (0 à 1), indépendantes de l'échelle de dessin.

Valeurs par défaut. Presque tous les champs optionnels ont une valeur par défaut dans le schéma (`.default(…)`) : un fichier écrit à la main, ou produit par une version qui ne connaissait pas encore un champ, se charge sans erreur, et le résultat de `parse` est toujours complet. C'est aussi ce qui permet d'ajouter un champ sans migration (voir § 3.8).

Identifiants. Toutes les entités ont un identifiant UUID (`IdSchema`) stable, utilisé pour toutes les références croisées ; aucune référence ne passe par un nom. `newId` tire un UUID version 4 (`crypto.randomUUID`, ou un repli sur `crypto.getRandomValues` dans un contexte non sécurisé). `builtinId` fabrique des identifiants fixes de la forme `00000000-0000-4000-8000-xxxxxxxxxxxx` pour les données intégrées : types de connecteurs et de liaisons par défaut, machines génériques et leurs ports. Ainsi un même type par défaut a le même identifiant dans tous les projets et toutes les bibliothèques, ce qui simplifie les imports (décision du 04/11/2026, lot 01 ; plages `0x5xx` pour les machines génériques, décision du 05/11/2026, lot 34).

Immuabilité. Dans l'application, le projet est un objet immuable : le magasin d'état (`src/store/`) le modifie par Immer, qui produit un nouvel objet à chaque action et partage les parties inchangées. Les fonctions du modèle ne modifient jamais leurs arguments ; elles renvoient un nouveau projet (`purgeUnusedMachines`, `projectFromExisting`) ou une valeur dérivée. Cette propriété est exploitée par l'index (§ 3.4) et par la mémoïsation du rendu.

3.2 Vue d'ensemble d'un projet

La figure 3.1 montre l'arbre d'un projet ; le tableau qui la suit donne les références par identifiant entre ses parties.

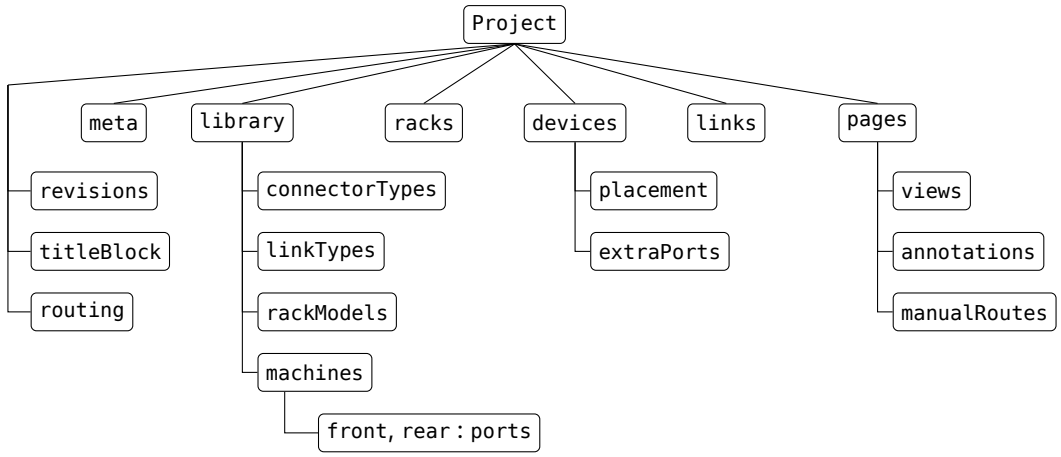


Fig. 3.1 : Arbre d'un projet.

Champ	Désigne	Remarque
`device.machineId`	machine	obligatoire (erreur si absente)
`placement.rackId`	armoire	placements `rack` et `zeroU`
`link.a`, `link.b`	équipement + port	avertissement si pendante (LIA-07)
`link.linkTypeId`	type de liaison	obligatoire
`port.connectorTypeId`	type de connecteur	ports de machine et ports ajoutés
`port.pairedPortId`	port de traversée	même machine, autre face, réciproque
`view.targetId`	armoire ou équipement	selon la nature de la vue
`manualRoute.linkId`	liaison	avertissement si absente
`point.viewId`	vue de la même page	avertissement si absente
`rack.modelId`	modèle d'armoire	information seulement, non vérifiée

3.2.1 Projet

`ProjectSchema` est la racine : `formatVersion` (littéral égal à `FORMAT_VERSION`, soit 2), `id`, les métadonnées (`ProjectMetaSchema` : nom, client, auteur, date de création, logo, notes), la liste des révisions (PRJ-11), la bibliothèque du projet, les armoires, les équipements placés, les liaisons, les pages, le cartouche et les réglages de routage (`routing.trayByLinkType`, goulotte de chaque type de liaison, ROU-02). Le champ optionnel `editedLibrary` n'existe que lorsque le document ouvert est une bibliothèque `.rkdglib` en cours d'édition (BIB-13) : il décrit cette bibliothèque et la version avec laquelle elle sera enregistrée ; le document n'a alors ni armoire ni page.

3.2.2 Bibliothèque : types de connecteurs et de liaisons

La bibliothèque du projet (`LibrarySchema`) est une copie locale : le projet est autonome et ne dépend d'aucun fichier externe. Elle contient quatre listes.

- Les **types de connecteurs** (§ 6.7) : `ConnectorTypeSchema` donne leur nom, leur famille parmi cuivre, optique, vidéo, USB, série, audio, électrique et autre (voir `ConnectorFamilySchema`), leur forme de rendu schématique (`shape`) et connecteur complémentaire éventuel (`complementId`, par exemple C13 et C14).
- Les **types de liaisons** (`LinkTypeSchema`, § 6.8) : nom, couleur, style et épaisseur de trait (en millimètres à l'échelle 1), connecteurs admis, préfixe de repère (`ETH-`…), présence dans la légende, et `markers` pour dessiner ce type en repères de destination plutôt qu'en traits (ROU-09).
- Les **modèles d'armoires** (`RackModelSchema`).
- Les **machines** (`MachineSchema`).

3.2.3 Armoires

Une armoire (`RackSchema`, § 6.3) et un modèle d'armoire partagent les mêmes caractéristiques (`RackSpecShape` interne) : hauteur de 1 à `RACK_MAX_HEIGHT_U` U, largeur et profondeur en millimètres, largeur de montage 19 ou 10 pouces, **profondeur utile** (distance entre montants, utilisée par le contrôle de profondeur EQP-02), type, portes, sens de numérotation, nombre d'emplacements 0U par face et par côté, couleur, et goulotte de chaque type de

liaison propre à cette armoire (`trayByLinkType`, qui l'emporte sur le réglage du projet). L'armoire garde en plus `modelId`, le modèle dont elle est issue, à titre d'information seulement : modifier le modèle ne modifie pas l'armoire.

Un raffinement (`superRefine`) impose `usableDepthMm` $\leq$ `depthMm`. À la création, la profondeur utile vaut par défaut « profondeur $-$ 200 mm » (`defaultUsableDepth`).

3.2.4 Équipements intégrés et machines

Depuis le lot 34 (format 2, décision du 05/11/2026), le modèle distingue deux notions.

L'équipement est un type intégré à Portolan (EQT-01), désigné par un `EquipmentKind` (`switch`, `patchPanel`, `pdu`, `monitor`, `generic`…). La liste est fermée (`EQUIPMENT_KINDS`) : l'utilisateur ne crée jamais de type. Chaque type appartient à une catégorie de rangement (`KIND_CATEGORY`, `EQUIPMENT_CATEGORIES`) ; la catégorie n'est pas stockée dans le fichier, elle est déduite (`machineCategory`). Les paramètres, les générateurs et les formulaires des équipements vivent dans `src/library/equipment/`, pas dans le modèle.

La machine est la personnalisation d'un équipement (§ 6.4) : son champ `equipment` porte le type (`kind`) et **tous** ses paramètres (`params`, enregistrement libre validé par le catalogue). Le générateur du catalogue en déduit la hauteur `heightU`, la largeur `width` (pleine ou demi), la taille affichée `displaySizeMm` des équipements extérieurs et les ports ; ces valeurs dérivées sont stockées dans la machine comme un cache, recalculé à chaque génération. Une machine `generic` est la machine générique de son équipement (EQT-02) : une seule par type, sans marque, jamais purgée.

Une machine a deux faces (`FaceSchema`, § 6.5) : mode de rendu (`default`, dessin généré par Portolan, ou `image`, photo importée), image, couleur et libellé du dessin par défaut, ports. Elle peut porter un logo (`MachineLogoSchema`, EDM-10), une image d'objet posé pour une étagère (`carried`, MAC-03), une profondeur (`null` : toute la profondeur utile de l'armoire, EQP-02), un dégagement arrière, une face de montage par défaut, des mots-clés et une origine (`MachineOriginSchema` : bibliothèque `.rkdglib` d'origine, sa version et l'identifiant de la machine dans celle-ci, pour détecter doublons et mises à jour).

Trois familles se déduisent du type et de la hauteur (`model/equipment`) :

- **extérieur** (catégorie `external` : écran, poste, portable…), placé hors des armoires, avec une taille affichée obligatoire (raffinement du schéma) ; `isExternal` ;
- **0U vertical** (multiprise verticale) : non extérieur et `heightU` nul ; `isZeroU`, longueur en U éventuelle donnée par le paramètre `lengthU` (`zeroULengthU`, STD-01) ;
- **en U** : tous les autres.

3.2.5 Ports

Un port (`PortSchema`, § 6.6) a un nom, un ordre, un type de connecteur, une position (`x`, `y` du centre) et une taille (`w`, `h`) en proportion de la face, un groupe, une direction de sortie (`exit` : haut, bas, gauche, droite ou automatique) et trois drapeaux : `multiple` (port acceptant plusieurs liaisons), `pairedPortId` (port de traversée sur l'autre face d'un panneau de brassage, pour les chemins de bout en bout) et `placed` (port positionné par l'utilisateur, dont position et taille sont conservées quand la machine est régénérée).

Les ports appartiennent à la machine : tous les équipements placés de cette machine ont les mêmes. L'équipement placé peut seulement :

- surcharger le nom affiché d'un port ou le désactiver (`portOverrides`, § 6.6) ; le nom affiché s'obtient par `portDisplayName` ;
- recevoir des ports supplémentaires (`extraPorts`, PRT-04, par exemple une carte d'extension), décrits par `InstancePortSchema` : un port plus la face de la machine qui le porte. Ces ports n'ont jamais de port de traversée.

Une extrémité de liaison est une `PortRef` : `deviceId` et `portId`. Le port peut être celui de la machine ou un port supplémentaire de l'équipement : `resolvePort` cherche dans les deux.

3.2.6 Équipements placés et types de placement

Les équipements placés (instances, `DeviceSchema`) forment une **seule** liste `devices`, qu'ils soient dans une armoire, en 0U ou à l'extérieur ; chacun porte un `placement` discriminé par `kind` (`PlacementSchema`) :

`kind`	Machine	Champs
`rack`	en U	`rackId`, `uBottom` (U la plus basse, depuis 1), `mountFace` (face de fixation, l'équipement s'étend vers l'autre, EQP-09), `halfSide` (côté d'un équipement demi-largeur, EQP-03)
`zeroU`	0U	`rackId`, `face`, `side`, `slot` (emplacement vertical, depuis 0)
`external`	extérieur	aucun : la position est celle de la vue extérieure sur la page

Ce choix (décision du 04/11/2026, lot 01) donne une seule forme d'extrémité de liaison et permet de déplacer un équipement d'une armoire à l'extérieur sans changer d'identifiant ; la face de montage remplace un drapeau « retourné ». L'instance porte aussi un nom, une description, des champs libres (clé et valeur : nom d'hôte, adresse, numéro de série…), le mode d'étiquette et `depthForced` (placement maintenu malgré un dépassement de profondeur, bouton « Forcer » d'EQP-02).

3.2.7 Liaisons et faisceaux

Une liaison (`LinkSchema`, § 6.9) relie deux extrémités `a` et `b` et porte un type de liaison, un repère (unique dans le projet, LIA-04), une longueur en mètres ou centimètres, une catégorie, une référence fournisseur, une couleur de gaine (information seulement : le trait garde la couleur du type), une goulotte (`auto`, `left` ou `right`, ROU-02), des notes, `compatibilityForced` (liaison maintenue malgré l'avertissement de compatibilité, LIA-06), un mode de dessin (`display` : selon la vue, trait ou repères, ROU-09) et `bundle` : le câble peut être fusionné en faisceau avec les câbles du même type entre les deux mêmes équipements (ROU-11) ; `false` signifie « dégroupé » par l'utilisateur. Le regroupement lui-même est activé par vue (`View.bundles`) et calculé par le routage.

3.2.8 Pages, vues, annotations, tracés manuels, cartouche

Une page (`PageSchema`, § 7.10) a un format (A4 à A0, Letter ou personnalisé : la taille est alors obligatoire, raffinement du schéma), une orientation, des marges, l'option d'échelle

uniforme des armoires (PAG-05), des vues, des annotations, des tracés manuels, l'affichage des repères de câbles (ROU-10), et la position de la légende et du tableau des révisions.

Une vue (`ViewSchema`) montre une armoire de face ou de dos, ou un équipement extérieur (`targetId` désigne l'armoire ou l'équipement), à une position en millimètres de papier et une échelle ; ses options règlent le titre, les étiquettes à côté de l'armoire (PAG-06, PAG-17), la taille forcée et le masquage des ports d'un extérieur (EXT-02, EXT-03), le mode repères (ROU-09), les faisceaux (ROU-11), le masquage des câbles (PAG-15) et la plage de U montrée (PAG-16).

Les annotations de l'onglet « Schéma » (`AnnotationSchema`, PAG-07) forment une union discriminée par `kind` : texte, trait ou flèche, rectangle (arrondi ou non) ou ellipse, image, zone titrée, bulle numérotée, note encadrée, tableau. Elles partagent une couleur, une couche (sous ou sur les vues) et une rotation par quart de tour.

Un tracé manuel (`ManualRouteSchema`, ROU-06) appartient à une page et désigne une liaison ; ses points sont **rattachés à une vue** de la page (`AttachedPointSchema`, ROU-16) : décalage en millimètres du dessin de la vue à l'échelle 1, depuis son coin haut gauche. Le point suit la vue quand elle se déplace et garde sa place sur le dessin quand l'échelle change (décision du 05/11/2026, lot 37).

Le cartouche (`TitleBlockSchema`, PAG-09, PAG-10) est réglé au niveau du projet : une liste de champs (`TitleBlockFieldSchema`) de nature fixe (logo, nom du projet, client, titre de page, révision, date, auteur, numéro de page) ou libre, visibles ou non, chacun placé dans une rangée (haut, bas, pleine hauteur ; `null` : place par défaut de la nature).

3.3 Fabriques

Les fabriques de `model/factories` créent des entités neuves : elles tirent un nouvel identifiant, complètent les champs obligatoires par des valeurs par défaut raisonnables, puis passent **toujours** par le schéma (`Schema.parse`), si bien que le résultat est valide et complet. Elles ne génèrent rien : une machine créée par `createMachine` garde les ports et la hauteur qu'on lui donne ; pour dériver ports et dimensions des paramètres, on passe par le catalogue (`generateMachine`, `createGenericMachine`, `src/library/`). De même `createProject` crée un projet vide avec une bibliothèque vide ; la bibliothèque par défaut et la première page sont ajoutées par `createNewProject` et `createStarterProject` (`src/library/index.ts`).

Les textes par défaut (« Armoire », « Page », « Nouveau projet ») viennent de `fr.json` (`defaults.*`). Le cartouche par défaut (`createDefaultTitleBlockFields`) comprend un champ logo, mais aucune image : le logo est fourni par l'utilisateur (CG-08).

3.4 Index et résolution des références

Les entités se référencent par identifiant ; les chercher par parcours de liste serait quadratique dans le routage et le rendu. `indexProject` construit une fois pour toutes un `ProjectIndex` : tables par identifiant des armoires, équipements, liaisons, machines, types de connecteurs et de liaisons ; ports de chaque machine (avec leur face) ; liaisons attachées à chaque port, sous la clé `portKey` (`deviceId/portId`).

L'index est mis en cache dans une `WeakMap` dont la clé est l'objet projet. Comme le projet est immuable, un projet inchangé retrouve son index. Par prudence (les tests et scripts modifient parfois le projet en place), l'entrée du cache garde une « empreinte » : les six listes indexées et leur longueur ; si l'une a changé de référence ou de longueur, l'index est reconstruit (décision du 04/11/2026, lot 02).

Limites. L'empreinte ne voit pas une modification en place qui garde la longueur d'une liste (remplacement d'un élément à la même position dans un tableau muté, changement d'un champ d'un élément) : ce cas n'arrive pas avec Immer, qui remplace la liste. Les ports supplémentaires des équipements (PRT-04) ne sont pas dans `machinePorts` : utiliser `resolvePort`, `deviceHasPort`, `devicePortEntries` ou `facePorts`, qui les prennent en compte, plutôt que l'index brut.

3.5 Validation et invariants

3.5.1 Deux étapes

`parseProject` valide une valeur inconnue (le `project.json` déjà migré, ou un brouillon de session) en deux temps (décision du 04/11/2026, lot 01) :

1. **Structure** : `ProjectSchema.safeParse`. Tout écart est une erreur ; les valeurs par défaut sont appliquées.
2. **Références croisées** : `checkReferences`, qui commence par `checkLibraryReferences` sur la bibliothèque. Chaque constat est une `ValidationIssue` de gravité `error` ou `warning`, avec un chemin JSON (`formatPath` : `racks[0].heightU`) et un message traduit (`validation.ref.*`).

S'il y a au moins une erreur, le résultat (`ParseResult`) est un échec qui ne contient que les erreurs, et le fichier est refusé avec un message explicite (FMT-02, REC-03). Sinon, le projet est rendu avec ses avertissements. `parseLibraryDocument` fait de même pour le `project.json` d'une bibliothèque `.rkdglib` (FMT-09), avec les seules vérifications de bibliothèque.

3.5.2 Erreurs et avertissements

La règle : une **référence indispensable cassée** est une erreur, car le projet serait inutilisable ; une situation que l'utilisateur peut corriger, et que le cahier prévoit (une liaison dont un port a disparu après une mise à jour de bibliothèque, LIA-07, BIB-14), est un avertissement.

Gravité	Constats (clé `validation.ref.*`)
erreur	identifiant en double dans une liste (types, modèles, machines, ports d'une machine, armoires, équipements, liaisons, pages, ports supplémentaires) (`duplicateId`) ; type de connecteur inconnu : complément, connecteur admis, port, port supplémentaire (`unknownConnector`) ; port de traversée absent ou non réciproque, ou sur la même face (`unknownPairedPort`, `invalidPairedPort`) ; port supplémentaire avec port de traversée ; deux machines génériques pour un même équipement (`duplicateGeneric`) ; paramètres de machine invalides selon le catalogue (`invalidParams`, BIB-09) ; machine absente (`unknownMachine`) ; armoire absente (`unknownRack`) ; type de placement incohérent avec la machine (`placementKind`) ; type de liaison absent (`unknownLinkType`) ; cible de vue absente (`unknownViewTarget`)
avertissement	surcharge d'un port inexistant (`unknownPort`) ; extrémité de liaison sur un équipement ou un port disparu (`linkDeviceMissing`, `linkPortMissing`) ; tracé manuel d'une liaison absente (`unknownRouteLink`) ou point rattaché à une vue absente de la page (`unknownRouteView`)

3.5.3 Où chaque invariant est tenu

Les invariants du modèle ne sont pas tous du même niveau ; savoir où chacun est tenu évite de le vérifier deux fois ou de l'oublier.

- **Par le schéma** (refusé à l'ouverture) : types et bornes des champs (hauteur de 1 à 52 U, proportions de port entre 0 et 1, couleurs `#rrggbb`, images nommées par leur empreinte SHA-256, dates ISO), profondeur utile $\leq$ profondeur, taille obligatoire d'un extérieur et d'une page personnalisée, plage de U croissante.
- **Par les références croisées** (`model/validate`) : unicité des identifiants, existence des cibles, cohérence placement / machine, ports de traversée réciproques, une seule machine générique par équipement.
- **Par les actions et les règles métier** (le projet reste ouvrable s'ils sont violés ; les vérifications les signalent comme anomalies, `listAnomalies`) : chevauchement des équipements et débordement d'armoire, profondeur (EQP-02), un port porte une seule liaison sauf port multiple, compatibilité des connecteurs (LIA-06), unicité et présence des repères de câbles (LIA-04), unicité des noms d'équipements. Ces règles sont dans `model/placement`, `model/links` et `model/anomalies`.
- **Par le catalogue** : cohérence des paramètres d'une machine et noms de ports uniques par face (`src/library/equipment/`) ; les valeurs dérivées (`heightU`, `width`, `displaySizeMm`, ports) ne sont cohérentes avec les paramètres que parce que toute modification passe par la génération.

3.6 Fonctions annexes

États des ports (PRT-03). `portStatesOnPage` liste chaque port dessiné sur une page, avec son rectangle sur la page et son état : occupé (au moins une liaison), désactivé (sur cet équipement) ou libre. Un équipement montré par plusieurs vues apparaît une fois par vue ; les ports des vues extérieures dont les ports sont masqués (EXT-03) sont omis. Le calque de mise en évidence de la zone de dessin (`src/ui/canvas/Highlights.tsx`) l'utilise.

Révisions et projet modèle (PRJ-10, PRJ-11). `nextRevisionIndex` propose l'indice de la révision suivante : un nombre final est incrémenté en gardant ses zéros (`B2` $\rightarrow$ `B3`, `09` $\rightarrow$ `10`), des lettres seules avancent comme un compteur en base 26 (`A` $\rightarrow$ `B`, `Z` $\rightarrow$ `AA`, `AZ` $\rightarrow$ `BA`) en gardant la casse ; tout autre indice reçoit un `+`. `projectFromExisting` crée un projet à partir d'un autre : copie profonde, nouvel identifiant, nouvelle date de création, aucune révision ; sans l'option « tout reprendre », armoires, équipements et liaisons sont vidés et les pages remplacées par une page vide reprenant la mise en page de la première (format, marges, légende, tableau des révisions). Bibliothèque, cartouche, propriétés et réglages de routage sont toujours repris : c'est ainsi qu'on réutilise un cartouche d'entreprise.

Purge (BIB-05). `purgeUnusedMachines` retire les machines qu'aucun équipement placé n'utilise, sauf les machines génériques, et estime le gain : taille des images qui ne servaient qu'aux machines retirées (ni à une autre machine, ni au logo du projet, ni à une annotation image) plus la taille JSON non compressée des machines retirées (décision du 04/11/2026, lot 04). Les modèles d'armoires et les types ne sont pas purgés. Les images elles-mêmes ne sont retirées de l'archive qu'à l'enregistrement, qui n'écrit que les images référencées (FMT-03).

3.7 Format de fichier et version

Le `project.json` d'une archive est le projet sérialisé après passage par le schéma (`serializeProject`, `src/io/archive.ts`) : l'ordre des clés est celui des schémas, l'écriture est déterministe. À la lecture, l'archive vérifie la version du manifeste : plus récente que `FORMAT_VERSION`, le fichier est refusé ; plus ancienne que `MIN_FORMAT_VERSION` (2), aussi (le format 1 n'est plus ouvert, décision du 05/11/2026, lot 34, FMT-04) ; entre les deux, la chaîne `MIGRATIONS` de `src/io/migrations.ts` (vide aujourd'hui) amène le document à la version courante, puis `parseProject` le valide.

Les schémas JSON publiés dans `public/schemas/` (projet `rkdg-project-v2.schema.json`, bibliothèque `rkdglib-library-v2.schema.json` et manifeste `rkdg-manifest-v2.schema.json`) sont générés depuis Zod par `npm run schema` (`tools/json-schema.ts`) ; le test `tools/json-schema.test.ts` échoue s'ils ne sont plus à jour.

3.8 Ajouter un champ au modèle

La démarche dépend de la compatibilité du changement avec les fichiers existants.

Champ optionnel avec valeur par défaut (cas le plus courant : nouvelle option d'une vue, d'une liaison…).

1. Ajouter le champ au schéma dans `src/model/schema.ts`, avec un commentaire JSDoc (unité, exigence) et une `.default(…)` ; pour une valeur objet ou tableau, une fonction (`.default(() => ({}))`) évite le partage d'une même instance. Le type TypeScript suit automatiquement.
2. Si la valeur par défaut d'une entité *neuve* doit différer de celle d'un fichier ancien, la fixer dans la fabrique (exemple : `defaultLabelsBeside`, étiquettes affichées sur une vue avant neuve, masquées dans un fichier qui ne connaît pas le réglage).
3. Lancer `npm run schema` pour régénérer `public/schemas/`.
4. Ne pas changer `FORMAT_VERSION` : un ancien fichier reste valide grâce à la valeur par défaut.
5. Si le champ référence une autre entité, ajouter la vérification dans `checkReferences` (erreur ou avertissement selon la règle ci-dessus), avec sa clé `validation.ref.*` dans `fr.json`.
6. Tests : `src/model/model.test.ts` (valeurs par défaut, aller-retour JSON), les tests de la règle métier concernée, et un test de lecture d'archive si le champ touche aux images (`collectImageRefs`).

Champ obligatoire, renommage ou changement de structure.

1. Incrémenter `FORMAT_VERSION`.
2. Ajouter dans `MIGRATIONS` (`src/io/migrations.ts`) l'étape `n` → `n+1`, qui transforme le JSON brut (avant validation) ; la tester dans `src/io/archive.test.ts`. Depuis la version 1 de l'application, un ancien fichier doit toujours s'ouvrir (FMT-04).

3. Lancer `npm run schema` : de nouveaux fichiers `…-v<n>.schema.json` sont produits ; conserver les anciens publiés.
4. Mettre à jour les fabriques, les jeux d'essai (`src/testing/fixtures.ts`) et `src/model/model.test.ts` : ce dernier vérifie, pour les principales entités, que chaque champ obligatoire manquant est refusé avec le bon chemin ; un nouveau champ obligatoire y est pris en compte automatiquement.
5. Consigner la décision dans `docs/decisions.md`.

Nouvelle entité ou nouveau type d'annotation. Ajouter le schéma et son type, la fabrique si l'interface en crée, l'entrée de l'index si on la cherche souvent par identifiant (`indexProject` ; penser à l'ajouter à l'empreinte du cache), les vérifications de références, et, si elle porte une image, l'ajouter à la collecte des images de l'archive et à la purge (`purgeUnusedMachines` ne garde que les images qu'elle connaît).

3.9 Tests

- `src/model/model.test.ts` : fabriques, valeurs par défaut, champs obligatoires, messages en français, références croisées, index, aller-retour JSON, documents de bibliothèque.
- `src/model/portStates.test.ts`, `projectTemplate.test.ts`, `purge.test.ts` : les fonctions du même nom.
- `tools/json-schema.test.ts` : schémas JSON publiés à jour.

Chapitre 4

Règles métier

Ce chapitre explique les règles qui décident de ce que l'utilisateur a le droit de faire sur un schéma et de ce que Portolan lui signale : placement des équipements dans une armoire, contrôle de profondeur, géométrie du dessin d'une armoire, dépôt à la souris, liaisons et chemins de bout en bout, anomalies, ports ajoutés à un équipement, alignement des éléments d'une page et mise en place automatique des vues. Le détail de chaque déclaration est au chapitre « Référence : règles métier du modèle ».

4.1 Principes communs

Toutes ces règles vivent dans `src/model/` et respectent les mêmes principes (règle 6 de `CLAUDE.md`) :

- **fonctions pures** : elles reçoivent le projet (ou une page, une armoire) et rendent un résultat ; elles ne modifient jamais leurs arguments. Une règle qui doit « simuler » une suite de changements (déplacer plusieurs équipements, en retourner plusieurs) travaille sur une copie superficielle du projet (`{ ...project, devices: … }`) ;
- **aucune dépendance à React** ni à l'état de l'interface : les règles sont appelées par les actions du magasin (`src/store/`), la zone de dessin, l'import tableur, le routage et les exports, et testées unitairement à côté de leur module (`placement.test.ts`, `links.test.ts`…) ;
- **refus rendus comme des valeurs** : un contrôle rend une union discriminée (`status: 'ok' | 'refused'`, `ok: true | false`) ou une liste de problèmes, jamais une exception. Chaque problème a une nature typée (pour les tests et le code appelant) et un message français déjà rédigé (pour l'interface) ;
- **messages dans `fr.json`** : les textes viennent des clés `placement.*`, `links.*`, `anomalies.*`, `instancePorts.*` par la fonction `t` de `src/i18n` ; les nombres sont formatés à la française (`formatNumber`) ;
- **accès par l'index** : les entités se retrouvent par `indexProject(project)` (`model/lookup`), qui met en cache des tables par identifiant et la liste des liaisons de chaque port (décision du 2026-11-04, lot 02) ; les ports d'un équipement, ports ajoutés compris, se résolvent par `resolvePort` et `devicePortEntries`.

Les unités suivent le schéma : profondeurs et dessin d'armoire en millimètres réels, pages en millimètres de papier, hauteurs en U ; les U sont comptées depuis le bas de l'armoire à

partir de 1, quelle que soit la numérotation affichée (`uNumbering` ne change que l'affichage, `uNumber`).

Le tableau 4.1 situe les modules.

Module	Rôle
`model/placement`	occupation des U, profondeur, 0U, réduction de hauteur, occupation et espace libre (EQP-02 à EQP-04, EQP-09, ARM-03, ARM-05)
`model/rackGeometry`	géométrie du dessin d'une face, fenêtre d'une plage d'U (ARM-04, PAG-16)
`model/dropTarget`	U sous le pointeur, place libre la plus proche, noms, flèches, retournement (EQP-01, EQP-06, EQP-09)
`model/depthInfo`	infobulle de profondeur d'une U (EQP-05)
`model/links`	une liaison par port, compatibilité, type proposé, repères, chemin de bout en bout (LIA-03 à LIA-11)
`model/anomalies`	panneau Vérifications (LIA-07, EQP-08)
`model/instancePorts`	ports ajoutés à un équipement placé (PRT-04)
`model/arrange`	grille, guides, alignement, répartition (PAG-08)
`model/autoLayout`	mise en place automatique des vues (PAG-13, PAG-17, EXT-04, réduction d'une page trop petite)

Tab. 4.1 : Modules des règles métier.

4.2 Placement dans une armoire

4.2.1 Trois sortes de placement

Le placement d'un équipement (`device.placement`) est une union de trois formes :

- `{ kind: 'rack', rackId, uBottom, mountFace, halfSide }` : fixé dans les U, de `uBottom` à `uBottom + heightU - 1`, sur la face `mountFace` ; il s'étend vers l'autre face sur sa profondeur. `halfSide` ne sert qu'aux machines demi-largeur ;
- `{ kind: 'zeroU', rackId, face, side, slot }` : équipement 0U (bandeau de prises vertical) dans l'emplacement latéral numéro `slot` (à partir de 0, de l'extérieur vers l'intérieur) du côté `side` de la face `face` (EQP-04) ;
- `{ kind: 'external' }` : équipement extérieur, hors armoire, montré par des vues de page (EXT-xx).

La nature de la machine décide de la forme admise : une machine extérieure ne se place qu'en `external`, une machine de hauteur 0 qu'en `zeroU`, les autres qu'en `rack`. Une incohérence donne les refus `notRackable`, `needsZeroU` ou `needsU`.

4.2.2 Le contrôle `checkPlacement`

`checkPlacement` est le point d'entrée unique ; tout chemin qui pose ou déplace un équipement passe par lui : pose depuis la bibliothèque et déplacement (`placeDevice`, `moveDevice`, `store/actions.ts`), aperçu du glisser-déposer (`ui/canvas/Canvas.tsx`), collage et duplication (`nearestFreePlacement`), flèches (`moveDevicesBy`), retournement (`flipDevices`), import tableur (`import/rules.ts`) et panneau Vérifications (`checkDevicePlacement`). Il prend un `PlacementCandidate` et procède ainsi :

1. machine inconnue → `unknownMachine` ;
2. cohérence nature / forme du placement (voir ci-dessus) ;
3. armoire inconnue → `unknownRack` ;
4. pour un placement dans les U :
   (a) débordement : `uBottom < 1` ou U haute au-dessus de la hauteur de l'armoire → `outOfRack`, et l'examen s'arrête là ;
   (b) **même face** (EQP-02, règle 1) : chaque autre équipement fixé sur la même face et qui partage au moins une U est en conflit, sauf si les deux sont des demi-largeurs de côtés physiques opposés. Tous les conflits sont réunis dans un seul problème `overlap` (U triées sans doublon, équipements en cause) ;
   (c) **profondeur** (EQP-02, règle 2) : pour chaque équipement fixé sur l'autre face qui partage une U (et qui n'est pas une demi-largeur du côté opposé), un problème `depth` si la somme dépasse la profondeur utile (section 4.2.4) ;
5. pour un emplacement 0U : l'emplacement doit exister (`zeroUSlotCount`, sinon `zeroUSlotMissing`) et être libre (sinon `zeroUSlotTaken`). Les équipements 0U ne participent ni au contrôle d'occupation des U ni à celui de profondeur.

Les problèmes trouvés deviennent des `PlacementMessage`. Le résultat est `ok` s'il n'y en a aucun ; `ok` avec `forced: true` et les dépassements en `warnings` si tous sont des dépassements de profondeur et que le candidat est forcé ; `refused` sinon, avec `forceable` vrai quand « Forcer » permettrait de passer. L'équipement exclu (`excludeDeviceId`) est celui que l'on déplace : il ne se gêne pas lui-même.

4.2.3 Demi-largeur et retournement

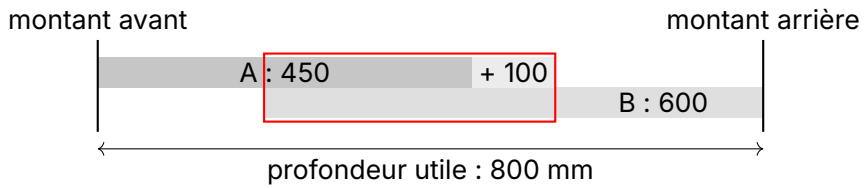
Un équipement demi-largeur (EQP-03, machine `width: 'half'`) occupe la moitié gauche ou droite de ses U. Le côté stocké, `halfSide`, est celui *vu depuis la face de montage* : c'est ce que l'utilisateur voit à l'écran quand il dépose l'équipement sur une vue (décision du 2026-11-05, lot 42). Les règles comparent en revanche le côté *physique*, vu de l'avant de l'armoire (`physicalHalfSide`) : inversé pour un équipement fixé à l'arrière. Ainsi un équipement à l'avant gauche et un autre fixé à l'arrière, côté droit vu de l'avant, ne sont pas l'un derrière l'autre : ils ne se gênent ni en occupation ni en profondeur. Une demi-largeur sans côté choisi (`null`) occupe toute l'U.

Retourner un équipement (EQP-09) change seulement sa face de montage : il n'y a pas de drapeau « retourné » (décision du lot 01, confirmée au lot 42). La face avant du modèle se trouve alors côté arrière de l'armoire. `flippedPlacement` garde la même U et le même côté physique, donc inverse le côté stocké. `flipDevices` retourne une sélection en tout ou rien : tous les équipements quittent d'abord leur face (deux équipements d'une même U peuvent ainsi échanger leurs faces), puis chacun est contrôlé contre ceux déjà retournés ; un second passage sur les positions finales marque comme forcés tous les équipements touchés par un dépassement accepté, pas seulement le dernier contrôlé.

4.2.4 Contrôle de profondeur

Pour chaque U où un équipement A est fixé à l'avant et un équipement B à l'arrière, il faut (EQP-02) :

$$\text{profondeur}(A) + \text{dégagement}(A) + \text{profondeur}(B) + \text{dégagement}(B) \leq \text{profondeur utile de l'armoire.}$$



Les termes sont construits par une fonction interne (`depthTerms`) partagée par le contrôle et par `rackOccupation` :

- une machine sans profondeur (`depthMm: null`) compte pour **toute la profondeur utile** (terme `unknownDepth`) : elle ne laisse rien à l'autre face, et le panneau Vérifications la signale (`machineWithoutDepth`) ;
- un dégagement nul n'est pas un terme : il n'apparaît pas dans le calcul affiché.

Le message donne le calcul en clair, « 450 + 100 + 600 = 1 150 mm > 800 mm » (`formatDepthCalculation`), et les U partagées (`formatURange`). Choix du lot 02 (2026-11-04) : seul un couple avant / arrière est contrôlé ; un équipement seul plus profond que la profondeur utile est accepté (les serveurs dépassent souvent les montants arrière). Le dépassement de profondeur est le **seul refus forçable** : l'utilisateur peut choisir « Forcer », l'équipement est alors enregistré avec `depthForced: true`, entouré en orange et listé dans les vérifications (`forcedDepth`). Le forçage est mémorisé par équipement : `checkDevicePlacement` le relit, si bien qu'un équipement forcé reste accepté tant que ses seuls problèmes sont des dépassements de profondeur ; tout autre problème en fait un conflit. Au clavier (flèches), un *nouveau* dépassement est refusé : le forçage passe par le glisser ou les propriétés (lot 16).

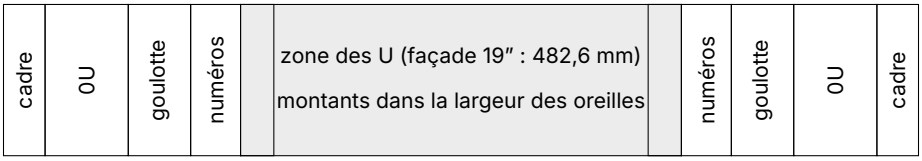
4.2.5 Hauteur, occupation et espace libre

Réduire la hauteur d'une armoire est refusé si un équipement fixé dans les U occupe une U supprimée, avec un message qui les nomme (ARM-03, `checkRackResize`, appelé par la modification d'armoire du magasin). Les équipements 0U ne sont pas contrôlés : leur emplacement couvre toujours toute la hauteur.

`rackOccupation` donne, pour chaque U, les équipements avant et arrière et la profondeur prise de chaque côté (le maximum quand deux demi-largeurs sont sur la même face), avec les mêmes termes que le contrôle : c'est ce qu'affiche l'infobulle de profondeur (EQP-05, `uDepthAt`) et ce qu'exporte le tableau d'occupation. `rackFreeSpace` et `freeSpaceSummary` donnent les U entièrement libres, à moitié libres et les plages libres (ARM-05) : couche « Occupation des armoires » (mode d'affichage, ni enregistré ni exporté) et propriétés de l'armoire.

4.3 Géométrie d'une armoire

`rackGeometry` calcule, en millimètres d'armoire et pour une face, où se trouve chaque élément du dessin (ARM-04, décision du lot 08). C'est la seule source de ces positions : le rendu, le routage (goulottes), le glisser-déposer, les étiquettes et l'éditeur de machine la consultent ; aucun de ces modules ne doit recalculer une position d'U par lui-même. Horizontalement, de l'extérieur vers le milieu :



- la zone des U a la largeur d'une façade 19″ (482,6 mm) ou 10″ (254 mm) ; les montants occupent la largeur des oreilles, de part et d'autre de l'ouverture (450 ou 222,25 mm) ; trois trous par U aux cotes EIA-310 ;
- chaque zone latérale porte, côté extérieur, les emplacements 0U de cette face (30 à 40 mm chacun) et, côté intérieur, la goulotte (au moins 12 mm, réservée au routage, ROU-01) ;
- le dessin garde la largeur de l'armoire, sauf si la place manque pour le côté le plus chargé (toutes faces confondues) : il est alors élargi, à l'identique pour les deux faces, pour que les vues avant et arrière restent alignées ;
- verticalement : toit (armoire fermée ou murale) ou simple cadre (ouverte), zone des U (44,45 mm par U), socle (armoire au sol) ou cadre (murale), puis pieds ou roulettes.

Gauche et droite s'entendent *vus depuis la face dessinée* : un emplacement 0U « gauche » de la face arrière est à droite vu de l'avant.

4.3.1 Plage d'U d'une vue

Une vue d'armoire peut n'afficher qu'une plage d'U (PAG-16, lot 47), stockée dans `view.uRange` en U physiques. `effectiveURange` la ramène dans l'armoire au moment de l'usage (une armoire rendue plus basse garde une plage valide ; une plage qui couvre tout équivaut à `null`). `rackWindow` donne la partie du dessin montrée : le contenu est coupé net aux limites des U (`clipTop`, `clipBottom`) ; au-delà d'une limite coupée, le cadre continue sur 40 mm et se termine par un zigzag (`cutLines`, `cutOutline`) ; toit, socle et pieds ne sont dessinés que si la plage les atteint. Toute conversion de millimètres d'armoire vers la page doit tenir compte de `window.top` (fonction `rackToPage` de `model/pageGeometry`).

4.4 Placer à la souris et au clavier

`model/dropTarget` traduit les gestes en placements, qu'il fait ensuite contrôler par `checkPlacement` :

- `rackViewAt` trouve la vue d'armoire sous le pointeur (la dernière dessinée l'emporte) et convertit le point en millimètres d'armoire ;
- `placementAt` aimante sur l'U sous le pointeur (EQP-01). Un modèle venu de la bibliothèque est tenu par sa U du haut ; un équipement déplacé garde l'U par laquelle il a été saisi (paramètre `grabU`). L'équipement est fixé sur la face de la vue où il est déposé ; une demi-largeur prend le côté vu à l'écran ; un 0U va dans l'emplacement sous le pointeur. La position est ramenée dans la plage d'U de la vue. L'aperçu est vert (accepté), orange (dépassement forçable) ou rouge (refusé), et le dépôt est toujours accepté par le navigateur pour que le refus soit expliqué par un message (lot 16) ;

- `nearestFreePlacement` cherche, pour coller ou dupliquer, la place libre la plus proche en alternant au-dessus et au-dessous de l'U d'origine ; une demi-largeur essaie l'autre moitié de chaque U avant l'U suivante ;
- `moveDevicesBy` déplace une sélection d'une U par flèche (EQP-06), tout ou rien, en traitant d'abord l'équipement le plus avancé dans le sens du mouvement ;
- noms automatiques : préfixe par nature d'équipement et compteur sur deux chiffres (`nextDeviceName`) ; nom rendu unique au collage (`uniqueDeviceName`). Les doublons restent possibles par saisie : ils ne sont qu'un avertissement (EQP-08).

Face de pose (décision de l'utilisateur du 2026-11-06, qui tranche le point ouvert du lot 42) : le glisser-déposer fixe l'équipement sur la face de la vue où il est déposé, et non sur la face de montage par défaut de la machine. La fenêtre « Placer un équipement », elle, propose la face par défaut.

4.5 Liaisons

4.5.1 Contrôles d'une liaison

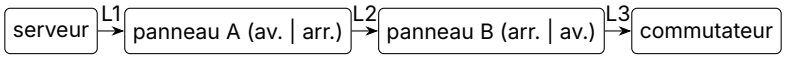
Créer une liaison (`addLink`, `store/actions.ts`) enchaîne deux niveaux de contrôle (LIA-06) :

1. **bloquant** : `checkLinkEnds` refuse deux fois le même port, un port inexistant, un port désactivé sur l'équipement, ou un port qui porte déjà une liaison, sauf s'il est marqué « multiple » ;
2. **avertissement** : `checkCompatibility` signale un connecteur non admis par le type de liaison (quand le type restreint ses connecteurs) et deux connecteurs incompatibles. Deux connecteurs sont compatibles s'ils sont identiques, complémentaires (C13 et C14) ou de la même famille (un module SFP+ et un LC) ; décision du lot 03 pour ne pas avertir sur chaque jarretière branchée dans un module optique tout en signalant RJ45 contre C13. L'interface propose « Créer quand même » ; la liaison est alors marquée `compatibilityForced` et reste listée dans les vérifications.

Le type proposé (LIA-03, `proposeLinkTypeForPorts`) est le premier type de la bibliothèque qui admet les deux connecteurs, puis le premier qui en admet un, puis le type par défaut (Ethernet). Le repère automatique (LIA-04, `nextLinkRef`) est le préfixe du type suivi du plus grand numéro existant plus un, sur trois chiffres (« ETH-013 ») ; les trous ne sont pas réutilisés, et un repère en double n'est pas refusé mais listé (`duplicateRefs`).

4.5.2 Chemin de bout en bout

Un panneau de brassage relie chaque port avant à un port arrière du même module : le champ `pairedPortId` du port de la machine. `endToEndPath` suit cette continuité depuis les deux extrémités d'une liaison, sans limite de nombre de panneaux (LIA-09) :



1. partir de l'extrémité B de la liaison : si le port atteint a un port de traversée (`pairedPort`) et que celui-ci porte une autre liaison, ajouter cette liaison au chemin, orientée, et recommencer depuis son autre extrémité ;

2. s'arrêter quand le port n'a pas de traversée (équipement final), quand la traversée n'est pas reliée (le chemin s'arrête sur le panneau, décision du lot 03), ou quand la liaison suivante a déjà été vue : c'est une **boucle**, signalée par `loop` ;

3. faire de même depuis l'extrémité A (avec le même ensemble de liaisons vues), puis mettre bout à bout la moitié côté A retournée, la liaison elle-même et la moitié côté B.

Le résultat (`EndToEndPath`) donne les liaisons dans l'ordre et les deux ports finaux. Il alimente les propriétés de la liaison, le tableau des liaisons, le carnet de câblage et les étiquettes (exports), l'anomalie `pathLoop` et la mise en évidence du chemin à l'écran (LIA-11, `pathElements`, lot 40). Une liaison sans panneau est son propre chemin.

4.6 Anomalies

`listAnomalies` recalcule toute la liste à chaque modification du projet ; le panneau Vérifications l'affiche avec son nombre sur l'onglet, et un clic sélectionne les cibles de l'anomalie (LIA-07). Le tableau 4.2 résume les natures.

Nature	Gravité	Condition
`danglingLink`	erreur	une extrémité de la liaison n'existe plus
`portOverbooked`	erreur	port non « multiple » portant plusieurs liaisons
`disabledPortLinked`	avert.	liaison sur un port désactivé
`incompatibleLink`	avert.	avertissement de `checkCompatibility`
`pathLoop`	avert.	boucle dans le chemin de bout en bout
`duplicateRef`	erreur	repère porté par plusieurs liaisons
`emptyRef`	avert.	liaison sans repère
`duplicateDeviceName`	avert.	même nom, casse et espaces de bord ignorés (EQP-08)
`placementConflict`	erreur	position refusée par `checkDevicePlacement`
`forcedDepth`	avert.	dépassement de profondeur accepté par « Forcer »
`machineWithoutDepth`	avert.	machine placée en armoire sans profondeur
`orphanPort`	avert.	port de panneau relié d'un seul côté

Tab. 4.2 : Anomalies du panneau Vérifications.

Les conflits de placement apparaissent dans un projet ouvert depuis un fichier, importé, ou modifié par un changement de machine (une machine devenue plus haute) : les actions interactives, elles, refusent de les créer. Le panneau en est le filet de sécurité.

4.7 Ports ajoutés à un équipement

Un équipement placé peut recevoir des ports que sa machine n'a pas (PRT-04, lot 40), par exemple une carte d'extension. Ils sont stockés sur l'équipement (`extraPorts`), jamais sur la machine, et ne peuvent pas être traversants. `model/instancePorts` fournit le nom par défaut (« X1 », « X2 »…, `instancePortName`), la première place libre d'une grille parcourue de haut en bas et de droite à gauche (`freePortPosition`), et les contrôles des propriétés : nom non vide et unique sur la face, connecteur connu, port entièrement dans la face (`instancePortProblems`). Tout le code qui énumère ou résout des ports passe par `model/lookup`, qui réunit ports de la machine et ports ajoutés : les règles de liaison s'appliquent donc à eux sans cas particulier.

4.8 Disposition des pages

4.8.1 Alignement et aimantation

`model/arrange` (PAG-08, lot 48) travaille sur des rectangles en millimètres de papier, ceux des vues (`viewBounds`, qui tient compte de la plage d'U) et des éléments de l'onglet Schéma, et rend des décalages que les actions appliquent en une action annulable :

- pendant un glisser, `snapMove` aligne, sur chaque axe, le bord ou le centre le plus proche d'un bord ou d'un centre d'un autre rectangle, à moins de 6 pixels d'écran (convertis en millimètres par la zone de dessin), et produit un guide ; sans guide proche et avec la grille de 5 mm active, le coin haut gauche va sur la grille. Alt désactive l'aimantation (côté interface) ;
- `alignOffsets` aligne une sélection sur un bord ou un centre de son union ; `distributeOffsets` répartit à écarts égaux en gardant le premier et le dernier.

4.8.2 Mise en place automatique des vues

`model/autoLayout` place les vues créées par Portolan :

- `rackViewsLayout` : vues d'une armoire ajoutée, à droite des vues existantes, à l'échelle de la première vue d'armoire de la page (ou à la plus grande qui tient) ; étiquettes à côté pour une vue avant (PAG-17, lot 46). Les vues existantes ne sont jamais déplacées ni réduites, même si les nouvelles dépassent le bord de la page ou chevauchent d'autres éléments (décision de l'utilisateur du 2026-11-06, PAG-03, qui remplace la réduction de toutes les vues décidée au lot 46) ;
- `rackPageLayout` : page par armoire (PAG-13, lot 43) ; vue avant, vue arrière, puis extérieurs reliés directement, en colonnes ; une seule échelle, la plus grande qui fait tout tenir au-dessus du tableau des révisions ;
- `externalViewLayout` : vue d'un équipement extérieur ajoutée à une page (EXT-04), qui reprend l'échelle, la taille et les ports masqués de sa vue sur une autre page.

Ces fonctions rendent des vues sans identifiant (`Omit<View, 'id'>`) : c'est l'action du magasin qui les crée, dans la même action annulable que l'ajout (`store/rackActions.ts`, `store/pageActions.ts`).

4.9 Limites connues

- Le contrôle de profondeur ignore les équipements 0U et un équipement seul plus profond que l'armoire (choix du lot 02).
- `formatURange` ne signale pas les trous : U 10 et 12 seules s'écrivent « 10 et 12 », mais 10, 12 et 13 s'écrivent « 10 à 13 ».
- L'infobulle de profondeur ne traite pas les 0U et calcule l'U sous le pointeur sans tenir compte de la bande dessinée d'une vue à plage d'U : dans la marge de 40 mm au-delà d'une coupure, elle peut décrire une U hors de la plage.
- Les vues ajoutées par `rackViewsLayout` peuvent sortir de la zone de dessin ou chevaucher un élément de l'onglet Schéma ou une vue d'équipement extérieur : c'est voulu (décision de l'utilisateur du 2026-11-06), l'utilisateur dispose la page.

4.10 Ajouter ou modifier une règle

4.10.1 Une nouvelle raison de refuser un placement

1. Ajouter une variante à `PlacementProblem`, avec les données utiles au message.
2. La produire dans la fonction interne concernée (`rackProblems` pour les U, `zeroUProblems` pour les 0U, ou la cascade de `checkPlacement`).
3. Ajouter son cas au `switch` de la fonction interne `describe` (TypeScript signale un oubli : le `switch` est exhaustif) et la clé `placement.<nature>` dans `src/i18n/fr.json`, avec les formes `_one` / `_other` si le message dépend d'un nombre.
4. Décider si elle est forçable (aujourd'hui, seule `depth` l'est : la ligne `forceable: problem.kind === 'depth'`); un nouveau refus forçable demande aussi de revoir le drapeau `depthForced` et l'anomalie `forcedDepth`.
5. Tester dans `src/model/placement.test.ts` le refus, le message et, le cas échéant, le forçage ; vérifier que `dropTarget.test.ts` (collage, flèches, retournement) passe toujours, puisque ces fonctions reposent sur le même contrôle.

Le panneau Vérifications reprend automatiquement le nouveau refus (`placementConflict`), et l'aperçu du glisser-déposer le colore en rouge.

4.10.2 Une nouvelle anomalie

1. Ajouter la nature à `AnomalyKind` et, si c'est une erreur, à l'ensemble `ERRORS` de `anomalies.ts`.
2. Ajouter le message `anomalies.<nature>` dans `fr.json`.
3. Calculer l'anomalie dans `listAnomalies` avec `add`, en donnant des cibles sélectionnables (équipement, liaison, machine ou armoire).
4. Tester dans `links.test.ts` (tests des anomalies) : présence, gravité, cibles, et absence sur un projet sain (le projet d'exemple ne doit présenter aucune anomalie, `store/exampleProject.test.ts`).
5. Mettre à jour le manuel (`manual/`) qui décrit le panneau Vérifications.

4.10.3 Une règle de liaison

Un refus va dans `checkLinkEnds` (nouvelle variante de `LinkEndProblem`, clé `links.<nature>`), un avertissement dans `checkCompatibility`. Les deux sont appelés par `addLink` et par l'import tableur (`import/rules.ts`) : vérifier les tests des deux. Si la règle doit aussi porter sur les liaisons existantes, ajouter l'anomalie correspondante.

4.10.4 Changer la géométrie d'une armoire

Toute cote du dessin se change dans `model/rackGeometry` et nulle part ailleurs. Mettre à jour `rackGeometry.test.ts` et `viewRange.test.ts`, lancer les tests du routage (les goulottes en dépendent), puis régénérer et examiner les captures de référence touchées (`npx playwright test --update-snapshots=all`) en signalant le changement visuel dans le commit (règle 8).

Chapitre 5

Pages, vues, cartouche et onglet Schéma

Ce chapitre explique comment Portolan passe d'un projet (armoires, équipements, liaisons) à des *pages* imprimables : format et zone de dessin, *vues* d'armoires et d'équipements extérieurs posées sur la page, échelle, plage de U et câbles masqués, cartouche configurable et légende, éléments libres de l'onglet « Schéma ». Il couvre aussi trois fonctions du modèle qui s'appuient sur cette géométrie ou vivent à côté d'elle : la recherche globale, la logique de l'éditeur de machine, et le traitement des bibliothèques (arborescence, import, export, mode édition). Toute cette logique est pure, hors de React, et testée unitairement ; la référence détaillée est au chapitre « Référence : pages, vues, cartouche, Schéma, recherche et bibliothèques ».

5.1 Repères et unités

Une page est un `<svg>` dont l'unité est le *millimètre de papier* (décision du 2026-11-04, lot 12). Trois repères coexistent ; confondre l'un avec l'autre est la première source d'erreurs dans ce domaine.

Repère	Contenu	Passage
mm réel d'armoire	dessin d'une face d'armoire (`RackGeometry`), origine au coin haut gauche du cadre, y vers le bas	`rackToPage`
mm de papier	page, vues, cartouche, éléments de l'onglet Schéma, tracés des câbles	échelle de la vue
proportion de face	position (centre) et taille d'un port dans la face de sa machine, entre 0 et 1	`portBox`

Une vue d'armoire applique au dessin de la face `translate(x y) scale(échelle)`, précédé d'un décalage vertical de `-window.top` quand sa plage de U coupe le haut de l'armoire. La formule de `rackToPage` est donc :

```
x page = x vue + x armoire × s,   y page = y vue + (y armoire − window.top) × s
```

où s est l'échelle *effective* (`effectiveScale`). Toute conversion du millimètre d'armoire vers la page doit passer par cette fonction ou par `deviceOnPage` / `portOnPage`, qui l'appliquent déjà : le rendu, le routage, le glisser-déposer, les étiquettes à côté de l'armoire et les contrôles des tests l'utilisent tous.

5.2 Format de page et zone de dessin

Le format (`A4` à `A0`, `Letter` ou `custom`), l'orientation et les marges sont des champs de la page (PAG-02). `PAPER_SIZES` donne les dimensions en portrait ; `pageSize` les oriente en prenant le petit et le grand côté, si bien qu'une taille personnalisée saisie « à l'envers » reste correcte. La *zone de dessin* (`drawingArea`) est l'intérieur des marges, moins les 26 mm du cartouche (`TITLE_BLOCK_HEIGHT`) en bas. Le cartouche occupe la bande juste en dessous ; le tableau des révisions (PAG-14) se place par défaut au-dessus de lui, dans la zone de dessin (sa hauteur est calculée par `revisionTableHeight`).

La mise en page automatique (page par armoire, PAG-13 ; place des vues d'une armoire ajoutée, après les vues existantes, qui ne bougent pas) vit dans `model/autoLayout`, décrite au chapitre du modèle ; elle s'appuie sur `viewBounds` et `sideLabelsWidth` pour mesurer les vues.

5.3 Vues

5.3.1 Trois sortes de vues

Une vue (`View`, PAG-03) est posée librement sur une page par son coin haut gauche (`x`, `y`, mm de papier) et montre :

- `rackFront` ou `rackRear` : une face d'armoire (`targetId` est l'armoire). Une même armoire peut apparaître plusieurs fois, sur une ou plusieurs pages ;
- `external` : un équipement extérieur (`targetId` est l'équipement), à la taille d'affichage de sa machine multipliée par l'échelle, ou à une taille forcée (`size`, redimensionnement non proportionnel, EXT-02).

La face montrée est donnée par `viewFace`. Le rectangle d'une vue, `viewBounds`, exclut son titre (réservé au-dessus, `VIEW_TITLE_SIZE`) et ses étiquettes à droite.

5.3.2 Échelle (PAG-05)

Chaque vue a son échelle (mm de papier par mm réel ; 0,2 pour du 1:5). Avec l'option « même échelle » de la page (`uniformRackScale`), toutes les vues d'armoires sont dessinées à la plus petite de leurs échelles ; les vues extérieures gardent la leur. Le champ `scale` de chaque vue n'est pas modifié : c'est `effectiveScale` qui tranche à chaque calcul. Une fonction qui utiliserait directement `view.scale` pour une vue d'armoire serait fausse dès que l'option est cochée.

5.3.3 Équipements et ports sur la page

`deviceInRackFace` place un équipement dans le dessin d'une face (mm d'armoire) et dit quelle face de sa machine y est vue : depuis sa face de montage, la face avant de la machine ; depuis l'autre, sa face arrière. Une machine demi-largeur occupe sa moitié physique, en miroir à l'arrière (lot 42). Un équipement 0U n'est vu que sur sa face.

`deviceOnPage` rend toutes les places où un équipement est dessiné sur une page, avec deux rectangles : `box`, tel que dessiné (coupé à la plage de U) et `fullBox`, entier, dans lequel les ports sont placés. `portOnPage` en déduit les rectangles d'un port, en ne gardant que les vues qui montrent la face de la machine qui porte ce port. Pour une vue extérieure aux ports

masqués (EXT-03), le « port » est un petit carré au milieu du bas de l'équipement : c'est là qu'aboutit le câble.

5.3.4 Plage de U (PAG-16)

Une vue d'armoire peut ne montrer qu'une plage de U (`uRange`, U physiques comptées depuis le bas, bornes comprises ; `null` = toute l'armoire). La décision du 2026-11-05 (lot 47) fixe le fonctionnement :

1. La plage est d'abord ramenée à l'armoire (`effectiveURange`) : une plage qui couvre toute l'armoire équivaut à `null`.
2. `rackWindow` calcule la fenêtre : la bande où le contenu est dessiné (`clipTop`, `clipBottom`, limites exactes des U) et, au-delà d'une limite coupée, 40 mm réels de cadre terminés par un zigzag (`top`, `bottom`).
3. La hauteur de la vue est celle de la fenêtre (`viewBounds`), et le dessin est décalé de `window.top` (`rackToPage`).
4. Un équipement à cheval sur une limite est coupé net (`clipToWindow`) ; entièrement hors de la bande, il n'est pas sur la page.
5. Un port est sur la page si son *centre* est dans la partie dessinée de son équipement ; sinon il est « coupé » : pas de liaison tirée depuis lui, et un câble vers lui suit la goulotte jusqu'à la coupure et se termine par un renvoi « → équipement · port » (routage, chapitre correspondant).
6. `devicesOnPage` (légende, numéro de page des renvois) ne compte plus un équipement hors de toutes les plages des vues de son armoire, et `rackSideLabels` n'étiquette que les équipements dessinés, alignés sur le milieu de leur partie visible.

Changer la plage conserve les tracés retouchés des câbles (décision de l'utilisateur, lot 47) : leurs points restent attachés au coin de la vue.

5.3.5 Câbles masqués (PAG-15)

`cablesHidden` masque les câbles *dans cette vue seulement* : aucun câble n'y est dessiné ni n'occupe de piste, ses ports restent visibles. Les autres vues de la page gardent leurs câbles ; un câble dont une extrémité n'est dessinée que dans des vues masquées se termine par un renvoi à la limite de l'autre vue. La géométrie de ce chapitre n'est pas touchée : tout se passe dans le routage, à partir de `hiddenEnds`, qui donne les extrémités dessinées seulement dans des vues aux câbles masqués (décision du 2026-11-05, lot 47).

5.3.6 Étiquettes à côté de l'armoire (PAG-06, PAG-17)

Avec `labelsBeside` (coché par défaut pour une nouvelle vue avant, lot 46), une colonne de 70 mm est réservée à 6 mm à droite de l'armoire (`SIDE_LABELS`). `rackSideLabels` calcule les étiquettes en millimètres d'armoire : chacune est ancrée au milieu de son équipement, puis, de haut en bas, décalée vers le bas si elle est à moins d'une hauteur de ligne de la précédente. L'algorithme ne remonte jamais une étiquette : sur une armoire très chargée, les dernières peuvent descendre sous le bas de l'armoire. Le texte suit le mode d'étiquette de l'équipement (`deviceLabel`, EQP-07).

5.3.7 Légende (PAG-11)

La légende liste les types de liaisons *effectivement utilisés* sur la page : `legendLinkTypes` retient les types d'au moins une liaison dont une extrémité est sur un équipement visible de la page, dans l'ordre de la bibliothèque, sauf ceux dont `showInLegend` est faux. Elle se dessine dans le cartouche (par défaut) ou en bloc libre (`page.legend`).

5.4 Cartouche (PAG-09, PAG-10)

Le cartouche est une liste ordonnée de champs au niveau du projet (`project.titleBlock.fields`) : logo, nom du projet, client, titre de la page, indice et date de révision, auteur, numéro de page « n sur N », champs libres. Chaque champ a un libellé (vide = libellé par défaut), une valeur (champ libre), un drapeau `visible` et une position `row` (`top`, `bottom`, `full` ou `null` = position de sa sorte, `defaultTitleBlockRow`).

La décision du 2026-11-05 (lot 43) rend la « position des champs » du cahier par *l'ordre de la liste* et cette position, plutôt que par des coordonnées libres (écartées : grille difficile à garder lisible sur toutes les tailles de page). `titleBlockLayout` procède ainsi :

1. Parcourir les champs visibles dans l'ordre. Un champ `full` ajoute une colonne. Le premier champ `top` ou `bottom` ajoute la colonne du *bloc d'informations* à cet endroit ; tous les suivants s'y rangent dans leur ligne.
2. Réserver la légende à droite (24 % de la largeur) si elle est dans le cartouche, puis chaque logo (12 %, 45 mm au plus).
3. Les autres colonnes pleine hauteur prennent chacune 22 % de la largeur, et toutes ensemble au plus 60 % de la place restante ; sans bloc d'informations, elles se partagent la place à parts égales. Le bloc d'informations prend le reste.
4. Dans le bloc, chaque ligne non vide prend une part égale de la hauteur, et ses champs une part égale de la largeur.

<table>
<tr><td rowspan="2">logo</td><td colspan="3">nom du projet</td><td rowspan="2">titre de la page</td><td rowspan="2">légende</td></tr>
<tr><td>révision</td><td>date</td><td>auteur…</td></tr>
</table>

Le cartouche par défaut (ci-dessus, schématique) est identique à celui d'avant le lot 43 ; la capture de référence `page-a4-reference` le garantit. Le dessin (`src/render/TitleBlock.tsx`) ne fait que remplir les cases calculées.

5.5 Onglet Schéma (PAG-07, PAG-08)

5.5.1 Modèle

Les éléments de l'onglet Schéma sont les `annotations` de la page (décision du 2026-11-05, lot 48) : ils appartiennent à la page et ne suivent pas les vues. Deux familles :

- les *lignes* (`LineAnnotation` : `line`, `arrow`), définies par leurs points, éventuellement coudées (`elbow`, premier segment selon `elbowStart`), avec une ou deux pointes ;

- les *cadres* (`BoxAnnotation` : texte, rectangle, rectangle arrondi, ellipse, image, zone, bulle, note, tableau), définis par un rectangle avant rotation et une rotation par quart de tour.

Champs communs : `color`, `layer` (`below` : dessiné avant les vues ; `above` : après les câbles, avant le cartouche) et `rotation`. Tous les champs ajoutés au lot 48 ont une valeur par défaut, si bien que le format 2 n'a pas changé.

Les éléments proposés par l'onglet sont des *préréglages* (`ANNOTATION_GROUPS`) : `doubleArrow` et `elbowArrow` créent une `arrow` avec `doubleHead` ou `elbow`. `createAnnotation` fixe les valeurs par défaut et centre l'élément sur le point de dépôt (ou le centre de la zone de dessin pour un clic). Une image glissée laisse un cadre vide à remplir (un dépôt ne peut pas ouvrir de sélecteur de fichier) ; une image cliquée demande d'abord le fichier, et son cadre prend ses proportions.

5.5.2 Géométrie

- *Encombrement* : `annotationBounds` échange largeur et hauteur d'un cadre tourné de 90° ou 270° autour de son centre. Il sert à la sélection au rectangle, aux poignées, aux guides et à l'alignement. Les pointes et l'épaisseur du trait n'y sont pas comptées.
- *Redimensionnement* : `setAnnotationBounds` part des bornes *visibles* voulues. Un cadre tourné reconstruit son cadre local autour du nouveau centre ; une ligne met ses points à l'échelle, et, le long d'un axe sans étendue, les répartit pour que la ligne s'incline.
- *Rotation* : `rotateAnnotation` ajoute 90° à un cadre ; une ligne fait tourner ses points et bascule `elbowStart`, sans quoi une flèche coudée serait retournée en miroir.
- *Dessin* : `linePath` (tracé coudé à un seul coude), `arrowHeads`, `dashArray` et `rotationTransform` alimentent `AnnotationsLayer`, qui dessine le même SVG à l'écran et dans les exports PDF, SVG et PNG (NF-07).

5.5.3 Gestes, alignement et répartition

Les actions annulables sont dans `src/store/annotationActions.ts`. `moveItems` déplace ensemble vues et éléments sélectionnés depuis leur position de départ : un glisser est une seule action. Les guides d'alignement (PAG-08) aimantent bords et centres à 6 pixels d'écran près sur ceux des autres vues et éléments (`snapMove`, Alt désactive) ; la grille de 5 mm est un état de l'interface, ni enregistré ni exporté. Aligner et répartir (`alignOffsets`, `distributeOffsets`) travaillent sur les rectangles de `viewBounds` et `annotationBounds`, et rendent des décalages à appliquer.

5.5.4 Limites connues

Pas de rotation libre ; un texte se saisit dans les propriétés, pas sur la page ; pas de fusion de cellules dans un tableau ; une flèche coudée n'a qu'un coude ; les éléments ne sont pas des obstacles pour le routage des câbles. Une vue ajoutée par Portolan (ajout d'armoire, de vue, duplication) peut recouvrir un élément : les vues existantes et les éléments ne sont jamais déplacés automatiquement (décision de l'utilisateur du 2026-11-06).

5.6 Sélection au rectangle (ERG-02)

`selectInRectangle` retient les vues entièrement contenues, et, dans une vue d'armoire touchée mais non contenue, les équipements dont le rectangle dessiné est entièrement contenu ; `annotationsInRectangle` ajoute les éléments de l'onglet Schéma. Une vue extérieure se sélectionne comme une vue, jamais comme un équipement.

5.7 Recherche globale (ERG-05)

La commande `edit.find` (Ctrl+F) ouvre la fenêtre « Rechercher », qui appelle `searchProject` à chaque frappe (décision du 2026-11-05, lot 40).

1. La recherche est découpée en mots normalisés (`normalizeSearch` : minuscules, accents retirés). Chaque mot doit être trouvé.
2. Câbles : dans le repère. Équipements : dans le nom, la description, les clés et valeurs des champs libres, le nom de la machine. Ports : dans « équipement port », avec au moins un mot dans le nom affiché du port, pour que le nom d'un équipement seul ne liste pas tous ses ports ; les ports ajoutés à l'équipement comptent.
3. Chaque résultat reçoit un rang : 0 si le nom est exactement la recherche (ou l'un de ses mots), 1 s'il commence par un mot cherché, 2 sinon.
4. Tri par rang, puis câbles, équipements, ports, puis ordre naturel (`localeCompare` numérique : « SW2 » avant « SW10 ») ; cent résultats au plus, avec le total.

Choisir un résultat active la page qui le dessine (`pageOfTarget` : la page active d'abord) et fait défiler la zone de dessin jusqu'au rectangle de `targetBoxOnPage`. La recherche du panneau Bibliothèque est une autre fonction (`matchesSearch`), qui ignore la casse mais pas les accents.

5.8 Éditeur de machine (EDM-03 à EDM-07)

`model/modelEditor` réunit les règles de l'éditeur de machine qui ne dépendent pas de l'interface (décisions des lots 18 et 44).

- *Gabarit* : `faceTemplateSize` (19 pouces ou demi-largeur sur la hauteur en U, ou taille d'affichage d'un extérieur). Une image est toujours ajustée sans déformation dans ce gabarit ; `checkImageRatio` avertit au-delà de 2 % d'écart et propose 2 000 px de large et la hauteur correspondante. Il n'y a pas d'éditeur d'images (EDM-03).
- *Poids* : `isImageTooHeavy`, seuil de 1 Mo par défaut, réglable dans les préférences du navigateur (EDM-04). Aucune recompression.
- *Ports en grille* : `arrangePorts` ne crée pas de ports (ils viennent des paramètres de l'équipement) mais fixe la position et la taille des ports désignés, dans l'ordre de numérotation (EDM-06).
- *Ports placés* : un port déplacé ou redimensionné à la main devient `placed` (`markPlaced`) ; la régénération de la machine garde alors sa position (EQT-05). `unplacePorts` le rend à la disposition automatique.

- *Reprise des positions* (EDM-07) : `positionSources` propose les machines qui ont des ports de même nom sur cette face, celles du même équipement en tête, et accepte des machines d'un autre fichier (lues sans être importées) ; `copyPortPositions` recopie position et taille par nom de port.

5.9 Bibliothèques dans le modèle

5.9.1 Arborescence du panneau (BIB-01)

`libraryTree` range les machines du projet sous les équipements intégrés, eux-mêmes sous leur catégorie (`equipmentTree`). Sans recherche, tous les équipements apparaissent, même sans machine ; avec une recherche, seulement les machines qui correspondent (nom, fabricant, référence, mots-clés, ou nom traduit de leur équipement ou de leur catégorie) et leurs équipements, que le panneau déroule. La machine générique d'un équipement est toujours en tête (`byGenericThenName`).

5.9.2 Identité d'une machine et import (BIB-08, BIB-10, BIB-11)

L'identité d'une machine importée est son *origine* : identifiant de la bibliothèque et identifiant de la machine dans la bibliothèque (décision du 2026-11-05, lot 30). Une machine importée d'un `.rkdglib` reçoit pour origine cette bibliothèque et sa version (`sourceOrigin`) ; « plus récente » veut dire même origine et version de bibliothèque supérieure. Depuis un `.rkdg`, l'origine est gardée, et sans origine c'est l'identifiant qui compte.

L'import se fait en trois temps, par des fonctions pures ; seul le magasin modifie le projet, à la fin :

1. La fenêtre liste les machines de la source et leurs conflits (`importCandidates`). Elle coche d'office les machines absentes et les versions plus récentes, avec le choix « Mettre à jour » (`defaultImportSelection`).
2. `planImport` calcule ce qui sera copié. Les types de connecteurs nécessaires et leurs compléments (`withComplements`) sont rattachés à un type du projet de même identifiant, ou de même nom et même famille, sinon copiés ; les ports et les paramètres de connecteur des machines suivent ce rattachement. Les types de liaisons utiles absents du projet sont copiés, les images déjà présentes ne le sont pas. Selon le choix (`ConflictChoice`), une machine en conflit est ignorée, remplace celle du projet en gardant son identifiant, ou est ajoutée sous un autre nom.
3. `checkImport` applique le plan : chaque remplacement passe par `changeMachine`, comme une machine modifiée, et est *refusé* si un équipement posé ne tiendrait plus (EQT-06) ; les liaisons sur des ports disparus sont listées, conservées et signalées par les vérifications (LIA-07). Le magasin (`applyImportPlan`) refait cette vérification sur le plan complet et applique le tout en une seule action annulable.

5.9.3 Export et mode édition (BIB-12, BIB-13)

`libraryForExport` produit la bibliothèque des machines cochées : copies non génériques avec la nouvelle origine, types de connecteurs et de liaisons nécessaires, images. `proposedDescription` reprend l'identifiant et la version suivante quand toutes les machines viennent d'une même bibliothèque publiée, jamais la bibliothèque par défaut.

En mode édition (décision du 2026-11-05, lot 44), un `.rkdglib` ouvert devient un `Project` marqué par `editedLibrary` (`libraryDocument`) : annulation, état « modifié » et brouillon de secours fonctionnent sans code nouveau. Le document porte la version d'enregistrement, *version ouverte + 1*, la même pour tous les enregistrements de la séance. `libraryToSave` reprend la règle de l'export et garde en plus les types créés dans la bibliothèque, même inutilisés ; le fichier est proposé sous le nom `<nom>_v<version>.rkdglib` (`libraryFileName`), si bien que la version précédente n'est pas écrasée. Limite connue : deux enregistrements d'une même séance portent le même numéro.

5.10 Modifier ce domaine

Ajouter un format de papier
: l'ajouter à `PageFormatSchema` (`src/model/schema.ts`, puis `npm run schema`) et à `PAPER_SIZES` ; ajouter le libellé dans `src/i18n/fr.json` ; compléter `pageLayout.test.ts`.

Ajouter un élément à l'onglet Schéma
: pour une nouvelle sorte, l'ajouter à `AnnotationSchema` (valeurs par défaut pour tous les champs, schémas régénérés) ; ajouter le préréglage à `ANNOTATION_GROUPS` et `PRESET_SIZES` et son cas dans `createAnnotation` (le `switch` exhaustif fait échouer la vérification de types sinon) ; le dessin dans `src/render/AnnotationsLayer.tsx`, les propriétés, la commande `annotation.add.<élément>` et sa description dans `manual/` ; tests `annotations.test.ts` et `tests/e2e/schema.spec.ts`.

Ajouter un champ au cartouche
: ajouter la sorte à `TitleBlockFieldSchema` ; lui donner une position par défaut (`defaultTitleBlockRow`), une valeur et un libellé dans `src/render/TitleBlock.tsx` ; vérifier que la capture `page-a4-reference` ne change pas si le champ n'est pas visible par défaut.

Changer la géométrie d'une vue
: toute nouvelle conversion du dessin d'armoire vers la page passe par `rackToPage` ou `deviceOnPage`, et tout test d'appartenance à une vue par la fenêtre de la plage de U (`inWindow`) ; tests `pageGeometry.test.ts`, captures `vue-plage-u` et e2e `views.spec.ts`.

Chercher un nouvel objet
: ajouter une variante à `SearchTarget`, sa boucle dans `searchProject`, son rang dans l'ordre des sortes et sa place dans `targetBoxOnPage` ; la fenêtre doit savoir le sélectionner.

Changer une règle d'import
: tout passe par `planImport` et `checkImport` ; les tests `libraryTransfer.test.ts`, `libraryEdit.test.ts` et les e2e `library-transfer.spec.ts`, `library-edit.spec.ts` couvrent conflits, mises à jour refusées et liaisons orphelines.

5.11 Points d'attention

- Dans `gridCell`, les numérotations `columns` et `oddTop` donnent la même disposition (colonne par colonne) ; la fenêtre propose pourtant les deux.
- `rackSideLabels` ignore les équipements 0U, et `annotationBounds` ignore pointes et épaisseur de trait.

Chapitre 6

Bibliothèques, équipements intégrés et démarrage

Ce chapitre couvre trois ensembles voisins, tous indépendants de React :

- le **catalogue des équipements intégrés** (`src/library/equipment/`) : types d'équipement, paramètres, générateurs de machines et changements de paramètres d'une machine déjà utilisée ;
- la **bibliothèque par défaut** (`src/library/`) : types de connecteurs et de liaisons, modèles d'armoires, machines génériques, et le projet de départ ;
- ce qui entoure l'application : la **page des bibliothèques de l'équipe** (`src/libraries/`, chiffrée), le **démarrage** (`src/main.tsx`, `src/app/`), l'**internationalisation** (`src/i18n/`) et le lecteur des **notes de version** (`src/help/markdown.ts`).

La référence de chaque déclaration se trouve au chapitre « Référence : bibliothèques, équipements intégrés et démarrage ».

6.1 Équipements, machines et paramètres

6.1.1 Vocabulaire

Le cahier des charges (§ 6 et EQT-01 à EQT-06) distingue trois niveaux :

Équipement un *type* intégré à Portolan (switch, multiprise, écran…), désigné dans le code par un `EquipmentKind`. La liste est fermée et fixée par le code : `EQUIPMENT_KINDS` dans `src/model/schema.ts`. Chaque équipement appartient à une catégorie (`KIND_CATEGORY`) qui range le panneau bibliothèque (BIB-01).

Machine une déclinaison concrète d'un équipement : un objet `Machine` de `project.library.machines`. Elle porte `equipment: { kind, params }`, ses deux faces, leurs ports, sa hauteur, sa largeur, sa profondeur et son origine. Les paramètres *appartiennent à la machine* (EQT-03) : ports, prises et dessin en sont déduits. Une machine peut être *générique* (`generic: true`, EQT-02).

Équipement placé un `Device` du projet, posé dans une armoire ou à l'extérieur ; il désigne sa machine par `machineId` et la suit quand elle change (EQT-06).

Depuis le lot 34 (décision du 2026-11-05, « Équipements intégrés et machines »), il n'y a plus de « modèles » figés : une seule machine générique par équipement dans la bibliothèque par défaut, et l'utilisateur crée ses déclinaisons en changeant les paramètres. Le format de fichier est passé à la version 2 à cette occasion.

6.1.2 Le catalogue

Le tableau 6.1 résume les 19 équipements, leur catégorie, les paramètres principaux (schémas de `library/equipment/params`) et la profondeur par défaut (`DEFAULT_DEPTHS`).

Tab. 6.1 : Les équipements intégrés.

Clé	Catégorie	Paramètres principaux	Prof. (mm)
`switch`, `router`, `firewall`	réseau	hauteur 1–4U, demi-largeur, alimentations (0–2, C14/C20), groupes de ports, console, gestion (STD-02)	300/250/250
`patchPanel`	réseau	hauteur, emplacements, module de remplissage, module et étiquette par emplacement (STD-05)	100
`fiberPanel`	réseau	cassettes, connecteur LC/SC/MPO, ports par cassette (STD-08)	300
`server`, `storage`	serveurs et stockage	hauteur 1–8U, alimentations (0–4), cartes réseau, gestion, VGA, USB avant/arrière ; baies de disques pour le stockage (STD-07)	600/650
`kvm`	serveurs et stockage	ports ordinateurs, console USB et VGA (STD-09)	500
`pdu`	alimentation	forme horizontale ou verticale, hauteur ou longueur, groupes de prises, entrée, fiche, interrupteur, parafoudre (STD-01)	100
`ups`	alimentation	hauteur, groupes de prises, entrée, carte réseau (STD-11)	600
`blankingPanel`	accessoires	hauteur, perforé, couleur (STD-03)	20
`cableManager`	accessoires	hauteur, type (STD-04)	80
`shelf`	accessoires	hauteur, coulissante, ports de la machine posée (STD-06, MAC-03)	450
`monitor`	extérieurs	diagonale, entrées vidéo, USB, alimentation (STD-12)	–
`workstation`, `laptop`, `keyboardMouse`, `controlPanel`	extérieurs	largeur et hauteur en mm, groupes de ports, alimentation (STD-12)	–
`generic`	générique	hauteur 1–52U, demi-largeur, libellé, couleur, groupes de ports libres (STD-10)	300

Un équipement extérieur n'a pas de profondeur (`null`) : sa taille affichée (`displaySizeMm`) remplace la hauteur en U, qui vaut 0. Une PDU verticale a elle aussi une hauteur de 0U et occupe un emplacement 0U de l'armoire.

6.1.3 Paramètres : des schémas Zod à valeurs par défaut

Chaque équipement a un schéma Zod dans `PARAM_SCHEMAS`. Règle centrale : **chaque champ a une valeur par défaut**, de sorte que `parse({})` donne exactement les paramètres de la machine générique (EQT-02). Deux types en découlent : `ParamsInput` (tel que saisi, tout

facultatif) et `Params` (complété). Les paramètres enregistrés dans une machine sont toujours la forme complétée.

Quelques schémas transforment la saisie :

- les groupes de prises (`OutletGroupSchema`) reçoivent un préfixe quand il manque : la première lettre libre (A, B…), calculée par `freeOutletPrefix` et stockée avec les paramètres. Les prises sont ainsi nommées A1, A2…, B1… et la taille d'un groupe ne renomme jamais celles d'un autre ;
- le panneau de brassage (`PatchPanelSchema`) complète `modules` jusqu'au nombre d'emplacements avec le module `fill` (« remplissage rapide ») ;
- la multiprise horizontale est limitée à `HORIZONTAL_PDU_MAX_OUTLETS` prises au total (`refine`).

Les messages d'erreur des schémas passent par `t()` : ils sont en français, et `schemaProblems` les préfixe du chemin du champ.

Connecteurs des groupes de ports (BIB-09, lot 39a). La valeur `connector` d'un groupe de ports libre est soit la *clé* d'un type de connecteur par défaut (`rj45`, `sfpPlus`…), soit l'*identifiant* (UUID) d'un type créé dans la bibliothèque du projet. Le schéma ne contrôle que la forme ; l'existence du type est contrôlée là où le projet est connu, par `connectorProblems`. Les paramètres concernés sont listés par `PORT_GROUP_PARAMS` (`portGroups`, ou `nics` pour serveur et stockage) ; les autres connecteurs (alimentations, prises, modules de panneau, cassettes, entrées vidéo) restent des listes fermées de clés. À l'import d'une bibliothèque, `remapParamConnectors` renumérote ces valeurs, et `connectorValueOf` ramène un type par défaut à sa clé.

6.1.4 Générateurs : des fonctions pures paramètres → machine

`library/equipment/generators` contient un générateur par équipement (table interne `GENERATORS`). Un générateur reçoit les paramètres complétés, une fonction de réutilisation d'identifiants (`IdReuse`) et une fonction de nommage des connecteurs ; il rend la hauteur, la largeur, la taille affichée et les ports des deux faces. `generateMachine` enchaîne :

1. `parseParams` valide et complète les paramètres (lève une erreur Zod s'ils sont invalides) ;
2. construction de la fonction de réutilisation à partir de la machine précédente (`previous`), si c'est une régénération ;
3. appel du générateur de l'équipement ; les ports sont disposés en coordonnées *relatives à la face* (0 à 1) par `grid` ou par une mise en blocs (numérotation colonne par colonne, 1 au-dessus de 2, comme sur un vrai switch) ; les ports arrière d'un panneau sont en miroir (`mirrored`) et appariés à leur port avant (`pairedPortId`, LIA-09) ;
4. refus (exception) si deux ports d'une même face portent le même nom ;
5. fusion des faces avec celles de la machine précédente : mode de rendu, image, couleur et libellé sont conservés (sauf pour l'obturateur et l'équipement générique, dont couleur et libellé sont des paramètres) ; un port de même identifiant garde son sens de sortie et « multiple », et sa position s'il a été placé à la main (`placed`) ;

6. sans machine précédente : nouvelle machine, nom proposé (`proposeMachineName`), profondeur par défaut ; avec elle : même identifiant, nom, marque et profondeur, revalidés par `MachineSchema`.

Stabilité des identifiants de ports. C'est l'invariant qui fait survivre les liaisons à un changement de paramètres. Les noms de ports sont déterministes (préfixe du groupe + numéro, chaque groupe numéroté à partir de 1). La fonction de réutilisation associe à chaque port généré l'identifiant d'un port précédent de *même face, même clé d'identité et même type de connecteur*, jamais deux fois le même ; à défaut, un identifiant neuf. La clé d'identité est le nom du port, sauf pour le panneau de brassage où c'est le numéro d'emplacement (`#1`, `#2`…) : une étiquette (STD-05) renomme le port sans en changer l'identité. Conséquences : ajouter des ports en fin de groupe ne renumérote pas les autres ; changer le connecteur d'un groupe crée de nouveaux ports, et une liaison ne passe jamais sur un port d'un autre connecteur.

Unicité des noms. Les noms des ports d'une face doivent être uniques, sinon la réutilisation serait ambiguë. Des paramètres valides pour leur schéma peuvent pourtant produire un doublon (deux groupes au même préfixe, préfixe « USB » qui rejoint un port de service…). `portNameProblems` les détecte en lançant le générateur à blanc ; `paramsProblems` réunit, dans cet ordre, problèmes de schéma, connecteurs manquants et noms en double. C'est la fonction à appeler avant toute génération venant de l'utilisateur ou d'un fichier (`equipmentParamsProblem` pour la validation à l'ouverture, erreur `invalidParams`).

Machines génériques. `createGenericMachine` génère la machine d'un équipement avec les paramètres par défaut et des identifiants *fixes* : machine `builtinId(0x500 + rang)`, ports `builtinId(0x10000 + rang × 0x1000 + n)`, où le rang est la position dans `EQUIPMENT_KINDS`. Son origine est la bibliothèque par défaut (`DEFAULT_LIBRARY_ID`, version `DEFAULT_LIBRARY_VERSION`). Ces identifiants fixes permettent de réimporter les machines génériques (BIB-07, `missingGenericMachines`) sans doublon. Une machine générique n'est jamais modifiée par ses paramètres : l'interface crée alors une variante.

6.1.5 Changer les paramètres d'une machine utilisée

`library/equipment/change` contient les opérations pures que le store (`src/store/machineActions.ts`) applique ; elles rendent un résultat discriminé (`MachineChange`) : soit `ok: true` avec le nouveau projet, la machine, les liaisons devenues orphelines et un avertissement par port perdu, soit `ok: false` avec des messages en français.

`changeMachineParams` régénère une machine avec de nouveaux paramètres (EQT-06) ;

`changeMachine` remplace une machine éditée dans l'éditeur de machine ;

`deriveMachine` crée une variante d'une machine (générique notamment) et la donne à certains équipements placés ;

`addVariantMachine` insère une variante déjà construite juste après sa source ;

`changeDeviceMachine` donne une autre machine du même équipement à un équipement placé (EQT-04).

Toutes suivent le même schéma : (1) validation des paramètres par `paramsProblems` avec les types de connecteurs du projet ; (2) construction du projet suivant, en rajoutant les types de connecteurs *par défaut* manquants (`withConnectorTypes` : un type supprimé par l'utilisateur revient avec la machine qui l'utilise ; un type du projet manquant n'est jamais recréé, les paramètres sont refusés) ; (3) contrôle de placement de chaque équipement placé de la machine (`checkDevicePlacement` : chevauchement, sortie de l'armoire, profondeur ; un équipement déjà forcé reste accepté) ; (4) relevé des liaisons dont le port a disparu. Ces liaisons **restent** dans le projet : elles deviennent des liaisons pendantes signalées par les vérifications (LIA-07) ; l'interface les liste avant d'appliquer. Les ports ajoutés à l'équipement placé lui-même (PRT-04) ne dépendent pas de la machine et ne sont jamais perdus.

6.2 La bibliothèque par défaut

`createDefaultLibrary` assemble la bibliothèque copiée dans chaque nouveau projet (PRJ-01, § 7.2) :

- 35 **types de connecteurs** (`createDefaultConnectorTypes`), d'identifiants fixes `builtinId(0x100 + rang)` ; leur nom est une désignation technique stockée comme donnée (« IEC C13 »), pas un texte d'interface ; les connecteurs d'alimentation enfichables ont un type complémentaire (C13 et C14, C19 et C20, C5 et C6) ;
- 7 **types de liaisons** (`createDefaultLinkTypes`), d'identifiants `builtinId(0x200 + rang)`, nommés depuis `fr.json`, avec couleur, préfixe de repère et connecteurs admis ; Ethernet est le type par défaut (`DEFAULT_LINK_TYPE_ID`, LIA-03) ;
- 10 **modèles d'armoires** (`createDefaultRackModels`) : 12, 18, 24, 42 et 47U, fixes ou roulantes, d'identifiants `builtinId(0x400 + n)` ;
- 19 **machines génériques**, une par équipement (`createDefaultMachines`).

Les identifiants fixes (décision du lot 01, 2026-11-04) font qu'un même type par défaut a le même identifiant dans tous les projets et toutes les bibliothèques : l'import d'une bibliothèque les rapproche sans ambiguïté (BIB-10). **Règle de maintenance** : un nouveau connecteur s'ajoute *à la fin* de la liste `CONNECTORS` (comme « F (coaxial) » au lot 10), sinon tous les identifiants suivants changeraient et les fichiers existants deviendraient incohérents. Il en va de même pour les types de liaisons et les modèles d'armoires.

La bibliothèque par défaut est neutre (CG-08) : aucun nom de marque, ni dans les données ni dans `fr.json`. `createStarterProject` ajoute au nouveau projet une première page A4 paysage ; c'est le projet chargé au démarrage et par « Nouveau projet ».

6.3 La page des bibliothèques de l'équipe

Les bibliothèques de marques ne sont pas dans l'application (CG-08) : leurs sources sont dans le dossier `libraries/` du dépôt, hors du contrôle de neutralité. Le script `tools/publish-libraries.ts` les publie *chiffrées* sur le site, avec un catalogue chiffré de même (WEB-03 à WEB-05, décision du lot 33, 2026-11-05).

6.3.1 Chiffrement

`libraries/crypto` utilise uniquement WebCrypto, si bien que le même code tourne dans le script de publication (Node) et dans la page (navigateur) :

- clé AES-256 dérivée du mot de passe de l'équipe par PBKDF2-SHA-256 ; nombre d'itérations : `PBKDF2_ITERATIONS` (au moins 600 000, WEB-03) ;
- sel de 16 octets et vecteur d'initialisation de 12 octets tirés au hasard pour chaque fichier ;
- format : `PLB1` (4 octets) | itérations (entier 32 bits gros-boutiste) | sel | IV | chiffré suivi de l'étiquette d'authentification GCM.

Le nombre d'itérations est écrit dans le fichier : on peut l'augmenter sans casser les fichiers déjà publiés. L'étiquette GCM rend indiscernables un mauvais mot de passe et un fichier altéré : les deux lèvent `WrongPasswordError`. Le mot de passe n'est **jamais** écrit dans le dépôt (WEB-04, REC-15) : le script le lit dans la variable d'environnement `LIBRARIES_PASSWORD`, alimentée par un secret de l'intégration continue, et refuse un mot de passe trop court. Sans secret, la publication continue sans bibliothèque.

6.3.2 Catalogue et page

Le catalogue (`CatalogueSchema`, fichier `CATALOGUE_PATH`) décrit chaque bibliothèque publiée : marque, version, date, auteur, licence, poids, chemin du fichier chiffré et liste des modèles. La page `site/bibliotheques/index.html` charge `libraries/page`, écrit en TypeScript sans React et construit par une configuration Vite à part (`vite.libraries.config.ts`, CSP propre) :

1. formulaire de mot de passe ; à l'envoi, lecture de `catalogue.bin`, déchiffrement et validation par Zod ;
2. une fiche par bibliothèque ; « Voir les modèles » télécharge et déchiffre le `.rkdglib`, puis lit `project.json` et les images de l'archive pour montrer chaque modèle avec sa face avant (ou sa couleur) ;
3. « Télécharger » donne le `.rkdglib` déchiffré, identique à l'original, à importer dans Portolan (BIB-10).

Le DOM est construit par `textContent` seulement (aucune injection HTML) ; le mot de passe reste en mémoire le temps de la page et n'est envoyé nulle part.

6.4 Démarrage de l'application

6.4.1 `main.tsx`, pas à pas

`main` est le point d'entrée du bundle. À l'import : initialisation de i18n, de la police Inter embarquée (graisses 400 à 700, NF-07b, aucun chargement externe) et des styles globaux ; titre du document ; adresse du module WebAssembly de libavoid. Puis `start()` :

1. enregistre les polices de symboles et *attend* les polices (`waitForFonts`) : le texte n'est mesuré qu'avec la police embarquée ;

2. crée le mesureur de texte (canvas), l'éditeur sur `createStarterProject`, l'état de l'interface (écran d'accueil affiché) ;
3. ouvre le stockage local (IndexedDB) ; en cas d'échec (navigation privée), pas de brouillon de secours ; relit la préférence `imageLimitMb` et s'abonne à ses changements (EDM-04) ;
4. crée la session (`createSession`), avec une confirmation qui passe par la fenêtre de l'interface ;
5. en développement seulement, expose `window.__portolan` (éditeur, session, interface, actions) pour les tests de bout en bout ;
6. monte React avec ses contextes (i18n, éditeur, interface, session, images, mesure du texte) ;
7. démarre la session (recherche d'un brouillon), marque `started` pour les tests une fois la fenêtre affichée, puis lance le mode hors ligne (`startOffline`).

6.4.2 Session : fichier courant et brouillon

`app/session` (décision du lot 06) gère le fichier courant, l'ouverture, l'enregistrement et le brouillon de secours ; l'interface s'abonne à `session.store`. Points importants :

- au démarrage, un brouillon *modifié* est proposé à la restauration (PRJ-05) ; l'enregistreur automatique ne démarre qu'après la décision de l'utilisateur, pour ne pas écraser le brouillon proposé ;
- l'archive est entièrement construite avant toute écriture (NF-04) ;
- un fichier `.rkdglib` ouvert passe en mode édition de bibliothèque (BIB-13, lot 44) : le premier enregistrement demande toujours un nom, pour ne pas écraser la version précédente ;
- toute action qui remplace le document demande confirmation s'il est modifié.

6.4.3 Hors ligne et mises à jour

`app/pwa` (lot 31) enregistre le service worker généré au build (aucun en développement). Une nouvelle version est *proposée*, jamais imposée (WEB-08) : `offerUpdate` place dans l'état de l'interface une fonction de rechargement qui écrit d'abord le brouillon, lève `updateReload` (pour suspendre l'avertissement de fermeture, PRJ-04) puis active la nouvelle version. Au redémarrage, le brouillon est proposé : aucun travail n'est perdu.

6.4.4 Marqueur du site

L'application doit savoir si elle est servie par le site public (où existent l'archive et la page des bibliothèques) pour activer les commandes `help.archive` et `library.teamLibraries` (lot 39b). `tools/build-site.ts` ajoute à la seule copie publiée de `index.html` une balise `<meta name="portolan-site" content="portolan-<version>.zip">` (`withSiteMarker`) ; l'application la lit dans sa propre page (`siteLinks`), sans requête, donc de façon identique hors ligne. Le contenu n'est accepté que sous la forme d'un nom d'archive (pas d'adresse arbitraire). Hors du site (build simple, archive, serveur de test), les deux commandes sont désactivées avec leur raison.

6.5 Internationalisation et textes

`i18n/index` crée une instance i18next indépendante de React (décision du lot 00) : le modèle et les règles métier produisent eux aussi des messages en français avec `t`, et React reçoit l'instance par `I18nextProvider`. Règles :

- **aucun texte en dur** (CG-05) : tout texte visible est une clé de `src/i18n/fr.json` ; la règle ESLint `i18next/no-literal-string` le vérifie dans les composants ;
- les valeurs sont formatées selon la langue de l'interface : `formatNumber`, `formatDate`, `formatBytes` ;
- l'interpolation n'échappe pas le HTML (`escapeValue: false`) : React et `textContent` s'en chargent ; ne jamais injecter le résultat de `t()` comme HTML.

Pour ajouter un texte : créer la clé dans `fr.json` au bon endroit de l'arborescence (`equipment.kinds`, `session`…), puis l'appeler par `t()`. Les désignations techniques (noms de connecteurs, « VGA », « USB », préfixes de ports) sont des données, pas des textes d'interface.

Notes de version. `src/help/notes-de-version.md` est importé tel quel dans le bundle (donc disponible hors ligne) et lu par `help/markdown`, un lecteur Markdown minimal : titres (niveaux 1 à 4), paragraphes, listes à puces, `` `code` `` et `**gras**` ; toute autre syntaxe reste du texte littéral. Il produit un arbre de données (`Block`) que la fenêtre d'aide rend en éléments React, sans `innerHTML`. C'est un contenu documentaire, comme le manuel : il n'est pas dans `fr.json`.

6.6 Comment faire

6.6.1 Ajouter un équipement

1. `src/model/schema.ts` : ajouter la clé *à la fin* de `EQUIPMENT_KINDS` (le rang fixe l'identifiant de la machine générique et de ses ports : insérer au milieu changerait ceux des équipements suivants) et sa catégorie dans `KIND_CATEGORY` ; si c'est un extérieur, il lui faut une taille affichée. Lancer `npm run schema`.
2. `params.ts` : écrire le schéma (chaque champ avec une valeur par défaut) et l'ajouter à `PARAM_SCHEMAS` ; s'il a des groupes de ports libres, déclarer le paramètre dans `PORT_GROUP_PARAMS`.
3. `generators.ts` : ajouter le générateur dans `GENERATORS`, sa profondeur dans `DEFAULT_DEPTHS`, sa couleur dans `EQUIPMENT_COLORS`, un cas dans `proposeMachineName` si le nom par défaut ne suffit pas. Noms de ports déterministes et uniques par face.
4. `fr.json` : `equipment.kinds`, `equipment.genericNames`, les noms proposés (`equipment.names`) et les libellés des champs du formulaire.
5. Interface : description du formulaire dans `src/ui/machine/paramFields.ts` ; dessin éventuel (décorations des équipements de baie, `src/render/`, ou `ExternalDrawing.tsx` et `logo.ts` pour un extérieur).
6. Tests : `src/library/equipment/equipment.test.ts` (génération, noms, réutilisation des identifiants), `src/library/defaultLibrary.test.ts` (nombre de machines génériques) ; captures de référence touchées (bibliothèque par défaut) à régénérer et signaler ; manuel (paramètres de l'équipement).

6.6.2 Ajouter un paramètre à un équipement

Ajouter le champ au schéma *avec une valeur par défaut* : les machines déjà enregistrées, dont les paramètres n'ont pas ce champ, restent valides et prennent la valeur par défaut à la prochaine génération. Si la valeur par défaut changeait le dessin des machines existantes, le signaler (captures de référence). Utiliser le paramètre dans le générateur sans renommer de port existant (sinon les liaisons des projets existants seraient signalées comme perdues). Ajouter le champ au formulaire (`paramFields.ts`), son libellé à `fr.json`, un test de génération et la ligne du manuel. Un connecteur choisi par l'utilisateur dans une liste ouverte doit passer par `PORT_GROUP_PARAMS` pour être contrôlé, exporté et rapproché à l'import.

6.6.3 Ajouter un type de connecteur ou de liaison par défaut

Ajouter l'entrée à la *fin* de `CONNECTORS` ou `LINK_TYPES` (`defaultTypes.ts`) ; pour un connecteur, son dessin dans `src/render/portShapes.ts` (`CONNECTOR_SHAPES`) ; pour un type de liaison, son nom dans `defaults.linkTypes` de `fr.json`. Vérifier `npm run check:neutral` (désignations normalisées seulement).

6.7 Limites connues

- Les ports retouchés dans l'éditeur de machine sont recréés d'après les paramètres à chaque régénération : seuls la position d'un port placé, le sens de sortie et « multiple » sont conservés (`regenerateEdited` se souvient en plus des ports perdus pendant une même édition).
- La profondeur par défaut ne dépend que de l'équipement, pas de ses paramètres (une multiprise verticale reçoit 100 mm comme une horizontale).
- Le lecteur Markdown ne connaît ni liens, ni italique, ni listes numérotées, ni blocs de code : la syntaxe correspondante apparaît telle quelle.

Chapitre 7

Routage des câbles

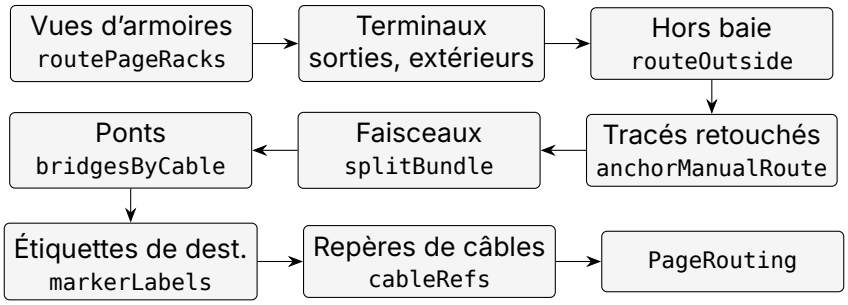
Le routage transforme les liaisons du projet (deux ports reliés) en traits dessinés sur une page : dans chaque vue d'armoire, le câble quitte son port, rejoint une goulotte, la parcourt puis rejoint l'autre port ou sort de l'armoire ; hors des armoires, il est tracé par libavoid jusqu'à l'autre armoire ou jusqu'à un équipement extérieur ; quand l'autre extrémité n'est pas sur la page, il se termine par un renvoi. Viennent ensuite les tracés retouchés à la main, les faisceaux, les ponts, le mode repères et les repères de câbles.

Tout le code est dans `src/routing/`. Il est **pur** (pas de React, pas de DOM, pas d'état global hormis des caches par identité) et **déterministe** (ROU-05) : à données identiques, le tracé est identique, ponts compris. La seule dépendance extérieure est libavoid-js (WebAssembly), passée en paramètre.

7.1 Vue d'ensemble

7.1.1 Le flux d'une page

Le point d'entrée est `routePage` (ou sa version mémorisée `routePageCached`). Il produit un `PageRouting` que consomment le rendu (`LinksLayer`, `src/render/LinksLayer.tsx`), la mise en évidence des chemins (`src/ui/canvas/Highlights.tsx`), les poignées de retouche (`src/ui/canvas/RouteHandles.tsx`), les commandes de faisceaux (`src/commands/route.ts`) et l'export SVG (donc PNG et PDF, qui en dérivent).



7.1.2 Les modules

Module	Rôle
`routing/trays`	routage dans une vue d'armoire : goulottes, pistes, sorties (ROU-01 à ROU-03, ROU-05, ROU-14, ROU-15, PAG-16)
`routing/lanes`	géométrie des couloirs libres de ports et ordre des pistes
`routing/external`	sortie d'un port d'équipement extérieur, sous l'équipement (EXT-03, ROU-14)
`routing/outside`	tracé hors baie par libavoid (ROU-04)
`routing/avoid`	chargement à la demande de libavoid-js
`routing/page`	orchestration d'une page, renvois (ROU-08)
`routing/hidden`	extrémités masquées par une vue (PAG-15)
`routing/markers`	mode repères (ROU-09)
`routing/manual`	retouche d'une polyligne (ROU-06)
`routing/attach`	points de passage rattachés aux vues (ROU-16)
`routing/bundles`	faisceaux (ROU-11) et places des textes (ROU-10)
`routing/bridges`	croisements et ponts (ROU-12, ROU-15)
`routing/straight`	ancrage d'un port (tracé provisoire du lot 21)

7.1.3 Unités et repères

Deux repères coexistent :

- le **millimètre d'armoire** (réel, échelle 1), origine au coin haut gauche du dessin de l'armoire (`RackGeometry` de `src/model/rackGeometry.ts`) : tout le calcul dans une vue d'armoire (`routing/trays`, `routing/lanes`) s'y fait ;
- le **millimètre de page** (papier) : toutes les sorties (`Leg`, `CableDrawing`…) et tout ce qui est hors des armoires. Le passage se fait par x_{page} `= view.x + x ×` e, y_{page} `= view.y + (y − window.top) ×` e, où e est l'échelle effective de la vue (`effectiveScale`) et `window` la fenêtre de la plage de U montrée (`rackWindow`, PAG-16).

Certaines constantes sont donc « réelles » (8 mm d'écartement maximal des pistes) et d'autres « sur le papier » (0,9 mm d'écartement minimal) : le nom de la constante le dit (`_MM` ou `_PAPER`, voir `MAX_TRACK_SPACING_MM` et `MIN_TRACK_SPACING_PAPER`).

7.2 Dans la baie : goulottes, couloirs et pistes

`routeRackView` trace les câbles d'une vue d'armoire. Chaque vue a deux goulottes verticales, à gauche et à droite, entre les montants et le bord de l'armoire (ROU-01). Le résultat, `RackViewRouting`, contient un `Leg` (« tronçon ») par câble et par vue, et l'occupation de chaque goulotte (`TrayUsage`).

7.2.1 Choix de la goulotte (ROU-02)

`traySide` : le côté forcé sur la liaison (`link.tray`), sinon le réglage de l'armoire (`rack.trayByLinkType`), sinon celui du projet (`trayByLinkType` de `project.routing`), sinon `defaultTraySide` : électrique à gauche, tout le reste à droite (point ouvert n° 2 du cahier des charges, décision du lot 22). Gauche et droite s'entendent *vues depuis la face affichée*, comme les emplacements 0U.

7.2.2 Étape 1 : ports et câbles de la vue

1. Une vue extérieure, une vue dont l'armoire n'existe plus ou une vue aux câbles masqués (PAG-15) rend un résultat vide.
2. Les ports de la face montrée de chaque équipement de l'armoire sont recensés (`visibleAnchors`) avec leur rectangle, celui de leur équipement et, pour un équipement 0U, le côté de son emplacement.
3. Les ports dont le centre est hors de la bande montrée par la plage de U sont *coupés* (PAG-16) : ils ne sont pas dessinés.
4. Chaque liaison qui a au moins une extrémité visible devient un câble à tracer, sauf si elle est dessinée en étiquettes de destination (ROU-09 : forcée, type en mode repères ou vue en mode repères) : elle ne prend alors aucune piste. Si l'unique extrémité visible a son autre extrémité sur un port *coupé* de la même face, le câble ira jusqu'à la coupure (haut ou bas).

7.2.3 Étape 2 : couloirs (ROU-14)

Un **couloir** (`Band`) est une bande horizontale qu'aucun port ne touche : entre deux rangées de ports, entre deux équipements, au-dessus du premier. `freeBands` les calcule comme le complément de l'union des hauteurs des ports, chacune élargie de 0,6 mm réels, dans la bande montrée (jusqu'au socle au plus). Trois jeux de couloirs sont calculés : celui des équipements montés en U, et un par côté pour les ports des équipements 0U.

Le choix est volontairement prudent (lot 38) : une bande libre sur *toute* la largeur sert à tous les câbles, y compris ceux d'un équipement demi-largeur voisin. C'est ce qui garantit l'invariant central : **un câble ne passe sur aucun port autre que ses extrémités**.

7.2.4 Étape 3 : sortie de chaque port

Pour chaque extrémité visible (`End` dans le code), dans cet ordre :

1. **Équipement 0U du côté de sa goulotte** : ligne droite jusqu'au bord extérieur de la goulotte si rien n'est en travers (`horizontalClear`) ; sinon sortie par un couloir des 0U.
2. **Équipement 0U de l'autre côté** : il est traité plus loin (« colonne de numérotation », ci-dessous).
3. **Équipement en U, chemin direct libre** : si la direction de sortie du port n'impose rien de contraire et qu'aucun port n'est entre lui et la goulotte, ligne droite à la hauteur du port.
4. **Sinon, sortie par un couloir** (`leaveRow`) :
   (a) côté préféré (`prefersUp`) : la direction de sortie du port si elle est « haut » ou « bas » ; sinon la moitié de l'équipement où se trouve le port (haute → vers le haut) ; pour la rangée du milieu, vers l'autre extrémité du câble (vers le bas pour une sortie d'armoire) ; à hauteur égale, vers le haut pour la goulotte de droite, vers le bas pour celle de gauche. Cette dernière règle vient d'une mesure : « droite → haut » seule faisait passer les croisements du schéma de référence de 5 à 17 ;
   (b) `chooseBand` : le couloir juste au-dessus ou juste au-dessous, s'il est atteint verticalement sans traverser de port (`verticalClear`), le côté préféré d'abord ;

(c) à défaut (rangées de ports collées, sans couloir entre elles), `sidestep` : un pas de côté dans la rangée jusqu'à la colonne libre la plus proche entre deux ports (vers la goulotte d'abord), puis vertical jusqu'au couloir au-dessus ou au-dessous des rangées ;

(d) à défaut encore (port *emmuré*, touché de tous côtés) : ligne droite à la hauteur du port, seul cas où un câble passe sur un port, faute de chemin.

5. **Sortie opposée à la goulotte** (port de direction « gauche » ou « droite » contraire) : demi-tour à 3 mm réels au-delà du bord de l'équipement, sur deux voies de couloir.

Colonne de numérotation (0U vers l'autre goulotte). Un câble d'une PDU verticale vers la goulotte d'en face sort de son emplacement jusqu'à la colonne de numérotation de son côté (quatre abscisses alternées, `(i mod 4 + 1) / 5` de sa largeur), en ligne droite si rien n'est en travers, sinon par un couloir des 0U ; il rejoint ensuite le couloir des équipements le plus proche et traverse l'armoire. Les voies des couloirs 0U sont attribuées *avant* celles des couloirs des équipements, car elles fixent la hauteur à laquelle le câble quitte son emplacement.

7.2.5 Étape 4 : voies dans les couloirs et biais à 45°

Dans un couloir, chaque câble a sa propre voie horizontale (`assignLanes`) :

- les câbles venus des ports du dessus prennent les voies hautes, ceux du dessous les voies basses ;
- parmi eux, le câble le plus proche de la goulotte prend la voie la plus proche des ports ; départage par demi-tour, distance, numéro de port, repère, identifiant ;
- écartement : $\min(\max(4 \text{ mm réels}, 0{,}6 \text{ mm papier}/e),\ h/(n+1))$, où h est la hauteur du couloir et n le nombre de voies (deux pour un demi-tour).

Du bord du couloir à sa voie, le câble fait un **biais à 45°** (`tuckIn`) qui le range à côté du câble du port voisin sans le croiser : les biais de deux ports voisins sont parallèles. Le biais ne dépasse jamais la moitié du chemin jusqu'à la goulotte. C'est le *seul* segment oblique d'un tracé automatique ; ROU-14 prime ici sur la phrase « tous les segments sont orthogonaux » de ROU-01. `isBevel` reconnaît ces biais (tolérance 0,02 mm, les tracés stockés étant arrondis au centième).

7.2.6 Étape 5 : ordre des pistes (ROU-03, ROU-15)

Chaque câble (ou chaque faisceau, traité comme un seul câble) occupe sa propre piste verticale dans la goulotte. L'ordre, de la plus proche des équipements vers l'extérieur, est calculé par `orderTracks` sur les hauteurs réelles d'entrée dans la goulotte (`TrayCable`) :

1. ordre initial : du plus court au plus long (une sortie d'armoire compte pour infiniment longue) ; un câble compris dans un autre passe dedans, ce qui est optimal quand toutes les entrées viennent des équipements (vérifié par force brute dans `lanes.test.ts`) ; à longueur égale, le sommet le plus bas d'abord, puis numéro de port, repère et identifiant (ROU-05) ;

2. échanges de pistes voisines tant qu'ils retirent des croisements, le coût d'un ordre de deux câbles étant donné par `pairCrossings` : les entrées du câble extérieur venues des équipements qui tombent dans la hauteur du câble intérieur, plus les entrées du câble intérieur venues des 0U qui tombent dans la hauteur de l'extérieur.

Parmi les câbles qui descendent (sorties et coupures en bas), ceux d'une coupure prennent les pistes intérieures, pour que les sorties latérales ne les croisent pas.

Écartement. max(*min*, min(8 mm réels, $L/(n+1)$)) où L est la largeur de la goulotte, n le nombre de pistes et *min* $= 0{,}9$ mm papier$/e$. Au-delà de la largeur, les pistes **débordent** hors de la goulotte (`widened`, choix « élargissement » du lot 22) ; sauf si des équipements 0U à ports sont de ce côté : l'écartement minimal tombe alors à zéro et les pistes se resserrent plutôt que de passer sur leurs prises (lot 38).

7.2.7 Étape 6 : fin du tronçon

- **Deux extrémités dans la vue** : port → voie → piste → voie → port ; le tronçon est `complete`.
- **Autre extrémité ailleurs** : le câble descend sa piste jusqu'au socle et **sort par le côté de l'armoire** (décision du lot 23 : les équipements extérieurs sont sous l'armoire sur le schéma de référence) ; une hauteur par câble, la piste la plus intérieure sortant le plus bas, si bien que les sorties ne se croisent pas. Avec une coupure en bas (PAG-16), la sortie traverse le cadre entre la limite et le zigzag. Le tronçon porte `exit` et `from`.
- **Autre extrémité coupée par la plage de U** (PAG-16) : le câble suit sa piste à travers la coupure et s'arrête au-delà, à `CUT_RENVOI_STUB` mm de papier, étagé de `CUT_RENVOI_STEP` mm (la piste la plus extérieure s'arrête au plus près de la coupure) ; le tronçon porte `cutTo` et `renvoiAlign` (texte écrit vers l'extérieur de l'armoire) et compte comme complet.

La polyligne est passée en millimètres de page puis simplifiée (`simplify` : points répétés retirés, segments alignés fusionnés, y compris les biais, sans fusionner un aller-retour).

7.2.8 Cache

Le routage d'une vue est mis en cache par identifiant de vue (500 vues au plus) et recalculé seulement si l'une de ses entrées change *d'identité* : armoire, vue, échelle effective, réglages de routage du projet, types de liaison de la bibliothèque (leur case « repères » retire les câbles des goulottes, ROU-09), tracés retouchés de la page et état des repères des vues (seulement si la vue a ses faisceaux), équipements de l'armoire, liaisons qui les touchent et leurs modèles. Toute donnée lue par le calcul d'une vue doit figurer dans cette liste : sinon, une modification n'est pas vue tant que rien d'autre ne change (les types de liaison en manquaient jusqu'à la relecture de la phase H). Comme l'état est immuable (Immer garde les objets inchangés), la comparaison par identité suffit. Conséquence pour les tests : ne jamais muter un projet en place entre deux appels (un test du lot 38 en a fait les frais).

7.3 Hors de la baie

7.3.1 Terminaux

Pour un câble sans tronçon complet, `terminals` (dans `routing/page`) rassemble ses *terminaux* sur la page :

- la sortie de chacun de ses tronçons, prolongée de 2,5 mm (`SHAPE_BUFFER_MM` + 1) hors de l'armoire du côté de sa goulotte, pour que libavoid puisse en partir ;
- chacun de ses ports dans une vue d'**équipement extérieur** (sauf vue aux câbles masqués) : le câble descend sous l'équipement et son nom (EXT-03) par `dropUnderDevice`.

`dropUnderDevice` trace un chemin orthogonal du centre du port jusqu'au-dessous de l'équipement sans traverser un autre port de la face : tout droit si la colonne est libre ; sinon il essaie les rangées et colonnes qui longent les autres ports (marges 0,5 puis 0,05 mm) et garde le plus court, un coude comptant pour 2 mm ; à égalité, le premier candidat ; sans chemin libre, tout droit. Ports masqués (`portsHidden`) : le câble part du bas de l'équipement.

7.3.2 libavoid

Avec deux terminaux, le câble est confié à `routeOutside` :

1. un routeur orthogonal est créé ; marge aux obstacles `SHAPE_BUFFER_MM` (1,5 mm), écartement des câbles parallèles `NUDGING_MM` (1 mm), pénalité de segment 10, écartement des chemins partagés à extrémité commune ;
2. les obstacles sont les rectangles des vues de la page (le nom sous un équipement extérieur compris, 6 mm) et le cartouche ;
3. toutes les connexions sont ajoutées, puis une seule transaction est traitée ;
4. chaque tracé est lu, arrondi au millième, ses extrémités remplacées par celles demandées (libavoid les déplace en écartant les câbles), puis `orthogonalize` ajoute un coin où deux points ne sont pas alignés ;
5. le routeur est détruit (`router.delete()`) dans tous les cas.

Le polyligne complet du câble (`CableDrawing.polyline`) enchaîne alors le chemin du premier terminal, le tracé libavoid et le chemin du second terminal à l'envers : c'est lui que l'on retouche.

7.3.3 Chargement et politique de sécurité

libavoid-js 0.5.0-beta.5 (LGPL-2.1-or-later), version épinglée, est chargée à la demande par `loadAvoid` : import dynamique (module à part dans le build) et fichier `libavoid.wasm` copié dans le build, dont l'URL est donnée au démarrage par `setAvoidWasmUrl` (`src/main.tsx`, alias Vite `libavoid-wasm?url`). Aucun chargement extérieur (CG-02) ; les fichiers sont mis en cache pour le hors-ligne. Tant que le module n'est pas chargé, les tracés hors baie sont des **lignes droites** : `useAvoid` (`src/render/LinksLayer.tsx`) redessine la page quand il arrive ; les exports PNG et PDF attendent `loadAvoid`, l'export SVG prend `loadedAvoid`.

Pas d'`eval`. La colle JavaScript d'embind générait des fonctions avec `new Function`, interdit par la politique de sécurité. Après un passage par `'unsafe-eval'` (phase E, abandonné), la décision de la phase F (2026-11-05) est de transformer cette colle au build : le greffon Vite `tools/libavoid-no-eval.ts` remplace ses deux générateurs par des fermetures équivalentes (comme Emscripten avec `DYNAMIC_EXECUTION=0`). Chaque motif doit être trouvé exactement une fois, sinon le build échoue ; la transformation s'applique au build, au serveur de développement et aux tests unitaires. La CSP du build est `script-src 'self' 'wasm-unsafe-eval'` (`vite.config.ts`). Monter de version libavoid-js demande donc de revoir ce greffon et son test (`tools/libavoid-no-eval.test.ts`).

7.4 Renvois (ROU-08)

Un câble qui n'a qu'un terminal sur la page (l'autre extrémité est sur une autre page ou une face non affichée) se termine par un **renvoi** (`Renvoi`) : une amorce de 4 mm sous la sortie, puis une étiquette ; les renvois d'une même sortie (même vue et même côté, ou même vue extérieure) sont étagés de `RENVOI_STEP` (3,2 mm). Le texte est donné par `renvoiText` : « → équipement · port, page n » (première autre page qui montre l'équipement) ou « (non dessiné) ».

Deux variantes utilisent le texte `routing.marker` (« → équipement · port », sans page, l'extrémité étant sur la même page) : le renvoi au bout d'un câble vers un port coupé par la plage de U (PAG-16, tronçon `cutTo`) et le renvoi vers une extrémité seulement dessinée dans une vue aux câbles masqués (PAG-15).

7.5 Câbles masqués (PAG-15)

Une vue peut masquer ses câbles *dans cette vue seulement* (décision de l'utilisateur, lot 47). Mise en œuvre :

- `routeRackView` rend une vue masquée vide : aucun tronçon, aucune piste ;
- `hiddenEnds` donne les extrémités dessinées seulement dans des vues masquées ; `routePage` y termine le câble par un renvoi `routing.marker` au lieu de `renvoiText` ;
- aucun terminal n'est pris dans une vue extérieure masquée ;
- le mode repères d'une vue masquée ne compte pas (`usesMarkers`) et `markerLabels` n'y place aucune étiquette.

Un câble dont aucune extrémité n'est dessinée dans une vue qui montre ses câbles n'est dessiné nulle part. Les ports d'une vue masquée restent dessinés.

7.6 Mode repères (ROU-09)

Dans une baie dense, un câble peut être dessiné comme une **étiquette de destination** à chacun de ses ports (« → SRV01 · eth0 ») au lieu d'un trait. La liaison est inchangée ; seul le dessin change. `usesMarkers` décide, par page : `link.display` forcé (`line` ou `markers`), sinon le type de liaison en mode repères, sinon l'une des extrémités dans une vue en mode repères (toute la liaison passe alors en étiquettes sur la page, pour ne pas laisser un câble sans fin).

`markerLabels` place une étiquette par port visible et par vue : écrite verticalement au-dessus de l'équipement (port de la moitié haute, ou rangée du milieu à numéro pair) ou au-dessous, à 0,4 mm du bord ; taille $0{,}9\times$ la largeur du port, bornée entre 0,8 et 2,2 mm ; texte avec le repère du câble si la vue le demande (`markerRefs`). Ces câbles ne prennent aucune piste dans les goulottes et n'entrent dans aucun faisceau.

7.7 Retouche manuelle et points rattachés (ROU-06, ROU-07, ROU-16)

7.7.1 Ce qui est stocké

Un tracé retouché est stocké dans `page.manualRoutes` : `{ linkId, points }`, chaque point étant un `AttachedPoint` `{ viewId, dx, dy }` : des millimètres *du dessin de la vue* (échelle 1) depuis son coin haut gauche (lot 37, format 2). Il remplace le tracé automatique de ce câble *sur cette page*.

7.7.2 Enregistrer une retouche

L'interface (`src/ui/canvas/Canvas.tsx`) modifie la polyligne en millimètres de page avec les fonctions pures de `routing/manual` :

- `moveSegment` : déplace un segment perpendiculairement ; les extrémités restent en place (un coin est ajouté si le premier ou le dernier segment bouge) ;
- `moveVertex` : déplace un coude, ses voisins intérieurs suivent pour garder leurs directions ;
- `insertPoint` (double-clic) : point posé à la projection du pointeur sur le segment le plus proche (`nearestSegment`), exactement sur un biais s'il s'agit d'un biais ;
- `removePoint` (clic droit) : retire un point intérieur.

Toutes rendent une polyligne orthogonale : `squareUp` transforme chaque biais à 45° du tracé automatique en coude droit au même endroit, dans le prolongement du segment précédent (donc dans le couloir). Les points alignés ajoutés par l'utilisateur sont conservés (`dedupe` ne retire que les doublons) : ce sont des coudes que l'on peut tirer.

`setManualRoute` (`src/store/routeActions.ts`) rattache ensuite chaque point (`attachPoints`) : à la vue qui le contient, sinon à la plus proche (`nearestView`, distance au rectangle de la vue, `distanceToBox`) ; à égalité, la première vue de la page. La position est arrondie au centième avant le calcul de l'écart. Un glisser est une seule action annulable (clé de regroupement). `resetRoutes` (ROU-07) retire les tracés.

7.7.3 Relire un tracé

À chaque routage, `routePage` :

1. résout les points (`resolvePoints`) à la position et à l'échelle *actuelles* de leur vue : `x = view.x + dx × e` ; un point dont la vue n'existe plus (fichier abîmé) est lu depuis le coin de la page ;

2. rétablit l'orthogonalité (`keepOrthogonal`) : un segment devenu oblique, parce que ses deux points sont rattachés à des vues déplacées différemment, reçoit **un** coude, calculé à chaque affichage et jamais stocké. Des deux coudes possibles, `(q.x, p.y)` (partir horizontalement) ou `(p.x, q.y)` (verticalement), on garde celui qui fait le moins de changements de direction avec les segments voisins, un demi-tour comptant pour trois ; à égalité, horizontalement. Le tronçon « s'étire » en prolongeant un voisin ;

3. rattache les extrémités aux ports actuels (`anchorManualRoute`) : la chaîne de segments alignés qui part de chaque extrémité suit le port, si bien qu'un équipement déplacé ne détache pas le câble ;

4. remplace les chemins du câble par ce tracé et déplace son renvoi éventuel au nouveau bout.

`manualPolyline` enchaîne les deux premières étapes.

7.7.4 Quand les vues changent

Vue déplacée rien à faire : ses points suivent par construction ; deux vues déplacées ensemble déplacent le tracé tel quel.

Échelle changée (celle de la vue, ou l'échelle commune des vues d'armoires) : les points gardent leur place sur le dessin de la vue, le tracé garde sa forme (ROU-16, cahier 1.7).

Vue supprimée (vue, armoire, équipement extérieur, bibliothèque) : `followRemovedViews`, appelé par `Editor.apply` dans la même action, utilise `reattachRemovedViews` : les points gardent leur place sur la page et se rattachent à la vue restante la plus proche ; sans vue restante, le tracé est retiré (retour à l'automatique).

Page dupliquée `remapRouteViews` rattache les tracés aux copies des vues.

Plage de U changée ou câbles masqués les tracés sont conservés tels quels (décision de l'utilisateur, lot 47) ; seules les extrémités suivent les ports.

7.8 Faisceaux et repères de câbles (ROU-10, ROU-11)

7.8.1 Qui forme un faisceau

Les faisceaux sont désactivés par défaut et s'activent par vue d'armoire (`View.bundles`, lot 41). Dans une vue, forment un faisceau les câbles (au moins deux) de même type de liaison, de même goulotte, entre les mêmes équipements (ceux des extrémités visibles et ceux des autres extrémités) : c'est la clé `bundleKey` de `routing/trays`. Restent seuls : un câble dégroupé (`Link.bundle` faux), un câble retouché sur la page, un câble en mode repères, un câble vers un port coupé par la plage de U.

7.8.2 Tracé

- Dans la goulotte, les câbles d'un faisceau partagent **une seule piste** (l'ordre des pistes traite le faisceau comme un câble dont les entrées sont celles de tous ses câbles) et la même sortie ; leurs voies de couloir restent individuelles (ROU-14 inchangé).

- Hors baie, `routePage` ne demande à libavoid qu'**un tracé par faisceau et par destination** ; vers un équipement extérieur, le tracé va sous le port médian, puis un « peigne » horizontal rejoint chaque port.
- Sur la page, les câbles qui partagent une piste dans *une* vue sont réunis (union des clés de leurs tronçons) : un câble sur une piste partagée est toujours compté et dessiné avec son faisceau.

7.8.3 Tronc et étiquette

`splitBundle` calcule le **tronc** géométriquement : les tronçons d'un câble qui longent un tronçon d'un autre câble du faisceau (même droite, recouvrement de plus de 0,01 mm) forment le tronc, fusionnés puis chaînés en polylignes ; le reste de chaque câble (de ses ports au tronc) forme ses *brins*, dessinés en trait fin. Le tronc est dessiné à `BUNDLE_WIDTH_FACTOR` fois l'épaisseur du type, avec l'étiquette « n × ETH » (`linkTypeShortName`).

Les étiquettes sont placées par `freeAlongPlace`, plus grand faisceau d'abord : sur les segments du tronc, les plus longs d'abord, au milieu puis de part et d'autre par pas d'un vingtième (0,5 mm au moins), au premier endroit qui ne recouvre aucune étiquette déjà placée. Une seule taille pour la page : la plus grande de `BUNDLE_FONT_SIZES` (2,4 / 1,8 / 1,4 / 1,1 mm) à laquelle toutes les étiquettes trouvent une place ; à la plus petite, une étiquette sans place va au milieu de son tronc. Les largeurs sont estimées (`textLength`, 0,62 em par caractère) : le routage ne mesure pas le texte.

7.8.4 Repères de câbles (ROU-10)

Propriété de page `cableRefs` : `none` (par défaut), `along` (le long du tracé : `freeAlongPlace` sur les segments propres du câble, en évitant les étiquettes de faisceaux et les repères déjà placés) ou `ends` (à 0,8 mm de chaque port, le long du premier segment, jamais à l'envers : `endPlace` ; pas de repère au bout d'un renvoi).

7.9 Croisements et ponts (ROU-12, ROU-15)

`findCrossings` cherche les croisements *propres* entre segments de câbles différents : les deux intérieurs se coupent en un point, à plus de 0,01 mm des extrémités (un câble qui en touche un autre, ou partage un tronçon avec lui, ne le croise pas). Les segments sont triés par abscisse minimale et balayés : seuls ceux dont les intervalles en x se recouvrent sont comparés. Le pont va sur le segment le plus horizontal. Le résultat est trié (câble, polyligne, segment, position) : déterministe.

`bridgesByCable` pose un arc de `BRIDGE_RADIUS_MM` (0,7 mm) de part et d'autre de chaque croisement ; les arcs qui se chevauchent fusionnent en un arc elliptique de même hauteur ; un arc ne dépasse jamais les extrémités de son segment. `cablePathData` écrit enfin le chemin SVG (commandes `A`, coordonnées au millième), arcs bombés vers le haut (vers la gauche sur un segment vertical). Le même chemin sert à l'écran et dans les exports ; svg2pdf.js convertit les arcs en courbes de Bézier.

Les ponts sont calculés *après* les tracés retouchés et les faisceaux : ils s'appliquent aux tracés retouchés, et un tronc compte comme un câble. `countCrossings` sert de mesure dans les tests (ROU-15).

7.10 Orchestration d'une page

`routePage`, pas à pas :

1. routage de chaque vue d'armoire (`routePageRacks`) ; tronçons regroupés par liaison ;
2. extrémités masquées (`hiddenEnds`) ;
3. pour chaque liaison dont le type n'est pas masqué : en mode repères, mise de côté ; sinon, dessin initial avec ses tronçons complets, renvois des coupures de plage ; sans tronçon complet, terminaux : deux terminaux → connexion hors baie en attente ; un seul → renvoi (si l'autre équipement existe) ;
4. connexions hors baie : une par câble, ou une par faisceau et destination ; tracé par libavoid, ou en ligne droite sans libavoid ; peigne des faisceaux vers un équipement extérieur ; polyligne complète du câble ;
5. tracés retouchés de la page (ROU-06, ROU-16) ;
6. faisceaux (`drawBundles`) : union des clés, tronc et brins, étiquettes ; un câble retouché quitte son faisceau ;
7. ponts sur tous les câbles et troncs ;
8. étiquettes de destination (`markerLabels`) et repères de câbles.

`routePageCached` garde le dernier résultat (une seule entrée) : le rendu, les poignées de retouche, la mise en évidence et les commandes de faisceaux partagent le même calcul tant que le projet, la page, libavoid et l'ensemble des types masqués sont les mêmes objets.

7.11 Invariants

- **Déterminisme** (ROU-05) : tout tri a un départage final par identifiant ou clé ; libavoid est déterministe à entrées identiques ; les tests comparent deux routages, ponts compris.
- **Jamais devant un port** (ROU-14) : aucun segment ne traverse l'intérieur d'un port étranger (toucher son bord est admis), sauf port emmuré ; vérifié par `src/testing/routingChecks.ts`.
- **Orthogonalité** : tous les segments sont horizontaux ou verticaux, sauf un biais à 45° en deuxième position depuis un port ; un tracé retouché est entièrement orthogonal.
- **Une piste par câble** (ou par faisceau) dans une goulotte ; deux voies horizontales ne se superposent pas dans un couloir.
- **Pureté** : aucune fonction ne modifie le projet ; les caches comparent des identités et supposent un état immuable.

7.12 Performances

Le routage des goulottes est quasi immédiat (200 câbles dans `trays.test.ts`) et mis en cache par vue. libavoid domine : environ 50 ms pour 40 câbles hors baie, 0,2 s pour 100 (mesures du lot 23). Le test de charge NF-02 (`outside.test.ts`) route une page de 300 câbles, dont 40 entre armoires, en moins d'une seconde. Pour garder ces temps : regrouper les connexions dans une seule transaction libavoid, ne pas recalculer une vue dont les entrées n'ont pas changé, et garder le balayage par abscisse de `findCrossings` (sinon quadratique sur des milliers de segments).

7.13 Tests et captures

Test	Ce qu'il couvre
`trays.test.ts`	goulottes et pistes, réglages ROU-02, sorties, ROU-14, déterminisme, 200 câbles, `simplify`
`lanes.test.ts`	couloirs, biais, coût des croisements, ordre optimal (force brute)
`clearance.test.ts`	aucun passage sur un port et nombre de croisements fixés sur le schéma de référence et les baies générées (`src/testing/generatedRack.ts`), rangées sans couloir, échelles, déterminisme
`viewRange.test.ts`	plage de U (PAG-16) et câbles masqués (PAG-15)
`outside.test.ts`	libavoid, renvois, charge NF-02
`external.test.ts`	descente sous un équipement extérieur
`attach.test.ts`, `manual.test.ts`	retouche et points rattachés
`bundles.test.ts`	faisceaux et repères
`bridges.test.ts`, `markers.test.ts`	ponts, mode repères
`src/export/pdfBridges.test.ts`	arcs des ponts dans le PDF
`tests/e2e/routing.spec.ts`	captures `baie-cablee.png` et `mode-reperes.png`, affectation des goulottes, renvoi puis tracé hors baie
`tests/e2e/retouch.spec.ts`	retouche à la souris, ROU-16, enregistrement
`tests/e2e/bundles.spec.ts`	capture `faisceaux.png`, dégroupage, repères
`tests/e2e/views.spec.ts`	capture `vue-plage-u.png`
`tests/e2e/externals.spec.ts`	capture `equipements-exterieurs.png`

7.14 Modifier une règle

Changer la goulotte par défaut d'un type `defaultTraySide` ; mettre à jour `trays.test.ts` et régénérer les captures qui montrent des câbles électriques ou réseau (`baie-cablee.png` au moins) et celles du manuel.

Changer l'écartement des pistes ou des voies constantes de `trays.ts` ; relancer `clearance.test.ts` (nombres de croisements fixés : au plus la valeur actuelle) et les captures.

Changer l'ordre des pistes `orderTracks` et `pairCrossings` ; `lanes.test.ts` vérifie l'optimalité par force brute, `clearance.test.ts` les nombres de croisements.

Changer le côté de sortie d'un port `prefersUp` dans `trays.ts` ; mesurer les croisements du schéma de référence avant et après (voir la règle « droite → haut » du lot 38).

Régler libavoid constantes et paramètres de `routeOutside` ; `outside.test.ts`, test de charge, captures hors baie.

Changer l'aspect des ponts `BRIDGE_RADIUS_MM` et `cablePathData` ; `bridges.test.ts`, `pdfBridges.test.ts`, captures.

Ajouter une entrée au routage d'une vue (nouveau champ de vue ou de liaison lu par `computeRackView`) : l'ajouter aussi à `dependencies` dans `trays.ts`, sinon le cache rend un résultat périmé.

Une capture de référence ne se régénère que pour un changement visuel voulu, signalé dans le commit (`npx playwright test --update-snapshots=all`, puis regarder l'image).

7.15 Limites connues

- Un couloir étroit emprunté par beaucoup de câbles serre ses voies sous l'épaisseur du trait (48 câbles dans 14,4 mm réels entre deux commutateurs de 48 ports) ; les faisceaux allègent les goulottes mais pas les couloirs ; le manuel renvoie au mode repères pour ces cas (lots 38 et 41).
- Hors baie, libavoid reste seul juge des croisements ; ceux qui restent viennent surtout de deux vues dont les ordres de sortie diffèrent.
- Pas de faisceau entre deux équipements extérieurs (un faisceau naît dans une vue d'armoire).
- ROU-13 (passe-câbles comme point de passage obligatoire, priorité C) n'est pas réalisé.
- Un point retouché dans une goulotte garde sa place sur la page quand la plage de U change : après une coupure en haut, il n'est plus en face des mêmes U.
- `straightLinks`, tracé provisoire du lot 21, n'est plus appelé que par ses tests ; seul `portAnchor` sert encore (tirer une liaison à la souris).

Chapitre 8

Rendu SVG

Ce chapitre décrit comment Portolan dessine une page : l'arbre des composants React de `src/render/`, les repères et les unités, la face d'un équipement (photo, dessin par défaut « réaliste sobre », dessins des extérieurs), les ports, la couche des câbles, celle des éléments de l'onglet « Schéma », le cartouche et la planche d'étiquettes. Il explique aussi ce que l'écran et les exports partagent, et comment modifier un dessin sans casser les captures de référence. La référence complète des modules est au chapitre « Référence : rendu SVG » ; les jeux d'essai de `src/testing/`, utilisés par les tests et le banc de rendu, sont décrits à la fin de ce chapitre.

8.1 Principes

Un seul dessin (NF-07). L'écran, les aperçus et les exports SVG, PNG et PDF utilisent *les mêmes composants*. Une page est un élément `<svg>` dont la `viewBox` est en millimètres de papier ; le plan de travail l'affiche à la taille `largeur × pixelsPerMm` et l'export SVG à la taille `largeur mm` (`PageContent`). L'export n'est donc jamais une sérialisation du DOM de l'écran : il rend `PageContent` hors du document avec `renderToStaticMarkup` (décision du 05/11/2026, lot 26, voir le chapitre sur les exports). Les conséquences sont fortes et valent pour tout le dossier :

- aucun `foreignObject`, aucun retour à la ligne laissé au navigateur : tout texte passe par `TextBlock`, découpé à l'avance en `<tspan>` (décision du 04/11/2026, lot 07) ;
- pas de `<defs>` partagés, de `<symbol>`/`<use>`, de dégradés ni de `paint-order` : ils sont mal pris en charge par l'export PDF (svg2pdf.js) ou dupliqueraient des identifiants entre le plan de travail, les aperçus et les exports. Les seuls `<defs>` sont les `clipPath` d'une vue sur une plage de U, dont l'identifiant est rendu unique par la vue (`clipId`) ;
- les couleurs sont écrites en dur dans les composants (hexadécimal), jamais par une feuille de style : le SVG exporté doit se suffire à lui-même ;
- ce qui n'appartient qu'à l'écran (zones de clic invisibles, emplacement d'une image non choisie) dépend d'un seul drapeau, `interactive`, faux à l'export.

Sans état propre. Les composants de rendu ne modifient rien : ils lisent le magasin de l'éditeur (`useEditorState`) ou reçoivent le projet en propriété. La géométrie est calculée par des fonctions pures hors de React, testées unitairement (règle 6 de `CLAUDE.md`) : `model/rackGeometry` (armoire), `model/pageLayout` et `model/pageGeometry` (page, vues), `model/titleBlockLayout` (cartouche), `model/annotations` (éléments du schéma), `routing/`

`page` (câbles), `text/layout` (texte), et, dans ce dossier, `render/portShapes` et `render/logo`. Les composants ne font qu'écrire des éléments SVG à partir de ces résultats.

Performance (NF-02b). Trois moyens imposés par le cahier des charges : seule la page active est montée (`PageCanvas`) ; chaque composant coûteux est mémoïsé par entité (`React.memo` et sélecteur par identifiant : `DeviceView` ne s'abonne qu'à son équipement et à sa machine) ; le niveau de détail des ports est une simple chaîne (`'full'`, `'simple'`, `'none'`) passée aux vues mémoïsées, de sorte qu'un zoom qui ne franchit pas le seuil ne redessine aucune vue. Les éléments répétés (vis, aérations, perforations, cadres de keystones, grille) sont regroupés dans un seul `<path>` pour limiter le nombre de nœuds (décision du 05/11/2026, lot 36). Les nombres écrits dans les chemins sont arrondis au millième (fonction locale `r3`) pour garder des SVG compacts et stables.

8.2 Repères et unités

Quatre repères s'emboîtent ; savoir dans lequel on se trouve évite la plupart des erreurs d'échelle.

Repère	Unité et usage
Page	millimètres de papier, origine en haut à gauche de la feuille. C'est la `viewBox` du `<svg>` ; cadre, cartouche, légende, câbles, renvois, étiquettes latérales, éléments du schéma et vues extérieures y sont dessinés directement.
Armoire	millimètres réels de l'armoire (`rackGeometry`). Une vue d'armoire applique `translate(x y) scale(échelle)` (échelle effective, `effectiveScale` : la plus petite des vues d'armoires si la page impose « même échelle »). `RackView`, `RackContents` et les faces des équipements posés sont dans ce repère.
Face	la boîte `box` d'une face (`x`, `y`, `width`, `height`) dans le repère courant ; les ports y sont placés en proportions (`port.x`, `port.y`, `port.w`, `port.h` entre 0 et 1) relatives au gabarit de la face, jamais à l'image (décision du 04/11/2026, lot 09). Le décor utilise `k`, unités de dessin par millimètre de l'équipement réel (`faceScale`).
Connecteur	millimètres du connecteur réel : chaque dessin de port est défini une fois à sa taille naturelle (`SHAPE_SIZES`) et placé par `translate … scale …` (`fitShape`).

Parce que le décor est exprimé en millimètres de l'équipement multipliés par `k`, une même machine se dessine à l'identique dans une baie (repère armoire, mm réels, $k \approx 1$), dans une vue extérieure (repère page, `k` petit), dans l'éditeur de machine et dans la bibliothèque (pixels). Une épaisseur de trait fixe (par exemple `strokeWidth={0.6}` du fond d'une face) est en revanche dans l'unité du repère courant : c'est voulu pour les contours, à éviter pour les détails fins.

Les tailles de texte suivent la même règle : `font.size` est dans l'unité du repère (mm de papier pour la page, mm réels dans une armoire : la numérotation des U, en 10 mm réels, devient environ 1,2 mm de papier à l'échelle 0,12).

8.3 Arbre des composants d'une page

`PageCanvas` lit la liste des pages et ne monte que la page active, avec son numéro et le nombre de pages ; `PageView` lit la page et le projet dans le magasin et délègue à `PageContent`,

qui dessine dans cet ordre (ordre de peinture SVG, le dernier est au-dessus) :

1. le fond blanc et le cadre de la feuille (à mi-marge, coins arrondis de 3 mm) ;
2. la grille de référence de `GRID_STEP` mm (export seulement, trait fort tous les 50 mm) ;
3. les éléments du schéma placés *sous* les vues (`AnnotationsLayer`, `layer="below"`) ;
4. chaque vue : une vue d'armoire (titre, `RackView` contenant `RackContents`, étiquettes à côté de l'armoire, PAG-06) ou une vue extérieure (`DeviceFace` dans la boîte de la vue, et son libellé dessous) ;
5. les câbles (`LinksLayer`), au-dessus des vues ;
6. les éléments du schéma placés *au-dessus* des vues ;
7. le tableau des révisions (PAG-14), la légende en bloc libre (PAG-11) et le cartouche (PAG-09, PAG-10), `render/TitleBlock`.

Les composants internes `RackViewOnPage` et `ExternalViewOnPage` sont mémoïsés : modifier une vue ne redessine pas les autres. Les étiquettes latérales (`SideLabels`) sont calculées par `rackSideLabels` (pur) et alignées sur le U de leur équipement, en police de `LABEL` ; le titre d'une vue utilise `VIEW_TITLE`. Ces deux constantes sont aussi lues par l'export SVG pour calculer l'emprise d'une vue exportée seule.

8.4 L'armoire

`RackView` dessine l'armoire vue de face ou de dos (ARM-04) à partir de `rackGeometry(rack, face)`, en millimètres réels :

- le cadre selon le type d'armoire : panneaux latéraux, toit et socle pour une armoire fermée ou murale ; montants pour une baie ouverte à quatre montants ; seulement une traverse haute et un pied élargi pour une baie à deux montants (`open2`). Le cadre prend la couleur de l'armoire, toit et socle une teinte assombrie à 75 % ;
- les emplacements 0U (rectangles en tirets), la numérotation des U des deux côtés (`uNumber` tient compte du sens de numérotation de l'armoire), les montants avec trois trous de fixation par U aux cotes EIA-310 (`HOLE_OFFSETS_MM`) ;
- le contenu (`children`) au-dessus des montants : c'est là que `RackContents` est inséré ;
- les roulettes ou les pieds, sauf pour une armoire murale.

Les portes ne sont pas dessinées et les goulottes sont seulement réservées dans la géométrie, où le routage les utilise (décision du 04/11/2026, lot 08).

Plage de U (PAG-16). Quand la vue a une plage de U, `rackWindow` donne les limites. Le dessin est décalé vers le haut de `window.top` ; le cadre est découpé par un `clipPath` en forme de contour en zigzag (`cutOutline`), le contenu (montants, numérotation, équipements) par un rectangle coupé net aux limites ; toit, socle et roulettes disparaissent du côté coupé, et deux lignes brisées (`cutLines`) marquent la coupure. Un équipement à cheval sur une limite est donc coupé net, comme l'exige le cahier.

Ordre des équipements (EQP-09). `rackDrawingOrder` dessine d'abord les équipements fixés sur l'autre face (vus de dos), puis ceux fixés sur la face affichée, qui les recouvrent : un commutateur retourné derrière un panneau de brassage est caché par ce dernier en vue avant. Chaque équipement est un `DeviceView` qui obtient sa boîte et la face du modèle à montrer par `deviceInRackFace` : vu depuis sa face de montage, un équipement montre la face avant de sa machine ; vu de l'autre côté, sa face arrière, et son côté de demi-largeur est inversé (décision du 04/11/2026, lot 09). Un équipement placé malgré un dépassement de profondeur (EQP-02, `depthForced`) reçoit un contour orange (`FORCED_COLOR`).

8.5 La face d'un équipement

`DeviceFace` est le cœur du dessin : il sert dans les armoires, les vues extérieures, l'éditeur de machine, l'aperçu de pose et le panneau de bibliothèque. Pour une face donnée, il choisit l'un de trois dessins :

1. **Photo** (MAC-01, EDM-03) : si la face est en mode `image` et que l'image est disponible, un `<image preserveAspectRatio="xMidYMid meet">` ajusté sans déformation dans la boîte de la face.
2. **Dessin d'extérieur** : un équipement extérieur dont la sous-catégorie a un dessin maison (`hasExternalDrawing` : écran, pupitre tactile, poste de travail, portable, clavier et souris) est dessiné par `ExternalDrawing`, en proportions de sa boîte, sans image ni marque (CG-08, décision du 04/11/2026, lot 11). Son logo éventuel est posé à l'endroit prévu (`externalLogoLayout`).
3. **Dessin par défaut** (MAC-02, EDM-10) : fond de la couleur de la face (sauf l'étagère, au fond transparent : `transparentBackground`), puis le décor (`StandardDecoration`), le logo, la machine posée sur une étagère (MAC-03) et le libellé.

Viennent ensuite, quel que soit le dessin, les ports (sauf niveau `none`) puis le contour orange d'une pose forcée.

Libellé. Le libellé est celui de la face, sinon le nom de la machine, sauf pour les équipements passifs « nommés par leurs paramètres » (obturateur, passe-câbles, étagère, PDU, panneaux, générique : `labelledByParams`), qui n'affichent que le libellé de leur face. Il occupe la zone libre calculée par `labelZone` : après l'oreille (et le logo), jusqu'au premier port ou jusqu'au décor propre au type (baies de disques d'un serveur à 29 % de la largeur, afficheur d'onduleur à 40 %, boutons d'un KVM à 48 %). Sa taille est `min(0,32 × hauteur, 12)` ; il n'est écrit que si la zone fait au moins `min(30, 2,5 × taille)`. Sa couleur contraste avec la façade (`isLight`).

Décor « réaliste sobre ». `StandardDecoration` dessine, au-dessus du fond et sous les ports, un léger relief (deux bandes semi-transparentes plutôt qu'un dégradé), les oreilles de 16 mm avec une vis par U en face avant des équipements de baie, puis les parties propres à chaque type d'équipement :

Type	Parties dessinées
commutateur, routeur, pare-feu	plaques des blocs de ports ; voyants en face avant ; aérations et blocs d'alimentation en face arrière
serveur, stockage	baies de disques (si la façade est libre), voyants et bouton d'alimentation en face avant ; plaques, aérations, alimentations en face arrière
KVM	voyants et boutons de sélection ; plaques et alimentations à l'arrière
PDU	barrette de prises ; interrupteur et voyant de parafoudre selon les paramètres en face avant (horizontale), boutons de fixation aux deux extrémités (verticale)
onduleur	voyants, afficheur, aérations ; plaques à l'arrière
panneau de brassage	keystones numérotés ; barre de maintien des câbles à l'arrière
tiroir optique	cassettes ; trait du tiroir en face avant
obturateur, passe-câbles	perforations éventuelles et vis ; anneaux ou brosse
étagère	plateau et équerres (coulissante ou non), sans relief
générique	aérations s'il n'a pas de ports, voyants

Les blocs d'alimentation sont reconnus aux ports nommés `PSU<n>` ; les plaques regroupent les ports par groupe, un groupe aux ports éloignés étant coupé en blocs de voisins. Toutes les tailles sont en millimètres de l'équipement (`k`).

Logo (EDM-10). Le logo d'une machine n'est dessiné que sur le dessin par défaut de sa face avant, jamais sur une photo ni en face arrière (décision du 05/11/2026, lot 39a). Sur un équipement de baie, `rackLogoLayout` le place au début de la zone du libellé, centré verticalement, au plus `LOGO_MAX_HEIGHT_MM` mm de haut, `LOGO_MAX_WIDTH_MM` mm de large et `LOGO_MAX_ZONE_SHARE` de la zone ; il est omis sous 3 mm. L'image est posée sur une plaque claire à filet noir, lisible sur une façade anthracite comme en impression noir et blanc. Les proportions du logo sont lues dans le modèle (`Machine.logo.aspect`), jamais en décodant l'image : le dessin est ainsi identique à l'écran et dans les exports.

Machine posée sur une étagère (MAC-03). Sur la face avant d'une étagère qui porte une image de machine (`model.carried`), `carriedLayout` place l'image debout sur le plateau, entre les équerres, aussi grande que la place le permet ; le libellé n'est alors pas écrit.

8.6 Les ports

8.6.1 Forme d'un port

Chaque port est dessiné d'après la *forme* de son type de connecteur (MAC-02, décision du 05/11/2026, lot 36) : 31 dessins (`PORT_SHAPES`), écrits une fois dans `SHAPE_DRAWINGS` à la taille naturelle du connecteur réel. La résolution d'un type de connecteur en dessin (`portLook`) suit trois étapes :

1. un type par défaut (identifiant fixe) prend toujours la forme *actuelle* de cet identifiant (`CONNECTOR_SHAPES`) : un projet enregistré avant un changement de dessin suit le nouveau dessin, sans migration ;

2. sinon, la forme enregistrée du type si elle est connue ; `generic` n'est retenu que pour la famille « autre » ;

3. sinon, un ancien nom (`round`, avant le lot 36) traduit selon la famille, puis le dessin de la famille (RJ45 pour le cuivre, SFP pour l'optique, HDMI pour la vidéo…).

Les **connecteurs personnalisés** (types créés dans la bibliothèque du projet, BIB-09, lot 39a) passent par la même résolution : un nouveau type a la forme `generic` et se dessine donc selon sa famille, à moins que l'utilisateur ne lui choisisse une forme. `usePortLooks` calcule la table identifiant → dessin pour tous les types du projet, recalculée seulement quand la liste des types change ; un port dont le type est introuvable reçoit `UNKNOWN_PORT_LOOK`.

`PortShape` place le dessin par `fitShape` : proportions naturelles, centré, aussi grand que le rectangle du port le permet. Le dessin d'un connecteur large (VGA) ne couvre qu'une partie du rectangle : un rectangle invisible `port-hit` (`fill="none"`, `pointer-events="all"`) couvre tout le rectangle, pour que le survol, l'infobulle et le tirage d'une liaison fonctionnent sur toute la surface du port. Sur un commutateur, un routeur ou un pare-feu en dessin par défaut, la rangée du haut d'une paire de rangées est retournée (ergot en haut, comme sur les vrais appareils) : `upperRowPorts`. Les ports ajoutés à un équipement posé (PRT-04, `instancePorts`) sont dessinés après ceux de la machine.

8.6.2 Niveau de détail (NF-02b)

Les ports sont *toujours* dessinés. Ils sont simplifiés en petits rectangles de la couleur de leur famille (`PORT_COLORS`) seulement au très fort dézoom. Le seuil, en pixels d'écran par millimètre de papier, est calculé par `portSimplifyThreshold` :

$$\text{seuil} = \min\Big(1{,}6\ ;\ \sqrt{\frac{L \times H}{4 \times 210 \times 297}}\Big)$$

où L et H sont la largeur et la hauteur du plan de travail en pixels. La constante `PORT_SIMPLIFY_BELOW_PX_PER_MM` vaut 1,6 : à ce zoom, une page A4 paysage fait environ 475 pixels de large, soit environ quatre pages A4 sur un écran de bureau ; la racine abaisse le seuil dans une petite fenêtre pour que les ports ne soient simplifiés que lorsque plus de quatre pages A4 (`PORT_SIMPLIFY_MIN_VISIBLE_MM2`) tiennent dans le plan de travail (une page A3 ajustée dans la fenêtre d'un portable garde ses ports détaillés). `portDetailAt` compare le zoom à ce seuil. Le seuil porte sur le zoom de la page et non sur l'échelle de chaque vue : toutes les vues changent de détail ensemble. Le plan de travail (`Canvas`) passe le niveau à `PageCanvas` ; sans taille de fenêtre, le seuil fixe s'applique ; les exports passent 100 pixels par millimètre et sont donc toujours détaillés. Le niveau `none` ne vient que du masquage volontaire des ports d'une vue extérieure (`portsHidden`).

8.7 La couche des câbles

`LinksLayer` dessine les câbles d'une page au-dessus des vues. Il ne calcule rien lui-même : `routePageCached(project, page, avoid, hidden)` (chapitre « Routage ») rend les tracés, faisceaux, repères, étiquettes de destination et renvois ; le dernier résultat est gardé (cache à une entrée : même projet, même page, même module, mêmes types masqués). Le module libavoid (WebAssembly) est chargé à la demande par `useAvoid` ; tant qu'il manque, le routage donne un tracé de repli et l'attribut `data-routed` vaut `false` (les tests de bout en bout attendent `true`). Ordre de peinture : câbles, troncs de faisceaux (ROU-11, largeur multipliée par `BUNDLE_WIDTH_FACTOR`), libellés des faisceaux, puis, si la partie « repères

de câbles » est demandée (EXP-05), références des câbles, étiquettes du mode repères (ROU-09) et renvois (ROU-08, police `RENVOI_FONT`).

Détails à connaître :

- un câble est un `<path>` dont les petits arcs de pont (ROU-12) sont produits par `cablePathData` ; style et épaisseur viennent du type de liaison, avec le motif de tirets de `dashArray` ;
- à l'écran, un second trait transparent plus large (`link-hit`) facilite le clic ; il n'est pas exporté ;
- un texte posé sur un câble (libellé de faisceau, référence) a un rectangle blanc dessous qui masque le trait ; une étiquette du mode repères a un halo blanc obtenu par une copie tracée dessous, et non par `paint-order`, inconnu de l'export PDF.

Les types de liaisons masqués (LIA-10) sont exclus par le routage, pas par le dessin.

8.8 Éléments du schéma

`AnnotationsLayer` dessine les éléments de l'onglet « Schéma » (PAG-07, lot 48) d'une couche donnée : textes, rectangles (arrondis ou non), ellipses, bulles, notes, tableaux, zones titrées, lignes et flèches, images. La géométrie (chemins des lignes, pointes de flèches, colonnes d'un tableau, rotation autour du centre) vient de `model/annotations`. Les textes passent par un `BoxText` qui calcule le nombre de lignes tenant dans la hauteur (texte coupé par « … » au-delà) et centre verticalement le bloc dans une bulle ou une cellule. À l'écran seulement, chaque élément reçoit une zone de sélection transparente ; une zone n'est sélectionnable que par son cadre et son titre, pour que les vues qu'elle entoure restent accessibles ; une image pas encore choisie est remplacée par un cadre en tirets (rien à l'export).

8.9 Cartouche, légende, révisions

`TitleBlock` dessine le cartouche (PAG-09, PAG-10) de `TITLE_BLOCK_HEIGHT` (26 mm) en bas de la zone utile. Les champs, leur ordre et leur place viennent du projet, mis en page par `titleBlockLayout` ; chaque champ montre son libellé (`titleBlockFieldLabel`) et sa valeur, découpée sur autant de lignes que la cellule le permet. Le logo du cartouche est fourni par l'utilisateur ; il n'y en a aucun par défaut (CG-08). La légende (PAG-11) liste les types de liaisons présents sur la page (`legendLinkTypes`) avec un échantillon du trait, soit dans le cartouche, soit en bloc libre (`LegendBlock`) ; sans cartouche, une légende prévue dans le cartouche est dessinée à sa place en bloc libre. Le tableau des révisions (`RevisionTable`, PAG-14) montre les dernières révisions, la plus récente en bas, ancré par son coin inférieur droit.

8.10 Planche d'étiquettes

`LabelSheet` dessine une planche d'étiquettes de câbles (EXP-07), en millimètres de papier, pour l'aperçu de la fenêtre d'export comme pour le PDF : la même règle « un seul dessin ». Le placement des étiquettes (`PlacedLabel`) et le choix des lignes et tailles de police (`fitLabel`) sont des fonctions pures de `export/labels` ; le composant ne fait qu'écrire le fond, les contours de découpe optionnels et les textes.

8.11 Images et texte : les deux contextes

Deux contextes React fournissent ce qui diffère entre l'écran et l'export, sans que les composants le sachent :

- `ImageUrlContext` donne l'URL d'une image du projet. À l'écran, `createImageUrlResolver` crée une URL d'objet (`blob:`) une seule fois par image (les images sont immuables : même référence, même contenu) ; l'export SVG fournit des URL `data:` pour que le fichier se suffise. Une image absente donne `undefined` : le composant n'écrit alors pas d'`<image>` (le logo garde sa plaque).
- `TextMeasureContext` donne la fonction de mesure du texte. Dans l'application, c'est `createCanvasMeasurer` (`canvas.measureText` avec Inter chargée, voir le chapitre « Texte ») ; `main.tsx` attend les quatre graisses d'Inter avant le premier rendu (NF-07b). Les exports réutilisent le même mesureur. `useMeasureText` lève une erreur si le contexte manque : tout nouveau point de montage d'un composant de rendu (banc, fenêtre, test) doit le fournir.

`TextBlock` est le seul composant qui écrit du texte : il appelle `layoutText` (lignes, positions, coupure par « … » au-delà de `maxLines`) et produit un `<text>` à un `<tspan>` positionné par ligne, en police Inter, avec `xml:space="preserve"`. La ligne de base est placée avec les métriques d'Inter, sans `dominant-baseline`.

8.12 Ce que partagent l'écran et les exports

`PageContent` est le point commun. Ses options distinguent les usages :

Option	Écran	Export
`pixelsPerMm`	zoom du plan de travail	100 (ports détaillés)
`portDetail`	calculé par le plan de travail	déduit de `pixelsPerMm` : `full`
`parts`	`SCREEN_PARTS`	choisies dans la fenêtre d'export (EXP-05)
`sizeInMm`	faux (taille en pixels)	vrai (`width="297mm"`)
`crop`	absent	emprise d'une vue exportée seule
`interactive`	vrai	faux : pas de zones de clic ni d'emplacements vides

Un test de bout en bout compare pixel à pixel le SVG exporté, affiché comme image, et le dessin de l'écran à la même taille (0 pixel de différence attendu) : tout écart entre les deux chemins est donc détecté.

8.13 Modifier un dessin

8.13.1 Où intervenir

- **Nouvelle forme de port** : ajouter la clé à `PORT_SHAPES`, sa taille naturelle à `SHAPE_SIZES` et son dessin (en mm du connecteur, origine en haut à gauche) à `SHAPE_DRAWINGS` ; TypeScript refuse une table incomplète. Si un type par défaut doit l'utiliser, changer sa forme dans `library/defaultTypes` (`CONNECTOR_SHAPES`) : les projets existants suivront. Mettre à jour la liste des formes dans le manuel et la scène `connectors` du banc.

- **Décor d'un type d'équipement** : ajouter un composant de partie au module `render/StandardDecoration` (tailles en mm × `k`, éléments répétés dans un seul chemin, pas de `<defs>`) et l'appeler dans le `switch` du type. Si la partie occupe la façade à gauche, ajuster `labelLimit` pour que le libellé s'arrête avant elle.
- **Nouvel extérieur dessiné** : ajouter une fonction de dessin à la table `DRAWINGS` de `ExternalDrawing.tsx`, et sa zone de logo à `EXTERNAL_AREAS` de `logo.ts`.
- **Nouvel élément de page** : le dessiner dans `PageContent` à sa place dans l'ordre de peinture ; s'il doit pouvoir être retiré d'un export, ajouter une partie à `PageParts` (et à `SCREEN_PARTS`, aux fenêtres d'export et à leur documentation).
- **Texte** : toujours `TextBlock`, jamais un `<text>` écrit à la main ; tout libellé visible passe par `src/i18n/fr.json`.

Les tests unitaires du dossier rendent les composants avec `renderToStaticMarkup` ou *Testing Library* et vérifient la structure (classes `std-…`, `port-shape-…`, nombre de vis, ordre des équipements, nombre de rendus) ; ils complètent les captures, qui vérifient l'aspect.

8.13.2 Captures de référence

Le banc de rendu `tests/harness/` (servi par `npm run dev`, hors du build, décision du 04/11/2026, lot 07) affiche des scènes SVG fixes (`/tests/harness/index.html?scene=<nom>` : `devices`, `standards`, `connectors`, `page`, `routedPage`, `viewRange`, `annotations`…) construites avec les jeux d'essai ci-dessous. Les tests de bout en bout (`tests/e2e/`) en prennent des captures comparées aux images de `tests/e2e/__screenshots__/`, dans Chromium avec un rendu du texte fixé (sans hinting, sans antialiasing sous-pixel, profil sRGB).

La tolérance est **absolue : 50 pixels différents** par capture (seuil de couleur 0,1), et non un pourcentage, qui laissait passer de vrais changements sur de grandes images (décision du 04/11/2026, lot 11). Pour un changement visuel *voulu* :

1. régénérer avec `npx playwright test --update-snapshots=all` (le mode par défaut ne réécrit pas une capture restée sous la tolérance, qui garderait alors l'ancien aspect) ; on peut limiter au fichier de test concerné ;
2. **regarder chaque image modifiée** et vérifier que seul le changement voulu apparaît ;
3. le signaler dans le message du commit, avec les captures concernées.

Une capture ne se met jamais à jour pour faire passer un test dont l'écart n'est pas voulu (règle 8 de `CLAUDE.md`). Utiliser le Chromium préinstallé, sans `playwright install`.

8.14 Jeux d'essai

Le dossier `src/testing/` n'est pas livré dans l'application : il fournit des projets déterministes partagés par les tests unitaires, les tests de rendu et le banc.

- `testing/fixtures` : projet d'essai complet et petit (`sampleProject`), chaîne serveur → panneaux → commutateur (`patchChain`), et petites fabriques (ports, machines générées). Les projets partent de `typesOnlyProject`, sans machines, pour que chaque test ne voie que les siennes.

- `testing/referenceProject` : la page du schéma de référence de `docs/references/`, avec des machines génériques.
- `testing/rangeProject` : vues sur une plage de U (PAG-16) et câbles vers des équipements coupés.
- `testing/generatedRack` : baies denses générées à partir d'une graine pour les tests de propriété du routage (lot 38).
- `testing/routingChecks` : rectangles des ports dessinés et détection d'un câble qui passe devant un port étranger (ROU-14).
- `testing/fonts` : Inter en TrueType lue sur le disque pour les tests sous Node.

Leurs identifiants sont fixes (compteur ou graine) : une capture du banc ne doit pas dépendre de `crypto.randomUUID`. Modifier l'un de ces projets change les captures et les tests qui l'utilisent : vérifier les deux.

8.15 Limites connues

- Les portes d'armoire ne sont pas dessinées ; les goulottes ne sont que réservées.
- La légende du cartouche utilise un motif de tirets fixe, indépendant de l'épaisseur du trait, alors que les câbles utilisent `dashArray`.

Chapitre 9

Texte : polices, mesure et découpage

Tout texte du schéma (nom d'un équipement, titre d'une vue, cartouche, renvoi, étiquette de destination, bloc de texte libre…) est dessiné en SVG avec une seule police, Inter, et découpé en lignes *avant* d'être dessiné. Ce chapitre explique pourquoi, comment la police est embarquée, comment le texte est mesuré et découpé, et comment les exports retrouvent exactement les mêmes glyphes. Le code se trouve dans `src/text/` ; la référence de ses modules est au chapitre « Référence : routage et texte ».

9.1 Les exigences et leurs conséquences

Deux exigences du cahier des charges gouvernent ce dossier :

NF-07 le rendu à l'écran et les exports PDF/SVG sont identiques : même SVG, même police, et **textes découpés à l'avance** en lignes (`<tspan>`) par un utilitaire de mesure (`canvas.measureText` avec la police embarquée), au lieu de compter sur le retour à la ligne automatique du navigateur ;

NF-07b une police unique, **Inter** (licence OFL), embarquée dans l'application (aucun chargement externe) et utilisée à l'écran comme dans les exports ; les mesures attendent que la police soit chargée.

S'y ajoutent CG-02 (aucun accès réseau hors de l'application : pas de CDN de polices) et EXP-02 (un SVG exporté est autonome : il embarque ses polices).

Les conséquences techniques, consignées le 2026-11-04 au lot 07 (« Mesure et découpage du texte ») :

- le découpage est une **fonction pure**, `layoutText`, qui reçoit la fonction de mesure en paramètre (`MeasureText`) : elle se teste sous Node sans navigateur, et l'écran comme les exports appellent la même fonction avec la même mesure ;
- le SVG ne contient jamais de `foreignObject` ni de retour à la ligne laissé au navigateur : chaque ligne est un `<tspan>` positionné ;
- la ligne de base est calculée à partir des métriques d'Inter (`FONT_ASCENT`, `FONT_DESCENT`) et non avec l'attribut `dominant-baseline`, mal géré par la conversion en PDF ;
- l'application attend le chargement des quatre graisses avant le premier rendu.

9.2 La police embarquée

9.2.1 Quatre graisses, deux familles de fichiers

Inter est utilisée en quatre graisses, `FONT_WEIGHTS` : 400, 500, 600 et 700. Le nom de famille, `FONT_FAMILY`, vaut `Inter` partout (CSS, attributs SVG, PDF). Deux jeux de fichiers de la même version de la police, aux mêmes métriques, sont livrés dans le build :

Sous-ensembles WOFF2 (écran) fournis par le paquet `@fontsource/inter` : un fichier par écriture (latin, latin étendu, cyrillique, grec, vietnamien…) et par graisse, chacun avec sa plage `unicode-range`. `src/main.tsx` importe les feuilles `@fontsource/inter/400.css` … `700.css` : le navigateur ne télécharge (depuis l'application elle-même) que les fichiers des caractères affichés. `WEB_FONT_FILES` recense ces fichiers au build (`import.meta.glob` sur `node_modules/@fontsource/inter`) avec leur plage, pour que l'export SVG puisse choisir les mêmes.

TrueType complet (PDF et symboles) fourni par le paquet `inter-ttf`, un fichier par graisse (`trueTypeUrl`). Il sert au PDF (jsPDF ne lit que le TrueType) et aux **symboles** qu'Inter dessine mais que les sous-ensembles de l'écran n'ont pas : flèches des renvois (« → »), signes mathématiques, formes, cercles numérotés…

Les plages des symboles sont données par `SYMBOLS_UNICODE_RANGE` et, sous forme analysée, par `SYMBOLS_RANGES`. À l'écran, `registerSymbolFaces` ajoute au document, pour chaque graisse, une `FontFace Inter` dont la source est le fichier TrueType restreint à cette plage : le navigateur l'utilise pour ces seuls caractères et ne le charge qu'à leur première apparition.

9.2.2 Démarrage : attendre la police

Au démarrage (`src/main.tsx`, fonction `start`) :

1. `registerSymbolFaces` déclare les faces des symboles ;
2. `waitForFonts` demande le chargement des quatre graisses (`document.fonts.load`) avec un échantillon qui contient une espace et `SYMBOLS_SAMPLE` (« → »), si bien que les faces des symboles sont chargées aussi, puis attend `document.fonts.ready` ;
3. `createCanvasMeasurer` crée la fonction de mesure, fournie aux composants par le contexte React `TextMeasureContext` (`src/render/TextMeasureContext.ts`) ;
4. l'application est montée.

Mesurer avant la fin du chargement mesurerait la police de secours et décalerait toutes les coupures : c'est la raison de cette attente (NF-07b). Le banc de rendu (`tests/harness/`) suit la même séquence avant de signaler `data-ready`.

9.3 Mesurer un texte

`MeasureText` est le type de toute fonction de mesure : `(text, font) => largeur`, la largeur étant exprimée dans l'unité de la taille de police (pixels à l'écran, millimètres de papier dans le dessin, où toutes les tailles sont en millimètres).

L'implémentation de l'application, `createCanvasMeasurer` :

1. crée un `OffscreenCanvas` d'un pixel (ou, à défaut, un `<canvas>` du document) et son contexte 2D ; une erreur est levée si le contexte est indisponible ;
2. mesure chaque texte à une **taille de référence de 100 px** (`context.font = cssFont({ size: 100, weight })`), puis divise par 100 : mesurer directement à 1,4 « px » (une taille en millimètres) donnerait des largeurs arrondies par le moteur de rendu ;
3. garde le résultat par `graisse|texte` dans un cache (20 000 entrées par défaut, vidé d'un coup quand il est plein) ;
4. multiplie la largeur unitaire par la taille demandée.

La chaîne CSS de police est construite par `cssFont` (« `600 12px Inter` »).

Dans les tests unitaires, on passe une mesure factice : `src/text/layout.test.ts` utilise une police « à chasse fixe » (chaque caractère vaut 0,5 × la taille), les tests de rendu et d'export une mesure semblable. C'est ce qui rend le découpage, la mise en page des cartouches, des étiquettes et des exports testables sans navigateur.

9.4 Découper un texte en lignes

`layoutText` reçoit le texte, des options (`LayoutOptions` : police, largeur maximale, nombre maximal de lignes, interligne, alignement) et la mesure. Son algorithme :

1. **Normaliser** : fins de ligne `\r\n` et `\r` ramenées à `\n`, espaces de début et de fin retirées. Un texte vide (ou `maxLines < 1`) donne un bloc sans ligne, de taille nulle ; `truncated` vaut vrai si un texte non vide a été entièrement coupé.
2. **Paragraphes** : le texte est coupé aux retours à la ligne explicites ; chaque paragraphe est découpé séparément.
3. **Mots** : un paragraphe est coupé aux blancs (plusieurs blancs comptent pour une espace). On ajoute les mots un à un à la ligne courante tant que la ligne mesurée tient dans `maxWidth` ; sinon la ligne est close et le mot commence la suivante.
4. **Mot trop long** : un mot plus large que la ligne à lui seul est coupé caractère par caractère (par *point de code* : `Array.from`, si bien qu'un caractère accentué ou hors du plan de base n'est jamais scindé) ; les morceaux pleins deviennent des lignes, le dernier commence la ligne courante.
5. **Nombre de lignes** : au-delà de `maxLines`, la dernière ligne permise reçoit *toute la suite* du texte, puis est raccourcie caractère par caractère jusqu'à tenir avec `ELLIPSIS` (« … ») ; `truncated` vaut vrai.
6. **Position** : la ligne de base de la première ligne est centrée dans sa hauteur de ligne :

$$y = \frac{h - (a + d)\,s}{2} + a\,s$$

où y est la ligne de base de la première ligne, h l'interligne (par défaut $1{,}2\,s$), s la taille, a et d les proportions d'ascendante et de descendante d'Inter ($1984/2048$ et $494/2048$) ; les lignes suivantes sont espacées de h. L'abscisse d'ancrage dépend de l'alignement : 0 (`start`), milieu (`middle`) ou bord droit (`end`) de la boîte, dont la largeur est `maxWidth`, ou la plus longue ligne quand `maxWidth` est infini.

Le résultat (`TextLayout`) donne les lignes (`LaidOutLine` : texte, largeur, ancrage, ligne de base), la largeur de la plus longue ligne, la hauteur du bloc (nombre de lignes × interligne) et l'indicateur de coupure. Le composant `TextBlock` (`src/render/TextBlock.tsx`) le traduit en un `<text>` et un `<tspan>` positionné par ligne, avec `text-anchor` selon l'alignement.

Invariants.

- Toute ligne produite mesure au plus `maxWidth`, sauf une ligne d'un seul caractère plus large que la boîte (aucune coupure possible) ou une ligne réduite à « … » dans une boîte plus étroite que ce signe.
- Le découpage ne dépend que du texte, des options et de la mesure : à mesure identique, l'écran et les exports produisent les mêmes lignes.
- Aucune ligne n'est vide, sauf un paragraphe vide (deux retours à la ligne successifs), conservé comme ligne vide.

Utilisateurs. `TextBlock` et `AnnotationsLayer` (`src/render/`), les étiquettes de câbles (`src/export/labels.ts`), et, indirectement, tout ce qui dessine un cartouche, un titre de vue ou un nom d'équipement. Le routage, lui, ne mesure pas : la largeur des étiquettes de faisceaux et des repères y est estimée (`textLength`), pour que le routage reste indépendant de la police et du navigateur.

9.5 Les polices dans les exports

9.5.1 SVG et PNG

Un SVG exporté doit s'afficher à l'identique hors de l'application (EXP-02). Le module `src/export/fonts.ts` relève, dans le balisage produit, les caractères dessinés par graisse (attribut `font-weight` de chaque `<text>`), puis écrit des règles `@font-face` en URL `data:` :

- pour chaque fichier WOFF2 de l'écran (`WEB_FONT_FILES`) de la bonne graisse dont la plage couvre au moins un caractère utilisé (`coversCodePoint`), le fichier entier, avec sa plage ;
- pour les symboles, un **sous-ensemble** du TrueType de la graisse (`subsetFont` avec `dropLayout`), limité aux symboles utilisés.

Les fichiers sont lus par `fontBytes`, qui les obtient de l'application elle-même (`fetch` sur une URL du build) et les garde en mémoire. Les sources sont rassemblées dans `APP_FONT_SOURCES` (`src/export/appFonts.ts`), injectable dans les tests.

9.5.2 PDF

L'export PDF (`src/export/pdfExport.ts`) enregistre dans jsPDF le fichier TrueType complet de chaque graisse utilisée (encodage `Identity-H`), puis convertit le SVG avec svg2pdf.js. Les lignes sont celles du SVG : aucune mise en page n'est refaite.

9.6 Lire et alléger une police TrueType

`src/text/sfnt.ts` est un lecteur minimal du format sfnt (TrueType à contours `glyf`). Il n'existe que pour embarquer, dans un SVG, les seuls glyphes de symboles utilisés (quelques kilo-octets au lieu de plusieurs centaines).

9.6.1 Lecture

`readSfnt` vérifie la signature `0x00010000` (TrueType ; une police CFF « OTTO » est refusée) et lit le répertoire des tables (étiquette, position, longueur). `glyphIndex` cherche le glyphe d'un point de code dans la table `cmap` : sous-table de format 12 (plateforme 3, encodage 10, tout Unicode) si elle existe, sinon format 4 (plateforme 3, encodage 1, plan de base), par segments. Il rend 0 (le glyphe « absent ») pour un caractère non couvert. `advanceWidth` lit l'avance d'un glyphe dans `hmtx` (le dernier enregistrement long vaut pour les glyphes suivants).

9.6.2 Sous-ensemble

`subsetFont` :

1. cherche le glyphe de chaque point de code demandé ; garde aussi le glyphe 0 ;
2. suit les **glyphes composites** (un « é » est fait d'un « e » et d'un accent) : les composants d'un glyphe gardé sont gardés, récursivement ;
3. reconstruit `glyf` en ne copiant que les contours gardés (alignés sur 4 octets) et une table `loca` au format long, où les glyphes retirés ont une longueur nulle ;
4. copie `head` en passant `indexToLocFormat` à 1, et remplace `post` par une version 3 (sans noms de glyphes) ;
5. retire, si `dropLayout` est demandé, les tables de substitution et de positionnement (`GSUB`, `GPOS`, `GDEF`, `STAT`, `DSIG`, `kern`) : pour des symboles isolés, ligatures et crénage ne servent à rien ;
6. réécrit le répertoire des tables (triées, alignées sur 4 octets, sommes de contrôle), puis l'ajustement de somme de contrôle de `head`.

Invariant essentiel. Les numéros de glyphes, `cmap`, `hmtx`, `hhea` et `maxp` sont conservés tels quels : le sous-ensemble **mesure exactement comme la police complète**, seuls les contours inutiles sont vidés. Une erreur est levée pour une police qui n'est pas du TrueType ou à qui il manque une table requise (`cmap`, `maxp`, `head`, `loca`, `glyf`, `post`). `outlinedGlyphs` compte les glyphes qui ont un contour (tests et diagnostic).

9.6.3 Plages Unicode

`src/text/unicodeRange.ts` lit les valeurs CSS `unicode-range` (`parseUnicodeRange` : « `U+0000-00FF,U+0131` »), teste la couverture d'un point de code (`coversCodePoint`) et découpe un texte en points de code (`codePoints`, paires de substitution réunies).

9.7 Pourquoi ce dossier ne dépend ni de React ni du DOM

La règle 6 de `CLAUDE.md` demande que la mise en page du texte soit indépendante de React et testée unitairement. Dans `src/text/` :

- `layout.ts`, `sfnt.ts` et `unicodeRange.ts` sont purs : ni DOM, ni React, ni état global ;
- `measure.ts` ne touche au DOM qu'à l'intérieur de `createCanvasMeasurer`, appelée une fois au démarrage ; le reste du code ne voit que le type `MeasureText` ;
- `fonts.ts` ne touche au DOM que dans `waitForFonts`, dont l'ensemble de polices est un paramètre (par défaut `document.fonts`) ;
- `fontFiles.ts` dépend du build (les URL des fichiers sont fournies par Vite) ; `registerSymbolFaces` prend aussi l'ensemble de polices en paramètre.

React n'intervient qu'au-dessus : le contexte `TextMeasureContext` distribue la mesure, `TextBlock` dessine la mise en page. Les tests unitaires de `src/text/` tournent donc sous Node : `layout.test.ts` (mesure factice) et `sfnt.test.ts` (fichier Inter Medium lu dans `node_modules` par `src/testing/fonts.ts`).

9.8 Tests et captures

Test	Ce qu'il garantit
`src/text/layout.test.ts`	coupure aux espaces, mots trop longs, accents, retours à la ligne explicites, points de suspension, position des lignes, largeur maximale respectée
`src/text/sfnt.test.ts`	plages Unicode, glyphes trouvés, sous-ensemble (composites gardés, métriques identiques, taille réduite, répertoire valide)
`tests/e2e/text.spec.ts`	dans Chromium, chaque `<tspan>` mesuré au canvas tient dans sa boîte une fois rendu ; pas de `foreignObject` ; capture de référence `blocs-de-texte.png` (scène `textBlocks` du banc)
`src/export/svgExport.test.tsx`	polices embarquées dans le SVG exporté
`tests/e2e/pdf.spec.ts`	PDF conforme à l'écran

9.9 Modifier le texte

Ajouter une graisse l'ajouter à `FONT_WEIGHTS` (le type `FontWeight` suit), importer la feuille `@fontsource` et le fichier TrueType correspondants (`src/main.tsx`, `fontFiles.ts`, motif `import.meta.glob`), puis vérifier les exports SVG et PDF.

Couvrir un nouveau symbole élargir `SYMBOLS_UNICODE_RANGE` (si le TrueType d'Inter le dessine) ; ajouter un caractère à `SYMBOLS_SAMPLE` seulement s'il doit être mesuré dès le premier rendu.

Changer une règle de coupure modifier `layout.ts` et `layout.test.ts` ; toute modification change potentiellement le dessin : relancer les captures de référence (`blocs-de-texte.png`, mais aussi celles des cartouches et des équipements) et les mettre à jour seulement si le changement est voulu.

Changer de police hors périmètre : NF-07b impose Inter ; les métriques `FONT_ASCENT` et `FONT_DESCENT` sont celles d'Inter.

Limites connues. La largeur des textes du routage est estimée et non mesurée (0,62 em par caractère) : une étiquette de faisceau ou un repère peuvent déborder un peu de la place réservée. Le cache de mesure est vidé en entier quand il atteint sa taille maximale. Le lecteur sfnt ignore les polices CFF et les collections (`ttcf`).

Chapitre 10

Fichiers, stockage et import

Ce chapitre décrit tout ce qui fait entrer ou sortir des données de l'application : le format des fichiers de projet (`.rkdg`) et de bibliothèque (`.rkdglib`), les schémas JSON publiés, la politique de versions du format, le brouillon de secours gardé dans le navigateur, l'ouverture et l'enregistrement des fichiers, l'import des images et l'import tableur simplifié.

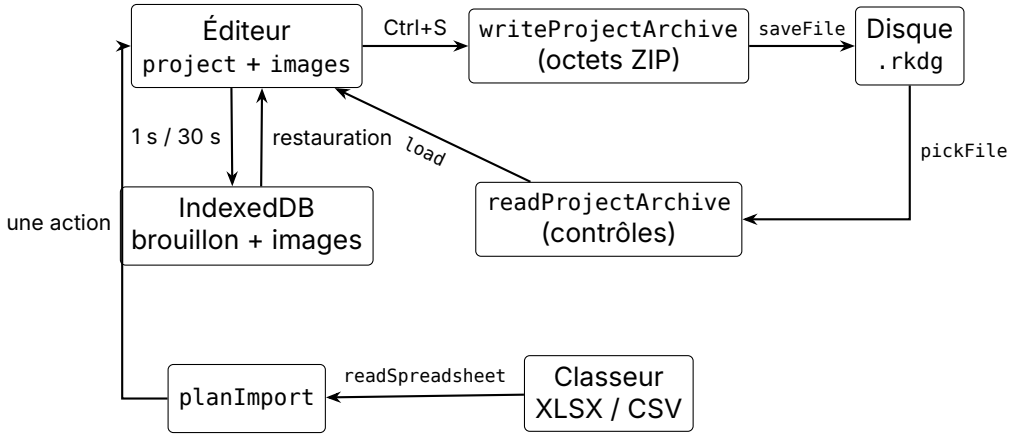
Le code se répartit en deux dossiers, tous deux indépendants de React et testés unitairement (règle 6 du fichier `CLAUDE.md`) :

Module	Rôle
`io/archive`	écriture et lecture des archives `.rkdg` / `.rkdglib`
`io/migrations`	chaîne de migrations entre versions du format
`io/images`	références d'images par empreinte, collecte et renommage
`io/hash`	empreinte SHA-256 (WebCrypto)
`io/svg`	nettoyage des SVG importés, proportions d'un SVG
`io/storage`	base IndexedDB : brouillon, images, préférences
`io/draft`	écriture automatique et restauration du brouillon
`io/files`	sélection, lecture et écriture de fichiers sur le disque
`io/index`	point d'entrée qui réexporte archive, hash, images, svg et migrations
`import/read`	lecture des classeurs XLSX et des fichiers CSV
`import/rules`	règles de l'import tableur (tout ou rien)
`import/template`	modèle de classeur vide et rapport d'erreurs

Ces modules ne conservent aucun état global, à l'exception du nettoyeur SVG par défaut (créé à la première utilisation). L'orchestration (fichier courant, poignée de fichier, démarrage du brouillon, messages d'erreur) vit dans `src/app/session.ts`, qui s'appuie sur eux ; l'import tableur est appliqué au projet par l'action `importSpreadsheet` de `src/store/importActions.ts`.

10.1 Vue d'ensemble des flux

Le projet en cours d'édition est tenu par l'`Editor` (chapitre sur l'état) : un objet `Project` conforme au schéma Zod et une table des images (`ImageMap`, octets par référence). Les flux sont les suivants :



Trois principes traversent ce chapitre :

- **rien n'est perdu** (NF-04) : l'archive est entièrement construite en mémoire avant toute écriture sur le disque, et un brouillon de secours est écrit en continu dans le navigateur ;
- **rien n'est cru sur parole** (NF-06) : un fichier ouvert est vérifié (décompression bornée, chemins, schéma, empreintes des images, SVG nettoyés) et une erreur est rendue avec un message en français, jamais par une exception ;
- **les images sont immuables** (FMT-03, NF-04b) : elles sont nommées par l'empreinte SHA-256 de leur contenu, stockées une seule fois et jamais réécrites.

10.2 Le format de fichier

10.2.1 Structure de l'archive

Un fichier `.rkdg` est une archive ZIP (§ 8 du cahier des charges, FMT-07, type MIME interne `application/vnd.rkdg+zip` : `RKDG_MIME_TYPE`). Le format n'est pas chiffré (FMT-08) : on peut l'ouvrir avec n'importe quel outil ZIP pour le déboguer.

```
projet.rkdg
├── manifest.json          identification du format (FMT-01)
├── project.json           tout le projet, normalisé
├── images/<sha256>.<ext>  png, jpg, jpeg, svg ou webp (FMT-03)
└── thumbnail.png          aperçu, facultatif
```

Un fichier `.rkdglib` a exactement la même structure (FMT-07, FMT-09) ; seuls le contenu du manifeste et celui de `project.json` changent (voir plus bas).

10.2.2 Le manifeste

`manifest.json` est décrit par `ManifestSchema` :

Champ	Contenu
`format`	`"rkdg"` (projet) ou `"rkdglib"` (bibliothèque)
`formatVersion`	entier, version du format du fichier (actuellement `2`, constante `FORMAT_VERSION` de `src/model/schema.ts`)
`appVersion`	version de l'application qui a écrit le fichier (information)
`createdAt`	date de création du projet (ISO 8601), reprise de `project.meta.createdAt`
`modifiedAt`	date de l'enregistrement (ISO 8601)
`library`	bibliothèque seulement (FMT-09) : identifiant (UUID), nom, marque, version, auteur, description, licence ou provenance des images (`LibraryInfoSchema`)

Le manifeste est écrit avec une indentation de deux espaces ; il est le seul fichier lu *avant* de connaître la version du format, il doit donc rester lisible par toutes les versions futures : n'y ajouter que des champs facultatifs.

10.2.3 Le contenu : `project.json`

Pour un projet, `project.json` est l'objet `Project` complet, avec son champ `formatVersion`. `serializeProject` le fait passer par `ProjectSchema.parse` avant de l'écrire : les valeurs par défaut sont complétées, les clés inconnues retirées et l'ordre des clés est celui du schéma, quel que soit l'ordre dans l'objet en mémoire. L'indentation est d'un espace, avec un saut de ligne final. Cette normalisation rend l'écriture déterministe : relire puis réécrire un fichier redonne un `project.json` identique octet pour octet (test « round-trips » de `src/io/archive.test.ts`, décision du 04/11/2026, lot 04).

Pour une bibliothèque, `project.json` contient seulement `{ formatVersion, library }` (`LibraryDocumentSchema`) : machines, types de connecteurs et types de liaisons, sans armoires, liaisons ni pages (FMT-09). Le champ `editedLibrary` d'un projet (bibliothèque ouverte pour être modifiée, BIB-13, décision du lot 44) n'apparaît jamais dans un `.rkdg` : à l'enregistrement d'un tel document, la session appelle `writeLibraryArchive` avec le contenu calculé par `libraryToSave`.

10.2.4 Les images

Une image est désignée dans `project.json` par sa *référence* (`ImageRef`) : les 64 chiffres hexadécimaux de l'empreinte SHA-256 de son contenu, un point et son extension, par exemple `3f2a…c1.webp`. Les références se trouvent à quatre endroits, que `collectImageRefs` parcourt :

- le logo du projet (`meta.logoImageId`, cartouche) ;
- les images des faces avant et arrière d'une machine (`front.imageId`, `rear.imageId`) ;
- le logo d'une machine (`logo.imageId`) et l'image d'un équipement porté (`carried.imageId`), via `machineImageRefs` du modèle ;
- les annotations d'image des pages (`kind: 'image'`).

Ce nommage par contenu apporte trois propriétés : une image utilisée par plusieurs machines n'est stockée qu'une fois (FMT-03) ; l'intégrité se vérifie à la lecture en recalculant l'empreinte ; une image n'est jamais modifiée sur place, ce qui permet de la garder hors de l'historique d'annulation et de ne l'écrire qu'une fois dans IndexedDB (NF-04b).

À l'écriture, seules les images *référencées* par le document sont mises dans l'archive : les images devenues inutiles disparaissent donc d'elles-mêmes à l'enregistrement (FMT-03).

Si une image référencée manque dans la table des images, l'écriture lève `ArchiveWriteError` plutôt que d'écrire un fichier incomplet. Les images matricielles (PNG, JPEG, WebP), déjà compressées, sont stockées sans compression ZIP (niveau 0) ; les SVG et les JSON sont compressés (niveau 6). Les entrées sont écrites dans un ordre fixe (manifeste, contenu, images triées, aperçu).

10.2.5 L'aperçu

`thumbnail.png` est prévu par le format (§ 8.1) et accepté en écriture (`WriteProjectOptions`, champ `thumbnail`) comme en lecture (`ReadProjectResult`, champ `thumbnail`). En l'état, la session n'en produit pas : le champ reste disponible pour un aperçu futur (liste des projets récents, PRJ-08).

10.3 Lecture défensive d'une archive

`readProjectArchive` et `readLibraryArchive` ne lèvent jamais d'exception pour un fichier mauvais : elles rendent un résultat discriminé par `ok`. En cas d'échec, `ReadError` porte un code (`ReadErrorCode`), un message en français prêt à afficher (clés `io.errors.*`) et des détails (une ligne par problème de schéma). L'interface affiche ce message dans une fenêtre d'erreur, jamais une page blanche (FMT-02).

10.3.1 Étapes

Les étapes communes aux deux types de fichiers sont dans la fonction interne `openContainer` :

1. **Signature.** Les deux premiers octets doivent être `PK` ; sinon `notArchive`.
2. **Décompression bornée.** L'archive est poussée par blocs de 16 Ko dans le décompresseur en flux de `fflate` (`Unzip`). Les tailles sont mesurées sur les octets réellement décompressés, jamais sur les tailles déclarées dans l'archive, qu'un fichier piégé peut falsifier. Limites par défaut (`DEFAULT_READ_LIMITS`) : 10 000 entrées, 512 Mo décompressés au total, 64 Mo par fichier JSON. Un dépassement interrompt la lecture (`tooManyEntries`, `tooLarge`).
3. **Chemins.** Un nom d'entrée absolu, contenant `\`, une lettre de lecteur, un segment `..` ou `.` refuse toute l'archive (`unsafePath`). Les entrées inconnues sont décompressées pour être mesurées, mais pas conservées ; seuls `manifest.json`, `project.json`, `thumbnail.png` et les `images/<empreinte>.<ext>` bien formées sont gardés.
4. **Manifeste.** Présence (`missingManifest`), JSON valide (`invalidJson`), conformité au schéma (`invalidManifest`, avec les champs fautifs en détail).
5. **Type.** Le champ `format` doit être celui attendu : un `.rkdglib` ouvert comme projet (ou l'inverse) donne `wrongFormat`. Un manifeste de bibliothèque sans bloc `library` est `invalidManifest`.
6. **Version.** Une version supérieure à `FORMAT_VERSION` est refusée avec un message invitant à mettre à jour l'application (`newerVersion`, FMT-05) ; une version inférieure à `MIN_FORMAT_VERSION` est refusée avec un message clair (`olderVersion`, FMT-04).

7. **Contenu.** `project.json` doit exister (`missingProject`), être du JSON (`invalidJson`) et un objet (`invalidContent`).
8. **Migrations.** `migrate` amène le document de la version du manifeste à la version courante (`migrationMissing` si une étape manque). C'est la version du *manifeste* qui fait foi.
9. **Validation.** Le document migré passe par `parseProject` ou `parseLibraryDocument` (`src/model/validate.ts`) : schéma Zod puis règles de cohérence. Les erreurs bloquantes donnent `invalidContent` avec la liste des problèmes ; les anomalies non bloquantes (LIA-07) sont rendues dans `warnings`.
10. **Images.** Pour chaque référence du document, dans l'ordre : l'image doit être présente (`missingImage`) et son empreinte recalculée doit correspondre à son nom (`corruptImage`). Un SVG est de plus nettoyé (section 10.8) ; si le nettoyage le modifie, il reçoit une nouvelle référence et le document est corrigé sur place (`remapProjectImages`, `remapLibraryImages`). Les images présentes dans l'archive mais non référencées sont ignorées.

Le résultat contient alors le document, le manifeste, les images vérifiées, l'aperçu éventuel, les avertissements et la liste des versions depuis lesquelles le fichier a été migré (`migratedFrom`, vide pour un fichier à jour).

10.3.2 Pourquoi un résultat plutôt qu'une exception

Le message affiché à l'utilisateur dépend du code d'erreur, et les tests vérifient ce code. Une exception aurait mélangé les fichiers mauvais (cas normal, prévu) et les défauts de programmation. Seule `readSpreadsheet` fait exception dans ce chapitre : une erreur d'ExcelJS sur un classeur illisible est levée et rattrapée par la fenêtre d'import.

10.4 Schémas JSON publiés

FMT-02 demande un schéma JSON versionné et publié avec l'application. Les schémas Zod de `src/model/schema.ts` et `ManifestSchema` sont la seule source de vérité (décision du 04/11/2026, lot 04) : `npm run schema` (`tools/build-json-schema.ts`, qui appelle `jsonSchemas` de `tools/json-schema.ts`) en tire trois fichiers JSON Schema (brouillon 2020-12) dans `public/schemas/`, copiés tels quels dans le build :

- `rkdg-project-v2.schema.json` : `project.json` d'un projet ;
- `rkdglib-library-v2.schema.json` : `project.json` d'une bibliothèque ;
- `rkdg-manifest-v2.schema.json` : `manifest.json` (les deux types).

Les schémas décrivent l'*entrée* acceptée (option `io: 'input'`) : un champ qui a une valeur par défaut y est facultatif. Les règles qui ne s'expriment pas en JSON Schema (raffinements Zod, règles de cohérence de `src/model/validate.ts`) n'y figurent pas : un fichier conforme au schéma publié peut encore être refusé à l'ouverture. Le test `tools/json-schema.test.ts` échoue si les fichiers de `public/schemas/` ne sont plus à jour : après toute modification de `src/model/schema.ts` ou du manifeste, lancer `npm run schema` et enregistrer les fichiers produits.

10.5 Versions du format et migrations

10.5.1 Politique actuelle

- **Fichier plus récent** que l'application : toujours refusé, avec un message qui invite à mettre à jour (FMT-05).
- **Fichier plus ancien** : FMT-04 prévoit, à partir de la version 1 (stable) de l'application, une chaîne de migrations v2 → v3 → … Avant la version 1, le format peut changer sans migration. Le format 2 (lot 34 : équipements intégrés et machines) a rompu avec le format 1 : `MIN_FORMAT_VERSION = 2`, un fichier de format 1 est refusé (`olderVersion`, décision du 05/11/2026, lot 34, reportée dans le cahier v1.7). Les schémas JSON v1 ont été retirés.
- La table `MIGRATIONS` est donc vide : le mécanisme est en place, testé avec des migrations factices (`src/io/archive.test.ts`), et attend le format 3.

10.5.2 Mécanisme

`MIGRATIONS[n]` transforme un document de version `n` en document de version `n + 1`. Il reçoit le JSON brut (`JsonObject`, avant toute validation) et rend un nouvel objet ; `migrate` applique les étapes dans l'ordre, réécrit `formatVersion` après chacune et note les versions traversées. Une étape manquante arrête tout (`missing`). Les migrations portent sur `project.json` seulement : le manifeste est réécrit à l'enregistrement suivant, et la même chaîne sert aux projets et aux bibliothèques (leurs `project.json` n'ont pas la même forme : une étape doit traiter les deux, en testant la présence de `meta` ou de `racks`).

10.5.3 Faire évoluer le format

Le champ `formatVersion` de `ProjectSchema` et de `LibraryDocumentSchema` est un littéral (`z.literal(FORMAT_VERSION)`) et Zod retire les clés inconnues : une version de l'application qui lit un fichier plus récent perdrait silencieusement les champs qu'elle ne connaît pas. D'où la règle : **toute modification qui ajoute, renomme ou réinterprète une donnée du fichier change la version du format**. Seule exception : un champ ajouté avec une valeur par défaut que les anciennes versions peuvent perdre sans dommage (cas de `editedLibrary`, jamais écrit dans un `.rkdg`). Procédure :

1. Modifier le schéma Zod dans `src/model/schema.ts` (et les règles de `src/model/validate.ts` si besoin).
2. Incrémenter `FORMAT_VERSION`.
3. Ajouter dans `src/io/migrations.ts` l'étape `MIGRATIONS[ancienne version]`, fonction pure qui ne fait *aucune* supposition non vérifiée sur le document (il n'est pas encore validé) ; laisser `MIN_FORMAT_VERSION` à 2 une fois la version 1 publiée.
4. Lancer `npm run schema` : les fichiers de la nouvelle version apparaissent dans `public/schemas/` ; garder ceux des versions encore lisibles (ils restent publiés), retirer ceux des versions refusées.
5. Ajouter dans `src/io/archive.test.ts` un test qui lit un fichier de l'ancienne version (construit à la main avec `zipSync`) et vérifie le résultat et `migratedFrom` ; adapter le test de bout en bout `tests/e2e/files.spec.ts`.

6. Penser au brouillon : `restoreDraft` *refuse* un brouillon d'une version plus ancienne au lieu de le migrer. Après un changement de version, un brouillon non enregistré de la version précédente est perdu, sauf à appeler `migrate` dans `restoreDraft` (à faire avec la première vraie migration).
7. Consigner la décision dans `docs/decisions.md` et mettre à jour le § 8 du cahier des charges si le format visible change.

10.6 Stockage dans le navigateur

10.6.1 La base IndexedDB

`openLocalStorage` ouvre (via la bibliothèque `idb`) la base `portolan`, version 1, avec trois magasins (décision du 04/11/2026, lot 06, NF-04b) :

Magasin	Clé	Contenu
`project`	`'draft'`	un seul enregistrement : le brouillon (`DraftRecord`)
`images`	référence d'image	octets de l'image, écrits seulement s'ils n'existent pas
`prefs`	nom libre	préférences de l'interface (valeurs quelconques)

Le brouillon ne contient que le JSON du projet (texte, non validé), son nom, la date d'écriture, le drapeau « modifications non enregistrées », le nom du fichier d'origine et, si le navigateur sait la stocker, la poignée de fichier (File System Access) ; si le stockage de la poignée échoue, le brouillon est réécrit sans elle. Les images, immuables, sont stockées à part et une seule fois : le brouillon ne réécrit donc que du JSON, et la compression ZIP n'a lieu qu'à l'enregistrement sur disque (NF-04b).

L'interface `LocalStorage` est l'unique point d'accès : la session et les tests n'utilisent qu'elle (les tests unitaires utilisent `fake-indexeddb`). Si l'ouverture de la base échoue (navigation privée stricte, quota), `src/main.tsx` démarre sans stockage : l'application fonctionne, sans brouillon.

10.6.2 Écriture du brouillon

`startDraftSaver` s'abonne au magasin de l'éditeur :

1. Un changement du projet, de l'entrée d'historique enregistrée (`savedEntryId`, qui fait changer le drapeau `dirty`) ou de la table des images marque le brouillon « à écrire » et relance une minuterie de 1 s (anti-rebond).
2. Un intervalle de 30 s écrit quand même si des changements continuent d'arriver sans pause (glisser prolongé).
3. Les écritures sont enchaînées sur une même promesse : deux écritures ne se chevauchent jamais, et `flush` rend une promesse résolue quand tout est écrit.
4. Les images sont écrites *avant* le JSON : un brouillon ne référence jamais une image absente du stockage.
5. Une erreur d'écriture est remise à `onError` et n'interrompt pas l'édition.

10.6.3 Restauration après une fermeture inattendue

Au démarrage (`session.start`), le brouillon est lu. S'il porte des modifications non enregistrées, la restauration est proposée (PRJ-05) et l'écriture automatique ne démarre pas tant que l'utilisateur n'a pas choisi, pour ne pas écraser le brouillon. Sinon, ou après le choix, la session supprime du magasin les images qui ne servent plus au projet courant (`pruneImages`), y écrit les images courantes puis démarre l'écriture automatique. `restoreDraft` relit le JSON, refuse une version de format plus ancienne, le valide avec `parseProject` comme un fichier ouvert, puis recharge les images ; il compte les images manquantes au lieu d'échouer. Le projet restauré est marqué « modifié » et la poignée de fichier est reprise, pour qu'« Enregistrer » réécrive le fichier d'origine.

Le brouillon **ne remplace jamais le fichier** (PRJ-05) : il n'est qu'un filet de sécurité, et l'enregistrement sur disque reste une action explicite.

10.6.4 Limites

- Un seul brouillon par navigateur : deux onglets ouverts sur Portolan écrivent le même enregistrement (le dernier écrit gagne), et la purge des images faite par l'un peut retirer les images de l'autre (elles restent dans la mémoire de cet onglet et sont réécrites à son prochain changement d'images).
- Pas de migration des brouillons (voir plus haut).
- Le stockage dépend du quota du navigateur ; un échec d'écriture n'est pas signalé à l'utilisateur (`onError` est vide dans la session).

10.7 Ouvrir et enregistrer des fichiers

`io/files` cache la différence entre navigateurs (CG-07, PRJ-02, PRJ-03) :

- avec l'API File System Access, présente dans les navigateurs à moteur Chromium (`supportsFileSystemAccess`), la fonction `pickFile` ouvre le sélecteur natif filtré par extension et rend une poignée (`WritableFileHandle`) ; `saveFile` réécrit le fichier de cette poignée (« Enregistrer ») après avoir vérifié, et redemandé au besoin, la permission d'écriture, ou ouvre le sélecteur d'enregistrement (« Enregistrer sous », poignée absente, permission refusée) ;
- sans cette API : ouverture par un `<input type="file">` caché, et enregistrement par téléchargement (`downloadBytes`) sous le nom proposé ; il n'y a alors pas de poignée et chaque enregistrement produit un nouveau téléchargement.

L'écriture est atomique au sens de NF-04 : `saveFile` reçoit des octets *déjà complets* (l'archive est construite en mémoire avant l'appel), et un flux d'écriture File System Access n'est validé sur le disque qu'à sa fermeture. Une annulation dans un sélecteur (`AbortError`) rend `null` ; les autres erreurs sont levées et affichées par la session. Le nom proposé vient de `suggestedFileName` (caractères interdits par les systèmes de fichiers remplacés par un tiret).

Pour une bibliothèque ouverte en édition (BIB-13), la session vide la poignée à l'ouverture : le premier enregistrement demande toujours un nom, proposé sous la forme `<nom>_v<version>.rkdglib`, pour ne pas écraser la version précédente. L'ouverture par glisser-déposer passe par `session.openFile`, qui appelle `readFile` sur l'objet `File` déposé ; un `.rkdglib` déposé est ouvert comme bibliothèque. Un fichier qui ne peut plus être lu

(déplacé ou supprimé entre le dépôt et la lecture) donne le message d'erreur d'ouverture, comme les autres voies.

10.8 Images importées

10.8.1 Préparation

Une image choisie par l'utilisateur (face de machine, logo, annotation) passe par `extensionFor` (type MIME ou nom de fichier ; `image/jpeg` donne `jpg`) puis par `prepareImage`. Les images matricielles sont gardées telles quelles, sans réencodage (EDM-03 : elles sont préparées dans un logiciel tiers) ; un SVG est d'abord nettoyé. La référence est calculée *après* le nettoyage, sur les octets réellement stockés. L'ajustement sans déformation dans le gabarit de la face relève du rendu, pas de ce module ; `svgAspect` donne les proportions d'un SVG que le navigateur ne sait pas mesurer.

10.8.2 Nettoyage des SVG (NF-06)

Un SVG peut contenir du code exécutable ou des références qui feraient sortir des requêtes réseau (règle 7 : aucun chargement externe). `createSvgSanitizer` configure DOMPurify avec le profil SVG (filtres compris) et :

- interdit `script`, `foreignObject`, `iframe`, `object`, `embed` ; DOMPurify retire de lui-même les gestionnaires d'événements (`onload`…) ;
- garde `use`, mais un attribut `href`, `xlink:href` ou `src` n'est conservé que s'il désigne un élément interne (`#id`) ou une image matricielle embarquée (`…`, idem JPEG, GIF, WebP) ;
- retire des attributs `style` et des éléments `<style>` les `url(…)` externes et les `@import`.

Le nettoyage a lieu à l'import *et* à chaque lecture d'archive : un fichier fabriqué hors de Portolan ne peut pas introduire un SVG dangereux. `sanitizeSvg` utilise la fenêtre du navigateur ; les tests passent une fenêtre jsdom à `createSvgSanitizer`.

Références internes. L'expression qui repère les `url(…)` externes écarte, par une assertion qui couvre le guillemet facultatif, toute référence interne : `url(#degrade)`, `url('#degrade')` et `url("#degrade")` sont conservées dans un style, si bien qu'un dégradé ou un motif d'un logo importé garde son remplissage. Les attributs de présentation (`fill="url(#degrade)"`) ne sont pas touchés.

10.9 Import tableur simplifié

L'import tableur (IMP-01 à IMP-07, lot 45) crée en une fois des équipements et des liaisons à partir d'un classeur. Il est *simplifié* : création seulement, jamais de mise à jour, ni armoire, ni machine, ni type créés. Il est *tout ou rien* : la moindre erreur n'importe rien et produit un rapport.

10.9.1 Chaîne de traitement

1. La fenêtre `ImportSpreadsheetDialog` fait choisir un fichier (`pickFile` avec `.xlsx` et `.csv`) et le lit avec `readSpreadsheet`.
2. Elle appelle aussitôt `planImport` sur le projet courant, sans rien modifier, et affiche les erreurs ou le résumé, avec les avertissements.
3. À la validation, l'action `importSpreadsheet` rejoue `planImport` sur le projet *à cet instant* puis ajoute équipements, liaisons et vues en un seul `editor.apply` : une seule action annulable (IMP-07). Les liaisons importées sont ensuite tracées par le routage normal.
4. En cas d'erreurs, le rapport (`buildImportReport`) peut être téléchargé en CSV.

10.9.2 Lecture des fichiers (IMP-04)

Un fichier dont les quatre premiers octets sont la signature ZIP est lu comme XLSX (ExcelJS, chargé à la demande pour ne pas alourdir le démarrage), sinon comme CSV. Pour un CSV :

- **encodage** (`detectEncoding`) : d'après la marque d'ordre des octets (UTF-8, UTF-16 LE ou BE), sinon UTF-8 si les octets sont de l'UTF-8 valide, sinon Windows-1252 (fichiers d'anciens tableurs) ;
- **séparateur** (`detectSeparator`) : celui de « ; », « , » et tabulation le plus fréquent sur la première ligne, hors guillemets ; « ; » par défaut, comme l'écrivent les tableurs en français ;
- **découpage** (`parseCsv`) : RFC 4180 (guillemets doublés, séparateurs et sauts de ligne entre guillemets) ; toutes les cellules restent du texte.

Pour un XLSX, chaque onglet devient une `Sheet` ; une formule donne son résultat, une date son jour ISO, un texte enrichi son texte brut, une cellule en erreur une cellule vide. Un CSV donne un seul onglet, nommé d'après le fichier : on importe donc équipements puis liaisons en deux fois.

10.9.3 Reconnaissance des onglets et des colonnes

Le format reconnu est celui de l'*export* (EXP-04) : un inventaire ou un carnet de câblage exporté se réimporte tel quel, et le modèle de classeur vide (`buildImportTemplate`, IMP-01) en reprend les en-têtes, plus un onglet « Notice ». Les colonnes sont identifiées par leur en-tête, dans n'importe quel ordre, en ignorant casse, accents, espaces superflus et unité finale entre parenthèses (`headerKey`). Un onglet est reconnu (`sheetKind`) par son nom (« Équipements » ou le titre de l'inventaire, « Liaisons » ou le titre du carnet de câblage), sinon par la présence de ses colonnes obligatoires ; les autres onglets sont ignorés.

Chaque colonne a un usage (`DEVICE_COLUMN_USE`, `LINK_COLUMN_USE`) : obligatoire, facultative, ou ignorée parce que calculée par l'export (type d'équipement, fabricant, hauteur ; armoire, U et extrémités finales d'une liaison). Une colonne obligatoire absente est une erreur sur la ligne d'en-tête. Toute colonne inconnue de l'onglet Équipements devient un champ libre de l'équipement (IMP-02) ; une colonne inconnue de l'onglet Liaisons est ignorée avec un avertissement.

10.9.4 Règles

`planImport` traite les onglets d'équipements, puis les onglets de liaisons, ligne par ligne. Chaque ligne est vérifiée contre le projet **complété des lignes déjà acceptées** : une U occupée par une ligne au-dessus, un port relié deux fois dans le fichier sont des erreurs, exactement comme dans l'interface. Les règles sont celles du modèle (`checkPlacement`, `checkLinkEnds`, `checkCompatibility`, `proposeLinkTypeForPorts`, `nextLinkRef`) : l'import n'a pas de règles à lui, il traduit seulement les cellules en appels.

Ligne d'équipement.

1. Nom obligatoire, absent du projet (IMP-06 : pas de mise à jour) et unique dans le fichier (l'erreur cite la première ligne). Les noms sont comparés sans casse ni espaces superflus (`nameKey`).
2. Machine retrouvée par son nom dans la bibliothèque du projet ; la colonne Référence ne sert qu'à départager deux machines de même nom.
3. Armoire vide ou « Extérieur » : équipement extérieur (une machine montée en armoire sans armoire est une erreur). Sinon, armoire existante et unique, face (« Avant » par défaut, « Arrière », suivie de « gauche » ou « droite »), U entière à partir de 1 pour une machine d'au moins 1U.
4. Un 0U prend le premier emplacement libre du côté écrit (ou des deux côtés) ; un demi-largeur le côté écrit ou le premier côté libre. L'export écrit le côté des deux (« Avant droite », « Arrière gauche (0U) ») : un classeur exporté puis réimporté garde chaque demi-largeur de son côté.
5. La pose est vérifiée par `checkPlacement` ; la profondeur insuffisante est une *erreur* (pas de « Forcer » à l'import).

Ligne de liaison.

1. Équipement A et B retrouvés par leur nom dans le projet complété ; un équipement du fichier refusé plus haut donne une erreur qui cite sa ligne.
2. Port retrouvé par son nom affiché, d'abord exact puis sans casse ; « 1 (Arrière) » désigne le port « 1 » de la face arrière (écriture produite par le carnet de câblage quand le nom existe sur les deux faces) ; un nom ambigu est une erreur qui propose les deux écritures.
3. Repère explicite absent du projet et unique dans le fichier ; un repère vide est attribué automatiquement *après* la lecture de tous les repères explicites de toutes les feuilles, pour ne pas en prendre un écrit plus bas.
4. Type retrouvé par son nom, ou proposé d'après les connecteurs ; longueur en mètres strictement positive ; couleur `#RRVVBB`.
5. Un port déjà relié est une erreur ; des connecteurs incompatibles sont un *avertissement* : la liaison est créée marquée « forcée » (`compatibilityForced`), comme une liaison tirée à la main (LIA-06).

Les équipements extérieurs importés reçoivent une vue sur la page indiquée (la page active), en cascade décalée de 10 mm à partir de (20, 20), à l'échelle des autres vues extérieures de la page. Un fichier sans onglet reconnu ou sans aucune ligne est une erreur. Chaque problème (`ImportIssue`) porte l'onglet, le numéro de ligne dans le tableur (1 = entête), l'en-tête de la colonne et le motif (IMP-05).

10.9.5 Ajouter une colonne

1. Ajouter la colonne à l'export (`src/export/tables.ts`) avec sa clé de libellé `tables.columns.inventory.*` ou `tables.columns.cabling.*` ;
2. l'ajouter, à la même position, à `DEVICE_COLUMNS` ou `LINK_COLUMNS`, avec son usage dans `DEVICE_COLUMN_USE` ou `LINK_COLUMN_USE` ;
3. si elle est lue, la traiter dans `importDevices` ou `importLinks` (`src/import/rules.ts`) ;
4. ajouter sa ligne de notice (`import.notice.devices.*` ou `import.notice.links.*` dans `src/i18n/fr.json`) ;
5. compléter `src/import/rules.test.ts` et, si le comportement visible change, `tests/e2e/import.spec.ts` et le manuel.

10.9.6 Limites

Création seulement ; pas d'aperçu différentiel ; un seul fichier par import ; vues extérieures empilées en cascade ; le rapport CSV ne contient que les erreurs (les avertissements restent dans la fenêtre) ; un équipement posé en forçant la profondeur ne se réimporte pas (décision du 05/11/2026, lot 45).

10.10 Tests

Fichier	Ce qu'il vérifie
`src/io/archive.test.ts`	aller-retour octet pour octet, manifeste, images partagées et purgées, refus (archive tronquée, bombe de décompression, chemins, versions, contenu, images manquantes ou corrompues, mauvais type), nettoyage SVG, bibliothèques, migrations
`src/io/storage.test.ts`	base IndexedDB (`fake-indexeddb`), brouillon automatique, restauration
`src/import/read.test.ts`	encodages, séparateurs, CSV entre guillemets, XLSX
`src/import/rules.test.ts`	toutes les règles d'import, ligne par ligne
`tools/json-schema.test.ts`	schémas de `public/schemas/` à jour
`tests/e2e/files.spec.ts`	ouvrir, enregistrer, brouillon, refus d'un format 1
`tests/e2e/import.spec.ts`	import tableur de bout en bout (REC-13, modèle, rapport)

Chapitre 11

Exports

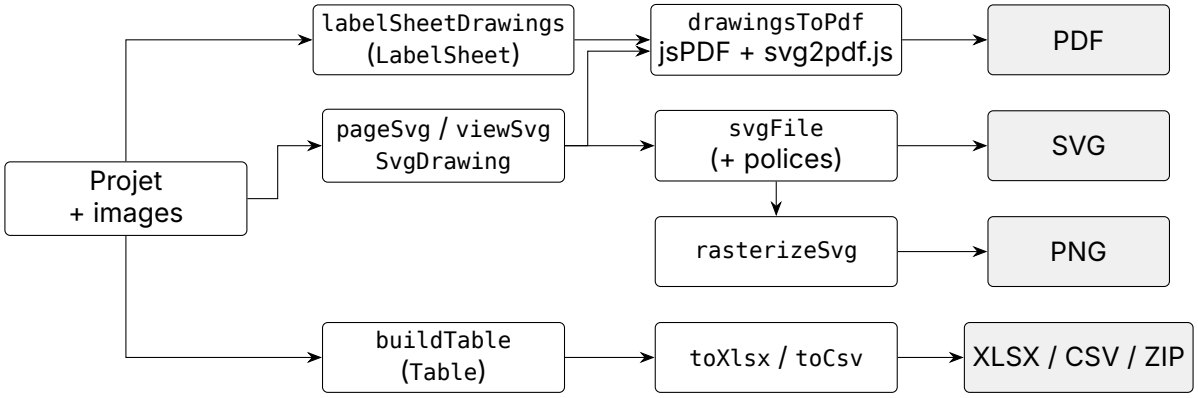
Portolan produit cinq sortes de fichiers à partir du projet ouvert, toutes construites dans le navigateur et téléchargées directement, sans serveur ni boîte d'impression (CG-01, CG-02) :

Commande	Exigences	Fichier produit
`export.pdf`	EXP-01, EXP-05	PDF vectoriel d'une ou plusieurs pages, chacune à son format
`export.svg`	EXP-02, EXP-05	SVG autonome d'une page ou d'une vue, polices intégrées
`export.png`	EXP-03, EXP-05	PNG d'une page ou d'une vue, résolution choisie
`export.spreadsheet`	EXP-04, EXP-05	classeur XLSX, ou CSV (ZIP s'il y a plusieurs listes)
`export.labels`	EXP-07	planches d'étiquettes de câbles en PDF, ou CSV pour imprimante

Le code vit dans `src/export/`. Les fenêtres qui posent les options sont dans `src/ui/exports/` (`ExportPdfDialog`, `ExportImageDialog`, `ExportSpreadsheetDialog`, `ExportLabelsDialog`) ; elles appellent une fonction « build » du module d'export, reçoivent un `ExportedFile` et le passent à `downloadExport` (`src/ui/exports/download.ts`), qui le télécharge. Les commandes elles-mêmes sont déclarées dans `src/commands/export.ts` et ne font qu'ouvrir ces fenêtres.

11.1 Principe : un seul dessin, plusieurs formats

L'exigence NF-07 impose que l'écran et les exports soient identiques : même SVG, même police, textes découpés à l'avance. Les exports graphiques ne réimplémentent donc aucun dessin : ils rendent **les mêmes composants React que l'écran** (`PageContent`, `render/PageView`) hors du document, avec `renderToStaticMarkup`, puis convertissent ce SVG dans le format voulu (décision du 2026-11-05, lot 26). Les étiquettes de câbles suivent le même schéma avec leur propre composant, `LabelSheet`. Le tableur, lui, ne dessine rien : il part de tableaux purs (`export/tables`) qui alimentent indifféremment XLSX et CSV (lot 28).



Les modules se partagent en trois familles, ce qui guide les tests :

Purs (testés sous Node)	`export/csv`, `export/encoding`, `export/fileNames`, `export/fonts`, `export/labels`, `export/tables`, et les parties calculatoires de `export/png` (taille, bloc `pHYs`, CRC), `export/pdfImages` (taille, transparence) et `export/pdfExport` (sélection des pages, styles de police)
Rendu React (Node ou jsdom)	`export/svgExport`, `export/labelsExport`, `export/imageExport`
Navigateur uniquement	`rasterizeSvg` et `convertImage` (`<canvas>`, `Image`), `drawingsToPdf` (DOM attaché, lu par svg2pdf.js)

Les bibliothèques lourdes (jsPDF, svg2pdf.js, ExcelJS) sont chargées **à la demande** par `import()` dynamique : elles forment des modules à part dans le build et ne pèsent pas sur le démarrage. Leurs versions sont épinglées (jsPDF 4.2.1, svg2pdf.js 2.9.0, ExcelJS 4.4.0, licences MIT ; lots 27 et 28).

11.2 Contrat commun

11.2.1 Fichier exporté et refus

Toute fonction « build » rend un `ExportedFile` : octets, nom proposé, type MIME. Un export impossible pour une raison que l'utilisateur peut corriger (page supprimée, image trop grande, gabarit d'étiquettes trop grand, aucune page cochée, aucune liaison) lève une `ExportError` dont le message, traduit, est affiché tel quel par la fenêtre. Les autres exceptions (canevas indisponible, police introuvable) sont des erreurs internes : la fenêtre les affiche aussi, sous le titre « Export impossible ».

11.2.2 Noms de fichiers

EXP-01 fixe la forme `<projet>_<indice>.pdf`. `exportBaseName` prend le nom du projet et l'indice de la **dernière** révision du cartouche, tous deux passés par `safeFileName` (caractères `\ / : * ? " < > |` remplacés par un tiret, blancs réduits) ; sans révision, le nom seul ; sans nom, le nom par défaut de `fr.json`. `exportFileName` ajoute un suffixe facultatif et l'extension :

Export	Nom proposé (projet « Salle A », indice B)
PDF des pages	`Salle A_B.pdf`
SVG / PNG d'une page	`Salle A_B_page-2.svg`
SVG / PNG d'une face d'armoire	`Salle A_B_Baie 1-avant.png`
SVG / PNG d'une vue extérieure	`Salle A_B_<nom de l'équipement>.png`
XLSX	`Salle A_B.xlsx`
CSV d'une liste	`Salle A_B_Carnet de câblage.csv`
CSV de plusieurs listes	`Salle A_B_csv.zip`
Étiquettes	`Salle A_B_etiquettes.pdf` / `.csv`

Les suffixes (`page-{{number}}`, `avant`, `arrière`, `etiquettes`) viennent de `fr.json` (clés `export.*` et `labels.fileSuffix`).

11.2.3 Encodages

`export/encoding` regroupe les conversions de bas niveau : `toBase64` (découpe par blocs de 32 768 octets pour ne pas dépasser le nombre d'arguments de `String.fromCharCode` sur une grande image), `dataUrl` (images et polices intégrées au SVG), `decodeXmlText` (texte d'un `<text>` tel que l'a sérialisé React, pour savoir quels caractères sont dessinés). Les fichiers texte (SVG, CSV) sont encodés en UTF-8 par `TextEncoder`.

11.3 Dessin d'une page ou d'une vue (SVG)

11.3.1 Rendu hors du document

`pageSvg` crée pour l'occasion un éditeur (`createEditor`) sur le projet courant, l'enveloppe dans les mêmes contextes que l'écran (i18n, éditeur, mesure du texte, résolution des images) et rend `PageContent` avec :

- `pixelsPerMm = 100` : un zoom bien au-dessus du seuil de simplification des ports (1,6 px/mm, lot 36), si bien que les exports gardent toujours le dessin détaillé des ports ;
- `sizeInMm` : l'élément `<svg>` porte sa largeur et sa hauteur en millimètres (format papier réel) ;
- `interactive=false` : les poignées invisibles de l'écran (traits larges pour le clic) ne sont pas dessinées ; le calque de sélection n'existe pas hors de l'écran ;
- les « parties » de la page (`PageParts`, EXP-05) : fond, cadre, cartouche avec tableau des révisions, légende, repères de câbles (renvois et étiquettes du mode repères), grille de 10 mm (traits forts tous les 50 mm, export seulement). Sans cartouche, une légende placée dans le cartouche est dessinée à sa place en bloc libre ;
- les types de liaisons masqués (LIA-10), comme à l'écran.

Le résultat est un `SvgDrawing` : le balisage `<svg>` **sans polices** et la taille en millimètres. Les images de façades sont, par défaut, des URL `data:` construites une seule fois par image (`dataUrl`) ; le PDF passe sa propre fonction (`RenderOptions`).

Le routage des câbles (libavoid) doit être chargé avant le rendu : les fonctions « build » appellent `loadAvoid()` et ignorent son échec, auquel cas les câbles hors baie sont tracés en lignes droites, comme à l'écran dans la même situation.

11.3.2 Export d'une vue

Une vue (face d'armoire ou vue extérieure) s'exporte seule, recadrée (décision du lot 26). `viewOnlyPage` réduit la page à cette vue (et retire les éléments libres du « Schéma », qui appartiennent à la page, PAG-07) : le routage de cette page réduite recalcule les câbles internes à la vue et transforme en renvois ceux qui partaient vers d'autres vues. `viewExportBox` calcule le cadrage :

1. rectangle de la vue (`viewBounds`) ; sans rectangle, pas d'export ;
2. vue extérieure : rectangle de l'étiquette sous l'équipement (15 mm de chaque côté, deux lignes de texte) ; face d'armoire : bande du titre au-dessus si le titre est affiché, colonne des étiquettes à droite si elles sont à côté ;
3. routage de la page réduite (`routePage`) : tous les points des tracés de câbles et de faisceaux ;
4. étiquettes de faisceaux et repères de câbles (ROU-10, ROU-11) : un carré centré sur leur position, de demi-côté « longueur du texte + taille », qui les contient quel que soit leur angle ;
5. si les repères de câbles sont exportés : rectangle de chaque renvoi (aligné à gauche ou à droite de son point) et de chaque marqueur vertical ;
6. union de ces rectangles, élargie de `VIEW_EXPORT_MARGIN` (3 mm) de chaque côté, arrondie au centième de millimètre.

`viewSvg` rend alors la page réduite recadrée sur ce rectangle, en forçant à faux le cadre, le cartouche, la légende et la grille, qui n'ont pas de sens pour une vue isolée. La fenêtre d'export image appelle aussi `viewExportBox` pour afficher la taille en millimètres et en pixels avant l'export.

11.3.3 Polices intégrées

Un SVG exporté doit s'afficher à l'identique sur un poste qui n'a pas Inter (EXP-02, NF-07b). `svgFile` insère au début de l'élément `<svg>` une feuille de style de règles `@font-face` en URL `data:`, construite par `embeddedFontCss` :

1. `textUsage` parcourt les éléments `<text>` du balisage (expression régulière), lit leur `font-weight` (400 par défaut), retire les balises internes (`<tspan>`), décode les entités et range les points de code par graisse ;
2. pour chaque graisse, chaque fichier WOFF2 de l'écran (sous-ensembles Fontsource par écriture : latin, latin étendu, cyrillique…) qui couvre au moins un caractère utilisé est intégré tel quel, avec son `unicode-range` ;
3. les symboles absents de ces sous-ensembles (flèches des renvois, signes mathématiques, formes) sont pris dans Inter TrueType, dont un sous-ensemble ne garde que les glyphes utilisés (`subsetFont` de `text/sfnt`, tables de mise en forme retirées) ; intégrer le fichier entier coûterait environ 450 ko par graisse.

Les sources (URL des fichiers, plages, chargement) sont injectées par un `FontSources` : `APP_FONT_SOURCES` pour l'application, des sources factices dans les tests. Le texte reste du texte (sélectionnable, cherchable) : la conversion en tracés, que permet aussi EXP-02, n'est pas implémentée (alternative écartée au lot 26).

11.4 PNG

Le PNG est le SVG complet (polices et images intégrées) dessiné dans un `<canvas>` (lot 26). `buildImageExport` enchaîne :

1. dessin de la page ou de la vue (`exportDrawing`) ; le fond blanc de la page n'est dessiné que pour un SVG ou un PNG à fond blanc : un PNG « transparent » n'a pas de feuille blanche ;
2. fichier SVG autonome (`svgFile`) ;
3. taille en pixels (`pngPixelSize`) : soit une résolution en ppp, soit une largeur en pixels dont on déduit la résolution (ppp $=$ pixels $\times\ 25{,}4 /$ largeur en mm) ; hauteur et largeur arrondies, au moins 1 px. Au-delà de 16 384 px de côté ou de $16\,384^2$ pixels de surface (limites des navigateurs), l'export est refusé (« Image trop grande ») ;
4. `rasterizeSvg` : le SVG devient une URL `blob:` chargée dans une `Image`, dessinée sur un canevas à la taille voulue (après un fond blanc si demandé), encodée par `toBlob` ;
5. la résolution est écrite dans le fichier (`withPngDpi`) : un bloc `pHYs` (pixels par mètre, CRC-32 calculé par `crc32`) inséré juste après `IHDR`, les éventuels `pHYs` existants retirés. Les logiciels de mise en page impriment ainsi l'image à la bonne taille.

11.5 PDF

11.5.1 Chaîne de construction

Le PDF est généré par l'application (EXP-01), sans boîte d'impression : le raccourci `Ctrl+P` ouvre la fenêtre d'export PDF (lot 27). `buildPdfExport` :

1. garde les pages cochées **dans l'ordre du projet** (`pagesToExport`) ; aucune page : refus ;
2. charge libavoid, puis dessine chaque page (`pageSvg`) avec son fond blanc ; les images y sont remplacées par un repère `urn:portolan-image:<ref>` ;
3. relève les images utilisées (`imagePlaceholders`) et les convertit **une seule fois pour tout le fichier** ;
4. remplace chaque repère par l'URL `data:` convertie ;
5. assemble le PDF avec `drawingsToPdf`.

11.5.2 Assemblage par jsPDF et svg2pdf.js

`drawingsToPdf` est la fonction commune aux pages et aux planches d'étiquettes :

1. chargement à la demande de jsPDF et svg2pdf.js ;
2. document en millimètres, compressé, au format de la première page (orientation paysage si la largeur dépasse la hauteur) ; titre = nom du projet, créateur = nom de l'application ;

3. polices : les graisses utilisées dans l'ensemble des dessins (`textUsage`) sont ajoutées une à une, depuis Inter TrueType (le même fichier que les symboles de l'écran), avec l'encodage `Identity-H`. jsPDF n'intègre alors que les glyphes dessinés et ajoute la table de correspondance Unicode : le texte reste sélectionnable et extractible, accents et flèches compris ;
4. chaque dessin est analysé (`DOMParser`), **attaché au document** dans un hôte invisible hors de la vue (svg2pdf.js lit les styles calculés), converti par `svg2pdf` sur toute la page, puis détaché ; chaque page après la première est ajoutée à son propre format (`addPage([l, h])`) : A4 et A3 se mélangent ;
5. le document est rendu en `Uint8Array`.

Piège consigné au lot 27 : jsPDF combine lui-même style et graisse. `addFont(fichier, 'Inter', 'normal', 500)` enregistre le style `500normal`, que svg2pdf.js cherche pour un texte de graisse 500 ; 400 et 700 deviennent `normal` et `bold`. `pdfFontStyle` reproduit ce nom, et `drawingsToPdf` vérifie après chaque ajout que le style est bien enregistré (sinon erreur « Font … not registered ») : une montée de version de jsPDF qui changerait cette règle se voit immédiatement.

Contraintes du dessin imposées par svg2pdf.js, à respecter dans `src/render/` : pas de `dominant-baseline` (lignes de texte placées par leur ligne de base), pas de `paint-order` (le liseré blanc des étiquettes du mode repères est une copie du texte tracée en blanc dessous), pas de `<symbol>`/`<use>` pour les formes de ports. Les arcs des ponts de câbles (ROU-12) sont convertis en courbes de Bézier par svg2pdf.js ; `src/export/pdfBridges.test.ts` le vérifie.

11.5.3 Images

`convertImage` décode l'image du projet dans le navigateur et la redessine sur un canevas :

- taille naturelle réduite à `MAX_IMAGE_SIDE` (2 400 px) au plus grand côté, jamais agrandie (`rasterSize`) ; une image vectorielle est d'abord dessinée sur `SVG_RASTER_WIDTH` (1 600 px) de large, à ses proportions ;
- PNG si un pixel n'est pas totalement opaque (`hasTransparency`), JPEG sinon, à la qualité choisie dans la fenêtre (60, 75, 85 ou 95 %, 85 % par défaut).

Les images vectorielles ne sont pas passées telles quelles à svg2pdf.js, qui ne les interprète que partiellement (alternative écartée au lot 27).

11.5.4 Injection des dépendances

`PdfDependencies` regroupe les deux accès au navigateur que les tests remplacent : le chargement d'une graisse d'Inter et la conversion d'une image. `src/export/pdfBuild.test.ts` construit ainsi un vrai PDF sous jsdom (avec une `getBBox` simulée), et les tests de bout en bout (`tests/e2e/pdf.spec.ts`) le relisent avec pdf.js : formats des pages, texte extractible, rendu comparé à l'écran (tolérance d'un pixel de décalage).

11.6 Tableur

11.6.1 Les cinq listes

`export/tables` construit des `Table` purs : identifiant, titre traduit, colonnes (`Column` : identifiant stable et libellé traduit, clé `tables.columns.<liste>.<colonne>`), lignes indexées par identifiant de colonne, cellules texte, nombre ou vide. Les listes d'EXP-04 :

Liste	Contenu et ordre
`inventory`	un équipement par ligne : nom, nature d'équipement, fabricant, machine, référence, armoire, U le plus bas, hauteur, face, description, puis une colonne par clé de champ libre (EQP-07, colonnes `field:<clé>`). Ordre (`orderedDevices`) : armoires dans l'ordre du projet, du haut vers le bas, puis les 0U par emplacement, puis les extérieurs ; à égalité, par nom
`cabling`	carnet de câblage : une ligne par liaison, triée par repère (ordre naturel : C2 avant C10) ; repère, type, extrémités A et B (armoire, U, équipement, port), **équipements et ports finaux** A et B, longueur en mètres, catégorie, référence fournisseur, couleur de gaine, notes
`occupation`	une ligne par U de chaque armoire, du haut vers le bas : équipements avant et arrière, Libre/Occupé
`ports`	une ligne par port de chaque équipement (ordre de l'inventaire) : nom affiché, face, connecteur, état Libre/Occupé/Désactivé, repères des câbles
`cableNeeds`	liaisons regroupées par type, catégorie et longueur, avec leur quantité

11.6.2 Extrémités finales du carnet

Les colonnes « équipement final / port final » (EXP-04, LIA-09) suivent la continuité à travers les panneaux de brassage, sans limite de nombre :

1. `endToEndPath` (`model/links`) donne le chemin complet de la liaison : ses sauts et ses deux extrémités finales `endA`, `endB` ;
2. sans chemin (boucle, port introuvable) ou sans panneau, les extrémités finales sont les extrémités directes de la liaison ;
3. le chemin est orienté depuis l'une de ses extrémités, pas forcément depuis le côté A de la liaison : si le saut de cette liaison part de son extrémité B, les deux extrémités finales sont échangées, pour que « final A » soit toujours du côté de l'extrémité A.

Le montage de recette REC-16 (serveur → panneau A → liaison de rocade → panneau B → commutateur) est vérifié depuis chacune de ses liaisons dans `src/export/spreadsheet.test.ts`.

Le nom d'un port dans les listes (`portCellName`) est son nom affiché, suivi de sa face lorsque l'autre face de l'équipement a un port du même nom (panneaux de brassage : port 1 avant et port 1 arrière). Sans cette précision, le carnet ne pourrait pas être réimporté sans ambiguïté (IMP-03).

11.6.3 Formats XLSX et CSV

`buildSpreadsheetExport` construit les listes demandées, ne garde que les colonnes cochées (`withColumns`, EXP-05), puis :

- **XLSX** (`toXlsx`, ExcelJS) : un onglet par liste (nom nettoyé et limité à 31 caractères, `sheetName`), ligne d'en-tête en gras, figée et filtrable, largeur de colonne selon le contenu (8 à 50 caractères), nombres écrits comme nombres ;
- **CSV** (`toCsv` sur `export/csv`) : UTF-8 avec BOM (sans lui, un tableur sous Windows lit le fichier en ANSI et casse les accents), fins de ligne CRLF, cellules entre guillemets si elles contiennent le séparateur, un guillemet ou un saut de ligne (guillemets doublés). Avec le séparateur « ; », les nombres prennent la virgule décimale (tableur en français) ; avec « , », le point. Une liste seule est téléchargée directement ; plusieurs listes forment **une archive ZIP** (fflate), car les navigateurs bloquent les téléchargements multiples.

Le module d'import réutilise ces briques : le modèle de classeur vide (IMP-01) est écrit par `toXlsx` et `csvCells`, et la reconnaissance des colonnes à l'import repose sur les mêmes libellés `tables.columns.inventory.*` et `tables.columns.cabling.*` de `fr.json`.

11.7 Étiquettes de câbles

EXP-07 demande, pour une sélection de liaisons ou toutes, une planche PDF A4 ou A3 et un CSV pour imprimante d'étiquettes (lot 29).

11.7.1 Contenu

`cableLabels` produit **une étiquette par extrémité** : A puis B de chaque liaison, dans l'ordre donné (la fenêtre passe la sélection, ou toutes les liaisons triées par repère). Chaque étiquette porte le repère, l'extrémité où elle se colle (« ici », équipement · port) et l'autre extrémité. Avec l'option « équipement final », et seulement si la liaison traverse au moins un panneau de brassage (chemin de plus d'un saut), une ligne « Final : ici ↔ là-bas » est ajoutée, orientée pour chaque extrémité comme dans le carnet. Le texte d'une étiquette (`labelLines`) : repère en gras à la taille s, extrémité locale en demi-gras à $0{,}72\,s$, « → autre extrémité » à $0{,}72\,s$, ligne finale à $0{,}62\,s$. « ↔ » a été préféré à la double flèche opposée (U+21C4), absente d'Inter.

11.7.2 Gabarit et planches

Le `LabelTemplate` décrit une feuille en portrait : format, taille d'une étiquette, colonnes, rangées, marges haute et gauche, espaces entre étiquettes ; par défaut la planche courante de 3 × 8 étiquettes de 63,5 × 33,9 mm (`DEFAULT_LABEL_TEMPLATE`). `templateProblem` refuse un gabarit incomplet ou qui dépasse de la feuille (la fenêtre désactive alors le bouton PDF ; le CSV reste possible). `labelSheets` remplit les emplacements ligne par ligne depuis le coin haut gauche, planche après planche.

11.7.3 Ajustement du texte

`fitLabel` cherche la plus grande taille qui tient :

1. zone utile = étiquette moins `LABEL_PADDING` (1,8 mm) de chaque côté ;
2. taille de départ du repère : $\min(5\text{ mm}, \text{hauteur utile}/4)$;

3. à chaque taille, chaque ligne est découpée à la largeur utile par `layoutText` (`text/layout`, NF-07) et empilée avec un interligne de 1,22 fois sa taille ;
4. si la hauteur totale tient, ces lignes sont retenues ; sinon la taille baisse de 0,2 mm ;
5. à la plus petite taille (la suivante passerait sous 1,4 mm), les lignes qui dépassent le bas sont coupées.

11.7.4 Fichiers

Chaque planche est dessinée par le composant `LabelSheet` (`render/LabelSheet`), le même qui sert à l'aperçu de la fenêtre, puis convertie en PDF par la fonction commune `drawingsToPdf` (`buildLabelsPdf`) ; le contour gris clair des étiquettes est une option, pour l'impression sur papier ordinaire. Le CSV (`buildLabelsCsv`, `labelsCsv`) a une ligne par étiquette : repère, extrémité (A ou B), ici, autre extrémité, final ici, final autre extrémité.

11.8 Limites connues

- **Polices du SVG** : `textUsage` lit le balisage par expressions régulières ; il suppose des éléments `<text>` non imbriqués et une graisse portée par l'attribut `font-weight` de l'élément `<text>` (pas par un `<tspan>` ni par du CSS). Un texte dessiné autrement serait rendu dans une police de repli sur un poste sans Inter.
- **Conversion en tracés** : non implémentée (EXP-02 la cite comme alternative) ; les textes restent des textes avec polices intégrées.
- **PNG** : limité à 16 384 px de côté ; le rendu dépend du moteur du navigateur (lissage), contrairement au SVG.
- **PDF** : svg2pdf.js n'interprète qu'une partie de SVG/CSS (voir les contraintes ci-dessus) ; une nouveauté de dessin doit être vérifiée dans le PDF, pas seulement à l'écran. Les images vectorielles des façades deviennent des images matricielles.
- **Mémoire** : tous les dessins et toutes les images converties d'un PDF sont en mémoire en même temps ; un projet de nombreuses pages A3 très illustrées peut être lent.
- **Tableur** : les colonnes de champs libres n'apparaissent qu'avec des clés non vides ; deux équipements aux clés de casse différente donnent deux colonnes. L'attribut `numeric` des colonnes est informatif : XLSX et CSV décident d'après le type JavaScript de la cellule.
- **Étiquettes** : à la plus petite taille, le texte qui ne tient pas est coupé sans signalement.

11.9 Faire évoluer les exports

11.9.1 Ajouter une colonne à une liste

1. Dans la fonction de la liste (`src/export/tables.ts`), ajouter `column('<liste>', '<id>', numérique?)` à la place voulue et la valeur dans chaque ligne (`null` pour une cellule vide, un nombre pour une valeur numérique) ;
2. ajouter le libellé `tables.columns.<liste>.<id>` dans `src/i18n/fr.json` ;

3. rien à faire pour XLSX, CSV ni pour la fenêtre : elle lit les colonnes de `buildTable` et propose la case à cocher ;
4. pour `inventory` ou `cabling`, vérifier l'import (`src/import/rules.ts`) : une colonne que l'import ne reconnaît pas y devient un champ libre de l'équipement (inventaire) ou un avertissement (carnet) ; la déclarer dans `DEVICE_COLUMNS` ou `LINK_COLUMNS` (`'ignored'` si elle ne se réimporte pas, comme les extrémités finales), et garder les mêmes en-têtes dans le modèle de classeur (IMP-01) ;
5. mettre à jour `src/export/spreadsheet.test.ts` (liste des colonnes attendues) et, si l'utilisateur doit en être informé, le chapitre du manuel qui décrit `\commande{export.spreadsheet}`.

11.9.2 Ajouter une liste

Ajouter l'identifiant à `TableId` et à `TABLE_IDS` (ordre des onglets et de la fenêtre), écrire sa fonction et l'inscrire dans `BUILDERS`, puis ajouter `tables.titles.<id>` et ses libellés de colonnes. Le titre sert de nom d'onglet (31 caractères au plus) et de nom de fichier CSV.

11.9.3 Ajouter une option d'export d'image ou de PDF

Une partie de la page à inclure ou non (EXP-05) est un champ de `PageParts` (`src/render/PageView.tsx`) : l'ajouter avec sa valeur d'écran dans `SCREEN_PARTS`, la respecter dans le rendu, l'exposer dans `ExportPartsFields` (`src/ui/exports/`), qui sert aux fenêtres SVG/PNG et PDF. Si la partie ne concerne pas une vue isolée, la forcer à faux dans `viewSvg` ; si elle déborde du rectangle de la vue, l'ajouter à `viewExportBox`. Une option propre au format (comme la qualité JPEG) se place dans la requête (`PdfExportRequest`, `ImageExportRequest`). Tests à compléter : `src/export/svgExport.test.tsx` (partie absente du SVG quand elle est décochée), `tests/e2e/exports.spec.ts` et `tests/e2e/pdf.spec.ts`.

11.9.4 Ajouter une graisse ou un caractère

Les graisses viennent de `FONT_WEIGHTS` (`text/fonts`) et les fichiers de `text/fontFiles` ; l'export SVG et le PDF les suivent sans modification, à condition que les fichiers WOFF2 et TrueType de la graisse soient importés. Un nouveau symbole est couvert automatiquement s'il existe dans Inter TrueType ; sinon il s'affichera dans une police de repli à l'écran et manquera dans le PDF : choisir un caractère présent dans Inter (comme « ↔ » au lot 29).

11.9.5 Ajouter un format

Écrire une fonction « build » qui rend un `ExportedFile` et lève une `ExportError` pour un refus explicable ; partir d'un `SvgDrawing` s'il s'agit d'un dessin ; charger toute bibliothèque lourde par `import()` ; nommer le fichier avec `exportFileName`. Déclarer la commande dans `src/commands/export.ts`, ajouter sa fenêtre dans `src/ui/exports/`, ses textes dans `fr.json` et sa description dans le manuel (`check:doc`). Aucune ressource ne doit être chargée depuis le réseau (CG-02).

11.9.6 Tests

Fichier	Ce qu'il garantit
`src/export/svgExport.test.tsx`	page au format papier, parties décochées, cadrage d'une vue, images et polices intégrées, noms de fichiers, refus
`src/export/png.test.ts`	taille en pixels, bloc `pHYs`, CRC-32
`src/export/pdfExport.test.ts`	ordre des pages, styles de police, images
`src/export/pdfBuild.test.ts`	PDF réel sous jsdom : une page par page, formats
`src/export/pdfBridges.test.ts`	arcs des ponts convertis en courbes
`src/export/spreadsheet.test.ts`	contenu des cinq listes, REC-16, CSV, XLSX relu
`src/export/labels.test.ts`	étiquettes, gabarit, ajustement, CSV, planches
`tests/e2e/exports.spec.ts`	SVG identique à l'écran au pixel près, PNG
`tests/e2e/pdf.spec.ts`	PDF relu par pdf.js, conforme à l'écran
`tests/e2e/labels.spec.ts`	planche A4, gabarit refusé, CSV

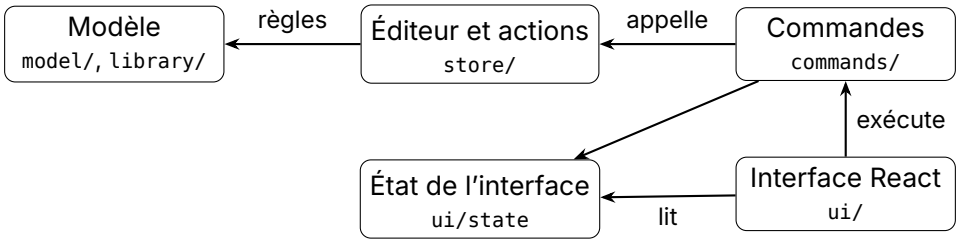
Chapitre 12

État, actions et commandes

Ce chapitre décrit comment le projet ouvert est conservé en mémoire, comment il est modifié (les *actions*), comment chaque modification devient annulable, et comment l'interface déclenche ces modifications au travers du *registre des commandes* (DOC-09). Il couvre les dossiers `src/store/` et `src/commands/` ; la référence détaillée de chaque déclaration se trouve au chapitre « Référence : état et commandes ».

12.1 Vue d'ensemble

Le code se répartit en quatre couches, de la plus pure à la plus proche de l'écran :



- Le **modèle** (`src/model/`, `src/library/`) contient les types du projet et les règles métier sous forme de fonctions pures : « ce placement est-il possible ? », « quel type de liaison proposer ? ». Il ne modifie rien.
- L'**éditeur** (`store/editor`) détient le projet courant dans un store Zustand « vanilla » et l'historique d'annulation. Les **actions** (`src/store/*Actions.ts`) vérifient les règles du modèle puis modifient le projet par `editor.apply`. Ni l'éditeur ni les actions ne dépendent de React (règle 6 de `CLAUDE.md`) : ils sont testés unitairement.
- Les **commandes** (`src/commands/`) sont les gestes de l'utilisateur : un identifiant stable, un libellé, un raccourci, une condition d'activation et une fonction `run`, qui appelle des actions ou ouvre un dialogue.
- L'**interface** (`src/ui/`) lit l'état par des sélecteurs, affiche les menus, la barre d'outils et le menu contextuel à partir du registre, et exécute les commandes au clic ou au clavier. L'état propre à l'interface (page active, sélection, zoom, dialogue ouvert, presse-papiers) est un second store Zustand, `ui/state`, hors historique et hors fichier (décision du 2026-11-04, lot 13).

Le sens des dépendances est strict : `store/` n'importe rien de `ui/` ni de `commands/` ; `commands/` importe `store/` et quelques modules d'interface non React (flux de confirmation, insertion d'élément) ; `ui/` importe tout.

12.2 L'éditeur et l'historique

12.2.1 État

L'éditeur est créé par `createEditor` (un par application, plus des éditeurs temporaires dans les tests, le projet d'exemple ou les simulations). Son état, `EditorState`, contient :

- `project` : le projet, objet immuable (Immer), jamais modifié en place ;
- `images` : les octets des images par référence d'empreinte. Contenu immuable, adressé par son empreinte : il est *hors historique* (`addImages` n'est pas annulable ; les images qui ne servent plus sont retirées à l'enregistrement du fichier) ;
- `past` et `future` : les entrées d'historique (`HistoryEntry`), chacune portant les *patches* Immer de la modification et leurs inverses ;
- `savedEntryId` : l'identifiant de l'entrée au sommet de `past` au moment du dernier enregistrement (0 pour l'état initial, `null` pour un brouillon restauré qui contient des modifications non enregistrées).

12.2.2 Modifier le projet : `apply`

Toute modification du projet passe par `editor.apply(label, recette, options)` (décision du 2026-11-04, lot 05). L'algorithme :

1. Immer `produceWithPatches` exécute la recette sur un brouillon du projet, puis appelle `followRemovedViews` : si la recette a retiré des vues, les points des tracés manuels qui leur étaient rattachés sont rattachés à la vue restante la plus proche (ROU-16), *dans la même action*. Aucune action n'a donc à s'en occuper.
2. Si la recette n'a rien changé (aucun patch), `apply` renvoie `false` et rien n'est enregistré : une entrée d'historique vide n'existe jamais.
3. Le nouveau projet est publié dans le store (les abonnés sont notifiés).
4. Si une transaction est ouverte, les patches s'ajoutent à elle (les inverses en tête, pour être rejoués dans l'ordre inverse). Sinon une entrée est empilée :
    - si `options.coalesceKey` est donnée, que l'entrée au sommet porte la même clé et que `future` est vide, les deux entrées fusionnent (patches concaténés, inverses dans l'ordre inverse) et l'entrée fusionnée reçoit un nouvel identifiant ; c'est ainsi qu'un glisser de cent mouvements ne fait qu'une étape d'annulation ;
    - sinon l'entrée est ajoutée, `future` est vidée et `past` est tronquée aux `HISTORY_LIMIT` (100) dernières entrées (PRJ-07).

Le `label` est une clé i18n de la section `history` de `fr.json` (`history.placeDevice`…). Il décrit l'action ; il n'est aujourd'hui pas affiché par l'interface, mais chaque action en fournit un.

12.2.3 Transactions

`editor.transaction(label, fn)` exécute `fn` ; tous les `apply` faits pendant `fn` forment *une seule* entrée d'historique, sous le libellé de la transaction. Une transaction imbriquée fait partie de la transaction englobante. Si `fn` lève une exception, les modifications déjà appliquées restent dans le projet et sont tout de même enregistrées comme une entrée (bloc `finally`) : le projet et l'historique restent cohérents. Exemples d'usage : `addRackWithViews` (armoire, puis vues calculées sur le projet qui contient l'armoire) ou `deleteSelection` (vues, éléments, liaisons, armoires et équipements supprimés en une étape).

12.2.4 Annuler, rétablir, état « modifié »

`undo` applique les patches inverses de l'entrée au sommet de `past` et la déplace en tête de `future` ; `redo` fait l'inverse. Les deux renvoient `false` s'il n'y a rien à faire. L'indicateur « modifié » (PRJ-04) est calculé par `isDirty` : le projet est modifié quand l'identifiant de l'entrée au sommet de `past` diffère de `savedEntryId`. Comme chaque entrée a un identifiant unique (même après fusion), annuler jusqu'à l'état enregistré fait disparaître l'indicateur, et refaire une modification différente le fait revenir.

`load` remplace le projet (nouveau fichier, ouverture, projet d'exemple) : il vide l'historique, abandonne une transaction ouverte et marque le projet enregistré, sauf avec `{ dirty: true }` (brouillon restauré). `markSaved(entryId)` est appelé par la session à la fin d'un enregistrement : l'identifiant passé est celui relevé *avant* l'écriture (`currentEntryId()`), de sorte qu'une modification faite pendant l'écriture asynchrone laisse le projet « modifié ».

12.2.5 Sélecteurs et React

React lit l'éditeur par `useEditorState` avec un sélecteur, et obtient l'éditeur par `useEditor` (fourni par `EditorContext`). Les sélecteurs de `store/selectors` par identifiant (`selectRack`, `selectDevice`…) s'appuient sur l'index en cache du projet (`model/lookup`) et sur le partage structurel d'Immer : une entité garde la même référence tant qu'elle n'est pas modifiée, donc un composant abonné à une entité ne se redessine que lorsqu'elle change (NF-02b). Un sélecteur paramétré (`selectRack(id)`) crée une fonction à chaque appel : dans un composant, le mémoïser (ou l'appeler dans le corps, ce que Zustand tolère puisque seule la valeur renvoyée est comparée).

12.3 Les actions

12.3.1 Convention

Une action est une fonction `(editor, …arguments) => ActionResult`. Elle lit le projet courant, vérifie les règles métier avec les fonctions du modèle, puis appelle `editor.apply` une seule fois (ou plusieurs fois dans une transaction). Elle renvoie un `ActionResult` :

- `{ ok: true, id?, warnings }` : l'action est faite ; `id` est l'entité créée ou modifiée ; `warnings` contient des avertissements à afficher (débordement de profondeur accepté, liaison incompatible forcée, équipement sans place lors d'un collage…) ;
- `{ ok: false, messages, forceable }` : l'action est refusée et le projet n'a pas changé ; `messages` explique pourquoi (déjà traduit) ; `forceable` indique que l'utilisateur peut « Forcer » (EQP-02, LIA-06) : le même appel avec `forced: true` passera.

Une action ne lève pas d'exception pour un refus métier : les exceptions sont réservées aux erreurs de programmation. Plusieurs actions de modification simples (`updateDevice`, `removeLinks`…) renvoient `ok: true` même si l'entité n'existe pas : l'appel ne fait alors rien. Le flux d'interface `placeWithConfirmation` transforme un refus « forçable » en fenêtre de confirmation puis rappelle l'action avec `forced`.

12.3.2 Répartition par domaine

Module	Contenu
`store/actions`	Socle : propriétés du projet, armoires, placement et déplacement d'équipements, liaisons ; suppression en cascade.
`store/deviceActions`	Placement avec nom automatique, équipements extérieurs avec leur vue, collage, déplacement par U, retournement (EQP-09).
`store/rackActions`	Armoire avec ses vues, vues d'une armoire, page d'une armoire (PAG-13), duplication (ARM-01).
`store/pageActions`	Pages et vues (PAG-01, PAG-03, EXT-04).
`store/annotationActions`	Éléments du « Schéma » : ajout, modification, collage, déplacement groupé, alignement et répartition (PAG-07, PAG-08).
`store/projectActions`	Révisions (PRJ-11) et cartouche (PAG-10).
`store/libraryActions`	Bibliothèque du projet : modèles, types de connecteurs et de liaisons, purge, réimportation, import (BIB-03 à BIB-14).
`store/machineActions`	Machines et paramètres d'équipement : création, nouveaux paramètres, variantes, éditeur de machine (EQT-02 à EQT-06).
`store/portActions`	Ports ajoutés à un équipement placé (PRT-04).
`store/routeActions`	Tracés manuels (ROU-06, ROU-07, ROU-16), faisceaux (ROU-11).
`store/routingActions`	Goulotte de chaque type de liaison (ROU-02).
`store/importActions`	Import d'un tableur, tout ou rien (IMP-05).
`store/exampleProject`	Construction du projet d'exemple (ERG-08).

12.3.3 Invariants de suppression

Les suppressions sont en cascade et passent par les fonctions internes de `store/actions` :

- supprimer des liaisons (`dropLinks`) retire aussi leurs tracés manuels de toutes les pages ;
- supprimer des équipements retire leurs liaisons (et donc leurs tracés) et les vues des équipements extérieurs ;
- supprimer une armoire (`removeRack`) retire ses équipements, comme ci-dessus, et toutes les vues de l'armoire ; les points de tracés rattachés à ces vues suivent la vue la plus proche (`followRemovedViews`, appelé par l'éditeur) ;
- supprimer une vue d'armoire ne supprime pas l'armoire ; supprimer la vue d'un équipement extérieur depuis l'interface supprime l'équipement (`deletionPlan`), car la vue le représente ;
- un projet garde toujours au moins une page (`removePage`) et au moins un type de liaison (`removeLinkType`) ; une machine générique n'est jamais supprimée (EQT-02).

12.3.4 Actions en deux temps : calcul, puis application

Les modifications de la bibliothèque sont calculées par des fonctions pures qui renvoient le projet *tel qu'il serait* (`MachineChange` de `library/equipment/change`, `ImportPlan`

de `model/libraryTransfer`, plan d'import de `import/rules`). L'interface affiche ces conséquences (équipements qui ne tiendraient plus, liaisons orphelines) avant que l'utilisateur ne confirme ; l'action recopie ensuite dans le brouillon seulement ce qui change. Exemples : `machineParamsChange` puis `applyMachineParams` ; `machineSaveChange` puis `saveMachine`.

12.3.5 Ajouter une action

1. Choisir le module du domaine (tableau ci-dessus) ; écrire la règle métier dans `src/model/` si elle n'existe pas (fonction pure, testée).
2. Écrire l'action : lire `editor.getState().project`, vérifier, refuser avec des messages traduits (`t(...)`), puis un seul `editor.apply` avec une clé `history.<nom>` ajoutée à `src/i18n/fr.json`. Pour un geste continu (glisser), accepter `{ coalesceKey }` et le passer à `apply`.
3. Si l'action doit être visible des tests de bout en bout, vérifier que son module est repris par `src/main.tsx` (`window.__portolan.actions`).
4. Tester dans `src/store/<module>.test.ts` : effet, refus, annulation (`editor.undo()` rend le projet d'avant), une seule entrée d'historique.

12.4 Le projet d'exemple

`createExampleProject` construit le projet d'exemple de l'écran d'accueil (ERG-08, décision du 2026-11-05, lot 43). Il est *construit par les actions*, comme le ferait un utilisateur, dans un éditeur temporaire : il respecte donc toujours les règles de placement et de liaison, et casse au test s'il ne les respecte plus (une action refusée lève une exception « example project »). Il ne contient que des machines génériques de la bibliothèque par défaut (CG-08) :

1. propriétés du projet, une révision datée de `EXAMPLE_REVISION_DATE` (date fixe, pour un document identique à chaque fois), cartouche sans logo et deux champs personnalisés ;
2. une armoire de 18U sur roulettes, garnie de haut en bas (panneau de brassage, passe-câbles, commutateur, étagère, deux serveurs, pare-feu, obturateur, bandeau de prises à l'arrière, onduleur), un écran et un portable extérieurs ;
3. une variante de l'étagère générique qui porte un portable (MAC-03, décision de l'utilisateur), dessiné par `EXAMPLE_LAPTOP_SVG`, avec un port réseau, enregistrée par `saveMachine` en mode `variant` ;
4. sept liaisons, puis une page pour l'armoire (`createRackPage`).

La fonction est asynchrone, comme le chargement d'un fichier, parce que l'empreinte SHA-256 de l'image du portable est calculée de façon asynchrone (`imageRefFor`). `npm run check:neutral` construit aussi ce projet et contrôle son JSON. Pour le modifier : changer la fonction, puis mettre à jour `src/store/exampleProject.test.ts`, l'e2e `tests/e2e/example.spec.ts` et ses captures, et la section « Le projet d'exemple » du manuel.

12.5 Le registre des commandes

12.5.1 Principe (DOC-09)

Chaque geste de l'interface (entrée de menu, bouton de barre d'outils, entrée de menu contextuel, raccourci) est une *commande* déclarée dans le registre `commands`. Le registre est la seule source : les menus, la barre d'outils, le menu contextuel, le dialogue des raccourcis et le contrôle de la documentation le lisent. Une commande est un objet `Command` :

Champ	Rôle
`id`	Identifiant stable, mots en minuscules séparés par des points (`file.save`, `view.zoomFit`) ; unique (test). Le préfixe désigne le domaine. C'est la clé du manuel (`\commande{id}`) et l'attribut `data-command` des boutons et menus, utilisé par les e2e.
`labelKey`	Clé i18n du libellé, présente dans `fr.json` (test).
`shortcut`	Raccourci principal, affiché dans les menus et infobulles.
`otherShortcuts`	Autres raccourcis acceptés (« Espace ou Ctrl+0 »), listés dans le dialogue des raccourcis (décision du 2026-11-05, lot 39).
`zone`	Zone de l'interface (`CommandZone`) : sert à grouper le dialogue des raccourcis et à situer la commande dans le manuel.
`allowInTextField`	Déclenchée aussi pendant la saisie dans un champ ou un dialogue (Enregistrer, Enregistrer sous, Ouvrir).
`isEnabled`	Condition d'activation ; absente = toujours active.
`isChecked`	Commande à bascule : coche affichée dans le menu.
`disabledHintKey`	Clé i18n expliquant pourquoi la commande est grisée, affichée sous l'entrée de menu.
`run`	L'exécution, éventuellement asynchrone.

Les commandes sont regroupées par domaine dans un fichier chacun (`fileCommands`, `editCommands`…), concaténés dans l'ordre de `commands`. Les fonctions partagées pour lire la sélection et la page active sont dans `commands/context`.

12.5.2 Le contexte d'exécution

Une commande reçoit un `CommandContext` : l'éditeur (toujours), la session de fichiers et l'état de l'interface (`ui`), tous deux absents dans les tests unitaires et les scripts. Une commande qui ouvre un dialogue ou lit la sélection teste donc `ui !== undefined` dans son `isEnabled` ; une commande qui ne fait qu'appeler des actions (`edit.undo`, `page.add`) fonctionne sans interface, ce qui la rend testable directement avec `executeCommand(id, { editor })`.

12.5.3 Activation

`isCommandEnabled` est la seule façon correcte de savoir si une commande est active. Elle ajoute à `isEnabled` le filtre du mode « bibliothèque » (BIB-13, décision du 2026-11-06) : quand le document ouvert est une bibliothèque (`project.editedLibrary` défini), seules les commandes de `LIBRARY_DOCUMENT_COMMANDS` et celles des préfixes `help.` et `app.` restent actives ; les autres sont grisées avec la raison `commands.libraryDocumentOnly` (`disabledHintKey`). Une nouvelle commande est donc grisée par défaut dans ce mode, ce qui est sûr ; l'ajouter à la liste si elle a un sens pour une bibliothèque.

`executeCommand` retrouve la commande, vérifie qu'elle est active et attend la fin de `run` ; elle renvoie `false` sans rien faire sinon. L'interface n'appelle jamais `command.run` directement.

12.5.4 Raccourcis

Un raccourci s'écrit « `Ctrl+Shift+S` » : modificateurs `Ctrl`, `Shift`, `Alt`, puis une touche. `Ctrl` accepte aussi `Cmd` (touche Méta). Les touches nommées sont `Plus`, `Minus`, `Delete`, `Escape`, `Space`, `F1`, `F2`, `Enter` et les flèches. `Plus` et `Minus` reconnaissent aussi « = » et « _ » et ignorent Maj, qui dépend de la disposition du clavier ; `Delete` reconnaît l'effacement arrière (`matchesShortcut`). L'affichage traduit les noms (« Ctrl+Maj+S », « Suppr », « Échap ») par `displayShortcut`.

Le crochet `useCommandShortcuts` écoute `keydown` sur la fenêtre :

1. la touche « appartient » au champ si le focus est dans un champ de saisie, si un dialogue modal est ouvert, ou si c'est Espace sur une case à cocher ou un bouton radio ;
2. `commandForKey` cherche la première commande dont un raccourci correspond ; quand la touche appartient au champ, seules les commandes `allowInTextField` sont candidates (Ctrl+Z dans un champ garde donc l'annulation native du champ, décision du 2026-11-04, lot 05) ;
3. une commande désactivée laisse la touche au navigateur (les flèches font défiler) ; sinon l'événement est annulé, la commande exécutée, et le relâchement de la même touche annulé aussi, pour qu'Espace n'active pas le bouton qui a le focus (décision du 2026-11-05, lot 39).

Les raccourcis sont uniques sur l'ensemble du registre (test de `src/commands/registry.test.ts`).

12.5.5 Liaison avec l'interface

Aucune liste de l'interface ne contient de libellé ni de raccourci : elles ne contiennent que des identifiants de commandes.

- **Menus** : `MENUS` liste, pour chaque menu de la barre, les identifiants dans l'ordre, « - » pour un séparateur. `menuEntries` saute les identifiants absents du registre et les séparateurs en trop ; `CommandList` affiche chaque entrée avec son libellé, son raccourci, sa coche (`isChecked`), son état grisé et sa raison (`disabledHintKey`), et l'exécute au clic.
- **Barre d'outils** : `TOOLBAR`, rendue par des `CommandButton` (libellé en infobulle avec le raccourci, bouton désactivé selon `isCommandEnabled`).
- **Menu contextuel** (ERG-04) : `contextMenuItems` choisit la liste selon la nature de ce qui est sélectionné (rien, liaison, port, équipement, élément, armoire, vue…) ; le menu réutilise `CommandList`.
- **Panneaux et fiches** : les boutons des panneaux (Bibliothèque, Schéma, fiche de bibliothèque) utilisent aussi `CommandButton` ou `useCommand`.
- **Dialogue des raccourcis** (Aide, F1) : généré depuis le registre, groupé par `zone`.

L'état activé d'une commande dépend du projet et de l'état de l'interface : `useStateVersion` abonne le composant aux deux stores pour que menus et boutons se mettent à jour. Les commandes lisent la sélection par `selection` et la page active par `activePage`.

12.5.6 La documentation des commandes : `\commande{id}`

Le manuel utilisateur (`manual/`) documente chaque commande dans la section qui la décrit, ouverte par la macro `\commande{id}` (ancre `cmd:<id>` et entrée d'index). `npm run check:doc` (`tools/check-doc.ts`) compare l'ensemble des identifiants du registre et l'ensemble des `\commande` des sources LaTeX : une commande non documentée, ou une section qui nomme une commande inexistante, fait échouer le contrôle, puis le manuel est compilé (règle 4 de `CLAUDE.md`). Renommer une commande impose donc de renommer sa section.

12.5.7 Ajouter une commande, de bout en bout

1. **Action** : si la commande modifie le projet, écrire ou réutiliser une action de `src/store/` (section précédente) ; la commande ne modifie jamais le projet autrement que par des actions ou `editor.apply`.
2. **Registre** : ajouter l'objet `Command` au tableau du domaine dans `src/commands/<domaine>.ts` (ou créer un fichier et l'ajouter à `commands` dans `registry.ts`). Choisir un `id` libre au format `domaine.nom`, la `zone`, un raccourci libre s'il en faut un, un `isEnabled` qui teste `ui` si `run` en a besoin.
3. **Textes** : ajouter le libellé (`commands.<domaine>.<nom>`) et, le cas échéant, la raison de désactivation dans `src/i18n/fr.json` ; aucun texte en dur.
4. **Mode bibliothèque** : ajouter l'identifiant à `LIBRARY_DOCUMENT_COMMANDS` si la commande a un sens quand une bibliothèque est ouverte.
5. **Interface** : placer l'identifiant dans `MENUS`, `TOOLBAR`, `contextMenuItems` ou un `CommandButton` de panneau.
6. **Manuel** : écrire la section dans le chapitre du manuel qui décrit la zone, avec `\commande{id}` ; `npm run check:doc` doit passer.
7. **Tests** : un test unitaire de la commande ou de son action (`src/commands/*.test.ts`, avec `executeCommand(id, { editor })` quand elle n'a pas besoin d'interface) ; un e2e dans `tests/e2e/` qui passe par le menu (aide `menu(page, 'Fichier', 'Libellé')`) ou par `[data-command="id"]`. Les tests du registre vérifient d'eux-mêmes l'unicité de l'identifiant et du raccourci et l'existence du libellé.

Exemple, la commande `help.technicalManual` de ce lot : déclarée dans `helpCommands`, elle ouvre `TECHNICAL_MANUAL_URL` (le PDF de cette documentation, livré avec l'application comme le manuel utilisateur et publié sur le site) dans un nouvel onglet ; libellé `commands.help.technicalManual`, entrée du menu Aide dans `MENUS`, active aussi en mode bibliothèque (préfixe `help.`).

12.6 Limites connues et conseils

- **Historique en mémoire seulement** : il n'est ni enregistré dans le fichier ni dans le brouillon ; ouvrir un fichier le vide.
- **Fusion** : deux gestes successifs qui utiliseraient la même `coalesceKey` fusionneraient ; les appelants construisent une clé unique par geste (identifiant de l'entité plus un compteur de glisser).

- **Libellés d'historique** : ils sont fournis par toutes les actions mais l'interface n'affiche pas encore « Annuler : … ».
- **Images** : `addImages` n'est pas annulable ; annuler l'import d'une image laisse ses octets dans `images` jusqu'à l'enregistrement, ce qui est sans effet visible.
- **Ordre du registre** : `commandForKey` prend la première commande qui correspond ; l'unicité des raccourcis rend cet ordre indifférent, et doit être gardée.
- **Zone `toolbar`** : prévue par le type mais utilisée par aucune commande aujourd'hui (les boutons de barre d'outils réutilisent des commandes de menu).

Chapitre 13

Interface utilisateur

L'interface de Portolan vit dans `src/ui/` : 69 fichiers React (TypeScript, JSX) et une feuille de style unique, `src/ui/global.css`. Elle ne contient pas de règle métier : elle lit le projet dans l'éditeur (`store/editor`), le modifie par les actions annulables de `src/store/`, exécute les commandes du registre (`commands/registry`) et dessine les pages avec les composants de `src/render/`. Ce qui lui est propre tient en trois choses : l'*état de l'interface* (page active, zoom, sélection, dialogue ouvert…), les *gestes* de la zone de dessin (sélection, placement, création de liaisons, retouche des tracés) et les *formulaires* (panneau des propriétés, dialogues).

Les composants sont écrits à la main, sans bibliothèque de composants (décision du 2026-11-04, lot 13) : menus, dialogues et champs restent simples et l'application reste légère. Le chapitre de référence (`ui/App` et suivants) décrit chaque module ; ce chapitre en donne l'organisation.

13.1 Démarrage et contextes React

Le point d'entrée `src/main.tsx` construit les objets indépendants de React avant le premier rendu :

1. les polices embarquées sont attendues (la mesure du texte n'utilise qu'Inter, NF-07b), puis la fonction de mesure est créée ;
2. l'éditeur (`createEditor(createStarterProject())`) et l'état de l'interface (`createUi`) ; l'écran d'accueil est demandé (`home: true`) ;
3. le stockage local est ouvert (s'il est disponible) ; le seuil d'alerte de poids des images (`prefs.imageLimitMb`, EDM-04) y est relu, puis réécrit à chaque changement des préférences ;
4. la session (`app/session`) reçoit `ui.confirm` pour ses confirmations : une ouverture qui perdrait des modifications passe par la fenêtre de message commune, jamais par `window.confirm`.

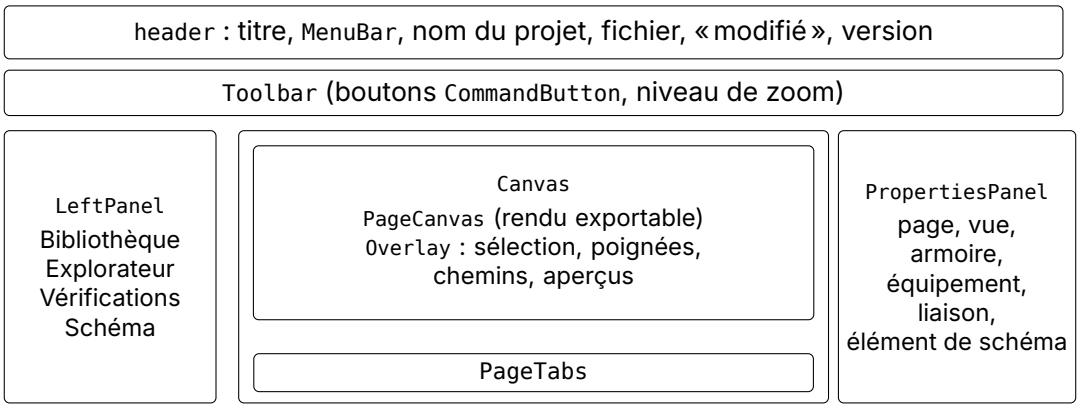
L'arbre React est ensuite enveloppé, de l'extérieur vers l'intérieur, par `I18nextProvider`, `EditorContext`, `UiContext`, `SessionContext`, `ImageUrlContext` (URL des images du projet) et `TextMeasureContext` (mesure du texte), autour de `App`. Les composants y accèdent par des crochets : `useEditor` et `useEditorState` pour le projet (`store/react`), `useUiApi` et `useUi` pour l'état de l'interface, `useSession` pour la session.

13.1.1 Le crochet de test `window.__portolan`

En développement seulement (`import.meta.env.DEV`, donc jamais dans le build publié), `main.tsx` expose `window.__portolan` avec `editor`, `session`, `ui` et `actions` (les actions de `store/actions`, `pageActions`, `deviceActions`, `machineActions` et `annotationActions` réunies). Les tests de bout en bout s'en servent pour préparer une scène sans passer par la souris (`tests/e2e/helpers.ts`, `withHook`). Une fois la fenêtre affichée (`onShown` de `App`, appelé après les effets qui écoutent les raccourcis) *et* la session démarrée, le crochet reçoit `started: true` ; `ready()` attend ce drapeau, sans quoi un raccourci pressé trop tôt serait perdu sur une machine chargée.

13.2 Arbre des composants

La fenêtre principale (ERG-01) est `App`. La figure 13.1 montre sa disposition ; les couches modales sont rendues après la zone de travail, dans cet ordre, si bien qu'un message s'affiche toujours au-dessus d'un dialogue.



Couches au-dessus : `HomeScreen`, `DialogHost`, `ContextMenu`, `DraftDialog`, `ErrorDialog`, `MessageHost`, `Toasts`, `UpdateBanner`.

Fig. 13.1 : Disposition de la fenêtre principale (`src/ui/App.tsx`).

En-tête
: `MenuBar` : un bouton par menu de `MENUS`, qui ouvre une liste `CommandList`. Puis le nom du projet, le nom du fichier (ou « sans titre »), l'indicateur « modifié » (`selectIsDirty`) et la version. En édition de bibliothèque (BIB-13), un badge indique la version de la bibliothèque.

Barre d'outils
: `Toolbar` : les commandes de `TOOLBAR`, chacune par `CommandButton`, et le niveau de zoom en pourcentage de la taille réelle.

Panneau de gauche
: `LeftPanel` : quatre onglets en icônes, nommés au survol (`leftTab`, `ui/panels/tabIcons`) — bibliothèque (`LibraryPanel`), explorateur (`ExplorerPanel`), vérifications (`ChecksPanel`, avec la pastille du nombre d'anomalies, ERG-11) et « Schéma » (`SchemaPanel`, PAG-07).

Zone centrale
: `Canvas` (la page active, ses gestes, ses infobulles), la légende des états de ports

(`PortOccupancyLegend`) et les onglets de pages (`PageTabs`). Quand une bibliothèque est ouverte pour édition, la zone centrale montre à la place `LibrarySheet` et le panneau des propriétés disparaît.

Panneau de droite
: `PropertiesPanel` : les propriétés de la sélection, ou de la page active quand rien n'est sélectionné.

Couches modales
: l'écran d'accueil (`HomeScreen`), le dialogue ouvert (`DialogHost`), le menu contextuel (`ContextMenu`), la restauration du brouillon (`DraftDialog`), l'erreur d'ouverture ou d'enregistrement (`ErrorDialog`), les messages (`MessageHost`), les notifications (`Toasts`) et le bandeau de mise à jour (`UpdateBanner`, WEB-08).

`App` porte aussi trois comportements globaux : les raccourcis clavier (`useCommandShortcuts`), l'avertissement à la fermeture de l'onglet quand le projet est modifié (PRJ-04 ; pas pendant un rechargement de mise à jour, qui a écrit le brouillon d'abord) et l'ouverture d'un fichier `.rkdg` ou `.rkdglib` déposé sur la fenêtre (PRJ-02, BIB-13).

13.3 L'état de l'interface

13.3.1 Un second store, hors du projet

Tout ce que l'utilisateur voit sans que cela appartienne au projet vit dans `UiState`, un store Zustand « vanilla » créé par `createUi` (décision du 2026-11-04, lot 13). Il n'entre ni dans l'historique ni dans le fichier : sélectionner, zoomer ou masquer un type de liaison n'est pas une modification annulable et ne rend pas le projet « modifié ». Étant indépendant de React, il est passé aux commandes dans leur contexte (`ctx.ui`) et testé sans navigateur (`src/ui/state.test.ts`) ; dans les tests et les scripts, `ctx.ui` est absent et les commandes qui en ont besoin sont désactivées.

13.3.2 Invariants maintenus par `createUi`

`createUi` s'abonne à l'éditeur. À chaque changement de projet (annulation, suppression, fichier ouvert) :

- si la page active n'existe plus, la page de même rang (ou la dernière) devient active dans le même projet, la première page dans un autre projet ;
- la sélection est filtrée par `selectionExists` : un élément supprimé ou annulé n'y reste jamais ; les vues et éléments de schéma ne sont gardés que s'ils sont sur la page active ;
- un projet d'un autre identifiant (nouveau, ouvert, restauré) ferme l'écran d'accueil.

Changer de page (`setActivePage`) retire de la sélection les vues et éléments de schéma, qui n'ont de sens que sur leur page ; armoires, équipements, liaisons et ports restent sélectionnés. Deux éléments sont identiques quand leur clé `selectionKey` l'est (sorte, équipement du port, identifiant).

Champs	Rôle
`activePageId`, `renamingPageId`	page affichée ; onglet en cours de renommage
`zoom`, `zoomAnchor`, `viewport`	zoom en pixels écran par mm de papier, point fixe du prochain zoom, taille de la zone de dessin
`selection`	liste de `SelectionItem` (vues, armoires, équipements, liaisons, ports, éléments de schéma)
`panels`, `leftTab`	panneaux visibles, onglet du panneau de gauche
`dialog`, `messages`, `toasts`, `contextMenu`	dialogue ouvert, file des messages, notifications, menu contextuel
`home`, `update`	écran d'accueil visible ; nouvelle version prête (WEB-08)
`clipboard`	équipements et éléments copiés (presse-papiers interne)
`libraryModelId`, `libraryKind`	machine ou équipement choisi dans la bibliothèque
`prefs`	préférences (seuil de poids des images), gardées dans le navigateur
`hiddenLinkTypes`, `showOccupation`, `portOccupancy`, `showGrid`	modes d'affichage (LIA-10, ARM-05, PRT-03, PAG-08), jamais enregistrés
`reveal`	rectangle à amener dans la vue une fois la page affichée (ERG-05)

Tab. 13.1 : Champs de `UiState`.

13.3.3 API impérative

L'objet `Ui` regroupe les opérations courantes : `select` (Maj + clic avec `toggle`), `setZoom` et `zoomBy` (bornés par `clampZoom`), `openDialog` et `closeDialog`, `confirm` et `alert` (promesses, ERG-07), `notify`, menus contextuels. Les champs sans opération dédiée (`leftTab`, `home`, `hiddenLinkTypes`…) se modifient directement par `ui.store.setState`.

Les messages forment une file : `confirm` et `alert` y ajoutent un `MessageRequest` et renvoient une promesse ; `MessageHost` affiche le plus ancien, au-dessus de tout dialogue, et `answer` la résout. Une fonction d'interface peut donc écrire :

```
if (await ui.confirm({ message: t('revisions.removeConfirm', { index }),
                       confirmLabel: t('common.delete'), danger: true }))
  removeRevision(editor, id);
```

13.3.4 Lire l'état dans un composant

`useUi` s'abonne à une tranche de l'état (`useUi((s) => s.zoom)`) : le composant n'est rendu à nouveau que si cette tranche change. Une tranche doit être une valeur stable (un champ, une valeur primitive) ; un tableau construit dans le sélecteur doit passer par `useShallow`, comme dans `PageTabs`. L'état d'activation des commandes dépend à la fois du projet et de l'interface : `useStateVersion` produit un numéro qui change avec l'un ou l'autre, et `useCommand` s'en sert pour garder boutons et menus à jour.

13.4 Commandes, menus et raccourcis

Les menus ne contiennent pas d'action : ce sont des listes d'identifiants de commandes du registre (`MENUS`, `TOOLBAR`, `contextMenuItems`), « - » marquant un séparateur. Libellé, raccourci, état activé, case cochée (`isChecked`, rôle `menuitemcheckbox`) et explication d'une commande désactivée viennent du registre ; `menuEntries` retire les commandes absentes et les séparateurs superflus. Ajouter une entrée de menu revient donc à ajouter son identifiant dans la liste voulue ; la commande elle-même est déclarée dans `src/commands/` et décrite dans le manuel par `\commande{id}` (contrôlé par `npm run check:doc`).

Le menu contextuel (ERG-04) est un `ContextMenuState` : position et identifiants. La zone de dessin et l'explorateur le remplissent par `contextMenuItems` selon la sorte d'éléments sélectionnés ; les onglets de page et la bibliothèque donnent leur propre liste.

`useCommandShortcuts` écoute le clavier sur toute la fenêtre (décisions des lots 05, 13 et 39) :

- quand un dialogue modal est ouvert (présence d'un `.modal-backdrop`) ou que le focus est dans un champ de texte (`isTextField`), seules les commandes `allowInTextField` (Enregistrer, Ouvrir) répondent ;
- Espace sur une case à cocher ou un bouton radio garde son rôle natif (`isToggleInput`) ;
- une commande désactivée laisse la touche au navigateur (les flèches font défiler) ;
- la touche qui a lancé une commande voit aussi son relâchement annulé, pour qu'un bouton qui a le focus ne soit pas activé par Espace (ERG-10).

13.5 La zone de dessin

13.5.1 Couches et coordonnées

`Canvas` est un conteneur défilant qui contient la feuille (`.sheet`) à la taille de la page multipliée par le zoom. Deux SVG s'y superposent, dans le même repère en millimètres de papier :

1. `PageCanvas` (`src/render/PageView.tsx`) : la page exactement comme elle sera exportée ;
2. `Overlay` : la couche de l'interface, jamais exportée — grille, occupation des armoires, contours de sélection, poignées de redimensionnement, aperçu de dépôt, rectangle de sélection, et en enfants les états de ports (`PortOccupancy`), le chemin de la liaison sélectionnée (`PathHighlight`), les poignées du tracé (`RouteHandles`), les guides d'alignement (`GuidesLayer`) et l'aperçu de la liaison en cours de tracé.

Les épaisseurs de la couche d'interface sont exprimées en pixels écran : on multiplie par `px = 1 / zoom` pour qu'un contour reste fin à tout zoom. `clientToPage` convertit un point du pointeur en millimètres de la page.

Le zoom est en pixels écran par millimètre de papier (`ZOOM_ACTUAL` = 96/25,4 correspond à 100 %). Le canevas garde fixe le point sous le pointeur (Ctrl + molette) ou le centre de la zone (boutons, raccourcis) : avant et après le changement il compare la position de la feuille et corrige le défilement. À l'ouverture ou à la création d'un projet, il ajuste la page à la fenêtre (`fitZoom`). Il publie la taille de la zone dans `viewport` (`ResizeObserver`) et applique une demande `reveal` une seule fois, puis l'efface.

13.5.2 Identifier ce qui est sous le pointeur

Le rendu porte des attributs de données : `data-view-id`, `data-device-id`, `data-port-id`, `data-link-id`, `data-annotation-id`. La recherche se fait dans la pile d'éléments sous le pointeur (`document.elementsFromPoint`), car les câbles sont dessinés au-dessus des équipements et le pointeur est capturé pendant un glisser. Les ports passent en premier (un port sous un câble reste atteignable), mais seulement ceux de l'équipement dessiné au-dessus (EQP-09) et pas sous un élément de schéma. `portAt` et `itemAt` traduisent l'élément trouvé en `SelectionItem` : port, liaison, élément de schéma, équipement (seulement dans une vue d'armoire), sinon vue.

13.5.3 Gestes

Un appui (`pointerdown`) choisit un geste, conservé dans une référence React et traité par `pointermove` puis `pointerup` ; le pointeur est capturé par la zone. Les gestes qui modifient le projet en continu passent une clé de fusion (`coalesceKey`) aux actions : tout le glisser forme une seule entrée d'historique. Un seuil de 3 pixels distingue un clic d'un glisser.

Geste	Déclencheur, effet
`pan`	bouton du milieu : fait défiler la zone (ERG-10).
`editRoute`	poignée du tracé de la liaison sélectionnée : déplace un point (`moveVertex`) ou un segment perpendiculairement (`moveSegment`) ; `setManualRoute` (ROU-06).
`resizeAnnotation`	poignée d'un élément de schéma : bord, coin ou extrémité de ligne ; aimantation, proportions gardées pour une image, une bulle ou avec Maj (PAG-07, PAG-08).
`resizeView`	poignée d'une vue d'équipement extérieur : échelle, ou taille libre avec Maj (EXT-02).
`band`	appui dans le vide : rectangle de sélection (vues, équipements, éléments entièrement compris) ; avec Maj, inverse leur sélection.
`drawLink`	appui sur un port : trace une liaison jusqu'au port relâché (LIA-01).
`moveDevice`	appui sur un équipement d'armoire ou 0U : aperçu puis déplacement (EQP-02).
`moveItems`	appui sur une vue ou un élément de schéma sélectionné : déplace ensemble vues et éléments, avec guides et grille ; Alt désactive l'aimantation.

Tab. 13.2 : Gestes de la zone de dessin (`Gesture` dans `Canvas.tsx`).

Sans geste en cours, le survol affiche l'infobulle du port (`PortTooltip`, PRT-02) ou, ailleurs dans une armoire, celle de la profondeur de la U (`DepthTooltip`, EQP-05). Le double-clic ajoute un point sur un câble (ROU-06) ou ouvre les propriétés (partie « Machine » d'un équipement, EQT-04 ; texte d'un élément de schéma) ; le clic droit sur un point du tracé sélectionné le supprime, ailleurs il ouvre le menu contextuel.

13.6 Flux d'interaction

Les règles sont vérifiées par le modèle et les actions ; l'interface ne fait que présenter un refus et proposer le forçage quand la règle l'autorise. Trois petits modules sans React rassemblent ces dialogues pour que tous les points d'entrée se comportent de la même façon.

13.6.1 Placement d'un équipement

`placeWithConfirmation` reçoit une tentative `(forced) => ActionResult`. Sans forçage d'abord ; si le refus est `forceable` (dépassement de profondeur seulement, EQP-02), il

demande « Forcer » par une confirmation destructive et rejoue la tentative forcée ; sinon il explique le refus. Il est utilisé par le dépôt depuis la bibliothèque, le déplacement à la souris, le dialogue `PlaceDeviceDialog` et les champs de placement des propriétés. Pendant le survol d'un dépôt, la zone de dessin calcule la position (`placementAt`) et colore l'aperçu (`DropPreview`) : vert si permis, orange si forçable, rouge si refusé ; le dépôt reste accepté par le navigateur pour que le refus soit expliqué (décision du lot 16).

Le glisser depuis la bibliothèque porte le type `MODEL_DRAG_TYPE` ; comme les données d'un glisser ne sont lisibles qu'au dépôt, l'identifiant de la machine est aussi gardé dans `libraryDrag` pour l'aperçu. Une machine extérieure est posée au point de dépôt (`placeExternal`) ; une machine d'armoire hors d'une armoire donne une notification.

13.6.2 Création d'une liaison

`createLinkWithConfirmation` appelle `addLink` ; un avertissement de compatibilité (`forceable`, LIA-06) demande s'il faut créer la liaison quand même, un refus (port déjà relié…) est expliqué. Deux points d'entrée : le geste `drawLink` de la zone de dessin (port à port sur la même page) et le dialogue `LinkDialog` (LIA-02, ports sur des pages différentes, type proposé selon les connecteurs). La liaison créée est sélectionnée, ce qui affiche son chemin et ses poignées.

13.6.3 Éléments de schéma

`insertAnnotation` pose un élément de l'onglet « Schéma » sur une page : centré sur le point de dépôt ou au centre de la zone de dessin, avec les textes par défaut de `presetTexts`. Une image cliquée demande d'abord son fichier ; déposée, elle ne le peut pas (pas d'activation utilisateur pendant un dépôt) et laisse un cadre à remplir depuis les propriétés (`replaceAnnotationImage`). L'image importée (`importImageFile`) donne ses proportions au cadre et déclenche une alerte au-delà du seuil des préférences (décision du lot 48).

13.7 Panneau des propriétés et champs

`PropertiesPanel` choisit les sections selon la sélection : rien → `PageProperties` ; plusieurs éléments → nombre, alignement (`ArrangeSection`, dès deux vues ou éléments de schéma) et propriétés communes des équipements ; un élément de schéma, une liaison, un équipement ou un port, une vue (avec l'équipement extérieur ou l'armoire qu'elle montre), une armoire → la section correspondante. Chaque section reçoit l'entité en propriété et une `key` égale à son identifiant, pour repartir d'un état local neuf quand la sélection change.

Les champs de `ui/fields` suivent une règle commune : un champ de texte ou de nombre garde une valeur locale (`useDraft`) et ne valide qu'en quittant le champ ou sur Entrée — une action annulable par modification, pas par frappe ; Échap rend la valeur. Après validation, le champ montre la valeur du projet : la nouvelle, ou l'ancienne si l'action a été refusée. Les nombres acceptent la virgule, sont bornés et arrondis si demandé, et les flèches haut et bas les modifient d'un pas. Les couleurs sont validées à chaque changement ; l'appelant fusionne ces modifications par une clé.

13.8 Dialogues

Un seul dialogue est ouvert à la fois : `ui.openDialog(type, props)` écrit `DialogRequest` dans l'état (et ferme le menu contextuel) ; `DialogHost` cherche le composant dans `DIALOGS` et lui passe `DialogProps` (`onClose`, `props`). Les commandes ouvrent les dialogues ainsi, par exemple `run: ({ ui }) => ui?.openDialog('linksTable')`. Le cadre commun est `Modal` : rôle `dialog` ou `alertdialog`, `aria-modal`, titre relié par `aria-labelledby`, focus sur le premier champ (ou l'élément `autoFocus`) puis rendu à l'élément précédent, Échap et clic sur le fond pour fermer.

L'écran d'accueil, la restauration du brouillon et l'erreur de session ne passent pas par `DialogHost` : ils dépendent d'un état propre (`home`, `draftOffer`, `error` de la session) et peuvent coexister avec un dialogue. L'accueil attend que la proposition de brouillon soit tranchée.

13.8.1 Ajouter un dialogue

1. Écrire le composant dans `src/ui/dialogs/` (ou `exports/`, `machine/`), de signature `({ onClose, props }: DialogProps)`, autour de `Modal` ; lire le projet par `useEditorState`, le modifier par une action du store ; afficher un refus par `ui.alert`.
2. L'enregistrer sous un nom dans `DIALOGS`.
3. Déclarer la commande qui l'ouvre dans `src/commands/` et l'ajouter à un menu de `MENUS` ; décrire la commande dans `manual/`.
4. Mettre tous les textes dans `src/i18n/fr.json`.
5. Ajouter un test de bout en bout qui ouvre le dialogue par son menu et le retrouve par `getByRole('dialog', { name: … })`.

13.8.2 Ajouter un panneau ou une section de propriétés

Un onglet du panneau de gauche est une valeur de plus de `LeftTab` (type du champ `leftTab` de `UiState`), sa place dans `LEFT_TABS`, son icône dans `TAB_ICON_PATHS` (le manuel la redessine seul), un bouton dans `LeftPanel` et un composant dans `src/ui/panels/`. Une section de propriétés est un composant de `src/ui/properties/` construit avec `Section` et les champs de `ui/fields`, branché dans `PropertiesPanel` selon la sélection. Une nouvelle sorte de sélection demande aussi `SelectionKind`, `selectionExists`, `itemAt`, `selectionBoxes` et `contextMenuItems`.

13.9 Textes et internationalisation

L'interface est en français, mais aucun texte n'est écrit en dur (CG-05) : tout passe par `src/i18n/fr.json`. L'instance i18next est créée hors de React (`i18n/index`) pour que le modèle et les flux d'interaction (fichiers `.ts` comme `ui/linkFlow`) produisent eux aussi des messages, par la fonction `t` ; les composants utilisent `useTranslation()` et reçoivent l'instance par `I18nextProvider`. Les nombres, dates et tailles sont formatés par `formatNumber`, `formatDate`, `formatBytes` ou `Intl` avec la langue de l'instance.

La règle ESLint `i18next/no-literal-string` (mode `jsx-only`) s'applique à `src/**/*.tsx`, hors tests (décisions du lot 00 et du lot 09). Elle refuse le texte littéral des éléments JSX et des attributs affichés : `title`, `alt`, `aria-label`, `placeholder`, `label`, `text`. Les autres attributs (`fill`, `className`, `data-testid`…) sont libres. Elle ne voit pas les chaînes des fichiers `.ts` :

un message construit hors JSX doit passer par `t` par discipline. Les rares caractères typographiques (« × » d'un bouton de fermeture, « + » d'ajout de page) sont des expressions et portent un `aria-label` ou un `title` traduit.

13.10 Accessibilité et tests de bout en bout

Les tests Playwright (`tests/e2e/`) trouvent l'interface comme un lecteur d'écran : par rôle et nom accessible, que l'interface doit donc fournir. Les conventions suivies :

- menus : élément `nav` nommé « Menus » (rôle `navigation`) ; boutons de menu avec `aria-haspopup` et `aria-expanded` ; listes `role="menu"`, entrées `menuitem` ou `menuitemcheckbox` (`aria-checked`), séparateurs `separator` ; chaque entrée et chaque bouton de commande porte `data-command` ;
- barre d'outils `role="toolbar"` ; onglets de page et du panneau de gauche `tablist` / `tab` avec `aria-selected` ;
- panneaux `aside` avec `aria-label` (rôle `complementary`), zone centrale `main` ; arbres de la bibliothèque et de l'explorateur `tree` / `treeitem` avec `aria-level`, `aria-selected`, `aria-expanded` ;
- dialogues `dialog` nommés par leur titre ; messages et erreurs `alertdialog` ; notifications et bandeau de mise à jour `status` avec `aria-live` ; infobulles `tooltip` ;
- champs : chaque `input` a un `label` relié (`useId`) ou un `aria-label` ; `data-testid` pour les champs cités souvent par les tests.

Les éléments du dessin, qui n'ont pas de rôle, sont trouvés par leurs attributs de données (`data-device-id`, `data-port-id`…). La couche d'interface (`Overlay`) est `aria-hidden`. Un test qui ne trouve plus un élément après un changement de libellé doit être corrigé avec le nouveau libellé, jamais affaibli.

13.11 Limites connues et conseils

- Le dialogue ouvert n'est pas empilable : ouvrir un dialogue depuis un autre remplace le premier. Les confirmations, elles, s'empilent dans la file des messages.
- La capture du pointeur et `elementsFromPoint` rendent les gestes de la zone de dessin difficiles à tester unitairement : les règles (position de dépôt, aimantation, sélection par rectangle, tracés retouchés) sont des fonctions pures de `src/model/` et `src/routing/`, testées à part ; les gestes eux-mêmes le sont de bout en bout.
- Les modes d'affichage et la sélection sont perdus au rechargement : c'est voulu (ils ne font pas partie du projet) ; seules les préférences sont gardées.
- `Canvas.tsx` (environ mille lignes) et `MachineEditorDialog.tsx` sont les deux composants les plus lourds. Un nouveau geste s'ajoute comme variante de `Gesture`, avec son traitement dans les trois gestionnaires du pointeur ; tout calcul non trivial va dans le modèle.

Chapitre 14

Tests

Portolan est testé à trois niveaux : des tests unitaires Vitest pour tout ce qui ne dépend pas du navigateur (modèle, règles métier, routage, format de fichier, mise en page du texte, exports, outils), des tests de bout en bout Playwright qui pilotent l'application dans Chromium (avec une comparaison de captures pour le rendu), et des contrôles automatiques propres au projet (neutralité, documentation utilisateur, documentation technique, site), décrits au chapitre 16. Tous font partie de la définition de fin d'un lot (section 14.5) et de l'intégration continue (chapitre 15).

14.1 Vue d'ensemble

Commande	Fichiers	Ce qui est testé
`npm test`	`src/**/*.test.ts(x)`, `tools/**/*.test.ts`	règles, modèle, routage, format, texte, exports, outils (Vitest, Node)
`npm run test:coverage`	idem	idem, avec seuils de couverture (CI)
`npm run e2e`	`tests/e2e/`	l'application sur le serveur de développement, captures de référence
`npm run e2e:build`	`tests/build/`	le build servi par `vite preview` : CSP, hors ligne, aucune requête externe
`npm run e2e:site`	`tests/site/`	le site construit (`site-dist/`)
`npm run doc:screenshots`	`tests/manual/`	pas un test : écrit les captures annotées du manuel

14.2 Tests unitaires (Vitest)

14.2.1 Où et comment

Les tests unitaires sont à côté du code testé : `src/model/placement.ts` a son `src/model/placement.test.ts`, `tools/site.ts` son `tools/site.test.ts`. La configuration est la section `test` de `vite.config.ts` :

- `include` : `src/**/*.test.ts`, `src/**/*.test.tsx` et `tools/**/*.test.ts` ;
- environnement `node` (pas de DOM par défaut ; un test qui a besoin d'un document crée une fenêtre jsdom lui-même, par exemple pour le nettoyage des SVG) ;
- `src/test-setup.ts` initialise i18next : les messages sont ceux de `src/i18n/fr.json`, comme dans l'application ;
- libavoid-js est intégré à la transformation (`server.deps.inline`) : les tests du routage passent par la colle transformée sans `new Function` (`tools/libavoid-no-eval`), exactement comme le build ;
- `svg2pdf.js` est redirigé vers sa version ES, la version Node ne trouvant pas jsPDF ;
- les alias de `resolve` (police Inter TrueType `inter-ttf/`, WebAssembly `libavoid-wasm`) sont les mêmes que pour l'application.

14.2.2 Ce qui doit être testé unitairement

`CLAUDE.md` (règle 6) et le plan (§ 1.4) imposent que le modèle, les règles métier, le routage, le format de fichier et la mise en page du texte ne dépendent pas de React *et* soient testés unitairement : `src/model`, `src/routing`, `src/io`, `src/text`, mais aussi `src/store` (commandes, annuler/rétablir), `src/library` (générateurs d'équipements), `src/export` (SVG, PNG, PDF, tableur relus par leurs bibliothèques) et `src/commands` (registre). Une nouvelle règle métier arrive avec ses tests, qui citent l'identifiant de l'exigence dans leur titre (`describe('… (ROU-14)')`). Les composants React sont vérifiés par les tests de bout en bout et les captures, pas par des tests unitaires de rendu.

14.2.3 Couverture

`npm run test:coverage` (lancé par la CI à la place de `npm test`) mesure la couverture avec le fournisseur v8 sur `src/model`, `src/io`, `src/store`, `src/commands`, `src/text`, `src/library`, `src/routing` et `src/export` (fichiers de test exclus). Les seuils sont de **70 %** pour les lignes, les fonctions, les branches et les instructions : en dessous, la commande échoue. Rapports : résumé texte et HTML (`coverage/`).

14.2.4 Données de test partagées (`src/testing`)

Le dossier `src/testing/` contient les projets et outils réutilisés par les tests unitaires, le banc de rendu et certains outils ; il n'est pas importé par l'application. Tous les projets sont construits avec les machines génériques (CG-08) et des identifiants déterministes, pour que les résultats et les captures soient stables.

- `testing/fixtures` : `typesOnlyProject` (projet vide avec les types de connecteurs et de liaisons par défaut), `sampleProject` (petit projet complet : armoires, équipements, liaisons), `patchChain` (chaîne de panneaux de brassage pour les chemins de bout en bout) et des aides (`machine`, `ports`, `portOf`…) ;
- `testing/referenceProject` : `referenceLikeProject`, le projet disposé comme le schéma de référence de `docs/references/`, partagé par le banc de rendu et les tests de croisements du routage ;

- `testing/rangeProject` : `rangeProject`, vues sur une plage de U et câbles masqués (PAG-15, PAG-16) ;
- `testing/generatedRack` : `generatedRackProject`, armoires denses générées à partir d'une graine (générateur `seededRandom`), pour les tests de propriétés du routage (lot 38) ;
- `testing/routingChecks` : rectangles des ports d'une page et détection des segments de câble qui passent sur un port étranger (`foreignPortHits`, ROU-14) ;
- `testing/fonts` : la police Inter TrueType lue depuis le paquet, pour les tests des exports PDF.

14.3 Tests de bout en bout (Playwright)

14.3.1 Navigateur et environnement

`@playwright/test` est épinglé (1.63.0) sur la version du Chromium préinstallé dans l'environnement de développement : on ne lance pas `playwright install` en local (la CI, elle, installe Chromium avec `npx playwright install --with-deps chromium`). Toutes les configurations fixent la langue `fr-FR` et le fuseau `Europe/Paris` ; `forbidOnly` est actif en CI (un `test.only` oublié fait échouer la suite) et aucun test n'est relancé en cas d'échec (`retries: 0`) : un test instable doit être corrigé, pas masqué.

14.3.2 Les quatre configurations

`playwright.config.ts` (`npm run e2e`, dossier `tests/e2e/`) : l'application sur le serveur de développement Vite (`npx vite --port <port> --strictPort`). Tests en parallèle, fenêtre de 1280 × 800, trace gardée en cas d'échec, rapport HTML en CI. Chromium est lancé avec une locale UTF-8 (sans elle, il remplace les noms de téléchargement accentués) et des options qui rendent le texte identique sur toute machine Linux : pas de lissage sous-pixel ni de *hinting*, profil de couleur sRGB, pas de GPU. Le serveur de développement n'a ni CSP ni service worker, mais expose `window.__portolan`.

`playwright.build.config.ts` (`npm run e2e:build`, dossier `tests/build/`) : le build (`npm run build`) servi par `vite preview`. On y vérifie ce que le serveur de développement ne peut pas montrer : CSP appliquée et libavoid chargé sans `'unsafe-eval'`, aucun `new Function` dans le build, exports PDF et tableur (modules chargés à la demande), fonctionnement hors ligne après la première visite (application, libavoid, manuel utilisateur, documentation technique, notes de version, CG-03), aucune requête vers une autre origine pendant un scénario complet (REC-06), et commandes du site désactivées hors du site.

`playwright.site.config.ts` (`npm run e2e:site`, dossier `tests/site/`) : le site construit, servi comme GitHub Pages le ferait (`vite preview --outDir site-dist`). Racine qui ouvre l'application non référencée, menus Aide (documentations PDF, notes, archive) et Bibliothèque, contenu de l'archive téléchargée, service worker à la racine qui laisse passer la page des bibliothèques et l'archive, page des bibliothèques avec mot de passe.

`playwright.manual.config.ts` (`npm run doc:screenshots`, dossier `tests/manual/`) : reprend la configuration de développement, en série (un seul processus), pour écrire les captures annotées du manuel dans `manual/screenshots/` (décision « Lot 13 — Captures

annotées », 2026-11-04). Ce ne sont pas des tests de non-régression : ces images sont versionnées et régénérées à chaque lot qui modifie l'interface.

14.3.3 Ports de test distincts

Chaque configuration lit son port dans une variable d'environnement, avec une valeur par défaut : `E2E_PORT` (5179, développement), `E2E_BUILD_PORT` (5180, build), `E2E_SITE_PORT` (5181, site). Le serveur est lancé avec `--strictPort` : si le port est pris, il échoue au lieu d'en choisir un autre. Hors CI, `reuseExistingServer` réutilise un serveur déjà lancé sur ce port ; c'est pourquoi les agents qui travaillent en parallèle (plan, § 1.5) doivent chacun choisir leurs propres ports, sans quoi l'un testerait le serveur, donc le code, d'un autre. Les captures du manuel (`playwright.manual.config.ts`) lisent `E2E_MANUAL_PORT`, à défaut `E2E_PORT`, à défaut 5179.

```
E2E_PORT=5279 E2E_BUILD_PORT=5280 E2E_SITE_PORT=5281 npm run e2e
```

14.3.4 Le crochet `window.__portolan`

En développement seulement (`import.meta.env.DEV`), `src/main.tsx` expose `window.__portolan` : l'éditeur (magasin du projet), la session, l'état de l'interface et les actions du magasin (projet, pages, équipements, machines, annotations). Les tests s'en servent pour préparer un projet rapidement et vérifier le modèle après une manipulation de l'interface. La propriété `started` est ajoutée quand la fenêtre est affichée, ses raccourcis écoutés et la session démarrée : la fonction `ready` de `tests/e2e/helpers.ts` l'attend, attend les polices, puis ferme l'écran d'accueil ; `withHook` exécute une fonction avec le crochet dans la page. Le crochet n'existe pas dans le build (décision « Lot 06 — Session et crochet de test »). Les tests de `tests/build/` et `tests/site/` passent donc uniquement par l'interface.

14.3.5 Banc de rendu (`tests/harness`)

`tests/harness/index.html` est une page servie par le serveur de développement, hors du build (décision « Lot 07 — Banc de rendu »). Elle affiche une *scène* choisie par l'adresse `/tests/harness/index.html?scene=<nom>` : blocs de texte, faces et sortes d'armoires, équipements, équipements standards, planche de la bibliothèque, page, page routée, page d'exemple, cartouches, vues, équipements extérieurs, repères, faisceaux, connecteurs, demi-largeur, plage de U, annotations (table `scenes` de `tests/harness/scenes.tsx`). Avant de rendre, le banc enregistre les polices de symboles, attend Inter et charge libavoid ; il pose ensuite `body[data-ready="true"]`, que les tests attendent avant la capture de l'élément `#scene`. Les scènes réutilisent les projets de `src/testing/`.

Ajouter une scène : écrire le composant dans un fichier de `tests/harness/`, l'inscrire dans `scenes`, puis ajouter le test de capture dans `tests/e2e/` :

```
test('capture de référence : maScene', async ({ page }) => {
  await page.goto('/tests/harness/index.html?scene=maScene');
  await page.waitForSelector('body[data-ready="true"] #scene');
  await expect(page.locator('#scene')).toHaveScreenshot('maScene.png');
});
```

14.3.6 Non-régression visuelle

Les captures de référence sont dans `tests/e2e/__screenshots__/<fichier de test>/<nom>.png`, un seul jeu pour Chromium sous Linux, sans suffixe de plateforme (le même rendu en local et en CI). La comparaison (`toHaveScreenshot`) tolère au plus **50 pixels différents** par capture (`maxDiffPixels: 50`, seuil par pixel 0,1), animations désactivées et curseur de saisie masqué. La limite est absolue et non relative : 0,2 % des pixels laissait passer de vrais changements sur les grandes captures (décision « Lot 11 — Tolérance des captures de référence », 2026-11-04).

Mettre à jour une capture. Seulement pour un changement visuel *voulu*.

1. Régénérer avec `npx playwright test --update-snapshots=all` (en limitant au fichier concerné). Le mode par défaut ne réécrit pas une capture restée sous la tolérance : `=all` est nécessaire pour qu'un petit changement voulu soit enregistré.
2. Regarder chaque image modifiée et vérifier que seul le changement attendu apparaît.
3. Le signaler dans le message de commit (quelles captures, pourquoi).

14.4 Règle : ne jamais affaiblir un test

Un test ou un contrôle n'est jamais désactivé, ignoré (`skip`, `only`), assoupli (tolérance relevée, assertion retirée, seuil de couverture abaissé, terme retiré de `tools/forbidden-terms.txt`) ni contourné pour faire passer un lot (`CLAUDE.md`, règle 8). Une capture de référence ne change que pour un changement visuel voulu et signalé. Si un test ne peut passer qu'en affaiblissant une exigence, on s'arrête et on pose la question (plan, § 1.2). Un test instable se corrige en le rendant déterministe (attendre un état précis, identifiants fixes, polices chargées), jamais par une relance.

14.5 Définition de fin

Un lot n'est terminé que lorsque toutes les conditions du § 1.3 de `docs/plan-developpement.md` sont remplies :

1. `npm run lint` et `npm run typecheck` (TypeScript strict) passent ;
2. `npm test` passe et les nouvelles règles métier sont couvertes ;
3. `npm run e2e` passe, captures de référence comprises ;
4. `npm run check:neutral` passe (CG-08) ;
5. `npm run check:doc` passe : chaque commande du registre est décrite dans le manuel, qui compile (DOC-09) ;
6. `npm run build` produit le site statique sans avertissement bloquant ;
7. le manuel (`manual/`) décrit tout ce que le lot ajoute à l'interface ;
8. aucun texte d'interface en dur : tout passe par `src/i18n/fr.json` (CG-05 ; une règle ESLint le vérifie) ;

9. le tableau d'avancement est coché et `docs/decisions.md` complété si besoin.

À partir du lot 49, elle comprend aussi la mise à jour de cette documentation technique : `npm run check:tech-doc` doit passer. En pratique, on lance aussi `npm run format:check` (exécuté par la CI, sans le masquer derrière un tube), et `npm run e2e:build`, `npm run site`, `npm run check:site` et `npm run e2e:site` quand le lot touche au build, au hors-ligne ou au site. Après intégration d'une branche de lot, toute la définition de fin est relancée sur le résultat, puis le résultat de GitHub Actions est vérifié après chaque push.

Chapitre 15

Intégration continue et publication

Deux workflows GitHub Actions accompagnent le dépôt : `.github/workflows/ci.yml` (« Contrôles »), qui rejoue la définition de fin à chaque push, et `.github/workflows/publish.yml` (« Publication »), qui construit le site public et le pousse vers le dépôt servi par GitHub Pages. Ce chapitre décrit ces deux chaînes, le contenu du site, son contrôle, le fonctionnement hors ligne et la politique de sécurité du contenu. La procédure de l'équipe (mise en place des secrets, publication d'une version, changement du mot de passe des bibliothèques) est dans `docs/publication.md`.

15.1 Principe : un dépôt privé, un site public non référencé

Le code est dans un dépôt GitHub *privé* ; le site est publié sur GitHub Pages, *public mais non référencé* (WEB-01). Avec l'offre GitHub gratuite, Pages n'est pas disponible pour un dépôt privé : le site construit est poussé dans un second dépôt, public, qui ne contient que lui (décision « Lot 32 — Site public non référencé et publication », point ouvert n° 1 tranché par l'utilisateur). Le dépôt cible est donné par la variable de dépôt `SITE_REPOSITORY` (`propriétaire/nom`), avec une valeur par défaut dans `publish.yml`.

Le site *ouvre directement l'application* (WEB-02, décision « Lot 39b — Site sans page d'accueil », 2026-11-05) : sa racine est le build de l'application. Depuis l'application, le menu Aide donne les documentations PDF, les notes de version et l'archive ; le menu Bibliothèque ouvre la page protégée des bibliothèques. Rien n'y est référencé : `robots.txt` interdit l'indexation et chaque page porte `<meta name="robots" content="noindex, nofollow">`.

15.1.1 Indexation : limite acceptée

Le site est un *site de projet* GitHub Pages, servi sous un sous-chemin (`https://<propriétaire>.github.io/portolan/`). Les moteurs de recherche ne lisent `robots.txt` qu'à la racine du domaine : celui du site, publié dans `portolan/`, est ignoré. Il reste publié et contrôlé par `check:site` (il servirait tel quel à la racine d'un domaine). Seule la balise `noindex` protège donc de l'indexation, et elle n'existe que dans les pages HTML. Les trois documentations PDF en accès libre (DOC-10) et l'archive de la version n'en portent pas : seul un en-tête HTTP `X-Robots-Tag` les protégerait, et GitHub Pages ne permet pas d'en ajouter. Un moteur peut les indexer s'il trouve un lien public vers elles (il n'en existe aucun).

Décision de l'utilisateur du 2026-11-06 (WEB-01, cahier des charges 1.13) : ce risque limité est *accepté et documenté* (les PDF ne contiennent ni marque, ni secret, ni donnée de l'équipe ; les bibliothèques sont chiffrées), sans modification du site ni de `check:site`. Détail

et pistes si le risque devait être supprimé : `docs/publication.md`, section « Indexation par les moteurs de recherche ».

15.2 Intégration continue : `ci.yml`

Le workflow « Contrôles » tourne à chaque push (toutes branches) et sur chaque pull request, sur `ubuntu-24.04`, en un seul job. Étapes, dans l'ordre :

1. récupération du dépôt, Node 22 avec cache npm, `npm ci` ;
2. outils LaTeX et polices (`texlive-xetex`, `texlive-lang-french`, `texlive-latex-extra`, `texlive-fonts-recommended`, `latexmk`, `fonts-inter`, `fonts-dejavu-core`) : la même liste que dans `CLAUDE.md` ;
3. Chromium pour Playwright (`npx playwright install --with-deps chromium`) ;
4. `npm run lint`, `npm run format:check`, `npm run typecheck` ;
5. `npm run test:coverage` (tests unitaires avec seuils de couverture) ;
6. `npm run check:neutral` ;
7. `npm run check:doc` : registre des commandes et documentation utilisateur, compilée dans `manual/build/portolan.pdf` (manuel) et `manual/build/prise-en-main-rapide.pdf` (prise en main rapide, lot 50) ;
8. `npm run check:tech-doc` (lot 49) : couverture de la documentation technique, puis compilation de `docs/technique/build/portolan-technique.pdf`. Il tourne *avant* le build, qui embarque les trois PDF ;
9. `npm run build`, puis `npm run site` et `npm run check:site` : le site est construit et contrôlé à chaque push, sans être publié (pas de bibliothèques chiffrées : la page des bibliothèques est construite vide) ;
10. `npm run e2e:site`, `npm run e2e`, `npm run e2e:build` ;
11. artefacts : `documentation` (manuel utilisateur et prise en main rapide) et `documentation-technique` (`portolan-technique.pdf`), joints même en cas d'échec (`if: always()`, sans erreur si le fichier manque) ; rapport Playwright (`playwright-report/`) en cas d'échec seulement.

Une étape en échec arrête le job. Le résultat est à vérifier après chaque push, pas seulement en fin de phase (`CLAUDE.md`). Ajouter un contrôle : l'ajouter comme script npm, à la définition de fin du plan, puis comme étape `run` ici, avant le build s'il produit un fichier que le build embarque.

15.3 Publication : `publish.yml`

Le workflow « Publication » est lancé par une étiquette `v*` (`git tag v0.2.0 && git push origin v0.2.0`) ou à la main (Actions › Publication › Run workflow). Une seule publication à la fois (`concurrency: publication`, sans annuler celle en cours) ; le jeton du workflow n'a que la lecture du dépôt (`permissions: contents: read`) : l'écriture dans le dépôt public passe par une clé de déploiement. Étapes :

1. récupération, Node 22, `npm ci`, outils LaTeX et polices (comme la CI) ;
2. `npm run check:neutral` : rien n'est publié si le cœur cite une marque ;
3. `npm run doc` (manuel utilisateur et prise en main rapide) et `npm run doc:tech` (documentation technique), puis `npm run build`, qui embarque les trois PDF ;
4. bibliothèques chiffrées : si le secret `LIBRARIES_PASSWORD` existe, `npm run publish:libraries` (`tools/publish-libraries`) ; sinon un avertissement, et la page des bibliothèques est publiée sans bibliothèque ;
5. `npm run site` puis `npm run check:site` ;
6. le dossier `site-dist/` est joint comme artefact `site` ;
7. `tools/publish-site.sh`, avec le secret `SITE_DEPLOY_KEY` (clé SSH privée, droit d'écriture sur le seul dépôt public) : clone du dépôt public, remplacement de tout son contenu par `site-dist/` et `site/README.public.md`, commit « Publication de la version … » et push sur `main` ; rien n'est poussé si le site n'a pas changé (détails au chapitre 16).

Aucune action tierce n'écrit dans le dépôt public : un simple script Git. Le numéro de version est celui de `package.json`, commun à l'application (À propos), aux documentations (`\appversion`), au site et au nom de l'archive.

15.4 Construction du site : `npm run site`

`tools/build-site.ts` (`tools/build-site`) assemble `site-dist/` à partir de trois entrées : `dist/` (build de l'application), `site/` (fichiers propres au site) et `library-data/` (bibliothèques chiffrées, facultatif). Le tableau donne le résultat.

Chemin	Contenu
`/` (`index.html`…)	l'application, copie de `dist/` sans les cartes de sources, avec son service worker `sw.js`, son manifeste et ses icônes ; `index.html` porte le marqueur de site
`manual/portolan.pdf`	le manuel utilisateur, embarqué par le build (menu Aide)
`manual/prise-en-main-rapide.pdf`	la prise en main rapide (lot 50), embarquée de même (menu Aide)
`manual/portolan-technique.pdf`	la documentation technique (DOC-10), embarquée de même (menu Aide› Documentation technique (PDF))
`portolan-<version>.zip`	l'archive pour un déploiement interne par simple copie (WEB-09)
`bibliotheques/`	la page protégée des bibliothèques et les fichiers chiffrés (WEB-03)
`robots.txt`, `.nojekyll`	interdiction d'indexer ; fichiers servis tels quels par GitHub Pages

Le site porte ainsi les documentations utilisateur en PDF ; leur organisation relève du manuel utilisateur et n'est pas décrite ici.

15.4.1 L'application à la racine et le marqueur de site

Le build de l'application ne sait pas s'il est servi par le site, par l'archive déployée ailleurs, par `vite preview` ou par le serveur de développement. `npm run site` ajoute donc à la seule copie publiée de `index.html` la balise `<meta name="portolan-site" content="portolan-<version>.zip">` (`withSiteMarker`, nom `SITE_MARKER`). L'application la lit dans sa propre page (`app/site`) : aucune requête, la même réponse en ligne et hors ligne. Le contenu n'est accepté que s'il a la forme d'un nom d'archive (`parseSiteMarker`), jamais une adresse arbitraire. Sans marqueur, les commandes « Télécharger l'archive » et « Bibliothèques de l'équipe… » sont désactivées, avec leur explication sous l'entrée de menu. Le service worker précache `index.html` sous la révision calculée avant l'ajout de la balise : Workbox ne vérifie pas le contenu, la copie en cache est bien celle du site.

15.4.2 Les documentations PDF

Le greffon `portolan-manual` de `vite.config.ts` émet avec le bundle, s'ils existent, `manual/build/portolan.pdf` sous `manual/portolan.pdf`, `manual/build/prise-en-main-rapide.pdf` sous `manual/prise-en-main-rapide.pdf` (lot 50) et, depuis le lot 49 (décision de l'utilisateur du 2026-11-06), `docs/technique/build/portolan-technique.pdf` sous `manual/portolan-technique.pdf`. Émis avec le bundle (et non copiés après), ils figurent dans le cache hors ligne. Ils sont donc en accès libre sur le site, dans l'archive et hors ligne. Le menu Aide les ouvre dans un nouvel onglet : « Prise en main rapide (PDF) » (`QUICK_START_URL`), « Manuel utilisateur (PDF) » (`MANUAL_URL`) et « Documentation technique (PDF) », commande `help.technicalManual` (`TECHNICAL_MANUAL_URL`) ; l'écran d'accueil et « À propos » proposent aussi les deux premiers (`DocumentLinks`). Comme le greffon ignore un PDF absent, c'est `npm run site` qui exige les trois fichiers dans `dist/manual/`, puis `check:site` qui exige leur présence sur le site. Ordre à respecter : `npm run doc` et `npm run doc:tech`, puis `npm run build`, puis `npm run site`.

15.4.3 La page des bibliothèques

Construite par Vite avec sa propre configuration `vite.libraries.config.ts` : source `site/bibliotheques/` (page, script, style), sortie `site-dist/bibliotheques/`, et sa propre CSP (`script-src 'self'`, `connect-src 'self'`, `form-action 'none'`, sans `'wasm-unsafe-eval'`). `npm run site` y copie le contenu de `library-data/` produit par `npm run publish:libraries` : chaque `.rkdglib` de `libraries/` et le catalogue, chiffrés (AES-256-GCM, clé dérivée par PBKDF2-SHA-256 du mot de passe de l'équipe, sel et IV aléatoires par fichier ; WEB-03, WEB-04). La page demande le mot de passe et déchiffre dans le navigateur ; le mot de passe n'est écrit ni dans le dépôt ni dans le site. Changer le mot de passe = changer le secret et relancer la publication.

15.4.4 L'archive téléchargeable

`portolan-<version>.zip` contient le build tel que `dist/` (application, manuel, documentation technique, service worker), sans cartes de sources, sous un dossier `portolan-<version>/`. Elle est faite depuis `dist/` et non depuis le site : elle n'a pas le marqueur, donc, déployée sur un serveur interne ou un réseau isolé, elle désactive les liens vers le site au lieu de pointer vers des pages absentes. Le build ayant une base relative (`base: './'`, NF-11), elle fonctionne servie depuis n'importe quel sous-chemin.

15.5 Contrôle du site : `npm run check:site`

`tools/check-site.ts` (`tools/check-site`, fonctions de `tools/site`) parcourt `site-dist/` et refuse :

- un fichier interdit : carte de source (`.map`, qui contiendrait tout le code TypeScript), source TypeScript, et toute *source* de document : chemin sous `docs/`, Markdown (`.md`), LaTeX (`.tex`). Le code et les documents de travail restent privés ; seule la documentation compilée en PDF est publiée ;
- un fichier obligatoire absent : `index.html`, `sw.js`, `manifest.webmanifest`, `icon.svg`, `manual/portolan.pdf`, `manual/prise-en-main-rapide.pdf`, `manual/portolan-technique.pdf`, `bibliotheques/index.html`, `robots.txt`, l'archive de la version ;
- une page HTML sans `noindex`, un lien ou une ressource vers une autre origine (attributs `href`, `src`, `action` ; `url(…)` dans le CSS, WEB-06) ;
- un `index.html` sans marqueur de site, un `robots.txt` sans `Disallow: /`.

Les tests `tests/site/` complètent ce contrôle statique dans un navigateur (voir chapitre 14).

15.6 Hors ligne : service worker et cache

L'application est une PWA installable qui fonctionne sans réseau une fois chargée (CG-03 ; décision « Lot 31 — Hors ligne et mises à jour », 2026-11-05). Le service worker est produit au build par vite-plugin-pwa (Workbox copié dans le build, rien n'est chargé de l'extérieur) ; il n'y en a pas en développement.

- **Précache** : dès la première visite, tous les fichiers du build (`**/*.{js,css,html,wasm,woff,woff2,ttf,svg,png,pdf,json,webmanifest}`) : scripts, y compris les modules chargés à la demande (PDF, tableur, libavoid), WebAssembly, police, icônes, schémas JSON et les trois documentations PDF (manuel, prise en main rapide, documentation technique). Limite par fichier relevée à 16 Mo ; les anciens caches sont nettoyés.
- **Navigation** : toute navigation dans la portée reçoit `index.html` (`navigateFallback`), sauf la page des bibliothèques et l'archive (`navigateFallbackDenylist` : `/bibliotheques/`, `.zip`), qui vont au réseau : sinon, la page des bibliothèques afficherait l'application.
- **Manifeste** : nom, couleurs, icônes 192, 512 et SVG ; `start_url` et `scope` relatifs (`./`) : installée depuis la racine du site, l'application a la racine pour portée.
- **Mises à jour** (WEB-08) : `registerType: 'prompt'`. Quand une nouvelle version est prête, un bandeau la propose (`offerUpdate`) ; le rechargement écrit d'abord le brouillon de secours, puis active la nouvelle version : rien n'est perdu. L'enregistrement est fait par `startOffline`, dans le build seulement.

Un fichier ajouté au build est mis en cache s'il correspond aux motifs ci-dessus ; un nouveau type de fichier demande d'ajouter son extension à `globPatterns`, et un fichier de plus de 16 Mo de relever la limite. Le test `tests/build/csp.spec.ts` coupe le réseau après la première visite et vérifie l'application, libavoid, les trois PDF et les notes de version.

15.7 Politique de sécurité du contenu (CSP)

Le build porte une CSP stricte, injectée dans `index.html` par le greffon `portolan-csp` de `vite.config.ts` (au build seulement : le serveur de développement a besoin de scripts en ligne pour le rechargement à chaud) (NF-05, CG-02) :

```
default-src 'self'; script-src 'self' 'wasm-unsafe-eval';
style-src 'self' 'unsafe-inline'; img-src 'self' data: blob:;
font-src 'self' data:; connect-src 'self'; worker-src 'self' blob:;
object-src 'none'; base-uri 'self'
```

Rien n'est chargé depuis une autre origine ni envoyé ailleurs (`connect-src 'self'`). `'wasm-unsafe-eval'` est nécessaire pour compiler le WebAssembly de libavoid ; il n'y a pas de `'unsafe-eval'` : la colle JavaScript de libavoid-js, qui fabriquait des fonctions par `new Function`, est transformée au build par `tools/libavoid-no-eval` (décision « Phase F — libavoid-js sans exécution dynamique, CSP stricte », 2026-11-05, qui remplace celle de la phase E). Les cartes de sources sont produites (`sourcemap: 'hidden'`) pour le débogage, sans lien dans les fichiers publiés, et ne sont jamais copiées sur le site. Le test `tests/build/csp.spec.ts` vérifie que la politique est appliquée, que libavoid se charge sans violation, qu'aucun fichier ne contient `new Function` et qu'aucune requête ne part vers une autre origine pendant un scénario complet (REC-06). Ne jamais ajouter de traceur, de police ou de script externe : `check:site` refuse tout lien sortant (WEB-06).

Chapitre 16

Outils du dépôt

Le dossier `tools/` regroupe les programmes de l'équipe qui ne font pas partie de l'application : contrôles de la définition de fin (neutralité, documentation), compilation des documentations LaTeX, génération des schémas JSON, construction et contrôle du site, chiffrement des bibliothèques de marques, et la transformation de libavoid-js appliquée au build. Ils tournent sous Node 22, sont écrits en TypeScript et lancés par `tsx` (sans étape de compilation) au travers des scripts de `package.json` ; seul `tools/libavoid-no-eval.ts` est chargé par Vite lui-même (greffon de `vite.config.ts`).

Deux familles de fichiers coexistent :

- les *points d'entrée* (`build-*.ts`, `check-*.ts`, `publish-libraries.ts`) : un script court qui lit des fichiers, appelle des fonctions, écrit un message en français et termine par `process.exit(1)` en cas d'échec ;
- les *modules d'aide* (`manual.ts`, `neutral.ts`, `site.ts`, `json-schema.ts`, `tech-doc.ts`, `tech-doc-files.ts`, `libavoid-no-eval.ts`) : les fonctions, pures autant que possible, testées unitairement par Vitest (les fichiers `tools/*.test.ts` font partie de `npm test`, voir chapitre 14).

Les modules de `tools/` n'exportent rien vers l'application : seule leur section `\module{tools/<nom>}` est exigée par `npm run check:tech-doc` (pas de section par déclaration exportée, réservée à `src/`). Un nouveau fichier `tools/<nom>.ts` (hors tests) doit donc recevoir sa section dans ce chapitre, sans quoi le contrôle échoue.

16.1 Scripts npm et outils

Le tableau suivant donne, pour chaque script de `package.json` qui passe par un outil du dépôt, le fichier lancé et ce qu'il produit. Les autres scripts appellent directement un outil tiers : `dev`, `build`, `preview` (Vite), `lint` (ESLint, zéro avertissement toléré), `format` et `format:check` (Prettier), `typecheck` (`tsc -b`), `test` et `test:coverage` (Vitest), `e2e`, `e2e:build`, `e2e:site` et `doc:screenshots` (Playwright, voir chapitre 14).

Script	Outil	Produit ou vérifie
`check:neutral`	`check-neutral.ts`	aucun terme interdit dans le cœur (CG-08)
`check:doc`	`check-doc.ts`	registre des commandes ↔ manuel, puis compilation du manuel (DOC-09)

Script	Outil	Produit ou vérifie
`doc`	`build-doc.ts`	`manual/build/portolan.pdf`
`check:tech-doc`	`check-tech-doc.ts`	couverture de la documentation technique, puis compilation (DOC-10)
`doc:tech`	`build-tech-doc.ts`	`portolan-technique.pdf` dans `docs/technique/build/`
`schema`	`build-json-schema.ts`	`public/schemas/*.schema.json` (FMT-02)
`site`	`build-site.ts`	le site dans `site-dist/` (WEB-01, WEB-02, WEB-09)
`check:site`	`check-site.ts`	contrôle de `site-dist/`
`publish:libraries`	`publish-libraries.ts`	bibliothèques chiffrées dans `library-data/` (WEB-03 à WEB-05)
`example:library`	`build-example-library.ts`	`exemple-generique.rkdglib` dans `libraries/`
build Vite	`libavoid-no-eval.ts`	colle de libavoid-js sans `new Function` (NF-05)
workflow Publication	`publish-site.sh`	pousse `site-dist/` vers le dépôt public

Les options passées après `--` sont transmises à l'outil, par exemple `npm run doc:tech -- --only=chapitres/outils --job=essai`.

16.2 `tools/build-doc`

Point d'entrée de `npm run doc` : compile les deux documents de l'utilisateur en appelant `compileManual` de `tools/manual`, qui écrit d'abord les fichiers générés de `manual/generated/` (version, icônes des onglets, liste des commandes) puis lance `latexmk -xelatex portolan.tex prise-en-main-rapide.tex` dans `manual/`. Résultat : `manual/build/portolan.pdf` (manuel complet) et `manual/build/prise-en-main-rapide.pdf` (lot 50), ensuite embarqués dans le build par le greffon `portolan-manual` de `vite.config.ts` (il faut donc lancer `npm run doc` *avant* `npm run build` pour que le menu Aide, l'écran d'accueil et le cache hors ligne trouvent les documents). Aucune option. En cas d'échec de `latexmk` (erreur LaTeX, outil absent), le script écrit « La compilation de la documentation a échoué. » et sort avec le code 1 ; le journal de LaTeX est affiché tel quel (sortie héritée). Pas de test unitaire propre : la compilation est vérifiée par `check:doc` en CI.

16.3 `tools/build-example-library`

Point d'entrée de `npm run example:library` : écrit `libraries/exemple-generique.rkdglib`, la bibliothèque d'exemple sans marque, modèle à suivre pour les bibliothèques de marques (WEB-05) et contenu de la page des bibliothèques tant qu'aucune n'existe. Elle part du projet de test `typesOnlyProject` (`src/testing/fixtures.ts`), y génère trois machines génériques (serveur, commutateur, écran) avec `generateMachine` et leurs paramètres par défaut, puis les exporte avec `libraryForExport` et `writeLibraryArchive`. Les identifiants (bibliothèque, machines, ports) et la date (5 novembre 2026) sont fixes : le fichier ne change que si son

contenu change, ce qui évite un binaire différent à chaque exécution. La version de l'application inscrite dans le manifeste vient de `package.json`. Le script lève une erreur si une machine exportée garde pour origine la bibliothèque par défaut. Aucune option, pas de test unitaire : le fichier produit est relu par `publish:libraries` et par les tests du site.

16.4 `tools/build-json-schema`

Point d'entrée de `npm run schema` : écrit dans `public/schemas/` les schémas JSON publiés avec l'application (FMT-02), tels que les calcule `jsonSchemas` de `tools/json-schema`, et affiche le chemin de chaque fichier écrit. Le dossier est créé au besoin. À lancer après toute modification de `src/model/schema.ts` ou du schéma du manifeste : le test `tools/json-schema.test.ts` échoue tant que les fichiers publiés ne sont pas à jour. Aucune option.

16.5 `tools/build-site`

Point d'entrée de `npm run site` : construit le site public dans `site-dist/` (exporte la constante `SITE_DIR`). Il suppose que `npm run doc`, `npm run doc:tech` et `npm run build` ont déjà tourné, et, pour publier des bibliothèques, `npm run publish:libraries`. Étapes :

1. vérifie que `dist/index.html` et les PDF embarqués par le build (`dist/manual/portolan.pdf`, `dist/manual/prise-en-main-rapide.pdf` depuis le lot 50 et `dist/manual/portolan-technique.pdf` depuis le lot 49) existent, sinon s'arrête en indiquant les commandes à lancer ;
2. vide `site-dist/`, y copie `dist/` *sans les cartes de sources* (`.map`, qui publieraient tout le code TypeScript du dépôt privé) ;
3. ajoute à la seule copie publiée de `index.html` le marqueur de site (`withSiteMarker`, balise `<meta name="portolan-site">` qui nomme l'archive) ; copie `site/robots.txt` et crée `.nojekyll` ;
4. construit la page des bibliothèques avec `vite.libraries.config.ts` (source `site/bibliotheques/`, sortie `site-dist/bibliotheques/`, CSP propre) et y copie `library-data/` s'il existe ;
5. écrit l'archive `portolan-<version>.zip` (nom donné par `archiveName` de `tools/site`) : le contenu de `dist/` sans cartes de sources, sous un dossier `portolan-<version>/`, compressé par fflate ; l'archive n'a pas le marqueur de site (WEB-09).

Aucune option ; la version vient de `package.json`. Le résultat est contrôlé par `tools/check-site` et par `npm run e2e:site`. Le détail du site est au chapitre 15.

16.6 `tools/build-tech-doc`

Point d'entrée de `npm run doc:tech` : compile la documentation technique (DOC-10) avec `compileTechDoc` de `tools/tech-doc-files`. Sans option, le résultat est `docs/technique/build/portolan-technique.pdf`, celui que le build embarque sous `manual/portolan-technique.pdf`. Options :

`--only=<chapitre>[,<chapitre>…]` ne compile que ces chapitres (chemins relatifs à `docs/technique/`, sans `.tex`, par exemple `chapitres/outils,reference/rendu`), au moyen de `\includeonly` ;

`--job=<nom>` compile sous ce nom de tâche, dans `docs/technique/build/<nom>/<nom>.pdf`, pour que plusieurs compilations tournent en même temps sans se gêner (chaque rédacteur ou agent a la sienne).

En cas d'échec de `latexmk`, le script écrit « La compilation de la documentation technique a échoué. » et sort avec le code 1 ; le journal est `docs/technique/build/portolan-technique.log`, ou `docs/technique/build/<job>/<job>.log` avec `--job`. Pas de test propre : la partie pure est testée dans `tools/tech-doc.test.ts`.

16.7 `tools/check-doc`

Point d'entrée de `npm run check:doc` (DOC-09), partie de la définition de fin : compare les identifiants du registre des commandes (`commands` de `src/commands`) avec les identifiants documentés par `\commande{id}` dans `manual/` (`documentedCommandIds` de `tools/manual`). Il échoue (code 1) s'il manque une section pour une commande du registre, ou si une section nomme une commande inexistante, en listant chaque identifiant fautif. Si tout correspond, il compile les deux PDF de l'utilisateur, manuel complet et prise en main rapide (même compilation que `npm run doc`), et échoue si elle échoue. `\commande{id}` n'apparaît que dans le manuel complet. Aucune option. Une nouvelle commande (par exemple `help.technicalManual`, lot 49) demande donc sa section dans le manuel dans le même lot.

16.8 `tools/check-neutral`

Point d'entrée de `npm run check:neutral` (CG-08, ERG-08) : lit la liste des termes interdits `tools/forbidden-terms.txt`, puis cherche chacun d'eux, sur des mots entiers et sans tenir compte de la casse, dans :

- tous les fichiers texte des dossiers `SCANNED_ROOTS` de `tools/neutral` : `src/` (bibliothèque par défaut comprise), `manual/`, `site/`, `public/` et, depuis le lot 49, `docs/technique/` ;
- `index.html` ;
- le projet d'exemple tel que l'application le construit (`createExampleProject`), en JSON, et le texte de chacune de ses images.

Chaque occurrence est affichée (fichier, ligne, terme, extrait) et le script sort avec le code 1. Le dossier `libraries/` (sources des bibliothèques de marques) n'est volontairement pas contrôlé. Tests : `tools/neutral.test.ts`.

16.9 `tools/check-site`

Point d'entrée de `npm run check:site`, à lancer après `npm run site` (en CI et dans le workflow de publication). Il liste tous les fichiers de `site-dist/` et cumule les problèmes donnés par `tools/site` :

- fichiers interdits (`forbiddenSiteFiles`) : cartes de sources, fichiers TypeScript, Markdown, LaTeX, ou tout ce qui est sous `docs/` — le code et les sources de la documentation restent privés (WEB-01) ;
- fichiers obligatoires absents (`requiredSiteFiles`) : application, service worker, manifeste, icône, PDF de documentation (manuel, prise en main rapide depuis le lot 50 et, depuis le lot 49, documentation technique), page des bibliothèques, `robots.txt`, archive ;
- problèmes de contenu des fichiers texte (`.html`, `.css`, `.txt`, `.webmanifest`) relevés par `siteProblems` : `noindex` absent, lien ou ressource externe, marqueur de site absent de `index.html`, `robots.txt` qui n'interdit pas l'indexation.

S'il y a au moins un problème, il les affiche tous et sort avec le code 1. Aucune option.

16.10 `tools/check-tech-doc`

Point d'entrée de `npm run check:tech-doc` (DOC-10), partie de la définition de fin à partir du lot 49 et lancé en CI avant le build. Il rassemble l'inventaire des déclarations exportées de `src/` (`srcSymbols`), la liste des modules à documenter (`documentedModules` : chaque fichier de `src/` et de `tools/`, hors tests) et ce que déclarent les sources LaTeX (`techTex`), tous trois de `tools/tech-doc-files`, puis appelle `coverageProblems` de `tools/tech-doc`. Les problèmes (section manquante, section sans code, section hors de son `\module`, description trop courte, `\voir` vers une clé inconnue, `\voirmodule` vers un module inconnu, doublons) sont listés en français et le script sort avec le code 1. Sinon il compile le PDF complet comme `npm run doc:tech`, puis lit le journal : un caractère absent des polices ou un tableau plus grand que sa page (`layoutProblems`) fait aussi échouer le contrôle, car du contenu manque alors au PDF. Options :

`--only=<module>[,<module>…]` limite le contrôle à ces modules ; un identifiant terminé par `/` prend un dossier entier (`--only=routing/,tools/`) ; les sections, modules et renvois des sources LaTeX sont filtrés de la même façon ;

`--no-pdf` s'arrête après le contrôle de couverture, sans compiler.

C'est l'outil à lancer pendant la rédaction d'un chapitre de référence, avec le dossier concerné et `--no-pdf`.

16.11 `tools/json-schema`

Module d'aide de `tools/build-json-schema` : produit les schémas JSON publiés (FMT-02) à partir des schémas Zod de l'application, avec `z.toJSONSchema` (forme « entrée », brouillon 2020-12, types non représentables remplacés par « n'importe quoi »). Exporte `SCHEMA_DIR` (`public/schemas`) et `jsonSchemas()`, qui renvoie, pour la version de format courante `FORMAT_VERSION`, trois fichiers : `rkdg-project-v<n>.schema.json` (`ProjectSchema`), `rkdglib-library-v<n>.schema.json` (`LibraryDocumentSchema`) et `rkdg-manifest-v<n>.schema.json` (`ManifestSchema`), chacun avec son `$id` et son titre, indenté de deux espaces et terminé par un saut de ligne. Test : `tools/json-schema.test.ts` vérifie que les fichiers de `public/schemas/` sont identiques à ce que produit `jsonSchemas()` ; un changement de format sans `npm run schema` le fait échouer.

16.12 `tools/libavoid-no-eval`

Transformation de la colle JavaScript de libavoid-js pour que l'application tourne sous une CSP sans `'unsafe-eval'` (NF-05 ; décision « Phase F — libavoid-js sans exécution dynamique », 2026-11-05). Le code embind généré par Emscripten fabrique deux sortes de fonctions par `new Function` : les appelants des fonctions C++ (*invokers*) et les appelants de méthodes JavaScript (*emval*). Le module les remplace par deux fermetures ajoutées en tête du module, `__portolanInvoker` et `__portolanMethodCaller`, qui font le même travail que le code généré : ce code n'est plus que lu (pour savoir s'il s'agit d'une méthode, s'il y a des destructeurs, une valeur de retour, quels décalages lire), jamais exécuté. Le WebAssembly n'est pas modifié (licence LGPL-2.1 : ce fichier décrit entièrement la modification).

Exports : `LIBAVOID_VERSION` (`0.5.0-beta.5`, version épinglée pour laquelle les motifs sont écrits), `LIBAVOID_GLUE` (expression qui reconnaît les fichiers de colle du paquet, versions navigateur et Node) et `removeDynamicExecution(code)`. Cette dernière applique chaque remplacement et *échoue* si un motif n'est pas trouvé exactement une fois (version différente de la colle) ou s'il reste un `new Function` ou un `eval(` après transformation : le build échoue alors, au lieu de livrer une application bloquée par la CSP. Utilisé par le greffon `portolan-libavoid-no-eval` de `vite.config.ts`, appliqué au build, au serveur de développement (libavoid-js n'y est pas pré-empaqueté) et aux tests unitaires (module « inline »). Test : `tools/libavoid-no-eval.test.ts` (version installée, plus aucun `new Function` dans les deux colles, refus d'un code inconnu) ; le test du build `tests/build/csp.spec.ts` vérifie le chargement sous la CSP. *Mettre à jour libavoid-js* : changer la version épinglée, adapter les deux expressions de `REPLACEMENTS` et `LIBAVOID_VERSION`, relancer les tests du routage et `npm run e2e:build`.

16.13 `tools/manual`

Module d'aide du manuel utilisateur, utilisé par `tools/build-doc`, `tools/check-doc` et, pour `BASE_DIR`, par les autres outils. Exports :

- `BASE_DIR` (racine du dépôt, calculée depuis l'emplacement du fichier), `MANUAL_DIR` (`manual/`), `MANUAL_PDF` (`manual/build/portolan.pdf`), `QUICK_START_PDF` (`manual/build/prise-en-main-rapide.pdf`) et `MANUAL_ROOTS` (les deux racines LaTeX compilées ensemble, `portolan.tex` et `prise-en-main-rapide.tex`, avec un préambule commun) ;
- `extractCommandIds(tex)` : les identifiants des `\commande{id}` d'une source LaTeX, commentaires ôtés (un `%` non échappé coupe la ligne), espaces autour de l'identifiant retirés, et sans la définition de la macro (argument `#1` ignoré) ;
- `documentedCommandIds()` : l'ensemble de ces identifiants pour tous les fichiers `.tex` de `manual/` (dossiers `build` et `generated` exclus) ;
- `writeVersionFile()` : écrit `manual/generated/version.tex` (macro `\appversion`, version de `package.json`, DOC-05) ;
- `tabIconCommand(id)` : nom de la macro LaTeX qui dessine l'icône d'un onglet du panneau de gauche (`iconeongletlibrary…`) ;
- `tabIconsTex()` : source LaTeX de ces macros, une par onglet, dessinées en TikZ (`svg.path`) depuis les mêmes tracés que l'interface (`ui/panels/tabIcons`), mises à l'échelle à 14 pt et retournées (`yscale=-1`, l'axe des y de TikZ montant) ;
- `writeTabIconsFile()` : écrit `manual/generated/tab-icons.tex` ;

- `compileManual()` : `writeVersionFile`, `writeTabIconsFile` et `writeCommandList` (`tools/manual-commands`), puis `latexmk -xelatex` sur `MANUAL_ROOTS`, sortie affichée ; renvoie vrai si la compilation des deux documents réussit.

Les outils LaTeX (XeLaTeX, latexmk, langue française, polices Inter et DejaVu) doivent être installés (liste dans `CLAUDE.md` et `.github/workflows/ci.yml`). Test : `tools/manual.test.ts` (`extractCommandIds`, icônes des onglets, liste des commandes).

16.14 `tools/manual-commands`

Annexe générée du manuel utilisateur (DOC-02, DOC-09, lot 50) : la liste de toutes les commandes du registre, avec leur libellé, leur identifiant, leurs raccourcis et la page de leur section (`\commande{id}` pose l'étiquette `cmd:<id>`). Écrite dans `manual/generated/commandes.tex` avant chaque compilation, elle ne peut être ni incomplète ni en retard sur l'application. Exports :

- `CommandEntry` : identifiant, libellé en français, raccourcis affichés, zone d'une commande ;
- `ZONE_ORDER` : ordre des zones dans la liste, celui du dialogue des raccourcis (général, menus, barre d'outils, zone de dessin, bibliothèque, explorateur, propriétés, pages, dialogues) ;
- `texEscape(text)` : texte échappé pour LaTeX (`\`, accolades, `$`, `&`, `%`, `#`, `_`, tilde, circonflexe) ;
- `commandListTex(entries, zoneLabels)` : un tableau `longtable` par zone non vide, dans `ZONE_ORDER`, chaque ligne liée à la section de la commande (`\hyperlink`, `\pageref`) ;
- `translate(key)` : texte français d'une clé pointée de `fr.json` ; lève une erreur « Texte introuvable » si la clé manque ;
- `registryEntries()` : les commandes du registre, libellés traduits et raccourcis tels qu'affichés ;
- `writeCommandList(manualDir)` : écrit le fichier généré.

Test : `tools/manual.test.ts`.

16.15 `tools/neutral`

Analyseur de neutralité (CG-08), utilisé par `tools/check-neutral`. Exports :

- `NeutralityHit` : une occurrence (fichier, ligne à partir de 1, terme, extrait de 120 caractères au plus) ;
- `parseTerms(content)` : un terme par ligne, lignes vides et commentaires `#` ignorés ;
- `termPattern(term)` : expression insensible à la casse qui ne reconnaît que des mots entiers (ni lettre ni chiffre Unicode avant ou après) ; un espace du terme accepte aussi plusieurs espaces, un `_`, un tiret ou rien (un terme en deux mots est aussi reconnu écrit attaché ou avec un tiret) ;

- `scanText(file, text, terms)` : les occurrences dans un texte, ligne par ligne ;
- `scanDirectories(baseDir, roots, terms)` : parcourt les dossiers (un dossier absent est ignoré), saute `node_modules`, `build`, `dist`, `.git` et les fichiers binaires (images, PDF, polices, archives, `.rkdg`, `.rkdglib`) ;
- `SCANNED_ROOTS` : `src`, `manual`, `site`, `public`, `docs/technique`.

Test : `tools/neutral.test.ts` (lecture des termes, mots entiers, fichier de test fautif détecté, bibliothèques de marques hors des dossiers contrôlés et hors du build). Ajouter un dossier au contrôle : l'ajouter à `SCANNED_ROOTS` et au test.

16.16 `tools/publish-libraries`

Point d'entrée de `npm run publish:libraries` et module testé (WEB-03 à WEB-05). Pour chaque fichier `.rkdglib` de `libraries/` (ordre alphabétique), il relit l'archive avec `readLibraryArchive` (et un nettoyeur SVG construit sur jsdom), la chiffre telle quelle dans `library-data/files/<id>-v<version>.bin`, et ajoute sa fiche au catalogue : identifiant, nom, marque, version, date, auteur, description, licence, taille, nom de fichier, chemin, et la liste des machines (nom, catégorie, fabricant, référence, hauteur en U). Le catalogue, validé par `CatalogueSchema`, est chiffré à son tour (`CATALOGUE_PATH`). Le chiffrement est celui de `src/libraries/crypto.ts` (AES-256-GCM, clé PBKDF2-SHA-256, sel et IV aléatoires par fichier). Le dossier de sortie est vidé à chaque exécution.

Exports : `LIBRARY_SOURCES`, `LIBRARY_DATA`, `PublishOptions` (`sourceDir`, `outDir`, `password`, `iterations` — par défaut `PBKDF2_ITERATIONS`, réduit dans les tests —, `now`) et `publishLibraries(options)`, qui renvoie le catalogue en clair (pour le journal). Erreurs : mot de passe de moins de 8 caractères ; archive illisible (le message nomme le fichier). Lancé en ligne de commande, le mot de passe est lu dans la variable d'environnement `LIBRARIES_PASSWORD` (secret GitHub Actions, jamais écrit dans le dépôt, WEB-04) ; s'il manque, le script s'arrête avec le code 1. Test : `tools/publish-libraries.test.ts` (chiffrement puis déchiffrement identique, mot de passe trop court refusé, aucun mot de passe écrit dans les fichiers suivis par Git).

16.17 `tools/site`

Fonctions pures du site public (WEB-01, WEB-02, WEB-09), utilisées par `tools/build-site` et `tools/check-site` :

- `archiveName(version)` : `portolan-<version>.zip` ;
- `requiredSiteFiles(version)` : les fichiers que tout site publié doit contenir : `index.html`, `sw.js`, `manifest.webmanifest`, `icon.svg`, `manual/portolan.pdf`, `manual/prise-en-main-rapide.pdf` (lot 50), `manual/portolan-technique.pdf` (lot 49), `bibliotheques/index.html`, `robots.txt` et l'archive ;
- `SiteFile` (chemin et texte d'un fichier) ;
- `forbiddenSiteFiles(paths)` : un message par fichier qui ne doit jamais être publié : carte de source (`.map`), source TypeScript (`.ts`, `.tsx`, `.mts`, `.cts`), et « documentation technique » au sens des *sources* : tout chemin sous `docs/`, tout `.md` ou `.tex`. Le PDF compilé de la documentation technique, sous `manual/`, n'est pas concerné ;

- `siteProblems(files, required)` : fichiers requis absents, page HTML sans `<meta name="robots" content="noindex…">`, attribut `href`, `src` ou `action` vers une autre origine (`//…` ou `schéma://…`), ressource `url(//…)` dans une feuille de style, `index.html` sans le marqueur `SITE_MARKER`, `robots.txt` sans `Disallow: /`.

Test : `tools/site.test.ts`. Ajouter un fichier obligatoire au site : l'ajouter à `requiredSiteFiles` et au test, puis vérifier `npm run e2e:site`.

16.18 `tools/tech-doc`

Partie pure (sans système de fichiers) de la documentation technique (DOC-10). Elle s'appuie sur l'API du compilateur TypeScript pour inventorier les déclarations exportées et sur des expressions régulières pour lire les sources LaTeX. Exports :

- `ExportKind` (fonction, constante, variable, classe, interface, type, énumération, espace de noms), `ExportedSymbol` (`module`, `name`, `key = module.nom`, `kind`, `signature` courte), `SourceFile` ;
- `isDocumentedSource(path)` : fichier `.ts` ou `.tsx` qui n'est ni un test ni une déclaration `.d.ts` ; `moduleId(path)` : chemin sans extension (`model/schema.ts` → `model/schema`) ;
- `exportedSymbols(file)` : les déclarations portant `export` et les déclarations locales listées par `export { … }` ; les réexportations (`export … from`, ou `export { x }` d'un `x` importé) sont documentées là où elles sont déclarées ; la signature est le texte de la déclaration sans modificateurs ni JSDoc, espaces réduits, tronquée pour un type ou une constante ; `inventory(files)` : toutes, triées par clé ;
- `DocumentedSection`, `TexMention`, `TechTex` et `parseTechTex(file, tex)` : lecture d'une source LaTeX (sections `\module`, sections `\fonction` avec leur texte jusqu'à la section, au module ou au titre suivant et le module auquel elles appartiennent, renvois `\voir` et `\voirmodule`), commentaires (un `%` non protégé, même après `\\`), exemples verbatim et définitions de macros ignorés ; `mergeTechTex(parts)` réunit plusieurs sources ; `plainText(tex)` donne le texte brut d'un fragment (sans commandes ni noms d'environnements), pour mesurer une description ;
- `MIN_DESCRIPTION` (40 caractères de texte brut au moins) et `coverageProblems(symbols, modules, tex)` : les problèmes de couverture en français, c'est-à-dire déclaration sans section, section sans déclaration, section en double, section hors du `\module` de son fichier, description vide ou trop courte, module sans section, section de module sans fichier, `\voir` vers une clé inexistante, `\voirmodule` vers un module inexistant ;
- `layoutProblems(log)` : dans le journal de LaTeX, les caractères absents des polices et les flottants plus grands que la page (contenu perdu malgré une compilation réussie) ;
- `escapeLatex(text)` et `signaturesTex(symbols)` : le fichier `generated/signatures.tex`, une ligne `\techsignature{clé}{nature}{signature}` par déclaration, lu par la macro `\fonction` pour afficher nature et signature.

Test : `tools/tech-doc.test.ts` (sources gardées, déclarations exportées sans les réexportations, signatures courtes, tri ; lecture des sources LaTeX, mesure d'une description, chaque sorte de problème de couverture, documentation complète acceptée ; échappement et `signatures.tex` ; cas relevés à la relecture du lot 49 : espaces de noms, exports renommés, commentaires après `\\`, environnements, `\voirmodule`, avertissements du journal). Changer une règle de couverture : modifier `coverageProblems`, son test, et l'en-tête de `docs/technique/macros.tex` qui rappelle les règles aux rédacteurs.

16.19 `tools/tech-doc-files`

Partie « fichiers » de la documentation technique, au-dessus de `tools/tech-doc` ; utilisée par `tools/check-tech-doc` et `tools/build-tech-doc`. Exports :

- `TECH_DIR` (`docs/technique`), `TECH_MAIN` (`portolan-technique`), `TECH_PDF`, `TECH_LOG` (`docs/technique/build/portolan-technique.pdf` et son journal `.log`) ;
- `srcFiles()` et `srcSymbols()` : les sources documentées de `src/` et leurs déclarations exportées ;
- `documentedModules()` : les identifiants de module attendus, ceux de `src/` et `tools/<nom>` pour chaque fichier `.ts` de `tools/` (hors tests ; `publish-site.sh` et `forbidden-terms.txt` n'en sont pas) ;
- `techTexFiles()` et `techTex()` : les sources `.tex` de `docs/technique/` hors `build/` et `generated/`, et ce qu'elles déclarent ;
- `writeGenerated(symbols)` : écrit `generated/version.tex` (`\appversion`) et `generated/signatures.tex` ;
- `CompileOptions` (`only`, `job`) et `compileTechDoc(options)` : écrit les fichiers générés, écrit `generated/only-<job>.tex`, qui définit `\techonly` pour `\includeonly` ou ne contient qu'un commentaire sans `--only` (toujours écrit : changer de sélection change le fichier, et `latexmk` recompile ; un PDF partiel ne passe jamais pour le PDF complet), crée les dossiers de sortie (`build/` pour la tâche principale, `build/<job>/` sinon, avec `chapitres/` et `reference/` pour les `.aux` de chaque `\include`), puis lance `latexmk -xelatex -jobname=<job> -outdir=<dossier> portolan-technique.tex` ; renvoie vrai si la compilation réussit.

Les dossiers `docs/technique/build/` et `docs/technique/generated/` sont ignorés par Git.

16.20 Autres fichiers de `tools/`

`tools/publish-site.sh`. Script Bash lancé par la dernière étape du workflow Publication (`.github/workflows/publish.yml`). Il pousse le contenu de `site-dist/` dans le dépôt public servi par GitHub Pages (WEB-07). Variables attendues : `SITE_DEPLOY_KEY`, la clé SSH privée de déploiement, avec droit d'écriture sur ce seul dépôt (secret GitHub Actions), et `SITE_REPOSITORY` (`propriétaire/nom`). Étapes : arrêt si une variable manque ou si `site-dist/index.html` est absent ; clé écrite dans un fichier temporaire (mode 600, supprimé à la sortie) ; empreinte de l'hôte GitHub ajoutée aux hôtes connus ; clone superficiel du dépôt public, branche `main` ; tout le contenu (sauf `.git`) remplacé par `site-dist/`, et `site/README.public.md` copié en `README.md` ; si rien ne change, « Site inchangé. » et fin ; sinon un commit « Publication de la version <version> (<commit>) » signé « Publication Portolan » et un `push`. Aucune action tierce n'est utilisée.

`tools/forbidden-terms.txt`. Liste des termes interdits du contrôle de neutralité (CG-08, REC-09) : un terme par ligne, comparaison insensible à la casse sur des mots entiers ; lignes vides et lignes commençant par `#` ignorées. Les termes sont groupés par domaine (réseau et sécurité ; serveurs, postes et stockage ; énergie et infrastructure de baie ; audiovisuel et périphériques ; éditeurs de logiciels). C'est le seul fichier du cœur qui nomme des marques ; il n'est pas contrôlé lui-même (`tools/` n'est pas dans `SCANNED_ROOTS`). Pour ajouter un terme : une ligne dans la bonne rubrique, puis `npm run check:neutral`.

Deuxième partie

Référence du code

Chapitre 17

Lire la référence

Cette partie décrit chaque module de `src/` et chacune de ses déclarations exportées, regroupés par domaine :

Chapitre	Modules
Modèle de données	`model/` : schémas, identifiants, fabriques, recherches, validation, états des ports, modèles de projet, purge.
Règles métier	`model/` : placement, géométrie des armoires, cible du dépôt, rangement, disposition automatique, ports des instances, liaisons, anomalies.
Pages et vues	`model/` : annotations, édition et transfert de bibliothèques, arborescence, éditeur de modèle, géométrie et disposition des pages, recherche, cartouche.
Bibliothèques et application	`library/`, `libraries/`, `app/`, `help/`, `i18n/`, `main`, `app-version`.
Routage et texte	`routing/`, `text/`.
Rendu et jeux d'essai	`render/`, `testing/`.
Fichiers et import	`io/`, `import/`.
Exports	`export/`.
État et commandes	`store/`, `commands/`.
Interface	`ui/`.

Pour chaque déclaration, l'en-tête donne le nom, la nature et la signature extraite du code ; la suite est rédigée (voir l'introduction, « Lire une entrée de la référence »). Les noms des paramètres sont ceux du code. Sauf mention contraire, les fonctions ne modifient pas leurs arguments : les modifications du projet passent par les actions du magasin d'état, qui travaillent sur un brouillon Immer.

Chapitre 18

Référence : modèle de base

Ce chapitre documente les modules de base du modèle (`src/model/`) : schémas et types de toutes les entités, identifiants, fabriques, catégories d'équipements, index, validation, états des ports, révisions et projet modèle, purge. La conception d'ensemble est expliquée au chapitre « Modèle de données » (page 21). Toutes les fonctions de ces modules sont pures, sauf la génération d'identifiants (aléatoire) et la date du jour ; aucune ne dépend de React.

18.1 `model/equipment`

Équipements intégrés (EQT-01) : catégorie de chaque type d'équipement et prédicats sur une machine qui ne dépendent que de son type (extérieur, 0U) ou de ses images. Dépend uniquement de `model/schema`. Les paramètres, générateurs et formulaires du catalogue sont dans `src/library/equipment/`. Utilisé par la validation, les fabriques, la purge, la géométrie des pages et des armoires, l'arbre de la bibliothèque (`model/libraryTree`), le panneau bibliothèque et l'éditeur de machine.

`WithKind` interface

```
interface WithKind { equipment }
```

Forme minimale de tout ce qui nomme son type d'équipement : une machine, ou une machine en cours d'édition dans l'interface. Permet aux prédicats de ce module de s'appliquer sans exiger une `Machine` complète.

Champs

`equipment.kind` type d'équipement intégré (`EquipmentKind`).

`machineImageRefs` fonction

```
function machineImageRefs(machine: Pick<Machine, 'front' | 'rear' | 'logo'> &
Partial<Pick<Machine, 'carried'>>): ImageRef[]
```

Images d'une machine : photos de ses faces avant et arrière, logo (EDM-02, EDM-10) et image de l'objet posé sur une étagère (`carried`, MAC-03), chacune une seule fois. Utilisée par la purge, le calcul du poids du fichier, la collecte des images de l'archive (`src/io/images.ts`) et l'import / export de bibliothèques.

Paramètres

`machine` machine (ou partie de machine) dont on lit `front.imageId`, `rear.imageId`, `logo` et, s'il est présent, `carried`.

Résultat. liste sans doublon des références d'images (`ImageRef`), dans l'ordre avant, arrière, logo, objet posé ; vide si la machine n'a aucune image.

Erreurs. Aucune.

`equipmentCategory` fonction

```
function equipmentCategory(kind: EquipmentKind): EquipmentCategory
```

Catégorie de rangement d'un type d'équipement (réseau, serveurs et stockage, alimentation, accessoires de baie, extérieurs, générique), lue dans `KIND_CATEGORY`.

Paramètres

`kind` type d'équipement.

Résultat. la catégorie (`EquipmentCategory`).

Erreurs. Aucune.

`machineCategory` fonction

```
function machineCategory(machine: WithKind): EquipmentCategory
```

Catégorie d'une machine, déduite de son équipement : la catégorie n'est pas stockée dans le fichier depuis le format 2 (lot 34).

Paramètres

`machine` machine ou objet `WithKind`.

Résultat. la catégorie de `machine.equipment.kind`.

Erreurs. Aucune.

`kindsOfCategory` fonction

```
function kindsOfCategory(category: EquipmentCategory): EquipmentKind[]
```

Types d'équipement d'une catégorie, dans l'ordre du catalogue (`EQUIPMENT_KINDS`). Sert à construire l'arbre du catalogue.

Paramètres

`category` catégorie recherchée.

Résultat. nouveau tableau des types de la catégorie (jamais vide pour une catégorie existante).

Erreurs. Aucune.

`equipmentTree` fonction

```
function equipmentTree(): { category: EquipmentCategory; kinds: EquipmentKind[] }[]
```

Arbre du catalogue des équipements (BIB-01) : les catégories dans l'ordre d'affichage (`EQUIPMENT_CATEGORIES`), chacune avec ses types. Utilisé par l'arbre de la bibliothèque et par l'éditeur de machine.

Résultat. tableau de `{ category, kinds }`, une entrée par catégorie.

Erreurs. Aucune.

`isExternalKind` fonction

```
function isExternalKind(kind: EquipmentKind): boolean
```

Indique si un type d'équipement est extérieur (catégorie `external` : écran, poste, portable, clavier et souris, pupitre), c'est-à-dire placé hors des armoires (EXT-xx). Utilisée par `createMachine` pour choisir ses valeurs par défaut.

Paramètres

`kind` type d'équipement.

Résultat. `true` pour un type extérieur.

Erreurs. Aucune.

`isExternal` fonction

```
function isExternal(machine: WithKind): boolean
```

Indique si une machine est un équipement extérieur, d'après son type. Un équipement placé d'une telle machine a un placement `external` (vérifié par `checkReferences`).

Paramètres

`machine` machine ou objet `WithKind`.

Résultat. `true` pour une machine extérieure.

Erreurs. Aucune.

`isZeroU` fonction

```
function isZeroU(machine: WithKind & Pick<Machine, 'heightU'>): boolean
```

Indique si une machine est un équipement vertical 0U (multiprise verticale) : non extérieure et de hauteur nulle. Ses équipements placés ont un placement `zeroU`.

Paramètres

`machine` objet portant `equipment.kind` et `heightU`.

Résultat. `true` pour une machine 0U.

Erreurs. Aucune.

`zeroULengthU` fonction

```
function zeroULengthU(machine: Pick<Machine, 'equipment' | 'heightU'>): number | null
```

Longueur en U d'un équipement vertical 0U (STD-01), lue dans le paramètre `lengthU` de la machine. Utilisée par la géométrie des pages et des armoires, la cible de dépôt, le panneau bibliothèque et le dessin des équipements standards.

Paramètres

`machine` objet portant `equipment` (avec ses paramètres) et `heightU`.

Résultat. la longueur (nombre fini strictement positif), ou `null` si la machine n'est pas 0U ou n'a pas de longueur valable : le dessin occupe alors tout l'emplacement 0U.

Erreurs. Aucune.

18.2 `model/factories`

Fabriques d'entités : chaque fonction crée une entité neuve avec un nouvel identifiant (`newId`) et des valeurs par défaut, puis la fait passer par son schéma Zod (`parse`) : le résultat est toujours valide et complet. Les textes par défaut viennent de `fr.json` (`defaults.*`). Aucune fabrique ne génère de ports ni ne lit le catalogue : la génération des machines est dans `src/library/`. Utilisé par les actions du magasin (`src/store/`), la bibliothèque par défaut, l'import, la mise en page automatique et les tests. Toutes les fabriques qui reçoivent des valeurs lèvent une `ZodError` si ces valeurs violent le schéma.

`USABLE_DEPTH_MARGIN_MM` constante

```
const USABLE_DEPTH_MARGIN_MM = 200
```

Marge en millimètres retranchée de la profondeur extérieure d'une armoire pour proposer sa profondeur utile (« profondeur − 200 mm », § 6.3). Vaut 200 ; utilisée par `defaultUsableDepth`.

`defaultUsableDepth` fonction

```
function defaultUsableDepth(depthMm: number): number
```

Profondeur utile proposée pour une armoire de profondeur donnée : profondeur moins `USABLE_DEPTH_MARGIN_MM`, au moins 1 mm pour rester valide (EQP-02 compare les équipements à cette valeur).

Paramètres

`depthMm` profondeur extérieure, en millimètres.

Résultat. la profondeur utile, en millimètres (≥ 1).

Erreurs. Aucune.

`today` fonction

```
function today(now: Date = new Date()): string
```

Date du jour au format `AAAA-MM-JJ`, en heure locale (et non UTC, pour ne pas changer de jour tard le soir). Sert de date par défaut à `createRevision`.

Paramètres

`now` instant de référence ; par défaut, maintenant (paramètre utile aux tests).

Résultat. la date, conforme à `IsoDateSchema`.

Erreurs. Aucune.

`createEmptyLibrary` fonction

```
function createEmptyLibrary(): Library
```

Bibliothèque vide : aucun type de connecteur, type de liaison, modèle d'armoire ni machine. Point de départ de `createProject` et de la bibliothèque par défaut (`createDefaultLibrary`, `src/library/`).

Résultat. une nouvelle `Library` vide.

Erreurs. Aucune.

`createRack` fonction

```
function createRack(input: RackInput = {}): Rack
```

Crée une armoire. Valeurs par défaut : nom « Armoire », 42 U, 600 mm de large, 800 mm de profondeur, profondeur utile calculée par `defaultUsableDepth` à partir de la profondeur donnée ; les autres champs prennent les valeurs du schéma.

Paramètres

`input` champs de l'armoire à imposer (tous facultatifs, sauf `id` qui est toujours neuf).

Résultat. la nouvelle `Rack`, `modelId` à `null`.

Erreurs. `ZodError` si les valeurs sont invalides (hauteur hors de 1 à 52 U, profondeur utile supérieure à la profondeur…).

```
const rack = createRack({ name: 'Baie 01', heightU: 24, depthMm: 1000 });
// rack.usableDepthMm === 800
```

`createRackModel` fonction

```
function createRackModel(input: Partial<Omit<RackModel, 'id'>> = {}): RackModel
```

Crée un modèle d'armoire de bibliothèque, avec les mêmes valeurs par défaut que `createRack`. Utilisée par la bibliothèque par défaut et l'éditeur de modèles d'armoires.

Paramètres

`input` caractéristiques à imposer (facultatives).

Résultat. le nouveau `RackModel`.

Erreurs. `ZodError` si les valeurs sont invalides.

`createRackFromModel` fonction

```
function createRackFromModel(model: RackModel, name?: string): Rack
```

Crée une armoire à partir d'un modèle d'armoire : copie de toutes ses caractéristiques, nouvel identifiant, `modelId` égal à l'identifiant du modèle (information seulement). Appelée par les actions d'armoires du magasin.

Paramètres

`model` modèle d'armoire source.

`name` nom de la nouvelle armoire ; par défaut, celui du modèle.

Résultat. la nouvelle `Rack`.

Erreurs. `ZodError` si le modèle est invalide (ne se produit pas avec un modèle validé).

`PortInput` type

```
type PortInput = Partial<Omit<Port, 'id'>> & Pick<Port, 'name' | 'connectorTypeId'>;
```

Valeurs acceptées par `createPort` : tous les champs d'un port sauf `id`, facultatifs, à l'exception de `name` et `connectorTypeId` qui sont obligatoires.

`createPort` fonction

```
function createPort(input: PortInput): Port
```

Crée un port de machine. Position par défaut au centre de la face (`x` = `y` = 0,5), taille 0,05 × 0,3 de la face ; les autres champs prennent les valeurs du schéma (sortie automatique, non placé…). Utilisée par les jeux d'essai (`src/testing/fixtures.ts`) ; les générateurs du catalogue et l'éditeur de machine construisent leurs ports eux-mêmes.

Paramètres

`input` nom, type de connecteur et champs facultatifs (`PortInput`).

Résultat. le nouveau `Port`.

Erreurs. `ZodError` si une proportion sort de 0 à 1 ou si le nom est vide.

`MachineInput` type

```
type MachineInput = Partial<Omit<Machine, 'id' | 'equipment'>> & { name: string; equipment:
{ kind: EquipmentKind; params?: Record<string, unknown> }; };
```

Valeurs acceptées par `createMachine` : champs facultatifs d'une machine (sauf `id`), avec un `name` obligatoire et un `equipment` dont seul `kind` est obligatoire (`params` vaut `{}` par défaut).

`createMachine` fonction

```
function createMachine(input: MachineInput): Machine
```

Crée une machine avec les champs donnés, telle quelle, sans génération : les ports et dimensions ne sont pas déduits des paramètres. Par défaut, `heightU` vaut 1 (0 pour un extérieur) et `displaySizeMm` vaut 300 × 200 mm pour un extérieur, `null` sinon. Pour une machine conforme à ses paramètres, utiliser `generateMachine` ou `createGenericMachine` (`src/library/`).

Paramètres

`input` nom, équipement et champs facultatifs (`MachineInput`).

Résultat. la nouvelle `Machine`.

Erreurs. `ZodError` si les valeurs sont invalides (nom vide, hauteur hors bornes, extérieur sans taille affichée).

```
const server = createMachine({
  name: 'Serveur 2U',
  equipment: { kind: 'server' },
  heightU: 2,
});
```

`DeviceInput` type

```
type DeviceInput = Partial<Omit<Device, 'id' | 'machineId' | 'placement'>> & { name:
string; };
```

Valeurs acceptées par `createDevice` : champs d'un équipement placé sauf `id`, `machineId` et `placement` (passés à part), tous facultatifs sauf `name`.

`createDevice` fonction

```
function createDevice(machineId: Id, placement: Placement, input: DeviceInput): Device
```

Crée un équipement placé (instance d'une machine). Ne vérifie pas que la place est libre ni que le placement convient à la machine : c'est le rôle des règles de placement (`canPlace`, `model/placement`) et de la validation.

Paramètres

- `machineId` identifiant de la machine dans la bibliothèque du projet.
- `placement` placement complet (`Placement`).
- `input` nom et champs facultatifs (`DeviceInput`).

Résultat. le nouveau `Device`.

Erreurs. `ZodError` si le nom est vide ou le placement invalide (U inférieure à 1…).

```
const device = createDevice(
  server.id,
  { kind: 'rack', rackId: rack.id, uBottom: 10,
    mountFace: 'front', halfSide: null },
  { name: 'srv-01' },
);
```

`LinkInput` type

```
type LinkInput = Partial<Omit<Link, 'id' | 'a' | 'b' | 'linkTypeId'>>;
```

Valeurs facultatives acceptées par `createLink` : tous les champs d'une liaison sauf `id`, les extrémités `a`, `b` et le type de liaison.

`createLink` fonction

```
function createLink(a: PortRef, b: PortRef, linkTypeId: Id, input: LinkInput = {}): Link
```

Crée une liaison entre deux ports. Le repère vaut la chaîne vide par défaut : l'action de création le calcule (`nextLinkRef`, `model/links`). Ne contrôle ni la compatibilité ni l'occupation des ports.

Paramètres

- `a` première extrémité (équipement et port).
- `b` seconde extrémité.
- `linkTypeId` type de liaison.
- `input` champs facultatifs (`LinkInput`).

Résultat. la nouvelle `Link`.

Erreurs. `ZodError` si une valeur est invalide (longueur négative, couleur mal formée…).

`createPage` fonction

```
function createPage(input: Partial<Omit<Page, 'id'>> = {}): Page
```

Crée une page : titre « Page » par défaut, A4 paysage, marges de 10 mm, sans vue ni annotation (valeurs du schéma).

Paramètres

`input` champs à imposer (facultatifs).

Résultat. la nouvelle `Page`.

Erreurs. `ZodError` si les valeurs sont invalides (format personnalisé sans taille…).

`ViewInput` type

```
type ViewInput = Partial<Omit<View, 'id'>> & Pick<View, 'kind' | 'targetId'>;
```

Valeurs acceptées par `createView` : champs facultatifs d'une vue (sauf `id`), avec `kind` et `targetId` obligatoires.

`defaultLabelsBeside` fonction

```
function defaultLabelsBeside(kind: View['kind']): boolean
```

Réglage par défaut des étiquettes à côté de l'armoire pour une vue neuve (PAG-17) : oui pour une vue avant, non pour une vue arrière ou extérieure. Un fichier qui ne connaît pas le réglage garde la valeur du schéma (masquées). Utilisée par `createView` et la mise en page automatique.

Paramètres

`kind` nature de la vue.

Résultat. `true` pour `rackFront`.

Erreurs. Aucune.

`createView` fonction

```
function createView(input: ViewInput): View
```

Crée une vue de page, en position (0, 0) par défaut, avec les étiquettes réglées par `defaultLabelsBeside` si `labelsBeside` n'est pas donné.

Paramètres

`input` nature, cible et champs facultatifs (`ViewInput`).

Résultat. la nouvelle `View`.

Erreurs. `ZodError` si les valeurs sont invalides (échelle nulle, plage de U décroissante…).

`createRevision` fonction

```
function createRevision(input: Partial<Omit<Revision, 'id'>> = {}): Revision
```

Crée une révision (PRJ-11) : indice « A » et date du jour (`today`) par défaut. L'action d'ajout de révision passe l'indice proposé par `nextRevisionIndex`.

Paramètres

`input` indice, date, auteur, description (facultatifs).

Résultat. la nouvelle `Revision`.

Erreurs. `ZodError` si l'indice est vide ou la date mal formée.

`createTitleBlockField` fonction

```
function createTitleBlockField(input: Partial<Omit<TitleBlockField, 'id'>> &
Pick<TitleBlockField, 'kind'>): TitleBlockField
```

Crée un champ de cartouche (PAG-10), visible, à la place par défaut de sa nature (`row` à `null`). Utilisée par l'action d'ajout de champ libre et par `createDefaultTitleBlockFields`.

Paramètres

`input` nature (`kind`, obligatoire), libellé, valeur, visibilité, rangée.

Résultat. le nouveau `TitleBlockField`.

Erreurs. `ZodError` si la nature est inconnue.

`createDefaultTitleBlockFields` fonction

```
function createDefaultTitleBlockFields(): TitleBlockField[]
```

Champs du cartouche d'un projet neuf (PAG-09) : logo, nom du projet, titre de page, révision, date de révision, auteur, numéro de page. Le champ logo est présent mais aucune image n'est fournie (CG-08) : le logo vient de l'utilisateur.

Résultat. sept nouveaux `TitleBlockField`.

Erreurs. Aucune.

`CreateProjectOptions` interface

```
interface CreateProjectOptions { name?; author?; library?; now? }
```

Options de `createProject`.

Champs

`name` nom du projet ; par défaut « Nouveau projet ».

`author` auteur ; par défaut vide.

`library` bibliothèque du projet ; par défaut vide.

`now` date de création ; par défaut maintenant.

`createProject` fonction

```
function createProject(options: CreateProjectOptions = {}): Project
```

Crée un projet vide au format courant : métadonnées, bibliothèque donnée (vide par défaut), cartouche par défaut, aucune armoire ni page. L'application utilise `createNewProject` (bibliothèque par défaut) et `createStarterProject` (plus une page A4), dans `src/library/index.ts`.

Paramètres

`options` voir `CreateProjectOptions`.

Résultat. le nouveau `Project`.

Erreurs. `ZodError` si la bibliothèque passée est invalide.

```
const project = createProject({ name: 'Salle serveurs' });
project.racks.push(createRack({ name: 'Baie 01' }));
```

18.3 `model/ids`

Génération des identifiants. Toutes les entités ont un UUID ; les données intégrées (types de connecteurs et de liaisons par défaut, machines génériques et leurs ports) ont des identifiants fixes, identiques dans tous les projets (décisions du 04/11/2026, lot 01, et du 05/11/2026, lot 34). Dépend seulement du type `Id`.

`newId` fonction

```
function newId(): Id
```

Tire un UUID aléatoire version 4 (RFC 4122). Utilise `crypto.randomUUID` quand il existe ; sinon (contexte non sécurisé, page servie en HTTP), construit l'UUID à partir de `crypto.getRandomValues` en fixant les bits de version et de variante.

Résultat. un identifiant conforme à `IdSchema`.

Erreurs. Aucune.

`builtinId` fonction

```
function builtinId(n: number): Id
```

Identifiant stable d'une donnée intégrée : `00000000-0000-4000-8000-` suivi de `n` en hexadécimal sur 12 chiffres. Plages utilisées : `0x1xx` pour les types de connecteurs, `0x2xx` pour les types de liaisons, `0x500` + rang pour les machines génériques, `0x10000` + rang × `0x1000` + `n` pour leurs ports (`src/library/`).

Paramètres

`n` numéro de la donnée (entier positif, au plus 12 chiffres hexadécimaux).

Résultat. l'identifiant, valide pour `IdSchema`.

Erreurs. Aucune.

```
builtinId(0x101); // '00000000-0000-4000-8000-000000000101'
```

18.4 `model/index`

Point d'entrée du modèle : réexporte `model/schema`, `model/ids`, `model/factories`, `model/lookup`, `model/validate`, `model/placement`, `model/links`, `model/anomalies`, `model/purge`, `model/rackGeometry`, `model/pageLayout`, `model/equipment` et `model/titleBlockLayout`. Les autres modules du dossier (géométrie des pages, routage des liaisons, recherche…) s'importent par leur chemin. Le module ne déclare rien lui-même.

18.5 `model/lookup`

Recherches rapides par identifiant et résolution des références de ports. L'index d'un projet est mis en cache par objet projet (`WeakMap`) : avec les mises à jour immuables (Immer), un projet inchangé réutilise son index ; une liste indexée modifiée en place (changement de référence ou de longueur) l'invalide (décision du 04/11/2026, lot 02). Dépend de `model/schema`. Utilisé par presque tout le reste : règles de liaison et d'anomalies, géométrie, routage, rendu, exports, import, interface.

`PortEntry` interface

```
interface PortEntry { port; face }
```

Port d'une machine accompagné de la face qui le porte : les ports sont rangés par face dans la machine, cette entrée évite de chercher la face ensuite.

Champs

`port` le port.

`face` face de la machine (`front` ou `rear`).

`ProjectIndex` interface

```
interface ProjectIndex { racks; devices; links; machines; connectorTypes; linkTypes;
machinePorts; linksByPort }
```

Index d'un projet, construit par `indexProject`. Toutes les tables sont en lecture seule pour l'appelant.

Champs

`racks` armoires par identifiant.

`devices` équipements placés par identifiant.

`links` liaisons par identifiant.

`machines` machines de la bibliothèque par identifiant.

`connectorTypes` types de connecteurs par identifiant.

`linkTypes` types de liaisons par identifiant.

`machinePorts` ports de chaque machine : identifiant de machine, puis identifiant de port, vers une `PortEntry`. Ne contient pas les ports ajoutés aux équipements (PRT-04).

`linksByPort` liaisons attachées à chaque port d'équipement, sous la clé `portKey` ; une liaison dont les deux extrémités sont sur le même port n'y figure qu'une fois.

`portKey` fonction

```
function portKey(ref: PortRef): string
```

Clé textuelle d'un port d'équipement placé, `deviceId/portId`, utilisée comme clé de table (`linksByPort`, états des ports, routage).

Paramètres

`ref` référence de port.

Résultat. la clé.

Erreurs. Aucune.

`machinePortEntries` fonction

```
function machinePortEntries(machine: Machine): PortEntry[]
```

Tous les ports d'une machine avec leur face : ceux de la face avant, puis ceux de la face arrière, dans l'ordre des listes.

Paramètres

`machine` la machine.

Résultat. nouveau tableau de `PortEntry`.

Erreurs. Aucune.

`facePorts` fonction

```
function facePorts(machine: Machine, device: Pick<Device, 'extraPorts'> | undefined, face:
FaceSide): Port[]
```

Ports d'une face d'un équipement placé : ceux de sa machine, puis ceux ajoutés à l'équipement sur cette face (PRT-04). Utilisée par le routage et les états des ports.

Paramètres

`machine` machine de l'équipement.

`device` équipement placé (seul `extraPorts` est lu) ; `undefined` pour les seuls ports de la machine.

`face` face de la machine.

Résultat. la liste des ports ; c'est la liste même de la machine (même référence) quand l'équipement n'ajoute aucun port sur cette face, ce qui préserve la mémoïsation.

Erreurs. Aucune.

`instancePorts` fonction

```
function instancePorts(device: Pick<Device, 'extraPorts'>, face: FaceSide): Port[]
```

Ports ajoutés à un équipement placé seulement (PRT-04), sur une face, sans leur champ `face`.

Paramètres

`device` équipement placé (seul `extraPorts` est lu).

`face` face recherchée.

Résultat. les ports ; une liste vide partagée (toujours la même référence) quand l'équipement n'a aucun port ajouté, pour que les dessins mémoïsés gardent leurs propriétés.

Erreurs. Aucune.

`devicePortEntries` fonction

```
function devicePortEntries(machine: Machine, device: Pick<Device, 'extraPorts'>
| undefined): PortEntry[]
```

Tous les ports d'un équipement placé avec leur face : ceux de la machine d'abord (`machinePortEntries`), puis ceux ajoutés à l'équipement. Utilisée par l'explorateur, la fenêtre de liaison, la recherche, les exports tableur et l'import.

Paramètres

`machine` machine de l'équipement.

`device` équipement placé, ou `undefined` pour les seuls ports de la machine.

Résultat. nouveau tableau de `PortEntry`.

Erreurs. Aucune.

`deviceHasPort` fonction

```
function deviceHasPort(project: Project, device: Device, portId: Id): boolean
```

Indique si un port existe sur un équipement placé : port de sa machine ou port ajouté à l'équipement. Utilisée par la validation (surcharges, extrémités de liaisons) et par l'état de l'interface.

Paramètres

`project` le projet (pour l'index des ports des machines).

`device` l'équipement placé.

`portId` identifiant du port.

Résultat. `true` si le port existe.

Erreurs. Aucune.

`indexProject` fonction

```
function indexProject(project: Project): ProjectIndex
```

Construit, ou retrouve dans le cache, l'index d'un projet (`ProjectIndex`). Le coût est linéaire en taille du projet au premier appel, constant ensuite tant que le projet (objet et listes indexées) ne change pas.

Paramètres

`project` le projet.

Résultat. l'index ; ne pas le modifier, il est partagé.

Erreurs. Aucune.

```
const index = indexProject(project);
const machine = index.machines.get(device.machineId);
const key = portKey({ deviceId: device.id, portId });
const linked = index.linksByPort.get(key) ?? [];
```

`resolvePort` fonction

```
function resolvePort(project: Project, ref: PortRef): { device: Device; machine: Machine;
port: Port; face: FaceSide } | undefined
```

Résout une référence de port vers son équipement, sa machine, le port et sa face. Le port est cherché parmi ceux de la machine, puis parmi ceux ajoutés à l'équipement (PRT-04). Utilisée par les liaisons, les anomalies, la géométrie des pages, l'info-bulle des ports, les exports et l'inspection.

Paramètres

`project` le projet.

`ref` la référence (équipement et port).

Résultat. `{ device, machine, port, face }`, ou `undefined` si la référence est pendante (équipement, machine ou port absent) : c'est le cas d'une liaison sur un port disparu (LIA-07).

Erreurs. Aucune.

`portDisplayName` fonction

```
function portDisplayName(device: Device, port: Port): string
```

Nom d'un port tel qu'affiché pour un équipement placé : le nom surchargé dans `portOverrides` s'il existe (§ 6.6), sinon le nom du port.

Paramètres

`device` l'équipement placé.

`port` le port.

Résultat. le nom affiché.

Erreurs. Aucune.

18.6 `model/portStates`

Ports libres et occupés dessinés sur une page (PRT-03), pour le mode d'affichage qui les met en évidence. Fonction pure ; dépend de `model/lookup` et de la géométrie des pages (`deviceOnPage`, `portBox`, `model/pageGeometry`). Utilisé par le calque `src/ui/canvas/Highlights.tsx`.

`PortState` interface

```
interface PortState { deviceId; portId; viewId; box; occupied; disabled }
```

État d'un port dessiné dans une vue d'une page.

Champs

`deviceId` équipement placé.

`portId` port (de la machine ou ajouté à l'équipement).

`viewId` vue dans laquelle le port est dessiné.

`box` rectangle du port sur la page, en millimètres de papier.

`occupied` au moins une liaison est attachée au port (même si elle est pendante à l'autre bout).

`disabled` port désactivé sur cet équipement (§ 6.6) : ni libre ni occupé pour l'affichage.

`portStatesOnPage` fonction

```
function portStatesOnPage(project: Project, page: Page): PortState[]
```

Liste tous les ports dessinés sur une page avec leur état (PRT-03), dans l'ordre de dessin : équipements dans l'ordre du projet, puis chaque apparition de l'équipement sur la page (un équipement montré par deux vues apparaît deux fois), puis les ports de la face montrée. Les équipements dont la machine manque sont ignorés, ainsi que les vues extérieures dont les ports sont masqués (EXT-03). Chaque cadre est placé dans le rectangle entier de l'équipement (`fullBox`), comme le port dessiné ; sur une vue limitée à une plage de U (PAG-16), un port dont le centre est coupé est omis (`centreVisible`) : les cadres coïncident avec `portOnPage`.

Paramètres

`project` le projet.

`page` la page.

Résultat. tableau de `PortState`.

Erreurs. Aucune.

18.7 `model/projectTemplate`

Révisions (PRJ-11) et « Nouveau projet à partir d'un projet existant » (PRJ-10). Fonctions pures ; dépend des fabriques et de `model/ids`. Utilisé par les actions de projet du magasin (`src/store/projectActions.ts`) et par la session (`src/app/session.ts`).

`nextRevisionIndex` fonction

```
function nextRevisionIndex(revisions: readonly Pick<Revision, 'index'>[]): string
```

Indice proposé pour la révision suivante (PRJ-11), d'après le dernier indice de la liste : un nombre final est incrémenté en gardant sa largeur (« B2 » → « B3 », « 09 » → « 10 ») ; des lettres seules avancent comme un compteur (« A » → « B », « Z » → « AA », « AZ » → « BA »), en gardant la casse ; tout autre indice reçoit un « + » final.

Paramètres

`revisions` révisions existantes, dans l'ordre (seul `index` est lu).

Résultat. l'indice proposé ; « A » pour la première révision (liste vide ou dernier indice blanc).

Erreurs. Aucune.

```
nextRevisionIndex([{ index: 'A' }, { index: 'B' }]); // 'C'
nextRevisionIndex([{ index: 'v9' }]); // 'v10'
```

`FromExistingOptions` interface

```
interface FromExistingOptions { keepContent; now? }
```

Options de `projectFromExisting`.

Champs

`keepContent` reprendre aussi armoires, équipements, liaisons et pages (« tout reprendre ») ; sinon le projet repart sans contenu.

`now` date de création du nouveau projet ; par défaut maintenant.

`projectFromExisting` fonction

```
function projectFromExisting(source: Project, options: FromExistingOptions): Project
```

Nouveau projet à partir d'un projet existant (PRJ-10) : copie profonde, nouvel identifiant, nouvelle date de création, aucune révision. Bibliothèque, types, cartouche, métadonnées et réglages de routage sont repris. Sans `keepContent`, armoires, équipements et liaisons sont vidés et les pages remplacées par une seule page vide « Page 1 » reprenant format, taille, orientation, marges, légende et tableau des révisions de la première page source.

Paramètres

`source` projet existant (non modifié).

`options` voir `FromExistingOptions`.

Résultat. le nouveau projet. Les identifiants des entités reprises sont conservés (ils sont uniques dans le nouveau projet).

Erreurs. Aucune.

18.8 `model/purge`

« Purger la bibliothèque » (BIB-05) : retire toutes les machines qu'aucun équipement placé n'utilise, sauf les machines génériques (EQT-02), et estime le gain de taille. Fonction pure ; dépend de `model/equipment`. Utilisé par les actions de bibliothèque du magasin et la fenêtre de purge (`src/ui/dialogs/LibraryDialogs.tsx`).

`PurgeResult` interface

```
interface PurgeResult { project; removedMachineIds; removedImages; savedBytes }
```

Résultat d'une purge.

Champs

`project` nouveau projet sans les machines retirées (le reste est partagé avec le projet d'origine).

`removedMachineIds` identifiants des machines retirées.

`removedImages` images qui ne servaient qu'aux machines retirées (ni à une machine gardée, ni au logo du projet, ni à une annotation image).

`savedBytes` gain estimé en octets : taille de ces images plus taille JSON (UTF-8, non compressée) des machines retirées.

`isPurgeable` fonction

```
function isPurgeable(project: Project, machine: Machine): boolean
```

Indique si une machine serait retirée par une purge : ni générique, ni utilisée par un équipement placé. Même critère que `purgeUnusedMachines` ; n'est appelée aujourd'hui que par les tests.

Paramètres

`project` le projet.

`machine` la machine.

Résultat. `true` si la machine est purgeable.

Erreurs. Aucune.

`purgeUnusedMachines` fonction

```
function purgeUnusedMachines(project: Project, imageSize: (ref: ImageRef) => number
| undefined = () => 0): PurgeResult
```

Calcule la purge de la bibliothèque. Les modèles d'armoires et les types de connecteurs et de liaisons ne sont jamais retirés (décision du 04/11/2026, lot 04). Les images ne sont pas supprimées ici : l'archive n'écrit à l'enregistrement que les images référencées.

Paramètres

`project` le projet (non modifié).

`imageSize` taille en octets d'une image du projet, `undefined` si elle est inconnue (comptée 0) ; par défaut, 0 pour toutes.

Résultat. un `PurgeResult`.

Erreurs. Aucune.

```
const { project: purged, removedMachineIds, savedBytes } =
  purgeUnusedMachines(project, (ref) => images.get(ref)?.byteLength);
```

18.9 `model/schema`

Modèle de données d'un projet Portolan (§ 6 du cahier) sous forme de schémas Zod ; les types TypeScript en sont déduits par `z.infer`, si bien que validation et typage ne divergent jamais. Les dimensions sont en millimètres, les hauteurs d'équipement en U. Les champs facultatifs ont une valeur par défaut, pour que les fichiers écrits à la main ou par une version antérieure du même format se chargent. Le module règle Zod pour des messages en français (`z.locales.fr()`, FMT-02, REC-03) ; ses propres messages viennent de `fr.json` (`validation.*`). Dépend de Zod et de `src/i18n`. Utilisé partout ; les schémas JSON publiés (`public/schemas/`) en sont générés par `npm run schema`.

Pour chaque entité, le schéma (`…Schema`) décrit les champs ; le type du même nom est le résultat d'un `parse` (valeurs par défaut appliquées, donc champs toujours présents).

`FORMAT_VERSION` constante

```
const FORMAT_VERSION = 2
```

Version courante du schéma de `project.json` (§ 8, FMT-04) : 2 depuis le lot 34. Écrite dans `formatVersion` du projet, des documents de bibliothèque et du manifeste ; comparée à la version d'un fichier ouvert, qui est refusé s'il est plus récent ou migré s'il est plus ancien (`src/io/migrations.ts`). À incrémenter pour tout changement incompatible, avec une migration.

`IdSchema` constante

```
const IdSchema = z.uuid()
```

Schéma d'un identifiant : un UUID (`z.uuid()`). Toutes les entités et toutes les références croisées l'utilisent.

`Id` type

```
type Id = z.infer<typeof IdSchema>;
```

Type d'un identifiant d'entité (chaîne UUID), déduit de `IdSchema` ; produit par `newId` ou `builtinId`.

`ColorSchema` constante

```
const ColorSchema = z.string().regex(/^#[0-9a-fA-F]{6}$/, { error: () =>
t('validation.color'), })
```

Schéma d'une couleur : chaîne `#rrggbb` (six chiffres hexadécimaux, casse libre), message `validation.color` sinon. Utilisé pour les couleurs d'armoire, de face, de type de liaison, de gaine et d'annotation.

`ImageRefSchema` constante

```
const ImageRefSchema = …
```

Schéma d'une référence d'image : nom du fichier dans l'archive, formé de l'empreinte SHA-256 du contenu (64 chiffres hexadécimaux minuscules) et de l'extension `png`, `jpg`, `jpeg`, `svg` ou `webp` (FMT-03). Deux images identiques n'ont donc qu'une entrée.

`ImageRef` type

```
type ImageRef = z.infer<typeof ImageRefSchema>;
```

Type d'une référence d'image (`ImageRefSchema`) : faces de machine, logo et objet posé, logo du projet, annotations image.

`FaceSideSchema` constante

```
const FaceSideSchema = z.enum(['front', 'rear'])
```

Schéma d'une face d'armoire ou de machine : `front` (avant) ou `rear` (arrière).

`FaceSide` type

```
type FaceSide = z.infer<typeof FaceSideSchema>;
```

Type d'une face, `'front' | 'rear'` (`FaceSideSchema`) : face de montage, face d'un port, face montrée par une vue.

`LateralSideSchema` constante

```
const LateralSideSchema = z.enum(['left', 'right'])
```

Schéma d'un côté : `left` ou `right`. Sert aux goulottes (ROU-02), aux emplacements 0U et au côté d'un équipement demi-largeur.

`LateralSide` type

```
type LateralSide = z.infer<typeof LateralSideSchema>;
```

Type d'un côté, `'left' | 'right'` (`LateralSideSchema`), vu de la face montrée.

`PointSchema` constante

```
const PointSchema = z.object({ x: z.number(), y: z.number() })
```

Schéma d'un point `{ x, y }` en millimètres de papier : points des traits et flèches d'annotation.

`Point` type

```
type Point = z.infer<typeof PointSchema>;
```

Type d'un point `{ x, y }` (`PointSchema`), en millimètres dans le repère de la page.

`ConnectorFamilySchema` constante

```
const ConnectorFamilySchema = …
```

Schéma de la famille d'un type de connecteur (§ 6.7) : `copper` (cuivre), `optical` (optique et modules), `video`, `usb`, `serial` (série / console), `audio`, `power` (électrique), `other`. Deux connecteurs de la même famille sont jugés compatibles (décision du 04/11/2026, lot 03).

`ConnectorFamily` type

```
type ConnectorFamily = z.infer<typeof ConnectorFamilySchema>;
```

Type de la famille d'un connecteur (`ConnectorFamilySchema`), utilisé par le contrôle de compatibilité et le rendu.

`ConnectorTypeSchema` constante

```
const ConnectorTypeSchema = …
```

Schéma d'un type de connecteur de la bibliothèque (§ 6.7, BIB-09).

Champs

- `id` identifiant (fixe pour les types par défaut, `builtinId`).
- `name` nom non vide (« RJ45 », « IEC C13 »…).
- `family` famille.
- `shape` forme du rendu schématique du port ; `generic` par défaut.
- `complementId` connecteur complémentaire (C13 et C14), ou `null`.

`ConnectorType` type

```
type ConnectorType = z.infer<typeof ConnectorTypeSchema>;
```

Type d'un type de connecteur (`ConnectorTypeSchema`), rangé dans `library.connectorTypes`.

`LineStyleSchema` constante

```
const LineStyleSchema = z.enum(['solid', 'dashed', 'dotted'])
```

Schéma du style de trait d'un type de liaison : `solid` (plein), `dashed` (tirets) ou `dotted` (pointillés).

`LinkTypeSchema` constante

```
const LinkTypeSchema = …
```

Schéma d'un type de liaison (§ 6.8).

Champs

- `id` identifiant.
- `name` nom non vide (« Ethernet »).
- `color` couleur du trait.
- `lineStyle` style de trait ; plein par défaut.
- `lineWidth` épaisseur en millimètres à l'échelle 1 ; 0,5 par défaut.
- `allowedConnectorIds` types de connecteurs admis aux extrémités (contrôle de compatibilité) ; vide par défaut.

`refPrefix` préfixe du repère automatique (`ETH-`) ; vide par défaut.

`showInLegend` présent dans la légende ; oui par défaut.

`markers` dessiné en repères de destination aux ports plutôt qu'en traits (ROU-09).

`LinkType` type

```
type LinkType = z.infer<typeof LinkTypeSchema>;
```

Type d'un type de liaison (`LinkTypeSchema`), rangé dans `library.linkTypes`.

`RACK_MAX_HEIGHT_U` constante

```
const RACK_MAX_HEIGHT_U = 52
```

Hauteur maximale d'une armoire, en U : 52 (§ 6.3). Borne aussi la hauteur d'une machine.

`RackModelSchema` constante

```
const RackModelSchema = …
```

Schéma d'un modèle d'armoire de la bibliothèque : un identifiant et les caractéristiques d'armoire décrites pour `RackSchema` (sans `modelId`), avec le même raffinement (profondeur utile $\leq$ profondeur, message `validation.usableDepth`).

`RackModel` type

```
type RackModel = z.infer<typeof RackModelSchema>;
```

Type d'un modèle d'armoire (`RackModelSchema`), rangé dans `library.rackModels` et copié par `createRackFromModel`.

`RackSchema` constante

```
const RackSchema = …
```

Schéma d'une armoire du projet (§ 6.3). Raffinement : `usableDepthMm` $\leq$ `depthMm`.

Champs

`id` identifiant.

`modelId` modèle d'armoire d'origine, pour information ; `null` par défaut.

`name` nom non vide.

`heightU` hauteur utile, entier de 1 à 52 U.

`widthMm` largeur extérieure, mm.

`mountingWidth` largeur de montage `19in` (défaut) ou `10in`.

`depthMm` profondeur extérieure, mm.

`usableDepthMm` profondeur utile entre montants, mm (contrôle EQP-02).

`castors` armoire sur roulettes.

`kind` `closed` (défaut), `open2`, `open4` (racks ouverts à 2 ou 4 montants), `wall` (coffret mural).

`frontDoor, rearDoor` `none` (défaut), `glass`, `perforated`, `solid`.

`uNumbering` `bottomUp` (défaut) ou `topDown` ; affichage seulement, les positions sont toujours stockées depuis le bas.

`zeroUSlots` nombre d'emplacements 0U verticaux par face et par côté (`frontLeft`, `frontRight`, `rearLeft`, `rearRight`) ; 0 par défaut.

`color` couleur du rendu ; gris `#8a8f98` par défaut.

`trayByLinkType` goulotte de chaque type de liaison dans cette armoire (ROU-02), prioritaire sur le réglage du projet ; vide par défaut.

Rack

type

```
type Rack = z.infer<typeof RackSchema>;
```

Type d'une armoire (`RackSchema`), rangée dans `project.racks` et désignée par `placement.rackId` et par le `targetId` des vues d'armoire.

EQUIPMENT_KINDS

constante

```
const EQUIPMENT_KINDS = …
```

Types d'équipements intégrés, dans l'ordre du catalogue (EQT-01) : `switch`, `router`, `firewall`, `patchPanel`, `fiberPanel`, `server`, `storage`, `kvm`, `pdu`, `ups`, `blankingPanel`, `cableManager`, `shelf`, `monitor`, `workstation`, `laptop`, `keyboardMouse`, `controlPanel`, `generic`. Liste fermée : l'utilisateur ne crée jamais de type. Ajouter un type demande un générateur dans `src/library/equipment/`, une entrée dans `KIND_CATEGORY` et une machine générique.

EquipmentKindSchema

constante

```
const EquipmentKindSchema = z.enum(EQUIPMENT_KINDS)
```

Schéma d'un type d'équipement : énumération de `EQUIPMENT_KINDS`. Utilisé dans `machine.equipment.kind`.

EquipmentKind

type

```
type EquipmentKind = z.infer<typeof EquipmentKindSchema>;
```

Type d'un type d'équipement intégré (`EquipmentKindSchema`), union des valeurs de `EQUIPMENT_KINDS`.

EQUIPMENT_CATEGORIES

constante

```
const EQUIPMENT_CATEGORIES = …
```

Catégories du catalogue dans l'ordre d'affichage (BIB-01) : `network` (réseau), `serversStorage` (serveurs et stockage), `power` (alimentation), `rackAccessories` (accessoires de baie), `external` (extérieurs), `generic`. Les libellés sont dans `fr.json`.

EquipmentCategory

type

```
type EquipmentCategory = (typeof EQUIPMENT_CATEGORIES)[number];
```

Type d'une catégorie d'équipement, union des valeurs de `EQUIPMENT_CATEGORIES`. Non stockée : déduite du type (`equipmentCategory`).

`KIND_CATEGORY` constante

```
const KIND_CATEGORY: Readonly<Record<EquipmentKind, EquipmentCategory>>
```

Catégorie de chaque type d'équipement (EQT-01) : réseau pour commutateur, routeur, pare-feu et panneaux ; serveurs et stockage pour serveur, stockage et KVM ; alimentation pour multiprise et onduleur ; accessoires pour obturateur, passe-câbles et étagère ; extérieurs pour écran, poste, portable, clavier et souris, pupitre ; générique pour l'équipement générique. La catégorie `external` détermine le placement hors armoire.

`PortExitSchema` constante

```
const PortExitSchema = z.enum(['up', 'down', 'left', 'right', 'auto'])
```

Schéma de la direction de sortie d'un câble depuis son port (§ 6.6) : `up`, `down`, `left`, `right` ou `auto` (choisie par le routage).

`PortSchema` constante

```
const PortSchema = …
```

Schéma d'un port de machine (§ 6.6).

Champs

`id` identifiant.

`name` nom non vide, unique sur sa face pour une machine générée.

`order` ordre de tri, entier ≥ 0 ; 0 par défaut.

`connectorTypeId` type de connecteur.

`x, y` position du centre, en proportion de la face (0 à 1).

`w, h` taille, en proportion de la face (dans]0, 1]).

`group` groupe (« Gestion », « Module 1 ») ; vide par défaut.

`exit` direction de sortie ; `auto` par défaut.

`multiple` port acceptant plusieurs liaisons (rare).

`pairedPortId` port de traversée sur l'autre face (module de panneau de brassage), réciproque, pour les chemins de bout en bout ; `null` par défaut.

`placed` port positionné par l'utilisateur (sur une photo) : sa position et sa taille sont conservées quand la machine est régénérée.

`Port` type

```
type Port = z.infer<typeof PortSchema>;
```

Type d'un port (`PortSchema`), rangé dans `front.ports` ou `rear.ports` d'une machine, ou extrait d'un port ajouté à un équipement.

`InstancePortSchema` constante

```
const InstancePortSchema = PortSchema.extend({ face: FaceSideSchema })
```

Schéma d'un port ajouté à un équipement placé seulement (PRT-04) : tous les champs d'un port plus `face`, la face de la machine qui le porte. Un tel port n'a pas de port de traversée (vérifié par `checkReferences`).

`InstancePort` type

```
type InstancePort = z.infer<typeof InstancePortSchema>;
```

Type d'un port ajouté à un équipement (`InstancePortSchema`), rangé dans `device.extraPorts`.

`FaceSchema` constante

```
const FaceSchema = …
```

Schéma d'une face de machine (§ 6.5).

Champs

- `renderMode` `default` (dessin généré par Portolan, défaut) ou `image` (photo importée).
- `imageId` image de la face, ou `null`.
- `color` couleur du boîtier pour le dessin par défaut ; `#3a3f47` par défaut.
- `label` libellé affiché sur le dessin par défaut.
- `ports` ports de la face ; vide par défaut.

`Face` type

```
type Face = z.infer<typeof FaceSchema>;
```

Type d'une face de machine (`FaceSchema`) : champs `front` et `rear` d'une machine.

`MachineOriginSchema` constante

```
const MachineOriginSchema = …
```

Schéma de l'origine d'une machine importée d'une bibliothèque `.rkdglib` : sert à détecter les doublons et les mises à jour (BIB-10, BIB-14).

Champs

- `libraryId` identifiant de la bibliothèque.
- `libraryName` nom de la bibliothèque.
- `libraryVersion` version (entier ≥ 1).
- `machineId` identifiant de la machine dans la bibliothèque.

`MachineOrigin` type

```
type MachineOrigin = z.infer<typeof MachineOriginSchema>;
```

Type de l'origine d'une machine (`MachineOriginSchema`) ; `null` pour une machine créée dans le projet.

`MachineLogoSchema` constante

```
const MachineLogoSchema = …
```

Schéma d'une image décorative d'une machine : logo (EDM-10) dessiné à côté du libellé de la face avant, ou objet posé sur une étagère (MAC-03).

Champs

- `imageId` image du projet.

`aspect` proportions largeur / hauteur de l'image (de 0,05 à 40), stockées pour dessiner sans décoder l'image.

`MachineLogo` type

```
type MachineLogo = z.infer<typeof MachineLogoSchema>;
```

Type d'une image décorative de machine (`MachineLogoSchema`) : champs `logo` et `carried`.

`MachineEquipmentSchema` constante

```
const MachineEquipmentSchema = …
```

Schéma de l'équipement d'une machine.

Champs

`kind` type d'équipement intégré.

`params` paramètres de l'équipement, enregistrement libre ; vide par défaut. Leur validité est vérifiée par le catalogue (`equipmentParamsProblem`, `src/library/equipment/`) à l'ouverture du fichier, pas par ce schéma.

`MachineEquipment` type

```
type MachineEquipment = z.infer<typeof MachineEquipmentSchema>;
```

Type de l'équipement d'une machine (`MachineEquipmentSchema`) : son type et ses paramètres.

`MachineSchema` constante

```
const MachineSchema = …
```

Schéma d'une machine, personnalisation d'un équipement intégré (§ 6.4). Elle porte tous les paramètres ; `heightU`, `width`, `displaySizeMm` et les ports en sont dérivés par le générateur et stockés ici. Raffinement : une machine extérieure a une taille affichée (`validation.externalSize`).

Champs

`id` identifiant.

`name` nom non vide.

`equipment` type et paramètres (`MachineEquipmentSchema`).

`generic` machine générique de son équipement (une par type, sans marque, EQT-02).

`manufacturer, reference` fabricant et référence ; vides par défaut.

`heightU` dérivé : hauteur en U, de 0 à 52 ; 0 pour un équipement 0U ou extérieur.

`width` dérivé : `full` (défaut) ou `half` (demi-largeur 19 pouces).

`displaySizeMm` dérivé : taille affichée d'un extérieur (`width`, `height` en mm), `null` sinon.

`depthMm` profondeur en mm ; `null` : toute la profondeur utile (EQP-02).

`clearanceMm` dégagement arrière en mm (ventilation, courbure) ; 0 par défaut.

`defaultMountFace` face de montage proposée ; avant par défaut.

`front, rear` faces (`FaceSchema`).

`logo` logo du dessin par défaut, ou `null`.

`carried` objet posé sur une étagère, ou `null`.

`tags` mots-clés de recherche.

`origin` origine (`MachineOriginSchema`) ou `null`.

`Machine` type

```
type Machine = z.infer<typeof MachineSchema>;
```

Type d'une machine (`MachineSchema`), rangée dans `library.machines` et désignée par `device.machineId`.

`LibrarySchema` constante

```
const LibrarySchema = …
```

Schéma de la bibliothèque d'un projet ou d'un fichier `.rkdglib`.

Champs

`connectorTypes` types de connecteurs.

`linkTypes` types de liaisons.

`rackModels` modèles d'armoires.

`machines` machines.

Toutes les listes sont vides par défaut.

`Library` type

```
type Library = z.infer<typeof LibrarySchema>;
```

Type d'une bibliothèque (`LibrarySchema`) : `project.library`, contenu d'un `.rkdglib`, résultat de `createEmptyLibrary`.

`RackPlacementSchema` constante

```
const RackPlacementSchema = …
```

Schéma du placement d'un équipement en U dans une armoire.

Champs

`kind` `rack`.

`rackId` armoire.

`uBottom` U la plus basse occupée, comptée depuis le bas (à partir de 1).

`mountFace` face de fixation ; l'équipement s'étend vers l'autre face (EQP-09).

`halfSide` côté d'un équipement demi-largeur (EQP-03), `null` sinon.

`ZeroUPlacementSchema` constante

```
const ZeroUPlacementSchema = …
```

Schéma du placement d'un équipement vertical 0U.

Champs

`kind` `zeroU`.

`rackId` armoire.

`face` face de l'armoire.

`side` côté (gauche ou droite).

`slot` emplacement vertical sur ce côté et cette face, à partir de 0.

`ExternalPlacementSchema` constante

```
const ExternalPlacementSchema = z.object({ kind: z.literal('external') })
```

Schéma du placement d'un équipement extérieur : `{ kind: 'external' }`, sans autre champ ; la position est celle de sa vue extérieure sur chaque page.

`PlacementSchema` constante

```
const PlacementSchema = …
```

Schéma d'un placement : union discriminée par `kind` des trois schémas précédents, `RackPlacementSchema`, `ZeroUPlacementSchema` et `ExternalPlacementSchema`. Le type doit correspondre à la machine (vérifié par `checkReferences`).

`RackPlacement` type

```
type RackPlacement = z.infer<typeof RackPlacementSchema>;
```

Type d'un placement en U (`RackPlacementSchema`), variante `rack` de `Placement`.

`ZeroUPlacement` type

```
type ZeroUPlacement = z.infer<typeof ZeroUPlacementSchema>;
```

Type d'un placement 0U (`ZeroUPlacementSchema`), variante `zeroU` de `Placement`.

`Placement` type

```
type Placement = z.infer<typeof PlacementSchema>;
```

Type d'un placement (`PlacementSchema`) : `rack`, `zeroU` ou `external`. Tester `placement.kind` avant de lire `rackId`.

`DeviceSchema` constante

```
const DeviceSchema = …
```

Schéma d'un équipement placé (instance d'une machine).

Champs

`id` identifiant.

`machineId` machine de la bibliothèque.

`name` nom non vide (unicité signalée par les vérifications).

`description` texte libre.

`fields` champs libres `{ key, value }` (nom d'hôte, adresse, numéro de série…).

`labelMode` étiquette : `name` (défaut), `nameModel` ou `none`.

`placement` placement (`PlacementSchema`).

`depthForced` placement maintenu malgré un dépassement de profondeur (« Forcer », EQP-02).

`portOverrides` surcharges par port (`portId`, `name` affiché ou `null`, `disabled`) (§ 6.6).

`extraPorts` ports ajoutés à cet équipement seulement (PRT-04).

`Device` type

```
type Device = z.infer<typeof DeviceSchema>;
```

Type d'un équipement placé (`DeviceSchema`), rangé dans `project.devices`, quel que soit son placement.

`PortRefSchema` constante

```
const PortRefSchema = z.object({ deviceId: IdSchema, portId: IdSchema })
```

Schéma d'une référence de port : `deviceId` (équipement placé) et `portId` (port de sa machine ou ajouté à l'équipement). Extrémité d'une liaison.

`PortRef` type

```
type PortRef = z.infer<typeof PortRefSchema>;
```

Type d'une référence de port (`PortRefSchema`) ; résolue par `resolvePort`, clé textuelle par `portKey`.

`LinkSchema` constante

```
const LinkSchema = …
```

Schéma d'une liaison (§ 6.9).

Champs

`id` identifiant.

`a, b` extrémités.

`linkTypeId` type de liaison.

`ref` repère du câble, unique dans le projet (LIA-04, signalé par les vérifications).

`length { value, unit }` (valeur positive, `m` ou `cm`), ou `null`.

`category, supplierRef` catégorie (Cat6, OM4…) et référence fournisseur.

`sheathColor` couleur de la gaine (information), ou `null`.

`tray` goulotte : `auto` (selon le type, défaut), `left`, `right` (ROU-02).

`notes` notes libres.

`compatibilityForced` liaison maintenue malgré l'avertissement de compatibilité (LIA-06).

`display` dessin : `auto` (selon la vue et le type), `line` ou `markers` (ROU-09).

`bundle` le câble peut être fusionné en faisceau avec les câbles de même type entre les deux mêmes équipements (ROU-11) ; `false` : dégroupé. Vrai par défaut.

`Link` type

```
type Link = z.infer<typeof LinkSchema>;
```

Type d'une liaison (`LinkSchema`), rangée dans `project.links`.

`ViewSchema` constante

```
const ViewSchema = …
```

Schéma d'une vue sur une page (§ 7.10).

Champs

- `id` identifiant.
- `kind` `rackFront`, `rackRear` ou `external`.
- `targetId` armoire (vues d'armoire) ou équipement (vue extérieure).
- `x, y` coin haut gauche sur la page, mm.
- `scale` échelle, positive ; 1 par défaut.
- `title` titre personnalisé, ou `null` pour le titre par défaut.
- `showTitle` titre affiché.
- `labelsBeside` étiquettes des équipements à côté de l'armoire (PAG-06, PAG-17).
- `size` vue extérieure : taille forcée en mm (EXT-02), ou `null`.
- `portsHidden` vue extérieure : ports non dessinés, câbles arrêtés au bas de l'équipement (EXT-03).
- `markers, markerRefs` mode repères (ROU-09) et ajout du repère du câble.
- `bundles` faisceaux de câbles (ROU-11).
- `cablesHidden` aucun câble dessiné dans la vue ni en partant (PAG-15).
- `uRange` vue d'armoire : plage de U `{ from, to }` montrée, bornes incluses, `from` $\leq$ `to` (PAG-16), ou `null` pour toute l'armoire.

`View` type

```
type View = z.infer<typeof ViewSchema>;
```

Type d'une vue (`ViewSchema`), rangée dans `page.views`.

`DashSchema` constante

```
const DashSchema = z.enum(['solid', 'dashed', 'dotted'])
```

Schéma du motif d'un trait ou d'un contour d'annotation (PAG-07) : `solid`, `dashed` ou `dotted`.

`Dash` type

```
type Dash = z.infer<typeof DashSchema>;
```

Type du motif de trait d'une annotation (`DashSchema`).

`QuarterTurnSchema` constante

```
const QuarterTurnSchema = z.union([
z.literal(0), z.literal(90), z.literal(180), z.literal(270), ])
```

Schéma de la rotation d'une annotation : 0, 90, 180 ou 270 degrés dans le sens horaire (PAG-07).

`QuarterTurn` type

```
type QuarterTurn = z.infer<typeof QuarterTurnSchema>;
```

Type d'une rotation par quart de tour (`QuarterTurnSchema`) : littéraux numériques 0, 90, 180 ou 270.

`AnnotationSchema` constante

```
const AnnotationSchema = …
```

Schéma d'un élément de l'onglet « Schéma » (PAG-07) : union discriminée par `kind`. Champs communs : `id`, `color` (texte, trait ou contour ; noir par défaut), `layer` (`below` ou `above` les vues, `above` par défaut), `rotation`. Les éléments en boîte ont `x`, `y`, `width`, `height` (mm de papier, avant rotation) ; les éléments à texte ont `fontSize` (mm, 3,5 par défaut), `bold`, `align` (`start`, `middle`, `end`) ; les éléments tracés ont `strokeWidth` (mm) et `dash`.

Champs

- `text` texte libre : boîte (hauteur 10 mm par défaut) et `text`.
- `line, arrow` trait ou flèche : `points` (au moins deux), `elbow` (coudé), `elbowStart` (premier segment horizontal ou vertical), `doubleHead` (pointe aux deux bouts ; sinon au dernier point).
- `rect, roundedRect, ellipse` forme : boîte, `outline`, `fill` (ou `null`).
- `image` image importée, sans déformation : boîte, `imageId` (`null` tant qu'elle n'est pas choisie).
- `zone` cadre titré : boîte, `title`, texte, trait (tirets par défaut), `fill`.
- `bubble` bulle numérotée : boîte, `text` (le numéro), texte, `fill` (couleur du disque).
- `note` note encadrée : boîte, `text`, texte, trait, `fill` (jaune pâle par défaut).
- `table` tableau : boîte, `cells` (lignes de cellules, au moins une), colonnes et lignes de taille égale, `headerRow` (première ligne en gras sur `fill`), texte, trait.

`Annotation` type

```
type Annotation = z.infer<typeof AnnotationSchema>;
```

Type d'une annotation de page (`AnnotationSchema`), rangée dans `page.annotations` ; discriminer par `kind` avant de lire les champs propres.

`AttachedPointSchema` constante

```
const AttachedPointSchema = z.object({ viewId: IdSchema, dx: z.number(), dy: z.number(), })
```

Schéma d'un point de tracé manuel rattaché à une vue de sa page (ROU-16) : `viewId` et le décalage (`dx`, `dy`) en millimètres du dessin de la vue à l'échelle 1, depuis son coin haut gauche. Sur la page, le point est en (`x` + `dx` × échelle effective, `y` + `dy` × échelle effective) : il suit la vue quand elle bouge ou change d'échelle (lot 37).

`AttachedPoint` type

```
type AttachedPoint = z.infer<typeof AttachedPointSchema>;
```

Type d'un point de passage rattaché à une vue (`AttachedPointSchema`), élément de `ManualRoute.points`.

`ManualRouteSchema` constante

```
const ManualRouteSchema = z.object({ linkId: IdSchema, points:
z.array(AttachedPointSchema), })
```

Schéma d'un tracé de câble retouché à la main sur une page (ROU-06) : `linkId` (la liaison) et `points`, la suite des points de passage rattachés aux vues (ROU-16). Un tracé dont la liaison ou une vue a disparu est signalé par un avertissement.

`ManualRoute` type

```
type ManualRoute = z.infer<typeof ManualRouteSchema>;
```

Type d'un tracé manuel (`ManualRouteSchema`), rangé dans `page.manualRoutes`.

`PageFormatSchema` constante

```
const PageFormatSchema = z.enum(['A4', 'A3', 'A2', 'A1', 'A0', 'Letter', 'custom'])
```

Schéma du format de papier d'une page : `A4`, `A3`, `A2`, `A1`, `A0`, `Letter` ou `custom` (taille donnée par `customSizeMm`).

`PageFormat` type

```
type PageFormat = z.infer<typeof PageFormatSchema>;
```

Type d'un format de page (`PageFormatSchema`) ; les dimensions de chaque format sont dans la mise en page (`model/pageLayout`).

`PageSchema` constante

```
const PageSchema = …
```

Schéma d'une page (§ 7.10). Raffinement : le format `custom` exige `customSizeMm` (`validation.customPageSize`).

Champs

- `id` identifiant.
- `title` titre (peut être vide).
- `format` format ; A4 par défaut.
- `customSizeMm` taille personnalisée `{ width, height }` en mm, ou `null`.
- `orientation` `portrait` ou `landscape` (défaut).
- `uniformRackScale` même échelle pour toutes les vues d'armoire (PAG-05).
- `marginsMm` marges haut, droite, bas, gauche en mm ; 10 par défaut.
- `views` vues.
- `annotations` annotations.
- `manualRoutes` tracés manuels.
- `cableRefs` repères des câbles : `none` (défaut), `along` (le long des tracés) ou `ends` (aux extrémités) (ROU-10).
- `legend` position de la légende : `titleBlock` (dans le cartouche, défaut) ou `free` avec `x`, `y` (mm, ou `null`).
- `revisionTable` tableau des révisions : `visible`, nombre de lignes `rows` (3 par défaut), position `x`, `y` ou `null` (place par défaut).

`Page` type

```
type Page = z.infer<typeof PageSchema>;
```

Type d'une page (`PageSchema`), rangée dans `project.pages`.

`TitleBlockFieldSchema` constante

```
const TitleBlockFieldSchema = …
```

Schéma d'un champ du cartouche (PAG-09, PAG-10).

Champs

- `id` identifiant.
- `kind` `logo`, `projectName`, `client`, `pageTitle`, `revision`, `revisionDate`, `author`, `pageNumber` ou `custom`.
- `label` libellé : celui d'un champ libre ; pour les autres, vide = libellé par défaut.
- `value` valeur d'un champ libre.
- `visible` champ affiché ; oui par défaut.
- `row` place : `top`, `bottom` (rangées du bloc d'informations) ou `full` (colonne sur toute la hauteur) ; `null` = place par défaut de la nature (`defaultTitleBlockRow`).

`TitleBlockField` type

```
type TitleBlockField = z.infer<typeof TitleBlockFieldSchema>;
```

Type d'un champ de cartouche (`TitleBlockFieldSchema`) ; créé par `createTitleBlockField`.

`TitleBlockSchema` constante

```
const TitleBlockSchema = z.object({ fields: z.array(TitleBlockFieldSchema).default([]), })
```

Schéma du cartouche du projet : `fields`, la liste ordonnée des champs (vide par défaut). Le cartouche est commun à toutes les pages et repris par PRJ-10.

`TitleBlock` type

```
type TitleBlock = z.infer<typeof TitleBlockSchema>;
```

Type du cartouche (`TitleBlockSchema`) : `project.titleBlock`, mis en page par `model/titleBlockLayout`.

`IsoDateSchema` constante

```
const IsoDateSchema = z.iso.date()
```

Schéma d'une date ISO `AAAA-MM-JJ` (`z.iso.date()`) : date des révisions, produite par `today`.

`IsoDateTimeSchema` constante

```
const IsoDateTimeSchema = z.iso.datetime({ offset: true })
```

Schéma d'un horodatage ISO 8601 avec décalage horaire (`z.iso.datetime({ offset: true })`) : date de création du projet.

`RevisionSchema` constante

```
const RevisionSchema = …
```

Schéma d'une révision du projet (§ 6.2, PRJ-11). La dernière alimente le cartouche.

Champs

- `id` identifiant.
- `index` indice non vide (A, B, C… ou libre).
- `date` date ISO.
- `author` auteur.
- `description` description des modifications.

`Revision` type

```
type Revision = z.infer<typeof RevisionSchema>;
```

Type d'une révision (`RevisionSchema`), rangée dans `project.revisions` dans l'ordre chronologique.

`ProjectMetaSchema` constante

```
const ProjectMetaSchema = …
```

Schéma des métadonnées du projet (§ 6.2).

Champs

- `name` nom du projet (peut être vide).
- `client` client ou site.
- `author` auteur (« Dessiné par »).
- `createdAt` horodatage ISO de création.
- `logoImageId` logo du cartouche, ou `null`.
- `notes` notes libres.

`ProjectMeta` type

```
type ProjectMeta = z.infer<typeof ProjectMetaSchema>;
```

Type des métadonnées du projet (`ProjectMetaSchema`) : `project.meta`.

`LibraryInfoSchema` constante

```
const LibraryInfoSchema = …
```

Schéma de la description d'un fichier de bibliothèque (manifeste d'un `.rkdglib`, FMT-09), utilisé aussi par `editedLibrary`.

Champs

- `id` identifiant de la bibliothèque.
- `name` nom non vide.
- `brand` marque (texte libre).
- `version` version, entier ≥ 1, incrémentée à chaque publication.
- `author, description` auteur et description.

`license` licence ou origine des images.

`LibraryInfo` type

```
type LibraryInfo = z.infer<typeof LibraryInfoSchema>;
```

Type de la description d'une bibliothèque (`LibraryInfoSchema`), lue dans le manifeste d'un `.rkdglib`.

`ProjectSchema` constante

```
const ProjectSchema = …
```

Schéma d'un projet, racine de `project.json` (§ 6.2).

Champs

`formatVersion` littéral `FORMAT_VERSION`.

`id` identifiant du projet.

`meta` métadonnées.

`revisions` révisions.

`library` bibliothèque du projet (obligatoire).

`racks` armoires.

`devices` équipements placés.

`links` liaisons.

`pages` pages.

`titleBlock` cartouche ; sans champ par défaut.

`routing` réglages de routage : `trayByLinkType`, goulotte de chaque type de liaison vue de la face montrée (ROU-02).

`editedLibrary` facultatif : bibliothèque `.rkdglib` ouverte pour édition (BIB-13), sans armoire ni page ; `version` est celle de l'enregistrement. Absent pour un projet.

`Project` type

```
type Project = z.infer<typeof ProjectSchema>;
```

Type d'un projet (`ProjectSchema`) : l'objet central de l'application, tenu par le magasin et passé à toutes les fonctions du modèle.

`LibraryDocumentSchema` constante

```
const LibraryDocumentSchema = z.object({ formatVersion: z.literal(FORMAT_VERSION), library: LibrarySchema, })
```

Schéma du `project.json` d'un fichier `.rkdglib` (FMT-09) : `formatVersion` et `library` seulement. Validé par `parseLibraryDocument`.

`LibraryDocument` type

```
type LibraryDocument = z.infer<typeof LibraryDocumentSchema>;
```

Type du document d'une bibliothèque (`LibraryDocumentSchema`), écrit et lu par `src/io/archive.ts`.

18.10 `model/validate`

Validation d'un projet : structure (schéma Zod, FMT-02), puis références croisées. Les erreurs rendent le projet inutilisable et le font refuser ; les avertissements sont des anomalies que l'utilisateur peut corriger (décision du 04/11/2026, lot 01). Dépend du schéma, de l'index et du catalogue des équipements (`equipmentParamsProblem`). Utilisé par la lecture des archives et des brouillons (`src/io/archive.ts`, `src/io/draft.ts`). Les messages viennent de `validation.ref.*`.

`ValidationIssue` interface

```
interface ValidationIssue { severity; path; message }
```

Constat de validation.

Champs

`severity` `error` (refuse le fichier) ou `warning`.

`path` emplacement dans le document JSON (`racks[0].heightU`), vide pour la racine.

`message` message en français.

`ParseResult` type

```
type ParseResult<T> = { ok: true; value: T; warnings: ValidationIssue[] } | { ok: false;
errors: ValidationIssue[] };
```

Résultat d'une validation : `{ ok: true, value, warnings }` avec la valeur validée (valeurs par défaut appliquées) et ses avertissements, ou `{ ok: false, errors }` avec les seules erreurs.

`formatPath` fonction

```
function formatPath(path: readonly PropertyKey[]): string
```

Met en forme un chemin JSON : nombres entre crochets, clés séparées par des points.

Paramètres

`path` chemin, tel que fourni par Zod.

Résultat. par exemple `racks[0].heightU` pour `['racks', 0, 'heightU']`.

Erreurs. Aucune.

`formatIssue` fonction

```
function formatIssue(issue: ValidationIssue): string
```

Message lisible d'un constat, `chemin : message`, ou le message seul si le chemin est vide. Utilisé pour le message de refus d'un fichier.

Paramètres

`issue` le constat.

Résultat. le texte.

Erreurs. Aucune.

`checkLibraryReferences` fonction

```
function checkLibraryReferences(library: Library, base = 'library'): ValidationIssue[]
```

Vérifie les références croisées d'une bibliothèque : identifiants uniques dans chaque liste ; compléments et connecteurs admis existants ; une seule machine générique par équipement ; paramètres de chaque machine valides pour le catalogue (BIB-09) ; pour chaque port de machine, identifiant unique dans la machine, type de connecteur existant et port de traversée existant, sur l'autre face et réciproque.

Paramètres

`library` la bibliothèque.

`base` préfixe des chemins des constats ; `library` par défaut.

Résultat. la liste des constats, tous de gravité `error`.

Erreurs. Aucune (les problèmes sont rendus, pas levés).

`checkReferences` fonction

```
function checkReferences(project: Project): ValidationIssue[]
```

Vérifie les références croisées d'un projet entier : celles de la bibliothèque, puis l'unicité des identifiants des armoires, équipements, liaisons et pages ; pour chaque équipement, machine et armoire existantes, type de placement conforme à la machine (extérieur, 0U ou en U), ports ajoutés valides ; pour chaque liaison, type existant et extrémités résolues ; pour chaque page, cibles des vues existantes et tracés manuels cohérents. Ne contrôle pas les règles métier (chevauchements, profondeur, repères) : ce sont des anomalies (`listAnomalies`).

Paramètres

`project` projet déjà conforme au schéma.

Résultat. les constats : erreurs pour les références indispensables, avertissements pour une surcharge de port inconnu, une liaison vers un équipement ou un port disparu, un tracé manuel orphelin ou rattaché à une vue absente.

Erreurs. Aucune (les problèmes sont rendus, pas levés).

`parseProject` fonction

```
function parseProject(data: unknown): ParseResult<Project>
```

Valide une valeur inconnue comme projet : schéma (`safeParse`), puis `checkReferences`. Appelée après migration, à l'ouverture d'une archive et à la reprise d'un brouillon.

Paramètres

`data` valeur à valider (JSON analysé).

Résultat. un `ParseResult` : le projet complété et ses avertissements, ou les erreurs (de structure seules si le schéma échoue).

Erreurs. Aucune exception : les erreurs sont dans le résultat.

```
const result = parseProject(JSON.parse(text));
if (!result.ok) throw new Error(result.errors.map(formatIssue).join('\n'));
const project = result.value;
```

`parseLibraryDocument` fonction

```
function parseLibraryDocument(data: unknown): ParseResult<LibraryDocument>
```

Valide le `project.json` d'un fichier `.rkdglib` : schéma `LibraryDocumentSchema`, puis `checkLibraryReferences`.

Paramètres

`data` valeur à valider.

Résultat. un `ParseResult` du document de bibliothèque.

Erreurs. Aucune exception : les erreurs sont dans le résultat.

Chapitre 19

Référence : règles métier du modèle

Ce chapitre décrit, déclaration par déclaration, les modules de `src/model/` qui portent les règles métier : placement et contrôle de profondeur, géométrie d'une armoire, placement à la souris et au clavier, infobulle de profondeur, liaisons et chemins de bout en bout, anomalies, ports ajoutés à un équipement, alignement et aimantation, mise en place automatique des vues. Tous ces modules sont purs : ils ne modifient jamais le projet reçu, ne dépendent pas de React et sont testés unitairement (`src/model/*.test.ts`). Le chapitre « Règles métier » de la première partie en explique la conception.

Conventions : les longueurs d'armoire sont en millimètres réels (« mm d'armoire »), celles de page en millimètres de papier ; les U sont comptées depuis le bas de l'armoire, à partir de 1 ; les emplacements 0U sont numérotés à partir de 0. Les messages destinés à l'utilisateur sont en français et viennent de `src/i18n/fr.json` (fonction `t`).

19.1 `model/placement`

Règles de placement d'un équipement (EQP-02, EQP-03, EQP-04, EQP-09, ARM-03, ARM-05) : occupation des U par face, contrôle de profondeur entre un équipement fixé à l'avant et un autre fixé à l'arrière, emplacements 0U, réduction de la hauteur d'une armoire, calcul de l'occupation et de l'espace libre. Dépend de `model/lookup` (`indexProject`), de `model/equipment` (`isExternal`) et de `i18n` (messages, nombres formatés). Utilisé par `model/dropTarget`, `model/anomalies`, `model/depthInfo`, les actions du magasin (`store/actions.ts`), le glisser-déposer (`ui/canvas/Canvas.tsx`), l'import tableur (`import/rules.ts`), le changement de machine (`library/equipment/change.ts`), l'export des tableaux et les propriétés d'armoire. Tests : `src/model/placement.test.ts`.

`PlacementCandidate` interface

```
interface PlacementCandidate { machineId; placement; excludeDeviceId?; forced? }
```

Placement proposé pour une machine, tel que le reçoit `checkPlacement` : pose depuis la bibliothèque, déplacement, collage, import.

Champs

- `machineId` identifiant de la machine (modèle) à placer ;
- `placement` position voulue : dans les U (`kind: 'rack'`), dans un emplacement 0U (`'zeroU'`) ou hors armoire (`'external'`) ;
- `excludeDeviceId` facultatif : équipement en cours de déplacement, ignoré dans la recherche des conflits (il ne se gêne pas lui-même) ;

`forced` facultatif : l'utilisateur a choisi « Forcer » malgré un dépassement de profondeur ; seuls ces dépassements sont alors acceptés.

`DepthTerm` interface

```
interface DepthTerm { deviceId; kind; mm }
```

Un terme de la somme des profondeurs affichée dans le message de refus (« 450 + 100 + 600 = 1 150 mm > 800 mm »).

Champs

`deviceId` équipement auquel se rapporte le terme ; `null` pour le candidat quand il n'est pas encore placé (pose d'un nouvel équipement) ;

`kind` `'depth'` (profondeur de la machine), `'unknownDepth'` (machine sans profondeur : le terme vaut toute la profondeur utile, EQP-02) ou `'clearance'` (dégagement, présent seulement s'il est strictement positif) ;

`mm` valeur du terme, en mm.

`DepthOverflow` interface

```
interface DepthOverflow { otherDeviceId; us; terms; totalMm; usableDepthMm }
```

Dépassement de profondeur entre le candidat et un équipement de l'autre face qui partage au moins une U avec lui (EQP-02, règle 2).

Champs

`otherDeviceId` équipement fixé sur l'autre face ;

`us` U partagées par les deux équipements, croissantes ;

`terms` termes de la somme, dans l'ordre : candidat (profondeur, dégagement), puis l'autre équipement ;

`totalMm` somme des termes, en mm ;

`usableDepthMm` profondeur utile de l'armoire, en mm ; on a toujours `totalMm > usableDepthMm`.

`PlacementProblem` type

```
type PlacementProblem = | { kind: 'unknownMachine' } | { kind: 'unknownRack' } | { kind:
'notRackable' } | { kind: 'needsZeroU' } | { kind: 'needsU' } | { kind: 'outOfRack'; fromU:
number; toU: number; rackHeightU: number } | { kind: 'overlap'; face: FaceSide; us:
number[]; deviceIds: Id[] } | { kind: 'zeroUSlotMissing'; available: number } | { kind:
'zeroU…
```

Raison d'un refus de placement (union discriminée par `kind`) :

Champs

`unknownMachine` la machine n'existe pas dans la bibliothèque du projet ;

`unknownRack` l'armoire visée n'existe pas ;

`notRackable` une machine extérieure dans une armoire, ou une machine d'armoire placée hors armoire ;

`needsZeroU` machine 0U (hauteur 0) placée dans les U ;

`needsU` machine qui occupe des U placée dans un emplacement 0U ;

`outOfRack` l'équipement sortirait de l'armoire : `fromU`, `toU` (U basse et haute qu'il occuperait) et `rackHeightU` ;

`overlap` U déjà occupées sur la même face : `face`, `us` (U en conflit, sans doublon, croissantes) et `deviceIds` (équipements en cause) ;

`zeroUSlotMissing` l'emplacement 0U demandé n'existe pas ; `available` donne le nombre d'emplacements de ce côté sur cette face ;

`zeroUSlotTaken` l'emplacement 0U est déjà pris par `deviceId` ;

`depth` dépassement de profondeur, avec les champs de `DepthOverflow` ; c'est le seul problème « forçable ».

`PlacementMessage` interface

```
interface PlacementMessage { problem; message; forceable }
```

Problème de placement accompagné de son message.

Champs

`problem` le problème (`PlacementProblem`) ;

`message` phrase française destinée à l'utilisateur (clés `placement.*` de `fr.json`), qui nomme les U et les équipements en cause ;

`forceable` vrai pour un dépassement de profondeur, faux sinon.

`PlacementResult` type

```
type PlacementResult = | { status: 'ok'; /** True when depth overflows were accepted with «
Forcer ». */ forced: boolean; /** Accepted depth overflows (empty unless
forced). */ warnings: PlacementMessage[]; } | { status: 'refused'; problems:
PlacementMessage[]; /** All problems are depth overflows: « Forcer » is
possible. */ forceable: boolean; };
```

Résultat de `checkPlacement`, union discriminée par `status` :

Champs

`ok` le placement est accepté ; `forced` est vrai quand des dépassements de profondeur ont été acceptés grâce à « Forcer », et `warnings` les liste (vide sinon) ;

`refused` le placement est refusé ; `problems` donne tous les problèmes trouvés et `forceable` est vrai quand tous sont des dépassements de profondeur (l'interface propose alors « Forcer »).

`occupiedUs` fonction

```
function occupiedUs(uBottom: number, heightU: number): number[]
```

U occupées par un équipement : de son U basse à son U haute, bornes comprises. Brique de base des contrôles d'occupation ; utilisée aussi par les propriétés d'armoire.

Paramètres

`uBottom` U basse (comptée depuis le bas, à partir de 1) ;

`heightU` hauteur de la machine, en U.

Résultat. Tableau croissant `[uBottom, …, uBottom + heightU - 1]` ; vide si la hauteur vaut 0.

Erreurs. Une hauteur négative lève une `RangeError` (`Array.from`) ; cela n'arrive pas avec une machine validée par le schéma.

`formatDepthCalculation` fonction

```
function formatDepthCalculation(overflow: Pick<DepthOverflow, 'terms' | 'totalMm'
| 'usableDepthMm'>): string
```

Écrit le calcul d'un dépassement de profondeur, tel qu'il apparaît dans les messages de refus et dans le panneau Vérifications : les termes séparés par « + », le total et la profondeur utile, nombres formatés à la française.

Paramètres

`overflow` objet portant `terms`, `totalMm` et `usableDepthMm` (un `DepthOverflow` convient).

Résultat. Chaîne de la forme « 450 + 100 + 600 = 1 150 mm > 800 mm » (clé `placement.depthCalculation`).

Erreurs. Aucune.

`formatURange` fonction

```
function formatURange(us: readonly number[]): string
```

Écrit une liste d'U pour un message : une U seule, deux U (« 10 et 11 ») ou un intervalle du plus petit au plus grand (« 10 à 12 »). Les U sont triées ; les trous éventuels ne sont pas signalés (seuls le minimum et le maximum comptent au-delà de deux valeurs). Sert aux messages de placement et aux plages libres des propriétés d'armoire.

Paramètres

`us` numéros d'U, dans un ordre quelconque.

Résultat. Texte français ; chaîne vide pour une liste vide.

Erreurs. Aucune.

```
formatURange([12, 10, 11]); // « 10 à 12 »
formatURange([4, 5]);       // « 4 et 5 »
```

`zeroUSlotCount` fonction

```
function zeroUSlotCount(rack: Rack, face: FaceSide, side: LateralSide): number
```

Nombre d'emplacements 0U d'un côté d'une face d'une armoire (champ `zeroUSlots` de l'armoire : `frontLeft`, `frontRight`, `rearLeft`, `rearRight`). Utilisé par le contrôle des emplacements 0U, les propriétés d'un équipement et l'import tableur.

Paramètres

`rack` l'armoire ;

`face` `'front'` ou `'rear'` ;

`side` `'left'` ou `'right'`, vu depuis cette face.

Résultat. Nombre entier positif ou nul.

Erreurs. Aucune.

`physicalHalfSide` fonction

```
function physicalHalfSide(model: Machine, placement: RackPlacement): LateralSide | null
```

Côté d'un équipement demi-largeur vu depuis l'avant de l'armoire (EQP-03). Le côté stocké dans le placement (`halfSide`) est celui vu depuis la face de montage : il est inversé pour un équipement fixé à l'arrière. C'est ce côté physique que comparent les règles d'occupation et de profondeur, et le calcul de l'espace libre ; il sert aussi au dessin (`model/pageGeometry`).

Paramètres

`model` la machine (seul son champ `width` est lu) ;

`placement` le placement dans les U.

Résultat. `'left'` ou `'right'`, ou `null` quand l'équipement prend toute la largeur (machine pleine largeur, ou demi-largeur sans côté choisi).

Erreurs. Aucune.

`flippedPlacement` fonction

```
function flippedPlacement(placement: RackPlacement): RackPlacement
```

Placement du même équipement retourné (EQP-09) : fixé sur l'autre face, à la même U et du même côté physique de l'armoire, ce qui inverse le côté stocké d'une demi-largeur. Il n'y a pas de drapeau « retourné » : retourner revient à changer `mountFace`. Utilisé par `flipDevices`.

Paramètres

`placement` placement dans les U de l'équipement.

Résultat. Nouveau placement (l'original n'est pas modifié).

Erreurs. Aucune.

`devicesInRack` fonction

```
function devicesInRack(project: Project, rackId: Id): PlacedInRack[]
```

Équipements fixés dans les U d'une armoire (les 0U et les extérieurs sont exclus), avec leur machine et leur placement typé. Un équipement dont la machine est introuvable est ignoré.

Paramètres

`project` le projet ;

`rackId` identifiant de l'armoire.

Résultat. Tableau d'objets `{ device, model, placement }` dans l'ordre du projet ; vide pour une armoire inconnue.

Erreurs. Aucune.

`checkPlacement` fonction

```
function checkPlacement(project: Project, candidate: PlacementCandidate): PlacementResult
```

Contrôle central du placement : dit si une machine peut être placée à une position et, sinon, pourquoi. Appelé à chaque aperçu de glisser-déposer, à chaque pose, déplacement, collage, retournement et import, et indirectement par le panneau Vérifications. Les étapes sont, dans l'ordre : machine connue ; cohérence entre la nature de la machine et celle du placement (extérieur, 0U, U) ; armoire connue ; puis, pour un placement dans les U, débordement

de l'armoire (qui arrête le contrôle), chevauchement sur la même face (EQP-02 règle 1, deux demi-largeurs de côtés physiques opposés ne se gênent pas) et profondeur avec chaque équipement de l'autre face partageant une U (EQP-02 règle 2) ; pour un emplacement 0U, existence puis disponibilité de l'emplacement.

Paramètres

`project` le projet (non modifié) ;

`candidate` la machine, la position, l'équipement déplacé à ignorer et le forçage (`PlacementCandidate`).

Résultat. `PlacementResult` : `ok` sans problème, `ok` avec `forced: true` quand seuls des dépassements de profondeur ont été trouvés et que le candidat est forcé, `refused` sinon.

Erreurs. Ne lève pas d'exception : les références inconnues donnent un refus (`unknownMachine`, `unknownRack`).

```
const result = checkPlacement(project, {
  machineId: serveur.id,
  placement: { kind: 'rack', rackId: baie.id, uBottom: 10,
               mountFace: 'front', halfSide: null },
});
if (result.status === 'refused' && result.forceable) {
  // proposer « Forcer », puis rappeler avec forced: true
}
```

`checkDevicePlacement` fonction

```
function checkDevicePlacement(project: Project, device: Device): PlacementResult
```

Contrôle d'un équipement déjà placé contre les autres : appelle `checkPlacement` avec sa machine, son placement, lui-même comme équipement exclu et son drapeau `depthForced`. Utilisé par `listAnomalies` (conflits et dépassements forcés) et par le changement de machine.

Paramètres

`project` le projet ;

`device` l'équipement à contrôler.

Résultat. Le `PlacementResult` de sa position actuelle.

Erreurs. Aucune.

`RackResizeResult` type

```
type RackResizeResult = { ok: true } | { ok: false; deviceIds: Id[]; message: string };
```

Résultat de `checkRackResize` :

Champs

`ok` `true` : la nouvelle hauteur est possible ;

`deviceIds` (si refus) équipements qui occupent des U supprimées ;

`message` (si refus) message français qui les nomme (ARM-03).

`checkRackResize` fonction

```
function checkRackResize(project: Project, rackId: Id, newHeightU: number):
RackResizeResult
```

Contrôle de la réduction de hauteur d'une armoire (ARM-03) : refusée si un équipement fixé dans les U a son U haute au-dessus de la nouvelle hauteur. Les équipements 0U ne sont pas contrôlés. Appelé par l'action de modification d'armoire du magasin.

Paramètres

- `project` le projet ;
- `rackId` l'armoire ;
- `newHeightU` la hauteur voulue, en U.

Résultat. `{ ok: true }` ou `{ ok: false, deviceIds, message }`.

Erreurs. Armoire inconnue : refus avec `deviceIds` vide et le message `placement.unknownRack`.

`UOccupation` interface

```
interface UOccupation { u; front; rear; frontDepthMm; rearDepthMm; freeDepthMm }
```

Occupation d'une U d'une armoire (ARM-05, EQP-05).

Champs

- `u` numéro physique de l'U (depuis le bas, à partir de 1) ;
- `front` identifiants des équipements fixés à l'avant sur cette U (deux au plus en pratique : deux demi-largeurs) ;
- `rear` idem à l'arrière ;
- `frontDepthMm` profondeur prise depuis l'avant (profondeur + dégagement, toute la profondeur utile pour une machine sans profondeur), le maximum des équipements avant ;
- `rearDepthMm` idem depuis l'arrière ;
- `freeDepthMm` profondeur utile moins les deux précédentes, jamais négative.

`rackOccupation` fonction

```
function rackOccupation(project: Project, rackId: Id): UOccupation[]
```

Occupation de chaque U d'une armoire, avec les mêmes termes de profondeur que le contrôle EQP-02. Utilisée par l'infobulle de profondeur (`uDepthAt`) et l'export des tableaux.

Paramètres

- `project` le projet ;
- `rackId` l'armoire.

Résultat. Tableau de `UOccupation`, de l'U 1 au sommet (l'indice `i` correspond à l'U `i + 1`) ; vide pour une armoire inconnue.

Erreurs. Aucune.

`freeUCount` fonction

```
function freeUCount(project: Project, rackId: Id): number
```

Nombre d'U où rien n'est fixé, ni à l'avant ni à l'arrière (ARM-05). Une U occupée par une seule demi-largeur compte comme occupée.

Paramètres

`project` le projet ;

`rackId` l'armoire.

Résultat. Nombre entier ; 0 pour une armoire inconnue.

Erreurs. Aucune.

`UFreeSpace` interface

```
interface UFreeSpace { u; left; right }
```

Moitiés libres d'une U, vues de l'avant de l'armoire (ARM-05, EQP-03).

Champs

`u` numéro physique de l'U ;

`left` aucun équipement, sur aucune face, ne couvre la moitié gauche ;

`right` idem pour la moitié droite.

`rackFreeSpace` fonction

```
function rackFreeSpace(project: Project, rackId: Id): UFreeSpace[]
```

Espace libre de chaque U d'une armoire : un équipement pleine largeur (ou demi-largeur sans côté) occupe les deux moitiés, une demi-largeur seulement sa moitié physique. Base de la couche « Occupation des armoires » (`model/pageGeometry`) et de `freeSpaceSummary`.

Paramètres

`project` le projet ;

`rackId` l'armoire.

Résultat. Tableau de `UFreeSpace`, de l'U 1 au sommet ; vide pour une armoire inconnue.

Erreurs. Aucune.

`FreeSpaceSummary` interface

```
interface FreeSpaceSummary { freeU; halfFreeU; ranges }
```

Résumé de l'espace libre d'une armoire.

Champs

`freeU` nombre d'U entièrement libres (ARM-05) ;

`halfFreeU` nombre d'U dont une seule moitié est libre (EQP-03) ;

`ranges` plages d'U entièrement libres, du bas vers le haut, sous la forme `[première, dernière]` (numéros physiques).

`freeSpaceSummary` fonction

```
function freeSpaceSummary(project: Project, rackId: Id): FreeSpaceSummary
```

Compte les U libres d'une armoire et les regroupe en plages consécutives. Utilisé par les propriétés d'armoire (U libres, à moitié libres, plages) et par la pastille du nombre d'U libres de la couche d'occupation (`ui/canvas/Overlay.tsx`).

Paramètres

`project` le projet ;

`rackId` l'armoire.

Résultat. `FreeSpaceSummary` ; tout à zéro pour une armoire inconnue.

Erreurs. Aucune.

19.2 `model/rackGeometry`

Géométrie du dessin d'une face d'armoire (ARM-04), en mm d'armoire, origine au coin haut gauche du dessin : cadre, zones latérales (emplacements 0U côté extérieur, goulotte côté intérieur), numérotation, montants avec trous de fixation, zone des U au milieu ; position d'une U, d'un équipement, d'un emplacement 0U ; fenêtre d'une vue limitée à une plage d'U (PAG-16) avec ses lignes de coupure en zigzag. Ne dépend que des types du schéma. Module central partagé par le rendu (`render/RackView.tsx`), la géométrie des pages (`model/pageGeometry`, `model/pageLayout`), le routage (goulottes, `routing/trays.ts`), le glisser-déposer et l'éditeur de machine. Tests : `rackGeometry.test.ts`, `viewRange.test.ts`.

`U_MM` constante

```
const U_MM = 44.45
```

Hauteur d'une U : 44,45 mm (1,75 pouce). Sert à toutes les conversions U → mm (dessin, glisser-déposer, infobulle de profondeur, générateurs d'équipements standards).

`PANEL_19_MM` constante

```
const PANEL_19_MM = 482.6
```

Largeur d'une façade 19 pouces, oreilles comprises : 482,6 mm. C'est la largeur de la zone des U d'une armoire 19" ; reprise par l'éditeur de machine et le décor des équipements standards.

`PANEL_10_MM` constante

```
const PANEL_10_MM = 254
```

Largeur d'une façade 10 pouces, oreilles comprises : 254 mm ; zone des U d'une armoire `mountingWidth: '10in'`.

`OPENING_19_MM` constante

```
const OPENING_19_MM = 450
```

Ouverture entre les montants d'une armoire 19" : 450 mm. La largeur d'un montant est `(PANEL_19_MM - OPENING_19_MM) / 2`.

OPENING_10_MM constante

```
const OPENING_10_MM = 222.25
```

Ouverture entre les montants d'une armoire 10" : 222,25 mm ; même usage que `OPENING_19_MM`.

FRAME_MM constante

```
const FRAME_MM = 18
```

Épaisseur du cadre de l'armoire : 18 mm. C'est aussi la hauteur du haut d'une armoire ouverte (sans toit) et du bas d'une armoire murale (sans socle). Lu par le rendu.

NUMBERING_MM constante

```
const NUMBERING_MM = 24
```

Largeur de la colonne de numérotation des U, de chaque côté de la zone des U : 24 mm.

ROOF_MM constante

```
const ROOF_MM = 40
```

Hauteur du toit d'une armoire fermée ou murale : 40 mm (une armoire ouverte n'a que le cadre, `FRAME_MM`).

PLINTH_MM constante

```
const PLINTH_MM = 45
```

Hauteur du socle d'une armoire au sol : 45 mm (une armoire murale n'a que le cadre).

FEET_MM constante

```
const FEET_MM = 18
```

Hauteur des pieds d'une armoire au sol sans roulettes : 18 mm, sous le socle. Lu par le rendu.

CASTOR_MM constante

```
const CASTOR_MM = 75
```

Hauteur des roulettes d'une armoire roulante (`castors`) : 75 mm, sous le socle. Lu par le rendu.

ZERO_U_SLOT_MAX_MM constante

```
const ZERO_U_SLOT_MAX_MM = 40
```

Largeur maximale d'un emplacement 0U : 40 mm. Quand la zone latérale est large, les emplacements gardent cette largeur et la goulotte prend le reste.

ZERO_U_SLOT_MIN_MM constante

```
const ZERO_U_SLOT_MIN_MM = 30
```

Largeur minimale d'un emplacement 0U : 30 mm. Sert à calculer la largeur minimale du dessin (le dessin est élargi s'il manque de place).

`TRAY_MIN_MM` constante

```
const TRAY_MIN_MM = 12
```

Largeur minimale d'une goulotte : 12 mm, toujours réservée dans la zone latérale, même avec des emplacements 0U (ROU-01).

`Box` interface

```
interface Box { x; y; width; height }
```

Rectangle aligné sur les axes, utilisé dans tout le modèle (armoire, page, alignement).

Champs

- `x` abscisse du bord gauche ;
- `y` ordonnée du bord haut (l'axe y descend) ;
- `width` largeur ;
- `height` hauteur.

L'unité dépend de l'usage : mm d'armoire dans ce module, mm de papier pour une page.

`Span` interface

```
interface Span { x; width }
```

Intervalle horizontal d'un élément vertical du dessin (montant, goulotte, emplacement 0U).

Champs

- `x` abscisse du bord gauche, mm d'armoire ;
- `width` largeur, mm.

`SideGeometry` interface

```
interface SideGeometry { numbering; rail; tray; zeroUSlots }
```

Éléments verticaux d'un côté (gauche ou droit, vu depuis la face dessinée) d'une armoire.

Champs

- `numbering` colonne de numérotation des U ;
- `rail` montant où se fixent les oreilles des équipements (trous de fixation) ;
- `tray` goulotte verticale, entre les emplacements 0U et la numérotation ;
- `zeroUSlots` emplacements 0U, de l'extérieur vers l'intérieur (l'indice est le numéro `slot` du placement 0U).

`RackGeometry` interface

```
interface RackGeometry { face; width; height; cabinet; uArea; left; right; plinthBottom;
heightU }
```

Géométrie complète d'une face d'armoire, en mm d'armoire.

Champs

- `face` face dessinée ;
- `width` largeur totale du dessin (au moins la largeur de l'armoire, élargie si les 0U et les goulottes manquent de place ; identique pour les deux faces) ;

`height` hauteur totale, roulettes ou pieds compris ;

`cabinet` rectangle du caisson (cadre), sans roulettes ni pieds ;

`uArea` zone des U, de la largeur d'une façade 19″ ou 10″, sous le toit ;

`left` , `right` : `SideGeometry` de chaque côté ;

`plinthBottom` bas du socle, où commencent roulettes ou pieds ;

`heightU` hauteur de l'armoire en U.

`rackGeometry` fonction

```
function rackGeometry(rack: Rack, face: FaceSide): RackGeometry
```

Calcule la géométrie d'une face d'armoire (ARM-04, décision du lot 08). Horizontalement, de l'extérieur vers l'intérieur de chaque côté : cadre, zone latérale (emplacements 0U de 30 à 40 mm à l'extérieur, goulotte d'au moins 12 mm à l'intérieur), numérotation, montant, puis la zone des U centrée. La largeur est celle de l'armoire, ou plus si le côté le plus chargé (toutes faces confondues) ne tient pas. Verticalement : toit (fermée, murale) ou cadre (ouverte), zone des U, socle (au sol) ou cadre (murale), puis pieds ou roulettes. Appelée par tout ce qui dessine ou mesure une armoire.

Paramètres

`rack` l'armoire ;

`face` face dessinée (gauche et droite s'entendent vus depuis cette face).

Résultat. Un `RackGeometry` neuf.

Erreurs. Aucune.

`uSpan` fonction

```
function uSpan(geometry: RackGeometry, u: number): { top: number; bottom: number }
```

Haut et bas d'une U dans le dessin.

Paramètres

`geometry` géométrie de la face ;

`u` numéro physique de l'U (depuis le bas, à partir de 1).

Résultat. `{ top, bottom }` en mm d'armoire, `bottom = top + U_MM`. Aucune borne n'est contrôlée : une U hors de l'armoire donne une position hors de la zone des U.

Erreurs. Aucune.

`deviceBox` fonction

```
function deviceBox(geometry: RackGeometry, uBottom: number, heightU: number, halfSide:
LateralSide | null = null): Box
```

Rectangle d'un équipement fixé dans les U : toute la largeur de la zone des U, ou sa moitié gauche ou droite pour une demi-largeur. Utilisé par le rendu, la géométrie des pages, le routage et l'aperçu de l'éditeur de machine.

Paramètres

`geometry` géométrie de la face ;

`uBottom` U basse ;

`heightU` hauteur en U ;

`halfSide` côté vu depuis la face dessinée, ou `null` (défaut) pour toute la largeur.

Résultat. `Box` en mm d'armoire.

Erreurs. Aucune.

`uNumber` fonction

```
function uNumber(rack: Pick<Rack, 'heightU' | 'uNumbering'>, u: number): number
```

Numéro affiché d'une U selon la numérotation de l'armoire (ARM-04) : identique au numéro physique pour `bottomUp`, inversé pour `topDown`. Les positions sont toujours stockées depuis le bas ; seul l'affichage change (dessin, explorateur, propriétés, infobulle).

Paramètres

`rack` l'armoire (champs `heightU` et `uNumbering`) ;

`u` numéro physique.

Résultat. Numéro à afficher.

Erreurs. Aucune.

`zeroUSlotBox` fonction

```
function zeroUSlotBox(geometry: RackGeometry, side: LateralSide, slot: number): Box
| undefined
```

Rectangle d'un emplacement 0U : sa largeur et toute la hauteur de la zone des U.

Paramètres

`geometry` géométrie de la face ;

`side` côté vu depuis cette face ;

`slot` numéro de l'emplacement (à partir de 0, de l'extérieur vers l'intérieur).

Résultat. `Box`, ou `undefined` si l'emplacement n'existe pas.

Erreurs. Aucune.

`HOLE_OFFSETS_MM` constante

```
const HOLE_OFFSETS_MM = [6.35, 22.225, 38.1] as const
```

Positions des trois trous de fixation (écrous cage) d'une U, depuis son haut : 6,35 ; 22,225 et 38,1 mm (cotes EIA-310). Lu par le rendu des montants.

`HOLE_SIZE_MM` constante

```
const HOLE_SIZE_MM = 6
```

Côté du trou de fixation carré dessiné sur les montants : 6 mm.

zeroUDeviceBox

fonction

```
function zeroUDeviceBox(geometry: RackGeometry, side: LateralSide, slot: number, lengthU: number | null): Box | undefined
```

Rectangle d'un équipement 0U dans son emplacement : `lengthU` U de long, centré sur la hauteur de la zone des U ; tout l'emplacement quand la longueur est inconnue ou dépasse la hauteur de l'armoire.

Paramètres

`geometry` géométrie de la face ;

`side` côté ;

`slot` numéro de l'emplacement ;

`lengthU` longueur de la machine en U (paramètre de l'équipement 0U), ou `null`.

Résultat. `Box`, ou `undefined` si l'emplacement n'existe pas.

Erreurs. Aucune.

URange

interface

```
interface URange { from; to }
```

Plage d'U physiques montrée par une vue d'armoire (PAG-16), stockée dans `view.uRange`.

Champs

`from` U basse, comptée depuis le bas, comprise ;

`to` U haute, comprise ; `from` $\leq$ `to`.

CUT_MARGIN_MM

constante

```
const CUT_MARGIN_MM = 40
```

Longueur du cadre gardée au-delà d'une limite de plage coupée : 40 mm d'armoire, où le cadre se termine par le zigzag (décision du lot 47).

ZIGZAG_DEPTH_MM

constante

```
const ZIGZAG_DEPTH_MM = 16
```

Profondeur du zigzag d'une ligne de coupure : 16 mm d'armoire, à l'intérieur de la marge `CUT_MARGIN_MM`.

ZIGZAG_TOOTH_MM

constante

```
const ZIGZAG_TOOTH_MM = 32
```

Largeur visée d'une dent du zigzag : 32 mm d'armoire (le nombre de dents est arrondi pour couvrir exactement la largeur du dessin).

`effectiveURange` fonction

```
function effectiveURange(range: URange | null, heightU: number): URange | null
```

Plage réellement montrée pour une armoire : bornes ramenées dans `[1, heightU]`, ce qui garde valide la plage d'une armoire rendue plus basse ; une plage qui couvre toute l'armoire équivaut à `null`.

Paramètres

`range` plage stockée dans la vue, ou `null` ;

`heightU` hauteur actuelle de l'armoire.

Résultat. `URange` ramenée, ou `null` pour « toute l'armoire » (aussi quand `heightU` est inférieur à 1).

Erreurs. Aucune.

```
effectiveURange({ from: 3, to: 60 }, 42); // { from: 3, to: 42 }
effectiveURange({ from: 1, to: 42 }, 42); // null
```

`RackWindow` interface

```
interface RackWindow { top; bottom; clipTop; clipBottom; cutTop; cutBottom; range }
```

Partie du dessin d'armoire montrée par une vue (mm d'armoire du dessin entier).

Champs

`top` , `bottom` : haut et bas de la vue ; avec une coupure, la marge de 40 mm au-delà de la limite est comprise ;

`clipTop` , `clipBottom` : bande où le contenu (équipements, montants, numérotation) est dessiné, coupée net aux limites des U ;

`cutTop` , `cutBottom` : vrai quand le haut (le bas) est coupé, donc sans toit (sans socle ni pieds) et terminé par un zigzag ;

`range` plage effective, `null` pour toute l'armoire.

`rackWindow` fonction

```
function rackWindow(geometry: RackGeometry, range: URange | null): RackWindow
```

Calcule la fenêtre d'une vue d'armoire (PAG-16) à partir de sa géométrie et de sa plage. Sans plage, toute la hauteur du dessin. Utilisée via `rackViewFrame` (`model/pageGeometry`) partout où l'on passe du mm d'armoire à la page : rendu, routage, glisser-déposer, étiquettes.

Paramètres

`geometry` géométrie de la face ;

`range` plage stockée (`null` pour toute l'armoire) ; elle est ramenée par `effectiveURange`.

Résultat. `RackWindow`.

Erreurs. Aucune.

`cutLines` fonction

```
function cutLines(geometry: RackGeometry, window: RackWindow): { top: { x: number; y:
number }[] | null; bottom: { x: number; y: number }[] | null }
```

Lignes de coupure d'une fenêtre, en zigzag sur toute la largeur du dessin : celle du haut de gauche à droite, celle du bas de droite à gauche, pour que `cutOutline` les enchaîne. Les pointes extérieures sont sur le bord de la fenêtre, les creux à `ZIGZAG_DEPTH_MM` vers l'intérieur.

Paramètres

`geometry` géométrie (seule la largeur compte) ;

`window` fenêtre de la vue.

Résultat. `{ top, bottom }` : listes de points `{ x, y }` en mm d'armoire, `null` pour un bord non coupé.

Erreurs. Aucune.

`cutOutline` fonction

```
function cutOutline(geometry: RackGeometry, window: RackWindow): { x: number; y: number }[]
```

Contour du cadre visible : un rectangle dont les bords coupés sont des zigzags. Sert de chemin de découpe (`clipPath`) au rendu.

Paramètres

`geometry` géométrie ;

`window` fenêtre de la vue.

Résultat. Polygone fermé implicitement, liste de points en mm d'armoire, dans le sens des aiguilles d'une montre.

Erreurs. Aucune.

`cutFrameBottom` fonction

```
function cutFrameBottom(window: RackWindow): number
```

Ordonnée la plus basse du cadre plein au-dessus d'une coupure basse (où commence le zigzag) : `window.bottom - ZIGZAG_DEPTH_MM`. Utilisée par le routage pour faire sortir les câbles entre la limite et le zigzag.

Paramètres

`window` fenêtre de la vue.

Résultat. Ordonnée en mm d'armoire.

Erreurs. Aucune.

`inWindow` fonction

```
function inWindow(window: RackWindow, y0: number, y1: number): boolean
```

Dit si une étendue verticale est visible dans la bande dessinée d'une fenêtre (au moins une partie strictement entre `clipTop` et `clipBottom`, à 10^{-6} près). Utilisée pour savoir si un équipement ou un 0U est dessiné dans une vue à plage d'U.

Paramètres

`window` fenêtre ;

`y0` , `y1` : haut et bas de l'étendue, mm d'armoire.

Résultat. Booléen.

Erreurs. Aucune.

19.3 `model/dropTarget`

Placement des équipements à la souris et au clavier (EQP-01, EQP-03, EQP-04, EQP-06, EQP-09) : vue d'armoire et U sous le pointeur, position libre la plus proche (copier, coller, dupliquer), noms automatiques et uniques, déplacement par les flèches, retournement. Dépend de `model/placement`, `model/rackGeometry`, `model/pageGeometry` et `model/pageLayout`. Utilisé par la zone de dessin (`ui/canvas/Canvas.tsx`), les actions des équipements (`store/deviceActions.ts`) et la fenêtre « Placer un équipement ». Tests : `dropTarget.test.ts`.

`rackViewAt` fonction

```
function rackViewAt(project: Project, page: Page, point: Point): { view: View; local: Point } | undefined
```

Vue d'armoire de la page sous un point, avec ce point converti en mm d'armoire du dessin entier (décalé de `window.top` pour une vue à plage d'U). Les vues sont parcourues de la dernière dessinée à la première : la vue du dessus l'emporte. Les vues d'équipements extérieurs et les vues d'armoires inconnues sont ignorées. Le point est accepté dans tout le rectangle de la vue, marges de coupure comprises.

Paramètres

`project` le projet ;

`page` la page ;

`point` point de la page, mm de papier.

Résultat. `{ view, local }`, ou `undefined` hors de toute vue d'armoire.

Erreurs. Aucune.

`placementAt` fonction

```
function placementAt(project: Project, page: Page, point: Point, model: Machine, grabU = model.heightU > 0 ? model.heightU - 1 : 0): Placement | undefined
```

Placement d'une machine déposée en un point d'une vue d'armoire (EQP-01, décision du lot 16), aimanté sur l'U sous le pointeur, sur la face montrée par la vue. Une demi-largeur prend la moitié sous le pointeur (vue depuis la face affichée, donc côté stocké). Une machine 0U va dans l'emplacement 0U sous le pointeur. Le placement est ramené dans la plage d'U de la vue (PAG-16) ; il n'est pas contrôlé (l'appelant passe le résultat à `checkPlacement` pour l'aperçu vert, orange ou rouge).

Paramètres

`project` le projet ;

`page` la page ;

`point` position du pointeur, mm de papier ;

`model` la machine déposée ;

`grabU` nombre d'U entre le bas de l'équipement et le pointeur ; par défaut `heightU - 1` (un modèle venu de la bibliothèque est tenu par sa U du haut) ; un équipement déplacé garde l'U par laquelle il a été saisi.

Résultat. `Placement` (`'rack'` ou `'zeroU'`), ou `undefined` : pas de vue d'armoire sous le point, machine extérieure, pointeur à plus d'une U au-dessus ou au-dessous de la zone des U, hors d'un emplacement 0U, 0U hors de la plage, ou machine plus haute que la plage de la vue.

Erreurs. Aucune.

`nearestFreePlacement` fonction

```
function nearestFreePlacement(project: Project, machineId: Id, preferred: RackPlacement
| ZeroUPlacement, excludeDeviceId?: Id): Placement | undefined
```

Position libre la plus proche d'une position préférée (copier / coller, dupliquer, décision du lot 16). Dans les U : l'U préférée, puis alternativement une U plus haut et une U plus bas, à distance croissante ; une demi-largeur essaie ses deux moitiés à chaque U (EQP-03). En 0U : l'emplacement préféré, puis tous ceux de la même face (gauche puis droite), puis ceux de l'autre face. Un dépassement de profondeur compte comme un refus.

Paramètres

`project` le projet ;

`machineId` la machine à placer ;

`preferred` position préférée (dans les U ou 0U), qui fixe l'armoire ;

`excludeDeviceId` facultatif : équipement déplacé, ignoré dans les conflits.

Résultat. Le premier placement accepté, ou `undefined` si l'armoire est pleine, ou si l'armoire ou la machine est inconnue.

Erreurs. Aucune.

`devicePrefix` fonction

```
function devicePrefix(kind: EquipmentKind): string
```

Préfixe des noms automatiques selon l'équipement (« SRV », « SW »…), lu dans `defaults.devicePrefixes` de `fr.json`.

Paramètres

`kind` nature d'équipement (`EquipmentKind`).

Résultat. Le préfixe.

Erreurs. Aucune (une clé absente donne le texte de repli de `t`).

`nextDeviceName` fonction

```
function nextDeviceName(project: Project, prefix: string): string
```

Premier nom libre formé du préfixe et d'un compteur sur deux chiffres : SRV01, SRV02… Appelé à la pose d'un équipement.

Paramètres

`project` le projet (noms existants) ;

`prefix` préfixe, en général donné par `devicePrefix`.

Résultat. Un nom qu'aucun équipement ne porte (comparaison exacte).

Erreurs. Aucune.

`uniqueDeviceName` fonction

```
function uniqueDeviceName(project: Project, name: string, taken = new Set<string>()):
string
```

Nom non utilisé par un autre équipement, pour un collage ou une duplication : le nom lui-même s'il est libre, sinon son nombre final augmenté en gardant le nombre de chiffres (SRV07 → SRV08), ou, sans nombre final, le nom suivi de « _02 », « _03 »…

Paramètres

`project` le projet ;

`name` nom voulu ;

`taken` noms déjà réservés en plus de ceux du projet (équipements collés dans la même action), vide par défaut.

Résultat. Nom libre.

Erreurs. Aucune.

```
const names = new Set<string>();
const name = uniqueDeviceName(project, 'SRV07', names); // « SRV08 » si SRV07 existe
names.add(name);
```

`moveDevicesBy` fonction

```
function moveDevicesBy(project: Project, deviceIds: readonly Id[], deltaU: number): { ok:
true; placements: Map<Id, Placement> } | { ok: false; message: string }
```

Déplace des équipements fixés dans les U d'un nombre d'U (flèches haut et bas, EQP-06), tout ou rien. Les équipements sont traités du plus éloigné dans le sens du mouvement au plus proche, et chaque nouvelle position est contrôlée contre les équipements déjà déplacés. Un équipement déjà forcé garde son forçage ; un nouveau dépassement de profondeur est refusé (le forçage passe par le glisser ou les propriétés). Les équipements 0U ou extérieurs de la liste sont ignorés.

Paramètres

`project` le projet ;

`deviceIds` équipements sélectionnés ;

`deltaU` nombre d'U, positif vers le haut.

Résultat. `{ ok: true, placements }` (nouvelle position par équipement), ou `{ ok: false, message }` avec les messages du premier refus, séparés par des sauts de ligne.

Erreurs. Le refus est rendu comme valeur, jamais levé.

`FlipResult` type

```
type FlipResult = | { ok: true; placements: Map<Id, Placement>; forced: Set<Id>; warnings:
string[] } | { ok: false; messages: string[]; forceable: boolean };
```

Résultat de `flipDevices` :

Champs

`ok` (`true`) `placements` : nouvelle position par équipement ; `forced` : équipements à marquer « profondeur forcée » ; `warnings` : messages des dépassements acceptés ;

`ok` (`false`) `messages` : tous les messages de refus ; `forceable` : vrai si tous les refus sont des dépassements de profondeur.

`flipDevices` fonction

```
function flipDevices(project: Project, deviceIds: readonly Id[], forced = false):
FlipResult
```

Retourne des équipements fixés dans les U (EQP-09, décision du lot 42) : chacun passe sur l'autre face, à la même U et du même côté physique (`flippedPlacement`). Tous les équipements quittent d'abord leur face, si bien que deux équipements d'une même U peuvent échanger leurs faces ; chacun est ensuite contrôlé contre ceux déjà retournés. Tout ou rien. Après succès, un second passage sur les positions finales marque comme forcés tous les équipements concernés par un dépassement, pas seulement le dernier contrôlé.

Paramètres

`project` le projet ;

`deviceIds` équipements à retourner (0U et extérieurs ignorés) ;

`forced` accepter les dépassements de profondeur (« Forcer »), faux par défaut ; un équipement déjà forcé l'est de toute façon.

Résultat. `FlipResult`.

Erreurs. Les refus sont rendus comme valeur.

19.4 `model/depthInfo`

Infobulle de profondeur d'une U (EQP-05, lot 40) : ce qui est fixé sur l'U survolée à l'avant et à l'arrière, la profondeur prise et la profondeur restante. Dépend de `rackViewAt`, `rackOccupation` et de `model/rackGeometry`. Utilisé par la zone de dessin (`ui/canvas/Canvas.tsx`). Tests : `depthInfo.test.ts`.

`UDepthInfo` interface

```
interface UDepthInfo { rack; u; label; occupation; front; rear }
```

Contenu de l'infobulle de profondeur d'une U.

Champs

`rack` l'armoire ;

`u` numéro physique de l'U (depuis le bas) ;

`label` numéro affiché selon la numérotation de l'armoire ;

`occupation` `UOccupation` de l'U (profondeurs en mm) ;

`front` équipements fixés à l'avant sur cette U ;

`rear` équipements fixés à l'arrière.

`uDepthAt` fonction

```
function uDepthAt(project: Project, page: Page, point: Point): UDepthInfo | undefined
```

U d'une vue d'armoire sous un point de la page, avec son occupation en profondeur (EQP-05). Le point est cherché dans la zone des U, sur toute la largeur de la vue ; sur une vue limitée à une plage de U (PAG-16), seulement dans la bande dessinée (entre `clipTop` et `clipBottom`) : les marges coupées en zigzag ne montrent aucune U. Les équipements 0U ne comptent pas.

Paramètres

- `project` le projet ;
- `page` la page ;
- `point` point de la page, mm de papier.

Résultat. `UDepthInfo`, ou `undefined` hors de la zone des U d'une vue d'armoire ou de sa plage.

Erreurs. Aucune.

19.5 `model/links`

Règles des liaisons (LIA-03, LIA-04, LIA-06, LIA-07, LIA-09, LIA-11) : une seule liaison par port (refus), compatibilité des connecteurs (avertissement), type de liaison proposé, repères automatiques et doublons, chemin de bout en bout à travers les panneaux de brassage. Dépend de `model/lookup` (`indexProject`, `resolvePort`, `portKey`, `portDisplayName`). Utilisé par les actions du magasin, l'import tableur, la fenêtre de liaison, le tableau des liaisons, les propriétés, les infobulles, le routage (renvois), les exports (tableaux, étiquettes), la recherche et `model/anomalies`. Tests : `links.test.ts`.

`LinkEndProblem` type

```
type LinkEndProblem = | { kind: 'samePort' } | { kind: 'unknownPort'; end: 'a' | 'b' } | { kind: 'portDisabled'; end: 'a' | 'b' } | { kind: 'portBusy'; end: 'a' | 'b'; linkId: Id };
```

Raison bloquante de refuser une liaison entre deux ports (LIA-06) :

Champs

- `samePort` les deux extrémités sont le même port ;
- `unknownPort` le port de l'extrémité `end` (`'a'` ou `'b'`) n'existe pas ;
- `portDisabled` le port de l'extrémité `end` est désactivé sur l'équipement ;
- `portBusy` le port de l'extrémité `end` porte déjà la liaison `linkId` (et n'est pas marqué « multiple »).

`LinkEndMessage` interface

```
interface LinkEndMessage { problem; message }
```

Problème d'extrémité avec son message.

Champs

- `problem` le `LinkEndProblem` ;

`message` phrase française (clés `links.*`) qui nomme le port et, pour un port occupé, le repère de la liaison existante.

`portLabel` — fonction

```
function portLabel(project: Project, ref: PortRef): string
```

Libellé « équipement · port » d'une référence de port, pour les messages, les renvois, les étiquettes et les tableaux. Le nom du port est le nom d'affichage (renommage éventuel sur l'équipement).

Paramètres

`project` le projet ;

`ref` la référence `{ deviceId, portId }`.

Résultat. Le libellé, ou le texte `links.unknownPortLabel` si le port n'existe pas.

Erreurs. Aucune.

`linksOnPort` — fonction

```
function linksOnPort(project: Project, ref: PortRef): Link[]
```

Liaisons attachées à un port, lues dans l'index du projet.

Paramètres

`project` le projet ;

`ref` le port.

Résultat. Tableau de liaisons (vide si aucune).

Erreurs. Aucune.

`checkLinkEnds` — fonction

```
function checkLinkEnds(project: Project, a: PortRef, b: PortRef, excludeLinkId?: Id):
LinkEndMessage[]
```

Contrôle bloquant des extrémités d'une liaison (LIA-06) : refus si les deux extrémités sont le même port, si un port n'existe pas, est désactivé ou porte déjà une autre liaison (sauf port « multiple »). Appelé à la création et à la modification d'une liaison, et par l'import.

Paramètres

`project` le projet ;

`a` , `b` : les deux ports ;

`excludeLinkId` facultatif : liaison en cours de modification, ignorée.

Résultat. Liste de `LinkEndMessage` ; vide quand la liaison est permise.

Erreurs. Aucune exception.

`connectorsCompatible` fonction

```
function connectorsCompatible(x: ConnectorType, y: ConnectorType): boolean
```

Deux connecteurs sont compatibles s'ils sont identiques, complémentaires (champ `complementId`, dans un sens ou dans l'autre, par exemple C13 et C14) ou de la même famille (un module SFP+ et un connecteur LC). Décision du lot 03.

Paramètres

`x` , `y` : les deux types de connecteurs.

Résultat. Booléen.

Erreurs. Aucune.

`CompatibilityProblem` type

```
type CompatibilityProblem = | { kind: 'connectorNotAllowed'; end: 'a' | 'b'; connectorId:
Id } | { kind: 'connectorsIncompatible'; connectorA: Id; connectorB: Id };
```

Avertissement de compatibilité (non bloquant, LIA-06) :

Champs

`connectorNotAllowed` le connecteur `connectorId` de l'extrémité `end` n'est pas admis par le type de liaison ;

`connectorsIncompatible` les connecteurs `connectorA` et `connectorB` ne sont pas compatibles entre eux.

`CompatibilityResult` interface

```
interface CompatibilityResult { status; problems }
```

Résultat de `checkCompatibility`.

Champs

`status` `'ok'` ou `'warning'` (au moins un problème) ;

`problems` problèmes, chacun avec son message français.

`checkCompatibility` fonction

```
function checkCompatibility(project: Project, a: PortRef, b: PortRef, linkTypeId: Id):
CompatibilityResult
```

Contrôle non bloquant d'une liaison (LIA-06) : chaque connecteur doit être admis par le type de liaison (si le type restreint ses connecteurs), et les deux connecteurs doivent être compatibles (`connectorsCompatible`). Un port, un connecteur ou un type inconnu ne produit pas d'avertissement pour la partie concernée. L'interface propose « Créer quand même » ; le panneau Vérifications liste les liaisons en avertissement.

Paramètres

`project` le projet ;

`a` , `b` : les deux ports ;

`linkTypeId` le type de liaison.

Résultat. `CompatibilityResult`.

Erreurs. Aucune.

`proposeLinkType` fonction

```
function proposeLinkType(project: Project, connectorA: Id | null, connectorB: Id
| null, defaultLinkTypeId?: Id): Id | undefined
```

Type de liaison proposé pour deux connecteurs (LIA-03, décision du lot 03) : le premier type de la bibliothèque (dans son ordre) qui admet les deux, sinon le premier qui en admet un, sinon le type par défaut, sinon le premier type de la bibliothèque.

Paramètres

`project` le projet (types de liaison de sa bibliothèque) ;

`connectorA` , `connectorB` : types de connecteurs, ou `null` ;

`defaultLinkTypeId` facultatif : type par défaut (Ethernet en pratique).

Résultat. Identifiant de type, ou `undefined` si la bibliothèque n'a aucun type.

Erreurs. Aucune.

`proposeLinkTypeForPorts` fonction

```
function proposeLinkTypeForPorts(project: Project, a: PortRef, b:
PortRef, defaultLinkTypeId?: Id): Id | undefined
```

Même règle que `proposeLinkType`, à partir de deux ports (leurs types de connecteurs). Utilisé par la fenêtre de liaison, le tirage d'une liaison et l'import.

Paramètres

`project` le projet ;

`a` , `b` : les deux ports (un port inconnu compte comme sans connecteur) ;

`defaultLinkTypeId` facultatif : type par défaut.

Résultat. Identifiant de type, ou `undefined`.

Erreurs. Aucune.

`refNumber` fonction

```
function refNumber(ref: string, prefix: string): number | null
```

Numéro d'un repère formé d'un préfixe suivi de chiffres seulement.

Paramètres

`ref` le repère ;

`prefix` le préfixe du type de liaison.

Résultat. Le nombre (`refNumber('ETH-012', 'ETH-')` vaut 12), ou `null` si le repère ne commence pas par le préfixe ou si la suite n'est pas faite de chiffres.

Erreurs. Aucune.

`nextLinkRef` fonction

```
function nextLinkRef(project: Project, linkTypeId: Id): string
```

Repère automatique suivant d'un type de liaison (LIA-04) : préfixe du type, puis le plus grand numéro existant avec ce préfixe plus un, sur trois chiffres au moins. Les trous ne sont pas réutilisés.

Paramètres

`project` le projet ;

`linkTypeId` le type de liaison (préfixe vide s'il est inconnu).

Résultat. Le repère, par exemple « ETH-013 ».

Erreurs. Aucune.

`duplicateRefs` fonction

```
function duplicateRefs(project: Project): Map<string, Id[]>
```

Repères portés par plusieurs liaisons (LIA-04, LIA-07). Les repères sont comparés après suppression des espaces de début et de fin ; les repères vides sont ignorés.

Paramètres

`project` le projet.

Résultat. Table repère → identifiants des liaisons qui le portent (deux au moins).

Erreurs. Aucune.

`PathHop` interface

```
interface PathHop { link; from; to }
```

Une liaison d'un chemin de bout en bout, orientée de l'extrémité finale A vers l'extrémité finale B.

Champs

`link` la liaison ;

`from` port de départ de ce tronçon dans le sens A → B ;

`to` port d'arrivée.

`EndToEndPath` interface

```
interface EndToEndPath { hops; endA; endB; loop }
```

Chemin de bout en bout d'une liaison (LIA-09).

Champs

`hops` liaisons dans l'ordre, de l'extrémité finale A à l'extrémité finale B ;

`endA` , `endB` : ports finaux (les extrémités de la liaison elle-même quand aucun panneau de brassage n'est traversé) ;

`loop` une boucle a été détectée ; le chemin s'arrête là où il reviendrait.

`pairedPort` fonction

```
function pairedPort(project: Project, ref: PortRef): PortRef | null
```

Port de traversée associé à un port d'un panneau de brassage (champ `pairedPortId` du port), sur l'autre face du même équipement.

Paramètres

`project` le projet ;

`ref` le port.

Résultat. Référence du port associé, ou `null` (port inconnu ou sans traversée).

Erreurs. Aucune.

`endToEndPath` fonction

```
function endToEndPath(project: Project, linkId: Id): EndToEndPath | undefined
```

Calcule le chemin de bout en bout d'une liaison à travers les panneaux de brassage, sans limite de nombre (LIA-09). Depuis chaque extrémité, tant que le port atteint a un port de traversée relié par une autre liaison, le chemin continue ; une liaison déjà vue arrête la marche et signale une boucle. Les deux moitiés sont ensuite mises bout à bout, la moitié côté A retournée. Utilisé par les propriétés de liaison, le tableau des liaisons, les exports et les anomalies (`pathLoop`).

Paramètres

`project` le projet ;

`linkId` la liaison de départ.

Résultat. `EndToEndPath`, ou `undefined` si la liaison n'existe pas.

Erreurs. Aucune.

```
const path = endToEndPath(project, link.id);
if (path) {
  const route = path.hops.map((hop) => hop.link.ref).join(' > ');
  const ends = [portLabel(project, path.endA), portLabel(project, path.endB)];
}
```

`pathElements` fonction

```
function pathElements(project: Project, linkId: Id): { linkIds: Id[]; ports: PortRef[] }
| undefined
```

Tout ce que traverse le chemin d'une liaison (LIA-11) : ses liaisons, de A vers B, et chaque port qu'elles relient, dans le même ordre (deux ports par liaison). Sert à la mise en évidence du chemin à l'écran (`ui/canvas/Highlights.tsx`).

Paramètres

`project` le projet ;

`linkId` la liaison sélectionnée.

Résultat. `{ linkIds, ports }`, ou `undefined` si la liaison n'existe pas.

Erreurs. Aucune.

19.6 `model/anomalies`

Liste des anomalies du projet pour le panneau Vérifications (LIA-07, EQP-08). Chaque anomalie a une gravité et désigne les entités à sélectionner sur le schéma. Dépend de `model/links`, `model/placement`, `model/lookup` et `model/equipment`. Utilisé par `ui/panels/ChecksPanel.tsx` (liste et pastille du nombre d'anomalies). Tests : dans `links.test.ts`.

`AnomalyKind` type

```
type AnomalyKind = | 'danglingLink' | 'portOverbooked' | 'disabledPortLinked'
| 'incompatibleLink' | 'duplicateRef' | 'emptyRef' | 'duplicateDeviceName'
| 'placementConflict' | 'forcedDepth' | 'machineWithoutDepth' | 'orphanPort' | 'pathLoop';
```

Nature d'une anomalie ; le message vient de la clé `anomalies.<nature>` de `fr.json`. Quatre natures sont des erreurs (marquées ci-dessous, ensemble `ERRORS` du module), les autres sont des avertissements.

Champs

- `danglingLink` (erreur) liaison dont un port ou un équipement n'existe plus ;
- `portOverbooked` (erreur) port non « multiple » qui porte plusieurs liaisons ;
- `disabledPortLinked` liaison sur un port désactivé ;
- `incompatibleLink` avertissement de compatibilité (LIA-06) ;
- `duplicateRef` (erreur) repère porté par plusieurs liaisons ;
- `emptyRef` liaison sans repère ;
- `duplicateDeviceName` nom porté par plusieurs équipements (EQP-08, casse ignorée) ;
- `placementConflict` (erreur) placement refusé par les règles (chevauchement, profondeur non forcée, emplacement 0U…) ;
- `forcedDepth` dépassement de profondeur accepté avec « Forcer » ;
- `machineWithoutDepth` machine placée dans une armoire sans profondeur ;
- `orphanPort` port de panneau de brassage relié sur une face alors que son port de traversée ne l'est pas ;
- `pathLoop` le chemin de bout en bout de la liaison forme une boucle.

`AnomalyTarget` interface

```
interface AnomalyTarget { kind; id }
```

Entité désignée par une anomalie, sélectionnée au clic dans le panneau.

Champs

- `kind` `'device'`, `'link'`, `'machine'` ou `'rack'` ;
- `id` son identifiant.

`Anomaly` interface

```
interface Anomaly { kind; severity; message; targets }
```

Une anomalie du projet.

Champs

- `kind` sa nature (`AnomalyKind`) ;

`severity` `'error'` ou `'warning'` ;

`message` phrase française ;

`targets` entités à sélectionner (une ou plusieurs).

`listAnomalies` fonction

```
function listAnomalies(project: Project): Anomaly[]
```

Recalcule toutes les anomalies du projet, à chaque modification, dans cet ordre : pour chaque liaison, extrémité inexistante (qui arrête l'examen de la liaison), port surchargé (une seule fois par port), port désactivé, compatibilité et boucle de chemin ; repères en double puis vides ; noms d'équipements en double ; conflits de placement et dépassements forcés (pour chaque équipement non extérieur, via `checkDevicePlacement`) ; machines sans profondeur utilisées dans une armoire ; ports orphelins.

Paramètres

`project` le projet.

Résultat. Liste d'`Anomaly` ; vide pour un projet sans anomalie.

Erreurs. Aucune.

19.7 `model/instancePorts`

Ports ajoutés à un équipement placé seulement (PRT-04, lot 40), par exemple une carte d'extension : stockés dans `device.extraPorts`, jamais sur la machine. Nom par défaut, place libre sur la face, contrôles. Dépend de `model/lookup` (`devicePortEntries`), `model/modelEditor` (`faceTemplateSize`) et du schéma (`InstancePortSchema`). Utilisé par les actions des ports (`store/portActions.ts`). Tests : `instancePorts.test.ts`.

`INSTANCE_PORT_MM` constante

```
const INSTANCE_PORT_MM = { width: 14, height: 12 } as const
```

Taille d'un port ajouté sur la face : 14 mm de large et 12 mm de haut (une prise RJ45 et sa marge). Convertie en proportion de la face par `newInstancePort`.

Champs

`width` 14 mm ;

`height` 12 mm.

`instancePortName` fonction

```
function instancePortName(machine: Machine, device: Device): string
```

Premier nom « X1 », « X2 »… (clé `instancePorts.defaultName`) qu'aucun port de l'équipement ne porte, sur l'une ou l'autre face, ports de la machine et ports ajoutés compris.

Paramètres

`machine` la machine de l'équipement ;

`device` l'équipement.

Résultat. Le nom.

Erreurs. Aucune.

`freePortPosition` fonction

```
function freePortPosition(ports: readonly Pick<Port, 'x' | 'y' | 'w' | 'h'>[], w:
number, h: number): { x: number; y: number }
```

Place libre pour un port de taille `w` × `h` parmi des ports existants, en proportions de la face (0 à 1) : première case libre d'une grille au pas de 1,25 fois la taille, rangées centrées sur la hauteur de la face parcourues de haut en bas, cases de droite à gauche (le libellé et la plupart des ports sont à gauche), en gardant 6 % de marge à gauche et à droite (oreilles). Un port est libre s'il ne chevauche aucun autre, avec 10 % de marge autour du nouveau.

Paramètres

`ports` ports déjà sur la face (centre `x`, `y` et taille `w`, `h`, en proportions) ;

`w` , `h` : taille du nouveau port, en proportions.

Résultat. Centre `{ x, y }` arrondi à 10^{-4}, ou le centre de la face `{ x: 0.5, y: 0.5 }` si rien n'est libre.

Erreurs. Aucune.

`newInstancePort` fonction

```
function newInstancePort(project: Project, deviceId: Id, face: FaceSide, connectorTypeId:
Id): InstancePort | undefined
```

Nouveau port ajouté à un équipement, sur une face de sa machine (la face avant pour un équipement extérieur, selon l'appelant), à une place libre, nommé « X*n* », de 14 × 12 mm (au plus la moitié de la largeur et 90 % de la hauteur de la face), marqué placé. Appelé par la commande `device.addPort`.

Paramètres

`project` le projet ;

`deviceId` l'équipement ;

`face` face de la machine ;

`connectorTypeId` type de connecteur du port (RJ45 en pratique).

Résultat. Port validé par `InstancePortSchema`, avec un nouvel identifiant, ou `undefined` si l'équipement ou sa machine est inconnu.

Erreurs. `InstancePortSchema.parse` lève une erreur de validation si les valeurs calculées sont invalides (cas non rencontré avec une machine valide).

`instancePortProblems` fonction

```
function instancePortProblems(project: Project, device: Device, port: InstancePort):
string[]
```

Contrôle d'un port ajouté modifié dans les propriétés : nom vide, nom déjà utilisé sur la même face par un autre port de l'équipement, type de connecteur inconnu, port qui sort de la face (bords à 10^{-9} près).

Paramètres

`project` le projet ;

`device` l'équipement ;

`port` le port tel qu'il serait enregistré.

Résultat. Messages français (clés `instancePorts.*`) ; vide si le port est valide.

Erreurs. Aucune.

19.8 `model/arrange`

Disposition des éléments d'une page (PAG-08, lot 48) : grille et aimantation, guides d'alignement pendant un glisser, alignement et répartition d'une sélection. Pur : travaille sur des rectangles en mm de papier (`Box`) et rend des décalages. Utilisé par la zone de dessin (`ui/canvas/Canvas.tsx`, guides et grille) et les actions d'alignement (`store/annotationActions.ts`). Tests : `arrange.test.ts`.

`GRID_SNAP_STEP` constante

```
const GRID_SNAP_STEP = 5
```

Pas de la grille à l'écran et de l'aimantation sur la grille : 5 mm de papier. Lu par la zone de dessin et la couche d'interface qui dessine la grille.

`AlignMode` type

```
type AlignMode = 'left' | 'centerX' | 'right' | 'top' | 'centerY' | 'bottom';
```

Bord ou centre sur lequel aligner une sélection : `'left'`, `'centerX'`, `'right'` (horizontalement), `'top'`, `'centerY'`, `'bottom'` (verticalement). Une commande `arrange.align*` par valeur.

`DistributeAxis` type

```
type DistributeAxis = 'horizontal' | 'vertical';
```

Axe de répartition : `'horizontal'` ou `'vertical'`.

`unionBox` fonction

```
function unionBox(boxes: readonly Box[]): Box | undefined
```

Plus petit rectangle qui contient tous les rectangles donnés (rectangle de la sélection).

Paramètres

`boxes` rectangles.

Résultat. Le rectangle englobant, ou `undefined` pour une liste vide.

Erreurs. Aucune.

`alignOffsets` fonction

```
function alignOffsets(boxes: readonly Box[], mode: AlignMode): Point[]
```

Décalages qui alignent des rectangles sur un bord ou un centre de leur union : bords gauches sur le plus à gauche, centres sur le centre de l'union, etc.

Paramètres

`boxes` rectangles de la sélection ;

`mode` bord ou centre visé.

Résultat. Un décalage `{ x, y }` par rectangle, dans le même ordre, arrondi au centième de mm ; vide pour une liste vide.

Erreurs. Aucune.

```
const offsets = alignOffsets(
  [{ x: 10, y: 0, width: 20, height: 5 }, { x: 40, y: 8, width: 10, height: 5 }],
  'left',
); // [{ x: 0, y: 0 }, { x: -30, y: 0 }]
```

`distributeOffsets` fonction

```
function distributeOffsets(boxes: readonly Box[], axis: DistributeAxis): Point[]
```

Décalages qui répartissent des rectangles à écarts égaux le long d'un axe : le premier et le dernier (par position de leur bord gauche ou haut) restent en place, les autres sont espacés entre eux en gardant leur ordre. Moins de trois rectangles : rien ne bouge.

Paramètres

`boxes` rectangles ;

`axis` axe de répartition.

Résultat. Un décalage par rectangle, dans l'ordre reçu.

Erreurs. Aucune.

`snapToGrid` fonction

```
function snapToGrid(value: number, step = GRID_SNAP_STEP): number
```

Arrondit une valeur au multiple le plus proche du pas.

Paramètres

`value` valeur en mm ;

`step` pas, `GRID_SNAP_STEP` par défaut.

Résultat. Valeur arrondie (au centième pour éviter les erreurs de virgule flottante).

Erreurs. Aucune.

`Guide` interface

```
interface Guide { axis; position; from; to }
```

Guide d'alignement dessiné pendant un glisser.

Champs

`axis` `'x'` pour une ligne verticale, `'y'` pour une ligne horizontale ;

`position` abscisse (axe x) ou ordonnée (axe y) de la ligne, mm de papier ;

`from` , `to` : étendue de la ligne sur l'autre axe, qui couvre le rectangle déplacé et tous ceux qui touchent le guide.

`SnapOptions` interface

```
interface SnapOptions { tolerance; grid? }
```

Réglages de l'aimantation.

Champs

`tolerance` plus grande distance aimantée sur un guide, mm de papier (la zone de dessin y convertit 6 pixels d'écran) ;

`grid` facultatif : pas de la grille quand elle est active, utilisé si aucun guide n'est assez proche ; `null` ou absent sans grille.

`SnapResult` interface

```
interface SnapResult { dx; dy; guides }
```

Résultat de `snapMove`.

Champs

`dx` , `dy` : correction à ajouter au déplacement, mm ;

`guides` guides à dessiner (zéro, un ou deux).

`snapMove` fonction

```
function snapMove(moving: Box, others: readonly Box[], options: SnapOptions): SnapResult
```

Aimantation d'un rectangle déplacé : sur chaque axe, celui de ses bords ou de son centre qui est le plus proche d'un bord ou d'un centre d'un autre rectangle, à moins de la tolérance, s'y aligne et un guide est produit ; sinon, si la grille est active, son coin haut gauche va sur la grille.

Paramètres

`moving` rectangle à sa position courante (avant correction) ;

`others` rectangles des autres vues et éléments ;

`options` tolérance et grille.

Résultat. `SnapResult`.

Erreurs. Aucune.

```
const snap = snapMove(box, others, { tolerance: 6 / zoom, grid: gridOn ? GRID_SNAP_STEP : null });
const moved = { ...box, x: box.x + snap.dx, y: box.y + snap.dy };
```

`snapValue` fonction

```
function snapValue(value: number, targets: readonly number[], { tolerance, grid }:
SnapOptions): { value: number; guide: number | null }
```

Aimantation d'une valeur déplacée seule (bord d'un rectangle redimensionné, bout d'une ligne) : vers la cible la plus proche à moins de la tolérance, sinon vers la grille si elle est active.

Paramètres

`value` valeur courante ;

`targets` valeurs cibles (`snapTargets`) ;

`options` tolérance et grille.

Résultat. `{ value, guide }` : valeur corrigée et position du guide à dessiner, ou `null` sans guide.

Erreurs. Aucune.

`snapTargets` fonction

```
function snapTargets(boxes: readonly Box[], axis: 'x' | 'y'): number[]
```

Bords et centres de rectangles le long d'un axe : trois valeurs par rectangle (début, milieu, fin), cibles de `snapValue`.

Paramètres

`boxes` rectangles ;

`axis` `'x'` ou `'y'`.

Résultat. Liste de valeurs, non triée.

Erreurs. Aucune.

19.9 `model/autoLayout`

Mise en place automatique des vues sur une page : vues d'une armoire ajoutées à droite des vues existantes (lot 15, PAG-17), page générée pour une armoire (PAG-13), vue d'un équipement extérieur ajoutée à une page (EXT-04). Les vues existantes d'une page ne sont jamais déplacées ni réduites (décision de l'utilisateur du 2026-11-06, PAG-03). Toutes les mesures sont en mm de papier. Dépend de `model/pageGeometry` (`viewBounds`, `externalViewSize`, `VIEW_TITLE_GAP`), `model/pageLayout` (`drawingArea`, `sideLabelsWidth`), `model/factories` (`defaultLabelsBeside`) et `model/rackGeometry`. Utilisé par les actions des armoires et des pages (`store/rackActions.ts`, `store/pageActions.ts`). Tests : `rackPage.test.ts`, `store/rackActions.test.ts`.

`VIEW_GAP_MM` constante

```
const VIEW_GAP_MM = 12
```

Écart entre deux vues placées automatiquement : 12 mm de papier.

`rackFitScale` fonction

```
function rackFitScale(rack: Rack, page: Page): number
```

Plus grande échelle, par pas de 0,01 et au plus 1, à laquelle la face avant d'une armoire tient dans la hauteur de la zone de dessin, titre de la vue compris (et 8 mm de marge).

Paramètres

`rack` l'armoire ;

`page` la page (format, marges, cartouche).

Résultat. Échelle entre 0,01 et 1.

Erreurs. Aucune.

`rackViewsLayout` fonction

```
function rackViewsLayout(project: Project, page: Page, rack: Rack, faces: readonly
FaceSide[]): Omit<View, 'id'>[]
```

Vues d'une armoire (avant et/ou arrière) placées côte à côte à droite des vues existantes de la page, en haut de la zone de dessin. L'échelle est celle de la première vue d'armoire de la page ; sans vue d'armoire, la plus grande qui fait tenir l'armoire en hauteur et ses vues, étiquettes à côté comprises, en largeur. Les étiquettes à côté suivent `defaultLabelsBeside` (vraies pour une vue avant, PAG-17). Utilisé par l'ajout d'armoire, les commandes d'ajout de vue et la duplication d'armoire, qui ajoutent ces vues après celles de la page sans toucher à ces dernières : les nouvelles vues peuvent sortir de la zone de dessin (décision de l'utilisateur du 2026-11-06).

Paramètres

`project` le projet ;

`page` la page ;

`rack` l'armoire ;

`faces` faces à montrer, dans l'ordre (`['front', 'rear']` en général).

Résultat. Vues sans identifiant (`Omit<View, 'id'>`), à créer par l'appelant ; elles peuvent dépasser la zone de dessin.

Erreurs. Aucune.

`SIDE_LABELS_MM` constante

```
const SIDE_LABELS_MM = SIDE_LABELS.gap + SIDE_LABELS.width
```

Place prise à droite d'une armoire par ses étiquettes à côté (PAG-06) : écart plus largeur des étiquettes de `SIDE_LABELS` (`model/pageLayout`), soit 76 mm de papier.

`EXTERNAL_LABEL_MM` constante

```
const EXTERNAL_LABEL_MM = 10
```

Hauteur réservée sous un équipement extérieur pour son nom (EXT-02) : 10 mm de papier.

`EXTERNAL_LABEL_OVERHANG_MM` constante

```
const EXTERNAL_LABEL_OVERHANG_MM = 15
```

Débord du nom d'un équipement extérieur de chaque côté du dessin (EXT-02) : 15 mm de papier, comptés dans l'encombrement de la vue.

`EXTERNAL_MAX_SCALE` constante

```
const EXTERNAL_MAX_SCALE = 0.1
```

Plus grande échelle des équipements extérieurs d'une page générée, et échelle par défaut d'une vue extérieure ajoutée sans autre référence : 0,1.

`linkedExternalDevices` fonction

```
function linkedExternalDevices(project: Project, rackId: Id): Device[]
```

Équipements extérieurs reliés directement (une liaison) à un équipement de l'armoire, dans l'ordre du projet (PAG-13). Les extérieurs reliés seulement à travers un autre extérieur ne sont pas pris.

Paramètres

`project` le projet ;

`rackId` l'armoire.

Résultat. Équipements, sans doublon.

Erreurs. Aucune.

`viewFootprint` fonction

```
function viewFootprint(project: Project, page: Page, view: View): Box | undefined
```

Rectangle pris par une vue sur sa page : son dessin, plus les étiquettes à côté d'une armoire, ou le nom sous un équipement extérieur avec son débord des deux côtés. Le titre d'une vue d'armoire n'est pas compté.

Paramètres

`project` le projet ;

`page` la page ;

`view` la vue.

Résultat. `Box` en mm de papier, ou `undefined` si la cible de la vue n'existe pas.

Erreurs. Aucune.

`revisionTableHeight` fonction

```
function revisionTableHeight(project: Project, page: Page): number
```

Hauteur gardée libre au-dessus du cartouche pour le tableau des révisions de la page (PAG-14) : 4,2 mm par ligne affichée (au plus le nombre de lignes réglé), plus l'en-tête et 2 mm.

Paramètres

`project` le projet (révisions) ;

`page` la page (réglage du tableau).

Résultat. Hauteur en mm ; 0 si le tableau est masqué ou s'il n'y a aucune révision.

Erreurs. Aucune.

`rackPageLayout` fonction

```
function rackPageLayout(project: Project, page: Page, rack: Rack): Omit<View, 'id'>[]
```

Vues d'une page créée pour une armoire (« Créer une page pour cette armoire », PAG-13, lot 43) : vue avant (étiquettes à côté) et vue arrière côte à côte, puis les équipements extérieurs reliés en colonnes à leur droite, de haut en bas. Une seule échelle pour les deux vues d'armoire, la plus grande (pas de 0,01, au plus 1) à laquelle tout tient dans la zone de dessin au-dessus du tableau des révisions ; les extérieurs prennent cette échelle, 0,1 au plus. Un extérieur déjà montré sur une autre page garde ses ports masqués et sa taille forcée, mise à

l'échelle. L'algorithme part de l'échelle qui fait tenir l'armoire en hauteur et la réduit à chaque essai d'au moins 10 % et d'au moins un centième, jusqu'à ce que tout tienne ou que l'échelle atteigne 0,01.

Paramètres

`project` le projet ;

`page` la page, supposée vide ;

`rack` l'armoire.

Résultat. Vues sans identifiant, à créer par l'appelant.

Erreurs. Aucune.

`externalViewLayout` fonction

```
function externalViewLayout(project: Project, page: Page, deviceId: Id): Omit<View, 'id'>
| undefined
```

Vue d'un équipement extérieur ajoutée à une page (« Afficher l'équipement extérieur sur cette page », EXT-04) : à droite de l'encombrement des vues de la page, en haut de la zone de dessin (au coin haut gauche s'il ne reste pas de place), avec l'échelle, la taille forcée et les ports masqués de sa vue sur une autre page, sinon l'échelle des autres vues extérieures de la page, sinon `EXTERNAL_MAX_SCALE`.

Paramètres

`project` le projet ;

`page` la page ;

`deviceId` l'équipement extérieur.

Résultat. La vue sans identifiant, ou `undefined` si l'équipement n'existe pas ou n'est pas extérieur.

Erreurs. Aucune.

Chapitre 20

Référence : pages, vues, cartouche, Schéma, recherche et bibliothèques

Ce chapitre décrit, déclaration par déclaration, les modules du modèle qui servent la mise en page (formats, échelles, géométrie des vues et des équipements sur la page, cartouche), les éléments de l'onglet Schéma, la recherche globale, la logique de l'éditeur de machine et le traitement des bibliothèques (arborescence du panneau, import, export, mode édition). Tous ces modules sont purs : ils ne dépendent ni de React ni du magasin, travaillent en millimètres de papier (sauf mention contraire) et sont testés unitairement dans le fichier `*.test.ts` voisin. La conception d'ensemble est expliquée au chapitre « Pages, vues, cartouche et onglet Schéma ».

20.1 `model/annotations`

Géométrie pure des éléments de l'onglet « Schéma » (PAG-07) : textes, formes, lignes et flèches, images, cadres de zone, bulles, notes et tableaux, rattachés à une page (`Page.annotations`). Toutes les coordonnées sont en millimètres de papier ; les valeurs calculées sont arrondies au centième de millimètre. Le module dépend du schéma (`Annotation`, `Point`, `QuarterTurn`), de `Box` (`model/rackGeometry`) et de `newId`. Il est utilisé par les actions annulables (`src/store/annotationActions.ts`), la couche de dessin (`src/render/AnnotationsLayer.tsx`), la zone de dessin (sélection, poignées), le panneau `SchemaPanel`, les propriétés d'un élément et les commandes `annotation.*`. Décision du 2026-11-05, lot 48.

`AnnotationKind` type

```
type AnnotationKind = Annotation['kind'];
```

Union des valeurs du discriminant `kind` d'un élément de page : `text`, `line`, `arrow`, `rect`, `roundedRect`, `ellipse`, `image`, `zone`, `bubble`, `note` et `table`. Elle est dérivée du schéma et suit donc automatiquement l'ajout d'une sorte d'élément. À ne pas confondre avec `AnnotationPreset`, qui liste les éléments proposés par l'onglet (une « flèche double » est une `arrow` avec `doubleHead`).

`LineAnnotation` type

```
type LineAnnotation = Extract<Annotation, { kind: 'line' | 'arrow' }>;
```

Élément défini par une suite de points (`kind` `line` ou `arrow`) : `points` (au moins deux, en mm de page), `strokeWidth`, `dash`, `elbow` (tracé coudé entre le premier et le dernier point),

`elbowStart` (`horizontal` ou `vertical` : sens du premier segment du coude) et `doubleHead` (pointe aux deux bouts, flèche seulement). Une ligne n'a pas de rotation propre : la faire pivoter tourne ses points.

`BoxAnnotation` type

```
type BoxAnnotation = Exclude<Annotation, LineAnnotation>;
```

Tous les autres éléments : ceux qui occupent un cadre `x`, `y`, `width`, `height` (coin haut gauche et taille avant rotation, en mm) et portent une `rotation` par quart de tour autour du centre du cadre.

`isLineAnnotation` fonction

```
function isLineAnnotation(annotation: Annotation): annotation is LineAnnotation
```

Garde de type qui distingue une ligne ou une flèche d'un élément à cadre. Presque toutes les fonctions du module commencent par ce test, car les deux familles se déplacent, se redimensionnent et pivotent différemment.

Paramètres

`annotation` l'élément à tester.

Résultat. `true` pour `kind` `line` ou `arrow`.

Erreurs. Aucune.

`hasText` fonction

```
function hasText(annotation: Annotation): annotation is Extract<Annotation, { kind: 'text'
| 'bubble' | 'note' }>
```

Garde de type des éléments qui portent un texte saisi par l'utilisateur dans leur champ `text` : texte, bulle numérotée et note. Utilisée par les propriétés et par le double-clic, qui donne le focus au champ de texte. Un cadre de zone (champ `title`) et un tableau (`cells`) ne sont pas concernés.

Paramètres

`annotation` l'élément à tester.

Résultat. `true` pour `text`, `bubble` et `note`.

Erreurs. Aucune.

`pointsBox` fonction

```
function pointsBox(points: readonly Point[]): Box
```

Rectangle englobant d'une liste de points, utilisé pour le cadre d'une ligne et pour ses redimensionnement et rotation.

Paramètres

`points` les points (au moins un).

Résultat. Le plus petit `Box` contenant les points ; largeur ou hauteur nulle pour une ligne horizontale ou verticale.

Erreurs. Aucune exception ; une liste vide donne des coordonnées infinies (`Math.min()` sans argument), ce que le schéma interdit en amont.

`localBox` fonction

```
function localBox(annotation: Annotation): Box
```

Cadre de l'élément avant rotation : son `x`, `y`, `width`, `height`, ou le rectangle englobant des points pour une ligne. C'est dans ce repère que se calculent le centre de rotation et le dessin.

Paramètres

`annotation` l'élément.

Résultat. Un `Box` en mm de page.

Erreurs. Aucune.

`isSideways` fonction

```
function isSideways(rotation: QuarterTurn): boolean
```

Indique si un quart de tour échange la largeur et la hauteur visibles de l'élément (rotations de 90° et 270°). Sert à l'encombrement et au redimensionnement.

Paramètres

`rotation` la rotation, 0, 90, 180 ou 270.

Résultat. `true` pour 90 et 270.

Erreurs. Aucune.

`annotationBounds` fonction

```
function annotationBounds(annotation: Annotation): Box
```

Rectangle réellement couvert sur la page, rotation comprise : pour un cadre tourné d'un quart de tour, le rectangle de mêmes centre et dimensions échangées. C'est la boîte utilisée pour la sélection au rectangle, les poignées, les guides d'alignement et l'alignement / la répartition.

Paramètres

`annotation` l'élément.

Résultat. Un `Box` en mm de page. Pour une ligne, l'englobant de ses points (les pointes et l'épaisseur du trait ne sont pas comptées).

Erreurs. Aucune.

`rotationTransform` fonction

```
function rotationTransform(annotation: Annotation): string | undefined
```

Attribut SVG `transform` qui dessine un élément à cadre tourné autour du centre de son cadre, par exemple `rotate(90 120 80)`. Utilisé par `src/render/AnnotationsLayer.tsx` : l'élément est dessiné dans son cadre non tourné, puis la transformation l'applique.

Paramètres

`annotation` l'élément.

Résultat. La chaîne `rotate(angle cx cy)`, ou `undefined` pour une ligne ou une rotation nulle.

Erreurs. Aucune.

`translateAnnotation` fonction

```
function translateAnnotation(annotation: Annotation, dx: number, dy: number): Annotation
```

Copie de l'élément déplacée d'un vecteur : les points d'une ligne, le coin d'un cadre. Utilisée par le glisser (`moveItems`), le collage et la duplication.

Paramètres

`annotation` l'élément (non modifié) ;

`dx` déplacement horizontal en mm ;

`dy` déplacement vertical en mm.

Résultat. Un nouvel élément, coordonnées arrondies au centième.

Erreurs. Aucune.

`MIN_ANNOTATION_SIZE` constante

```
const MIN_ANNOTATION_SIZE = 1
```

Plus petite dimension d'un côté de cadre, 1 mm : `setAnnotationBounds` ne descend jamais en dessous, pour qu'un élément reste visible et saisissable.

`setAnnotationBounds` fonction

```
function setAnnotationBounds(annotation: Annotation, bounds: Box): Annotation
```

Ajuste un élément à de nouvelles bornes visibles (poignées de redimensionnement, champs de position et de taille des propriétés). Un élément à cadre garde sa rotation : son cadre local est recalculé autour du nouveau centre, largeur et hauteur échangées s'il est de côté. Une ligne voit ses points mis à l'échelle dans les nouvelles bornes ; le long d'un axe où la ligne n'avait aucune étendue (ligne horizontale ou verticale), les points sont répartis du premier au dernier, si bien que la ligne s'incline au lieu d'ignorer la valeur saisie (correction de la relecture du lot 48).

Paramètres

`annotation` l'élément ;

`bounds` les bornes visibles voulues, en mm de page.

Résultat. Un nouvel élément ; un cadre a au moins `MIN_ANNOTATION_SIZE` de côté.

Erreurs. Aucune.

```
const resized = setAnnotationBounds(rect, { x: 10, y: 10, width: 60, height: 30 });
```

`rotateAnnotation` fonction

```
function rotateAnnotation(annotation: Annotation): Annotation
```

Quart de tour dans le sens horaire autour du centre (commande `annotation.rotate`). Un élément à cadre ajoute 90° à sa `rotation` (modulo 360) ; une ligne fait tourner ses points autour du centre de leur englobant et bascule `elbowStart`, sans quoi une flèche coudée serait retournée en miroir (correction de la relecture du lot 48).

Paramètres

`annotation` l'élément.

Résultat. Un nouvel élément.

Erreurs. Aucune.

`linePath` fonction

```
function linePath(annotation: LineAnnotation): Point[]
```

Points du tracé dessiné d'une ligne. Sans coude, ce sont ses points. Avec coude, un chemin d'au plus trois points entre le premier et le dernier : horizontal puis vertical (`elbowStart` `horizontal`) ou l'inverse ; deux points seulement si les extrémités sont alignées. Les points intermédiaires d'une ligne coudée sont ignorés.

Paramètres

`annotation` la ligne ou la flèche.

Résultat. La liste des sommets de la polyligne, en mm.

Erreurs. Aucune.

`arrowHeadLength` fonction

```
function arrowHeadLength(strokeWidth: number): number
```

Longueur d'une pointe de flèche selon l'épaisseur du trait : six fois l'épaisseur, au moins 2,5 mm.

Paramètres

`strokeWidth` épaisseur du trait en mm.

Résultat. La longueur en mm.

Erreurs. Aucune.

`arrowHead` fonction

```
function arrowHead(from: Point, tip: Point, length: number): Point[] | null
```

Triangle d'une pointe de flèche dont la pointe est en `tip`, orientée selon le segment venant de `from`. La base est à `length` de la pointe, sa demi-largeur vaut 0,4 × `length`.

Paramètres

`from` point précédent du tracé ;

`tip` extrémité où se dessine la pointe ;

`length` longueur de la pointe en mm.

Résultat. Trois points (pointe, puis les deux coins de la base), ou `null` si les deux points sont confondus (direction indéfinie).

Erreurs. Aucune.

`arrowHeads` fonction

```
function arrowHeads(annotation: LineAnnotation): Point[][]
```

Pointes d'une ligne telles que dessinées : aucune pour une ligne simple, une au dernier point pour une flèche, une de plus au premier point avec `doubleHead`. Les directions sont prises sur le tracé de `linePath`, donc sur le dernier segment du coude.

Paramètres

`annotation` la ligne ou la flèche.

Résultat. Une liste de triangles (zéro, un ou deux), sans les pointes dégénérées.

Erreurs. Aucune.

`dashArray` fonction

```
function dashArray(dash: 'solid' | 'dashed' | 'dotted', width: number): string | undefined
```

Motif SVG `stroke-dasharray` d'un trait proportionnel à son épaisseur : tirets de 6 × l'épaisseur séparés de 4 ×, pointillés de 1 × séparés de 2 ×.

Paramètres

`dash` `solid`, `dashed` ou `dotted` ;

`width` épaisseur du trait en mm.

Résultat. La chaîne du motif, ou `undefined` pour un trait plein.

Erreurs. Aucune.

`annotationsInRectangle` fonction

```
function annotationsInRectangle(page: Page, rect: Box): string[]
```

Sélection au rectangle (ERG-02) : identifiants des éléments de la page dont l'encombrement (`annotationBounds`) est entièrement dans le rectangle. Appelée par la zone de dessin, avec `selectInRectangle` pour les vues et équipements.

Paramètres

`page` la page ;

`rect` le rectangle de sélection en mm de page.

Résultat. Les identifiants, dans l'ordre de la page.

Erreurs. Aucune.

`nextBubbleNumber` fonction

```
function nextBubbleNumber(page: Page): string
```

Numéro de la prochaine bulle numérotée : un de plus que le plus grand nombre lu en tête du texte des bulles de la page (`parseInt`), « 1 » s'il n'y en a pas. Utilisée à la création d'une bulle et au collage (une bulle collée prend le numéro suivant).

Paramètres

`page` la page.

Résultat. Le numéro, en texte.

Erreurs. Aucune ; les bulles dont le texte ne commence pas par un nombre sont ignorées.

`ANNOTATION_GROUPS` constante

```
const ANNOTATION_GROUPS = …
```

Éléments proposés par l'onglet Schéma, en quatre groupes affichés par `SchemaPanel` (PAG-07) :

Champs

- `texts` texte, rectangle, rectangle arrondi, ellipse ;
- `lines` ligne, flèche, flèche double, flèche coudée ;
- `images` image ;
- `markers` cadre de zone, bulle, note, tableau.

L'ordre des tableaux est celui de l'affichage. Chaque élément correspond à une commande `annotation.add.<élément>`.

`AnnotationPreset` type

```
type AnnotationPreset = (typeof ANNOTATION_GROUPS)[keyof typeof ANNOTATION_GROUPS][number];
```

Union des éléments de `ANNOTATION_GROUPS` (`text`, `rect`, …, `elbowArrow`, …, `table`). Un préréglage n'est pas une sorte d'élément : `doubleArrow` et `elbowArrow` donnent une `arrow`.

`ANNOTATION_PRESETS` constante

```
const ANNOTATION_PRESETS: readonly AnnotationPreset[]
```

Liste à plat de tous les préréglages, dans l'ordre des groupes. Sert à valider le type de glisser `application/x-portolan-annotation` et à déclarer les commandes.

`isAnnotationPreset` fonction

```
function isAnnotationPreset(value: string): value is AnnotationPreset
```

Garde de type d'une chaîne reçue de l'extérieur (données d'un glisser-déposer, identifiant de commande).

Paramètres

- `value` la chaîne.

Résultat. `true` si c'est un préréglage connu.

Erreurs. Aucune.

`PresetTexts` interface

```
interface PresetTexts { text; zone; note; table }
```

Textes par défaut des nouveaux éléments, traduits par l'appelant (le modèle n'utilise pas directement les libellés de l'interface ici).

Champs

- `text` texte d'un nouvel élément texte ;
- `zone` titre d'un nouveau cadre de zone ;
- `note` texte d'une nouvelle note ;
- `table` cellules d'un nouveau tableau : ligne d'en-tête et première ligne.

`PRESET_SIZES` constante

```
const PRESET_SIZES: Record<AnnotationPreset, { width: number; height: number }>
```

Taille par défaut d'un nouvel élément, en mm, par préréglage (texte 50 × 8, formes 40 × 25, lignes 40 de long, flèche coudée 40 × 20, image 60 × 40, zone 120 × 80, bulle 8 × 8, note 60 × 20, tableau 80 × 18). Pour une ligne, `width` et `height` donnent le vecteur du premier au second point. Utilisée par `createAnnotation`.

`createAnnotation` fonction

```
function createAnnotation(preset: AnnotationPreset, center: Point, page: Page, texts:
PresetTexts, image: { imageId: string; aspect: number } | null = null): Annotation
```

Nouvel élément d'un préréglage, centré sur un point (point de dépôt d'un glisser, ou centre de la zone de dessin pour un clic). Fixe toutes les valeurs par défaut : couleur noire, au-dessus des vues, sans rotation, texte de 3,5 mm aligné à gauche, trait de 0,35 mm plein. Les zones sont dessinées sous les vues en tirets bleus, les bulles sont blanches sur fond bleu, numérotées par `nextBubbleNumber`, les notes ont un fond jaune pâle, les tableaux une ligne d'en-tête. Une image déjà choisie impose ses proportions (hauteur = largeur / `aspect`) ; sans image, le cadre attend un fichier (`imageId` `null`).

Paramètres

`preset` le préréglage ;

`center` centre voulu, en mm de page ;

`page` la page, pour le numéro de bulle ;

`texts` les textes par défaut traduits ;

`image` image importée et ses proportions (largeur / hauteur), ou `null`.

Résultat. L'élément, avec un nouvel identifiant ; il n'est pas ajouté à la page.

Erreurs. Aucune.

```
const note = createAnnotation('note', { x: 148, y: 105 }, page, {
  text: 'Texte', zone: 'Zone', note: 'Note', table: [['A', 'B'], ['', '']],
});
```

`copyAnnotations` fonction

```
function copyAnnotations(annotations: readonly Annotation[], dx: number, dy: number):
Annotation[]
```

Copies profondes d'éléments avec de nouveaux identifiants, décalées d'un vecteur (coller, dupliquer ; le magasin décale chaque collage de 5 mm de plus).

Paramètres

`annotations` les éléments à copier ;

`dx` décalage horizontal en mm ;

`dy` décalage vertical en mm.

Résultat. Les copies, dans le même ordre.

Erreurs. Aucune.

`resizeTable` fonction

```
function resizeTable(cells: readonly string[][], rows: number, columns: number): string[][]
```

Cellules d'un tableau redimensionné à `rows` lignes et `columns` colonnes (au moins une de chaque), en gardant le texte des cellules existantes et en complétant par des chaînes vides. Utilisée par les propriétés d'un tableau.

Paramètres

`cells` les cellules actuelles, ligne par ligne ;

`rows` nombre de lignes voulu ;

`columns` nombre de colonnes voulu.

Résultat. Un tableau rectangulaire neuf.

Erreurs. Aucune.

`tableColumns` fonction

```
function tableColumns(cells: readonly string[][]): number
```

Nombre de colonnes d'un tableau : la longueur de sa plus longue ligne, au moins une. Le dessin donne à toutes les colonnes la même largeur.

Paramètres

`cells` les cellules.

Résultat. Le nombre de colonnes.

Erreurs. Aucune.

20.2 `model/libraryEdit`

Mode édition de bibliothèque (BIB-13, décision du 2026-11-05, lot 44) : un `.rkdglib` ouvert directement devient un *document* du même type qu'un projet (`Project`), marqué par le champ facultatif `editedLibrary`, qui ne contient que des machines et des types (ni armoires, ni liaisons, ni pages). Ainsi l'annulation, l'état « modifié » et le brouillon de secours fonctionnent sans code nouveau. Le module dépend de `model/libraryTransfer` (règle d'export), des types par défaut (`src/library/defaultTypes.ts`) et de `createProject`. Il est utilisé par la session (`src/app/session.ts` : ouverture, enregistrement) et par la commande `file.newLibrary`.

`isLibraryDocument` fonction

```
function isLibraryDocument(project: Project): boolean
```

Indique si le document ouvert est une bibliothèque en cours de modification. Le registre des commandes s'en sert pour griser les commandes sans objet dans ce mode, et l'interface pour afficher la fiche de bibliothèque à la place de la zone de dessin.

Paramètres

`project` le document.

Résultat. `true` si `project.editedLibrary` est défini.

Erreurs. Aucune.

`LibraryDocumentOptions` interface

```
interface LibraryDocumentOptions { createdAt?; now? }
```

Options de `libraryDocument`.

Champs

`createdAt` facultatif : date de création du fichier de bibliothèque (manifeste), reprise dans `meta.createdAt` et donc conservée à chaque enregistrement ;

`now` facultatif : date courante (tests), transmise à `createProject`.

`libraryDocument` fonction

```
function libraryDocument(library: Library, info: LibraryInfo, options:
LibraryDocumentOptions = {}): Project
```

Construit le document qui permet de modifier une bibliothèque lue d'un fichier. Les types de connecteurs et de liaisons de la bibliothèque sont complétés par les types par défaut de Portolan (un type de même identifiant de la bibliothèque remplace celui par défaut), pour que tout équipement intégré puisse se générer. Les machines sont copiées et marquées non génériques (une bibliothèque n'a pas de machine générique, EQT-02). Le document porte la version sous laquelle il sera enregistré : `info.version + 1`.

Paramètres

`library` contenu de la bibliothèque (machines et types) ;

`info` description du fichier (identifiant, nom, marque, version, …) ;

`options` voir `LibraryDocumentOptions`.

Résultat. Un `Project` neuf nommé comme la bibliothèque, sans armoire ni modèle d'armoire, avec `editedLibrary`.

Erreurs. Aucune ; les entrées sont copiées (`structuredClone`), jamais modifiées.

```
const doc = libraryDocument(result.library, result.info, {
  createdAt: result.manifest.createdAt,
});
// doc.editedLibrary.version === result.info.version + 1
```

`newLibraryDocument` fonction

```
function newLibraryDocument(now = new Date()): Project
```

Document d'une nouvelle bibliothèque vide (commande `file.newLibrary`) : nouvel identifiant, nom par défaut traduit (« Nouvelle bibliothèque »), métadonnées vides, version 0, donc enregistrée en version 1.

Paramètres

`now` date de création (par défaut maintenant).

Résultat. Le document, prêt à recevoir des machines.

Erreurs. Aucune.

`libraryToSave` fonction

```
function libraryToSave(project: Project): { library: Library; info: LibraryInfo; images: Set<ImageRef>; }
```

Contenu à écrire dans le `.rkdglib` lors de l'enregistrement d'un document de bibliothèque. Reprend la règle de l'export (`libraryForExport`) : les machines non génériques, avec pour origine cette bibliothèque dans la version du document, les types de connecteurs qu'elles utilisent, les types de liaisons qui les acceptent et les types complémentaires. Garde en plus les types de connecteurs et de liaisons *créés dans la bibliothèque* (identifiant absent des types par défaut), même si aucune machine ne les utilise encore ; un type de liaison par défaut n'est gardé que s'il accepte un connecteur utilisé.

Paramètres

`project` le document de bibliothèque.

Résultat. `library` (sans modèles d'armoires), `info` (copie de `editedLibrary`) et `images` (références des images des machines).

Erreurs. Lève `Error('not a library document')` si le document n'a pas `editedLibrary`.

`libraryFileName` fonction

```
function libraryFileName(info: Pick<LibraryInfo, 'name' | 'version'>): string
```

Nom de fichier proposé pour une bibliothèque, identique à l'export (BIB-12), de la forme `<nom>_v<version>.rkdglib`. Les caractères interdits dans un nom de fichier (`\ / : * ? " < > |`) sont remplacés par un tiret, les blancs réduits à une espace ; un nom vide prend le nom par défaut traduit.

Paramètres

`info` le nom et la version de la bibliothèque.

Résultat. Le nom du fichier, par exemple `Commutateurs_v4.rkdglib`.

Erreurs. Aucune.

20.3 `model/libraryTransfer`

Import de machines depuis une bibliothèque (`.rkdglib`) ou un autre projet (`.rkdg`), mise à jour vers une version plus récente, et export d'une sélection de machines en bibliothèque (BIB-08, BIB-10 à BIB-12, BIB-14, REC-10). Le module est pur : il *planifie* ce qu'il faut copier, le magasin (`applyImportPlan`, `src/store/libraryActions.ts`) l'applique en une seule action annulable. Il dépend de `changeMachine` et `machineConnectorTypeIds` (`src/library/equipment/change.ts`), du remappage des paramètres (`src/library/equipment/params.ts`) et de `machineImageRefs`. Il est utilisé par les fenêtres d'import et d'export, les commandes de fichiers de bibliothèque et `model/libraryEdit`. Décisions du 2026-11-05, lots 30 et 44.

`LibraryDescription` interface

```
interface LibraryDescription { id; name; brand; version; author; description; license }
```

Description d'un fichier de bibliothèque (manifeste d'un `.rkdglib`), de même forme que `LibraryInfo` du schéma.

Champs

`id` identifiant stable de la bibliothèque, conservé d'une version à l'autre ;

`name` nom affiché ;

`brand` marque (texte libre ; jamais dans la bibliothèque par défaut) ;

`version` numéro de version entier, croissant ;

`author` auteur ;

`description` description libre ;

`license` licence ou provenance des images.

`ImportSource` interface

```
interface ImportSource { library; images; info }
```

D'où viennent les machines à importer.

Champs

`library` contenu lu du fichier (machines, types) ;

`images` images du fichier, par référence ;

`info` description du `.rkdglib`, ou `null` pour un import depuis un projet (BIB-08) : l'origine des machines est alors gardée telle quelle.

`ConflictChoice` type

```
type ConflictChoice = 'keep' | 'replace' | 'both';
```

Choix de l'utilisateur pour une machine déjà présente (BIB-11) : `keep` conserve la machine du projet et n'importe rien (défaut) ; `replace` remplace la machine du projet en gardant son identifiant, les équipements posés suivent (c'est aussi la « mise à jour » de BIB-14) ; `both` garde les deux, la copie importée étant renommée « … (importé) » avec un nouvel identifiant.

`ImportCandidate` interface

```
interface ImportCandidate { machine; origin; existing; newer }
```

Une machine de la source, avec son éventuel conflit avec le projet.

Champs

`machine` la machine telle que dans la source ;

`origin` origine qu'elle aura dans le projet (`sourceOrigin`) ;

`existing` machine du projet qui est « la même » (même origine, ou même identifiant pour deux machines sans origine), sinon `null` ;

`newer` la source en a une version plus récente (version de bibliothèque supérieure, BIB-11).

`sourceOrigin` fonction

```
function sourceOrigin(machine: Machine, info: LibraryDescription | null): MachineOrigin
| null
```

Origine qu'une machine de la source aura une fois importée. Depuis un `.rkdglib`, c'est cette bibliothèque (identifiant, nom, version) et l'identifiant de la machine dans la bibliothèque : celui de son origine si elle vient déjà de cette bibliothèque, sinon son propre identifiant. Depuis un projet, l'origine existante est gardée.

Paramètres

`machine` la machine de la source ;

`info` description de la bibliothèque, ou `null` pour un projet.

Résultat. Une `MachineOrigin`, ou `null` (projet, machine sans origine).

Erreurs. Aucune.

`importCandidates` fonction

```
function importCandidates(project: Project, source: ImportSource): ImportCandidate[]
```

Liste les machines de la source avec leurs conflits (BIB-10, BIB-11) ; c'est le contenu de la fenêtre d'import. L'identité d'une machine est son origine (bibliothèque, identifiant d'origine) ; sans origine, son identifiant.

Paramètres

`project` le projet ouvert ;

`source` la source.

Résultat. Un `ImportCandidate` par machine de la source, dans son ordre.

Erreurs. Aucune.

`connectorIds` fonction

```
function connectorIds(machines: readonly Machine[]): Set<Id>
```

Types de connecteurs dont des machines ont besoin (BIB-09) : ceux de leurs ports et ceux du projet choisis dans les paramètres de leur équipement.

Paramètres

`machines` les machines.

Résultat. L'ensemble des identifiants de types de connecteurs.

Erreurs. Aucune.

`withComplements` fonction

```
function withComplements(ids: Iterable<Id>, connectorTypes: readonly ConnectorType[]):
Set<Id>
```

Complète un ensemble de types de connecteurs par leurs types complémentaires, de proche en proche : un type copié nomme son complément (`complementId`), qui doit être copié aussi pour que le fichier reste valide.

Paramètres

`ids` identifiants de départ ;

`connectorTypes` types où chercher les compléments.

Résultat. Un nouvel ensemble contenant `ids` et leurs compléments.

Erreurs. Aucune ; un complément absent de `connectorTypes` est ajouté sans être suivi plus loin.

`ImportPlan` interface

```
interface ImportPlan { added; replaced; connectorTypes; linkTypes; images; addedBytes }
```

Ce qu'un import copiera dans le projet.

Champs

`added` machines ajoutées (nouvel identifiant si celui-ci est déjà pris) ;

`replaced` machines qui remplacent une machine du projet (même identifiant que la machine remplacée) ;

`connectorTypes` types de connecteurs à copier (absents du projet) ;

`linkTypes` types de liaisons à copier ;

`images` images absentes du projet, par référence ;

`addedBytes` poids ajouté approximatif, en octets (JSON des objets copiés + images), affiché avant validation (BIB-10).

`planImport` fonction

```
function planImport(project: Project, source: ImportSource, machineIds:
ReadonlySet<Id>, choices: ReadonlyMap<Id, ConflictChoice> = new Map(), defaultChoice:
ConflictChoice = 'keep'): ImportPlan
```

Calcule ce que copie l'import des machines cochées (BIB-10). Les candidats en conflit dont le choix est `keep` sont écartés. Les types de connecteurs nécessaires (et leurs compléments) sont réutilisés quand le projet en a un de même identifiant, ou de même nom (sans casse) et même famille, sinon copiés ; les ports et les paramètres de connecteur des machines copiées sont rattachés au type retenu, et le complément d'un type copié aussi. Les types de liaisons de la source qui acceptent un connecteur importé et que le projet n'a pas (même identifiant ou même nom) sont copiés. Une machine remplacée reprend l'identifiant et le caractère générique de la machine du projet ; une machine ajoutée devient non générique et reçoit un nouvel identifiant si le sien est pris. Seules les images absentes du projet (machines et logo) sont copiées.

Paramètres

`project` le projet ;

`source` la source ;

`machineIds` identifiants (dans la source) des machines cochées ;

`choices` choix par machine en conflit (par identifiant dans la source) ;

`defaultChoice` choix des conflits sans choix propre (« pour tous »), `keep` par défaut.

Résultat. Un `ImportPlan`, non encore vérifié : passer par `checkImport` avant de l'appliquer.

Erreurs. Aucune ; la source n'est pas modifiée.

```
const candidates = importCandidates(project, source);
const { checked, choices } = defaultImportSelection(candidates);
const plan = planImport(project, source, checked, choices);
const check = checkImport(project, plan);
```

`defaultImportSelection` fonction

```
function defaultImportSelection(candidates: readonly ImportCandidate[]): { checked:
Set<Id>; choices: Map<Id, ConflictChoice>; }
```

Sélection initiale de la fenêtre d'import (BIB-14) : sont cochées les machines absentes du projet et celles dont la source a une version plus récente ; ces dernières reçoivent le choix `replace`, affiché « Mettre à jour ». Une machine déjà présente dans la même version n'est pas cochée.

Paramètres

`candidates` les candidats de `importCandidates`.

Résultat. `checked` (identifiants cochés) et `choices` (choix par machine).

Erreurs. Aucune.

`ImportCheck` interface

```
interface ImportCheck { plan; project; refused; orphanLinks; warnings }
```

Conséquences d'un plan d'import pour le projet.

Champs

`plan` le plan sans les remplacements refusés, sans les types dont seuls ceux-ci avaient besoin, poids recalculé ;

`project` le projet une fois le plan appliqué ;

`refused` remplacements refusés parce qu'un équipement posé ne tiendrait plus (EQT-06) : la machine, son nom dans le projet et les problèmes en français ;

`orphanLinks` liaisons dont un port disparaît avec une mise à jour (conservées, signalées par les vérifications, LIA-07) ;

`warnings` un message par port perdu, par exemple « SW01 · 25 (liaison « ETH-004 ») ».

`checkImport` fonction

```
function checkImport(project: Project, plan: ImportPlan): ImportCheck
```

Applique un plan au projet et en vérifie les conséquences (BIB-11, BIB-14). Les machines et types ajoutés sont joints ; chaque machine remplacée passe par `changeMachine`, comme une machine modifiée dans l'éditeur : ses équipements posés la suivent, et la mise à jour est refusée si l'un d'eux ne tiendrait plus (chevauchement, sortie de l'armoire, profondeur). Les types de connecteurs et de liaisons dont seule une mise à jour refusée avait besoin sont retirés, comme les images inutiles. La fenêtre l'appelle pour afficher « À vérifier avant l'import », et `applyImportPlan` l'appelle de nouveau avec le plan complet.

Paramètres

`project` le projet avant import ;

`plan` le plan de `planImport`.

Résultat. Un `ImportCheck`.

Erreurs. Aucune exception : les refus sont rendus dans `refused`.

`proposedDescription` fonction

```
function proposedDescription(machines: readonly Machine[], excludedLibraryId?: Id):
LibraryDescription
```

Description proposée par la fenêtre d'export (BIB-12). Si toutes les machines viennent d'une même bibliothèque, autre que `excludedLibraryId`, son identifiant et son nom sont repris avec la version suivant la plus haute version d'origine ; sinon, nouvel identifiant, nom vide, version 1. La marque proposée est le fabricant de la première machine.

Paramètres

`machines` les machines cochées ;

`excludedLibraryId` identifiant à ne jamais reprendre (la bibliothèque par défaut, qui n'est pas publiée).

Résultat. Une `LibraryDescription` à compléter.

Erreurs. Aucune.

`libraryForExport` fonction

```
function libraryForExport(project: Project, machineIds: ReadonlySet<Id>, info:
LibraryDescription): { library: Library; images: Set<ImageRef> }
```

Bibliothèque des machines choisies (BIB-12) : copies non génériques des machines, avec pour origine la bibliothèque décrite (identifiant d'origine conservé si elles en viennent déjà) ; les types de liaisons qui acceptent un connecteur utilisé ; les types de connecteurs des ports, ceux qu'acceptent ces types de liaisons et leurs compléments. Aucun modèle d'armoire n'est exporté.

Paramètres

`project` le projet ;

`machineIds` identifiants des machines à exporter ;

`info` description de la bibliothèque produite.

Résultat. `library` et `images` (références des images à joindre).

Erreurs. Aucune.

20.4 `model/libraryTree`

Arborescence du panneau Bibliothèque (BIB-01) : catégorie, puis équipement intégré, puis machines (la machine générique en tête), filtrée par une recherche plein texte. Le module dépend de `equipmentTree` (`model/equipment`) ; il est utilisé par `LibraryPanel`, la fenêtre d'import (même arborescence) et les propriétés d'un équipement (liste des machines du même équipement, triée par `byGenericThenName`).

`searchWords` fonction

```
function searchWords(query: string): string[]
```

Découpe une recherche en mots en minuscules (blancs comme séparateurs). Les accents ne sont pas retirés, contrairement à la recherche globale (`normalizeSearch`).

Paramètres

`query` le texte saisi.

Résultat. Les mots, vide si rien n'est cherché.

Erreurs. Aucune.

`matchesSearch` fonction

```
function matchesSearch(model: Machine, query: string, extra: readonly string[] = []):
boolean
```

Recherche plein texte sur une machine (BIB-01) : chaque mot doit apparaître, sans tenir compte de la casse, dans le nom, le fabricant, la référence, les mots-clés ou les textes supplémentaires (nom traduit de l'équipement et de la catégorie).

Paramètres

`model` la machine ;

`query` la recherche ;

`extra` textes supplémentaires où chercher.

Résultat. `true` si tous les mots sont trouvés, ou si la recherche est vide.

Erreurs. Aucune.

`byGenericThenName` fonction

```
function byGenericThenName(a: Machine, b: Machine): number
```

Fonction de comparaison pour `Array.sort` : la machine générique d'abord, puis les autres par nom (ordre alphabétique français).

Paramètres

`a` première machine ;

`b` seconde machine.

Résultat. Un nombre négatif, nul ou positif.

Erreurs. Aucune.

`LibraryTreeEquipment` interface

```
interface LibraryTreeEquipment { kind; generic; machines }
```

Un équipement intégré dans l'arborescence.

Champs

`kind` l'équipement ;

`generic` sa machine générique, posée quand on glisse l'équipement lui-même ; `undefined` si le projet n'en a pas ;

`machines` machines affichées sous l'équipement (filtrées par la recherche), la générique en tête.

`LibraryTreeCategory` interface

```
interface LibraryTreeCategory { category; equipments }
```

Une catégorie de l'arborescence.

Champs

`category` la catégorie (réseau, serveurs et stockage, …) ;

`equipments` ses équipements, dans l'ordre d'`equipmentTree`.

`LibraryTree` interface

```
interface LibraryTree { categories; searching }
```

Arborescence complète du panneau.

Champs

`categories` les catégories affichées ;

`searching` une recherche est active : le panneau déroule alors les équipements affichés.

`libraryTree` fonction

```
function libraryTree(machines: readonly Machine[], query: string, labels: { kind: (kind:
EquipmentKind) => string; category: (category: EquipmentCategory) => string; }):
LibraryTree
```

Construit l'arborescence (BIB-01). Sans recherche, tous les équipements intégrés par catégorie, avec toutes leurs machines (un équipement sans machine reste affiché). Avec une recherche, seulement les machines qui correspondent (y compris par le nom de leur équipement ou de leur catégorie) et les équipements qui en ont ; une catégorie vide disparaît.

Paramètres

`machines` les machines du projet ;

`query` la recherche ;

`labels` fonctions qui donnent le nom traduit d'un équipement (`kind`) et d'une catégorie (`category`).

Résultat. Un `LibraryTree`.

Erreurs. Aucune.

```
const tree = libraryTree(project.library.machines, 'serveur 2u', {
  kind: (k) => kindLabel(k),        // nom traduit de l'équipement
  category: (c) => categoryLabel(c), // nom traduit de la catégorie
});
```

20.5 `model/modelEditor`

Règles de l'éditeur de machine (EDM-03 à EDM-07) : gabarit de face, contrôle des proportions d'une image avec dimensions recommandées, alerte de poids, ports disposés en grille ou repris d'une autre machine, ports « placés » à la main. Les positions et tailles des ports sont des proportions de la face (0 à 1, centre du port). Le module dépend de `model/rackGeometry` (`PANEL_19_MM`, `U_MM`) et de `isExternal`. Il est utilisé par `MachineEditorDialog`, `FaceEditor`, `PortGridForm`, `PortsTable`, `RackPreview`, `src/ui/annotationFlow.ts` (image d'un

élément de page) et `src/model/instancePorts.ts`. Décisions du 2026-11-04 (lot 18) et du 2026-11-05 (lot 44).

`HALF_PANEL_MM` constante

```
const HALF_PANEL_MM = PANEL_19_MM / 2
```

Largeur d'une face de machine demi-largeur : la moitié de la façade 19 pouces (`PANEL_19_MM / 2`), en mm.

`RATIO_TOLERANCE` constante

```
const RATIO_TOLERANCE = 0.02
```

Écart relatif toléré entre les proportions d'une image et celles du gabarit avant avertissement : 0,02, soit 2 % (EDM-03).

`RECOMMENDED_WIDTH_PX` constante

```
const RECOMMENDED_WIDTH_PX = 2000
```

Largeur recommandée d'une image de face, 2 000 pixels ; la hauteur recommandée en découle par le ratio du gabarit (« 1U : 2 000 × 184 px », EDM-03).

`DEFAULT_IMAGE_LIMIT_MB` constante

```
const DEFAULT_IMAGE_LIMIT_MB = 1
```

Seuil d'alerte de poids d'une image par défaut, 1 Mo (EDM-04). La valeur effective est une préférence du navigateur, pas une donnée du projet.

`faceTemplateSize` fonction

```
function faceTemplateSize(machine: WithKind & Pick<Machine, 'heightU' | 'width'
| 'displaySizeMm'>): { width: number; height: number }
```

Taille du gabarit d'une face, en mm : 19 pouces (ou la moitié pour une machine demi-largeur) sur la hauteur en U (au moins 1 U, pour qu'une machine 0U ait un gabarit), ou la taille d'affichage d'un équipement extérieur (300 × 200 mm si elle n'est pas définie). C'est le cadre dans lequel une image est ajustée sans déformation.

Paramètres

`machine` l'équipement, la hauteur en U, la largeur et la taille d'affichage.

Résultat. `{ width, height }` en mm.

Erreurs. Aucune.

`RatioCheck` interface

```
interface RatioCheck { ok; imageRatio; templateRatio; deviation; recommended }
```

Résultat du contrôle des proportions d'une image (EDM-03).

Champs

`ok` l'écart est dans la tolérance ;

`imageRatio` largeur / hauteur de l'image ;

`templateRatio` largeur / hauteur du gabarit ;

`deviation` écart relatif (0,05 = 5 %) ;

`recommended` taille recommandée en pixels pour ce gabarit.

`checkImageRatio` fonction

```
function checkImageRatio(template: { width: number; height: number }, pixels: { width: number; height: number }): RatioCheck
```

Compare les proportions d'une image à celles du gabarit de la face. L'image est de toute façon ajustée sans déformation ; le résultat sert seulement à l'avertissement.

Paramètres

`template` taille du gabarit (`faceTemplateSize`) ;

`pixels` taille de l'image en pixels.

Résultat. Un `RatioCheck`.

Erreurs. Aucune ; une hauteur nulle donne un ratio infini, donc `ok` faux.

```
const check = checkImageRatio(faceTemplateSize(machine), { width: 1800, height: 200 });
if (!check.ok) console.warn(check.deviation, check.recommended);
```

`isImageTooHeavy` fonction

```
function isImageTooHeavy(bytes: number, limitMb = DEFAULT_IMAGE_LIMIT_MB): boolean
```

Indique si une image dépasse le seuil d'alerte de poids (EDM-04) ; l'image est stockée telle quelle dans tous les cas.

Paramètres

`bytes` poids de l'image en octets ;

`limitMb` seuil en mégaoctets (1 024 × 1 024 octets), 1 par défaut.

Résultat. `true` au-delà du seuil.

Erreurs. Aucune.

`GridNumbering` type

```
type GridNumbering = 'rows' | 'columns' | 'oddTop';
```

Sens de numérotation d'une grille de ports : `rows` (ligne par ligne), `columns` (colonne par colonne) et `oddTop` (impairs en haut, pairs en dessous : 1 au-dessus de 2, 3 au-dessus de 4). Dans `gridCell`, `columns` et `oddTop` donnent la même disposition.

`PortGridParams` interface

```
interface PortGridParams { rows; columns; x; y; dx; dy; w; h; numbering }
```

Réglages de l'ajout en série (EDM-06). Toutes les longueurs sont des proportions de la face.

Champs

`rows` nombre de lignes ;

`columns` nombre de colonnes ;

`x` abscisse du centre du premier port (haut gauche) ;

`y` ordonnée de ce centre ;

`dx` distance horizontale entre centres de ports ;

`dy` distance verticale entre centres de ports ;

`w` largeur d'un port ;

`h` hauteur d'un port ;

`numbering` sens de numérotation (`GridNumbering`).

`gridCell` fonction

```
function gridCell(index: number, rows: number, columns: number, numbering: GridNumbering): { row: number; col: number }
```

Cellule du n^e^ port (à partir de 0) d'une grille.

Paramètres

`index` rang du port ;

`rows` nombre de lignes ;

`columns` nombre de colonnes ;

`numbering` sens de numérotation.

Résultat. `{ row, col }` à partir de 0.

Erreurs. Aucune.

`arrangePorts` fonction

```
function arrangePorts(ports: readonly Port[], ids: readonly Id[], grid: PortGridParams): Port[]
```

Place des ports sur une grille en une opération (EDM-06), par exemple 24 ports en deux rangées de 12, impairs en haut. Les ports viennent des paramètres de la machine : la grille ne fixe que leur position et leur taille, dans l'ordre de `ids`, au plus lignes × colonnes ports. Les ports disposés deviennent « placés » (leur position est gardée quand les paramètres changent) et restent dans la face (`clampPort`).

Paramètres

`ports` les ports de la face ;

`ids` identifiants des ports à disposer, dans l'ordre de numérotation ;

`grid` les réglages (lignes et colonnes arrondies, au moins 1).

Résultat. Tous les ports de la face ; ceux qui ne sont pas disposés sont inchangés.

Erreurs. Aucune.

`copyPortPositions` fonction

```
function copyPortPositions(ports: readonly Port[], source: readonly Port[]): { ports: Port[]; count: number }
```

Reprend les positions des ports d'une autre face ou d'une autre machine (EDM-07) : chaque port prend la position et la taille du port de même nom de la source, et devient placé.

Paramètres

`ports` les ports à modifier ;

`source` les ports de référence.

Résultat. `ports` (la nouvelle liste) et `count` (nombre de ports repris).

Erreurs. Aucune.

`positionSources` fonction

```
function positionSources(machines: readonly Machine[], target: Pick<Machine, 'id'
| 'equipment'> & Pick<Machine, 'front' | 'rear'>, face: 'front' | 'rear', otherFile =
false): Machine[]
```

Machines proposées par « Reprendre les positions de » pour une face (EDM-07) : celles qui ont au moins un port de même nom sur cette face, celles du même équipement d'abord, puis les autres dans leur ordre (deux familles de façades voisines peuvent servir l'une à l'autre).

Paramètres

`machines` les machines candidates ;

`target` la machine modifiée (identifiant, équipement, faces) ;

`face` `front` ou `rear` ;

`otherFile` les machines viennent d'un autre fichier : une machine de même identifiant (version précédente) est alors gardée.

Résultat. Les machines retenues.

Erreurs. Aucune.

`clampPort` fonction

```
function clampPort<T extends Pick<Port, 'x' | 'y' | 'w' | 'h'>>(port: T): T
```

Garde un port dans sa face : largeur et hauteur ramenées entre 0,002 et 1, centre décalé pour que le port ne dépasse pas.

Paramètres

`port` un objet avec `x`, `y`, `w`, `h` (proportions).

Résultat. Une copie corrigée, du même type que l'entrée.

Erreurs. Aucune.

`markPlaced` fonction

```
function markPlaced(before: readonly Port[], after: readonly Port[]): Port[]
```

Marque comme placés les ports que l'utilisateur a déplacés ou redimensionnés (ou ajoutés) : la régénération de la machine garde alors leur position (EQT-05). Appelée par l'éditeur à chaque modification graphique des ports.

Paramètres

`before` les ports avant la modification ;

`after` les ports après.

Résultat. `after`, avec `placed: true` sur les ports changés.

Erreurs. Aucune.

`unplacePorts` fonction

```
function unplacePorts(ports: readonly Port[], ids: ReadonlySet<Id> | null): Port[]
```

Rend des ports à la disposition automatique de l'équipement : ils ne sont plus placés et leur position revient à la prochaine régénération.

Paramètres

`ports` les ports de la face ;

`ids` les ports concernés, ou `null` pour tous.

Résultat. La nouvelle liste.

Erreurs. Aucune.

20.6 `model/pageGeometry`

Géométrie des éléments d'une page en millimètres de papier : encombrement des vues, place des équipements et de leurs ports dans chaque vue, sélection au rectangle, aperçu d'un dépôt, place libre. Le module passe du millimètre d'armoire (dessin d'une face, `model/rackGeometry`) au millimètre de page, en tenant compte de l'échelle effective (`effectiveScale`) et de la plage de U de la vue (PAG-16, fenêtre `rackWindow`). Il est partagé par la couche d'interaction (sélection, glisser-déposer, liaisons), le routage (`src/routing/`), le rendu, les exports et la recherche. Décisions du 2026-11-04 (lot 12) et du 2026-11-05 (lot 47).

`VIEW_TITLE_SIZE` constante

```
const VIEW_TITLE_SIZE = 7
```

Hauteur réservée au titre d'une vue au-dessus de son dessin, 7 mm de papier.

`VIEW_TITLE_GAP` constante

```
const VIEW_TITLE_GAP = VIEW_TITLE_SIZE * 1.6
```

Distance entre le haut du dessin d'une vue et la ligne de base de son titre : 1,6 × `VIEW_TITLE_SIZE`, soit 11,2 mm de papier.

`viewFace` fonction

```
function viewFace(view: Pick<View, 'kind'>): FaceSide
```

Face d'armoire montrée par une vue : `rear` pour une vue arrière, `front` sinon (y compris, par convention, pour une vue d'équipement extérieur).

Paramètres

`view` la vue (seul `kind` compte).

Résultat. `front` ou `rear`.

Erreurs. Aucune.

`externalViewSize` fonction

```
function externalViewSize(view: View, model: Machine | undefined, scale = view.scale): { width: number; height: number }
```

Taille sur la page d'une vue d'équipement extérieur (PAG-05, EXT-02) : la taille forcée de la vue si elle existe, sinon la taille d'affichage de la machine (100 × 100 mm par défaut) multipliée par l'échelle.

Paramètres

- `view` la vue extérieure ;
- `model` la machine de l'équipement, si connue ;
- `scale` échelle à appliquer, celle de la vue par défaut.

Résultat. `{ width, height }` en mm de papier.

Erreurs. Aucune.

`rackViewFrame` fonction

```
function rackViewFrame(rack: Rack, view: Pick<View, 'kind' | 'uRange'>): { geometry: RackGeometry; window: RackWindow }
```

Géométrie de la face d'armoire montrée par une vue et fenêtre de sa plage de U (PAG-16). Point de départ de tout calcul sur une vue d'armoire.

Paramètres

- `rack` l'armoire ;
- `view` la vue (`kind` et `uRange`).

Résultat. `geometry` (`RackGeometry`) et `window` (`RackWindow` : haut et bas de la vue, bande où le contenu est dessiné, coupures).

Erreurs. Aucune.

`rackToPage` fonction

```
function rackToPage(view: Pick<View, 'x' | 'y'>, window: Pick<RackWindow, 'top'>, scale: number, point: { x: number; y: number }): { x: number; y: number }
```

Position sur la page d'un point du dessin d'armoire : décalée du haut de la fenêtre (`window.top`, non nul quand la plage coupe le haut), mise à l'échelle et translatée au coin de la vue. À utiliser partout où l'on passe du millimètre d'armoire à la page.

Paramètres

- `view` coin haut gauche de la vue sur la page ;
- `window` fenêtre de la vue (seul `top` compte) ;
- `scale` échelle effective ;
- `point` point en mm d'armoire.

Résultat. Le point en mm de page.

Erreurs. Aucune.

`viewBounds` fonction

```
function viewBounds(project: Project, page: Page, view: View): Box | undefined
```

Rectangle du dessin d'une vue sur sa page, titre et étiquettes à côté de l'armoire exclus : pour une vue d'armoire, la largeur de la face et la hauteur de la fenêtre de la plage de U à l'échelle effective ; pour une vue extérieure, `externalViewSize`. Sert à la sélection, aux guides, à l'alignement, à la mise en page automatique.

Paramètres

`project` le projet ;

`page` la page de la vue ;

`view` la vue.

Résultat. Un `Box` en mm de page, ou `undefined` si la cible de la vue (armoire ou équipement) n'existe plus.

Erreurs. Aucune.

`deviceInRackFace` fonction

```
function deviceInRackFace(device: Device, model: Machine, geometry: RackGeometry): { box: Box; modelFace: FaceSide } | undefined
```

Rectangle d'un équipement posé dans le dessin d'une face d'armoire (mm d'armoire) et face de sa machine qui y est vue. Un équipement monté sur cette face montre la face avant de sa machine, l'autre face montre sa face arrière ; une machine demi-largeur est dessinée dans sa moitié, miroir à l'arrière. Un équipement 0U n'est visible que sur sa face.

Paramètres

`device` l'équipement ;

`model` sa machine ;

`geometry` géométrie de la face d'armoire.

Résultat. `box` et `modelFace`, ou `undefined` si l'équipement n'est pas visible sur cette face (extérieur, 0U de l'autre face, emplacement 0U introuvable).

Erreurs. Aucune.

`clipToWindow` fonction

```
function clipToWindow(box: Box, window: RackWindow): Box | undefined
```

Partie d'un rectangle (mm d'armoire) comprise dans la bande dessinée d'une fenêtre : c'est la coupe nette d'un équipement à cheval sur une limite de plage (PAG-16).

Paramètres

`box` le rectangle ;

`window` la fenêtre de la vue.

Résultat. Le rectangle coupé en hauteur, ou `undefined` s'il est entièrement hors de la bande.

Erreurs. Aucune.

`DeviceOnPage` interface

```
interface DeviceOnPage { viewId; box; fullBox; modelFace }
```

Une place où un équipement est dessiné sur une page.

Champs

`viewId` la vue qui le dessine ;

`box` rectangle tel que dessiné, coupé aux limites de la plage de U (sélection, contour, étiquettes) ;

`fullBox` rectangle entier, éventuellement hors de la vue : les ports s'y placent ;

`modelFace` face de la machine dessinée dans cette vue.

`deviceOnPage` fonction

```
function deviceOnPage(project: Project, page: Page, deviceId: Id): DeviceOnPage[]
```

Toutes les places où un équipement est dessiné sur une page : sa vue extérieure, ou les vues de son armoire (avant, arrière, autant de fois que l'armoire figure sur la page) où il est au moins en partie dans la plage de U.

Paramètres

`project` le projet ;

`page` la page ;

`deviceId` l'équipement.

Résultat. Une liste de `DeviceOnPage`, vide si l'équipement n'est pas sur la page ou n'existe pas.

Erreurs. Aucune.

`PortOnPage` interface

```
interface PortOnPage { viewId; box; deviceBox; port }
```

Une place où un port est dessiné sur une page.

Champs

`viewId` la vue ;

`box` rectangle du port, en mm de page ;

`deviceBox` rectangle dessiné de son équipement (coupé par la plage) ;

`port` le port.

`centreVisible` fonction

```
function centreVisible(box: Box, drawn: Box): boolean
```

Indique si le centre d'un port tombe dans la partie dessinée de son équipement : un équipement coupé par la plage de U de la vue (PAG-16) ne montre que les ports dont le centre est dans la bande dessinée. Partagée par `portOnPage` et `portStatesOnPage`, pour que les deux retiennent les mêmes ports.

Paramètres

`box` rectangle du port, placé dans le rectangle entier de l'équipement ;

`drawn` rectangle dessiné de l'équipement (coupé par la plage).

Résultat. `true` si le centre vertical du port est dans `drawn` (à 10^{-6} mm près).

Erreurs. Aucune.

`portBox` fonction

```
function portBox(port: Pick<Port, 'x' | 'y' | 'w' | 'h'>, box: Box): Box
```

Rectangle d'un port dans le rectangle de son équipement : la position du port est celle de son centre, en proportion de la face.

Paramètres

- `port` position et taille du port (proportions) ;
- `box` rectangle de l'équipement.

Résultat. Le rectangle du port, dans le repère de `box`.

Erreurs. Aucune.

`portOnPage` fonction

```
function portOnPage(project: Project, page: Page, deviceId: Id, portId: Id): PortOnPage[]
```

Toutes les places où un port est dessiné sur une page : dans chaque vue qui montre la face de la machine portant ce port. Pour une vue extérieure aux ports masqués (EXT-03), le port est un carré de 1 mm au milieu du bas de l'équipement, où aboutit le câble. Un port dont le centre est hors de la partie dessinée (coupé par la plage, PAG-16) n'est pas sur la page. Utilisée par le routage, la création de liaisons, les surbrillances et la recherche.

Paramètres

- `project` le projet ;
- `page` la page ;
- `deviceId` l'équipement ;
- `portId` le port (port de la machine ou port ajouté à l'équipement).

Résultat. Une liste de `PortOnPage`, vide si le port est introuvable ou n'est pas dessiné.

Erreurs. Aucune.

`boxesIntersect` fonction

```
function boxesIntersect(a: Box, b: Box): boolean
```

Indique si deux rectangles se chevauchent (intérieurs ; deux rectangles qui se touchent seulement par un bord ne se chevauchent pas).

Paramètres

- `a` premier rectangle ;
- `b` second rectangle.

Résultat. `true` en cas de chevauchement.

Erreurs. Aucune.

`boxContains` fonction

```
function boxContains(outer: Box, inner: Box): boolean
```

Indique si un rectangle en contient entièrement un autre, bords compris.

Paramètres

`outer` rectangle contenant ;

`inner` rectangle contenu.

Résultat. `true` si `inner` est dans `outer`.

Erreurs. Aucune.

`boxFromPoints` fonction

```
function boxFromPoints(a: { x: number; y: number }, b: { x: number; y: number }): Box
```

Rectangle défini par deux coins opposés dans n'importe quel ordre (rectangle de sélection tiré à la souris).

Paramètres

`a` premier coin ;

`b` second coin.

Résultat. Le rectangle, de largeur et hauteur positives ou nulles.

Erreurs. Aucune.

`selectInRectangle` fonction

```
function selectInRectangle(project: Project, page: Page, rect: Box): { views: Id[];
devices: Id[] }
```

Sélection au rectangle (ERG-02) : les vues entièrement dans le rectangle, et les équipements entièrement dans le rectangle (rectangle dessiné) dont la vue d'armoire, elle, n'y est pas entièrement. Les éléments de l'onglet Schéma sont traités à part par `annotationsInRectangle`.

Paramètres

`project` le projet ;

`page` la page ;

`rect` le rectangle, en mm de page.

Résultat. `views` et `devices` : identifiants retenus.

Erreurs. Aucune.

`placementBox` fonction

```
function placementBox(project: Project, page: Page, view: View, model: Machine, placement:
Device['placement']): Box | undefined
```

Rectangle sur la page qu'occuperait une machine à un placement donné dans une vue d'armoire : aperçu d'un glisser-déposer, coupé à la plage de U.

Paramètres

`project` le projet ;

`page` la page ;

`view` la vue d'armoire visée ;

`model` la machine déposée ;

`placement` le placement envisagé.

Résultat. Un `Box` en mm de page, ou `undefined` (vue extérieure, armoire absente, placement invisible sur cette face ou hors plage).

Erreurs. Aucune.

`freeSpaceBoxes` fonction

```
function freeSpaceBoxes(project: Project, page: Page, view: View): Box[]
```

Place libre d'une vue d'armoire (ARM-05), montrée pendant un glisser : rectangles de page couvrant les U libres, ou la moitié libre d'une U à côté d'un équipement demi-largeur, vue depuis la face de la vue (moitié en miroir à l'arrière). Les U consécutives de même partie libre sont fusionnées. Sur une vue limitée à une plage de U (PAG-16), les rectangles sont coupés à la bande dessinée (`clipToWindow`) et placés depuis le haut de la fenêtre, comme les équipements : une U hors plage n'est jamais teintée.

Paramètres

`project` le projet ;

`page` la page ;

`view` la vue.

Résultat. Les rectangles, de bas en haut ; vide pour une vue extérieure ou une armoire absente.

Erreurs. Aucune.

20.7 `model/pageLayout`

Mise en page (PAG-02 à PAG-06, PAG-11, PAG-16) : formats de papier, zone de dessin, échelle effective des vues, équipements visibles et légende, étiquettes à côté des armoires. Fonctions pures en millimètres de papier (sauf les étiquettes, en mm d'armoire). Le module dépend de `model/rackGeometry` et de l'index du projet ; il est réexporté par `src/model/index.ts` et utilisé par le rendu de la page (`PageView`), le routage (`src/routing/page.ts`), la mise en page automatique (`model/autoLayout`), les exports et les propriétés de la page. Décision du 2026-11-04, lot 12.

`PAPER_SIZES` constante

```
const PAPER_SIZES: Record< Exclude<PageFormat, 'custom'>, { width: number; height: number }
>
```

Dimensions en portrait des formats de papier normalisés, en mm : A4 (210 × 297) à A0 (841 × 1 189), et Letter (215,9 × 279,4). Le format personnalisé n'y figure pas : sa taille est dans `page.customSizeMm`.

`pageSize` fonction

```
function pageSize(page: Pick<Page, 'format' | 'customSizeMm' | 'orientation'>): { width: number; height: number; }
```

Taille d'une page dans son orientation (PAG-02) : le petit côté en largeur en portrait, le grand côté en paysage. Un format personnalisé sans taille retombe sur A4.

Paramètres

`page` format, taille personnalisée et orientation.

Résultat. `{ width, height }` en mm.

Erreurs. Aucune.

`TITLE_BLOCK_HEIGHT` constante

```
const TITLE_BLOCK_HEIGHT = 26
```

Hauteur du cartouche, 26 mm, en bas de la zone utile de la page (PAG-09).

`drawingArea` fonction

```
function drawingArea(page: Page): { x: number; y: number; width: number; height: number }
```

Zone de dessin d'une page : à l'intérieur des marges, au-dessus du cartouche. Utilisée par la mise en page automatique, le routage (limites des tracés) et le placement du cartouche (juste en dessous).

Paramètres

`page` la page.

Résultat. Rectangle en mm de page.

Erreurs. Aucune ; des marges excessives donnent une taille négative.

`effectiveScale` fonction

```
function effectiveScale(page: Page, view: View): number
```

Échelle à laquelle une vue est dessinée (PAG-05) : la sienne, ou, avec l'option « même échelle » de la page (`uniformRackScale`), la plus petite des échelles des vues d'armoires de la page. Une vue extérieure garde toujours la sienne.

Paramètres

`page` la page ;

`view` la vue.

Résultat. L'échelle (mm de papier par mm réel).

Erreurs. Aucune.

`devicesOnPage` fonction

```
function devicesOnPage(project: Project, page: Page): Set<Id>
```

Équipements visibles sur une page : ceux des armoires de ses vues d'armoires, dans la plage de U d'au moins une de ces vues (PAG-16), et ceux de ses vues extérieures. Sert à la légende et au numéro de page des renvois (« page n »).

Paramètres

`project` le projet ;

`page` la page.

Résultat. Ensemble d'identifiants d'équipements.

Erreurs. Aucune.

`legendLinkTypes` fonction

```
function legendLinkTypes(project: Project, page: Page): LinkType[]
```

Types de liaisons de la légende (PAG-11) : ceux d'au moins une liaison dont une extrémité est sur un équipement visible de la page, dans l'ordre de la bibliothèque, sans les types exclus de la légende (`showInLegend` faux).

Paramètres

`project` le projet ;

`page` la page.

Résultat. Les types de liaisons, éventuellement aucun.

Erreurs. Aucune.

`deviceLabel` fonction

```
function deviceLabel(project: Project, device: Device): string | null
```

Texte de l'étiquette d'un équipement selon son mode (EQP-07) : aucun, son nom, ou son nom suivi du nom de sa machine entre parenthèses.

Paramètres

`project` le projet (pour la machine) ;

`device` l'équipement.

Résultat. Le texte, ou `null` en mode `none`.

Erreurs. Aucune ; machine absente : le nom seul.

`SIDE_LABELS` constante

```
const SIDE_LABELS = { gap: 6, width: 70 }
```

Colonne des étiquettes à côté d'une armoire (PAG-06), en mm de papier.

Champs

`gap` écart entre le bord droit de l'armoire et la colonne, 6 mm ;

`width` largeur de la colonne, 70 mm.

`sideLabelsWidth` fonction

```
function sideLabelsWidth(view: Pick<View, 'kind' | 'labelsBeside'>): number
```

Largeur occupée à droite d'une vue par ses étiquettes à côté de l'armoire (`gap + width`), 0 sans étiquettes ou pour une vue extérieure. Utilisée par la mise en page automatique pour espacer les vues.

Paramètres

`view` la vue (`kind`, `labelsBeside`).

Résultat. La largeur en mm de papier.

Erreurs. Aucune.

`SideLabel` interface

```
interface SideLabel { deviceId; text; anchorY; y }
```

Une étiquette à côté d'une armoire.

Champs

`deviceId` l'équipement ;

`text` son texte (`deviceLabel`) ;

`anchorY` milieu de la partie dessinée de l'équipement, en mm d'armoire relatifs au haut de la vue ;

`y` position de l'étiquette après écartement.

`rackSideLabels` fonction

```
function rackSideLabels(project: Project, view: View, minGap: number): SideLabel[]
```

Étiquettes des équipements d'une vue d'armoire (PAG-06) : une par équipement posé en U (pas les 0U) qui a un texte et dont une partie est dans la plage de U, alignée sur le milieu de sa partie dessinée ; triées de haut en bas, puis écartées vers le bas pour qu'aucune ne soit à moins de `minGap` de la précédente.

Paramètres

`project` le projet ;

`view` la vue d'armoire ;

`minGap` écart minimal en mm d'armoire (le rendu passe une hauteur de ligne divisée par l'échelle).

Résultat. Les étiquettes, vide pour une vue extérieure ou une armoire absente.

Erreurs. Aucune.

20.8 `model/search`

Recherche globale (ERG-05, commande `edit.find`, Ctrl+F) : équipements, ports et repères de câbles du projet, et page où un résultat est dessiné. Le module dépend de `model/pageGeometry` (places sur la page), de l'index du projet et de `portLabel` ; il est utilisé par `SearchDialog`. Décision du 2026-11-05, lot 40.

`SearchTarget` type

```
type SearchTarget = | { kind: 'device'; deviceId: Id } | { kind: 'port'; deviceId: Id;
portId: Id } | { kind: 'link'; linkId: Id };
```

Objet désigné par un résultat : `device` (`deviceId`), `port` (`deviceId`, `portId`) ou `link` (`linkId`). Une fois choisi, il est sélectionné dans le magasin.

`SearchHit` interface

```
interface SearchHit { target; label; detail; rank }
```

Un résultat de recherche.

Champs

`target` l'objet trouvé ;

`label` texte principal : nom de l'équipement, « équipement · port », repère du câble (« (sans repère) » s'il est vide) ;

`detail` texte secondaire : machine ; type de connecteur et repères des liaisons du port ; extrémités du câble ;

`rank` 0 : nom exact ; 1 : nom commençant par un mot cherché ; 2 : trouvé ailleurs.

`SearchResult` interface

```
interface SearchResult { hits; total }
```

Résultat d'une recherche.

Champs

`hits` les résultats retenus, triés, au plus la limite ;

`total` nombre total de correspondances avant la limite.

`normalizeSearch` fonction

```
function normalizeSearch(text: string): string
```

Normalise un texte pour la comparaison : décomposition Unicode, suppression des signes diacritiques combinants, minuscules ; « Équipement » devient « equipement ».

Paramètres

`text` le texte.

Résultat. Le texte normalisé.

Erreurs. Aucune.

`searchProject` fonction

```
function searchProject(project: Project, query: string, limit = 100): SearchResult
```

Recherche dans le projet (ERG-05). Chaque mot de la recherche doit être trouvé, sans tenir compte de la casse ni des accents. Un câble est trouvé par son repère ; un équipement par son nom, sa description, ses champs libres (clés et valeurs) ou le nom de sa machine ; un port par « nom de l'équipement nom du port », avec au moins un mot dans le nom du port (le nom d'un équipement seul ne liste pas tous ses ports). Tri : rang, puis sorte (câbles, équipements, ports), puis ordre naturel des libellés.

Paramètres

`project` le projet ;

`query` la recherche ;

`limit` nombre maximal de résultats rendus, 100 par défaut.

Résultat. Un `SearchResult` ; vide pour une recherche vide.

Erreurs. Aucune.

```
const { hits, total } = searchProject(project, 'sw01 25');
// hits[0].target : { kind: 'port', deviceId, portId }
```

`targetBoxOnPage` fonction

```
function targetBoxOnPage(project: Project, pageId: Id, target: SearchTarget): Box
| undefined
```

Rectangle d'une page où un résultat est dessiné : union des places de l'équipement, du port, ou des deux extrémités d'un câble.

Paramètres

`project` le projet ;

`pageId` la page ;

`target` l'objet.

Résultat. Un `Box` en mm de page, ou `undefined` si la page n'existe pas ou ne dessine pas l'objet.

Erreurs. Aucune.

`pageOfTarget` fonction

```
function pageOfTarget(project: Project, target: SearchTarget, preferredPageId: Id | null):
{ pageId: Id; box: Box } | undefined
```

Page qui montre un résultat : la page préférée (la page active) si elle le dessine, sinon la première page qui le dessine, dans l'ordre du projet ; avec le rectangle à faire défiler jusqu'à la vue.

Paramètres

`project` le projet ;

`target` l'objet ;

`preferredPageId` la page active, ou `null`.

Résultat. `{ pageId, box }`, ou `undefined` si aucune page ne le dessine (l'objet est alors seulement sélectionné, avec un message).

Erreurs. Aucune.

20.9 `model/titleBlockLayout`

Mise en page du cartouche (PAG-09, PAG-10) : où va chaque champ visible. Un champ « sur toute la hauteur » (`full`) forme une colonne (par défaut le logo et le titre de la page) ; les autres se partagent un bloc d'informations de deux lignes (`top`, `bottom`) placé à l'endroit de

son premier champ dans l'ordre de la liste ; la légende (PAG-11), quand elle est dans le cartouche, prend l'extrémité droite. Le module ne dépend que du schéma (`TitleBlockField`) ; il est réexporté par `src/model/index.ts` et utilisé par `TitleBlock` (`src/render/TitleBlock.tsx`) et l'aperçu de la fenêtre « Cartouche ». Décision du 2026-11-05, lot 43.

`TitleBlockRow` type

```
type TitleBlockRow = 'top' | 'bottom' | 'full';
```

Place d'un champ : `top` (ligne du haut du bloc d'informations), `bottom` (ligne du bas) ou `full` (colonne sur toute la hauteur du cartouche).

`defaultTitleBlockRow` fonction

```
function defaultTitleBlockRow(kind: TitleBlockField['kind']): TitleBlockRow
```

Place d'un champ dont le projet ne choisit pas la position (`row` `null`) : logo et titre de la page sur toute la hauteur, nom du projet en haut, tous les autres en bas. C'est ce qui garde le cartouche par défaut identique à celui d'avant le lot 43.

Paramètres

`kind` la sorte de champ.

Résultat. La place par défaut.

Erreurs. Aucune.

`titleBlockRow` fonction

```
function titleBlockRow(field: Pick<TitleBlockField, 'kind' | 'row'>): TitleBlockRow
```

Place effective d'un champ : son choix, sinon celle de sa sorte.

Paramètres

`field` le champ (`kind`, `row`).

Résultat. `top`, `bottom` ou `full`.

Erreurs. Aucune.

`TitleBlockCell` interface

```
interface TitleBlockCell { x; y; width; height }
```

Rectangle d'une case du cartouche, en mm de page.

Champs

`x` bord gauche ;

`y` bord haut ;

`width` largeur ;

`height` hauteur.

`TitleBlockLayout` interface

```
interface TitleBlockLayout { fields; legend }
```

Résultat de la mise en page.

Champs

`fields` les champs visibles avec leur case, dans l'ordre de la liste ;

`legend` case de la légende, ou `null` si elle n'est pas dans le cartouche.

`LEGEND_SHARE` constante

```
const LEGEND_SHARE = 0.24
```

Part de la largeur du cartouche prise par la légende : 0,24 (24 %).

`COLUMN_SHARE` constante

```
const COLUMN_SHARE = 0.22
```

Part de la largeur du cartouche d'une colonne pleine hauteur autre que le logo, à côté du bloc d'informations : 0,22 (22 %), bornée à 60 % de la place restante pour toutes ces colonnes ensemble.

`LOGO_SHARE` constante

```
const LOGO_SHARE = 0.12
```

Part de la largeur du cartouche prise par la colonne du logo : 0,12 (12 %), bornée par `LOGO_MAX_MM`.

`LOGO_MAX_MM` constante

```
const LOGO_MAX_MM = 45
```

Largeur maximale de la colonne du logo, 45 mm, pour qu'un logo ne mange pas le cartouche d'une grande page.

`titleBlockLayout` fonction

```
function titleBlockLayout(fields: readonly TitleBlockField[], box: TitleBlockCell, options:
{ legend: boolean }): TitleBlockLayout
```

Calcule les cases des champs visibles du cartouche dans un rectangle. Les colonnes sont prises dans l'ordre de la liste ; le bloc d'informations est inséré à la place de son premier champ ; dans une ligne, les champs se partagent la largeur à parts égales ; une ligne sans champ laisse toute la hauteur à l'autre. Sans bloc d'informations, les colonnes autres que le logo se partagent à parts égales la place laissée par le logo et la légende.

Paramètres

`fields` les champs du cartouche du projet, dans leur ordre (les masqués sont ignorés) ;

`box` le rectangle du cartouche, en mm de page ;

`options` `legend` : la légende est dans le cartouche.

Résultat. Un `TitleBlockLayout`.

Erreurs. Aucune.

```
const layout = titleBlockLayout(project.titleBlock.fields,
  { x: 10, y: 174, width: 277, height: TITLE_BLOCK_HEIGHT },
  { legend: page.legend.position === 'titleBlock' });
```

Chapitre 21

Référence : bibliothèques, équipements intégrés et démarrage

Ce chapitre décrit, module par module, le catalogue des équipements intégrés (`src/library/equipment/`), la bibliothèque par défaut (`src/library/`), la page des bibliothèques chiffrées (`src/libraries/`), le démarrage de l'application (`src/main.tsx`, `src/app/`), l'internationalisation (`src/i18n/`) et le lecteur des notes de version (`src/help/`). La conception d'ensemble est expliquée au chapitre « Bibliothèques, équipements intégrés et démarrage ». Sauf mention contraire, ces modules ne dépendent pas de React et sont testés unitairement.

21.1 `app-version`

Numéro de version de l'application, lu dans `package.json` au moment de la construction. Vite remplace la constante globale `__APP_VERSION__` (déclarée dans `src/vite-env.d.ts`, définie dans `vite.config.ts`). Utilisé par la session (manifeste des archives `.rkdg` et `.rkdglib`) et par la fenêtre « À propos ». Le même numéro alimente le manuel, le site et le nom de l'archive (décision du lot 32).

`APP_VERSION` — constante

```
const APP_VERSION: string
```

Version de l'application (par exemple `1.4.0`), écrite dans le manifeste de chaque fichier enregistré (`appVersion`) et affichée dans l'aide. Hors de Vite (scripts exécutés par `tsx`, certains tests), la constante globale n'existe pas et la valeur vaut `0.0.0`.

21.2 `app/pwa`

Fonctionnement hors ligne et mises à jour (CG-03, WEB-08, décision du lot 31). Le service worker, généré au build par le greffon PWA de Vite, met en cache toute l'application ; une nouvelle version est proposée, jamais imposée. Dépend de `i18n/index`, de l'état de l'interface (`Ui`) et de la session ; appelé par `main`. Le bandeau de mise à jour (`src/ui/App.tsx`) lit `update` dans l'état de l'interface.

`updateReload` constante

```
const updateReload = { pending: false }
```

Drapeau mutable partagé : `pending` passe à `true` juste avant le rechargement d'une mise à jour. L'application (`src/ui/App.tsx`) le consulte dans son gestionnaire `beforeunload` pour ne pas afficher l'avertissement de fermeture (PRJ-04) : le brouillon vient d'être écrit et sera proposé au redémarrage.

Champs

`pending` `true` pendant le rechargement de mise à jour, `false` sinon.

`offerUpdate` fonction

```
function offerUpdate(ui: Ui, session: Session, activate: () => Promise<void>)
```

Propose une nouvelle version : place dans l'état de l'interface un objet `update` dont la fonction `reload` écrit d'abord le brouillon de secours (`session.flushDraft()`), lève `updateReload`, puis active la nouvelle version. Appelée par `startOffline` quand le service worker signale une version en attente ; exportée pour être testée sans service worker.

Paramètres

`ui` état de l'interface, où le bandeau est déclaré ;

`session` session courante, dont le brouillon est écrit avant le rechargement ;

`activate` fonction qui active le nouveau service worker et recharge la page.

Résultat. Aucun ; l'effet est la modification de l'état de l'interface. Rien n'est rechargé tant que l'utilisateur n'a pas choisi « Recharger ».

Erreurs. Aucune à l'appel. Lors du rechargement, une erreur d'écriture du brouillon rejette la promesse de `reload` et la nouvelle version n'est pas activée.

`startOffline` fonction

```
async function startOffline(ui: Ui, session: Session)
```

Enregistre le service worker (module virtuel `virtual:pwa-register`, chargé à la demande), dans l'application construite seulement : en développement, ou si le navigateur ne connaît pas les service workers, la fonction ne fait rien. Une version en attente déclenche `offerUpdate` ; la première mise en cache complète affiche la notification « prêt à fonctionner hors ligne ». Appelée en dernier par `start()` dans `main`.

Paramètres

`ui` état de l'interface (bandeau et notifications) ;

`session` session courante, transmise à `offerUpdate`.

Résultat. Promesse résolue une fois l'enregistrement demandé.

Erreurs. Un échec du chargement du module d'enregistrement rejette la promesse ; les erreurs d'enregistrement du service worker lui-même sont gérées par le greffon.

21.3 `app/session`

Session de travail (décision du lot 06) : fichier courant, ouverture (PRJ-02), enregistrement (PRJ-03), brouillon de secours et sa restauration au démarrage (PRJ-05), nouveau projet (PRJ-01, PRJ-10) et ouverture d'une bibliothèque pour la modifier (BIB-13). Indépendant

de React : l'interface s'abonne à `session.store` (Zustand). Dépend de l'éditeur (`src/store/editor.ts`), du stockage local et du brouillon (`src/io/storage.ts`, `src/io/draft.ts`), des archives (`src/io/archive.ts`), des fichiers (`src/io/files.ts`) et de `src/model/libraryEdit.ts`. Créée par `main`, utilisée par les commandes du menu Fichier et par `app/pwa`. Testée par `src/app/session.test.ts`.

`SessionMessage` interface

```
interface SessionMessage { message; details }
```

Erreur à montrer à l'utilisateur après une ouverture, un enregistrement ou une restauration manqués ; la fenêtre d'erreur de l'interface l'affiche.

Champs

- `message` phrase principale en français (déjà traduite) ;
- `details` détails éventuels, un par ligne (erreurs de validation du fichier) ; liste vide sinon.

`SessionState` interface

```
interface SessionState { fileName; fileHandle; draftOffer; error; busy }
```

État observable de la session, dans `session.store`.

Champs

- `fileName` nom du fichier courant, ou `null` pour un projet jamais enregistré ;
- `fileHandle` poignée File System Access pour réécrire le même fichier (« Enregistrer »), ou `null` (fichier choisi sans cette API, bibliothèque ouverte en édition, nouveau projet) ;
- `draftOffer` brouillon modifié trouvé au démarrage, en attente de la décision de l'utilisateur, ou `null` ;
- `error` erreur à afficher (`SessionMessage`) ou `null` ;
- `busy` `true` pendant une lecture ou une écriture de fichier.

`SessionOptions` interface

```
interface SessionOptions { confirm; draftDebounceMs?; draftIntervalMs?; files? }
```

Options de `createSession`.

Champs

- `confirm` demande une confirmation à l'utilisateur (message traduit) ; rend `true` pour continuer. Dans l'application, ouvre la fenêtre de confirmation de l'interface ;
- `draftDebounceMs` facultatif : délai d'écriture du brouillon après une modification, en ms (1000 par défaut, voir `src/io/draft.ts`) ;
- `draftIntervalMs` facultatif : intervalle de l'écriture périodique, en ms (30 000 par défaut) ;
- `files` facultatif : remplaçants de `pickFile` et `saveFile` pour les tests.

`Session` interface

```
interface Session { store; start; restoreDraft; dismissDraft; open; openLibrary; openFile;
save; saveAs; newProject; newFromExisting; dismissError; flushDraft; stop }
```

Objet rendu par `createSession`. Les méthodes asynchrones ne lèvent pas pour une erreur de fichier : elles la placent dans `store` (`error`).

Champs

`store` magasin Zustand de l'état (`SessionState`) ;

`start` cherche un brouillon modifié : s'il existe, le propose (`draftOffer`) sans démarrer l'enregistreur automatique ; sinon démarre l'enregistreur, qui écrit aussitôt un premier brouillon ;

`restoreDraft` restaure le brouillon proposé (document marqué modifié, nom et poignée du fichier repris), puis démarre l'enregistreur ;

`dismissDraft` efface le brouillon proposé, puis démarre l'enregistreur ;

`open` après confirmation si le document est modifié, choisit un fichier et l'ouvre (projet, ou bibliothèque en édition selon l'extension) ;

`openLibrary` idem, en ne proposant que des `.rkdglib` (BIB-13) ;

`openFile` ouvre un fichier déjà obtenu (glisser-déposer, lancement), avec sa poignée éventuelle ;

`save, saveAs` enregistrent le projet, ou la bibliothèque éditée (nom obligatoire, nom de fichier proposé `<nom>_v<version>.rkdglib`) ; l'archive est construite entièrement avant l'écriture (NF-04) ; un enregistrement annulé ne change rien ;

`newProject` remplace le document par le projet donné (et ses images), après confirmation ; rend `false` si l'utilisateur renonce ;

`newFromExisting` choisit un `.rkdg` et démarre un nouveau projet qui en reprend la bibliothèque, le cartouche et les propriétés, et le contenu si `keepContent` (PRJ-10) ; rend `true` si un projet a été créé ;

`dismissError` efface l'erreur affichée ;

`flushDraft` écrit le brouillon tout de suite (avant un rechargement, tests) ;

`stop` arrête l'enregistreur automatique.

`createSession` fonction

```
function createSession(editor: Editor, storage: LocalStorage | null, options:
SessionOptions): Session
```

Crée la session de l'application, une seule fois, dans `start()` de `main` ; les tests en créent avec un stockage en mémoire et des fichiers simulés.

Paramètres

`editor` éditeur du document (historique, état « modifié ») ;

`storage` stockage local (IndexedDB), ou `null` s'il est indisponible : il n'y a alors ni brouillon ni restauration ;

`options` voir `SessionOptions`.

Résultat. La session (`Session`), état initial vide, enregistreur non démarré (appeler `start`).

Erreurs. Aucune à la création. Les erreurs de lecture ou d'écriture de fichier sont placées dans `store.getState().error`, y compris un fichier déposé devenu illisible dans `openFile` ; une erreur du stockage local pendant `start` ou `dismissDraft` rejette la promesse.

```
const session = createSession(editor, storage, {
  confirm: (message) => window.confirm(message),
});
await session.start();
if (session.store.getState().draftOffer) await session.restoreDraft();
```

21.4 `app/site`

Liens vers le site public de Portolan (WEB-02, WEB-09, décision du lot 39b) : l'archive téléchargeable (menu Aide) et la page des bibliothèques protégée (menu Bibliothèque). Ils n'existent que sur le site : `tools/build-site.ts` marque la copie publiée de `index.html` d'une balise `<meta>` ; ailleurs, les commandes `help.archive` et `library.teamLibraries` (`src/commands/help.ts`, `src/commands/libraryFiles.ts`) sont désactivées avec leur raison. Fonctions pures, sauf `siteLinks` qui lit le document. Testé par `src/app/site.test.ts`.

`SITE_MARKER` constante

```
const SITE_MARKER = 'portolan-site'
```

Valeur de l'attribut `name` de la balise `<meta>` posée par la construction du site (`portolan-site`). Sert à l'écrire (`siteMarkerTag`) et à la retrouver (`siteLinks`).

`LIBRARIES_PAGE` constante

```
const LIBRARIES_PAGE = 'bibliotheques/'
```

Chemin de la page des bibliothèques, relatif à l'application (`bibliotheques/`, WEB-03) : sur le site, l'application est à la racine et la page juste à côté.

`SiteLinks` interface

```
interface SiteLinks { archive; libraries }
```

Liens du site, relatifs à l'application, rendus par `parseSiteMarker`.

Champs

`archive` nom de l'archive de l'application et de son manuel (`portolan-<version>.zip`) ;

`libraries` page des bibliothèques (`LIBRARIES_PAGE`).

`parseSiteMarker` fonction

```
function parseSiteMarker(content: string | null | undefined): SiteLinks | null
```

Interprète le contenu de la balise du site. Seul est accepté un nom de la forme `portolan-<version>.zip` (lettres, chiffres, `.` `+` `-`) : la balise ne peut pas faire pointer l'application vers une adresse arbitraire.

Paramètres

`content` contenu de l'attribut `content`, éventuellement absent ; les espaces autour sont ignorés.

Résultat. Les liens (`SiteLinks`), ou `null` si le contenu manque ou n'a pas la forme attendue.

Erreurs. Aucune.

```
parseSiteMarker('portolan-1.2.0.zip');
// { archive: 'portolan-1.2.0.zip', libraries: 'bibliotheques/' }
parseSiteMarker('https://exemple.test/x.zip'); // null
```

`siteMarkerTag` fonction

```
function siteMarkerTag(archive: string): string
```

Texte HTML de la balise du site pour une archive donnée, utilisé par `withSiteMarker`.

Paramètres

`archive` nom de l'archive (`portolan-<version>.zip`).

Résultat. `<meta name="portolan-site" content="…" />`.

Erreurs. Lève une `Error` si le nom d'archive n'est pas accepté par `parseSiteMarker`.

`withSiteMarker` fonction

```
function withSiteMarker(html: string, archive: string): string
```

Ajoute la balise du site dans l'en-tête de la page HTML de l'application. Appelée par `tools/build-site.ts` sur la seule copie publiée de `index.html` ; l'archive, faite depuis `dist/`, n'a donc pas la balise.

Paramètres

`html` contenu de `index.html` ;

`archive` nom de l'archive publiée à côté.

Résultat. La page avec la balise insérée juste avant `</head>`.

Erreurs. Lève une `Error` si la balise est déjà présente, si la page n'a pas de `</head>`, ou si le nom d'archive est invalide.

`siteLinks` fonction

```
function siteLinks(): SiteLinks | null
```

Liens du site de l'application en cours d'exécution, lus dans sa propre page (aucune requête, donc même réponse en ligne et hors ligne). Appelée par les commandes du menu Aide et du menu Bibliothèque pour savoir si elles sont actives et où mener.

Résultat. Les liens (`SiteLinks`), ou `null` hors du site ou hors d'un navigateur (pas de `document`).

Erreurs. Aucune.

21.5 `help/documents`

Les documents de l'utilisateur livrés avec l'application (DOC-01, DOC-04, DOC-11, lot 50) : la prise en main rapide et le manuel utilisateur complet, compilés depuis `manual/` et copiés dans le build (donc dans le cache hors ligne, sur le site et dans l'archive). Données pures, lues par les commandes du menu Aide (`commands/help`) et par les liens de l'écran d'accueil et de « À propos » (`DocumentLinks`). La documentation technique n'en fait pas partie : son chemin est `TECHNICAL_MANUAL_URL`. Testé par `src/help/documents.test.ts` (chemins, commandes, textes présents dans `fr.json`).

`UserDocument` interface

```
interface UserDocument { id; url; linkKey }
```

Description d'un document de l'utilisateur.

Champs

- `id` `quickStart` ou `manual` : identifiant, aussi le suffixe de la commande d'aide qui l'ouvre (`help.quickStart`, `help.manual`) ;
- `url` chemin du PDF, relatif à l'application ;
- `linkKey` clé de traduction du libellé de son lien (écran d'accueil, « À propos »).

`QUICK_START_URL` constante

```
const QUICK_START_URL = 'manual/prise-en-main-rapide.pdf'
```

Chemin relatif de la prise en main rapide dans l'application déployée (`manual/prise-en-main-rapide.pdf`, DOC-11). Le chemin ne porte pas la version : elle est sur la page de titre et dans les métadonnées du PDF, ce qui garde stables les liens et le cache hors ligne.

`MANUAL_URL` constante

```
const MANUAL_URL = 'manual/portolan.pdf'
```

Chemin relatif du manuel utilisateur complet dans l'application déployée (`manual/portolan.pdf`, DOC-01 à DOC-04) ; le PDF est copié dans le build et précaché, donc disponible hors ligne. Ouvert par `help.manual`, l'écran d'accueil et le dialogue « À propos ».

`USER_DOCUMENTS` constante

```
const USER_DOCUMENTS: readonly UserDocument[]
```

Les deux documents de l'utilisateur, dans l'ordre de lecture : prise en main rapide, puis manuel utilisateur (clés de liens `help.quickStartLink` et `help.manualLink`). L'ordre est celui des liens affichés par `DocumentLinks`.

21.6 `help/markdown`

Lecteur Markdown minimal des notes de version affichées par le menu Aide (WEB-02, décision du lot 39b) : titres, paragraphes, listes à puces, `` `code` `` et `**gras**`. Pur : le résultat est un arbre de données que la fenêtre (`src/ui/dialogs/HelpDialogs.tsx`) rend en éléments React, en nœuds texte seulement ; toute autre syntaxe reste du texte littéral. La source, `src/help/notes-de-version.md`, est importée telle quelle dans le bundle. Testé par `src/help/markdown.test.ts`.

`Span` interface

```
interface Span { kind; text }
```

Morceau de texte d'une ligne, avec son style.

Champs

- `kind` `text` (texte ordinaire), `code` (texte entre accents graves, pris littéralement) ou `strong` (texte entre doubles astérisques) ;
- `text` texte, sans les marques Markdown.

`Block` type

```
type Block = | { kind: 'heading'; level: 1 | 2 | 3 | 4; spans: Span[] } | { kind:
'paragraph'; spans: Span[] } | { kind: 'list'; items: Span[][] };
```

Bloc d'un document Markdown, union de trois variantes :

Champs

`heading` titre : `level` de 1 à 4 (nombre de `#`) et ses morceaux `spans` ; la fenêtre prend le titre de niveau 1 comme titre ;

`paragraph` paragraphe : lignes consécutives réunies par une espace, en morceaux `spans` ;

`list` liste à puces : `items`, un tableau de morceaux par élément.

`parseInline` fonction

```
function parseInline(text: string): Span[]
```

Découpe une ligne en morceaux : `` `code` `` et `**gras**` ; les morceaux de texte ordinaire voisins sont fusionnés et les morceaux vides omis. Appelée par `parseMarkdown` pour chaque titre, paragraphe et élément de liste.

Paramètres

`text` texte d'une ligne (ou d'un paragraphe déjà réuni).

Résultat. La liste des morceaux (`Span`), vide pour un texte vide.

Erreurs. Aucune ; une marque non fermée reste du texte.

`parseMarkdown` fonction

```
function parseMarkdown(markdown: string): Block[]
```

Découpe un texte Markdown en blocs. Une ligne vide termine le paragraphe ou la liste en cours ; une ligne `-` ou `*` suivie d'une espace commence un élément de liste ; une ligne indentée qui suit un élément le prolonge ; une ligne non indentée après une liste commence un paragraphe. Les fins de ligne Windows sont acceptées.

Paramètres

`markdown` texte complet du fichier.

Résultat. Les blocs (`Block`) dans l'ordre du texte.

Erreurs. Aucune.

```
parseMarkdown('# Notes\n\n- **Ajout** de `x`');
// [ { kind: 'heading', level: 1, spans: [{ kind: 'text', text: 'Notes' }] },
//   { kind: 'list', items: [[ { kind: 'strong', text: 'Ajout' },
//     { kind: 'text', text: ' de ' }, { kind: 'code', text: 'x' } ]] } ]
```

21.7 `i18n/index`

Internationalisation (CG-05, décision du lot 00). Crée une instance i18next indépendante de React, chargée de `src/i18n/fr.json`, initialisée de façon synchrone à l'import : le modèle, les règles métier, les générateurs et la page des bibliothèques produisent leurs messages français par `t` ; React reçoit l'instance par `I18nextProvider` (`main`). Fournit aussi le formatage

des nombres, dates et tailles selon la langue de l'interface. Importé en premier par `main` et par `test-setup`. Testé par `src/i18n/i18n.test.ts`.

`i18n` constante

```
const i18n = i18next.createInstance()
```

Instance i18next de l'application : langue `fr` (aussi langue de repli), ressources de `fr.json`, initialisation synchrone, interpolation sans échappement HTML (le rendu se fait par React ou `textContent`). Passée à `I18nextProvider` ; sa propriété `language` sert aux formats `Intl`.

`t` fonction

```
function t(key: string, options?: TOptions): string
```

Traduit une clé de `fr.json`, avec interpolation (`{{nom}}`) et pluriels (`count`). C'est la seule voie d'un texte visible : aucun texte en dur dans le code.

Paramètres

`key` clé pointée (`session.openFailed`) ;

`options` facultatif : valeurs à interpoler, `count` pour le pluriel, etc.

Résultat. Le texte traduit ; la clé elle-même si elle n'existe pas.

Erreurs. Aucune.

```
t('equipment.names.kvm', { count: 8 }); // « Console KVM 8 ports »
```

`formatNumber` fonction

```
function formatNumber(value: number): string
```

Formate un nombre selon la langue de l'interface (séparateur de milliers, virgule décimale). Utilisée par les messages de placement, le tableau des liaisons, l'export PDF et `formatBytes`.

Paramètres

`value` nombre à formater.

Résultat. Le texte, par exemple « 1 150 » (avec une espace insécable) ou « 1,5 ».

Erreurs. Aucune.

`formatDate` fonction

```
function formatDate(isoDate: string): string
```

Formate une date ISO (`AAAA-MM-JJ`) en date courte de la langue de l'interface : « 4 nov. 2026 ». La date est construite en heure locale (pas de décalage d'un jour selon le fuseau). Utilisée par le cartouche, les fenêtres de projet et la page des bibliothèques.

Paramètres

`isoDate` date `AAAA-MM-JJ` (sans heure).

Résultat. La date formatée ; le texte reçu tel quel si l'une des trois parties manque ou n'est pas un nombre non nul.

Erreurs. Aucune.

`formatBytes` fonction

```
function formatBytes(bytes: number): string
```

Taille en octets dans une unité lisible : octets en dessous de 1024, sinon Ko ou Mo avec une décimale (« 1,2 Mo »). Les unités viennent de `fr.json` (`units.bytes`, `units.kilobytes`, `units.megabytes`) et le nombre est formaté par `formatNumber`. Utilisée pour le poids des images et des bibliothèques (panneau bibliothèque, éditeur de machine, import et export de bibliothèques, page des bibliothèques du site).

Paramètres

`bytes` taille en octets.

Résultat. Le texte, par exemple « 512 o », « 3,4 Ko », « 1,2 Mo ».

Erreurs. Aucune.

21.8 `libraries/catalogue`

Catalogue des bibliothèques publiées (WEB-03, WEB-05, décision du lot 33) : schémas Zod de la fiche de chaque bibliothèque. Le catalogue est écrit par `tools/publish-libraries.ts`, chiffré comme les bibliothèques (`libraries/crypto`), et lu par `libraries/page` après déchiffrement. Ne dépend que de Zod.

`CatalogueModelSchema` constante

```
const CatalogueModelSchema = …
```

Schéma d'un modèle listé dans une fiche (aperçu textuel avant téléchargement).

Champs

`name` nom de la machine ;

`category` catégorie (clé de catégorie d'équipement) ;

`manufacturer` fabricant, texte libre de la bibliothèque de marque ;

`reference` référence commerciale ;

`heightU` hauteur en U (0 pour un équipement 0U ou extérieur).

`CatalogueEntrySchema` constante

```
const CatalogueEntrySchema = …
```

Schéma de la fiche d'une bibliothèque publiée.

Champs

`id` identifiant de la bibliothèque ;

`name` nom affiché ;

`brand` marque (vide pour une bibliothèque sans marque) ;

`version` version de la bibliothèque (entier) ;

`date` date du fichier, ISO ;

`author, description, license` métadonnées du fichier `.rkdglib` ;

`size` poids du `.rkdglib` en clair, en octets ;

`fileName` nom du fichier à télécharger ;

`path` chemin du fichier chiffré, relatif à la page ;

`models` modèles (`CatalogueModelSchema`).

`CatalogueEntry` type

```
type CatalogueEntry = z.infer<typeof CatalogueEntrySchema>;
```

Type d'une fiche de bibliothèque, déduit de `CatalogueEntrySchema` ; utilisé par la page pour construire une carte et par le script de publication pour la produire.

`CatalogueSchema` constante

```
const CatalogueSchema = z.object({ publishedAt: z.string(), libraries:
z.array(CatalogueEntrySchema), })
```

Schéma du catalogue complet, validé par la page après déchiffrement.

Champs

`publishedAt` date et heure de la publication, ISO ;

`libraries` fiches des bibliothèques (`CatalogueEntrySchema`).

`Catalogue` type

```
type Catalogue = z.infer<typeof CatalogueSchema>;
```

Type du catalogue, déduit de `CatalogueSchema`.

`CATALOGUE_PATH` constante

```
const CATALOGUE_PATH = 'catalogue.bin'
```

Nom du fichier chiffré du catalogue (`catalogue.bin`), relatif à la page des bibliothèques ; écrit par le script de publication, lu par la page.

21.9 `libraries/crypto`

Chiffrement des bibliothèques publiées (WEB-03, WEB-04, décision du lot 33) : AES-256-GCM, clé dérivée du mot de passe de l'équipe par PBKDF2-SHA-256, sel et vecteur d'initialisation aléatoires pour chaque fichier. Uniquement WebCrypto : le même code sert au script de publication (Node, `tools/publish-libraries.ts`) et à la page (`libraries/page`). Format d'un fichier : `PLB1` | itérations (entier 32 bits gros-boutiste) | sel (16 octets) | IV (12 octets) | chiffré et étiquette. Aucun mot de passe n'est stocké. Testé par `src/libraries/crypto.test.ts`.

`PBKDF2_ITERATIONS` constante

```
const PBKDF2_ITERATIONS = 600_000
```

Nombre d'itérations PBKDF2 utilisé par défaut pour chiffrer (600 000, minimum exigé par WEB-03). Il est écrit dans chaque fichier : on peut l'augmenter sans rendre illisibles les fichiers déjà publiés. Les tests passent une valeur plus faible pour aller vite.

`WrongPasswordError` classe

```
class WrongPasswordError extends Error
```

Erreur levée par `decrypt` quand l'authentification GCM échoue : mauvais mot de passe ou fichier altéré, que l'algorithme ne distingue pas. La page la reconnaît (`instanceof`) pour afficher « mot de passe incorrect » plutôt qu'une erreur de lecture.

`NotEncryptedError` classe

```
class NotEncryptedError extends Error
```

Erreur levée par `decrypt` quand les données ne commencent pas par l'en-tête `PLB1` ou sont plus courtes que l'en-tête : ce n'est pas un fichier chiffré par Portolan.

`encrypt` fonction

```
async function encrypt(plain: Uint8Array, password: string, iterations =
PBKDF2_ITERATIONS): Promise<Uint8Array>
```

Chiffre des données avec un mot de passe : tire un sel et un IV, dérive la clé, chiffre, puis écrit l'en-tête et le chiffré. Appelée par le script de publication pour chaque bibliothèque et pour le catalogue.

Paramètres

`plain` données en clair (contenu d'un `.rkdglib`, ou JSON du catalogue) ;

`password` mot de passe de l'équipe (lu par le script dans une variable d'environnement, jamais écrit dans le dépôt) ;

`iterations` facultatif : nombre d'itérations PBKDF2 (`PBKDF2_ITERATIONS` par défaut).

Résultat. Le fichier chiffré complet (en-tête compris). Deux appels sur les mêmes données donnent des résultats différents (sel et IV aléatoires).

Erreurs. Rejette si WebCrypto refuse l'opération (par exemple un nombre d'itérations nul).

`decrypt` fonction

```
async function decrypt(data: Uint8Array, password: string): Promise<Uint8Array>
```

Déchiffre un fichier produit par `encrypt` : lit l'en-tête (nombre d'itérations compris), dérive la clé, déchiffre et authentifie. Appelée par la page pour le catalogue et pour chaque bibliothèque, et par les tests de publication.

Paramètres

`data` fichier chiffré ;

`password` mot de passe saisi.

Résultat. Les données en clair, identiques à celles qui ont été chiffrées.

Erreurs. `NotEncryptedError` si l'en-tête manque ; `WrongPasswordError` si le mot de passe est faux ou le fichier altéré.

```
const data = await encrypt(new TextEncoder().encode('{}'), password);
const plain = await decrypt(data, password); // Uint8Array de '{}'
```

21.10 `libraries/page`

Script de la page des bibliothèques de l'équipe (WEB-03 à WEB-05, décision du lot 33), chargé par `site/bibliotheques/index.html` et construit par `vite.libraries.config.ts`, avec sa propre politique de sécurité du contenu. Il n'exporte rien : à l'import, il cherche l'élément `#libraries` et y monte la page. Écrit sans React ; le DOM est construit par `createElement` et `textContent` seulement (aucune injection HTML). Déroulement : formulaire de mot de passe ; lecture de `CATALOGUE_PATH` (sans cache), déchiffrement (`decrypt`) et validation (`CatalogueSchema`) ; une carte par bibliothèque (marque, version et date, nombre de modèles et poids, auteur, licence, description) avec « Télécharger » (le `.rkdglib` déchiffré, identique à l'original, à importer par BIB-10) et « Voir les modèles » (déchiffre le fichier, lit `project.json` et les images de l'archive avec fflate, montre chaque machine avec l'image de sa face avant, ou sa couleur). Un catalogue absent (404) affiche « rien de publié » ; un mauvais mot de passe et une autre erreur ont chacun leur message. Le mot de passe reste en mémoire le temps de la page, n'est ni enregistré ni envoyé. Textes par `t` (clés `libraryPage.*`). Testé de bout en bout sur le site construit.

21.11 `library/defaultModels`

Contenu « modèles » de la bibliothèque par défaut (§ 7.2, EQT-02, EQT-03, décision des lots 11 et 34) : modèles d'armoires et machine générique de chaque équipement, sans déclinaison. Neutre (CG-08). Les identifiants sont fixes et chaque machine a la bibliothèque par défaut pour origine, pour pouvoir être réimportée (BIB-07). Réexporte aussi `DEFAULT_LIBRARY_ID` et `DEFAULT_LIBRARY_VERSION`. Utilisé par `createDefaultLibrary`. Testé par `src/library/defaultLibrary.test.ts`.

`createDefaultRackModels` fonction

```
function createDefaultRackModels(): RackModel[]
```

Crée les 10 modèles d'armoires par défaut : 12U (600 × 600 mm), 18U et 24U (600 × 800 mm), 42U et 47U (800 × 1000 mm), chacun fixe et roulant. Identifiants fixes `builtinId(0x400 + n)` ; noms traduits (« Armoire 42U 800 × 1000 roulante ») ; emplacements 0U : un à l'arrière gauche, plus un à l'arrière droite à partir de 24U.

Résultat. Une nouvelle liste de `RackModel` à chaque appel (copiable sans risque).

Erreurs. Aucune.

`createDefaultMachines` fonction

```
function createDefaultMachines(): Machine[]
```

Crée les machines génériques de tous les équipements, dans l'ordre du catalogue (`createGenericMachines`).

Résultat. 19 machines génériques, identifiants fixes.

Erreurs. Aucune.

21.12 `library/defaultTypes`

Types de connecteurs (§ 6.7) et types de liaisons (§ 6.8) par défaut (décision du lot 01). Leurs identifiants sont fixes (`builtinId(0x100 + rang)` pour les connecteurs, `builtinId(0x200 +`

rang) pour les liaisons), si bien que projets et bibliothèques les partagent ; on n'ajoute donc qu'en fin de liste. Les noms de connecteurs sont des désignations techniques stockées comme données ; les noms des types de liaisons viennent de fr.json (defaults.linkTypes). Utilisé par les paramètres et générateurs d'équipements, le dessin des ports (src/render/portShapes.ts), le routage (goulottes), l'import tableur et le store. Le module est réexporté par library/index.

CONNECTOR_KEYS — constante

```
const CONNECTOR_KEYS: readonly string[]
```

Clés des 35 types de connecteurs par défaut (rj45, sfpPlus, c13…), dans l'ordre de la liste. Le formulaire des paramètres (src/ui/machine/paramFields.ts) en tire les choix des groupes de ports libres.

connectorName — fonction

```
function connectorName(key: string): string
```

Nom affiché d'un type de connecteur par défaut, par sa clé (sfpPlus → « SFP+ »). Utilisée pour nommer les groupes de ports et proposer le nom d'une machine.

Paramètres

`key` clé d'un type par défaut.

Résultat. Le nom, ou la clé elle-même si elle est inconnue.

Erreurs. Aucune.

CONNECTOR_IDS — constante

```
const CONNECTOR_IDS: Readonly<Record<string, Id>>
```

Identifiants fixes des types de connecteurs par défaut, par clé (CONNECTOR_IDS.rj45). Sert à résoudre les clés des paramètres en identifiants (connectorTypeIdOf) et à reconnaître une clé (isDefaultConnector).

CONNECTOR_SHAPES — constante

```
const CONNECTOR_SHAPES: ReadonlyMap<Id, string>
```

Dessin (forme) actuel de chaque type de connecteur par défaut, par identifiant (MAC-02). Le rendu des ports (src/render/portShapes.ts) s'en sert pour dessiner un type par défaut avec sa forme d'origine, même si celle enregistrée dans un ancien projet diffère.

createDefaultConnectorTypes — fonction

```
function createDefaultConnectorTypes(): ConnectorType[]
```

Crée les 35 types de connecteurs par défaut : identifiant fixe, nom, famille (cuivre, optique, vidéo, USB, série, audio, alimentation), forme et type complémentaire (C13 et C14, C19 et C20, C5 et C6). Appelée pour la bibliothèque par défaut et pour rajouter à un projet les types par défaut qui lui manquent (missingConnectorTypes).

Résultat. Une nouvelle liste de ConnectorType.

Erreurs. Lève une Error seulement si la liste interne référence une clé inconnue (incohérence du code, vérifiée par les tests).

`LINK_TYPE_IDS` constante

```
const LINK_TYPE_IDS: Readonly<Record<string, Id>>
```

Identifiants fixes des 7 types de liaisons par défaut, par clé : `ethernet`, `power`, `video`, `fiber`, `usb`, `serial`, `audio`. Le routage des goulottes (`src/routing/trays.ts`) s'en sert pour séparer courants forts et faibles.

`DEFAULT_LINK_TYPE_ID` constante

```
const DEFAULT_LINK_TYPE_ID: Id
```

Identifiant du type de liaison Ethernet, type par défaut d'une nouvelle liaison (LIA-03) et repli de l'import tableur et de la fenêtre de liaison.

`createDefaultLinkTypes` fonction

```
function createDefaultLinkTypes(): LinkType[]
```

Crée les 7 types de liaisons par défaut : nom traduit, couleur, trait plein de 0,5 mm, connecteurs admis, préfixe de repère (`ETH-`, `PWR-`, `VID-`, `FO-`, `USB-`, `SER-`, `AUD-`), présence dans la légende, sans marqueurs.

Résultat. Une nouvelle liste de `LinkType`.

Erreurs. Lève une `Error` seulement en cas d'incohérence interne (clé de connecteur inconnue).

21.13 `library/equipment/change`

Changements d'une machine déjà utilisée (EQT-04, EQT-06, BIB-03, décision du lot 34) : nouveaux paramètres, machine éditée dans l'éditeur, variante d'une machine, autre machine donnée à un équipement placé. Tous les équipements placés de la machine la suivent ; le changement est refusé si l'un d'eux chevauche, sort de l'armoire ou dépasse la profondeur ; les liaisons sur des ports disparus sont signalées et conservées (LIA-07). Fonctions pures : le store (`src/store/machineActions.ts`) applique le résultat. Dépend des générateurs, de `src/model/placement.ts`, `src/model/lookup.ts` et `src/model/links.ts`. Testé par `equipment.test.ts`, `machineEdit.test.ts` et `customConnectors.test.ts`.

`MachineChange` type

```
type MachineChange = | { ok: true; /** The regenerated machine (same id). */ machine: Machine; /** The project with the regenerated machine. */ project: Project; /** Links attached to ports that no longer exist. */ orphanLinks: Link[]; /** One French message per lost port: « SW01 · 25 (liaison « ETH-004 ») ». */ warnings: string[]; } | { ok: false; /** Fren…
```

Résultat d'un changement de machine, union discriminée par `ok`.

Champs

- `ok: true` changement possible : `machine` (la machine nouvelle ou régénérée), `project` (le projet à appliquer), `orphanLinks` (liaisons dont un port a disparu, laissées dans le projet) et `warnings` (un message par port perdu : « SW01 · 25 (liaison « ETH-004 ») ») ;
- `ok: false` changement refusé : `problems`, messages en français (paramètres invalides, machine inconnue, équipements qui ne tiendraient plus).

`withMachine` fonction

```
function withMachine(project: Project, machine: Machine): Project
```

Remplace dans la bibliothèque du projet la machine de même identifiant, sans rien toucher d'autre (ni équipements, ni types de connecteurs).

Paramètres

`project` projet de départ (non modifié) ;

`machine` nouvelle version de la machine.

Résultat. Un nouveau projet ; inchangé en substance si aucune machine n'a cet identifiant.

Erreurs. Aucune.

`missingConnectorTypes` fonction

```
function missingConnectorTypes(project: Project, machines: readonly Machine[]):
ConnectorType[]
```

Types de connecteurs *par défaut* utilisés par les ports des machines données et absents du projet (supprimés par l'utilisateur, BIB-09) : ils reviennent avec ces machines. Un type créé dans le projet n'est jamais recréé : ses paramètres sont refusés tant qu'il manque. Utilisée par le store (création de machine, réimport des génériques).

Paramètres

`project` projet ;

`machines` machines à ajouter ou régénérer.

Résultat. Les types manquants, copies neuves de `createDefaultConnectorTypes`.

Erreurs. Aucune.

`machineConnectorTypeIds` fonction

```
function machineConnectorTypeIds(machine: Machine): Set<Id>
```

Types de connecteurs utilisés par une machine (BIB-09) : ceux de ses ports, plus les types du projet choisis dans ses paramètres (un type choisi compte même si aucun port n'a été produit). Utilisée par l'export de bibliothèque (`src/model/libraryTransfer.ts`) et par `connectorTypeUsers`.

Paramètres

`machine` machine examinée.

Résultat. L'ensemble des identifiants.

Erreurs. Aucune.

`connectorTypeUsers` fonction

```
function connectorTypeUsers(project: Project, connectorId: Id): Machine[]
```

Machines du projet qui utilisent un type de connecteur (ports ou paramètres). Sert à refuser la suppression d'un type utilisé et à remplir la colonne « Utilisé par » de la fenêtre des types de connecteurs.

Paramètres

`project` projet ;

`connectorId` identifiant du type de connecteur.

Résultat. Les machines concernées, dans l'ordre de la bibliothèque.

Erreurs. Aucune.

`withConnectorTypes` fonction

```
function withConnectorTypes(project: Project, machines: readonly Machine[]): Project
```

Projet complété des types de connecteurs par défaut dont les machines données ont besoin (`missingConnectorTypes`), ajoutés en fin de liste.

Paramètres

`project` projet ;

`machines` machines qui vont y entrer ou y être régénérées.

Résultat. Le même objet projet s'il ne manque rien, sinon un nouveau projet.

Erreurs. Aucune.

`changeMachine` fonction

```
function changeMachine(project: Project, machine: Machine): MachineChange
```

Remplace la machine `machine.id` par `machine`, telle qu'éditée (paramètres, faces, ports placés sur une photo ; BIB-03, EQT-06). Étapes : machine connue ; paramètres valides avec les types du projet (`paramsProblems`) ; types par défaut manquants rajoutés ; placement de chaque équipement de la machine revérifié ; liaisons perdues relevées. Utilisée par le store et par la mise à jour des machines importées.

Paramètres

`project` projet ;

`machine` nouvelle version de la machine (même identifiant).

Résultat. `MachineChange`.

Erreurs. Aucune exception : refus rendu dans `problems` (machine inconnue, paramètres invalides, équipement refusé).

`changeMachineParams` fonction

```
function changeMachineParams(project: Project, machineId: Id, params:
Record<string, unknown>): MachineChange
```

Régénère une machine avec de nouveaux paramètres (EQT-03, EQT-06) et vérifie les conséquences pour ses équipements placés et leurs liaisons, par `changeMachine`. Les identifiants des ports sont repris par (face, nom, connecteur) : les liaisons des ports qui restent suivent.

Paramètres

`project` projet ;

`machineId` machine à changer (pas une machine générique : l'interface crée alors une variante par `deriveMachine`) ;

`params` nouveaux paramètres complets, tels que saisis.

Résultat. `MachineChange`.

Erreurs. Aucune exception : machine inconnue ou paramètres invalides rendus dans `problems`.

```
const result = changeMachineParams(project, machine.id, {
  ...machine.equipment.params,
  heightU: 2,
});
if (result.ok) console.log(result.warnings);
else console.log(result.problems);
```

`addVariantMachine` fonction

```
function addVariantMachine(project: Project, sourceId: Id, machine: Machine, deviceIds:
readonly Id[] = []): MachineChange
```

Ajoute une variante déjà construite d'une machine source, juste après elle dans la bibliothèque, et la donne aux équipements choisis qui utilisent la source ; la source n'est pas modifiée (EQT-02, EQT-04). Mêmes contrôles que `changeMachine`.

Paramètres

`project` projet ;

`sourceId` machine dont la variante est issue ;

`machine` la variante (identifiant neuf, ports repris de la source pour que les liaisons suivent) ;

`deviceIds` facultatif : équipements à faire passer sur la variante ; ceux qui n'utilisent pas la source sont ignorés.

Résultat. `MachineChange`.

Erreurs. Aucune exception : source inconnue, paramètres invalides ou équipement refusé rendus dans `problems`.

`deriveMachine` fonction

```
function deriveMachine(project: Project, machineId: Id, params:
Record<string, unknown>, deviceIds: readonly Id[] = [], name?: string): MachineChange
```

Crée une variante d'une machine avec d'autres paramètres (identifiant neuf, non générique, sans origine), la nomme, puis l'ajoute par `addVariantMachine`. C'est la voie obligée pour une machine générique, qui n'est jamais modifiée.

Paramètres

`project` projet ;

`machineId` machine source ;

`params` paramètres de la variante ;

`deviceIds` facultatif : équipements de la source à faire passer sur la variante ;

`name` facultatif : nom de la variante ; vide ou absent, le nom proposé par `proposeMachineName`.

Résultat. `MachineChange`.

Erreurs. Aucune exception : refus rendus dans `problems`.

`regenerateEdited` fonction

```
function regenerateEdited(draft: Machine, params: Record<string, unknown>, original?:
Machine, connectorTypes?: readonly ConnectorType[]): Machine
```

Régénère la machine en cours d'édition dans l'éditeur de machine avec de nouveaux paramètres, comme `regenerateMachine`, mais en se souvenant des ports de la machine d'origine que le brouillon a perdus : un port retiré puis remis pendant la même édition garde son identifiant, sa position sur la photo et ses liaisons.

Paramètres

- `draft` machine en cours d'édition ;
- `params` nouveaux paramètres ;
- `original` facultatif : machine à l'ouverture de l'éditeur ;
- `connectorTypes` facultatif : types du projet, pour nommer les groupes.

Résultat. La machine régénérée (même identifiant que `draft`).

Erreurs. Celles de `generateMachine` : paramètres invalides ou noms de ports en double.

`DeviceMachineChange` type

```
type DeviceMachineChange = | { ok: true; project: Project; /** Links moved to the port of
the same name of the new machine. */ kept: number; /** Links whose port has no equivalent
on the new machine (kept, dangling). */ orphanLinks: Link[]; warnings: string[]; } | { ok:
false; problems: string[] };
```

Résultat de `changeDeviceMachine`, union discriminée par `ok`.

Champs

- `ok: true` `project` (projet modifié), `kept` (nombre d'extrémités de liaisons passées sur un port équivalent), `orphanLinks` (liaisons sans port équivalent, conservées et pendantes) et `warnings` (un message par port perdu) ;
- `ok: false` `problems`, messages en français.

`equivalentPort` fonction

```
function equivalentPort(machine: Machine, face: 'front' | 'rear', port: { name: string;
connectorTypeId: Id }): { id: Id } | undefined
```

Port d'une machine équivalent à un port d'une autre : même face, même nom et même type de connecteur (une liaison ne passe jamais sur un autre connecteur).

Paramètres

- `machine` machine d'arrivée ;
- `face` face du port (`front` ou `rear`) ;
- `port` nom et type de connecteur du port de départ.

Résultat. Le port équivalent, ou `undefined`.

Erreurs. Aucune.

`changeDeviceMachine` fonction

```
function changeDeviceMachine(project: Project, deviceId: Id, machineId: Id):
DeviceMachineChange
```

Donne une autre machine, du même équipement, à un équipement placé (EQT-04). Les liaisons de l'équipement passent sur les ports équivalents de la nouvelle machine (`equivalentPort`) ; les autres restent et sont signalées. Les ports ajoutés à l'équipement lui-même (PRT-04) ne bougent pas. L'indicateur de profondeur forcée est recalculé. Utilisée par les propriétés de l'équipement et le store.

Paramètres

`project` projet ;

`deviceId` équipement placé ;

`machineId` nouvelle machine.

Résultat. `DeviceMachineChange` ; projet inchangé et `kept` nul si la machine est déjà la sienne.

Erreurs. Aucune exception : équipement ou machine inconnus, machine d'un autre équipement, placement refusé rendus dans `problems`.

21.14 `library/equipment/generators`

Générateurs des équipements intégrés (§ 7.4.3) : fonctions pures paramètres → machine (hauteur, largeur, taille affichée, faces et ports), un générateur par équipement dans la table interne `GENERATORS`. Les identifiants des ports sont repris de la version précédente de la machine par (face, nom, connecteur), par (face, emplacement) pour un panneau de brassage ; un port placé par l'utilisateur garde sa position. Contient aussi la validation complète des paramètres (schéma, connecteurs, noms uniques) et les machines génériques (EQT-02). Dépend de `library/equipment/params`, `library/defaultTypes`, `src/model/factories.ts` et `src/model/schema.ts`. Testé par `equipment.test.ts`, `shelf.test.ts` et `customConnectors.test.ts`.

`DEFAULT_DEPTHS` constante

```
const DEFAULT_DEPTHS: Readonly<Record<EquipmentKind, number | null>>
```

Profondeur par défaut d'une nouvelle machine, en mm, par équipement (§ 6.4) : 300 pour un switch, 600 pour un serveur, 100 pour une multiprise ou un panneau de brassage, etc. ; `null` pour les extérieurs (profondeur inconnue : toute la profondeur utile). Lue par `generateMachine` à la création seulement : une régénération garde la profondeur de la machine.

`EQUIPMENT_COLORS` constante

```
const EQUIPMENT_COLORS: Readonly<Record<EquipmentKind, string>>
```

Couleur de façade du dessin par défaut de chaque équipement (anthracite pour l'actif, gris clair pour l'étagère…), en hexadécimal. Appliquée à la création ; une régénération garde la couleur de la face, sauf pour l'obturateur et l'équipement générique dont la couleur est un paramètre.

`IdReuse` type

```
type IdReuse = (face: FaceSide, key: string, connectorTypeId: Id) => Id;
```

Fonction qui donne l'identifiant d'un port généré d'après sa face, sa clé d'identité et son type de connecteur : l'identifiant du premier port précédent de même face, même clé et même connecteur pas encore donné, sinon un identifiant neuf ; jamais deux fois le même. La clé est le nom du port, ou `#n` (numéro d'emplacement) pour un panneau de brassage. Plusieurs ports précédents peuvent partager face et clé quand l'éditeur se souvient des ports d'origine (`regenerateEdited`) : le bon connecteur est cherché dans toute la liste. Passée à `grid`.

`connectorTypeIdOf` fonction

```
function connectorTypeIdOf(value: string): Id
```

Identifiant de type de connecteur d'une valeur des paramètres : l'identifiant fixe d'une clé par défaut, ou la valeur elle-même pour un type du projet (UUID).

Paramètres

`value` clé par défaut (`rj45`) ou identifiant d'un type du projet.

Résultat. L'identifiant.

Erreurs. Lève une `Error` si la valeur n'est ni une clé connue ni un UUID (les schémas l'interdisent : ne se produit pas après `parseParams`).

`ConnectorNames` type

```
type ConnectorNames = (value: string) => string;
```

Fonction qui donne le nom affiché d'une valeur de connecteur des paramètres (« SFP+ », ou le nom d'un type du projet), rendue par `connectorNames`.

`connectorNames` fonction

```
function connectorNames(connectorTypes: readonly ConnectorType[] = []): ConnectorNames
```

Fabrique la fonction de nommage des connecteurs : clés par défaut par leur nom (`connectorName`), types du projet par le leur ; un type du projet absent de la liste s'appelle « type inconnu ». Sert aux noms des groupes de ports (« Groupe 2 (SFP+) »), au nom proposé et au formulaire des paramètres.

Paramètres

`connectorTypes` facultatif : types de connecteurs du projet (liste vide par défaut).

Résultat. La fonction de nommage (`ConnectorNames`).

Erreurs. Aucune.

`PortSpec` interface

```
interface PortSpec { name; connectorKey; group?; key? }
```

Description d'un port à générer, avant sa disposition sur la face.

Champs

`name` nom du port (unique sur sa face) ;

`connectorKey` valeur de connecteur des paramètres (clé par défaut ou identifiant d'un type du projet) ;

`group` facultatif : nom du groupe affiché (« Groupe 1 (RJ45) »), vide par défaut ;

`key` facultatif : clé d'identité pour la reprise de l'identifiant (le nom par défaut ; le numéro d'emplacement pour un panneau de brassage).

`grid` fonction

```
function grid(specs: PortSpec[], face: FaceSide, rows: number, reuse: IdReuse, area: Area = FULL_AREA, size = 0.7): Port[]
```

Dispose des ports en grille dans une zone de la face : `rows` rangées, remplies de gauche à droite puis de haut en bas ; coordonnées relatives à la face (0 à 1), taille bornée. Utilisée par les générateurs de multiprise, panneaux, tiroir optique et extérieurs.

Paramètres

`specs` ports à disposer, dans l'ordre ;

`face` face (`front` ou `rear`) ;

`rows` nombre de rangées ;

`reuse` fonction de reprise des identifiants (`IdReuse`) ;

`area` facultatif : zone `x0, x1, y0, y1` en fractions de la face (par défaut la façade hors oreilles) ;

`size` facultatif : part de la cellule occupée par un port (0,7 par défaut).

Résultat. Les ports, `order` de 0 à n-1, non placés, sortie `auto`.

Erreurs. Celles de `connectorTypeIdOf`.

`mirrored` fonction

```
function mirrored(ports: Port[]): Port[]
```

Ports vus de l'arrière : l'abscisse de chaque port est remplacée par son symétrique (`1 - x`), pour que le port arrière soit derrière son port avant (le port 1 est à droite vu de l'arrière). Utilisée pour les panneaux de brassage et le tiroir optique.

Paramètres

`ports` ports disposés pour l'avant.

Résultat. De nouveaux ports, seule l'abscisse change.

Erreurs. Aucune.

`screenSizeMm` fonction

```
function screenSizeMm(diagonalInches: number): { width: number; height: number }
```

Taille affichée d'un écran 16 :9 d'après sa diagonale. Utilisée par le générateur d'écran.

Paramètres

`diagonalInches` diagonale en pouces.

Résultat. Largeur et hauteur en mm, arrondies : 24 pouces donnent 531 × 299 mm.

Erreurs. Aucune.

`proposeMachineName` fonction

```
function proposeMachineName<K extends EquipmentKind>(kind: K, params: ParamsInput<K>
| Record<string, unknown> = {}, connectorTypes: readonly ConnectorType[] = []): string
```

Nom proposé pour une machine d'après ses paramètres, en français : « Switch 24 RJ45 + 4 SFP+ », « Serveur 2U », « Multiprise 8 prises », « Écran 24 pouces »… L'équipement générique prend son libellé s'il en a un. Utilisée à la création d'une machine, d'une variante et par l'éditeur de machine.

Paramètres

`kind` équipement ;

`params` facultatif : paramètres tels que saisis (défauts sinon) ;

`connectorTypes` facultatif : types du projet, pour nommer leurs groupes.

Résultat. Le nom proposé.

Erreurs. Lève l'erreur Zod de `parseParams` si les paramètres sont invalides.

`portNameProblems` fonction

```
function portNameProblems(kind: EquipmentKind, params: Record<string, unknown>): string[]
```

Problèmes de noms des ports que des paramètres valides produiraient : deux ports d'une face au même nom (deux groupes au même préfixe, étiquette répétée, préfixe qui rejoint un port de service…). Lance le générateur avec des identifiants factices. Utilisée par `paramsProblems` et par le formulaire des paramètres.

Paramètres

`kind` équipement ;

`params` paramètres tels que saisis.

Résultat. Un message en français par nom en double ; liste vide si les noms sont uniques ou si les paramètres ne passent pas leur schéma (ce cas est rapporté par `schemaProblems`).

Erreurs. Aucune.

`paramsProblems` fonction

```
function paramsProblems(kind: EquipmentKind, params:
Record<string, unknown>, connectorTypes?: readonly ConnectorType[]): string[]
```

Validation complète de paramètres, en français : d'abord le schéma (`schemaProblems`) ; puis, si les types du projet sont donnés, l'existence des types du projet choisis (`connectorProblems`) ; enfin l'unicité des noms de ports (`portNameProblems`). Seule la première catégorie de problèmes non vide est rendue. Fonction à appeler avant toute génération venant de l'utilisateur ou d'un fichier ; utilisée par toutes les opérations de `library/equipment/change`.

Paramètres

`kind` équipement ;

`params` paramètres tels que saisis ;

`connectorTypes` facultatif : types de connecteurs du projet ; absent (projet inconnu : bancs, outils), seule la forme des valeurs de connecteur est contrôlée.

Résultat. Les messages ; liste vide si les paramètres sont valides.

Erreurs. Aucune.

`equipmentParamsProblem` fonction

```
function equipmentParamsProblem(kind: EquipmentKind, params:
Record<string, unknown>, connectorTypes?: readonly ConnectorType[]): string | null
```

Premier problème de paramètres, pour la validation d'un fichier ouvert ou d'une bibliothèque importée (`src/model/validate.ts`, erreur `invalidParams`).

Paramètres

`kind` équipement ;

`params` paramètres enregistrés ;

`connectorTypes` facultatif : types de connecteurs du fichier.

Résultat. Le premier message de `paramsProblems`, ou `null`.

Erreurs. Aucune.

`GenerateOptions` interface

```
interface GenerateOptions { newPortId?; connectorTypes? }
```

Options de `generateMachine`.

Champs

`newPortId` facultatif : source des identifiants des nouveaux ports (UUID aléatoires par défaut ; identifiants fixes pour les machines génériques) ;

`connectorTypes` facultatif : types de connecteurs du projet, pour nommer les groupes de ports et la machine quand ils utilisent un type du projet (BIB-09).

`generateMachine` fonction

```
function generateMachine<K extends EquipmentKind>(kind: K, params: ParamsInput<K>
| Record<string, unknown> = {}, previous?: Machine, options: GenerateOptions = {}): Machine
```

Génère la machine d'un équipement : paramètres validés et complétés, ports disposés, faces fusionnées avec la version précédente. Sans `previous`, crée une machine neuve (nom proposé, profondeur et couleur par défaut). Avec `previous`, la régénère : même identifiant, nom, marque, profondeur, image, mode de rendu, couleur et libellé des faces (sauf obturateur et générique), identifiants de ports repris, réglages des ports conservés (sortie, « multiple », position d'un port placé). Voir le chapitre de conception pour le détail des étapes.

Paramètres

`kind` équipement ;

`params` facultatif : paramètres tels que saisis (défauts de l'équipement sinon) ;

`previous` facultatif : version précédente de la machine ;

`options` facultatif : voir `GenerateOptions`.

Résultat. La machine, avec `equipment.params` sous forme complétée.

Erreurs. Lève l'erreur Zod si les paramètres sont invalides, une `Error` (messages des doublons, un par ligne) si deux ports d'une face portent le même nom, et l'erreur de `MachineSchema` si la machine régénérée est invalide. Appeler `paramsProblems` d'abord pour éviter ces exceptions.

```
const sw = generateMachine('switch', {
  portGroups: [{ connector: 'rj45', count: 48 }],
});
const twoU = regenerateMachine(sw, { ...sw.equipment.params, heightU: 2 });
// twoU.id === sw.id ; ports 1 à 48 : mêmes identifiants
```

`regenerateMachine` fonction

```
function regenerateMachine(machine: Machine, params:
Record<string, unknown>, connectorTypes?: readonly ConnectorType[]): Machine
```

Régénère une machine avec de nouveaux paramètres (EQT-03) : raccourci de `generateMachine` avec la machine comme version précédente. Ses équipements placés la suivent puisqu'ils la désignent par identifiant.

Paramètres

`machine` machine actuelle ;

`params` nouveaux paramètres ;

`connectorTypes` facultatif : types de connecteurs du projet.

Résultat. La machine régénérée, même identifiant.

Erreurs. Celles de `generateMachine`.

`DEFAULT_LIBRARY_ID` constante

```
const DEFAULT_LIBRARY_ID = builtinId(0x300)
```

Identifiant fixe de la bibliothèque par défaut (`builtinId(0x300)`), origine des machines génériques (BIB-07). Sert à reconnaître les machines issues de la bibliothèque par défaut à l'import et à la réimportation.

`DEFAULT_LIBRARY_VERSION` constante

```
const DEFAULT_LIBRARY_VERSION = 2
```

Version de la bibliothèque par défaut, inscrite dans l'origine des machines génériques (2 depuis le passage aux machines générées, lot 34). À augmenter si le contenu de la bibliothèque par défaut change de façon à justifier une mise à jour.

`genericMachineId` fonction

```
function genericMachineId(kind: EquipmentKind): Id
```

Identifiant fixe de la machine générique d'un équipement : `builtinId(0x500 + rang)`, le rang étant la position dans `EQUIPMENT_KINDS`.

Paramètres

`kind` équipement.

Résultat. L'identifiant.

Erreurs. Aucune.

`defaultLibraryOrigin` fonction

```
function defaultLibraryOrigin(machineId: Id): MachineOrigin
```

Origine « bibliothèque par défaut » d'une machine : identifiant et version de la bibliothèque par défaut, son nom traduit, et l'identifiant de la machine d'origine.

Paramètres

`machineId` identifiant de la machine dans la bibliothèque par défaut.

Résultat. Un objet `MachineOrigin`.

Erreurs. Aucune.

`createGenericMachine` fonction

```
function createGenericMachine(kind: EquipmentKind): Machine
```

Crée la machine générique d'un équipement (EQT-02) : paramètres par défaut, sans marque, identifiant fixe (`genericMachineId`), ports d'identifiants fixes (`builtinId(0x10000 + rang × 0x1000 + n)`), nom traduit (« Switch générique »), `generic: true`, origine = bibliothèque par défaut. Deux appels donnent des machines identiques.

Paramètres

`kind` équipement.

Résultat. La machine générique.

Erreurs. Aucune (les paramètres par défaut sont valides, vérifié par les tests).

`createGenericMachines` fonction

```
function createGenericMachines(): Machine[]
```

Machines génériques de tous les équipements, dans l'ordre de `EQUIPMENT_KINDS`. Contenu machines de la bibliothèque par défaut.

Résultat. 19 machines.

Erreurs. Aucune.

`missingGenericMachines` fonction

```
function missingGenericMachines(machines: readonly Machine[]): Machine[]
```

Machines génériques absentes d'une liste (une par équipement) : sert à « Réimporter depuis la bibliothèque par défaut » (BIB-07) et à compléter une bibliothèque qui n'en contient pas toutes.

Paramètres

`machines` machines du projet.

Résultat. Les machines génériques des équipements qui n'en ont pas, neuves.

Erreurs. Aucune.

`findGenericMachine` fonction

```
function findGenericMachine(machines: readonly Machine[], kind: EquipmentKind): Machine
| undefined
```

Machine générique d'un équipement dans une liste ; utilisée par le store quand un équipement est glissé depuis le panneau bibliothèque.

Paramètres

`machines` machines du projet ;

`kind` équipement.

Résultat. La machine, ou `undefined` si elle manque.

Erreurs. Aucune.

21.15 `library/equipment/index`

Point d'entrée du catalogue des équipements intégrés (EQT-01). Ne fait que réexporter : `EQUIPMENT_KINDS`, `EQUIPMENT_CATEGORIES`, `KIND_CATEGORY` et leurs types (`src/model/schema.ts`), les fonctions de catégorie de `src/model/equipment.ts`, puis tout `library/equipment/params`, `library/equipment/generators` et `library/equipment/change`. L'interface et le store importent le catalogue par ce module (ou par `library/index`).

21.16 `library/equipment/params`

Paramètres des équipements intégrés (§ 7.4.3, STD-01 à STD-12), sous forme de schémas Zod. Chaque champ a une valeur par défaut : `parse({})` donne les paramètres de la machine générique (EQT-02). Les messages d'erreur sont traduits. Contient aussi les fonctions sur les connecteurs choisis dans les paramètres (BIB-09, lot 39a). Dépend de `library/defaultTypes` et `i18n/index` ; utilisé par les générateurs, le formulaire des paramètres (`src/ui/machine/`) et les transferts de bibliothèque. Les bornes ci-dessous sont celles du code.

`isDefaultConnector` fonction

```
function isDefaultConnector(value: string): boolean
```

Indique si une valeur est la clé d'un type de connecteur par défaut (`rj45`, `sfpPlus`…).

Paramètres

`value` valeur de connecteur.

Résultat. `true` pour une clé de `CONNECTOR_IDS`.

Erreurs. Aucune.

`isCustomConnector` fonction

```
function isCustomConnector(value: string): boolean
```

Indique si une valeur est l'identifiant (UUID) d'un type de connecteur du projet (créé dans sa bibliothèque, BIB-09), plutôt qu'une clé par défaut.

Paramètres

`value` valeur de connecteur.

Résultat. `true` pour un UUID qui n'est pas une clé par défaut ; ne vérifie pas que le type existe.

Erreurs. Aucune.

`connectorValueOf` fonction

```
function connectorValueOf(id: Id): string
```

Valeur des paramètres pour un type de connecteur : la clé d'un type par défaut pour son identifiant fixe, l'identifiant lui-même sinon. Utilisée à l'import d'une bibliothèque pour qu'une machine dont le type a été rapproché d'un type par défaut se régénère à l'identique.

Paramètres

`id` identifiant du type de connecteur.

Résultat. La clé (`rj45`) ou l'identifiant.

Erreurs. Aucune.

`NETWORK_CONNECTORS` constante

```
const NETWORK_CONNECTORS = …
```

Clés des connecteurs proposés en premier pour les ports réseau des équipements actifs : RJ45, SFP, SFP+, SFP28, QSFP+, QSFP28, LC, SC. Lue par le formulaire des paramètres.

`PortGroupSchema` constante

```
const PortGroupSchema = …
```

Schéma d'un groupe de ports identiques des équipements de baie : connecteur × nombre, sur une face, noms préfixe + numéro à partir de 1.

Champs

`connector` clé par défaut ou identifiant d'un type du projet (forme seule contrôlée) ;

`count` nombre de ports, 1 à 96 ;

`face` `front` (défaut) ou `rear` ;

`prefix` préfixe des noms (vide par défaut : ports « 1 », « 2 »…).

`PortGroup` type

```
type PortGroup = z.output<typeof PortGroupSchema>;
```

Type d'un groupe de ports complété, déduit de `PortGroupSchema`.

`ExternalPortGroupSchema` constante

```
const ExternalPortGroupSchema = …
```

Schéma d'un groupe de ports d'un équipement extérieur (une seule face, l'avant).

Champs

`connector` clé par défaut ou identifiant d'un type du projet ;

`count` nombre de ports, 1 à 32 ;

`prefix` préfixe des noms (vide par défaut).

`ExternalPortGroup` type

```
type ExternalPortGroup = z.output<typeof ExternalPortGroupSchema>;
```

Type d'un groupe de ports d'extérieur complété, déduit de `ExternalPortGroupSchema`.

`PSU_TYPES` constante

```
const PSU_TYPES = ['c14', 'c20'] as const
```

Connecteurs d'entrée possibles des alimentations d'un équipement de baie : `c14`, `c20`. Les alimentations sont nommées PSU1, PSU2…, en face arrière.

`CONSOLE_TYPES` constante

```
const CONSOLE_TYPES = ['rj45Console', 'usbB', 'none'] as const
```

Port de console d'un équipement réseau : `rj45Console`, `usbB` ou `none` (pas de console).

`SwitchSchema` constante

```
const SwitchSchema = networkActive(() => [group('rj45', 24), group('sfpPlus', 4, 'SFP+')])
```

Paramètres d'un switch (STD-02), communs aux trois équipements réseau actifs.

Champs

`heightU` hauteur, 1 à 4U (1) ;

`halfWidth` demi-largeur (`false`) ;

`psu` alimentations : `count` 0 à 2 (1), `type` parmi `PSU_TYPES` (`c14`) ;

`portGroups` jusqu'à 16 groupes (`PortGroupSchema`) ; par défaut 24 RJ45 et 4 SFP+ préfixés « SFP+ » ;

`console` voir `CONSOLE_TYPES` (`rj45Console`) ;

`management` port de gestion RJ45 en face avant (`true`).

`RouterSchema` constante

```
const RouterSchema = networkActive(() => [group('rj45', 4), group('sfpPlus', 2, 'SFP+')])
```

Paramètres d'un routeur (STD-02) : mêmes champs que `SwitchSchema` ; groupes par défaut 4 RJ45 et 2 SFP+.

`FirewallSchema` constante

```
const FirewallSchema = networkActive(() => [group('rj45', 8), group('sfpPlus', 2, 'SFP+')])
```

Paramètres d'un pare-feu (STD-02) : mêmes champs que `SwitchSchema` ; groupes par défaut 8 RJ45 et 2 SFP+.

`ServerSchema` constante

```
const ServerSchema = serverLike(1)
```

Paramètres d'un serveur (STD-07).

Champs

`heightU` hauteur, 1 à 8U (1) ;

`halfWidth` demi-largeur (`false`) ;

`psu` alimentations : `count` 0 à 4 (2), `type` (`c14`) ;

`nics` cartes réseau, jusqu'à 16 groupes ; par défaut 2 RJ45 arrière NIC1, NIC2 ;

`management` port de gestion RJ45 arrière (`true`) ;

`video` port VGA arrière (`true`) ;

`usbFront, usbRear` nombre de ports USB-A, 0 à 8 (1 et 2).

`StorageSchema` constante

```
const StorageSchema = …
```

Paramètres d'une baie de stockage (STD-07) : ceux de `ServerSchema` avec une hauteur par défaut de 2U, plus :

Champs

`driveBays` nombre de baies de disques dessinées en face avant, 0 à 96 (12) ; dessin seulement, sans port.

`KvmSchema` constante

```
const KvmSchema = …
```

Paramètres d'une console KVM (STD-09).

Champs

`heightU` toujours 1 ;

`computerPorts` ports ordinateurs RJ45 arrière, 1 à 64 (8) ;

`consoleUsb, consoleVga` console locale (clavier et souris USB, écran VGA) en face arrière (`true`) ;

`psu` alimentations : 0 à 2 (1), `c14` par défaut.

`OUTLET_TYPES` constante

```
const OUTLET_TYPES = ['c13', 'c19', 'schuko', 'typeE'] as const
```

Types de prises des multiprises et onduleurs : `c13`, `c19`, `schuko`, `typeE`.

`OutletGroupSchema` constante

```
const OutletGroupSchema = …
```

Schéma d'un groupe de prises : nombre × type, noms préfixe + numéro (A1, A2…). Le préfixe garde les noms d'un groupe quelle que soit la taille des autres ; absent, la liste qui le contient lui donne la première lettre libre (`freeOutletPrefix`), qui est alors enregistrée avec les paramètres.

Champs

`count` nombre de prises, 1 à 48 ;

`type` voir `OUTLET_TYPES` ;

`prefix` facultatif : préfixe des noms.

`OutletGroup` type

```
type OutletGroup = z.output<typeof OutletGroupSchema> & { prefix: string };
```

Type d'un groupe de prises complété : celui du schéma, avec un `prefix` toujours présent.

`freeOutletPrefix` fonction

```
function freeOutletPrefix(groups: readonly { prefix?: string }[]): string
```

Première lettre (A à Z) qu'aucun groupe n'utilise comme préfixe ; si toutes sont prises, `G<n>-` où n est le nombre de groupes plus un. Utilisée par les schémas et par le formulaire quand on ajoute un groupe.

Paramètres

`groups` groupes existants (seul `prefix` est lu).

Résultat. Le préfixe libre, par exemple `B` après un groupe `A`.

Erreurs. Aucune.

`INLET_TYPES` constante

```
const INLET_TYPES = ['c14', 'c20', 'p17', 'fixedCable'] as const
```

Entrées d'une multiprise : `c14`, `c20`, `p17` ou `fixedCable` (câble fixe, dont le port prend le connecteur de la fiche, `PLUG_TYPES`).

`PLUG_TYPES` constante

```
const PLUG_TYPES = ['schuko', 'typeE', 'p17'] as const
```

Fiches possibles du câble fixe d'une multiprise : `schuko`, `typeE`, `p17`.

`HORIZONTAL_PDU_MAX_OUTLETS` constante

```
const HORIZONTAL_PDU_MAX_OUTLETS = 24
```

Nombre maximal de prises d'une multiprise horizontale, tous groupes confondus (24) ; au delà, le schéma refuse les paramètres (message sur `groups`). Une PDU verticale n'a pas cette limite.

`PduSchema` constante

```
const PduSchema = …
```

Paramètres d'une multiprise ou PDU (STD-01), formes horizontale et verticale réunies.

Champs

`form` `horizontal` (défaut) ou `vertical` (bande 0U) ;

`heightU` hauteur d'une multiprise horizontale, 1 ou 2U (1) ;

`lengthU` longueur d'une PDU verticale, en U couverts, 4 à 52 (18) ;

`groups` 1 à 12 groupes de prises (`OutletGroupSchema`), types mélangés permis ; par défaut 8 C13 préfixées A ;

`inlet` entrée (`INLET_TYPES`, `c14`) ;

`plug` fiche du câble fixe (`typeE`) ;

`switch, surgeProtection` interrupteur et parafoudre, dessinés seulement (`false`).

`UpsSchema` constante

```
const UpsSchema = …
```

Paramètres d'un onduleur (STD-11).

Champs

`heightU` hauteur, 1 à 4U (2) ;

`outlets` 1 à 8 groupes de prises de sortie, en face arrière ; par défaut 6 C13 préfixées A ;

`inlet` entrée `c14` ou `c20` (`c20`) ;

`network` carte réseau RJ45 arrière (`true`).

`BlankingPanelSchema` constante

```
const BlankingPanelSchema = …
```

Paramètres d'un obturateur (STD-03), sans port.

Champs

`heightU` hauteur, 1 à 4U (1) ;

`perforated` perforé (`false`) ;

`color` couleur des faces, `#rrggbb` (`#2f3236`).

`CABLE_MANAGER_TYPES` constante

```
const CABLE_MANAGER_TYPES = ['rings', 'brush', 'curtain', 'cover'] as const
```

Types de passe-câbles : `rings` (anneaux), `brush` (brosse), `curtain` (rideau à brosse), `cover` (capot).

`CableManagerSchema` constante

```
const CableManagerSchema = …
```

Paramètres d'un passe-câbles (STD-04), sans port.

Champs

`heightU` 1 ou 2U (1) ;

`type` voir `CABLE_MANAGER_TYPES` (`rings`).

`ShelfSchema` constante

```
const ShelfSchema = …
```

Paramètres d'une étagère (STD-06).

Champs

`heightU` 1 ou 2U (1) ;

`sliding` coulissante (`false`) ;

`portGroups` ports de la machine posée sur l'étagère (MAC-03), jusqu'à 32 groupes, tout connecteur, l'une ou l'autre face ; aucun par défaut.

`GenericSchema` constante

```
const GenericSchema = …
```

Paramètres de l'équipement générique (STD-10).

Champs

`heightU` hauteur, 1 à 52U (1) ;

`halfWidth` demi-largeur (`false`) ;

`label` libellé des faces, aussi nom proposé (vide) ;

`color` couleur des faces (`#4a4f57`) ;

`portGroups` groupes de ports libres, jusqu'à 32, tout connecteur, l'une ou l'autre face ; aucun par défaut.

`PATCH_MODULES` constante

```
const PATCH_MODULES = …
```

Modules d'un emplacement de panneau de brassage (STD-05) : RJ45 (Cat5e, Cat6, Cat6A, STP, UTP), HDMI, DisplayPort, USB, fibre LC ou SC, coaxial BNC ou F, XLR, prise électrique (type E) et `blank` (obturateur : pas de port). Chaque module correspond à un connecteur par défaut dans le générateur.

`PatchModule` type

```
type PatchModule = (typeof PATCH_MODULES)[number];
```

Type d'un module de panneau de brassage, une valeur de `PATCH_MODULES`.

`PatchPanelSchema` constante

```
const PatchPanelSchema = …
```

Paramètres d'un panneau de brassage modulaire (STD-05). Après validation, `modules` a exactement `slots` éléments : les emplacements non précisés reçoivent `fill`.

Champs

`heightU` 1 ou 2U (1) ;

`slots` nombre d'emplacements, 1 à 96 (24) ;

`fill` module des emplacements non précisés, « remplissage rapide » (`rj45Cat6a`) ;

`modules` module de chaque emplacement, à partir du premier ;

`labels` étiquette facultative de chaque emplacement, utilisée comme nom du port (le numéro sinon) ; l'identité du port reste l'emplacement.

`FIBER_CONNECTORS` constante

```
const FIBER_CONNECTORS = ['lc', 'sc', 'mpo'] as const
```

Connecteurs d'un tiroir optique : `lc`, `sc`, `mpo`.

`FiberPanelSchema` constante

```
const FiberPanelSchema = …
```

Paramètres d'un tiroir optique (STD-08). Les ports sont nommés cassette.port (« 1.1 », « 1.2 »…), à l'avant et à l'arrière, appariés.

Champs

`heightU` 1 ou 2U (1) ;

`cassettes` nombre de cassettes, 1 à 12 (4) ;

`connector` voir `FIBER_CONNECTORS` (`lc`) ;

`portsPerCassette` ports par cassette, 1 à 24 (12).

`VIDEO_INPUTS` constante

```
const VIDEO_INPUTS = ['hdmi', 'displayPort', 'vga', 'dvi', 'usbC', 'sdi'] as const
```

Entrées vidéo possibles d'un écran : `hdmi`, `displayPort`, `vga`, `dvi`, `usbC`, `sdi`. Une entrée répétée est numérotée (« HDMI », « HDMI2 »).

`EXTERNAL_POWER` constante

```
const EXTERNAL_POWER = ['c14', 'c6', 'c7', 'usbC', 'none'] as const
```

Alimentation d'un équipement extérieur : `c14`, `c6`, `c7`, `usbC` ou `none` (pas de port d'alimentation).

`MonitorSchema` constante

```
const MonitorSchema = …
```

Paramètres d'un écran (STD-12) ; la taille affichée est calculée pour un écran 16 :9 (`screenSizeMm`).

Champs

`diagonalInches` diagonale en pouces, 5 à 120 (24) ;

`videoInputs` jusqu'à 8 entrées (`VIDEO_INPUTS`) ; par défaut HDMI et DisplayPort ;

`usb` ports USB-A, 0 à 8 (0) ;

`power` alimentation (`EXTERNAL_POWER`, `c14`).

`WorkstationSchema` constante

```
const WorkstationSchema = …
```

Paramètres d'un poste de travail (STD-12), forme commune des boîtiers extérieurs.

Champs

`widthMm, heightMm` taille affichée en mm, 20 à 5000 (190 × 420) ;

`portGroups` jusqu'à 16 groupes (`ExternalPortGroupSchema`) ; par défaut LAN1, DP1–DP2, HDMI1, USB1–USB2 ;

`power` alimentation (`c14`).

`LaptopSchema` constante

```
const LaptopSchema = …
```

Paramètres d'un ordinateur portable (STD-12) : mêmes champs que `WorkstationSchema` ; 360 × 250 mm, ports LAN, HDMI et USB, alimentation USB-C.

`KeyboardMouseSchema` constante

```
const KeyboardMouseSchema = externalBox(440, 160, () => [ext('usbA', 2, 'USB')], 'none')
```

Paramètres d'un ensemble clavier et souris (STD-12) : mêmes champs que `WorkstationSchema` ; 440 × 160 mm, deux ports USB, sans alimentation.

`ControlPanelSchema` constante

```
const ControlPanelSchema = …
```

Paramètres d'un pupitre ou écran tactile (STD-12) : mêmes champs que `WorkstationSchema` ; 280 × 230 mm, ports LAN et USB, alimentation USB-C.

`PARAM_SCHEMAS` constante

```
const PARAM_SCHEMAS = …
```

Table des schémas de paramètres, par équipement ; le compilateur vérifie qu'elle couvre tous les `EquipmentKind`. Point d'entrée de toute validation (`parseParams`, `schemaProblems`) et source des types `Params` et `ParamsInput`. Tout nouvel équipement doit y avoir son schéma.

`ParamsInput` type

```
type ParamsInput<K extends EquipmentKind> = z.input<(typeof PARAM_SCHEMAS)[K]>;
```

Type des paramètres d'un équipement tels que saisis : tous les champs à valeur par défaut sont facultatifs. Accepté par `generateMachine` et `parseParams`.

`Params` type

```
type Params<K extends EquipmentKind> = z.output<(typeof PARAM_SCHEMAS)[K]>;
```

Type des paramètres d'un équipement complétés par leurs valeurs par défaut (sortie du schéma, transformations comprises), tels qu'enregistrés dans `machine.equipment.params` et reçus par les générateurs.

`parseParams` fonction

```
function parseParams<K extends EquipmentKind>(kind: K, params: ParamsInput<K>
| Record<string, unknown> = {}): Params<K>
```

Valide les paramètres d'un équipement et les complète avec les valeurs par défaut.

Paramètres

`kind` équipement ;

`params` facultatif : paramètres tels que saisis (objet vide par défaut).

Résultat. Les paramètres complétés (`Params`).

Erreurs. Lève une erreur Zod si les paramètres sont invalides ; utiliser `schemaProblems` pour obtenir des messages sans exception.

```
const pdu = parseParams('pdu', { groups: [{ count: 8, type: 'c13' }, { count: 4, type: 'c19' }] });
// pdu.groups[1].prefix === 'B'
```

`defaultParams` fonction

```
function defaultParams<K extends EquipmentKind>(kind: K): Params<K>
```

Paramètres par défaut d'un équipement, c'est-à-dire ceux de sa machine générique.

Paramètres

`kind` équipement.

Résultat. `parseParams(kind, {})`.

Erreurs. Aucune (les défauts sont valides).

`PORT_GROUP_PARAMS` constante

```
const PORT_GROUP_PARAMS: Readonly<Partial<Record<EquipmentKind, string>>>
```

Paramètre qui porte les groupes de ports libres (connecteur choisi par l'utilisateur) de chaque équipement : `portGroups` pour les équipements réseau actifs, l'étagère, l'équipement générique et les extérieurs à boîtier, `nics` pour serveur et stockage. Les équipements absents n'ont pas de connecteur libre. Lu par `paramConnectors` et `remapParamConnectors`.

`paramConnectors` fonction

```
function paramConnectors(kind: EquipmentKind, params: Record<string, unknown>): string[]
```

Valeurs de connecteur (clés ou identifiants) des groupes de ports libres de paramètres, lues sans validation (les éléments mal formés sont ignorés).

Paramètres

`kind` équipement ;

`params` paramètres, éventuellement non validés.

Résultat. Les valeurs, dans l'ordre des groupes, avec doublons.

Erreurs. Aucune.

`paramCustomConnectors` fonction

```
function paramCustomConnectors(kind: EquipmentKind, params: Record<string, unknown>): Id[]
```

Identifiants des types de connecteurs du projet que des paramètres utilisent (sans les clés par défaut), sans doublon. Utilisée par `machineConnectorTypeIds` et `connectorProblems`.

Paramètres

- `kind` équipement ;
- `params` paramètres.

Résultat. Les identifiants.

Erreurs. Aucune.

`connectorProblems` fonction

```
function connectorProblems(kind: EquipmentKind, params:
Record<string, unknown>, connectorTypes: readonly Pick<ConnectorType, 'id'>[]): string[]
```

Problèmes des connecteurs de paramètres face aux types du projet : un type du projet choisi qui n'existe plus (supprimé, ou absent d'un fichier).

Paramètres

- `kind` équipement ;
- `params` paramètres ;
- `connectorTypes` types de connecteurs du projet (seul `id` est lu).

Résultat. Un message en français par type manquant, qui cite son identifiant ; vide si tous existent.

Erreurs. Aucune.

`remapParamConnectors` fonction

```
function remapParamConnectors(kind: EquipmentKind, params: Record<string, unknown>, map:
(value: string) => string): Record<string, unknown>
```

Remplace les valeurs de connecteur des groupes de ports libres par une fonction de correspondance : import d'une machine dont les types de connecteurs sont rapprochés par identifiant puis par nom et famille (BIB-10). Les autres valeurs ne changent pas.

Paramètres

- `kind` équipement ;
- `params` paramètres de la machine importée ;
- `map` correspondance valeur d'origine → valeur dans le projet.

Résultat. De nouveaux paramètres si une valeur change, sinon le même objet.

Erreurs. Aucune.

`schemaProblems` fonction

```
function schemaProblems(kind: EquipmentKind, params: Record<string, unknown>): string[]
```

Messages en français des problèmes de paramètres face à leur schéma, chacun préfixé du chemin du champ (`portGroups.0.count : …`). `paramsProblems` y ajoute les connecteurs et les noms de ports.

Paramètres

`kind` équipement ;

`params` paramètres tels que saisis.

Résultat. Les messages ; vide si les paramètres sont valides.

Erreurs. Aucune.

21.17 `library/index`

Bibliothèque par défaut livrée avec l'application (PRJ-01, § 7.2), neutre (CG-08), et création des nouveaux projets. Assemble `library/defaultTypes` (réexporté) et `library/defaultModels` ; dépend de `src/model/factories.ts`. Utilisé par `main`, la commande « Nouveau projet », la fenêtre de nouveau projet et le projet d'exemple. Testé par `src/library/defaultLibrary.test.ts`.

`createDefaultLibrary` fonction

```
function createDefaultLibrary(): Library
```

Crée une copie neuve de la bibliothèque par défaut : 35 types de connecteurs, 7 types de liaisons, 10 modèles d'armoires et 19 machines génériques, identifiants fixes.

Résultat. Un objet `Library` indépendant (chaque projet a sa copie).

Erreurs. Aucune.

`createNewProject` fonction

```
function createNewProject(options: Omit<CreateProjectOptions, 'library'> = {}): Project
```

Crée un projet vide initialisé avec une copie de la bibliothèque par défaut (PRJ-01), sans page. Utilisée par le projet d'exemple et par `createStarterProject`.

Paramètres

`options` facultatif : options de `createProject` (nom, cartouche…), sauf la bibliothèque, toujours celle par défaut.

Résultat. Le projet.

Erreurs. Aucune.

`createStarterProject` fonction

```
function createStarterProject(options: Omit<CreateProjectOptions, 'library'> = {}): Project
```

Projet ouvert au démarrage et par « Nouveau projet » : bibliothèque par défaut et une page A4 paysage vide intitulée « Page 1 ».

Paramètres

`options` facultatif : comme `createNewProject`.

Résultat. Le projet.

Erreurs. Aucune.

21.18 `main`

Point d'entrée de l'application (chargé par `index.html`) ; n'exporte rien. À l'import : initialisation de `i18n/index`, police Inter embarquée (graisses 400, 500, 600 et 700, aucun chargement externe), styles globaux, titre du document, adresse du WebAssembly de libavoid. Puis `start()` : enregistrement des polices de symboles et attente des polices (le texte n'est mesuré qu'avec la police embarquée, NF-07b) ; mesureur de texte ; éditeur sur `createStarterProject` ; état de l'interface avec l'écran d'accueil ; ouverture du stockage local (sans brouillon s'il est indisponible) et préférence `imageLimitMb` (EDM-04) ; session (`createSession`) ; en développement, crochet de test `window.__portolan` (éditeur, session, interface, actions du store) ; montage de React avec ses contextes (i18n, éditeur, interface, session, adresses des images, mesure du texte) ; démarrage de la session ; en développement, `started` quand la fenêtre est affichée et la session prête ; enfin `startOffline`. Voir le chapitre de conception pour l'ordre et ses raisons.

21.19 `test-setup`

Préparation des tests unitaires, déclarée dans `setupFiles` de la configuration Vitest (`vite.config.ts`). N'exporte rien : il importe `i18n/index` pour que l'instance i18next soit initialisée avant tout test, de sorte que les messages produits par le modèle et les règles métier soient en français, comme dans l'application.

Chapitre 22

Référence : routage

Ce chapitre décrit, module par module, chaque déclaration exportée de `src/routing/`. La conception (algorithmes, décisions, tests) est expliquée au chapitre « Routage des câbles ». Sauf mention contraire, les coordonnées sont en millimètres de page (papier) et les fonctions sont pures et déterministes (ROU-05).

22.1 `routing/attach`

Points d'un tracé retouché rattachés aux vues de la page (ROU-16, lot 37). Chaque point est stocké relativement à une vue (`AttachedPoint` : `viewId`, `dx`, `dy` en millimètres du dessin de la vue, à l'échelle 1, depuis son coin haut gauche) ; il suit la vue quand elle est déplacée et garde sa place sur son dessin quand l'échelle change. Dépend de `viewBounds` (`model/pageGeometry`) et de `effectiveScale` (`model/pageLayout`). Utilisé par `routing/page` (relecture), `store/routeActions.ts` (enregistrement, suppression de vues) et `store/pageActions.ts` (duplication de page).

`distanceToBox` fonction

```
function distanceToBox(at: Point, box: Box): number
```

Distance euclidienne d'un point à un rectangle, nulle si le point est à l'intérieur ou sur le bord. Sert à choisir la vue de rattachement d'un point (`nearestView`).

Paramètres

`at` le point (mm de page) ;

`box` le rectangle (`x`, `y`, `width`, `height`, mm de page).

Résultat. la distance en millimètres, ≥ 0.

Erreurs. Aucune.

`nearestView` fonction

```
function nearestView(project: Project, views: readonly View[], page: Page, at: Point): View
| undefined
```

Vue à laquelle un point se rattache : celle qui le contient, sinon la plus proche par distance à son rectangle (`viewBounds` ; une vue sans rectangle compte comme un point à sa position). À égalité (vues superposées, distances égales à 10^{-6} près), la première vue de la liste gagne. Appelée par `attachPoints` et `reattachRemovedViews`.

Paramètres

`project` le projet (pour mesurer les vues) ;

`views` les vues candidates, dans l'ordre de la page ;

`page` la page qui porte ces vues ;

`at` le point, mm de page.

Résultat. la vue choisie, ou `undefined` si `views` est vide.

Erreurs. Aucune.

`attachPoints` fonction

```
function attachPoints(project: Project, page: Page, points: Point[]): AttachedPoint[]
| null
```

Rattache les points d'un tracé (mm de page) aux vues de la page : chaque point est arrondi au centième, rattaché à `nearestView`, puis exprimé en millimètres du dessin de la vue (écart divisé par l'échelle effective). Appelée par `setManualRoute` à chaque retouche.

Paramètres

`project` le projet ;

`page` la page du tracé ;

`points` la polyligne retouchée, mm de page.

Résultat. les points rattachés, dans l'ordre ; `null` si la page n'a aucune vue (la retouche est alors refusée).

Erreurs. Aucune exception.

```
const attached = attachPoints(project, page, [{ x: 40, y: 120 }, { x: 40, y: 180 }]);
if (attached) page.manualRoutes.push({ linkId: link.id, points: attached });
```

`resolvePoints` fonction

```
function resolvePoints(page: Page, points: readonly AttachedPoint[]): Point[]
```

Inverse de `attachPoints` : positions sur la page des points rattachés, à la position et à l'échelle *actuelles* de leur vue (`view.x + dx × échelle`), arrondies au millionième. Un point dont la vue n'existe plus (fichier abîmé) est lu depuis le coin haut gauche de la page (`dx`, `dy` pris comme millimètres de papier).

Paramètres

`page` la page, avec ses vues ;

`points` les points stockés.

Résultat. les points en mm de page, même nombre et même ordre.

Erreurs. Aucune.

`keepOrthogonal` fonction

```
function keepOrthogonal(points: readonly Point[]): Point[]
```

Rend un tracé orthogonal après que certaines de ses vues ont bougé : chaque segment devenu oblique reçoit un coude. Des deux coudes possibles, `(q.x, p.y)` (partir horizontalement) ou `(p.x, q.y)` (verticalement), celui qui fait le moins de changements de direction avec les segments voisins gagne, un demi-tour comptant pour trois ; à égalité, horizontalement. Le coude n'est jamais stocké : il est recalculé à chaque affichage.

Paramètres

`points` le tracé résolu, mm de page.

Résultat. un nouveau tableau ; un tracé déjà orthogonal est rendu à l'identique (copie).

Erreurs. Aucune.

```
keepOrthogonal([{ x: 0, y: 0 }, { x: 10, y: 5 }]);
// [{ x: 0, y: 0 }, { x: 10, y: 0 }, { x: 10, y: 5 }]
```

`manualPolyline` fonction

```
function manualPolyline(page: Page, route: Pick<ManualRoute, 'points'>): Point[]
```

Polyligne de page d'un tracé retouché stocké : `resolvePoints` puis `keepOrthogonal`. Appelée par `routePage` avant de rattacher les extrémités aux ports (`anchorManualRoute`).

Paramètres

`page` la page du tracé ;

`route` le tracé stocké (seul `points` est lu).

Résultat. la polyligne orthogonale, mm de page.

Erreurs. Aucune.

`reattachRemovedViews` fonction

```
function reattachRemovedViews(project: Project, page: Page, removed: readonly
View[], previous: Page): ManualRoute[]
```

Tracés retouchés d'une page après la suppression de certaines de ses vues : les points rattachés à une vue supprimée gardent leur place sur la page (calculée sur la page d'avant, à l'échelle d'avant) et se rattachent à la vue restante la plus proche. Sans vue restante, le tracé est retiré (retour au tracé automatique). Appelée par `followRemovedViews` dans `Editor.apply`, dans la même action annulable.

Paramètres

`project` le projet après la suppression ;

`page` la page après la suppression ;

`removed` les vues supprimées ;

`previous` la page avant la suppression (positions et échelles d'origine).

Résultat. la nouvelle liste des tracés de la page ; les tracés sans point sur une vue supprimée sont rendus tels quels.

Erreurs. Aucune.

`remapRouteViews` fonction

```
function remapRouteViews(routes: readonly ManualRoute[], ids: ReadonlyMap<Id, Id>):
ManualRoute[]
```

Tracés retouchés d'une page dupliquée : chaque point est rattaché à la copie de sa vue. Un identifiant absent de la table est conservé. Appelée lors de la duplication d'une page (`store/pageActions.ts`).

Paramètres

`routes` les tracés de la page d'origine ;

`ids` table identifiant de vue d'origine → identifiant de la copie.

Résultat. des copies des tracés.

Erreurs. Aucune.

22.2 `routing/avoid`

Chargement à la demande de libavoid-js (WebAssembly, LGPL-2.1-or-later, version épinglée 0.5.0-beta.5) : le module JavaScript et son fichier `libavoid.wasm` sont des fichiers séparés du build, sans accès réseau extérieur (CG-02), mis en cache pour le hors-ligne. Leur colle embind est transformée au build pour ne contenir aucun `new Function` (`tools/libavoid-no-eval.ts`, phase F). Utilisé par `main.tsx`, `LinksLayer`, les exports et les commandes de routage.

`AvoidModule` type

```
type AvoidModule = any;
```

Type de l'instance de libavoid (API embind : `Router`, `ShapeRef`, `ConnRef`, `Point`, `Rectangle`…). Volontairement `any` : le paquet ne fournit pas de types exploitables. Seul `routeOutside` l'utilise réellement ; ailleurs il circule comme une valeur opaque (`null` tant que le module n'est pas chargé).

`setAvoidWasmUrl` fonction

```
function setAvoidWasmUrl(url: string)
```

Indique l'URL du fichier `libavoid.wasm` dans le navigateur. Appelée une fois au démarrage (`src/main.tsx`, et le banc de rendu) avec l'URL fournie par Vite (`libavoid-wasm?url`). Sous Node (tests), elle n'est pas appelée et la bibliothèque trouve son fichier elle-même.

Paramètres

`url` URL du fichier WebAssembly, dans l'application.

Résultat. aucun.

Erreurs. Aucune.

`loadAvoid` fonction

```
function loadAvoid(): Promise<AvoidModule>
```

Charge libavoid une fois (import dynamique puis `AvoidLib.load`) ; les appels suivants rendent la même promesse, ou l'instance déjà chargée. Appelée par `useAvoid` (rendu), par les exports PNG et PDF avant de tracer, par le banc de rendu et par les tests.

Résultat. une promesse de l'instance `AvoidModule`.

Erreurs. La promesse est rejetée si le module ou le WebAssembly ne se chargent pas ; ce rejet est mémorisé (un nouvel appel rend la même promesse rejetée). Les appelants l'ignorent et gardent des lignes droites hors baie.

```
const avoid = await loadAvoid();
const routing = routePage(project, page, avoid);
```

`loadedAvoid` fonction

```
function loadedAvoid(): AvoidModule | null
```

Rend l'instance si elle est déjà chargée, sans rien déclencher. Utilisée là où l'on ne peut pas attendre (export SVG synchrone, commandes, poignées de retouche, état initial de `useAvoid`).

Résultat. l'instance, ou `null`.

Erreurs. Aucune.

22.3 `routing/bridges`

Croisements entre câbles et ponts (ROU-12, ROU-15) : là où deux câbles se croisent, un petit arc sur le segment le plus horizontal enjambe l'autre. Pur et déterministe ; le dessin (écran et exports) ne fait que traduire les ponts en chemin SVG (`cablePathData`). Utilisé par `routePage`, `LinksLayer`, `ui/canvas/Highlights.tsx` et les tests de croisements.

`BRIDGE_RADIUS_MM` constante

```
const BRIDGE_RADIUS_MM = 0.7
```

Rayon d'un pont, en millimètres de papier : 0,7, soit au moins 1,5 fois l'épaisseur d'un câble. Valeur par défaut de `bridgesByCable` et `cablePathData`.

`CablePaths` interface

```
interface CablePaths { id; paths }
```

Un câble tel que dessiné, en entrée de la recherche des croisements.

Champs

`id` identifiant (liaison, ou clé de faisceau pour un tronc) ; informatif, les résultats désignent les câbles par leur rang ;

`paths` ses polylignes, mm de page.

`SegmentRef` interface

```
interface SegmentRef { cable; path; segment }
```

Désigne un segment parmi les câbles passés à `findCrossings`.

Champs

`cable` rang du câble dans la liste ;

`path` rang de la polyligne dans `paths` ;

`segment` rang du segment, entre les points `segment` et `segment + 1`.

`Crossing` interface

```
interface Crossing { at; over; under }
```

Un croisement propre entre deux câbles différents.

Champs

`at` point de croisement, mm de page ;

`over` segment qui porte le pont (le plus horizontal) ;

`under` segment enjambé.

`Bridge` interface

```
interface Bridge { path; segment; start; end }
```

Un pont sur un segment d'un câble : arc de `start` à `end`, le long du segment, bombé vers le haut (vers la gauche sur un segment vertical).

Champs

`path` rang de la polyligne du câble ;

`segment` rang du segment dans la polyligne ;

`start` début de l'arc, sur le segment (mm de page) ;

`end` fin de l'arc ; `start` et `end` ne dépassent jamais les extrémités du segment.

`findCrossings` fonction

```
function findCrossings(cables: CablePaths[]): Crossing[]
```

Tous les croisements entre câbles différents. Un croisement est propre : les intérieurs des deux segments se coupent en un point situé à plus de 0,01 mm de leurs extrémités (un câble qui en touche un autre, ou partage un tronçon avec lui, ne le croise pas ; un câble ne se croise pas lui-même). Balayage par abscisse : seuls les segments dont les intervalles en x se recouvrent sont comparés. Le pont va sur le segment le plus horizontal, sur le premier câble à égalité.

Paramètres

`cables` les câbles, dans un ordre stable.

Résultat. les croisements triés par câble, polyligne et segment porteurs, puis position.

Erreurs. Aucune.

`countCrossings` fonction

```
function countCrossings(cables: CablePaths[]): number
```

Nombre de croisements entre câbles différents (longueur de `findCrossings`) : mesure de ROU-15 dans les tests (`clearance.test.ts`).

Paramètres

`cables` les câbles.

Résultat. un entier ≥ 0.

Erreurs. Aucune.

`bridgesByCable` fonction

```
function bridgesByCable(cables: CablePaths[], radius = BRIDGE_RADIUS_MM):
Map<number, Bridge[]>
```

Ponts de chaque câble (ROU-12) : un arc de `radius` de part et d'autre de chaque croisement, le long du segment porteur ; les arcs qui se chevauchent fusionnent ; un arc est borné aux extrémités de son segment, et un arc rogné à moins de la moitié du rayon est abandonné. Appelée par `routePage` sur les câbles puis les troncs de faisceaux.

Paramètres

`cables` les câbles dessinés (tracés retouchés et troncs compris) ;

`radius` rayon, mm de papier (défaut `BRIDGE_RADIUS_MM`).

Résultat. table rang du câble → ponts triés par polyligne et segment ; un câble sans pont est absent de la table.

Erreurs. Aucune.

`cablePathData` fonction

```
function cablePathData(paths: Point[][], bridges: Bridge[] = [], radius = BRIDGE_RADIUS_MM)
```

Données du chemin SVG (attribut `d`) d'un câble : ses polylignes (`M`, `L`) et, pour chaque pont, une commande d'arc `A` (elliptique pour des ponts fusionnés : aussi haute qu'un pont simple, aussi longue que nécessaire), coordonnées au millième. Utilisée par `LinksLayer` (écran et exports) et `Highlights`.

Paramètres

`paths` les polylignes du câble ;

`bridges` ses ponts (défaut : aucun) ;

`radius` hauteur maximale d'un arc (défaut `BRIDGE_RADIUS_MM`).

Résultat. la chaîne `d`, vide pour un câble sans point.

Erreurs. Aucune.

```
const d = cablePathData(drawing.paths, drawing.bridges);
// "M10 20 L10 40 L14.3 40 A0.7 0.7 0 0 1 15.7 40 L30 40"
```

22.4 `routing/bundles`

Faisceaux de câbles (ROU-11) et places des textes le long des tracés (ROU-10, étiquettes de faisceaux). Les câbles d'un faisceau partagent leur chemin ; leurs tronçons communs forment le tronc (trait épais annoté « 24 × ETH »), le reste forme les brins de chaque câble. Pur et déterministe. Utilisé par `routePage`, `LinksLayer`, l'export SVG et `commands/route.ts`.

`BUNDLE_WIDTH_FACTOR` constante

```
const BUNDLE_WIDTH_FACTOR = 2.4
```

Épaisseur du tronc d'un faisceau : 2,4 fois celle du type de liaison (reste plus fin que l'arc d'un pont). Utilisée par `LinksLayer`.

`BUNDLE_FONT_SIZE` constante

```
const BUNDLE_FONT_SIZE = 2.4
```

Taille de l'étiquette d'un faisceau, en millimètres de papier : 2,4. Première valeur de `BUNDLE_FONT_SIZES` et valeur initiale de `BundleDrawing.fontSize`.

`BUNDLE_FONT_SIZES` constante

```
const BUNDLE_FONT_SIZES = [BUNDLE_FONT_SIZE, 1.8, 1.4, 1.1]
```

Tailles essayées pour les étiquettes de faisceaux d'une page, de la plus grande à la plus petite (2,4 ; 1,8 ; 1,4 ; 1,1 mm) : la page prend la plus grande à laquelle toutes les étiquettes trouvent une place libre (faisceaux côte à côte dans une goulotte).

`CABLE_REF_FONT_SIZE` constante

```
const CABLE_REF_FONT_SIZE = 1.4
```

Taille d'un repère de câble (ROU-10), en millimètres de papier : 1,4. Utilisée pour placer les repères (`routePage`) et pour les dessiner (`LinksLayer`, export SVG).

`linkTypeShortName` fonction

```
function linkTypeShortName(linkType: Pick<LinkType, 'refPrefix' | 'name'>): string
```

Nom court d'un type de liaison dans l'étiquette d'un faisceau : son préfixe de repère débarrassé des caractères qui ne sont ni lettres ni chiffres en début et en fin (« ETH- » → « ETH »), sinon son nom.

Paramètres

`linkType` le type (seuls `refPrefix` et `name` sont lus).

Résultat. le nom court.

Erreurs. Aucune.

`BundleSplit` interface

```
interface BundleSplit { trunk; fans }
```

Découpage d'un faisceau en tronc et brins, rendu par `splitBundle`.

Champs

`trunk` tronçons partagés par au moins deux câbles, fusionnés et chaînés en polylignes (mm de page) ;

`fans` pour chaque câble (même ordre qu'en entrée), ses propres tronçons, de ses ports jusqu'au tronc.

`splitBundle` fonction

```
function splitBundle(cables: readonly (readonly Point[][])[]): BundleSplit
```

Tronc et brins d'un faisceau, à partir des chemins dessinés de chacun de ses câbles : un tronçon qui longe un tronçon d'un autre câble du faisceau (même droite, recouvrement de plus de 0,01 mm) appartient au tronc ; les morceaux de tronc sont fusionnés par droite puis chaînés bout à bout, de façon déterministe.

Paramètres

`cables` pour chaque câble, ses polylignes.

Résultat. le découpage ; un tronc vide signifie que les câbles ne partagent rien (ils sont alors dessinés seuls).

Erreurs. Aucune.

```
const { trunk, fans } = splitBundle([
  [[{ x: 0, y: 0 }, { x: 0, y: 10 }, { x: 20, y: 10 }]],
  [[{ x: 5, y: 0 }, { x: 5, y: 10 }, { x: 20, y: 10 }]],
]);
// trunk : [[{ x: 5, y: 10 }, { x: 20, y: 10 }]] ; fans[1] : [[{ x: 5, y: 0 }, { x: 5, y: 10 }]]
```

`TextPlace` interface

```
interface TextPlace { at; angle; anchor }
```

Place d'un texte le long d'une ligne.

Champs

`at` point d'ancrage, mm de page ;

`angle` angle en degrés, choisi pour que le texte ne soit jamais à l'envers (il se lit de gauche à droite, ou de bas en haut sur un segment vertical) ;

`anchor` le texte commence en `at` (`start`), s'y termine (`end`) ou y est centré (`middle`).

`alongPlace` fonction

```
function alongPlace(paths: readonly Point[][]): TextPlace | null
```

Milieu du plus long segment des polylignes (le premier à longueur égale), avec l'angle de lecture du segment. Version simple de `freeAlongPlace`, sans évitement des autres étiquettes.

Paramètres

`paths` les polylignes.

Résultat. la place, ancre `middle` ; `null` sans segment de longueur non nulle.

Erreurs. Aucune.

`Area` interface

```
interface Area { x0; y0; x1; y1 }
```

Rectangle sur la page (mm), aligné sur les axes, occupé par un texte.

Champs

- `x0` bord gauche ;
- `y0` bord haut ;
- `x1` bord droit ($\geq$ `x0`) ;
- `y1` bord bas ($\geq$ `y0`).

`textLength` fonction

```
function textLength(text: string, fontSize: number): number
```

Largeur estimée d'un texte, en millimètres : environ 0,62 em par caractère (Inter demi-gras) plus 0,5 em de marges du fond blanc. Le routage ne mesure pas le texte, pour rester indépendant de la police et du navigateur.

Paramètres

- `text` le texte ;
- `fontSize` la taille, mm de papier.

Résultat. `(text.length × 0,62 + 0,5) × fontSize`.

Erreurs. Aucune.

`textArea` fonction

```
function textArea(place: TextPlace, length: number, fontSize: number): Area
```

Rectangle englobant d'un texte de longueur donnée posé à une place, quel que soit son angle (hauteur prise à 1,2 fois la taille). Sert à tester les recouvrements.

Paramètres

- `place` la place du texte ;
- `length` sa longueur estimée, mm ;
- `fontSize` sa taille, mm.

Résultat. le rectangle englobant.

Erreurs. Aucune.

`freeAlongPlace` fonction

```
function freeAlongPlace(paths: readonly Point[][], length: number, fontSize: number, taken:
Area[], fallback = true): TextPlace | null
```

Place d'un texte le long d'un câble (repères ROU-10, étiquettes de faisceaux ROU-11) : sur ses segments, les plus longs d'abord (ceux qui contiennent le texte avant les autres), au milieu puis de plus en plus loin de part et d'autre par pas d'un vingtième du segment (0,5 mm au moins), jusqu'à ce que le texte ne touche plus que l'extrémité du segment ; la première place qui ne recouvre aucune zone de `taken` est retenue et ajoutée à `taken` (la fonction **modifie** ce tableau).

Paramètres

`paths` les polylignes du câble ou du tronc ;

`length` longueur estimée du texte (`textLength`) ;

`fontSize` taille du texte ;

`taken` zones déjà occupées par d'autres textes ; complété en sortie ;

`fallback` vrai par défaut : sans place libre, le milieu du plus long segment (ajouté lui aussi à `taken`) ; faux : `null`.

Résultat. la place, ancre `middle` ; `null` sans segment, ou sans place libre quand `fallback` est faux.

Erreurs. Aucune.

```
const taken: Area[] = [];
const place = freeAlongPlace(bundle.paths, textLength('24 × ETH', 2.4), 2.4, taken);
```

`endPlace` fonction

```
function endPlace(polyline: readonly Point[]): TextPlace | null
```

Place d'un repère au début d'une polyligne (ROU-10, mode « aux extrémités ») : le long du premier segment non nul, à 0,8 mm du port ; ancre `start`, ou `end` quand le texte a dû être retourné pour ne pas se lire à l'envers. Appelée deux fois par câble (la polyligne et son inverse), une seule fois pour un câble terminé par un renvoi.

Paramètres

`polyline` la polyligne, partant du port.

Résultat. la place, ou `null` sans deuxième point distinct du premier.

Erreurs. Aucune.

`bundlePartners` fonction

```
function bundlePartners(project: Project, linkIds: Iterable<Id>): Id[]
```

Câbles qui peuvent former un faisceau avec ceux donnés (ROU-11) : même type de liaison, mêmes deux équipements, dans un sens ou dans l'autre. Les câbles donnés sont inclus. Utilisée par la commande « Regrouper en faisceau » (`route.group`).

Paramètres

`project` le projet ;

`linkIds` les câbles de départ (la sélection).

Résultat. les identifiants, dans l'ordre de `project.links`.

Erreurs. Aucune ; un identifiant inconnu est ignoré.

22.5 `routing/external`

Sortie d'un câble depuis un port d'une vue d'équipement extérieur (EXT-03, ROU-14) : il descend sous l'équipement sans passer sur un autre port de celui-ci. Pur. Utilisé par `routePage` (terminaux extérieurs).

`segmentCrossesBox` fonction

```
function segmentCrossesBox(a: Point, b: Point, box: Box): boolean
```

Vrai quand un segment horizontal ou vertical traverse l'intérieur d'un rectangle ; toucher son bord est admis (tolérance de 10^{-4} mm). Un rectangle d'épaisseur nulle n'est jamais traversé.

Paramètres

- `a` première extrémité ;
- `b` seconde extrémité (même x ou même y que `a`) ;
- `box` le rectangle.

Résultat. un booléen.

Erreurs. Aucune ; pour un segment oblique, le test porte sur son rectangle englobant.

`dropUnderDevice` fonction

```
function dropUnderDevice(centre: Point, own: Box, others: readonly Box[], device:
Box, bottom: number): Point[]
```

Chemin orthogonal du centre d'un port d'équipement extérieur jusqu'à `bottom`, sans traverser un autre port de l'équipement. Tout droit quand la colonne sous le port est libre ; sinon descente jusqu'à une rangée libre, pas de côté jusqu'à une colonne libre à l'intérieur de l'équipement, puis descente. Les rangées et colonnes candidates longent les autres ports (marges 0,5 puis 0,05 mm) ; le chemin le plus court gagne, un coude comptant pour 2 mm ; à égalité le premier candidat (déterministe).

Paramètres

- `centre` centre du port, mm de page ;
- `own` rectangle du port ;
- `others` rectangles des autres ports de la face ;
- `device` rectangle de l'équipement (les colonnes doivent y rester) ;
- `bottom` ordonnée d'arrivée, sous l'équipement et son nom.

Résultat. la polyligne, du centre du port à `bottom`.

Erreurs. Aucune ; sans chemin libre, la descente droite est rendue.

```
const path = dropUnderDevice(
  { x: 50, y: 10 }, { x: 49, y: 9, width: 2, height: 2 },
  [{ x: 49, y: 14, width: 2, height: 2 }], { x: 40, y: 5, width: 30, height: 15 }, 30,
);
```

22.6 `routing/hidden`

Câbles masqués par une vue (PAG-15) : une vue aux câbles masqués ne dessine aucun câble, mais ses ports restent visibles ; les autres vues gardent leurs câbles et terminent par un renvoi ceux dont l'autre extrémité n'est dessinée que dans une telle vue. Pur, avec un cache d'une entrée. Utilisé par `routePage`.

`hiddenEnds` fonction

```
function hiddenEnds(project: Project, page: Page): ReadonlySet<string>
```

Extrémités de liaisons masquées sur une page : dessinées dans au moins une vue aux câbles masqués et dans aucune vue qui montre ses câbles. Un port coupé par la plage de U d'une vue (PAG-16) n'y est pas dessiné. Le dernier résultat est mémorisé (même projet et même page, comparés par identité).

Paramètres

- `project` le projet ;
- `page` la page.

Résultat. l'ensemble des clés `portKey` des extrémités masquées (vide, partagé, si la page n'a pas de vue masquée). À ne pas modifier.

Erreurs. Aucune.

22.7 `routing/lanes`

Géométrie des couloirs libres de ports d'une vue d'armoire (ROU-14) et ordre des pistes d'une goulotte (ROU-03, ROU-15). Pur : millimètres d'armoire en entrée et en sortie. Un couloir est une bande horizontale qu'aucun port ne touche ; un câble ne court horizontalement que dans un couloir (ou à la hauteur de son propre port quand rien n'est entre lui et la goulotte). Utilisé par `routing/trays` ; `isBevel` l'est aussi par `routing/manual`.

`isBevel` fonction

```
function isBevel(a: { x: number; y: number }, b: { x: number; y: number }): boolean
```

Vrai pour un segment à 45° (biais à côté d'un port, ROU-14), avec une tolérance de 0,02 mm (les tracés stockés sont arrondis au centième) ; un segment de moins de 0,02 mm de large n'en est pas un.

Paramètres

- `a` première extrémité ;
- `b` seconde extrémité.

Résultat. un booléen.

Erreurs. Aucune.

`Rect` interface

```
interface Rect { x0; y0; x1; y1 }
```

Rectangle d'un port, en millimètres d'armoire, par ses bords.

Champs

- `x0` bord gauche ;
- `y0` bord haut ;
- `x1` bord droit ;
- `y1` bord bas.

`Band` interface

```
interface Band { y0; y1 }
```

Bande horizontale libre (couloir), en millimètres d'armoire.

Champs

- `y0` bord haut ;
- `y1` bord bas (`y0 < y1`).

`freeBands` fonction

```
function freeBands(blockers: readonly Rect[], top: number, bottom: number, margin:
number, minHeight = 0.2): Band[]
```

Couloirs entre `top` et `bottom` : complément de l'union des hauteurs des obstacles, chacune élargie de `margin` ; les bandes plus fines que `minHeight` sont abandonnées.

Paramètres

- `blockers` rectangles des ports ;
- `top` haut de la zone, mm d'armoire ;
- `bottom` bas de la zone ;
- `margin` marge gardée au-dessus et au-dessous de chaque port (0,6 mm réels dans `routing/trays`) ;
- `minHeight` hauteur minimale d'un couloir (défaut 0,2 mm).

Résultat. les couloirs, de haut en bas.

Erreurs. Aucune.

```
freeBands([{ x0: 0, y0: 10, x1: 5, y1: 12 }], 0, 30, 0.5);
// [{ y0: 0, y1: 9.5 }, { y0: 12.5, y1: 30 }]
```

`horizontalClear` fonction

```
function horizontalClear(blockers: readonly Rect[], y: number, xa: number, xb:
number, except?: Rect): boolean
```

Vrai quand le segment horizontal de `xa` à `xb` à la hauteur `y` ne traverse l'intérieur d'aucun obstacle (toucher un bord est admis).

Paramètres

- `blockers` rectangles des ports ;
- `y` hauteur du segment ;
- `xa` première abscisse ;
- `xb` seconde abscisse ;
- `except` obstacle ignoré (le port de départ), facultatif.

Résultat. un booléen.

Erreurs. Aucune.

`verticalClear` fonction

```
function verticalClear(blockers: readonly Rect[], x: number, ya: number, yb:
number, except?: Rect): boolean
```

Vrai quand le segment vertical de `ya` à `yb` à l'abscisse `x` ne traverse l'intérieur d'aucun obstacle (toucher un bord est admis).

Paramètres

- `blockers` rectangles des ports ;
- `x` abscisse du segment ;
- `ya` première ordonnée ;
- `yb` seconde ordonnée ;
- `except` obstacle ignoré, facultatif.

Résultat. un booléen.

Erreurs. Aucune.

`Entry` interface

```
interface Entry { y; inside }
```

Arrivée horizontale d'un câble dans la goulotte, pour le calcul de l'ordre des pistes.

Champs

- `y` hauteur de l'arrivée, mm d'armoire ;
- `inside` vrai si elle vient du côté des équipements, faux si elle vient des emplacements 0U (côté extérieur de la goulotte).

`TrayCable` interface

```
interface TrayCable { entries; top; bottom; exits }
```

Un câble (ou un faisceau) d'une goulotte, pour l'ordre des pistes.

Champs

- `entries` ses arrivées dans la goulotte ;
- `top` haut de son étendue verticale dans la goulotte ;
- `bottom` bas de son étendue (une sortie descend jusqu'au socle) ;
- `exits` vrai s'il quitte la vue au socle (sa sortie est placée une fois l'ordre connu).

`pairCrossings` fonction

```
function pairCrossings(inner: TrayCable, outer: TrayCable): number
```

Nombre de croisements entre deux câbles quand `inner` a la piste la plus proche des équipements : les arrivées de `outer` venues des équipements qui tombent dans l'étendue de `inner`, plus les arrivées de `inner` venues des 0U qui tombent dans l'étendue de `outer`. Les sorties ne se croisent jamais (la piste intérieure sort le plus bas).

Paramètres

- `inner` câble de la piste intérieure ;
- `outer` câble de la piste extérieure.

Résultat. un entier ≥ 0.

Erreurs. Aucune.

`orderTracks` fonction

```
function orderTracks<T extends TrayCable>(cables: readonly T[], tieBreak: (p: T, q: T) =>
number): T[]
```

Ordre des pistes, la plus proche des équipements d'abord (ROU-03, ROU-15) : du plus court au plus long (une sortie est infiniment longue ; un câble compris dans un autre ne le croise pas), le sommet le plus bas d'abord à longueur égale, puis `tieBreak` ; ensuite échanges de pistes voisines tant qu'ils retirent des croisements (`pairCrossings`), au plus autant de passes que de câbles.

Paramètres

`cables` les câbles de la goulotte ;

`tieBreak` comparaison finale, qui rend l'ordre déterministe (ROU-05) : numéro de port, repère, identifiant dans `routing/trays`.

Résultat. un nouveau tableau ordonné (l'entrée n'est pas modifiée).

Erreurs. Aucune.

`tuckIn` fonction

```
function tuckIn(x: number, edge: number, lane: number, to: number): { x: number; y: number
}[]
```

Points d'un câble entre le bord de son couloir et sa voie, en direction de la goulotte : vertical, puis biais à 45° qui le range à côté du câble du port voisin (ROU-14). Le biais ne dépasse jamais la moitié du chemin jusqu'à la goulotte.

Paramètres

`x` abscisse du câble (celle du port) ;

`edge` ordonnée du bord du couloir par lequel il entre ;

`lane` ordonnée de sa voie ;

`to` abscisse de sa piste dans la goulotte.

Résultat. deux points : début du biais (à l'abscisse `x`) et fin du biais (sur la voie).

Erreurs. Aucune.

```
tuckIn(10, 0, 3, 100); // [{ x: 10, y: 0 }, { x: 13, y: 3 }]
```

22.8 `routing/manual`

Retouche d'un tracé de câble (ROU-06) : déplacer un segment entier en gardant le tracé orthogonal, déplacer un coude, ajouter ou retirer un point, rattacher un tracé aux ports actuels. Pur : chaque fonction rend une nouvelle polyligne en millimètres de page. Un tracé retouché est orthogonal : les biais à 45° du tracé automatique (ROU-14) deviennent des coudes droits au même endroit. Utilisé par `ui/canvas/Canvas.tsx` (gestes) et `routePage` (`anchorManualRoute`).

`squareUp` fonction

```
function squareUp(points: Point[]): Point[]
```

Ajoute un coin là où deux points consécutifs ne sont pas alignés (un biais à 45° compris) : le coin prolonge le segment précédent (verticalement après un segment vertical ou au départ, horizontalement sinon), si bien qu'un biais à côté d'un port devient un coude droit au même endroit, dans le couloir.

Paramètres

`points` la polyligne.

Résultat. une polyligne orthogonale.

Erreurs. Aucune.

`clean` fonction

```
function clean(points: Point[]): Point[]
```

Retire les points répétés et fusionne les segments alignés (horizontaux ou verticaux). Contrairement à `dedupe`, elle supprime les points intermédiaires alignés ; elle n'est utilisée que par les tests.

Paramètres

`points` la polyligne.

Résultat. la polyligne simplifiée.

Erreurs. Aucune.

`moveSegment` fonction

```
function moveSegment(points: Point[], index: number, offset: number): Point[]
```

Déplace le segment `index` (entre les points `index` et `index + 1`) perpendiculairement à sa direction : un segment horizontal monte ou descend, un vertical va à gauche ou à droite. Les extrémités du tracé ne bougent pas : un coin est ajouté quand le premier ou le dernier segment se déplace.

Paramètres

`points` la polyligne ;

`index` rang du segment ;

`offset` décalage en mm (vers le bas ou la droite s'il est positif).

Résultat. la nouvelle polyligne ; `points` inchangé (même objet) si `index` est hors limites.

Erreurs. Aucune.

```
moveSegment([{ x: 0, y: 0 }, { x: 0, y: 10 }, { x: 20, y: 10 }, { x: 20, y: 20 }], 1, 5);
// [{ x: 0, y: 0 }, { x: 0, y: 15 }, { x: 20, y: 15 }, { x: 20, y: 20 }]
```

`moveVertex` fonction

```
function moveVertex(points: Point[], index: number, to: Point): Point[]
```

Déplace un point intérieur (un coude) ; ses voisins intérieurs suivent pour que les segments adjacents gardent leur direction (le tracé reste orthogonal). Les extrémités ne se déplacent pas.

Paramètres

`points` la polyligne ;

`index` rang du point, ni le premier ni le dernier ;

`to` nouvelle position.

Résultat. la nouvelle polyligne ; `points` inchangé si `index` désigne une extrémité ou est hors limites.

Erreurs. Aucune.

`insertPoint` fonction

```
function insertPoint(points: Point[], index: number, at: Point): Point[]
```

Ajoute un point sur le segment `index`, à la projection de `at` sur lui (exactement sur le biais pour un segment à 45°). Le point est aligné : il ne change pas la forme, mais devient un coude quand on le tire. Double-clic sur un câble.

Paramètres

`points` la polyligne ;

`index` rang du segment (`nearestSegment`) ;

`at` position du pointeur.

Résultat. la nouvelle polyligne ; `points` inchangé si `index` est hors limites.

Erreurs. Aucune.

`removePoint` fonction

```
function removePoint(points: Point[], index: number): Point[]
```

Retire un point intérieur, puis rétablit l'orthogonalité (`squareUp`). Clic droit sur un point du câble sélectionné.

Paramètres

`points` la polyligne ;

`index` rang du point, ni le premier ni le dernier.

Résultat. la nouvelle polyligne ; `points` inchangé pour une extrémité.

Erreurs. Aucune.

`dedupe` fonction

```
function dedupe(points: Point[]): Point[]
```

Retire seulement les points répétés (à 10^{-6} mm près) : les points alignés ajoutés par l'utilisateur restent, car ce sont des coudes qu'il peut tirer.

Paramètres

`points` la polyligne.

Résultat. la polyligne sans doublons consécutifs.

Erreurs. Aucune.

`nearestSegment` fonction

```
function nearestSegment(points: Point[], at: Point): { index: number; distance: number }
```

Segment le plus proche d'un point (distance à sa projection bornée au segment). Sert à choisir où insérer un point au double-clic.

Paramètres

`points` la polyligne ;

`at` le point, mm de page.

Résultat. `{ index, distance }` ; `index` vaut -1 et `distance` `Infinity` pour une polyligne de moins de deux points.

Erreurs. Aucune.

`anchorManualRoute` fonction

```
function anchorManualRoute(manual: Point[], start: Point, end: Point): Point[]
```

Rattache un tracé retouché aux extrémités actuelles du tracé automatique (les ports, qui ont pu bouger) : la chaîne de segments alignés qui part de chaque extrémité (d'après les directions du tracé stocké) suit cette extrémité ; puis l'orthogonalité est rétablie. Appelée par `routePage`.

Paramètres

`manual` le tracé retouché résolu (`manualPolyline`) ;

`start` premier point du tracé automatique ;

`end` dernier point du tracé automatique.

Résultat. le tracé rattaché ; un tracé de moins de deux points devient la liaison orthogonale de `start` à `end`.

Erreurs. Aucune.

22.9 `routing/markers`

Mode repères (ROU-09) : dans une baie dense, un câble est dessiné comme une étiquette de destination à chacun de ses ports (« → SRV01 · eth0 »), repère du câble en option) au lieu d'un trait. La liaison est inchangée. Pur. Utilisé par `routePage`, `routing/trays` et `LinksLayer`.

`MarkerLabel` interface

```
interface MarkerLabel { link; viewId; at; direction; text; fontSize }
```

Étiquette de destination d'un câble à l'un de ses ports.

Champs

- `link` la liaison ;
- `viewId` la vue où le port est dessiné ;
- `at` début du texte, au bord de l'équipement (à 0,4 mm), mm de page ;
- `direction` `up` : le texte monte au-dessus de l'équipement (tourné, lu de bas en haut) ; `down` : il descend au-dessous ;
- `text` « → équipement · port », suivi du repère entre parenthèses si la vue le demande ;
- `fontSize` taille en mm de papier : 0,9 × la largeur du port, entre 0,8 et 2,2.

`usesMarkers` fonction

```
function usesMarkers(project: Project, page: Page, link: Link): boolean
```

Dit si une liaison est dessinée en étiquettes sur cette page : selon `link.display` s'il est forcé (`line` ou `markers`) ; sinon oui si son type de liaison est en mode repères, ou si l'une de ses extrémités est dans une vue en mode repères dont les câbles ne sont pas masqués (PAG-15).

Paramètres

- `project` le projet ;
- `page` la page ;
- `link` la liaison.

Résultat. un booléen.

Erreurs. Aucune.

`markerLabels` fonction

```
function markerLabels(project: Project, page: Page, links: readonly Link[]): MarkerLabel[]
```

Étiquettes de destination des liaisons en mode repères : une par extrémité, par vue où le port est dessiné (sauf vue aux câbles masqués). Le texte part vers le haut pour un port de la moitié haute (ou de la rangée du milieu à numéro d'ordre pair), vers le bas sinon.

Paramètres

- `project` le projet ;
- `page` la page ;
- `links` les liaisons en mode repères (`usesMarkers`).

Résultat. les étiquettes, dans l'ordre des liaisons puis des extrémités.

Erreurs. Aucune.

22.10 `routing/outside`

Routage hors des armoires (ROU-04) : tracés orthogonaux entre terminaux (sorties sous les armoires, ports d'équipements extérieurs), qui contournent les vues de la page, calculés par libavoid. Déterministe à entrées identiques. Utilisé par `routePage` ; `orthogonalize` l'est aussi par `routing/manual`.

`OutsideConnection` interface

```
interface OutsideConnection { id; from; to }
```

Une connexion à tracer hors baie.

Champs

`id` identifiant (celui de la liaison, ou du premier câble d'un faisceau) ;

`from` terminal de départ, mm de page ;

`to` terminal d'arrivée.

`OutsideProblem` interface

```
interface OutsideProblem { obstacles; connections }
```

Problème soumis à libavoid.

Champs

`obstacles` rectangles à contourner (vues, nom des équipements extérieurs, cartouche), mm de page ;

`connections` les connexions.

`SHAPE_BUFFER_MM` constante

```
const SHAPE_BUFFER_MM = 1.5
```

Distance gardée entre un tracé et un obstacle, en mm de papier : 1,5. Sert aussi à écarter les terminaux des armoires (`TERMINAL_GAP` = cette valeur + 1).

`NUDGING_MM` constante

```
const NUDGING_MM = 1
```

Écartement souhaité entre deux tracés parallèles, en mm de papier : 1.

`routeOutside` fonction

```
function routeOutside(avoid: AvoidModule, problem: OutsideProblem): Map<string, Point[]>
```

Trace toutes les connexions en une transaction libavoid (routage orthogonal, marge `SHAPE_BUFFER_MM`, écartement `NUDGING_MM`, pénalité de segment 10, chemins à extrémité commune écartés). Chaque tracé est arrondi au millième, ses extrémités sont remplacées par celles demandées, puis il est rendu orthogonal. Le routeur est toujours détruit.

Paramètres

`avoid` l'instance de libavoid (`loadAvoid`) ;

`problem` obstacles et connexions.

Résultat. table identifiant → polyligne de `from` à `to` ; vide sans connexion.

Erreurs. Les exceptions de libavoid sont propagées.

```
const avoid = await loadAvoid();
const routes = routeOutside(avoid, {
  obstacles: [{ x: 40, y: 0, width: 20, height: 50 }],
  connections: [{ id: 'l1', from: { x: 30, y: 25 }, to: { x: 70, y: 25 } }],
});
```

`orthogonalize` fonction

```
function orthogonalize(points: Point[]): Point[]
```

Insère un coin là où deux points consécutifs ne sont pas alignés (le coin garde l'abscisse du premier et l'ordonnée du second).

Paramètres

`points` la polyligne.

Résultat. une polyligne orthogonale.

Erreurs. Aucune.

22.11 `routing/page`

Dessin des câbles d'une page (ROU-01 à ROU-12) : tronçons dans les vues d'armoires, tracés hors baie entre les terminaux d'une liaison (libavoid, ou lignes droites tant qu'il n'est pas chargé), renvois quand l'autre extrémité n'est pas sur la page, tracés retouchés, faisceaux, repères de câbles et ponts. Pur hormis libavoid. Dépend de tous les autres modules du routage et du modèle (`model/lookup`, `model/pageLayout`, `model/pageGeometry`) ; utilisé par le rendu, l'interface, les commandes et l'export SVG.

`Renvoi` interface

```
interface Renvoi { link; at; text; align? }
```

Renvoi (ROU-08, PAG-15, PAG-16) au bout d'un câble dont l'autre extrémité n'est pas dessinée.

Champs

`link` la liaison ;

`at` début de l'étiquette (bout de l'amorce), mm de page ;

`text` texte : « → équipement · port, page n », « … (non dessiné) » ou « → équipement · port » ;

`align` facultatif : `start` (défaut) le texte part de `at`, `end` il s'y termine (renvoi d'une goulotte gauche au-delà d'une coupure).

`CableDrawing` interface

```
interface CableDrawing { link; paths; polyline; manual; bridges; bundle }
```

Dessin d'un câble sur la page.

Champs

`link` la liaison ;

`paths` ses polylignes, mm de page ; dans un faisceau, seulement ses brins ;

`polyline` le câble entier en une polyligne, d'une extrémité à l'autre (ou à son renvoi) : le tracé que l'on retouche (ROU-06) ; `null` si inconnu ;

`manual` vrai s'il vient d'un tracé retouché de la page ;

`bridges` ses ponts (ROU-12) ;

`bundle` clé de son faisceau (ROU-11), `null` s'il est dessiné seul.

`BundleDrawing` interface

```
interface BundleDrawing { key; links; linkTypeId; paths; bridges; text; label; fontSize }
```

Un faisceau dessiné (ROU-11).

Champs

`key` clé du faisceau (stable) ;

`links` ses câbles ;

`linkTypeId` leur type de liaison ;

`paths` le tronc, mm de page ;

`bridges` ponts du tronc ;

`text` « 24 × ETH » ;

`label` place du texte le long du tronc, `null` si aucune ;

`fontSize` taille du texte, mm de papier (une des `BUNDLE_FONT_SIZES`, la même pour toute la page).

`CableRefLabel` interface

```
interface CableRefLabel { link; text; place }
```

Repère d'un câble (ROU-10), le long du tracé ou à l'une de ses extrémités.

Champs

`link` la liaison ;

`text` son repère (`link.ref`) ;

`place` la place du texte.

`PageRouting` interface

```
interface PageRouting { cables; renvois; markers; bundles; refs }
```

Résultat du routage d'une page.

Champs

`cables` dessins des câbles (types masqués et câbles en mode repères exclus) ;

`renvois` renvois ;

`markers` étiquettes de destination (ROU-09) ;

`bundles` faisceaux ;

`refs` repères des câbles, selon `page.cableRefs`.

`RENVOI_STEP` constante

```
const RENVOI_STEP = 3.2
```

Écart vertical entre deux renvois empilés à la même sortie, en mm de papier : 3,2. L'amorce du premier mesure 4 mm.

`renvoiText` fonction

```
function renvoiText(project: Project, page: Page, end: PortRef): string
```

Texte d'un renvoi (ROU-08) : l'autre extrémité et la première autre page qui montre son équipement (« → SW01 · Gi1, page 2 »), ou « → SW01 · Gi1 (non dessiné) » si aucune.

Paramètres

`project` le projet ;

`page` la page du renvoi (exclue de la recherche) ;

`end` l'extrémité absente de la page.

Résultat. le texte traduit (`routing.renvoiPage` ou `routing.renvoiNone`).

Erreurs. Aucune.

`routePage` fonction

```
function routePage(project: Project, page: Page, avoid: AvoidModule
| null, hiddenLinkTypes: ReadonlySet<Id> = new Set()): PageRouting
```

Câbles d'une page. Étapes : routage des vues d'armoires (`routePageRacks`) ; pour chaque liaison d'un type visible, mise de côté des liaisons en mode repères, tronçons complets et renvois de coupure, terminaux et connexions hors baie ou renvoi ; tracé hors baie (un par faisceau et destination) ; tracés retouchés ; faisceaux et leurs étiquettes ; ponts ; étiquettes de destination ; repères des câbles. Sans libavoid, les tracés hors baie sont des lignes droites.

Paramètres

`project` le projet ;

`page` la page ;

`avoid` l'instance de libavoid, ou `null` ;

`hiddenLinkTypes` types de liaisons masqués à l'écran (défaut : aucun).

Résultat. le `PageRouting` de la page.

Erreurs. Aucune attendue ; une exception de libavoid serait propagée.

```
const routing = routePage(project, page, loadedAvoid(), new Set());
for (const cable of routing.cables) console.log(cable.link.ref, cable.paths.length);
```

`routePageCached` fonction

```
function routePageCached(project: Project, page: Page, avoid: AvoidModule
| null, hiddenLinkTypes: ReadonlySet<Id> = EMPTY): PageRouting
```

`routePage` avec mémorisation du dernier résultat : le rendu, les poignées de retouche, la mise en évidence et les commandes partagent le même calcul tant que le projet, la page, libavoid et l'ensemble des types masqués sont les mêmes objets.

Paramètres

`project` le projet ;

`page` la page ;

`avoid` l'instance de libavoid, ou `null` ;

`hiddenLinkTypes` types masqués ; passer toujours le même objet pour profiter du cache (défaut : un ensemble vide partagé).

Résultat. le routage, éventuellement le même objet qu'à l'appel précédent (ne pas le modifier).

Erreurs. Comme `routePage`.

22.12 `routing/straight`

Tracé provisoire des liaisons du lot 21 : une ligne droite entre deux ports visibles sur la page. Remplacé par le routage des lots 22 et 23 ; seul `portAnchor` sert encore (`ui/canvas/Canvas.tsx`, tirer une liaison). Pur.

`LinkDrawing` interface

```
interface LinkDrawing { link; points }
```

Ligne droite d'une liaison.

Champs

`link` la liaison ;

`points` polyligne en mm de page, de l'extrémité A à l'extrémité B.

`portAnchor` fonction

```
function portAnchor(project: Project, page: Page, ref: PortRef): Point | undefined
```

Centre d'un port sur la page, dans la première vue qui le montre. Sert de point de départ et d'arrivée du trait provisoire quand on tire une liaison à la souris.

Paramètres

`project` le projet ;

`page` la page ;

`ref` le port (`deviceId`, `portId`).

Résultat. le centre, mm de page ; `undefined` si le port n'est pas sur la page.

Erreurs. Aucune.

`straightLinks` fonction

```
function straightLinks(project: Project, page: Page, hiddenLinkTypes: ReadonlySet<Id> = new Set()): LinkDrawing[]
```

Lignes droites des liaisons dont les deux extrémités sont visibles sur la page, types masqués exclus. N'est plus appelée que par ses tests.

Paramètres

`project` le projet ;

`page` la page ;

`hiddenLinkTypes` types masqués (défaut : aucun).

Résultat. les dessins, dans l'ordre de `project.links`.

Erreurs. Aucune.

22.13 `routing/trays`

Routage dans une vue d'armoire (ROU-01 à ROU-03, ROU-05, ROU-14, ROU-15, PAG-15, PAG-16) : chaque câble quitte son port par un couloir libre de ports (`routing/lanes`), se range par un biais à 45°, rejoint sa piste dans la goulotte de son côté, la parcourt, puis rejoint l'autre port de la vue, sort de l'armoire par le côté au niveau du socle, ou s'arrête au-delà d'une coupure de plage avec un renvoi. Deux câbles ne partagent ni piste ni voie, sauf les câbles d'un faisceau (une piste). Pur, déterministe, mis en cache par vue. Dépend du modèle (`model/lookup`, `model/pageGeometry`, `model/rackGeometry`, `model/pageLayout`) et de `library/defaultTypes` ; utilisé par `routing/page` et par l'interface des goulottes.

`defaultTraySide` fonction

```
function defaultTraySide(linkTypeId: Id): LateralSide
```

Goulotte par défaut d'un type de liaison : électrique à gauche, tout le reste à droite (point ouvert n° 2 du cahier des charges, lot 22). Affichée comme valeur « Auto » dans la fenêtre des goulottes et les propriétés d'une armoire.

Paramètres

`linkTypeId` identifiant du type de liaison.

Résultat. `'left'` pour le type électrique par défaut, `'right'` sinon.

Erreurs. Aucune.

`traySide` fonction

```
function traySide(project: Project, rack: Rack, link: Link): LateralSide
```

Côté de la goulotte d'une liaison dans une armoire (ROU-02), vu depuis la face montrée : forcé sur la liaison, sinon réglage de l'armoire, sinon réglage du projet, sinon `defaultTraySide`.

Paramètres

`project` le projet (réglage `routing.trayByLinkType`) ;

`rack` l'armoire (`trayByLinkType`) ;

`link` la liaison (`tray`).

Résultat. `'left'` ou `'right'`.

Erreurs. Aucune.

`MIN_TRACK_SPACING_PAPER` constante

```
const MIN_TRACK_SPACING_PAPER = 0.9
```

Écartement minimal entre deux pistes, en mm de papier : 0,9 (lisibilité). Divisé par l'échelle de la vue pour passer en mm d'armoire ; ramené à 0 quand des équipements 0U à ports sont du côté de la goulotte (les pistes se resserrent au lieu de déborder sur eux).

`MAX_TRACK_SPACING_MM` constante

```
const MAX_TRACK_SPACING_MM = 8
```

Écartement maximal entre deux pistes, en mm réels : 8. L'écartement vaut `largeur de la goulotte / (n + 1)` borné par ces deux valeurs.

`Leg` interface

```
interface Leg { link; viewId; side; track; points; complete; exit?; from?; bundle?; cutTo?; renvoiAlign? }
```

Tronçon d'un câble dans une vue d'armoire.

Champs

- `link` la liaison ;
- `viewId` la vue ;
- `side` côté de la goulotte ;
- `track` rang de la piste, 0 = la plus proche des équipements ;
- `points` polyligne en mm de page ;
- `complete` vrai si les deux extrémités sont dans la vue, ou si l'autre est coupée par la plage (rien d'autre n'est alors tracé depuis cette vue) ; faux pour une sortie ;
- `exit` facultatif : bout d'un tronçon incomplet, au côté de l'armoire au niveau du socle (mm de page) ;
- `from` facultatif : extrémité de la liaison d'où part un tronçon incomplet ;
- `bundle` facultatif : clé du faisceau (ROU-11), partagée par les tronçons qui ont la même piste ;
- `cutTo` facultatif : autre extrémité, dans la même armoire mais coupée par la plage de U (PAG-16) ; le tronçon suit la goulotte jusqu'à la coupure et finit par un renvoi ;
- `renvoiAlign` facultatif : côté du bout où va le texte de ce renvoi (vers l'extérieur de l'armoire).

`CUT_RENVOI_STUB` constante

```
const CUT_RENVOI_STUB = 4
```

Longueur de l'amorce au-delà d'une coupure de plage de U, en mm de papier : 4.

`CUT_RENVOI_STEP` constante

```
const CUT_RENVOI_STEP = 3.2
```

Écart entre deux renvois étagés au-delà d'une coupure, en mm de papier : 3,2 ; la piste la plus extérieure s'arrête au plus près de la coupure.

`TrayUsage` interface

```
interface TrayUsage { tracks; widened }
```

Occupation d'une goulotte.

Champs

- `tracks` abscisses des pistes, en mm d'armoire, de l'intérieur vers l'extérieur ;

`widened` vrai si les pistes, à l'écartement minimal, dépassent la largeur de la goulotte (elles sont dessinées au-delà).

`RackViewRouting` interface

```
interface RackViewRouting { viewId; legs; trays }
```

Routage d'une vue d'armoire.

Champs

`viewId` la vue ;

`legs` un tronçon par câble tracé dans la vue ;

`trays` occupation des goulottes `left` et `right`.

`simplify` fonction

```
function simplify(points: Point[]): Point[]
```

Retire les points répétés et fusionne les segments alignés (droits ou à 45°), sans fusionner un aller-retour sur la même droite. Appliquée à chaque tronçon et à la polyligne complète d'un câble.

Paramètres

`points` la polyligne.

Résultat. la polyligne simplifiée.

Erreurs. Aucune.

```
simplify([{ x: 0, y: 0 }, { x: 0, y: 5 }, { x: 0, y: 10 }, { x: 4, y: 10 }]);
// [{ x: 0, y: 0 }, { x: 0, y: 10 }, { x: 4, y: 10 }]
```

`routeRackView` fonction

```
function routeRackView(project: Project, page: Page, view: View): RackViewRouting
```

Routage d'une vue d'armoire, mis en cache par identifiant de vue jusqu'à ce qu'une de ses entrées change d'identité (armoire, vue, échelle effective, réglages de routage du projet, types de liaison de la bibliothèque, tracés retouchés et état des repères des vues quand la vue a ses faisceaux, équipements de l'armoire, liaisons qui les touchent, leurs modèles ; 500 vues au plus). Une vue extérieure, sans armoire, ou aux câbles masqués rend un résultat vide.

Paramètres

`project` le projet ;

`page` la page de la vue ;

`view` la vue d'armoire.

Résultat. le `RackViewRouting` de la vue (objet partagé par le cache, à ne pas modifier).

Erreurs. Aucune.

`routePageRacks` fonction

```
function routePageRacks(project: Project, page: Page): RackViewRouting[]
```

Routage de chaque vue d'armoire d'une page (`routeRackView` sur toutes les vues non extérieures, dans l'ordre de la page). Premier pas de `routePage`.

Paramètres

`project` le projet ;

`page` la page.

Résultat. un routage par vue d'armoire.

Erreurs. Aucune.

Chapitre 23

Référence : texte

Ce chapitre décrit chaque déclaration exportée de `src/text/` : police embarquée, mesure, découpage en lignes, lecture et allègement des polices TrueType. La conception est expliquée au chapitre « Texte : polices, mesure et découpage ».

23.1 `text/fontFiles`

Fichiers de la police embarquée (NF-07b), tous servis par l'application : les sous-ensembles WOFF2 d'Inter de l'écran (un fichier par écriture et par graisse, paquet `@fontsource/inter`) et Inter en TrueType (même version, mêmes métriques, paquet `inter-ttf`), pour le PDF et pour les symboles absents des sous-ensembles. Dépend du build (URL fournies par Vite, `import.meta.glob`) ; utilisé par `main.tsx`, le banc de rendu et les exports (`export/appFonts.ts`, `export/pdfExport.ts`).

`WebFontFile` interface

```
interface WebFontFile { subset; weight; ranges; unicodeRange; url }
```

Un fichier WOFF2 de l'écran.

Champs

- `subset` nom de l'écriture (`latin`, `latin-ext`, `cyrillic`…) ;
- `weight` graisse ;
- `ranges` plages couvertes, analysées ;
- `unicodeRange` les mêmes, en valeur CSS `unicode-range` ;
- `url` URL du fichier dans l'application.

`WEB_FONT_FILES` constante

```
const WEB_FONT_FILES: WebFontFile[]
```

Liste des fichiers WOFF2 de l'écran, construite au chargement du module à partir des fichiers du paquet et de sa table `unicode.json` ; un fichier sans plage connue est ignoré. Sert à l'export SVG pour embarquer les mêmes fichiers que l'écran (`APP_FONT_SOURCES`).

`trueTypeUrl` fonction

```
function trueTypeUrl(weight: FontWeight): string
```

URL du fichier TrueType d'une graisse d'Inter. Utilisée par l'export PDF (police complète) et par l'export SVG et l'écran pour les symboles.

Paramètres

`weight` la graisse.

Résultat. l'URL dans l'application.

Erreurs. Aucune.

`SYMBOLS_UNICODE_RANGE` constante

```
const SYMBOLS_UNICODE_RANGE = …
```

Plages, en valeur CSS, des symboles qu'Inter dessine mais que les sous-ensembles de l'écran n'ont pas : exposants et indices, formes de nombres, flèches (« → » des renvois), opérateurs mathématiques, signes techniques, cercles numérotés, filets et formes, symboles divers, dingbats. Ces caractères sont pris dans le fichier TrueType.

`SYMBOLS_RANGES` constante

```
const SYMBOLS_RANGES = parseUnicodeRange(SYMBOLS_UNICODE_RANGE)
```

`SYMBOLS_UNICODE_RANGE` analysée (`parseUnicodeRange`), pour tester la couverture d'un caractère dans l'export SVG.

`SYMBOLS_SAMPLE` constante

```
const SYMBOLS_SAMPLE = '→'
```

Échantillon de caractères chargés avant la première mesure : « → ». Passé à `waitForFonts` pour que les faces des symboles soient prêtes au premier rendu.

`registerSymbolFaces` fonction

```
function registerSymbolFaces(fontFaces: FontFaceSet = document.fonts)
```

Ajoute au document, pour chaque graisse, une face `Inter` dont la source est le fichier TrueType restreint à `SYMBOLS_UNICODE_RANGE` ; le navigateur ne la charge qu'au premier besoin. Appelée au démarrage, avant `waitForFonts`.

Paramètres

`fontFaces` l'ensemble de polices (défaut `document.fonts`).

Résultat. aucun.

Erreurs. Aucune en pratique (exige `FontFace`, donc un navigateur).

`fontBytes` fonction

```
function fontBytes(url: string): Promise<Uint8Array>
```

Octets d'un fichier de police de l'application, obtenus par `fetch` une seule fois par URL (la promesse est gardée en mémoire). Utilisée par les exports SVG et PDF.

Paramètres

`url` URL du fichier (`trueTypeUrl` ou d'un `WebFontFile`).

Résultat. une promesse des octets.

Erreurs. Promesse rejetée par `Error('Font not found: …')` si la réponse n'est pas correcte, ou par l'erreur réseau ; la promesse rejetée reste en cache.

23.2 `text/fonts`

La police unique de l'application (NF-07b) : Inter, embarquée, utilisée à l'écran et dans les exports ; ses graisses, ses métriques verticales et l'attente de son chargement. Pur sauf `waitForFonts`. Utilisé par tout le rendu de texte, la mesure et les exports.

`FONT_FAMILY` constante

```
const FONT_FAMILY = 'Inter'
```

Nom de famille de la police : `Inter`. Utilisé dans les attributs `font-family` du SVG, les règles `@font-face` des exports et l'enregistrement de la police dans le PDF.

`FONT_WEIGHTS` constante

```
const FONT_WEIGHTS = [400, 500, 600, 700] as const
```

Graisses embarquées : 400, 500, 600 et 700 (tableau constant). Les feuilles de `main.tsx` et les fichiers TrueType de `text/fontFiles` correspondent à cette liste.

`FontWeight` type

```
type FontWeight = (typeof FONT_WEIGHTS)[number];
```

Union des graisses de `FONT_WEIGHTS` : `400 | 500 | 600 | 700`.

`FontSpec` interface

```
interface FontSpec { size; weight? }
```

Police d'un texte à mesurer ou à mettre en page.

Champs

`size` taille en unités de l'utilisateur : pixels à l'écran, millimètres dans le dessin ;

`weight` graisse, facultative (400 par défaut).

`FONT_ASCENT` constante

```
const FONT_ASCENT = 1984 / 2048
```

Ascendante d'Inter en proportion de la taille : $1984/2048$ (`unitsPerEm` 2048). Sert à placer la ligne de base dans `layoutText`.

`FONT_DESCENT` constante

```
const FONT_DESCENT = 494 / 2048
```

Descendante d'Inter en proportion de la taille : $494/2048$.

`cssFont` fonction

```
function cssFont(font: FontSpec): string
```

Forme abrégée CSS d'une police, pour la mesure au canvas : « `600 12px Inter` ».

Paramètres

`font` la police (graisse 400 par défaut).

Résultat. la chaîne CSS.

Erreurs. Aucune.

`waitForFonts` fonction

```
async function waitForFonts(fontFaces: FontFaceSet = document.fonts, sample = ' '):
Promise<void>
```

Attend que chaque graisse embarquée d'Inter soit chargée, puis que l'ensemble de polices soit prêt : un texte ne doit jamais être mesuré avec une police de secours (NF-07b). Appelée au démarrage de l'application et du banc de rendu.

Paramètres

`fontFaces` l'ensemble de polices (défaut `document.fonts`) ;

`sample` caractères dont les faces doivent aussi être chargées (défaut : une espace ; l'application passe aussi `SYMBOLS_SAMPLE`).

Résultat. une promesse résolue quand tout est chargé.

Erreurs. Promesse rejetée si le chargement d'une face échoue.

```
registerSymbolFaces();
await waitForFonts(document.fonts, ` ${SYMBOLS_SAMPLE}`);
const measureText = createCanvasMeasurer();
```

23.3 `text/index`

Point d'entrée du dossier : réexporte `text/fonts`, `text/measure` et `text/layout` (police, mesure, découpage). Les modules propres aux exports (`text/fontFiles`, `text/sfnt`, `text/unicodeRange`) n'y sont pas : on les importe directement.

23.4 `text/layout`

Mise en page du texte (NF-07) : découpage à l'avance en lignes, pour que l'écran et les exports PDF/SVG montrent exactement les mêmes lignes. Fonction pure, qui reçoit la mesure en paramètre. Utilisé par `TextBlock`, `AnnotationsLayer` et les étiquettes de câbles.

`TextAlign` type

```
type TextAlign = 'start' | 'middle' | 'end';
```

Alignement horizontal d'un bloc : `start` (à gauche), `middle` (centré), `end` (à droite) ; mêmes valeurs que l'attribut SVG `text-anchor`.

`LayoutOptions` interface

```
interface LayoutOptions { font; maxWidth; maxLines?; lineHeight?; align? }
```

Options de `layoutText`.

Champs

`font` police (taille et graisse) ;

`maxWidth` largeur maximale d'une ligne, dans l'unité de la taille ; `Infinity` : pas de retour à la ligne automatique ;

`maxLines` nombre maximal de lignes, facultatif ; la dernière se termine par « … » s'il reste du texte ;

`lineHeight` distance entre lignes de base, facultative (défaut 1,2 × la taille) ;

`align` alignement, facultatif (défaut `start`).

`LaidOutLine` interface

```
interface LaidOutLine { text; width; x; y }
```

Une ligne mise en page.

Champs

`text` son texte ;

`width` sa largeur mesurée ;

`x` abscisse d'ancrage, depuis le bord gauche du bloc (0, milieu ou bord droit selon l'alignement) ;

`y` ordonnée de la ligne de base, depuis le haut du bloc.

`TextLayout` interface

```
interface TextLayout { lines; width; height; truncated }
```

Résultat de `layoutText`.

Champs

`lines` les lignes ;

`width` largeur de la plus longue ligne ;

`height` hauteur du bloc : nombre de lignes × interligne ;

`truncated` vrai si du texte a été coupé pour respecter `maxLines`.

`ELLIPSIS` constante

```
const ELLIPSIS = '…'
```

Points de suspension ajoutés à la dernière ligne d'un texte coupé : « … » (un seul caractère).

`layoutText` fonction

```
function layoutText(text: string, options: LayoutOptions, measure: MeasureText): TextLayout
```

Découpe un texte en lignes : fins de ligne normalisées et blancs de bord retirés ; paragraphes aux retours à la ligne ; coupure aux blancs, puis caractère par caractère (points de code) pour un mot plus large que la ligne ; au-delà de `maxLines`, la dernière ligne reçoit la suite du texte et finit par « … » ; ligne de base de la première ligne centrée dans l'interligne d'après les métriques d'Inter.

Paramètres

`text` le texte, avec d'éventuels retours à la ligne ;

`options` police, largeur, nombre de lignes, interligne, alignement (`LayoutOptions`) ;

`measure` la fonction de mesure (`MeasureText`).

Résultat. la mise en page ; aucune ligne pour un texte vide ou `maxLines < 1`.

Erreurs. Aucune.

```
const layout = layoutText(
  'Commutateur 48 ports',
  { font: { size: 2.5, weight: 600 }, maxWidth: 20, maxLines: 2, align: 'middle' },
  measureText,
);
for (const line of layout.lines) console.log(line.text, line.x, line.y);
```

23.5 `text/measure`

Mesure du texte par `canvas.measureText` avec la police embarquée (NF-07), avec un cache. Les fonctions de mise en page reçoivent une `MeasureText`, si bien qu'elles se testent sans navigateur. Utilisé par `main.tsx` (création) et, par le type, par le rendu et les exports.

`MeasureText` type

```
type MeasureText = (text: string, font: FontSpec) => number;
```

Type d'une fonction de mesure : largeur d'un texte dans une police, dans la même unité que la taille de la police. L'application en fournit une par `TextMeasureContext` ; les tests passent une mesure factice (par exemple 0,5 × la taille par caractère).

`createCanvasMeasurer` fonction

```
function createCanvasMeasurer(maxEntries = 20_000): MeasureText
```

Crée la fonction de mesure de l'application : `OffscreenCanvas` (ou un `<canvas>` du document), mesure à 100 px puis mise à l'échelle (précise pour les petites tailles en millimètres), cache par graisse et texte vidé d'un coup quand il est plein. Appelée une fois au démarrage, après `waitForFonts`.

Paramètres

`maxEntries` taille maximale du cache (défaut 20 000).

Résultat. une `MeasureText`.

Erreurs. `Error('Canvas 2D context unavailable')` si le contexte 2D ne peut être obtenu.

23.6 `text/sfnt`

Lecture minimale et allègement des polices TrueType (contours `glyf`), pour n'embarquer que les glyphes utilisés par un export (EXP-02). Numéros de glyphes, métriques et table des caractères sont conservés : seuls les contours inutiles sont vidés, si bien que le sous-ensemble mesure exactement comme la police complète. Pur. Utilisé par `export/fonts.ts`.

`Sfnt` interface

```
interface Sfnt { bytes; view; tables }
```

Police lue par `readSfnt`.

Champs

`bytes` les octets du fichier ;

`view` une `DataView` sur ces octets ;

`tables` répertoire des tables : étiquette → `{ tag, offset, length }`.

`readSfnt` fonction

```
function readSfnt(bytes: Uint8Array): Sfnt
```

Lit l'en-tête et le répertoire des tables d'une police TrueType.

Paramètres

`bytes` les octets du fichier.

Résultat. la police lue (les octets ne sont pas copiés).

Erreurs. `Error('Not a TrueType font')` si la signature n'est pas `0x00010000` (police CFF, collection, WOFF…).

`glyphIndex` fonction

```
function glyphIndex(font: Sfnt, codePoint: number): number
```

Glyphe d'un point de code, d'après la table `cmap` : sous-table de format 12 (Windows, Unicode complet) si elle existe, sinon format 4 (Windows, plan de base).

Paramètres

`font` la police lue ;

`codePoint` le point de code.

Résultat. le numéro de glyphe, 0 si le caractère est absent.

Erreurs. `Error('Missing table cmap')` sans table `cmap`.

`SubsetOptions` interface

```
interface SubsetOptions { dropLayout? }
```

Options de `subsetFont`.

Champs

`dropLayout` facultatif : retire les tables de substitution et de positionnement (`GSUB`, `GPOS`, `GDEF`, `STAT`, `DSIG`, `kern`), pour une police qui ne dessine que des symboles isolés.

`subsetFont` fonction

```
function subsetFont(bytes: Uint8Array, codePoints: Iterable<number>, options: SubsetOptions = {}): Uint8Array
```

Copie de la police qui ne garde que les contours des glyphes des points de code demandés, de leurs composants (glyphes composites, suivis récursivement) et du glyphe 0. Reconstruit `glyf` et une `loca` longue, met `post` en version 3, réécrit le répertoire, les sommes de contrôle et l'ajustement de `head`.

Paramètres

`bytes` la police complète ;

`codePoints` les caractères à garder ;

`options` options (`SubsetOptions`).

Résultat. les octets de la nouvelle police.

Erreurs. Celles de `readSfnt` ; `Error('Missing table …')` s'il manque `cmap`, `maxp`, `head`, `loca`, `glyf` ou `post`.

```
const full = await fontBytes(trueTypeUrl(500));
const subset = subsetFont(full, codePoints('→↑'), { dropLayout: true });
```

`outlinedGlyphs` fonction

```
function outlinedGlyphs(bytes: Uint8Array): number
```

Nombre de glyphes qui ont un contour (longueur non nulle dans `loca`). Sert aux tests et au diagnostic d'un sous-ensemble.

Paramètres

`bytes` la police.

Résultat. un entier.

Erreurs. Comme `readSfnt` ; table manquante.

`advanceWidth` fonction

```
function advanceWidth(bytes: Uint8Array, glyph: number): number
```

Avance d'un glyphe, en unités de la police, lue dans `hmtx` (au-delà du nombre de métriques longues, la dernière vaut).

Paramètres

`bytes` la police ;

`glyph` le numéro de glyphe.

Résultat. l'avance, en unités (`unitsPerEm` 2048 pour Inter).

Erreurs. Comme `readSfnt` ; table `hhea` ou `hmtx` manquante.

23.7 `text/unicodeRange`

Valeurs CSS `unicode-range` (« `U+0000-00FF,U+0131` ») : analyse et couverture, pour choisir les fichiers de police qu'un export doit embarquer. Pur. Utilisé par `text/fontFiles` et `export/fonts.ts`.

`CodeRange` type

```
type CodeRange = readonly [start: number, end: number];
```

Intervalle de points de code, bornes comprises : `[début, fin]` (n-uplet en lecture seule).

`parseUnicodeRange` fonction

```
function parseUnicodeRange(value: string): CodeRange[]
```

Analyse une valeur `unicode-range` : éléments séparés par des virgules, préfixe `U+` facultatif (casse indifférente), point de code seul ou intervalle `début-fin` en hexadécimal.

Paramètres

`value` la valeur CSS.

Résultat. les intervalles, dans l'ordre ; un point seul donne `[a, a]`.

Erreurs. Aucune exception ; une valeur mal formée donne des bornes `NaN` (qui ne couvrent rien). Les jokers (`U+4??`) ne sont pas gérés.

```
parseUnicodeRange('U+0000-00FF, U+0131'); // [[0, 255], [305, 305]]
```

`coversCodePoint` fonction

```
function coversCodePoint(ranges: readonly CodeRange[], codePoint: number): boolean
```

Vrai si l'un des intervalles contient le point de code.

Paramètres

`ranges` les intervalles ;

`codePoint` le point de code.

Résultat. un booléen.

Erreurs. Aucune.

`codePoints` fonction

```
function codePoints(text: string): number[]
```

Points de code d'un texte, paires de substitution réunies (un caractère hors du plan de base donne un seul point, par exemple `0x1F600`).

Paramètres

`text` le texte.

Résultat. les points de code, dans l'ordre.

Erreurs. Aucune.

Chapitre 24

Référence : rendu SVG

Ce chapitre documente les modules de `src/render/` : les composants React qui écrivent le SVG d'une page (à l'écran et dans les exports, NF-07) et les fonctions pures qui les accompagnent (formes des ports, logos). La conception d'ensemble, les repères et le niveau de détail sont expliqués au chapitre « Rendu SVG ». Sauf mention contraire, les coordonnées sont dans l'unité du repère où le composant est monté : millimètres de papier sur une page, millimètres réels dans une armoire.

24.1 `render/AnnotationsLayer`

Dessin des éléments de l'onglet « Schéma » d'une page (PAG-07) : textes, formes, lignes et flèches, bulles, notes, tableaux, zones, images. Dépend de `model/annotations` (géométrie pure : chemins, pointes, rotation, colonnes), de `text/layout` et de `TextBlock` pour les textes, de `useImageUrl` pour les images. Monté deux fois par `PageContent` (sous les vues et au-dessus), donc présent à l'écran et dans les exports.

`AnnotationsLayer` constante

```
const AnnotationsLayer = …
```

Composant mémoïsé qui dessine, dans un groupe `annotations-<couche>`, les éléments de la page dont `layer` vaut la couche demandée, dans l'ordre de la page ; il ne rend rien s'il n'y en a aucun. Chaque élément est un groupe tourné autour de son centre (`rotationTransform`). Les textes sont découpés à l'avance, limités au nombre de lignes qui tiennent dans la hauteur (coupure par « … ») et centrés verticalement dans une bulle ou une cellule de tableau.

Champs

`page` la page dont on dessine les éléments ;

`layer` `'below'` (sous les vues) ou `'above'` (au-dessus) ;

`interactive` vrai à l'écran : chaque élément porte `data-annotation-id` et une zone de sélection transparente (trait élargi pour une ligne ; cadre et bandeau de titre seulement pour une zone, afin que les vues qu'elle entoure restent accessibles) ; une image non choisie est remplacée par un cadre en tirets et un texte d'attente. Faux à l'export : rien de tout cela n'est écrit.

24.2 `render/DeviceFace`

Dessin d'une face de machine : photo, dessin d'un extérieur ou dessin par défaut (MAC-01, MAC-02, EDM-03, EDM-10), avec le logo, la machine posée sur une étagère (MAC-03), le libellé, les ports (PRT-01) et le contour d'une pose forcée (EQP-02). Dépend de `render/StandardDecoration`, `render/ExternalDrawing`, `render/logo`, `render/PortShape`, `render/portShapes` et `render/images`. Utilisé par `DeviceView`, les vues extérieures de `render/PageView`, l'éditeur de machine, l'aperçu de pose et le panneau de bibliothèque.

`isLight` fonction

```
function isLight(hex: string): boolean
```

Indique si une couleur de fond est claire, pour choisir la couleur du libellé d'une face (foncé sur fond clair, blanc sur fond sombre). La luminance relative est calculée avec les coefficients 0,2126 / 0,7152 / 0,0722 sur les canaux 0–255, sans correction gamma, et comparée à 150.

Paramètres

`hex` couleur au format `#rrggbb`.

Résultat. `true` si la luminance dépasse 150.

Erreurs. Aucune exception ; une chaîne mal formée donne `NaN` pour les canaux et donc `false`.

`FORCED_COLOR` constante

```
const FORCED_COLOR = '#f08c00'
```

Couleur orange (`#f08c00`) du contour épais (2,5 unités) dessiné autour d'un équipement placé malgré un dépassement de profondeur (EQP-02, `depthForced`). Utilisée par `DeviceFace` seulement.

`DeviceFaceProps` interface

```
interface DeviceFaceProps { model; face; box; looks; portDetail?; forced?; extraPorts? }
```

Propriétés de `DeviceFace`.

Champs

`model` la machine dessinée ;

`face` la face de la machine à dessiner (`'front'` ou `'rear'`), qui n'est pas forcément la face de l'armoire affichée ;

`box` rectangle de la face dans le repère courant (mm réels dans une armoire, mm de papier dans une vue extérieure, pixels dans un aperçu) ;

`looks` dessin de chaque type de connecteur, par identifiant (`usePortLooks`) ;

`portDetail` niveau de détail des ports (défaut `'full'`) ; `'none'` seulement quand l'utilisateur masque les ports d'une vue extérieure ;

`forced` facultatif : dessine le contour orange de pose forcée ;

`extraPorts` facultatif : ports ajoutés à l'équipement posé sur cette face (PRT-04), dessinés après ceux de la machine.

`upperRowPorts` fonction

```
function upperRowPorts(ports: readonly Port[]): ReadonlySet<Id>
```

Trouve les ports de la rangée du haut d'une paire de rangées, pour les dessiner retournés (ergot en haut) comme sur les vrais commutateurs. Les ports sont regroupés par colonne (même groupe et même abscisse arrondie au 1/2000) ; une colonne d'exactement deux ports dont l'écart vertical atteint 60 % de la somme de leurs hauteurs donne son port du haut. Appelée par `DeviceFace` pour les commutateurs, routeurs et pare-feu en dessin par défaut et au niveau de détail `full`.

Paramètres

`ports` les ports d'une face (positions en proportions de la face).

Résultat. L'ensemble des identifiants des ports du haut.

Erreurs. Aucune.

```
const flipped = upperRowPorts(model.front.ports);
const upsideDown = flipped.has(port.id);
```

`DeviceFace` constante

```
const DeviceFace = …
```

Composant mémoïsé qui dessine une face de machine dans `box`. Ses propriétés sont décrites par `DeviceFaceProps`. Il écrit un groupe `device-face` avec, dans l'ordre : soit la photo de la face (mode `image` et image disponible, `preserveAspectRatio="xMidYMid meet"`) ; soit, pour un extérieur qui a un dessin maison, `ExternalDrawing` et son logo ; soit le dessin par défaut : fond de la couleur de la face (sauf étagère), décor (`StandardDecoration`), logo sur sa plaque claire, image de la machine posée sur l'étagère, libellé. Le libellé (libellé de la face, sinon nom de la machine sauf équipement « nommé par ses paramètres ») est écrit sur une ligne dans la zone de `labelZone`, en taille `min(0,32 × hauteur, 12)`, s'il y a au moins `min(30, 2,5 × taille)` de place. Puis chaque port dans un groupe `port-slot` portant `data-port-id`, et enfin le contour `FORCED_COLOR`.

24.3 `render/ExternalDrawing`

Dessins faits par Portolan pour les équipements extérieurs génériques : écran, pupitre tactile, poste de travail, portable, clavier et souris. Ils évitent de stocker une image dans chaque nouveau projet et ne montrent aucune marque (CG-08, décision du 04/11/2026, lot 11). Toutes les tailles sont des proportions de la boîte. Utilisé par `DeviceFace` ; les zones de logo correspondantes sont dans `render/logo`.

`hasExternalDrawing` fonction

```
function hasExternalDrawing(kind: EquipmentKind): boolean
```

Indique si un type d'équipement a un dessin maison. `DeviceFace` l'interroge pour un équipement extérieur sans photo ; un extérieur sans dessin maison reçoit le dessin par défaut.

Paramètres

`kind` le type d'équipement de la machine.

Résultat. `true` pour `monitor`, `controlPanel`, `workstation`, `laptop` et `keyboardMouse`.

Erreurs. Aucune.

`ExternalDrawing` constante

```
const ExternalDrawing = …
```

Composant mémoïsé qui dessine l'extérieur de type `kind` dans le rectangle `box`, dans un groupe `external-<type>` ; il ne rend rien pour un type sans dessin.

Champs

`kind` type d'équipement ;

`box` rectangle de la face, dans le repère courant.

24.4 `render/LabelSheet`

Planche d'étiquettes de câbles (EXP-07) en millimètres de papier. Le même composant sert à l'aperçu de la fenêtre d'export (`ExportLabelsDialog`) et au PDF (`export/labelsExport`). Dépend de `export/labels` (formats de papier, marge intérieure, découpage du texte d'une étiquette par `fitLabel`) et de `useMeasureText`.

`LabelSheet` fonction

```
function LabelSheet({ labels, template, outlines, pixelsPerMm, }: { labels: readonly
PlacedLabel[]; template: LabelTemplate; outlines: boolean; /** Screen size; absent = size
in millimetres (export). */ pixelsPerMm?: number; })
```

Dessine une planche : un `<svg>` de la taille du papier du gabarit, fond blanc, contours de découpe facultatifs (gris clair, coins de 1 mm) et, pour chaque étiquette placée, ses lignes de texte (groupe `cable-label` portant l'identifiant de la liaison et l'extrémité). Chaque ligne est un `TextBlock` sans retour à la ligne, de la police choisie par `fitLabel`.

Paramètres

`labels` étiquettes placées sur cette planche (coin haut gauche en mm) ;

`template` gabarit d'étiquettes (papier, taille d'une étiquette) ;

`outlines` dessine les contours, pour découper du papier ordinaire ;

`pixelsPerMm` facultatif : taille à l'écran ; absent, le `<svg>` est dimensionné en millimètres (export).

Résultat. L'élément `<svg>` de la planche.

Erreurs. Lève l'erreur de `useMeasureText` si aucun mesureur de texte n'est fourni.

24.5 `render/LinksLayer`

Couche des câbles d'une page (LIA-01, ROU-01 à ROU-12) : tracés, troncs de faisceaux, libellés, références, étiquettes du mode repères et renvois. Le calcul vient de `routing/page` (`routePageCached`) et le module libavoid de `routing/avoid`. Monté par `PageContent` ; `useAvoid` est aussi utilisé par les surlignages du plan de travail et `RENVOI_FONT` par l'export SVG.

`dashArray` fonction

```
function dashArray(style: LinkType['lineStyle'], width: number): string | undefined
```

Motif de tirets d'un style de trait de type de liaison, proportionnel à l'épaisseur pour garder le même aspect quel que soit le trait. Utilisé pour les câbles et les troncs de faisceaux.

Paramètres

`style` `'solid'`, `'dashed'` ou `'dotted'` ;

`width` épaisseur du trait.

Résultat. `"4w 2,5w"` pour les tirets, `"0,1w 2w"` pour les pointillés (à extrémités rondes), `undefined` pour un trait plein.

Erreurs. Aucune.

```
dashArray('dashed', 0.5); // '2 1.25'
dashArray('solid', 0.5);  // undefined
```

`useAvoid` fonction

```
function useAvoid(): AvoidModule | null
```

Crochet React qui rend le module libavoid (ROU-04) une fois chargé, `null` avant. Il part de `loadedAvoid` (déjà chargé ailleurs) et lance sinon `loadAvoid` ; l'échec du chargement est ignoré (les câbles restent en tracé de repli). Utilisé par `LinksLayer` et par les surlignages du plan de travail.

Résultat. Le module libavoid ou `null`.

Erreurs. Aucune : une erreur de chargement est absorbée.

`RENVOI_FONT` constante

```
const RENVOI_FONT = { size: 2.2, weight: 500 as const }
```

Police d'un renvoi « → équipement · port » (ROU-08) : 2,2 mm de papier, graisse 500. La largeur d'un renvoi est limitée à 80 mm. L'export SVG la lit pour inclure les renvois dans l'emprise d'une vue exportée seule.

Champs

`size` taille en mm de papier ;

`weight` graisse d'Inter.

`LinksLayer` constante

```
const LinksLayer = …
```

Composant mémoïsé qui dessine les câbles de la page dans un groupe `links` dont l'attribut `data-routed` vaut `true` une fois libavoid chargé. Ordre : câbles (avec leurs ponts, ROU-12), troncs de faisceaux (ROU-11), libellés de faisceaux, puis, si `labels`, références des câbles, étiquettes du mode repères (ROU-09, halo blanc par copie tracée dessous) et renvois. La couleur, l'épaisseur et le style viennent du type de liaison (gris `#495057` et 0,5 mm si le type manque).

Champs

`project` le projet (routage et types de liaisons) ;

`page` la page ;

`hiddenLinkTypes` facultatif : types de liaisons masqués (LIA-10), exclus par le routage ;

`labels` renvois et étiquettes de destination (EXP-05, « repères de câbles »), vrai par défaut ;

`interactive` traits transparents élargis (`link-hit`) pour le pointeur, écran seulement, vrai par défaut.

24.6 `render/PageView`

Dessin d'une page (PAG-02 à PAG-17) : feuille, cadre, grille d'export, vues d'armoires et vues extérieures, câbles, éléments du schéma, tableau des révisions, légende et cartouche, en millimètres de papier. C'est le point commun de l'écran et des exports (NF-07). Dépend de `model/pageLayout`, `model/pageGeometry`, `model/rackGeometry`, du magasin de l'éditeur et des autres modules du dossier. Utilisé par le plan de travail (`Canvas`), les exports SVG, PNG et PDF, leurs fenêtres et le banc de rendu.

`VIEW_TITLE` constante

```
const VIEW_TITLE = { size: 7, color: '#1f5f8b' }
```

Style du titre d'une vue d'armoire (« Vue avant », « Vue arrière » ou titre choisi) : 7 mm de papier, bleu `#1f5f8b`, centré au-dessus de l'armoire. Lu aussi par l'export SVG pour l'emprise d'une vue.

Champs

`size` taille en mm de papier ;

`color` couleur du texte.

`LABEL` constante

```
const LABEL = { size: 3, color: '#1f5f8b', width: SIDE_LABELS.width }
```

Style des étiquettes d'équipements à côté d'une armoire (PAG-06) et des libellés de vues extérieures (taille + 0,5 mm) : 3 mm de papier, bleu `#1f5f8b`, largeur `SIDE_LABELS.width` (70 mm). Lu aussi par l'export SVG.

Champs

`size` taille en mm de papier ;

`color` couleur ;

`width` largeur maximale d'une étiquette, en mm.

`PageViewProps` interface

```
interface PageViewProps { pageId; hiddenLinkTypes?; pixelsPerMm; portDetail?; pageNumber; pageCount }
```

Propriétés de `PageView` (et, sauf `pageId`, de `PageContent`).

Champs

`pageId` identifiant de la page ;

`hiddenLinkTypes` facultatif : types de liaisons non dessinés (LIA-10) ;

`pixelsPerMm` pixels d'écran par millimètre de papier (zoom) : taille du `<svg>` et niveau de détail par défaut ;

`portDetail` facultatif : niveau de détail imposé (calculé par le plan de travail selon la taille de sa fenêtre) ; défaut `portDetailAt(pixelsPerMm)` ;

`pageNumber` , `pageCount` : numéro de la page (à partir de 1) et nombre de pages, pour le cartouche.

`PageView` constante

```
const PageView = …
```

Composant mémoïsé qui lit la page `pageId` et le projet dans le magasin de l'éditeur et rend `PageContent` avec les réglages de l'écran. Il ne rend rien si la page n'existe pas. Propriétés : `PageViewProps`. Monté par `PageCanvas`.

`PageParts` interface

```
interface PageParts { background; frame; titleBlock; legend; cableLabels; grid }
```

Parties d'une page qu'un export peut retirer (EXP-05) ; chacune est un booléen.

Champs

`background` feuille blanche (faux : PNG transparent) ;

`frame` cadre de la feuille ;

`titleBlock` cartouche et tableau des révisions ;

`legend` légende, dans le cartouche ou en bloc libre ;

`cableLabels` renvois, références et étiquettes de destination des câbles ;

`grid` grille de référence (export seulement).

`SCREEN_PARTS` constante

```
const SCREEN_PARTS: PageParts
```

Parties dessinées à l'écran : toutes, sauf la grille. C'est aussi la valeur initiale des options des fenêtres d'export d'image et de PDF.

`GRID_STEP` constante

```
const GRID_STEP = 10
```

Pas de la grille de référence d'export : 10 mm de papier, un trait plus foncé et plus épais toutes les cinq lignes (50 mm). La grille couvre la zone utile de la page.

`PageContentProps` interface

```
interface PageContentProps extends Omit<PageViewProps, 'pageId'> { page; project; parts?;
sizeInMm?; crop?; interactive? }
```

Propriétés de `PageContent` : celles de `PageViewProps` sans `pageId`, plus :

Champs

`page` la page à dessiner ;

`project` le projet ;

`parts` parties dessinées (défaut `SCREEN_PARTS`) ;

`sizeInMm` taille du `<svg>` en millimètres (export) au lieu de pixels ;

`crop` partie de la page dessinée, en mm de page (export d'une vue seule) ; devient la `viewBox` ;

`interactive` zones de clic et emplacements d'attente (écran), vrai par défaut.

`PageContent` fonction

```
function PageContent({ page, project, pixelsPerMm, portDetail:
portDetailProp, pageNumber, pageCount, hiddenLinkTypes, parts = SCREEN_PARTS, sizeInMm =
false, crop, interactive = true, }: PageContentProps)
```

Dessine une page complète dans un `<svg>` dont la `viewBox` est en millimètres de papier (la page, ou `crop`). Ordre : fond, cadre (à mi-marge), grille, éléments du schéma du dessous, vues (échelle effective `effectiveScale` ; vue d'armoire avec titre et étiquettes latérales, ou vue extérieure avec son libellé), câbles, éléments du dessus, tableau des révisions (en bas à droite de la zone utile, ou à sa position enregistrée), légende libre, cartouche. Sans cartouche, une légende prévue dans le cartouche est dessinée en bloc libre à sa place. Le niveau de détail est `portDetail`, sinon `portDetailAt(pixelsPerMm)`. C'est une fonction (non mémoïsée) pour pouvoir être rendue hors du document par l'export.

Paramètres

`props` `PageContentProps`.

Résultat. L'élément `<svg class="page">` portant `data-page-id`.

Erreurs. Lève une erreur si le contexte de mesure du texte ou celui de l'éditeur manque.

```
const markup = renderToStaticMarkup(
  <EditorContext.Provider value={editor}>
    <TextMeasureContext.Provider value={measure}>
      <PageContent page={page} project={project} pixelsPerMm={100}
        pageNumber={1} pageCount={1} sizeInMm interactive={false} />
    </TextMeasureContext.Provider>
  </EditorContext.Provider>,
);
```

`PageCanvas` constante

```
const PageCanvas = …
```

Composant mémoïsé monté par le plan de travail : il ne monte que la page active (NF-02b), avec son numéro et le nombre de pages, et ne rend rien sans page active ou si elle n'existe plus.

Champs

`activePageId` identifiant de la page active, ou `null` ;

`pixelsPerMm` zoom ;

`portDetail` facultatif : niveau de détail calculé selon la fenêtre ;

`hiddenLinkTypes` facultatif : types de liaisons masqués.

24.7 `render/PortShape`

Dessins des ports (MAC-02) : un dessin par forme, défini à la taille naturelle du connecteur réel (`SHAPE_SIZES`), en couleurs contrastées lisibles en noir et blanc (métal clair, ouvertures

noires, contacts clairs), et le port simplifié du très fort dézoom (NF-02b). Dépend de `render/portShapes`. Utilisé par `DeviceFace` ; la scène `connectors` du banc montre tous les dessins.

`PORT_COLORS` constante

```
const PORT_COLORS: Record<ConnectorFamily, string>
```

Couleur de remplissage d'un port simplifié, par famille de connecteur : gris clair pour le cuivre, bleu-gris pour l'optique, beige pour la vidéo, vert pâle pour l'USB, brique pâle pour la série, mauve pour l'audio, presque noir pour l'alimentation, gris pour « autre ».

`SHAPE_DRAWINGS` constante

```
const SHAPE_DRAWINGS: Record<PortShapeKey, ReactElement>
```

Dessin SVG de chaque forme de port (`PortShapeKey`), en millimètres du connecteur, origine en haut à gauche de sa taille naturelle. Les dessins sont des éléments React créés une fois au chargement du module et réutilisés par tous les ports. Les chemins répétés (contacts, broches) sont regroupés dans un seul `<path>`. Ajouter une forme demande une entrée ici, dans `PORT_SHAPES` et dans `SHAPE_SIZES`.

`PortShapeProps` interface

```
interface PortShapeProps { look; x; y; width; height; detail; flipped? }
```

Propriétés de `PortShape`.

Champs

- `look` famille et forme du port (`PortLook`) ;
- `x` , `y`, `width`, `height` : rectangle du port, dans le repère courant ;
- `detail` `'full'` (dessin du connecteur) ou `'simple'` (rectangle de la couleur de la famille) ;
- `flipped` facultatif : dessin retourné verticalement (rangée du haut d'un commutateur, ergot en haut).

`PortShape` constante

```
const PortShape = …
```

Composant mémoïsé qui dessine un port. Au niveau `simple`, un rectangle de la couleur de `PORT_COLORS` (filet proportionnel à la taille). Au niveau `full`, un groupe `port port-<famille> port-shape-<forme>` contenant un rectangle invisible couvrant tout le port (`port-hit`, zone de survol et de tirage d'une liaison) et le dessin de la forme placé par `fitShape` (`translate` puis `scale`, avec une échelle verticale négative s'il est retourné).

24.8 `render/RackContents`

Équipements d'une armoire vus depuis une face, chacun mémoïsé et abonné à sa seule entité (NF-02b). Dépend de `model/pageGeometry` (`deviceInRackFace`), `model/lookup` (`instancePorts`), des sélecteurs du magasin et de `DeviceFace`. Monté dans `RackView` par les vues de page et l'aperçu de pose ; `usePortLooks` sert aussi à l'éditeur de machine et à la bibliothèque.

`DeviceView` constante

```
const DeviceView = …
```

Composant mémoïsé qui dessine un équipement posé : il lit l'équipement et sa machine par leur identifiant (modifier un autre équipement ne le redessine pas), obtient la boîte et la face du modèle vue depuis la face affichée (`deviceInRackFace`) et rend `DeviceFace` dans un groupe `device` (ou `device device-zero-u`) portant `data-device-id`. Il ne rend rien si l'équipement ou sa machine manque ou s'il n'est pas visible depuis cette face. Le contour de pose forcée est demandé pour un équipement de baie `depthForced`.

Champs

`deviceId` identifiant de l'équipement ;

`geometry` géométrie de l'armoire pour la face affichée ;

`viewFace` face de l'armoire affichée (transmise, non utilisée : la géométrie porte déjà la face) ;

`looks` dessins des types de connecteurs ;

`portDetail` niveau de détail des ports.

`usePortLooks` fonction

```
function usePortLooks(): ReadonlyMap<Id, PortLook>
```

Crochet React qui rend le dessin de chaque type de connecteur du projet (`portLooks`), recalculé seulement quand la liste des types de la bibliothèque change.

Résultat. Table identifiant de type → `PortLook`.

Erreurs. Lève une erreur hors d'un fournisseur du magasin de l'éditeur.

`rackDrawingOrder` fonction

```
function rackDrawingOrder(devices: readonly Device[], rackId: Id, face: FaceSide): Id[]
```

Ordre de dessin des équipements d'une armoire pour une face (EQP-09) : d'abord les équipements de baie fixés sur l'autre face (vus de dos), puis les autres (fixés sur cette face, 0U), chacun dans l'ordre du projet. Le dernier dessiné recouvre les précédents : un commutateur retourné derrière un panneau de brassage est caché par ce dernier. Utilisé par `RackContents` dans un sélecteur à comparaison superficielle.

Paramètres

`devices` les équipements du projet ;

`rackId` l'armoire ;

`face` face de l'armoire dessinée.

Résultat. Les identifiants des équipements de l'armoire (extérieurs exclus), dans l'ordre de dessin.

Erreurs. Aucune.

```
const ids = rackDrawingOrder(project.devices, rack.id, 'front');
```

`RackContentsProps` interface

```
interface RackContentsProps { rackId; geometry; face; portDetail? }
```

Propriétés de `RackContents`.

Champs

- `rackId` l'armoire ;
- `geometry` sa géométrie pour la face (`rackGeometry`) ;
- `face` face dessinée ;
- `portDetail` facultatif : niveau de détail (défaut `'full'`).

`RackContents` constante

```
const RackContents = …
```

Composant mémoïsé qui dessine, dans un groupe `rack-contents`, un `DeviceView` par équipement de l'armoire, dans l'ordre de `rackDrawingOrder`. Il ne se redessine que si la liste des identifiants change ; les modifications d'un équipement ne redessinent que sa vue.

24.9 `render/RackView`

Dessin d'une armoire vue de face ou de dos (ARM-04), en millimètres réels, à partir de `model/rackGeometry` : cadre selon le type, emplacements 0U, numérotation des U, montants à trous EIA-310, roulettes ou pieds, coupure d'une plage de U (PAG-16). Utilisé par les vues d'armoires de `render/PageView`, l'aperçu de pose et le banc.

`RackViewProps` interface

```
interface RackViewProps { rack; face; uRange?; clipId?; children? }
```

Propriétés de `RackView`.

Champs

- `rack` l'armoire ;
- `face` face dessinée ;
- `uRange` facultatif : plage de U affichée (PAG-16) ; `null` ou absente, l'armoire entière ;
- `clipId` facultatif : clé unique du dessin dans le document, utilisée pour les identifiants des `clipPath` d'une plage (défaut : identifiant de l'armoire ; la page passe l'identifiant de la vue, car une armoire peut y figurer deux fois) ;
- `children` contenu dessiné dans la zone des U, au-dessus des montants (les équipements).

`RackView` constante

```
const RackView = …
```

Composant mémoïsé qui dessine l'armoire dans un groupe `rack rack-<type>` portant `data-rack-id` et `data-face`. Armoire fermée ou murale : panneaux latéraux, toit et socle (teinte assombrie). Baie ouverte : montants, traverses. Baie à deux montants : traverse haute et pied élargi, sans fond. Puis emplacements 0U, numérotation (`uNumber`, selon le sens de numérotation), montants, `children`, et roulettes ou pieds (aucun pour une armoire murale). Sur une plage de U, le groupe est décalé de `window.top`, le cadre est découpé selon un contour en zigzag, le contenu coupé net aux limites, toit, socle et roulettes omis du côté coupé, et les traits de coupure ajoutés (classe `rack-range`).

24.10 `render/StandardDecoration`

Décor « réaliste sobre » du dessin par défaut des équipements (MAC-02, décision du 05/11/2026, lot 36) : relief, oreilles et vis, aérations, voyants et parties propres à chaque type. Toutes les tailles sont en millimètres de l'équipement réel multipliés par `k` (`faceScale`), pour un dessin identique quelle que soit l'échelle. Pas de `<defs>` ni de dégradé (export PDF). Dépend de `model/rackGeometry`, `model/equipment` et `render/logo`. Utilisé par `DeviceFace`.

`faceScale` fonction

```
function faceScale(model: Machine, box: Box): number
```

Unités de dessin par millimètre de l'équipement réel, pour une face dessinée dans `box`. Calculée par la hauteur pour un équipement de baie (`heightU` × 44,45 mm), par la longueur pour un équipement 0U, sinon par la largeur (`displaySizeMm` d'un extérieur, à défaut une façade 19″ de 482,6 mm).

Paramètres

`model` la machine ;

`box` rectangle de la face.

Résultat. Le facteur `k` (strictement positif pour une boîte non vide).

Erreurs. Aucune ; une boîte de taille nulle donne 0.

```
// Commutateur 1U dessiné en mm réels : k = 44.45 / 44.45 = 1
const k = faceScale(switch1u, { x: 0, y: 0, width: 482.6, height: 44.45 });
```

`labelZone` fonction

```
function labelZone(model: Machine, face: FaceSide, box: Box, extraPorts: readonly
Pick<Port, 'x' | 'w'>[] = []): { x: number; right: number; logo: LogoLayout | null }
```

Zone horizontale du libellé d'une face : après l'oreille et une marge de 4 mm, jusqu'au premier port ou à la limite du décor propre au type (29 % de la largeur pour un serveur ou un stockage, 40 % pour un onduleur, 48 % pour un KVM, en face avant), moins 4 mm. En face avant d'un équipement de baie qui a un logo, le logo prend le début de la zone (`rackLogoLayout`) et le libellé commence 3 mm après sa plaque. Utilisée par `DeviceFace` (libellé, logo) et par les voyants du décor.

Paramètres

`model` la machine ;

`face` la face ;

`box` rectangle de la face ;

`extraPorts` ports ajoutés à l'équipement posé sur cette face (PRT-04), dont le libellé s'arrête avant, comme pour ceux de la machine (aucun par défaut).

Résultat. `{ x, right, logo }` : début et fin de la zone du libellé, en unités de dessin, et géométrie du logo ou `null`. `right` peut être inférieur à `x` quand il n'y a pas de place.

Erreurs. Aucune.

`StandardDecorationProps` interface

```
interface StandardDecorationProps { model; face; box }
```

Propriétés de `StandardDecoration`.

Champs

`model` la machine ;

`face` la face dessinée ;

`box` rectangle de la face.

`StandardDecoration` constante

```
const StandardDecoration = …
```

Composant mémoïsé qui dessine, dans un groupe `std std-<type>`, au-dessus du fond et sous les ports : le relief (sauf étagère et PDU verticale), les oreilles avec une vis par U (face avant des équipements de baie qui en ont), puis les parties du type : plaques des blocs de ports, voyants, aérations, blocs d'alimentation (ports nommés `PSU<n>`), baies de disques, bouton d'alimentation, afficheur d'onduleur, boutons de KVM, keystones numérotés, cassettes, perforations, anneaux ou brosse, plateau d'étagère, interrupteur et parafoudre d'une PDU. Le tableau du chapitre « Rendu SVG » résume les parties par type.

`transparentBackground` fonction

```
function transparentBackground(model: Machine): boolean
```

Indique si la face d'une machine n'a pas de fond : seule l'étagère, pour montrer ce qu'elle porte. `DeviceFace` omet alors le rectangle de fond.

Paramètres

`model` la machine.

Résultat. `true` pour une étagère.

Erreurs. Aucune.

`labelledByParams` fonction

```
function labelledByParams(model: Machine): boolean
```

Indique si la machine n'affiche que le libellé de sa face, sans se rabattre sur son nom : équipements passifs ou génériques dont le nom décrit les paramètres (obturateur, passe-câbles, étagère, PDU, panneau de brassage, tiroir optique, générique).

Paramètres

`model` la machine.

Résultat. `true` pour ces types d'équipements.

Erreurs. Aucune.

24.11 `render/TextBlock`

Le seul composant qui écrit du texte dans le dessin : texte découpé à l'avance en lignes `<tspan>` positionnées (NF-07), jamais de `foreignObject` ni de retour à la ligne automatique.

Dépend de `text/layout` (`layoutText`), `text/fonts` et `useMeasureText`. Utilisé par presque tous les composants du dossier.

`TextBlockProps` interface

```
interface TextBlockProps { text; x; y; width; font; maxLines?; lineHeight?; align?; fill?; className? }
```

Propriétés de `TextBlock`.

Champs

- `text` le texte (les espaces multiples sont réduits par la mise en page) ;
- `x` , `y` : coin haut gauche du bloc ;
- `width` largeur maximale d'une ligne ; `Infinity` : pas de retour à la ligne ;
- `font` police : taille dans l'unité du repère, graisse (400 par défaut) ;
- `maxLines` facultatif : nombre maximal de lignes, la dernière se terminant par « … » si le texte est plus long ;
- `lineHeight` facultatif : hauteur de ligne (défaut de `layoutText`) ;
- `align` `'start'`, `'middle'` ou `'end'` (défaut `'start'`) ;
- `fill` couleur (noir par défaut) ;
- `className` classe CSS facultative.

`TextBlock` constante

```
const TextBlock = …
```

Composant mémoïsé qui met le texte en page avec le mesureur du contexte et écrit un `<text>` en police Inter, `xml:space="preserve"`, avec un `<tspan>` par ligne aux positions calculées et l'ancre de l'alignement. Ne rend rien pour un texte vide. La mise en page est recalculée seulement si le texte, la largeur, la police, le nombre de lignes, la hauteur de ligne, l'alignement ou le mesureur changent.

24.12 `render/TextMeasureContext`

Contexte React qui fournit la fonction de mesure du texte au dessin. L'application y place le mesureur sur canvas (Inter chargée, `main.tsx`), le banc de rendu le sien ; les exports et les fenêtres d'aperçu le reprennent.

`TextMeasureContext` constante

```
const TextMeasureContext = createContext<MeasureText | null>(null)
```

Contexte contenant la fonction `MeasureText` (largeur d'un texte dans une police), `null` par défaut. Tout arbre qui monte un composant du dessin doit le fournir.

`useMeasureText` fonction

```
function useMeasureText(): MeasureText
```

Crochet React qui rend la fonction de mesure du contexte.

Résultat. La fonction `MeasureText`.

Erreurs. Lève `Error('TextMeasureContext is missing')` si aucun fournisseur n'est monté : l'oubli est visible immédiatement plutôt qu'un texte mal découpé.

24.13 `render/TitleBlock`

Cartouche (PAG-09, PAG-10), légende (PAG-11) et tableau des révisions (PAG-14), en millimètres de papier. La place des champs vient de `model/titleBlockLayout`, les libellés de `fr.json`, les dates de `formatDate`. Utilisé par `PageContent` et par la fenêtre des propriétés du projet (aperçu du cartouche, libellés par défaut).

`TitleBlockProps` interface

```
interface TitleBlockProps { project; pageTitle; pageNumber; pageCount; x; y; width; height; legend }
```

Propriétés de `TitleBlock`.

Champs

- `project` le projet (champs, métadonnées, révisions, logo) ;
- `pageTitle` titre de la page ;
- `pageNumber` , `pageCount` : pour « page n sur N » ;
- `x` , `y`, `width`, `height` : rectangle du cartouche sur la page ;
- `legend` types de liaisons de la légende dans le cartouche, ou `null` quand la légende est un bloc libre (ou absente).

`titleBlockFieldLabel` fonction

```
function titleBlockFieldLabel(field: Pick<TitleBlockField, 'kind' | 'label'>, translate: (key: string) => string): string
```

Libellé d'un champ du cartouche : le sien s'il n'est pas vide (ou si le champ est personnalisé), sinon le libellé par défaut de sa nature (PAG-10). Utilisée par le cartouche et comme texte d'attente du champ « libellé » dans la fenêtre du projet.

Paramètres

- `field` nature et libellé du champ ;
- `translate` fonction de traduction (clés `render.titleBlock.*`).

Résultat. Le libellé à afficher.

Erreurs. Aucune.

```
titleBlockFieldLabel({ kind: 'author', label: '' }, t); // libellé par défaut
```

`TitleBlock` constante

```
const TitleBlock = …
```

Composant mémoïsé qui dessine le cadre blanc du cartouche, puis chaque champ placé par `titleBlockLayout` : le logo du projet (image ajustée, cadre vide sans logo ; aucun logo par défaut, CG-08) ou une cellule libellé + valeur, la valeur sur autant de lignes que la hauteur le permet. Valeurs : nom du projet, client, titre de la page, indice et date de la dernière révision, auteur, « page n sur N », valeur saisie d'un champ personnalisé. La légende est dessinée dans sa cellule si elle est demandée.

`RevisionTableProps` interface

```
interface RevisionTableProps { project; rows; right; bottom; width }
```

Propriétés de `RevisionTable`.

Champs

`project` le projet (révisions) ;

`rows` nombre de révisions montrées (les dernières) ;

`right` , `bottom` : coin inférieur droit du tableau, en mm de papier ;

`width` largeur du tableau, en mm.

`RevisionTable` constante

```
const RevisionTable = …
```

Composant mémoïsé qui dessine le tableau des révisions (PAG-14) : une ligne d'en-tête (indice, date, auteur, description, 10/20/22/48 % de la largeur) puis les `rows` dernières révisions, la plus récente en bas, en lignes de 4,2 mm. Ne rend rien sans révision.

`LegendBlock` constante

```
const LegendBlock = …
```

Composant mémoïsé qui dessine la légende en bloc libre (PAG-11), de 50 mm de large : un titre puis, pour chaque type de liaison, un échantillon de trait (couleur, épaisseur d'au moins 0,6 mm, tirets ou pointillés) et son nom.

Champs

`linkTypes` types de liaisons à montrer (`legendLinkTypes`) ;

`x` , `y` : coin haut gauche, en mm de papier.

24.14 `render/images`

URL des images du projet pour le dessin (photos des faces, logos, images posées sur une étagère, logo du cartouche, images du schéma). Les composants ne savent pas d'où vient l'URL : l'application fournit des URL d'objet, l'export SVG des URL `data:`. Dépend de `io/images` (type MIME d'une référence).

`ImageUrlResolver` type

```
type ImageUrlResolver = (ref: ImageRef) => string | undefined;
```

Fonction qui rend l'URL d'une image du projet à partir de sa référence, ou `undefined` si l'image n'est pas disponible.

`createImageUrlResolver` fonction

```
function createImageUrlResolver(getImages: () => ImageMap): ImageUrlResolver
```

Crée le résolveur de l'application : à la première demande d'une image, une URL d'objet (`blob:`) est créée à partir de ses octets et de son type MIME, puis gardée pour les demandes suivantes (les images sont immuables : même référence, même contenu). Quand la table des images change d'identité (autre projet ouvert, image retirée), les URL des images qui n'y sont plus sont révoquées : leur contenu ne reste pas en mémoire toute la session. Appelée une fois par `main.tsx` ; la fenêtre d'import de modèles fournit son propre résolveur pour les images de la source importée.

Paramètres

`getImages` fonction qui rend la table actuelle des images (lue à chaque demande).

Résultat. Un `ImageUrlResolver`.

Erreurs. Aucune ; une image absente donne `undefined`.

```
const resolve = createImageUrlResolver(() => editor.getState().images);
const url = resolve(imageRef); // 'blob:…' ou undefined
```

`ImageUrlContext` constante

```
const ImageUrlContext = createContext<ImageUrlResolver>(() => undefined)
```

Contexte React contenant le résolveur ; par défaut, un résolveur qui ne trouve aucune image (les images ne sont alors pas dessinées, sans erreur).

`useImageUrl` fonction

```
function useImageUrl(ref: ImageRef | null): string | undefined
```

Crochet React qui résout une référence d'image avec le résolveur du contexte.

Paramètres

`ref` la référence, ou `null`.

Résultat. L'URL, ou `undefined` si `ref` est `null` ou l'image absente.

Erreurs. Aucune.

24.15 `render/logo`

Géométrie pure (sans React) du logo d'une machine sur le dessin par défaut de sa face avant (EDM-10) et de l'image d'une machine posée sur une étagère (MAC-03). Tailles en unités de dessin, `k` unités par millimètre de l'équipement réel. Testé dans `logo.test.tsx`. Utilisé par `render/StandardDecoration` et `DeviceFace`.

`LogoLayout` interface

```
interface LogoLayout { plate; image }
```

Géométrie d'un logo.

Champs

`plate` rectangle de la plaque claire derrière le logo ;

`image` rectangle de l'image, aux proportions du logo, à l'intérieur de la plaque.

`LOGO_MAX_HEIGHT_MM` constante

```
const LOGO_MAX_HEIGHT_MM = 16
```

Hauteur maximale de la plaque d'un logo sur un équipement de baie : 16 mm (tient sur une façade 1U de 44,45 mm avec le libellé et les voyants).

`LOGO_MAX_WIDTH_MM` constante

```
const LOGO_MAX_WIDTH_MM = 70
```

Largeur maximale de la plaque d'un logo sur un équipement de baie : 70 mm.

`LOGO_MAX_ZONE_SHARE` constante

```
const LOGO_MAX_ZONE_SHARE = 0.45
```

Part maximale de la zone du libellé que le logo peut prendre : 0,45 ; le libellé et les ports gardent le reste.

`fitLogo` fonction

```
function fitLogo(area: Box, aspect: number, pad: number, align: 'start' | 'center' =
'start'): LogoLayout
```

Ajuste un logo de proportions `aspect` dans une zone, avec une marge intérieure : aussi haut que la zone le permet, moins haut s'il serait trop large ; aligné à gauche ou centré horizontalement, toujours centré verticalement.

Paramètres

`area` zone disponible ;

`aspect` proportions du logo (largeur / hauteur, strictement positives) ;

`pad` marge entre la plaque et l'image ;

`align` `'start'` (défaut) ou `'center'`.

Résultat. Un `LogoLayout` ; dimensions nulles si la zone est plus petite que les marges.

Erreurs. Aucune.

```
const layout = fitLogo({ x: 0, y: 0, width: 40, height: 10 }, 2, 1);
// image 16 x 8, plaque 18 x 10 en (0, 0)
```

`rackLogoLayout` fonction

```
function rackLogoLayout(zone: { x: number; right: number }, box: Box, k: number, aspect:
number): LogoLayout | null
```

Logo d'un équipement de baie : au début de la zone du libellé (après l'oreille), centré verticalement, d'au plus la moitié de la hauteur de la face et `LOGO_MAX_HEIGHT_MM` mm, au plus `LOGO_MAX_WIDTH_MM` mm et `LOGO_MAX_ZONE_SHARE` de la zone en largeur, marge `min(1,5 mm, 12 % de la hauteur)`. Appelée par `labelZone`.

Paramètres

`zone` début `x` et fin `right` de la zone du libellé ;

`box` rectangle de la face ;

`k` unités de dessin par mm ;

`aspect` proportions du logo.

Résultat. La géométrie, ou `null` s'il n'y a pas de place ou si l'image ferait moins de 3 mm de large (ou 1,8 mm de haut).

Erreurs. Aucune.

`externalLogoLayout` fonction

```
function externalLogoLayout(kind: EquipmentKind, box: Box, aspect: number): LogoLayout
| null
```

Logo d'un équipement extérieur dessiné par Portolan, centré à sa place : sur le cadre sous l'écran (écran, pupitre, portable), sous les aérations (poste de travail), au-dessus des touches (clavier). La marge vaut 12 % de la hauteur de la zone.

Paramètres

`kind` type d'équipement ;

`box` rectangle de la face ;

`aspect` proportions du logo.

Résultat. La géométrie, ou `null` pour un type sans place de logo.

Erreurs. Aucune.

`carriedLayout` fonction

```
function carriedLayout(box: Box, k: number, aspect: number): Box | null
```

Rectangle de l'image d'une machine posée sur une étagère (MAC-03), sur la face avant de l'étagère : debout sur le plateau (au plus 6 mm, 18 % de la face), entre les deux équerres de 10 mm, avec 1 mm de jeu, aussi grande que la place le permet dans ses proportions, centrée. Les tailles correspondent au dessin de l'étagère dans `render/StandardDecoration`.

Paramètres

`box` rectangle de la face de l'étagère ;

`k` unités de dessin par mm ;

`aspect` proportions de l'image.

Résultat. Le rectangle, ou `null` si l'image ferait moins de 3 mm de côté.

Erreurs. Aucune.

24.16 `render/portShapes`

Formes des ports et niveau de détail (MAC-02, NF-02b), en fonctions pures sans React : liste des formes, résolution d'un type de connecteur en dessin, tailles naturelles, ajustement dans le rectangle d'un port, seuil de simplification. Dépend de `library/defaultTypes` (`CONNECTOR_SHAPES`). Utilisé par `render/PortShape`, `render/RackContents`, `render/DeviceFace`, `render/PageView` et le plan de travail (`portDetailAt`).

`PORT_SHAPES` constante

```
const PORT_SHAPES = …
```

Liste des 31 formes de ports, une par connecteur réel : RJ45, RJ11, SFP, QSFP, LC, SC, ST, MPO, HDMI, DisplayPort, DVI, VGA, D-sub, USB A, B et C, BNC, F coaxial, jack, XLR, C13, C14, C19, C20, C5, C6, C7, prise à terre latérale, type E, P17, et `generic`. L'ordre est celui des listes de l'interface.

`PortShapeKey` type

```
type PortShapeKey = (typeof PORT_SHAPES)[number];
```

Union des clés de `PORT_SHAPES` : nom d'une forme de port.

`PortLook` interface

```
interface PortLook { family; shape }
```

Dessin des ports d'un type de connecteur.

Champs

`family` famille du connecteur (cuivre, optique, vidéo, USB, série, audio, alimentation, autre), qui donne la couleur du port simplifié ;

`shape` forme dessinée au niveau de détail `full`.

`portLook` fonction

```
function portLook(type: Pick<ConnectorType, 'id' | 'family' | 'shape'>): PortLook
```

Résout le dessin d'un type de connecteur (décision du 05/11/2026, lot 36) : la forme actuelle d'un type par défaut (par son identifiant fixe, pour que les anciens projets suivent un changement de dessin) ; sinon la forme du type si elle est connue, sauf `generic` hors de la famille « autre » ; sinon un ancien nom traduit selon la famille (`round` → ST, BNC ou jack) ; sinon le dessin de la famille. Un nouveau type de connecteur (forme `generic`) se dessine donc selon sa famille.

Paramètres

`type` identifiant, famille et forme du type de connecteur.

Résultat. Un `PortLook`.

Erreurs. Aucune : tout nom inconnu retombe sur le dessin de la famille.

```
portLook({ id: newId(), family: 'optical', shape: 'generic' });
// { family: 'optical', shape: 'sfp' }
```

`portLooks` fonction

```
function portLooks(types: readonly Pick<ConnectorType, 'id' | 'family' | 'shape'>[]):
ReadonlyMap<Id, PortLook>
```

Applique `portLook` à une liste de types de connecteurs.

Paramètres

`types` les types (identifiant, famille, forme).

Résultat. Table identifiant → `PortLook`.

Erreurs. Aucune.

`UNKNOWN_PORT_LOOK` constante

```
const UNKNOWN_PORT_LOOK: PortLook
```

Dessin d'un port dont le type de connecteur est introuvable : famille « autre », forme `generic` (rectangle arrondi gris).

`PortDetail` type

```
type PortDetail = 'full' | 'simple' | 'none';
```

Niveau de détail des ports : `'full'` (dessin du connecteur réel), `'simple'` (petit rectangle de la couleur de la famille, très fort dézoom) ou `'none'` (ports non dessinés : masqués par l'utilisateur dans une vue extérieure).

`PORT_SIMPLIFY_BELOW_PX_PER_MM` constante

```
const PORT_SIMPLIFY_BELOW_PX_PER_MM = 1.6
```

Zoom, en pixels d'écran par millimètre de papier, sous lequel les ports peuvent être simplifiés (NF-02b) : 1,6, où environ quatre pages A4 tiennent sur un écran de bureau.

`PORT_SIMPLIFY_MIN_VISIBLE_MM2` constante

```
const PORT_SIMPLIFY_MIN_VISIBLE_MM2 = 4 * 210 * 297
```

Surface de papier visible au-delà de laquelle les ports peuvent être simplifiés : quatre pages A4 (4 × 210 × 297 mm²).

`portSimplifyThreshold` fonction

```
function portSimplifyThreshold(viewport?: { width: number; height: number }): number
```

Seuil de simplification pour une fenêtre : `min(1,6 ; √(surface de la fenêtre en px² / surface de quatre A4 en mm²))`. Dans une petite fenêtre, le seuil baisse pour que les ports ne soient simplifiés que lorsque plus de quatre pages A4 y tiennent.

Paramètres

`viewport` facultatif : taille du plan de travail en pixels.

Résultat. Le seuil en pixels par mm ; 1,6 sans fenêtre ou pour une taille nulle.

Erreurs. Aucune.

`portDetailAt` fonction

```
function portDetailAt(pixelsPerMm: number, viewport?: { width: number; height: number }): PortDetail
```

Niveau de détail des ports à un zoom, dans une fenêtre : le niveau vaut `'simple'` sous le seuil de `portSimplifyThreshold`, et `'full'` sinon. Appelée par le plan de travail et, par défaut, par `PageContent` ; les exports passent 100 pixels par mm et obtiennent donc toujours `'full'`.

Paramètres

`pixelsPerMm` zoom ;

`viewport` facultatif : taille du plan de travail en pixels.

Résultat. `'full'` ou `'simple'` (jamais `'none'`).

Erreurs. Aucune.

```
portDetailAt(1.5);                                  // 'simple'
portDetailAt(1.5, { width: 700, height: 700 });   // 'full' (seuil abaissé à 1,4)
```

`SHAPE_SIZES` constante

```
const SHAPE_SIZES: Record<PortShapeKey, { w: number; h: number }>
```

Taille naturelle de chaque dessin de port, largeur et hauteur en millimètres du connecteur réel (par exemple RJ45 12 × 12, SFP 14 × 9, USB C 9 × 3,6). Le dessin garde ces proportions et est centré dans le rectangle du port (`fitShape`).

`fitShape` fonction

```
function fitShape(shape: PortShapeKey, rect: { x: number; y: number; width: number; height: number }): { x: number; y: number; scale: number; width: number; height: number }
```

Rectangle du dessin d'une forme dans le rectangle d'un port : proportions naturelles, centré, aussi grand que possible.

Paramètres

`shape` la forme ;

`rect` rectangle du port.

Résultat. `{ x, y, scale, width, height }` : coin haut gauche, facteur appliqué au dessin naturel et taille obtenue.

Erreurs. Aucune.

```
fitShape('rj45', { x: 0, y: 0, width: 24, height: 12 });
// { x: 6, y: 0, scale: 1, width: 12, height: 12 }
```

Chapitre 25

Référence : jeux d’essai

Les modules de `src/testing/` ne font pas partie de l’application livrée : ils construisent des projets déterministes et des contrôles partagés par les tests unitaires, les tests de rendu et le banc de rendu (`tests/harness/`). Leurs noms de projets et d’équipements restent génériques (CG-08). Modifier l’un d’eux change les tests et les captures qui l’utilisent.

25.1 `testing/fixtures`

Petites fabriques et projets d’essai complets, construits avec des machines générées à partir des paramètres des équipements intégrés (`library/equipment`). Utilisé par la plupart des tests du modèle, du magasin, du routage, des commandes et des exports.

`typesOnlyProject` fonction

```
function typesOnlyProject(options: { name?: string; author?: string } = {}): Project
```

Nouveau projet avec les types de connecteurs et de liaisons par défaut, mais sans modèles d’armoires ni machines, pour que chaque test ne voie que ses propres machines.

Paramètres

`options` facultatif : `name` et `author` du projet.

Résultat. Le projet.

Erreurs. Aucune.

`connector` fonction

```
function connector(key: string): Id
```

Identifiant fixe d’un type de connecteur par défaut à partir de sa clé (`rj45`, `sfpPlus`…).

Paramètres

`key` clé du type par défaut.

Résultat. L’identifiant.

Erreurs. Lève `Error('unknown connector <clé>')` pour une clé inconnue.

`linkType` fonction

```
function linkType(key: string): Id
```

Identifiant fixe d'un type de liaison par défaut à partir de sa clé (`ethernet`, `power`…).

Paramètres

- `key` clé du type de liaison.

Résultat. L'identifiant, ou celui du type de liaison par défaut pour une clé inconnue.

Erreurs. Aucune.

`ports` fonction

```
function ports(prefix: string, count: number, connectorKey: string, y = 0.5): Port[]
```

Crée `count` ports régulièrement espacés sur une rangée : noms `<préfixe>1`, `<préfixe>2`…, ordre croissant, abscisses `(i + 0,5) / count`.

Paramètres

- `prefix` préfixe des noms ;
- `count` nombre de ports ;
- `connectorKey` clé du type de connecteur par défaut ;
- `y` ordonnée de la rangée en proportion de la face (défaut 0,5).

Résultat. Les ports.

Erreurs. Lève l'erreur de `connector` pour une clé inconnue.

```
const front = ports('Gi', 24, 'rj45');
```

`machine` fonction

```
function machine(input: MachineInput): Machine
```

Machine construite telle quelle à partir des champs donnés (ports faits à la main, sans génération) ; simple appel de `createMachine`.

Paramètres

- `input` champs de la machine (`MachineInput`).

Résultat. La machine.

Erreurs. Celles de `createMachine`.

`generated` fonction

```
function generated(kind: EquipmentKind, params: Record<string, unknown> = {}, fields:
Partial<Omit<Machine, 'id' | 'equipment'>> = {}): Machine
```

Machine générée à partir des paramètres d'un équipement intégré, dont certains champs (nom, profondeur, dégagement…) sont ensuite remplacés.

Paramètres

- `kind` type d'équipement ;
- `params` paramètres (défaut : ceux du type) ;
- `fields` champs remplacés après génération.

Résultat. La machine.

Erreurs. Celles de `generateMachine` pour des paramètres invalides.

```
const sw = generated('switch', {}, { name: 'Commutateur 48 ports', depthMm: 300 });
```

`patchPanelMachine` fonction

```
function patchPanelMachine(count: number, name = 'Panneau de brassage'): Machine
```

Panneau de brassage de `count` modules RJ45 Cat6, 80 mm de profondeur, dont le port avant n est apparié au port arrière n (STD-05).

Paramètres

`count` nombre de modules ;

`name` nom (défaut « Panneau de brassage »).

Résultat. La machine.

Erreurs. Celles de `generateMachine`.

`SampleProject` interface

```
interface SampleProject { project; rack; machines }
```

Résultat de `sampleProject`.

Champs

`project` le projet ;

`rack` l'armoire 18U ;

`machines` les machines par rôle : `server`, `switch`, `patch`, `pdu`, `cableManager`, `monitor`.

`sampleProject` fonction

```
function sampleProject(): SampleProject
```

Petit projet complet : une armoire 18U (1000 mm de profondeur, 800 mm utiles, roulettes, un emplacement 0U de chaque côté à l'arrière) avec un commutateur 24 ports (U16), un serveur 2U (U10), un panneau de brassage 24 ports (U17), une PDU verticale 0U à l'arrière gauche, un écran extérieur ; deux liaisons (Ethernet serveur → commutateur, alimentation serveur → PDU) ; une page A4 paysage avec les vues avant et arrière (étiquettes latérales sur l'arrière) et la vue de l'écran, une note de montage et un tracé retouché de deux points attachés à la vue avant ; une révision A.

Résultat. Un `SampleProject`.

Erreurs. Lève `Error('fixture ports')` si une machine générée n'a pas les ports attendus (paramètres des équipements modifiés).

`PatchChain` interface

```
interface PatchChain { project; server; switchPort; panels; links }
```

Résultat de `patchChain`.

Champs

`project` le projet ;

`server` port du serveur au début de la chaîne ;

`switchPort` port du commutateur à la fin ;

`panels` les panneaux traversés, dans l'ordre ;

`links` les liaisons de la chaîne, dans l'ordre.

`portOf` fonction

```
function portOf(device: Device, m: Machine, face: FaceSide, i = 0): PortRef
```

Référence du i^{e} port d'une face d'un équipement.

Paramètres

`device` l'équipement ;

`m` sa machine ;

`face` la face ;

`i` rang du port (défaut 0).

Résultat. La référence `{ deviceId, portId }`.

Erreurs. Lève `Error('no port')` si la face n'a pas ce port.

`patchChain` fonction

```
function patchChain(n: number): PatchChain
```

Chaîne serveur → n panneaux de brassage → commutateur dans une armoire 42U (LIA-09, REC-16) : chaque panneau est entré par une face et quitté par l'autre, les faces d'entrée alternant (avant, arrière, avant…). Sert aux tests du chemin de bout en bout.

Paramètres

`n` nombre de panneaux traversés (0 : liaison directe).

Résultat. Un `PatchChain` ; les liaisons sont nommées `ETH-001`, `ETH-002`…

Erreurs. Aucune pour `n` positif ou nul.

```
const { project, links } = patchChain(3); // 4 liaisons
```

25.2 `testing/fonts`

Fichiers de police pour les tests unitaires sous Node (exports SVG et PDF, sous-ensembles TrueType), lus dans le paquet par l'alias de Vite. Ne fonctionne que sous Vitest.

`interMediumTrueType` fonction

```
function interMediumTrueType(): Uint8Array
```

Lit Inter Medium (graisse 500) au format TrueType depuis le paquet installé.

Résultat. Les octets du fichier.

Erreurs. Lève l'erreur de `readFileSync` si le fichier est introuvable (le chemin est résolu depuis le répertoire courant, qui doit être la racine du dépôt).

25.3 `testing/generatedRack`

Baies denses générées à partir d'une graine, pour les tests de propriété du routage (lot 38 : aucun câble devant un port étranger, croisements). Identifiants dérivés de la graine : la même graine donne toujours le même projet. Utilisé par les tests de `routing/` (couloirs, dégagement, faisceaux).

`seededRandom` fonction

```
function seededRandom(seed: number): () => number
```

Petit générateur pseudo-aléatoire déterministe (mulberry32).

Paramètres

`seed` graine entière (ramenée à 32 bits non signés).

Résultat. Une fonction qui rend à chaque appel un nombre dans $[0, 1[$.

Erreurs. Aucune.

```
const random = seededRandom(42);
const a = random(); // toujours la même suite pour la graine 42
```

`GeneratedRackOptions` interface

```
interface GeneratedRackOptions { switches?; servers?; zeroUPerSide? }
```

Options de `generatedRackProject`.

Champs

`switches` facultatif : nombre de commutateurs 48 ports, empilés en haut (défaut 3) ;

`servers` facultatif : nombre de serveurs 1U (défaut 8) ;

`zeroUPerSide` facultatif : 1 (emplacement 0U avant droit vide, trois PDU verticales) ou 2 (une PDU verticale dans chacun des deux emplacements de chaque côté, sur les deux faces).

`generatedRackProject` fonction

```
function generatedRackProject(seed: number, options: GeneratedRackOptions = {}): Project
```

Projet d'une armoire 42U de 800 mm de large entièrement câblée : commutateurs 48 ports + 4 SFP+ en haut, serveurs 1U à quatre cartes réseau à l'arrière, une PDU horizontale fixée à l'arrière en U2, des PDU verticales 0U, un écran extérieur. Chaque port RJ45 des commutateurs est relié (tirage selon la graine) à une carte d'un serveur (autre vue), à un autre port de commutateur (même vue) ou à l'écran ; chaque alimentation est reliée à une prise. Une page A3 paysage montre les vues avant et arrière (échelle 0,12) et la vue de l'écran.

Paramètres

`seed` graine (fixe aussi les identifiants) ;

`options` `GeneratedRackOptions`.

Résultat. Le projet.

Erreurs. Aucune attendue ; une modification des paramètres des équipements intégrés peut faire échouer les accès non vérifiés (`!`) avec une `TypeError`.

```
const project = generatedRackProject(7, { switches: 2, zeroUPerSide: 2 });
```

25.4 `testing/rangeProject`

Projet des vues sur une plage de U (PAG-16) et des câbles masqués (PAG-15), partagé par les tests unitaires et la scène `viewRange` du banc. Identifiants déterministes.

`RangeProject` interface

```
interface RangeProject { project; fullViewId; rangeViewId; devices; machines; links }
```

Résultat de `rangeProject`.

Champs

`project` le projet ;

`fullViewId` vue avant de l'armoire entière ;

`rangeViewId` même face, de 3U à 15U ;

`devices` équipements par rôle : `sw1` (commutateur 2U à cheval sur la limite haute), `sw2` (au-dessus de la plage), `pp1`, `pp2` (dessous), `srv` (à cheval sur la limite basse) ;

`machines` machines par rôle : `switch2u`, `switch`, `patch`, `server` ;

`links` liaisons par rôle : `inside` (dans la plage), `cutAbove` (vers un port coupé de `sw1`), `aboveDevice` (vers `sw2`), `below` (vers `pp2`), `outside` (les deux extrémités hors plage).

`rangeProject` fonction

```
function rangeProject(): RangeProject
```

Construit le projet (aussi utilisé par un test de l'export SVG) : une armoire 18U de 600 × 800 mm à roulettes, cinq équipements répartis autour de la plage 3U–15U, cinq liaisons Ethernet `C01` à `C05`, une page A4 paysage avec deux vues avant à l'échelle 0,15 sans étiquettes latérales (« Baie entière » et « De 3U à 15U »).

Résultat. Un `RangeProject`.

Erreurs. Aucune attendue ; un port introuvable après une modification des générateurs lève une `TypeError`.

25.5 `testing/referenceProject`

Projet disposé comme le schéma de référence de `docs/references/`, avec des machines génériques seulement. Partagé par le banc de rendu (scènes `page`, `routedPage`, `titleBlocks`, `newViews`, `annotations`) et les tests unitaires du routage (croisements, lot 38).

`referenceLikeProject` fonction

```
function referenceLikeProject(): Project
```

Construit le projet « Baie de démonstration » : une armoire 18U à roulettes de la bibliothèque par défaut avec étagère coulissante 2U, commutateur, deux serveurs, pare-feu, panneau de brassage mixte (12 HDMI, 12 RJ45 Cat6a), PDU verticale 0U ; trois écrans et trois pupitres extérieurs ; les câbles du schéma de référence (Ethernet, alimentation, vidéo) ; une page A4 paysage avec les vues avant (étiquettes latérales) et arrière à l'échelle 0,12 et les six vues extérieures ; une révision A. Le compteur d'identifiants est remis à zéro à chaque appel : deux appels donnent le même projet.

Résultat. Le projet.

Erreurs. Aucune attendue ; si la bibliothèque par défaut perd son modèle 18U à roulettes ou une machine générique, un accès non vérifié lève une `TypeError`.

25.6 `testing/routingChecks`

Contrôles du routage utilisés par les tests (lot 38) : rectangles des ports dessinés dans les vues d'une page et segments de câbles qui passent devant un port qui n'est pas une de leurs extrémités (ROU-14). Dépend de `model/pageGeometry`, `model/pageLayout` et `model/lookup`.

`PortRect` interface

```
interface PortRect { key; x0; y0; x1; y1 }
```

Rectangle d'un port dessiné sur la page, en mm de papier.

Champs

`key` clé du port (`portKey` : équipement et port) ;

`x0` , `y0`, `x1`, `y1` : coins haut gauche et bas droit.

`portRects` fonction

```
function portRects(project: Project, page: Page): PortRect[]
```

Rectangles de tous les ports dessinés dans les vues d'une page : ports des équipements d'armoire vus depuis chaque vue (échelle effective, décalage d'une plage de U ; un port dont le centre est coupé par la plage est omis) et ports des vues extérieures (sauf ports masqués). Les ports ajoutés à un équipement posé ne sont pas comptés.

Paramètres

`project` le projet ;

`page` la page.

Résultat. La liste des `PortRect`.

Erreurs. Aucune.

`segmentHitsRect` fonction

```
function segmentHitsRect(a: Point, b: Point, rect: PortRect, margin = 1e-4): boolean
```

Indique si un segment traverse l'intérieur d'un rectangle (algorithme de Liang–Barsky) ; toucher le bord est permis, le rectangle étant réduit de `margin` de chaque côté.

Paramètres

`a` , `b` : extrémités du segment ;

`rect` le rectangle ;

`margin` réduction du rectangle (défaut 10^{-4} mm).

Résultat. `true` si une partie de longueur non nulle du segment est à l'intérieur.

Erreurs. Aucune.

```
const rect = { key: 'p', x0: 0, y0: 0, x1: 10, y1: 10 };
segmentHitsRect({ x: -5, y: 5 }, { x: 15, y: 5 }, rect); // true
segmentHitsRect({ x: 0, y: -5 }, { x: 0, y: 15 }, rect); // false (sur le bord)
```

`ForeignPortHit` interface

```
interface ForeignPortHit { linkRef; port; from; to }
```

Passage d'un câble devant un port étranger.

Champs

- `linkRef` référence du câble ;
- `port` clé du port traversé ;
- `from` , `to` : extrémités du segment fautif, en mm de papier.

`foreignPortHits` fonction

```
function foreignPortHits(cables: CableDrawing[], rects: PortRect[]): ForeignPortHit[]
```

Segments des câbles qui passent devant un port autre que leurs deux extrémités (ROU-14) : pour chaque segment de chaque tracé, un filtre par boîtes englobantes puis `segmentHitsRect`. Un test de routage attend une liste vide.

Paramètres

- `cables` tracés des câbles (`routePage(…).cables`) ;
- `rects` rectangles des ports (`portRects`).

Résultat. La liste des `ForeignPortHit`.

Erreurs. Aucune.

```
const routing = routePage(project, page, avoid);
const hits = foreignPortHits(routing.cables, portRects(project, page));
expect(hits).toEqual([]);
```

Chapitre 26

Référence : fichiers, stockage et import

Ce chapitre documente une à une les déclarations exportées des dossiers `src/import/` (import tableur simplifié) et `src/io/` (format de fichier, images, stockage du navigateur, ouverture et enregistrement). La conception d'ensemble, les algorithmes et les décisions sont expliqués au chapitre 10. Aucun de ces modules ne dépend de React.

26.1 `import/read`

Lecture des fichiers à importer (IMP-04) : classeurs XLSX, lus par ExcelJS chargé à la demande (import dynamique, comme l'export tableur), et fichiers CSV lus à la main, dont l'encodage et le séparateur sont détectés. Le résultat est une liste d'onglets faits de lignes de cellules, sans interprétation : le sens des colonnes est l'affaire de `import/rules`. Utilisé par la fenêtre `ImportSpreadsheetDialog` (`src/ui/dialogs/`) et par les tests. Ne dépend que d'ExcelJS.

`SheetCell` type

```
type SheetCell = string | number | null;
```

Valeur d'une cellule lue : texte, nombre ou `null` pour une cellule vide. Les cellules d'un CSV sont toujours du texte ; celles d'un XLSX peuvent être des nombres.

Champs

`string` texte de la cellule (non rogné) ;

`number` nombre d'une cellule numérique XLSX ;

`null` cellule vide, formule en erreur ou valeur absente.

`Sheet` interface

```
interface Sheet { name; rows }
```

Un onglet lu, tel que le reçoit `planImport`.

Champs

`name` nom de l'onglet (XLSX) ou nom du fichier sans extension (CSV) ; sert à reconnaître le type d'onglet et à désigner l'onglet dans le rapport ;

`rows` lignes dans l'ordre du fichier ; `rows[0]` est la ligne d'en-tête et `rows[n]` la ligne `n + 1` du tableur. Une ligne peut être plus courte que l'en-tête.

`ReadWorkbook` interface

```
interface ReadWorkbook { format; sheets }
```

Résultat de `readSpreadsheet`.

Champs

`format` `'xlsx'` ou `'csv'`, selon les premiers octets du fichier ;

`sheets` onglets du classeur, dans l'ordre (un seul pour un CSV).

`CsvEncoding` type

```
type CsvEncoding = 'utf-8' | 'utf-16le' | 'utf-16be' | 'windows-1252';
```

Encodages reconnus pour un CSV, noms acceptés par `TextDecoder`.

Champs

`'utf-8'` UTF-8, avec ou sans marque d'ordre des octets ;

`'utf-16le'` UTF-16 petit-boutiste (marque `FF FE`) ;

`'utf-16be'` UTF-16 gros-boutiste (marque `FE FF`) ;

`'windows-1252'` repli pour un fichier qui n'est pas de l'UTF-8 valide.

`detectEncoding` fonction

```
function detectEncoding(bytes: Uint8Array): { encoding: CsvEncoding; bomLength: number }
```

Détermine l'encodage d'un fichier CSV (IMP-04) : d'après sa marque d'ordre des octets si elle est présente, sinon UTF-8 si les octets sont de l'UTF-8 valide (décodage strict), sinon Windows-1252. Appelée par `decodeCsv`.

Paramètres

`bytes` contenu brut du fichier.

Résultat. `{ encoding, bomLength }` : l'encodage et la longueur de la marque à sauter (3 pour UTF-8, 2 pour UTF-16, 0 sans marque).

Erreurs. Aucune.

```
detectEncoding(new Uint8Array([0xef, 0xbb, 0xbf, 0x41]));
// { encoding: 'utf-8', bomLength: 3 }
detectEncoding(new Uint8Array([0x45, 0xe9])); // é en Windows-1252
// { encoding: 'windows-1252', bomLength: 0 }
```

`decodeCsv` fonction

```
function decodeCsv(bytes: Uint8Array): string
```

Décode un fichier CSV en texte avec l'encodage détecté par `detectEncoding`, marque d'ordre des octets retirée.

Paramètres

`bytes` contenu brut du fichier.

Résultat. Le texte du fichier. Les séquences invalides sont remplacées par le caractère de remplacement Unicode (décodage non strict).

Erreurs. Aucune.

`DetectedSeparator` type

```
type DetectedSeparator = ';' | ',' | '\t';
```

Séparateurs de colonnes reconnus dans un CSV : `';'` (tableurs en français, valeur par défaut), `','` ou la tabulation `'\t'`.

`detectSeparator` fonction

```
function detectSeparator(text: string): DetectedSeparator
```

Choisit le séparateur d'un texte CSV : parmi « ; », « , » et la tabulation, celui qui apparaît le plus souvent sur la première ligne, hors des passages entre guillemets. À égalité, l'ordre « ; », « , », tabulation l'emporte ; si aucun n'apparaît, « ; ».

Paramètres

`text` texte décodé du fichier ; seule la première ligne (jusqu'au premier saut de ligne hors guillemets) est examinée.

Résultat. Le séparateur retenu.

Erreurs. Aucune.

```
detectSeparator('Nom,Machine,Armoire\nSW01,Commutateur 48 ports,A1'); // ','
detectSeparator('"Nom; complet"\tMachine');                          // '\t'
```

`parseCsv` fonction

```
function parseCsv(text: string, separator: DetectedSeparator): string[][]
```

Découpe un texte CSV en lignes et cellules selon la RFC 4180 : un champ qui commence par un guillemet peut contenir séparateurs et sauts de ligne, un guillemet doublé `""` y vaut un guillemet. Les fins de ligne `\r\n`, `\n` et `\r` sont acceptées ; un saut de ligne final n'ajoute pas de ligne vide.

Paramètres

`text` texte décodé ;

`separator` séparateur de colonnes.

Résultat. Les lignes, chacune un tableau de cellules texte (non rognées).

Erreurs. Aucune : un guillemet non fermé englobe simplement la fin du texte.

```
parseCsv('a;"b;c"\n"d ""e""";f', ';');
// [['a', 'b;c'], ['d "e"', 'f']]
```

`readCsv` fonction

```
function readCsv(bytes: Uint8Array, name: string): Sheet
```

Lit un fichier CSV complet : décodage (`decodeCsv`), détection du séparateur, découpage. Appelée par `readSpreadsheet`.

Paramètres

`bytes` contenu brut du fichier ;

`name` nom à donner à l'onglet (nom du fichier sans extension).

Résultat. Un onglet `Sheet` dont toutes les cellules sont du texte.

Erreurs. Aucune.

`readXlsx` fonction

```
async function readXlsx(bytes: Uint8Array): Promise<Sheet[]>
```

Lit tous les onglets d'un classeur XLSX avec ExcelJS (chargé à la première utilisation). Chaque cellule est convertie : nombre et texte tels quels, booléen en texte, date en jour ISO (`AAAA-MM-JJ`), texte enrichi en texte brut, formule en son résultat calculé, cellule en erreur en `null`. Les lignes et cellules vides intermédiaires sont conservées, pour que l'indice d'une ligne reste son numéro moins un.

Paramètres

`bytes` contenu du fichier XLSX.

Résultat. Promesse des onglets, dans l'ordre du classeur.

Erreurs. Rejette la promesse si ExcelJS ne peut pas lire le classeur (fichier endommagé ou qui n'est pas un XLSX).

`readSpreadsheet` fonction

```
async function readSpreadsheet(bytes: Uint8Array, fileName: string): Promise<ReadWorkbook>
```

Point d'entrée de la lecture : un fichier qui commence par la signature ZIP (`PK\3\4`) est lu comme XLSX (`readXlsx`), tout autre comme CSV (`readCsv`). Appelée par la fenêtre d'import dès que le fichier est choisi.

Paramètres

`bytes` contenu du fichier ;

`fileName` nom du fichier ; pour un CSV, son nom sans extension devient le nom de l'onglet.

Résultat. Promesse de `ReadWorkbook`.

Erreurs. Rejette la promesse quand un classeur XLSX est illisible ; la fenêtre affiche alors un message.

```
const workbook = await readSpreadsheet(opened.bytes, opened.name);
const result = planImport(project, workbook.sheets, { pageId });
```

26.2 `import/rules`

Règles de l'import tableur simplifié (IMP-02 à IMP-07) : reconnaissance des onglets et des colonnes (celles de l'export, EXP-04), puis vérification de chaque ligne avec les règles habituelles du modèle, contre le projet complété des lignes déjà acceptées. Création seulement, tout ou rien : le résultat est soit ce qu'il faut ajouter au projet, soit la liste des erreurs (onglet, ligne, colonne, motif). Module pur : il ne modifie pas le projet reçu. Dépend de `src/model/` (`checkPlacement`, `checkLinkEnds`, `checkCompatibility`, `proposeLinkTypeForPorts`, `nextLinkRef`, fabriques `createDevice`, `createLink`, `createView`) et des libellés de colonnes de l'export (`tables.columns.*`). Utilisé par l'action `importSpreadsheet` (`src/store/importActions.ts`), la fenêtre d'import et `import/template`.

`ImportSheetKind` type

```
type ImportSheetKind = 'devices' | 'links';
```

Type d'un onglet reconnu : `'devices'` (équipements, inventaire) ou `'links'` (liaisons, carnet de câblage).

`ImportIssue` interface

```
interface ImportIssue { sheet; row; column; message }
```

Un problème du rapport d'import (IMP-05), erreur ou avertissement.

Champs

- `sheet` nom de l'onglet (XLSX) ou du fichier (CSV) ; vide pour un problème du fichier entier (aucun onglet reconnu, fichier vide) ;
- `row` numéro de ligne dans le tableur (1 = ligne d'en-tête, utilisé pour une colonne manquante) ; `null` pour un problème du fichier entier ;
- `column` en-tête de la colonne en cause ; vide quand le problème concerne la ligne entière ;
- `message` motif, en français.

`ImportPlan` interface

```
interface ImportPlan { devices; links; views; pageId }
```

Ce qu'un import réussi ajoute au projet ; appliqué en une seule action annulable par `importSpreadsheet` (IMP-07).

Champs

- `devices` équipements créés, avec leur pose déjà vérifiée ;
- `links` liaisons créées, avec leur repère (explicite ou automatique) ;
- `views` vues des équipements extérieurs importés, à ajouter à la page `pageId` (vide sans page) ;
- `pageId` page qui reçoit ces vues, ou `null`.

`ImportResult` type

```
type ImportResult = | { ok: true; plan: ImportPlan; warnings: ImportIssue[] } | { ok:
false; errors: ImportIssue[]; warnings: ImportIssue[] };
```

Résultat de `planImport`.

Champs

`ok: true` `plan` (`ImportPlan`) et `warnings` (connecteurs incompatibles, colonnes de liaison ignorées) ;

`ok: false` `errors` (au moins une, rien n'est importé) et `warnings`.

`ImportOptions` interface

```
interface ImportOptions { pageId? }
```

Options de `planImport`.

Champs

`pageId` facultatif : page qui reçoit les vues des équipements extérieurs importés (la page active dans l'interface) ; `null` ou absent : pas de vue.

`DEVICE_COLUMNS` constante

```
const DEVICE_COLUMNS = …
```

Identifiants des colonnes de l'onglet Équipements, dans l'ordre de l'export de l'inventaire : `name`, `category`, `manufacturer`, `machine`, `reference`, `rack`, `u`, `height`, `face`, `description`. Le libellé d'en-tête de chacune vient de `deviceColumnLabel`. Sert à reconnaître les colonnes et à construire le modèle de classeur.

`LINK_COLUMNS` constante

```
const LINK_COLUMNS = …
```

Identifiants des colonnes de l'onglet Liaisons, dans l'ordre de l'export du carnet de câblage : repère (`ref`), `type`, armoire, U, équipement et port de chaque extrémité (`aRack`, `aU`, `aDevice`, `aPort`, puis `b`…), extrémités finales (`finalADevice`, `finalAPort`, `finalBDevice`, `finalBPort`), `length`, `category`, `supplierRef`, `sheathColor`, `notes`.

`ColumnUse` type

```
type ColumnUse = 'required' | 'optional' | 'ignored';
```

Usage d'une colonne par l'import.

Champs

`'required'` obligatoire : son absence est une erreur sur la ligne d'en-tête, une cellule vide est une erreur de la ligne ;

`'optional'` lue si elle est présente ;

`'ignored'` reconnue mais non lue (valeur calculée par l'export : hauteur, fabricant, extrémités finales…).

`DEVICE_COLUMN_USE` constante

```
const DEVICE_COLUMN_USE: Record<DeviceColumn, ColumnUse>
```

Usage de chaque colonne de `DEVICE_COLUMNS` : nom et machine obligatoires ; référence (pour départager deux machines de même nom), armoire, U, face et description facultatives ; type d'équipement (`category`, en-tête « Équipement »), fabricant et hauteur ignorés, car calculés par l'export. Sert aussi à la notice du modèle de classeur.

`LINK_COLUMN_USE` constante

```
const LINK_COLUMN_USE: Record<LinkColumn, ColumnUse>
```

Usage de chaque colonne de `LINK_COLUMNS` : équipement et port des deux extrémités obligatoires ; repère, type, longueur, catégorie, référence fournisseur, couleur de gaine et notes facultatifs ; armoires, U et extrémités finales ignorées.

`deviceColumnLabel` fonction

```
function deviceColumnLabel(id: DeviceColumn): string
```

Libellé d'en-tête d'une colonne d'équipement, identique à celui de l'export de l'inventaire (clé `tables.columns.inventory.<id>`).

Paramètres

- `id` identifiant de la colonne (élément de `DEVICE_COLUMNS`).

Résultat. Le libellé en français, par exemple « Nom ».

Erreurs. Aucune.

`linkColumnLabel` fonction

```
function linkColumnLabel(id: LinkColumn): string
```

Libellé d'en-tête d'une colonne de liaison, identique à celui du carnet de câblage (clé `tables.columns.cabling.<id>`).

Paramètres

- `id` identifiant de la colonne (élément de `LINK_COLUMNS`).

Résultat. Le libellé en français.

Erreurs. Aucune.

`nameKey` fonction

```
function nameKey(text: string): string
```

Clé de comparaison d'un nom (équipement, machine, armoire, type de liaison) : texte rogné, espaces multiples réduits à un, minuscules (règles du français). Deux noms de même clé sont considérés comme identiques par l'import (IMP-06).

Paramètres

- `text` nom à comparer.

Résultat. La clé.

Erreurs. Aucune.

```
nameKey('  Serveur   2U ') === nameKey('serveur 2u'); // true
```

`headerKey` fonction

```
function headerKey(text: string): string
```

Clé de comparaison d'un en-tête ou d'un nom d'onglet : `nameKey`, puis accents retirés (décomposition Unicode) et apostrophe typographique ramenée à l'apostrophe droite. Le suffixe d'unité entre parenthèses n'est pas retiré ici : c'est la table des en-têtes connus qui enregistre chaque libellé avec et sans ce suffixe (« Longueur (m) » et « Longueur »).

Paramètres

`text` en-tête lu dans le fichier, ou libellé attendu.

Résultat. La clé.

Erreurs. Aucune.

```
headerKey('Équipements') === headerKey('equipements'); // true
```

`cellText` fonction

```
function cellText(cell: SheetCell | undefined): string
```

Texte rogné d'une cellule ; un nombre est converti en texte, une cellule vide ou absente donne la chaîne vide.

Paramètres

`cell` cellule lue, éventuellement `undefined` (ligne plus courte que l'en-tête).

Résultat. Le texte, sans espaces de début et de fin.

Erreurs. Aucune.

`cellNumber` fonction

```
function cellNumber(cell: SheetCell | undefined): number | null
```

Nombre contenu dans une cellule (U, longueur) : une cellule numérique est rendue telle quelle ; un texte est lu après suppression des espaces, avec point ou virgule décimale.

Paramètres

`cell` cellule lue.

Résultat. Le nombre ; `null` pour une cellule vide ; `NaN` pour un texte qui n'est pas un nombre (à l'appelant d'en faire une erreur).

Erreurs. Aucune.

```
cellNumber('2,5');   // 2.5
cellNumber('1 000'); // 1000
cellNumber('');      // null
cellNumber('dix');   // NaN
```

`sheetKind` fonction

```
function sheetKind(sheet: Sheet): ImportSheetKind | null
```

Reconnaît le type d'un onglet : d'abord par son nom (« Équipements » ou titre de l'inventaire ; « Liaisons » ou titre du carnet de câblage), sinon par ses en-têtes (colonnes Nom et Machine pour les équipements, les quatre colonnes d'extrémités pour les liaisons), ce qui couvre un CSV ou un onglet renommé.

Paramètres

`sheet` onglet lu.

Résultat. `'devices'`, `'links'`, ou `null` pour un onglet ignoré (notice, autre contenu).

Erreurs. Aucune.

`planImport` fonction

```
function planImport(project: Project, sheets: readonly Sheet[], options: ImportOptions =
{}): ImportResult
```

Vérifie les onglets d'un classeur et construit ce qu'il faut ajouter au projet (IMP-02 à IMP-06). Les onglets d'équipements sont traités avant ceux de liaisons, les autres ignorés ; chaque ligne non vide est vérifiée contre le projet complété des lignes déjà acceptées. Les repères automatiques sont attribués à la fin, une fois tous les repères explicites connus. Appelée par la fenêtre d'import (analyse sans modification) puis par `importSpreadsheet` (revalidation sur le projet courant avant l'ajout).

Paramètres

`project` projet courant, non modifié ;

`sheets` onglets lus par `readSpreadsheet` ;

`options` `ImportOptions` (page des vues extérieures).

Résultat. `ImportResult` : le plan et les avertissements, ou toutes les erreurs trouvées (pas seulement la première) et les avertissements.

Erreurs. Aucune exception : un fichier sans onglet reconnu, sans ligne, ou une ligne fautive donnent un résultat `ok: false`.

```
const sheets: Sheet[] = [{
  name: 'Équipements',
  rows: [['Nom', 'Machine', 'Armoire', 'U (bas)', 'Face'],
         ['SW01', 'Commutateur 48 ports', 'A1', 40, 'Avant']],
}];
const result = planImport(project, sheets);
if (result.ok) console.log(result.plan.devices.length); // 1
else console.log(result.errors.map((e) => `${e.row} ${e.column} : ${e.message}`));
```

26.3 `import/template`

Fichiers de l'import tableur : le modèle de classeur vide (IMP-01), avec les onglets Équipements et Liaisons portant les en-têtes de l'export (EXP-04) et un onglet de notice, et le rapport d'erreurs exportable en CSV (IMP-05). Dépend de `import/rules` (colonnes et usages) et de l'export tableur (`toXlsx` de `src/export/spreadsheet.ts`, `csvCells` de `src/export/csv.ts`). Utilisé par la commande `import.template` (`src/commands/import.ts`) et la fenêtre d'import.

`templateTables` fonction

```
function templateTables(): Table[]
```

Tables du modèle de classeur : l'onglet Équipements et l'onglet Liaisons, vides avec leurs en-têtes, et l'onglet « Notice » qui donne pour chaque colonne son onglet, son en-tête, son usage (obligatoire, facultative, ignorée) et ce qu'il faut y écrire, plus une ligne sur les colonnes libres de l'onglet Équipements. Séparée de `buildImportTemplate` pour être testée sans produire de XLSX.

Résultat. Trois tables (`Table` de `src/export/tables.ts`).

Erreurs. Aucune.

`buildImportTemplate` fonction

```
async function buildImportTemplate(project: Project): Promise<ExportedFile>
```

Produit le modèle de classeur vide (IMP-01), téléchargé par la commande `import.template`.

Paramètres

`project` projet courant, transmis à `toXlsx` (propriétés du classeur).

Résultat. Promesse d'un `ExportedFile` : octets XLSX, nom « Modèle d'import Portolan.xlsx » (clé `import.templateFileName`), type MIME XLSX.

Erreurs. Rejette la promesse si la génération XLSX échoue (ExcelJS).

`buildImportReport` fonction

```
function buildImportReport(issues: readonly ImportIssue[], fileName: string): ExportedFile
```

Produit le rapport d'erreurs d'un import refusé (IMP-05), en CSV séparé par « ; », colonnes Onglet, Ligne, Colonne, Motif.

Paramètres

`issues` erreurs à lister (la fenêtre passe `result.errors` ; les avertissements n'y figurent pas) ;

`fileName` nom du fichier importé ; son nom sans extension sert à nommer le rapport.

Résultat. Un `ExportedFile` nommé `<nom>_rapport d'import.csv`, type `text/csv`.

Erreurs. Aucune.

```
if (!result.ok) downloadExport(buildImportReport(result.errors, file.name));
```

26.4 `io/archive`

Fichiers de projet `.rkdg` et de bibliothèque `.rkdglib` (§ 8 du cahier, FMT-01 à FMT-09) : archive ZIP contenant `manifest.json`, `project.json`, `images/<sha256>.<ext>` et un `thumbnail.png` facultatif. L'écriture est déterministe et entièrement faite en mémoire (NF-04) ; la lecture est défensive : décompression bornée, chemins sûrs, validation par le schéma, intégrité des images, SVG nettoyés (NF-06, REC-03). Dépend de `fflate`, de Zod, du schéma et de la validation du modèle, de `io/images`, `io/hash`, `io/migrations` et `io/svg`. Utilisé par la session (`src/app/session.ts`), l'import et l'export de bibliothèques, l'éditeur de machine (reprise de positions depuis un autre fichier) et `tools/json-schema.ts`. Le module réexporte aussi `LibraryInfoSchema` et `LibraryInfo`, définis dans `src/model/schema.ts`.

`RKDG_MIME_TYPE` constante

```
const RKDG_MIME_TYPE = 'application/vnd.rkdg+zip'
```

Type MIME interne d'un fichier de projet, `application/vnd.rkdg+zip` (FMT-07). Utilisé par le sélecteur de fichiers et pour le téléchargement d'un `.rkdg`.

`RKDG_EXTENSION` constante

```
const RKDG_EXTENSION = '.rkdg'
```

Extension d'un fichier de projet, `.rkdg`. C'est la valeur par défaut des sélecteurs de fichiers et de `suggestedFileName`.

`RKDGLIB_EXTENSION` constante

```
const RKDGLIB_EXTENSION = '.rkdglib'
```

Extension d'un fichier de bibliothèque, `.rkdglib` (FMT-07, BIB-13). La session s'en sert pour reconnaître un fichier de bibliothèque ouvert ou déposé.

`ManifestSchema` constante

```
const ManifestSchema = …
```

Schéma Zod de `manifest.json` (FMT-01, FMT-09), utilisé à la lecture et pour générer `rkdg-manifest-v2.schema.json`.

Champs

- `format` `'rkdg'` ou `'rkdglib'` ;
- `formatVersion` entier ≥ 1, version du format du fichier ;
- `appVersion` version de l'application qui a écrit le fichier ;
- `createdAt` date de création (ISO 8601) ;
- `modifiedAt` date d'enregistrement (ISO 8601) ;
- `library` facultatif, obligatoire pour un `.rkdglib` (vérifié à la lecture) : description de la bibliothèque (`LibraryInfoSchema` : `id` UUID, `name`, `brand`, `version` ≥ 1, `author`, `description`, `license`).

`Manifest` type

```
type Manifest = z.infer<typeof ManifestSchema>;
```

Type TypeScript déduit de `ManifestSchema` ; rendu dans le résultat de lecture (`manifest`). Mêmes champs que le schéma.

`ArchiveWriteError` classe

```
class ArchiveWriteError extends Error
```

Erreur levée par `writeProjectArchive` et `writeLibraryArchive` quand une image référencée par le document n'est pas dans la table des images : l'archive n'est pas produite plutôt que d'être incomplète. Son message (clé `io.errors.imageNotLoaded`) nomme l'image ; la session l'affiche comme un échec d'enregistrement.

`serializeProject` fonction

```
function serializeProject(project: Project): string
```

Texte normalisé de `project.json` pour un projet : l'objet passe par `ProjectSchema.parse` (valeurs par défaut complétées, clés inconnues retirées, ordre des clés du schéma), puis est écrit avec une indentation d'un espace et un saut de ligne final. Garantit l'aller-retour octet pour octet.

Paramètres

`project` projet à écrire.

Résultat. Le texte JSON.

Erreurs. Lève une `ZodError` si le projet n'est pas conforme au schéma (défaut de programmation : le magasin ne produit que des projets valides).

`WriteProjectOptions` interface

```
interface WriteProjectOptions { project; images; appVersion; thumbnail?; now? }
```

Paramètres de `writeProjectArchive`.

Champs

`project` projet à enregistrer ;

`images` images disponibles (octets par référence) ; seules celles que le projet référence sont écrites (FMT-03) ;

`appVersion` version de l'application, écrite dans le manifeste ;

`thumbnail` facultatif : PNG d'aperçu, écrit en `thumbnail.png` ;

`now` facultatif : date d'enregistrement (`modifiedAt` et date des entrées ZIP) ; maintenant par défaut, fixée par les tests.

`writeProjectArchive` fonction

```
function writeProjectArchive(options: WriteProjectOptions): Uint8Array
```

Construit en mémoire une archive `.rkdg` complète : manifeste (format, `FORMAT_VERSION`, `createdAt` du projet, `modifiedAt` = `now`), `project.json` normalisé (`serializeProject`), images référencées triées par nom (PNG, JPEG, WebP sans compression, SVG compressés), aperçu éventuel. Appelée par la session à l'enregistrement, avant toute écriture sur le disque (NF-04).

Paramètres

`options` voir `WriteProjectOptions`.

Résultat. Les octets du fichier.

Erreurs. `ArchiveWriteError` si une image référencée manque ; `ZodError` si le projet est invalide.

```
const bytes = writeProjectArchive({ project, images, appVersion: APP_VERSION });
const saved = await saveFile(bytes, {
  suggestedName: suggestedFileName(project.meta.name),
  description: 'Projet Portolan',
});
```

`WriteLibraryOptions` interface

```
interface WriteLibraryOptions { library; info; images; appVersion; createdAt?; now? }
```

Paramètres de `writeLibraryArchive`.

Champs

- `library` bibliothèque à écrire (machines, types de connecteurs et de liaisons) ;
- `info` description de la bibliothèque, écrite dans le manifeste ; elle est validée par `LibraryInfoSchema` ;
- `images` images disponibles ; seules celles des machines sont écrites ;
- `appVersion` version de l'application ;
- `createdAt` facultatif : date de création (ISO) ; `now` par défaut ;
- `now` facultatif : date d'enregistrement.

`writeLibraryArchive` fonction

```
function writeLibraryArchive(options: WriteLibraryOptions): Uint8Array
```

Construit une archive `.rkdglib` (FMT-09) : manifeste enrichi du bloc `library`, `project.json` réduit à `{ formatVersion, library }` (`LibraryDocumentSchema`), images des machines. Appelée par l'export de bibliothèque (BIB-12) et par la session pour enregistrer une bibliothèque en édition (BIB-13).

Paramètres

- `options` voir `WriteLibraryOptions`.

Résultat. Les octets du fichier.

Erreurs. `ArchiveWriteError` si une image manque ; `ZodError` si la bibliothèque ou sa description est invalide (nom vide, version < 1…).

`ReadLimits` interface

```
interface ReadLimits { maxEntries; maxTotalBytes; maxJsonBytes }
```

Limites de la décompression (NF-06), mesurées sur les octets réellement décompressés.

Champs

- `maxEntries` nombre maximal d'entrées dans l'archive ;
- `maxTotalBytes` taille décompressée totale maximale, en octets (aussi la limite d'une image) ;
- `maxJsonBytes` taille maximale d'un fichier `.json`, en octets.

`DEFAULT_READ_LIMITS` constante

```
const DEFAULT_READ_LIMITS: ReadLimits
```

Limites par défaut de la lecture : 10 000 entrées, 512 Mo décompressés au total, 64 Mo par fichier JSON (décision du 04/11/2026, lot 04). Les tests les abaissent par `ReadOptions.limits` pour simuler une bombe de décompression.

`ReadErrorCode` type

```
type ReadErrorCode = | 'notArchive' | 'tooManyEntries' | 'tooLarge' | 'unsafePath'
| 'missingManifest' | 'invalidManifest' | 'wrongFormat' | 'newerVersion' | 'olderVersion'
| 'migrationMissing' | 'missingProject' | 'invalidJson' | 'invalidContent' | 'missingImage'
| 'corruptImage';
```

Codes d'erreur de lecture ; chacun a son message `io.errors.<code>`.

Champs

- `notArchive` pas une archive ZIP, archive tronquée ou vide ;
- `tooManyEntries` plus de `maxEntries` entrées ;
- `tooLarge` taille décompressée au-delà des limites ;
- `unsafePath` chemin absolu, avec `..`, `.`, `\` ou lettre de lecteur ;
- `missingManifest` `manifest.json` absent ;
- `invalidManifest` manifeste non conforme, ou bibliothèque sans bloc `library` ;
- `wrongFormat` projet ouvert comme bibliothèque ou l'inverse ;
- `newerVersion` format plus récent que l'application (FMT-05) ;
- `olderVersion` format antérieur à `MIN_FORMAT_VERSION` (FMT-04) ;
- `migrationMissing` une étape de migration manque ;
- `missingProject` `project.json` absent ;
- `invalidJson` JSON illisible (manifeste ou contenu) ;
- `invalidContent` contenu non objet, ou erreurs de schéma ou de cohérence (détaillées) ;
- `missingImage` image référencée absente de l'archive ;
- `corruptImage` contenu d'une image différent de son empreinte.

`ReadError` interface

```
interface ReadError { ok; code; message; details }
```

Échec de lecture, rendu (jamais levé) par les fonctions de lecture.

Champs

- `ok` toujours `false` ;
- `code` `ReadErrorCode` ;
- `message` message pour l'utilisateur, en français ;
- `details` problèmes détaillés, un par ligne (erreurs de schéma au format `chemin : message`) ; souvent vide.

`ReadProjectResult` type

```
type ReadProjectResult = (ReadCommon & { format: 'rkdg'; project: Project; thumbnail?:
Uint8Array }) | ReadError;
```

Résultat de `readProjectArchive` : `ReadError`, ou succès avec les champs communs et ceux d'un projet.

Champs

- `ok` `true` ;
- `format` `'rkdg'` ;

`manifest` manifeste lu ;

`project` projet validé (et migré), références d'images à jour ;

`images` images vérifiées, SVG nettoyés ;

`thumbnail` facultatif : octets de `thumbnail.png` (non vérifiés) ;

`warnings` anomalies non bloquantes de la validation (LIA-07) ;

`migratedFrom` versions depuis lesquelles le fichier a été migré (vide si à jour).

`ReadLibraryResult` type

```
type ReadLibraryResult = (ReadCommon & { format: 'rkdglib'; library: Library; info:
LibraryInfo }) | ReadError;
```

Résultat de `readLibraryArchive` : `ReadError`, ou succès avec `format: 'rkdglib'`, `manifest`, `library` (bibliothèque validée), `info` (description lue dans le manifeste), `images`, `warnings` et `migratedFrom`, de même sens que pour un projet.

`ReadOptions` interface

```
interface ReadOptions { limits?; sanitizer?; migrations? }
```

Options de lecture, toutes facultatives (surtout utiles aux tests).

Champs

`limits` limites partielles, fusionnées avec `DEFAULT_READ_LIMITS` ;

`sanitizer` nettoyeur SVG ; `sanitizeSvg` par défaut (les tests passent un nettoyeur jsdom) ;

`migrations` table de migrations ; `MIGRATIONS` par défaut.

`readProjectArchive` fonction

```
async function readProjectArchive(bytes: Uint8Array, options: ReadOptions = {}):
Promise<ReadProjectResult>
```

Lit un fichier `.rkdg` : décompression bornée, manifeste, type et version, migrations, validation par `parseProject`, puis vérification des images (présence, empreinte, nettoyage SVG avec renommage des références). Les étapes sont détaillées au chapitre 10, section 10.3. Appelée par la session (ouvrir, glisser-déposer, nouveau projet depuis un existant) et par l'import de modèles depuis un `.rkdg` (BIB-08).

Paramètres

`bytes` contenu du fichier ;

`options` `ReadOptions`.

Résultat. Promesse de `ReadProjectResult`.

Erreurs. Ne lève jamais pour un fichier mauvais : l'erreur est rendue avec un message en français (`ReadError`). Lève seulement si le nettoyeur SVG par défaut n'a pas de fenêtre DOM (hors navigateur, sans `sanitizer`).

```
const result = await readProjectArchive(opened.bytes);
if (!result.ok) return fail(result.message, result.details);
editor.load(result.project, result.images);
```

`readLibraryArchive` fonction

```
async function readLibraryArchive(bytes: Uint8Array, options: ReadOptions = {}):
Promise<ReadLibraryResult>
```

Lit un fichier `.rkdglib` : mêmes étapes que `readProjectArchive`, avec `parseLibraryDocument` et les seules images des machines. Appelée par l'import de bibliothèque (BIB-10), l'ouverture d'une bibliothèque en édition (BIB-13) et la reprise de positions de ports depuis un autre fichier (EDM-07).

Paramètres

`bytes` contenu du fichier ;

`options` `ReadOptions`.

Résultat. Promesse de `ReadLibraryResult`.

Erreurs. Comme `readProjectArchive` ; un `.rkdg` donne `wrongFormat`.

26.5 `io/draft`

Brouillon de secours (PRJ-05, NF-04) : le JSON du projet est écrit dans IndexedDB après chaque modification (anti-rebond) et à intervalle régulier ; les images sont écrites une fois, avant le JSON. Le brouillon ne remplace jamais le fichier. Dépend de `io/storage` (interface `LocalStorage`), de l'éditeur (`src/store/editor.ts`) et de la validation du modèle. Utilisé par la session (`src/app/session.ts`).

`DraftSaverOptions` interface

```
interface DraftSaverOptions { debounceMs?; intervalMs?; fileInfo?; onError? }
```

Options de `startDraftSaver`, toutes facultatives.

Champs

`debounceMs` délai après la dernière modification avant d'écrire, en millisecondes (1 000 par défaut) ;

`intervalMs` durée maximale entre deux écritures tant que les modifications continuent, en millisecondes (30 000 par défaut) ;

`fileInfo` fonction qui rend le nom et la poignée du fichier courant, lue à chaque écriture ;

`onError` appelée avec l'erreur d'une écriture échouée (l'enregistreur continue).

`DraftSaver` interface

```
interface DraftSaver { flush; stop }
```

Enregistreur de brouillon en marche, rendu par `startDraftSaver`.

Champs

`flush` écrit tout de suite s'il y a quelque chose à écrire ; la promesse est résolue quand les écritures en cours et celle-ci sont terminées. Appelée par la session après un chargement ou un enregistrement, avant de fermer, et par les tests ;

`stop` se désabonne de l'éditeur et arrête les minuteries (sans écrire).

`draftRecord` fonction

```
function draftRecord(state: EditorState, fileName: string | null, fileHandle?: unknown, now = new Date()): DraftRecord
```

Construit l'enregistrement de brouillon d'un état de l'éditeur : JSON du projet, nom du projet, date, drapeau « non enregistré » (`isDirty`), nom et poignée du fichier.

Paramètres

`state` état de l'éditeur ;

`fileName` nom du fichier d'origine, ou `null` ;

`fileHandle` facultatif : poignée File System Access ;

`now` facultatif : date d'écriture (maintenant par défaut).

Résultat. Un `DraftRecord`.

Erreurs. Aucune.

`startDraftSaver` fonction

```
function startDraftSaver(editor: Editor, storage: LocalStorage, options: DraftSaverOptions = {}): DraftSaver
```

Démarre l'écriture automatique du brouillon : abonnement au magasin de l'éditeur (un changement du projet, de l'entrée enregistrée ou des images relance l'anti-rebond), intervalle de sécurité, écritures enchaînées sans chevauchement, images avant le JSON. Appelée par la session au démarrage, une fois la proposition de restauration réglée.

Paramètres

`editor` éditeur à surveiller ;

`storage` stockage local ouvert ;

`options` `DraftSaverOptions`.

Résultat. Un `DraftSaver`.

Erreurs. Aucune : les erreurs d'écriture sont remises à `onError`.

```
const saver = startDraftSaver(editor, storage, {
  fileInfo: () => ({ fileName: 'baie.rkdg' }),
  onError: (error) => console.warn(error),
});
// … avant de fermer :
await saver.flush();
saver.stop();
```

`RestoreResult` type

```
type RestoreResult = | { ok: true; project: Project; images: ImageMap; draft: DraftRecord; warnings: ValidationIssue[]; missingImages: number; } | { ok: false; errors: string[] };
```

Résultat de `restoreDraft`.

Champs

`ok: true` `project` (validé), `images` (celles trouvées dans le stockage), `draft` (l'enregistrement restauré), `warnings` (anomalies non bloquantes), `missingImages` (nombre d'images référencées absentes du stockage) ;

`ok: false` `errors` : messages (JSON illisible, format plus ancien, erreurs de validation au format `chemin : message`).

`restoreDraft` fonction

```
async function restoreDraft(storage: LocalStorage, draft: DraftRecord):
Promise<RestoreResult>
```

Reconstruit le projet d'un brouillon : lecture du JSON, refus d'une version de format antérieure à la version courante (pas de migration des brouillons), validation par `parseProject`, puis lecture des images référencées dans le stockage. Appelée par la session quand l'utilisateur accepte la restauration proposée au démarrage.

Paramètres

`storage` stockage local ;

`draft` brouillon lu par `loadDraft`.

Résultat. Promesse de `RestoreResult`. Une image manquante n'empêche pas la restauration (elle est comptée).

Erreurs. Les problèmes du brouillon sont rendus (`ok: false`) ; une erreur d'IndexedDB à la lecture des images rejette la promesse.

26.6 `io/files`

Ouverture et enregistrement de fichiers sur le disque (CG-07, PRJ-02, PRJ-03). Avec l'API File System Access (navigateurs à moteur Chromium), « Enregistrer » réécrit le fichier ouvert ; sans elle, l'ouverture passe par une sélection de fichier et l'enregistrement par un téléchargement. Le module contient le typage minimal de cette API, absente de la bibliothèque DOM de TypeScript. Dépend de `io/archive` (extension et type MIME). Utilisé par la session, les commandes et fenêtres d'import et d'export (`downloadBytes`, `pickFile`).

`WritableFileHandle` interface

```
interface WritableFileHandle { name; getFile; createWritable; queryPermission?;
requestPermission? }
```

Typage minimal d'une poignée de fichier File System Access (`FileSystemFileHandle`). Conservée par la session (et dans le brouillon) pour réécrire le même fichier.

Champs

`name` nom du fichier ;

`getFile` lit le fichier (`File`) ;

`createWritable` ouvre un flux d'écriture (`write`, `close`) ; le contenu n'est validé sur le disque qu'à `close` ;

`queryPermission` facultatif : état de la permission d'écriture ;

`requestPermission` facultatif : demande la permission d'écriture.

`OpenedFile` interface

```
interface OpenedFile { name; bytes; handle }
```

Fichier choisi et lu.

Champs

- `name` nom du fichier (avec extension) ;
- `bytes` contenu ;
- `handle` poignée pour réécrire le même fichier (File System Access seulement), sinon `null`.

`supportsFileSystemAccess` fonction

```
function supportsFileSystemAccess(): boolean
```

Indique si le navigateur offre `showOpenFilePicker` et `showSaveFilePicker`. Les tests de bout en bout retirent ces fonctions pour éprouver le repli par téléchargement.

Résultat. `true` si les deux sélecteurs existent.

Erreurs. Aucune.

`readFile` fonction

```
async function readFile(file: File, handle: WritableFileHandle | null = null):
Promise<OpenedFile>
```

Lit un objet `File` (sélection, glisser-déposer) en mémoire.

Paramètres

- `file` fichier à lire ;
- `handle` facultatif : poignée associée (`null` par défaut).

Résultat. Promesse de `OpenedFile`.

Erreurs. Rejette la promesse si le navigateur ne peut pas lire le fichier.

`pickFile` fonction

```
async function pickFile(description: string, accept: string | readonly string[] =
RKDG_EXTENSION): Promise<OpenedFile | null>
```

Fait choisir un fichier à l'utilisateur : sélecteur File System Access filtré par extension (une entrée par type MIME), sinon champ `<input type="file">` caché. Appelée par la session (ouvrir un projet ou une bibliothèque), l'import de bibliothèque, l'import tableur et l'éditeur de machine.

Paramètres

- `description` libellé du type de fichier dans le sélecteur ;
- `accept` extension acceptée, ou liste d'extensions (`.rkdg` par défaut).

Résultat. Promesse du fichier lu, ou `null` si l'utilisateur annule.

Erreurs. Rejette la promesse pour une erreur autre qu'une annulation (`AbortError`). Sans File System Access, la promesse ne se résout à l'annulation que si le navigateur émet l'évènement `cancel`.

```
const opened = await pickFile('Tableur', ['.xlsx', '.csv']);
if (opened) await readSpreadsheet(opened.bytes, opened.name);
```

`SaveOptions` interface

```
interface SaveOptions { handle?; saveAs?; suggestedName; description; extension? }
```

Options de `saveFile`.

Champs

`handle` facultatif : fichier à réécrire (« Enregistrer ») ; absent ou `null` : un nom est demandé ;

`saveAs` facultatif : demande toujours un nom (« Enregistrer sous ») ;

`suggestedName` nom proposé (et nom du téléchargement sans File System Access) ;

`description` libellé du type de fichier dans le sélecteur ;

`extension` facultatif : `.rkdg` par défaut, `.rkdglib` pour une bibliothèque (BIB-13).

`SavedFile` interface

```
interface SavedFile { name; handle }
```

Fichier enregistré.

Champs

`name` nom effectif du fichier ;

`handle` poignée à réutiliser au prochain « Enregistrer », ou `null` après un téléchargement.

`saveFile` fonction

```
async function saveFile(bytes: Uint8Array, options: SaveOptions): Promise<SavedFile | null>
```

Écrit des octets déjà complets (NF-04). Avec File System Access : réutilise la poignée si elle est fournie, que `saveAs` est faux et que la permission d'écriture est accordée (redemandée au besoin), sinon ouvre le sélecteur d'enregistrement ; l'écriture est validée à la fermeture du flux. Sans File System Access : téléchargement sous `suggestedName`. Appelée par la session (Ctrl+S, Ctrl+Maj+S).

Paramètres

`bytes` contenu complet du fichier ;

`options` `SaveOptions`.

Résultat. Promesse de `SavedFile`, ou `null` si l'utilisateur annule le sélecteur.

Erreurs. Rejette la promesse pour une erreur d'écriture ou de sélecteur autre qu'une annulation ; la session l'affiche comme un échec d'enregistrement.

`downloadBytes` fonction

```
function downloadBytes(bytes: Uint8Array, fileName: string, mimeType: string)
```

Télécharge des octets sous un nom : URL d'objet, lien caché cliqué puis retiré, URL révoquée après 10 s. Repli d'enregistrement sans File System Access, et voie normale des exports (images, PDF, tableur, étiquettes, bibliothèques, modèle d'import).

Paramètres

`bytes` contenu ;

`fileName` nom du fichier téléchargé ;

`mimeType` type MIME du `Blob`.

Erreurs. Aucune.

`suggestedFileName` fonction

```
function suggestedFileName(projectName: string, extension = RKDG_EXTENSION): string
```

Nom de fichier proposé pour un projet : son nom, avec les caractères interdits par les systèmes de fichiers (`\ / : * ? " < > |`) remplacés par un tiret, les espaces réduits, et l'extension ajoutée si elle manque ; nom par défaut (`defaults.fileName`) si le résultat est vide.

Paramètres

`projectName` nom du projet ;

`extension` facultatif : `.rkdg` par défaut.

Résultat. Le nom de fichier.

Erreurs. Aucune.

```
suggestedFileName('Salle A / baie 3'); // 'Salle A - baie 3.rkdg'
```

26.7 `io/hash`

Empreinte SHA-256 par WebCrypto, qui nomme les images (FMT-03, NF-04b). Aucune dépendance. Utilisé par `io/images` et `io/archive`.

`sha256Hex` fonction

```
async function sha256Hex(bytes: Uint8Array): Promise<string>
```

Calcule l'empreinte SHA-256 d'octets.

Paramètres

`bytes` contenu.

Résultat. Promesse de l'empreinte en 64 chiffres hexadécimaux minuscules.

Erreurs. Rejette la promesse si `crypto.subtle` est indisponible (page hors contexte sécurisé : ni HTTPS ni `localhost`).

26.8 `io/images`

Images d'un projet (FMT-03) : stockées à part de `project.json`, nommées par l'empreinte SHA-256 de leur contenu suivie de leur extension, de sorte qu'une image utilisée par plusieurs machines n'est stockée qu'une fois. Le module calcule les références, prépare une image importée, collecte les références d'un document et les remplace. Dépend de `io/hash`, `io/svg` et de `machineImageRefs` (`src/model/equipment.ts`). Utilisé par `io/archive`, `io/draft`, la session, les exports et l'interface (import d'images, éditeur de machine).

`ImageExtension` type

```
type ImageExtension = 'png' | 'jpg' | 'jpeg' | 'svg' | 'webp';
```

Extensions d'image acceptées : `'png'`, `'jpg'`, `'jpeg'`, `'svg'`, `'webp'` (les mêmes que `ImageRefSchema`).

`ImageMap` type

```
type ImageMap = Map<ImageRef, Uint8Array>;
```

Octets des images par référence (`Map<ImageRef, Uint8Array>`). Forme sous laquelle l'éditeur, l'archive, le stockage et les exports s'échangent les images ; le contenu d'une entrée n'est jamais modifié.

`imageExtension` fonction

```
function imageExtension(ref: ImageRef): ImageExtension
```

Extension d'une référence d'image (ce qui suit le dernier point).

Paramètres

`ref` référence d'image.

Résultat. L'extension.

Erreurs. Aucune (la référence est supposée valide).

`imageMimeType` fonction

```
function imageMimeType(ref: ImageRef): string
```

Type MIME d'une image d'après sa référence (`image/png`, `image/jpeg`, `image/svg+xml`, `image/webp`). Utilisé pour créer des `Blob`, des URL `data:` (export SVG) et convertir les images de l'export PDF.

Paramètres

`ref` référence d'image.

Résultat. Le type MIME.

Erreurs. Aucune.

`extensionFor` fonction

```
function extensionFor(mimeOrName: string): ImageExtension | undefined
```

Extension correspondant à un type MIME ou à un nom de fichier, sans tenir compte de la casse ; `image/jpeg` donne `jpg`. Appelée à l'import d'une image pour la refuser si son type n'est pas pris en charge.

Paramètres

`mimeOrName` type MIME (`File.type`) ou nom de fichier.

Résultat. L'extension, ou `undefined` si le format n'est pas pris en charge.

Erreurs. Aucune.

```
extensionFor('image/jpeg'); // 'jpg'
extensionFor('Face.PNG');   // 'png'
extensionFor('image/gif');  // undefined
```

`imageRefFor` fonction

```
async function imageRefFor(bytes: Uint8Array, extension: ImageExtension): Promise<ImageRef>
```

Référence d'une image : empreinte de ses octets, un point, l'extension.

Paramètres

`bytes` contenu de l'image (déjà nettoyé pour un SVG) ;

`extension` extension à utiliser.

Résultat. Promesse de la référence.

Erreurs. Celles de `sha256Hex`.

`prepareImage` fonction

```
async function prepareImage(bytes: Uint8Array, extension: ImageExtension, sanitizer:
SvgSanitizer = sanitizeSvg): Promise<{ ref: ImageRef; bytes: Uint8Array }>
```

Prépare une image importée par l'utilisateur : un SVG est nettoyé (NF-06), les autres images sont gardées telles quelles (EDM-03, pas de réencodage) ; la référence est calculée sur les octets finaux.

Paramètres

`bytes` contenu du fichier choisi ;

`extension` extension déterminée par `extensionFor` ;

`sanitizer` facultatif : nettoyeur SVG (`sanitizeSvg` par défaut).

Résultat. Promesse de `{ ref, bytes }` : la référence et les octets à stocker.

Erreurs. Celles du nettoyeur (pas de fenêtre DOM) et de `sha256Hex`.

```
const extension = extensionFor(file.type || file.name);
if (extension) {
  const { ref, bytes } = await prepareImage(
    new Uint8Array(await file.arrayBuffer()), extension);
}
```

`collectImageRefs` fonction

```
function collectImageRefs(project: Project): Set<ImageRef>
```

Toutes les références d'images d'un projet : logo du projet, images des machines de sa bibliothèque (faces, logo, équipement porté) et annotations d'image des pages. Sert à choisir les images à écrire dans l'archive, à vérifier à la lecture, à purger du stockage et à recharger d'un brouillon.

Paramètres

`project` projet.

Résultat. Ensemble des références.

Erreurs. Aucune.

`collectLibraryImageRefs` fonction

```
function collectLibraryImageRefs(library: Library): Set<ImageRef>
```

Références d'images des machines d'une bibliothèque seule (fichier `.rkdglib`).

Paramètres

`library` bibliothèque.

Résultat. Ensemble des références.

Erreurs. Aucune.

`remapLibraryImages` fonction

```
function remapLibraryImages(library: Library, map: ReadonlyMap<ImageRef, ImageRef>)
```

Remplace, sur place, les références d'images des machines d'une bibliothèque (faces avant et arrière, logo, équipement porté) selon une table ancien → nouveau ; une référence absente de la table est gardée. Appelée après la lecture quand le nettoyage a modifié un SVG.

Paramètres

`library` bibliothèque à modifier ;

`map` table des renommages.

Erreurs. Aucune.

`remapProjectImages` fonction

```
function remapProjectImages(project: Project, map: ReadonlyMap<ImageRef, ImageRef>)
```

Remplace, sur place, les références d'images d'un projet : logo du projet, machines de la bibliothèque (`remapLibraryImages`) et annotations d'image.

Paramètres

`project` projet à modifier ;

`map` table des renommages.

Erreurs. Aucune.

26.9 `io/index`

Point d'entrée du dossier : réexporte `io/archive`, `io/hash`, `io/images`, `io/svg` et `io/migrations`. Les modules liés à IndexedDB et aux fichiers (`io/storage`, `io/draft`, `io/files`) n'en font pas partie et s'importent directement. Ne déclare rien lui-même.

26.10 `io/migrations`

Migrations de `project.json` entre versions du format (FMT-04, PRJ-09) : `MIGRATIONS[n]` transforme un document de version `n` en document de version `n + 1`. Le format 2 (lot 34) a rompu avec le format 1 : les fichiers antérieurs à `MIN_FORMAT_VERSION` sont refusés sans migration, et la table est vide tant que le format reste en version 2. Dépend seulement de `FORMAT_VERSION`. Utilisé par `io/archive` ; voir le chapitre 10 pour la procédure d'évolution du format.

`MIN_FORMAT_VERSION` constante

```
const MIN_FORMAT_VERSION = 2
```

Plus ancienne version de format encore ouverte (migrée si besoin) : `2`. Un fichier plus ancien est refusé (`olderVersion`). À laisser à 2 une fois la version 1 de l'application publiée (FMT-04).

`JsonObject` type

```
type JsonObject = Record<string, unknown>;
```

Objet JSON brut (`Record<string, unknown>`) : document `project.json` avant validation, tel que le manipulent les migrations.

`Migration` type

```
type Migration = (doc: JsonObject) => JsonObject;
```

Étape de migration : fonction pure qui reçoit un document d'une version et rend le document de la version suivante (le champ `formatVersion` est réécrit par `migrate`). Elle ne doit rien supposer que le document ne garantit pas : il n'est pas encore validé.

`MIGRATIONS` constante

```
const MIGRATIONS: Readonly<Record<number, Migration>>
```

Table des étapes, indexée par la version de départ. Vide pour l'instant (format 2) ; la première entrée sera `2: (doc) => …` lors du passage au format 3.

`MigrationResult` type

```
type MigrationResult = | { ok: true; doc: JsonObject; applied: number[] } | { ok: false;
reason: 'newer' | 'older' | 'missing' | 'invalidVersion'; version: number };
```

Résultat de `migrate`.

Champs

- `ok: true` `doc` (document à la version cible) et `applied` (versions de départ des étapes appliquées, dans l'ordre) ;
- `ok: false` `reason` et `version` : `'newer'` (version supérieure à la cible), `'older'` (inférieure au minimum), `'missing'` (étape absente pour `version`), `'invalidVersion'` (pas un entier ≥ 1).

`migrate` fonction

```
function migrate(doc: JsonObject, fromVersion: number, target: number =
FORMAT_VERSION, migrations: Readonly<Record<number, Migration>> = MIGRATIONS, minVersion:
number = MIN_FORMAT_VERSION): MigrationResult
```

Amène un document d'une version à une autre en appliquant les étapes dans l'ordre ; chaque étape reçoit le résultat de la précédente et `formatVersion` est mis à jour après chacune. Appelée par la lecture d'archive avec la version du manifeste.

Paramètres

- `doc` document brut ;
- `fromVersion` version du document ;

`target` facultatif : version visée (`FORMAT_VERSION` par défaut) ;

`migrations` facultatif : table des étapes (`MIGRATIONS` par défaut) ;

`minVersion` facultatif : version minimale acceptée (`MIN_FORMAT_VERSION` par défaut).

Résultat. `MigrationResult`. Un document déjà à la version cible est rendu tel quel, `applied` vide.

Erreurs. Aucune exception propre (une étape qui lève propage son exception).

```
const steps = { 2: (doc: JsonObject) => ({ ...doc, routing: {} }) };
const result = migrate({ formatVersion: 2 }, 2, 3, steps);
// { ok: true, doc: { formatVersion: 3, routing: {} }, applied: [2] }
```

26.11 `io/storage`

Stockage du navigateur (NF-04b) : base IndexedDB `portolan` (via `idb`) avec trois magasins, le brouillon de secours (JSON du projet seulement), les images en blobs immuables indexés par leur empreinte (écrits une fois) et les préférences. Utilisé par `src/main.tsx` (ouverture, préférences), la session et `io/draft`.

`DraftRecord` interface

```
interface DraftRecord { projectJson; projectName; savedAt; dirty; fileName; fileHandle? }
```

Enregistrement du brouillon (magasin `project`, clé `draft`).

Champs

`projectJson` contenu de `project.json`, non validé (il l'est à la restauration) ;

`projectName` nom du projet, pour la proposition de restauration ;

`savedAt` date d'écriture du brouillon (ISO) ;

`dirty` le brouillon contient des modifications non enregistrées dans un fichier ; la restauration n'est proposée que dans ce cas ;

`fileName` nom du fichier d'origine, ou `null` ;

`fileHandle` facultatif : poignée File System Access, pour réécrire le même fichier après restauration ; omise si le navigateur ne sait pas la stocker.

`LocalStorage` interface

```
interface LocalStorage { saveDraft; loadDraft; clearDraft; putImages; getImages;
pruneImages; getPref; setPref; close }
```

Interface d'accès au stockage, rendue par `openLocalStorage` ; remplaçable dans les tests.

Champs

`saveDraft` écrit le brouillon (réessaie sans poignée si celle-ci ne peut pas être stockée) ;

`loadDraft` lit le brouillon, `undefined` s'il n'y en a pas ;

`clearDraft` supprime le brouillon (« Ignorer » à la proposition) ;

`putImages` écrit les images absentes du magasin, dans une transaction ; rend le nombre d'images écrites ;

`getImages` lit les images demandées présentes dans le magasin ;

`pruneImages` supprime les images qui ne sont pas dans `keep` ; rend le nombre d'images supprimées ;

`getPref` lit une préférence (type au choix de l'appelant) ;

`setPref` écrit une préférence ;

`close` ferme la base.

`openLocalStorage` fonction

```
async function openLocalStorage(name = 'portolan'): Promise<LocalStorage>
```

Ouvre (et crée au premier lancement, version 1 du schéma de base) la base IndexedDB et rend son interface d'accès. Appelée une fois au démarrage par `src/main.tsx` ; les tests l'appellent avec `fake-indexeddb` et un nom de base propre.

Paramètres

`name` facultatif : nom de la base (`portolan` par défaut).

Résultat. Promesse de `LocalStorage`.

Erreurs. Rejette la promesse si IndexedDB est indisponible ou refusé (certaines navigations privées) ; l'application démarre alors sans brouillon.

26.12 `io/svg`

Nettoyage des images SVG importées (NF-06) : scripts, gestionnaires d'événements, objets étrangers et références externes sont retirés avant stockage et affichage, à l'import comme à la lecture d'un fichier. Calcule aussi les proportions d'un SVG. Dépend de DOMPurify. Utilisé par `io/images`, `io/archive` et l'interface (éditeur de machine, annotations d'image).

`SvgSanitizer` type

```
type SvgSanitizer = (svg: string) => string;
```

Fonction de nettoyage : reçoit le texte d'un SVG et rend le texte nettoyé. Injectée dans la lecture d'archive et la préparation d'images pour pouvoir utiliser jsdom dans les tests.

`createSvgSanitizer` fonction

```
function createSvgSanitizer(window: WindowLike): SvgSanitizer
```

Crée un nettoyeur lié à une fenêtre DOM. Configuration de DOMPurify : profils SVG et filtres SVG ; `script`, `foreignObject`, `iframe`, `object`, `embed` interdits ; `use` autorisé. Un crochet ne garde `href`, `xlink:href` et `src` que pour une référence interne (`#id`) ou une image matricielle embarquée en base 64 (PNG, JPEG, GIF, WebP) ; un autre retire des attributs `style` et des éléments `<style>` les `url(…)` externes et les `@import` ; une référence interne (`url(#id)`, avec ou sans guillemets) est conservée (voir chapitre 10).

Paramètres

`window` fenêtre du navigateur, ou fenêtre jsdom dans les tests.

Résultat. Un `SvgSanitizer`.

Erreurs. Aucune.

```
const sanitize = createSvgSanitizer(new JSDOM('').window);
sanitize('<svg onload="alert(1)"><script>x()</script><rect/></svg>');
// '<svg><rect></rect></svg>'
```

`sanitizeSvg` fonction

```
function sanitizeSvg(svg: string): string
```

Nettoyeur par défaut, lié à la fenêtre courante (créé à la première utilisation). Valeur par défaut des paramètres `sanitizer` de `prepareImage` et de la lecture d'archive.

Paramètres

`svg` texte du SVG.

Résultat. Le texte nettoyé.

Erreurs. Lève une `Error` hors navigateur (pas de `window`) : les tests doivent passer leur propre nettoyeur.

`svgAspect` fonction

```
function svgAspect(svg: string): number | null
```

Proportions (largeur / hauteur) d'un SVG, d'après les attributs `width` et `height` de son élément racine s'ils sont des longueurs absolues (pas de pourcentage), sinon d'après sa `viewBox`. Un navigateur ne donne pas toujours la taille d'une telle image ; l'éditeur de machine (logo, EDM-10) et les annotations d'image s'en servent pour l'ajuster sans déformation.

Paramètres

`svg` texte du SVG.

Résultat. Le rapport largeur / hauteur, ou `null` si aucune taille positive n'est trouvée.

Erreurs. Aucune.

```
svgAspect('<svg width="200" height="50">…</svg>');  // 4
svgAspect('<svg viewBox="0 0 16 9">…</svg>');       // 1.777…
svgAspect('<svg width="100%" height="100%">');       // null
```

Chapitre 27

Référence : exports

Ce chapitre documente les modules de `src/export/`, dans l'ordre alphabétique de leurs fichiers. Le fonctionnement d'ensemble (chaîne SVG → PNG / PDF, tableur, étiquettes) est décrit au chapitre « Exports ». Les longueurs sont en millimètres de papier sauf mention contraire.

27.1 `export/appFonts`

Sources des polices de l'application pour les exports SVG : relie `export/fonts`, qui ne connaît que l'interface `FontSources`, aux fichiers réels déclarés par `text/fontFiles` (sous-ensembles WOFF2 de l'écran, Inter TrueType, plages des symboles). Séparé pour que `export/fonts` reste pur et testable avec des sources factices. Utilisé par défaut par `buildImageExport` (EXP-02, NF-07b).

`APP_FONT_SOURCES` constante

```
const APP_FONT_SOURCES: FontSources
```

Sources des polices utilisées par les exports SVG et PNG de l'application ; valeur par défaut du paramètre `fonts` de `buildImageExport`.

Champs

`webFonts` les fichiers WOFF2 de l'écran (`WEB_FONT_FILES`), un par écriture et par graisse ;

`symbols` plages et `unicode-range` des symboles absents de ces fichiers, et `trueTypeUrl`, l'URL d'Inter TrueType d'une graisse ;

`load` `fontBytes` : télécharge (depuis l'application elle-même) et met en cache les octets d'un fichier de police.

27.2 `export/csv`

Écriture des fichiers CSV des exports (EXP-04, EXP-07) : UTF-8 avec BOM, fins de ligne CRLF, guillemets selon RFC 4180, virgule décimale avec le séparateur « ; ». Pur, sans dépendance. Utilisé par `export/spreadsheet` (listes), `export/labels` (CSV pour imprimante d'étiquettes) et par le modèle de classeur de l'import (`src/import/template.ts`).

`CsvSeparator` type

```
type CsvSeparator = ';' | ',';
```

Séparateur de champs choisi dans la fenêtre d'export.

Champs

- `';'` point-virgule, usage des tableurs en français ; les nombres s'écrivent alors avec la virgule décimale ;
- `','` virgule ; les nombres gardent le point décimal.

`CsvCell` type

```
type CsvCell = string | number | null;
```

Valeur d'une cellule CSV : texte, nombre (écrit selon le séparateur) ou `null` (cellule vide).

`csvValue` fonction

```
function csvValue(value: CsvCell, separator: CsvSeparator): string
```

Texte d'une cellule dans une ligne CSV. Un nombre est converti par `String` ; avec le séparateur « ; » son point décimal devient une virgule. Le texte est entouré de guillemets (guillemets internes doublés) s'il contient le séparateur, un guillemet, un retour chariot ou un saut de ligne.

Paramètres

- `value` la cellule ;
- `separator` le séparateur du fichier.

Résultat. Le texte de la cellule, vide pour `null`.

Erreurs. Aucune.

```
csvValue(1.5, ';');            // '1,5'
csvValue('Baie 1; avant', ';'); // '"Baie 1; avant"'
csvValue('dit "A"', ',');       // '"dit ""A"""'
```

`csvCells` fonction

```
function csvCells(rows: readonly (readonly CsvCell[])[], separator: CsvSeparator): string
```

Texte complet d'un fichier CSV : BOM UTF-8, une ligne par rangée (la première est l'en-tête), cellules séparées par `separator`, chaque ligne terminée par CRLF (la dernière aussi).

Paramètres

- `rows` les rangées de cellules, en-tête compris ;
- `separator` le séparateur.

Résultat. Le texte du fichier, à encoder en UTF-8 (`TextEncoder`).

Erreurs. Aucune.

27.3 `export/encoding`

Conversions de bas niveau des fichiers exportés : Base64, URL `data:`, décodage des entités XML, noms de fichiers sûrs. Pur. Utilisé par les exports SVG (images et polices intégrées), PDF (polices passées à jsPDF), les noms de fichiers, l'export de bibliothèque (`ExportLibraryDialog`) et plusieurs tests de rendu.

`toBase64` fonction

```
function toBase64(bytes: Uint8Array): string
```

Encode des octets en Base64. Les octets sont convertis en chaîne binaire par blocs de 32 768 (`0x8000`) pour ne pas dépasser le nombre maximal d'arguments de `String.fromCharCode` sur une grande image ou une police.

Paramètres

`bytes` les octets.

Résultat. La chaîne Base64 (alphabet standard, avec remplissage).

Erreurs. Aucune.

`dataUrl` fonction

```
function dataUrl(bytes: Uint8Array, mimeType: string): string
```

URL `data:` d'un fichier, utilisée pour intégrer images et polices dans un SVG exporté.

Paramètres

`bytes` le contenu du fichier ;

`mimeType` son type MIME (`image/png`, `font/woff2`…).

Résultat. `data:<type>; base64,<données>`.

Erreurs. Aucune.

`decodeXmlText` fonction

```
function decodeXmlText(text: string): string
```

Décode les entités d'un texte XML : références numériques décimales (`é`) et hexadécimales (`é`), entités nommées `amp`, `lt`, `gt`, `quot`, `apos`, `nbsp`. Une entité nommée inconnue est laissée telle quelle. Sert à `textUsage` pour connaître les caractères réellement dessinés.

Paramètres

`text` le contenu textuel d'un élément, tel que sérialisé.

Résultat. Le texte décodé.

Erreurs. Une référence numérique hors de l'espace Unicode lève la `RangeError` de `String.fromCodePoint` ; React n'en produit pas.

`safeFileName` fonction

```
function safeFileName(name: string): string
```

Rend un nom utilisable comme nom de fichier sur tous les systèmes : les caractères `\ / : * ? " < > |` deviennent des tirets, les suites de blancs une espace, les blancs de début et de fin sont retirés.

Paramètres

`name` le nom saisi (projet, indice, liste, vue…).

Résultat. Le nom nettoyé, éventuellement vide.

Erreurs. Aucune.

```
safeFileName('Salle A/B : plan  ');   // 'Salle A-B - plan'
```

27.4 `export/fileNames`

Noms proposés pour les fichiers exportés, de la forme `<projet>_<indice>` (EXP-01). Dépend de `export/encoding` et de l'i18n (nom par défaut). Utilisé par tous les exports (image, PDF, tableur, étiquettes).

`exportBaseName` fonction

```
function exportBaseName(project: Project): string
```

Base commune des noms de fichiers : nom du projet, suivi de _ et de l'indice de la dernière révision du cartouche s'il y en a une. Les deux parties passent par `safeFileName` ; un projet sans nom prend le nom par défaut (`defaults.fileName`).

Paramètres

`project` le projet.

Résultat. La base du nom, sans extension.

Erreurs. Aucune.

`exportFileName` fonction

```
function exportFileName(project: Project, extension: string, suffix?: string): string
```

Nom complet d'un fichier exporté : base (`exportBaseName`), suffixe facultatif précédé de _, extension.

Paramètres

`project` le projet ;

`extension` l'extension sans point (`pdf`, `svg`…) ;

`suffix` facultatif : précision ajoutée au nom (`page-2`, nom de la vue, `etiquettes`), nettoyée par `safeFileName` ; ignorée si elle devient vide.

Résultat. Le nom de fichier proposé au téléchargement.

Erreurs. Aucune.

```
exportFileName(project, 'png', 'Baie 1-avant'); // 'Salle A_B_Baie 1-avant.png'
```

27.5 `export/fonts`

Polices intégrées dans un SVG exporté (EXP-02, NF-07, NF-07b) : les mêmes fichiers Inter que l'écran, seulement pour les graisses et les écritures utilisées, plus un sous-ensemble d'Inter TrueType pour les symboles absents des fichiers de l'écran. Pur : les fichiers sont fournis par un `FontSources`. Dépend de `text/sfnt` (sous-ensemble TrueType), `text/unicodeRange` et `export/encoding`. Utilisé par `export/svgExport`, et `textUsage` par `export/pdfExport` (graisses à charger).

`textUsage` fonction

```
function textUsage(svg: string): Map<FontWeight, Set<number>>
```

Relève les caractères dessinés par graisse : pour chaque élément `<text>` du balisage, la graisse de son attribut `font-weight` (400 sans attribut), le texte sans balises internes et entités décodées, ajouté à l'ensemble des points de code de cette graisse. Analyse par expressions régulières : suppose des `<text>` non imbriqués, la graisse portée par l'élément `<text>` lui-même.

Paramètres

`svg` le balisage SVG.

Résultat. Une table graisse → ensemble de points de code ; vide sans texte.

Erreurs. Aucune.

`FontSources` interface

```
interface FontSources { webFonts; symbols; load }
```

Fichiers de police disponibles pour l'intégration, injectés pour tester `embeddedFontCss` sans réseau. Valeur de l'application : `APP_FONT_SOURCES`.

Champs

`webFonts` fichiers WOFF2 de l'écran : graisse, plages couvertes (`ranges`), règle CSS `unicodeRange` correspondante, URL ;

`symbols` symboles absents de ces fichiers : plages, `unicodeRange`, et `trueTypeUrl(graisse)` qui donne l'URL d'Inter TrueType de la graisse ;

`load` chargement asynchrone des octets d'une URL.

`embeddedFontCss` fonction

```
async function embeddedFontCss(svg: string, sources: FontSources): Promise<string>
```

Construit les règles `@font-face` (famille Inter, URL `data:`) nécessaires au texte d'un SVG, par graisse croissante : chaque fichier WOFF2 de la graisse qui couvre au moins un caractère utilisé, intégré entier avec son `unicode-range` ; puis, si des symboles sont utilisés, un sous-ensemble d'Inter TrueType de la graisse réduit à ces glyphes (`subsetFont`, tables de mise en forme retirées). Appelée par `svgFile`.

Paramètres

`svg` le balisage SVG (sans polices) ;

`sources` les fichiers disponibles.

Résultat. Le texte CSS, une règle par ligne ; vide si le dessin n'a pas de texte.

Erreurs. Rejette la promesse si un fichier ne peut être chargé (`sources.load`) ou si le fichier TrueType est invalide (`subsetFont`).

27.6 `export/imageExport`

Export d'une page ou d'une vue en SVG ou PNG (EXP-02, EXP-03, EXP-05) : choisit le dessin, intègre les polices, rastérise si besoin et nomme le fichier. Dépend de `export/svgExport`, `export/png`, `export/fileNames`, du routage (`loadAvoid`). Utilisé par la fenêtre `ExportImageDialog` (commandes `export.svg` et `export.png`). Réexporte aussi `ExportError` et `ExportedFile` pour les fenêtres.

`ImageFormat` type

```
type ImageFormat = 'svg' | 'png';
```

Format d'image demandé.

Champs

- `'svg'` dessin vectoriel autonome, polices intégrées ;
- `'png'` image matricielle, résolution et fond choisis.

`ImageExportRequest` interface

```
interface ImageExportRequest { format; pageId; viewId; parts; hiddenLinkTypes; png }
```

Options de la fenêtre d'export d'image.

Champs

- `format` `'svg'` ou `'png'` ;
- `pageId` la page dessinée ;
- `viewId` la vue exportée seule (face d'armoire ou vue extérieure), ou `null` pour la page entière ;
- `parts` parties de la page à dessiner (cadre, cartouche, légende, repères de câbles, grille ; EXP-05) ; le champ `background` est recalculé ;
- `hiddenLinkTypes` types de liaisons non dessinés (LIA-10) ;
- `png` pour le PNG : `resolution` (`PngResolution`) et `background` (`PngBackground`) ; ignoré pour le SVG.

`exportDrawing` fonction

```
function exportDrawing(project: Project, images: ImageMap, request:
ImageExportRequest, measure: MeasureText): SvgDrawing
```

Dessin à exporter, sans polices : la page entière (`pageSvg`) ou la vue recadrée (`viewSvg`). La partie `background` (feuille blanche) est mise à vrai pour un SVG et pour un PNG à fond blanc, à faux pour un PNG transparent. Appelée par `buildImageExport` ; la fenêtre s'en sert aussi pour sa taille.

Paramètres

- `project` le projet ;

`images` les images du projet ;

`request` la demande ;

`measure` la mesure du texte (police chargée).

Résultat. Le `SvgDrawing` de la page ou de la vue.

Erreurs. `ExportError` si la page n'existe plus (`export.noPage`), ou si la vue n'existe plus ou n'a pas de rectangle (`export.noView`).

`buildImageExport` fonction

```
async function buildImageExport(state: { project: Project; images: ImageMap }, request:
ImageExportRequest, measure: MeasureText, fonts: FontSources = APP_FONT_SOURCES):
Promise<ExportedFile>
```

Construit le fichier SVG ou PNG prêt à télécharger : charge libavoid (échec ignoré : lignes droites), dessine (`exportDrawing`), produit le SVG autonome (`svgFile`) puis, pour le PNG, calcule la taille (`pngPixelSize`) et rastérise (`rasterizeSvg`). Le nom porte le suffixe `page-<n>`, `<armoire>-avant` / `<armoire>-arrière`, ou le nom de l'équipement d'une vue extérieure.

Paramètres

`state` le projet et ses images ;

`request` la demande (`ImageExportRequest`) ;

`measure` la mesure du texte ;

`fonts` sources des polices à intégrer (par défaut `APP_FONT_SOURCES`).

Résultat. Un `ExportedFile` de type `image/svg+xml` ou `image/png`.

Erreurs. `ExportError` : page ou vue disparue, ou PNG trop grand (`export.tooLarge`). Rejette aussi si une police ne se charge pas ou si la rastérisation échoue (`rasterizeSvg`).

```
const file = await buildImageExport(
  { project, images },
  { format: 'png', pageId, viewId: null, parts, hiddenLinkTypes: new Set(),
    png: { resolution: { kind: 'dpi', dpi: 150 }, background: 'white' } },
  measure,
);
```

27.7 `export/labels`

Étiquettes de câbles (EXP-07, REC-14), partie pure : contenu des étiquettes (une par extrémité, équipement final en option à travers les panneaux de brassage, LIA-09), gabarits de planches A4/A3, placement, ajustement du texte et CSV pour imprimante d'étiquettes. Dépend de `model/links` (`endToEndPath`, `portLabel`), `text/layout` et `export/csv`. Utilisé par `export/labelsExport`, le composant `LabelSheet` (`render/LabelSheet`) et la fenêtre `ExportLabelsDialog`.

`LabelPaper` type

```
type LabelPaper = 'A4' | 'A3';
```

Format de feuille des planches : `'A4'` ou `'A3'`, toujours en portrait.

`LABEL_PAPERS` constante

```
const LABEL_PAPERS: Record<LabelPaper, { width: number; height: number }>
```

Dimensions des feuilles en millimètres, en portrait : A4 = 210 × 297, A3 = 297 × 420. Utilisé par `templateProblem` et pour la taille des pages du PDF.

Champs

`A4` `{ width: 210, height: 297 }`;

`A3` `{ width: 297, height: 420 }`.

`LabelTemplate` interface

```
interface LabelTemplate { paper; labelWidth; labelHeight; columns; rows; marginTop; marginLeft; gapX; gapY }
```

Gabarit d'une planche d'étiquettes (feuille en portrait), en millimètres, réglé dans la fenêtre.

Champs

`paper` format de la feuille ;

`labelWidth, labelHeight` taille d'une étiquette (> 0) ;

`columns, rows` nombre d'étiquettes par rangée et de rangées (> 0) ;

`marginTop, marginLeft` position de la première étiquette depuis le coin haut gauche de la feuille ;

`gapX, gapY` espace horizontal et vertical entre deux étiquettes voisines.

`DEFAULT_LABEL_TEMPLATE` constante

```
const DEFAULT_LABEL_TEMPLATE: LabelTemplate
```

Gabarit proposé par défaut : planche A4 courante de 3 colonnes × 8 rangées d'étiquettes de 63,5 × 33,9 mm, marges haute 12,9 mm et gauche 7,2 mm, espace horizontal 2,5 mm, sans espace vertical.

`CableLabel` interface

```
interface CableLabel { link; end; ref; here; there; final }
```

Contenu d'une étiquette, avant mise en page.

Champs

`link` la liaison étiquetée ;

`end` extrémité sur laquelle l'étiquette est collée, `'a'` ou `'b'` ;

`ref` repère de la liaison (éventuellement vide) ;

`here` texte de l'extrémité locale (`portLabel` : équipement · port) ;

`there` texte de l'autre extrémité ;

`final` équipements finaux, `here` du côté de cette extrémité et `there` de l'autre, ou `null` si l'option est désactivée ou si la liaison ne traverse aucun panneau de brassage.

`cableLabels` fonction

```
function cableLabels(project: Project, linkIds: readonly Id[], options: { showFinal:
boolean }): CableLabel[]
```

Étiquettes des liaisons, dans l'ordre donné : extrémité A puis extrémité B de chaque liaison. Avec `showFinal`, calcule le chemin de bout en bout (`endToEndPath`) ; s'il compte plus d'un saut, les extrémités finales sont orientées comme la liaison (si le saut de cette liaison part de son extrémité B, elles sont échangées).

Paramètres

`project` le projet ;

`linkIds` les liaisons à étiqueter ; un identifiant inconnu est ignoré ;

`options` `showFinal` : ajouter les équipements finaux.

Résultat. Deux étiquettes par liaison trouvée.

Erreurs. Aucune.

`templateProblem` fonction

```
function templateProblem(template: LabelTemplate): string | null
```

Vérifie un gabarit : dimensions, colonnes et rangées strictement positives, produit colonnes × rangées entier, et planche contenue dans la feuille (marge + étiquettes + espaces, à 10^{-6} mm près). Appelée par la fenêtre (bouton PDF désactivé) et par `labelSheetDrawings`.

Paramètres

`template` le gabarit.

Résultat. Le message traduit du problème (`labels.invalid` ou `labels.tooLarge`), ou `null` si le gabarit convient.

Erreurs. Aucune.

`PlacedLabel` interface

```
interface PlacedLabel { label; x; y }
```

Étiquette placée sur une planche.

Champs

`label` son contenu ;

`x, y` coin haut gauche de l'étiquette sur la feuille, en millimètres.

`labelSheets` fonction

```
function labelSheets(labels: readonly CableLabel[], template: LabelTemplate):
PlacedLabel[][]
```

Répartit les étiquettes sur des planches : emplacements remplis ligne par ligne depuis le coin haut gauche, nouvelle planche quand la précédente est pleine. Le gabarit doit avoir été validé par `templateProblem`.

Paramètres

`labels` les étiquettes, dans l'ordre ;

`template` le gabarit.

Résultat. Une liste de planches, chacune une liste de `PlacedLabel` ; vide sans étiquette.

Erreurs. Aucune.

`LABEL_PADDING` constante

```
const LABEL_PADDING = 1.8
```

Marge intérieure d'une étiquette, 1,8 mm de chaque côté : réduit la zone de texte dans `fitLabel` et décale le texte dans `LabelSheet`.

`LabelLine` interface

```
interface LabelLine { text; font }
```

Ligne logique d'une étiquette, avant découpage à la largeur.

Champs

- `text` le texte ;
- `font` taille (mm) et graisse.

`labelLines` fonction

```
function labelLines(label: CableLabel, size: number): LabelLine[]
```

Lignes d'une étiquette pour une taille de repère donnée : repère (ou « (sans repère) ») en graisse 700 à `size` ; extrémité locale en 600 à $0{,}72\times$ `size` ; « → autre extrémité » en 400 à $0{,}72\times$ `size` ; si présente, « Final : ici ↔ là-bas » en 400 à $0{,}62\times$ `size`.

Paramètres

- `label` l'étiquette ;
- `size` taille du repère, en millimètres.

Résultat. Trois ou quatre lignes.

Erreurs. Aucune.

`FittedLine` interface

```
interface FittedLine { text; font; y }
```

Ligne de texte d'une étiquette après ajustement.

Champs

- `text` le texte de la ligne (une partie d'une ligne logique découpée) ;
- `font` sa police ;
- `y` haut de la boîte de la ligne depuis le haut de la zone de texte (marge intérieure non comprise), en millimètres.

`fitLabel` fonction

```
function fitLabel(label: CableLabel, template: Pick<LabelTemplate, 'labelWidth'
| 'labelHeight'>, measure: MeasureText): FittedLine[]
```

Met en page le texte d'une étiquette à la plus grande taille qui tient : part de min$(5, h/4)$ (h : hauteur utile), découpe chaque ligne à la largeur utile (`layoutText`), interligne de 1,22 fois la taille ; si la hauteur totale dépasse, réduit de 0,2 mm et recommence. Quand la taille suivante passerait sous 1,4 mm, garde les lignes qui tiennent et coupe les autres. Appelée par `LabelSheet`.

Paramètres

`label` l'étiquette ;

`template` la taille de l'étiquette (`labelWidth`, `labelHeight`) ;

`measure` la mesure du texte.

Résultat. Les lignes placées, de haut en bas.

Erreurs. Aucune.

`labelsCsv` fonction

```
function labelsCsv(labels: readonly CableLabel[], separator: CsvSeparator): string
```

CSV des étiquettes pour une imprimante d'étiquettes : en-tête traduit (repère, extrémité, ici, autre extrémité, final ici, final autre extrémité), puis une ligne par étiquette ; l'extrémité est écrite `A` ou `B`, les colonnes finales sont vides sans équipement final. Format de `csvCells`.

Paramètres

`labels` les étiquettes ;

`separator` le séparateur.

Résultat. Le texte du fichier CSV.

Erreurs. Aucune.

27.8 `export/labelsExport`

Fichiers des étiquettes de câbles (EXP-07) : planches PDF et CSV. Rend chaque planche avec le composant `LabelSheet` (`render/LabelSheet`) puis la convertit par `drawingsToPdf`. Dépend de `export/labels`. Utilisé par la fenêtre `ExportLabelsDialog` (commande `export.labels`).

`LabelsRequest` interface

```
interface LabelsRequest { linkIds; template; showFinal; outlines }
```

Options de la fenêtre des étiquettes.

Champs

`linkIds` liaisons à étiqueter, dans l'ordre voulu ;

`template` gabarit de planche ;

`showFinal` ajouter les équipements finaux ;

`outlines` tracer le contour gris clair de chaque étiquette (papier ordinaire).

`labelSheetDrawings` fonction

```
function labelSheetDrawings(project: Project, request: LabelsRequest, measure:
MeasureText): SvgDrawing[]
```

Dessine chaque planche en SVG, à la taille de la feuille : vérifie le gabarit, calcule les étiquettes et les répartit, puis rend `LabelSheet` hors du document.

Paramètres

- `project` le projet ;
- `request` la demande ;
- `measure` la mesure du texte.

Résultat. Un `SvgDrawing` par planche.

Erreurs. `ExportError` si le gabarit est invalide (message de `templateProblem`) ou s'il n'y a aucune étiquette (`labels.none`).

`buildLabelsPdf` fonction

```
async function buildLabelsPdf(project: Project, request: LabelsRequest, measure:
MeasureText, deps?: Pick<PdfDependencies, 'loadFont'>): Promise<ExportedFile>
```

PDF des planches d'étiquettes, une page par planche, titré « <projet> — Étiquettes de câbles », nommé `<projet>_<indice>_etiquettes.pdf`.

Paramètres

- `project` le projet ;
- `request` la demande ;
- `measure` la mesure du texte ;
- `deps` facultatif : chargement des polices (tests) ; par défaut celui de l'application.

Résultat. Un `ExportedFile` `application/pdf`.

Erreurs. Celles de `labelSheetDrawings` ; rejette si une police ne se charge pas ou si la conversion échoue.

`buildLabelsCsv` fonction

```
function buildLabelsCsv(project: Project, request: Pick<LabelsRequest, 'linkIds'
| 'showFinal'>, separator: CsvSeparator): ExportedFile
```

CSV des étiquettes (`labelsCsv`), nommé `<projet>_<indice>_etiquettes.csv`. N'utilise pas le gabarit : possible même quand il est refusé.

Paramètres

- `project` le projet ;
- `request` `linkIds` et `showFinal` ;
- `separator` le séparateur.

Résultat. Un `ExportedFile` `text/csv`.

Erreurs. `ExportError` s'il n'y a aucune étiquette.

27.9 `export/pdfExport`

PDF généré par l'application (EXP-01, EXP-05, NF-07) : les pages dessinées comme à l'écran (`pageSvg`), converties en PDF vectoriel par svg2pdf.js dans un document jsPDF, Inter TrueType intégrée (sous-ensemble, texte sélectionnable), chaque page à son format, images converties une seule fois (`export/pdfImages`). jsPDF et svg2pdf.js sont chargés à la demande. Utilisé par la fenêtre `ExportPdfDialog` (commande `export.pdf`, `Ctrl+P`) et par `export/labelsExport`.

`PdfExportRequest` interface

```
interface PdfExportRequest { pageIds; parts; hiddenLinkTypes; jpegQuality }
```

Options de la fenêtre d'export PDF.

Champs

`pageIds` pages à exporter ; l'ordre du PDF reste celui du projet ;

`parts` parties de la page à dessiner (EXP-05) ; le fond est toujours dessiné ;

`hiddenLinkTypes` types de liaisons non dessinés ;

`jpegQuality` qualité des images JPEG, de 0,1 à 1 (la fenêtre propose 0,6, 0,75, 0,85 et 0,95).

`pdfFontStyle` fonction

```
function pdfFontStyle(weight: FontWeight): string
```

Nom de style sous lequel jsPDF enregistre une graisse d'Inter ajoutée avec le style `normal`, et sous lequel svg2pdf.js la cherche : `normal` pour 400, `bold` pour 700, `<graisse>normal` sinon (`500normal`). Sert à vérifier l'enregistrement dans `drawingsToPdf`.

Paramètres

`weight` la graisse (400, 500, 600 ou 700).

Résultat. Le nom du style.

Erreurs. Aucune.

`pagesToExport` fonction

```
function pagesToExport(project: Project, pageIds: readonly Id[])
```

Pages du projet dont l'identifiant est demandé, dans l'ordre du projet (pas dans celui de la demande). Les identifiants inconnus sont ignorés.

Paramètres

`project` le projet ;

`pageIds` les pages cochées.

Résultat. Les pages, éventuellement aucune.

Erreurs. Aucune.

`imagePlaceholders` fonction

```
function imagePlaceholders(markup: string): ImageRef[]
```

Références des images d'un dessin, sans doublon. Dans le dessin, chaque image a pour URL provisoire un repère `urn:portolan-image:<ref>`, posé par `buildPdfExport` et remplacé après conversion.

Paramètres

`markup` le balisage SVG.

Résultat. Les références d'images, dans l'ordre de première apparition.

Erreurs. Aucune.

`PdfDependencies` interface

```
interface PdfDependencies { loadFont; convertImage }
```

Accès au navigateur et aux fichiers dont dépend la construction du PDF, injectables pour les tests sous jsdom.

Champs

`loadFont` octets d'Inter TrueType d'une graisse ;

`convertImage` conversion d'une image du projet en PNG ou JPEG (URL `data:`), à la qualité JPEG donnée ; par défaut `convertImage`.

`buildPdfExport` fonction

```
async function buildPdfExport(state: { project: Project; images: ImageMap }, request:
PdfExportRequest, measure: MeasureText, deps: PdfDependencies = APP_PDF_DEPENDENCIES):
Promise<ExportedFile>
```

Construit le PDF des pages choisies : sélection (`pagesToExport`), chargement de libavoid (échec ignoré), dessin de chaque page avec fond, images en repères, conversion unique de chaque image utilisée, remplacement des repères, assemblage par `drawingsToPdf`. Le titre du document est le nom du projet ; le fichier s'appelle `<projet>_<indice>.pdf`.

Paramètres

`state` le projet et ses images ;

`request` la demande ;

`measure` la mesure du texte ;

`deps` facultatif : polices et conversion d'images (par défaut celles de l'application).

Résultat. Un `ExportedFile` `application/pdf`.

Erreurs. `ExportError` si aucune page n'est retenue (`export.noPageSelected`) ; rejette si une image ne peut être décodée, si une police ne se charge pas ou n'est pas enregistrée par jsPDF.

```
const file = await buildPdfExport(
  { project, images },
  { pageIds: project.pages.map((p) => p.id), parts: SCREEN_PARTS,
    hiddenLinkTypes: new Set(), jpegQuality: 0.85 },
  measure,
);
```

`drawingsToPdf` fonction

```
async function drawingsToPdf(drawings: readonly SvgDrawing[], title: string, deps:
Pick<PdfDependencies, 'loadFont'> = APP_PDF_DEPENDENCIES): Promise<Uint8Array>
```

Assemble un PDF à partir de dessins SVG, une page par dessin à sa propre taille (en millimètres, orientation déduite) : charge jsPDF et svg2pdf.js, ajoute pour chaque graisse utilisée dans l'ensemble des dessins Inter TrueType avec l'encodage `Identity-H` (jsPDF ne garde que les glyphes dessinés), puis convertit chaque SVG après l'avoir attaché au document dans un hôte invisible. Fonction commune aux pages (`buildPdfExport`) et aux étiquettes (`buildLabelsPdf`). Navigateur (ou jsdom outillé) uniquement.

Paramètres

`drawings` les dessins, au moins un ; les images doivent déjà être en URL `data:` ;

`title` titre du document (propriétés du PDF) ;

`deps` facultatif : `loadFont`.

Résultat. Les octets du PDF compressé.

Erreurs. Lève `Error('Font <graisse> not registered')` si jsPDF n'enregistre pas une graisse sous le style attendu ; rejette si une police ne se charge pas. Une liste vide n'est pas prévue (lecture de `drawings[0]`) : les appelants la refusent avant.

`pageFormats` fonction

```
function pageFormats(project: Project)
```

Taille de chaque page du projet (`pageSize`), pour afficher les formats dans la fenêtre d'export PDF.

Paramètres

`project` le projet.

Résultat. Pour chaque page, dans l'ordre : `{ page, width, height }` en millimètres.

Erreurs. Aucune.

27.10 `export/pdfImages`

Images du PDF (EXP-01) : chaque image de façade est redessinée sur un canevas et convertie en PNG si elle a de la transparence, en JPEG à qualité réglable sinon, réduite si elle est trop grande. Les calculs de taille et de transparence sont purs ; `convertImage` a besoin du navigateur. Utilisé par `export/pdfExport`.

`MAX_IMAGE_SIDE` constante

```
const MAX_IMAGE_SIDE = 2400
```

Plus grand côté d'une image convertie : 2 400 px, soit environ 300 ppp sur une grande vue imprimée ; les images plus grandes sont réduites, jamais agrandies.

`SVG_RASTER_WIDTH` constante

```
const SVG_RASTER_WIDTH = 1600
```

Largeur, 1600 px, à laquelle une image vectorielle (SVG) est dessinée avant conversion, à ses proportions.

`hasTransparency` fonction

```
function hasTransparency(rgba: ArrayLike<number>): boolean
```

Indique si un tampon RGBA contient un pixel qui n'est pas totalement opaque (alpha inférieur à 255).

Paramètres

- `rgba` les composantes, quatre par pixel (`ImageData.data`).

Résultat. `true` dès le premier pixel non opaque.

Erreurs. Aucune.

`rasterSize` fonction

```
function rasterSize(width: number, height: number, max = MAX_IMAGE_SIDE): { width: number; height: number }
```

Taille de l'image convertie : la taille naturelle, réduite proportionnellement pour que le plus grand côté ne dépasse pas `max`, arrondie, au moins 1 px de chaque côté.

Paramètres

- `width, height` taille naturelle en pixels ;
- `max` plus grand côté permis (par défaut `MAX_IMAGE_SIDE`).

Résultat. `{ width, height }` en pixels.

Erreurs. Aucune.

`ConvertedImage` interface

```
interface ConvertedImage { dataUrl; format }
```

Image convertie pour le PDF.

Champs

- `dataUrl` l'image en URL `data:` ;
- `format` `'png'` (transparence) ou `'jpeg'`.

`convertImage` fonction

```
async function convertImage(bytes: Uint8Array, mimeType: string, jpegQuality: number): Promise<ConvertedImage>
```

Convertit une image du projet : la décode (URL `blob:` temporaire, libérée ensuite), fixe sa taille (`SVG_RASTER_WIDTH` pour un SVG, `rasterSize`), la dessine sur un canevas puis l'encode en PNG si elle a de la transparence, en JPEG sinon.

Paramètres

- `bytes` le contenu de l'image ;

`mimeType` son type MIME ;

`jpegQuality` qualité JPEG, de 0 à 1.

Résultat. Un `ConvertedImage`.

Erreurs. Rejette si l'image ne peut être décodée (`image.decode()`) ou si le contexte 2D du canevas est indisponible (`Error('Canvas unavailable')`).

27.11 `export/png`

PNG d'une page ou d'une vue (EXP-03) : taille en pixels selon une résolution ou une largeur, fond blanc ou transparent, résolution écrite dans le fichier (bloc `pHYs`). Les calculs et la manipulation des blocs PNG sont purs ; `rasterizeSvg` a besoin du navigateur. Utilisé par `export/imageExport` et par la fenêtre `ExportImageDialog` (aperçu de la taille).

`PngResolution` type

```
type PngResolution = { kind: 'dpi'; dpi: number } | { kind: 'width'; pixels: number };
```

Façon de fixer la taille du PNG.

Champs

`{ kind: 'dpi', dpi }` résolution en points par pouce ;

`{ kind: 'width', pixels }` largeur en pixels, la résolution s'en déduit.

`PixelSize` interface

```
interface PixelSize { width; height; dpi }
```

Taille d'un PNG.

Champs

`width, height` dimensions en pixels (au moins 1) ;

`dpi` résolution résultante en points par pouce, écrite dans le fichier.

`MAX_PNG_SIDE` constante

```
const MAX_PNG_SIDE = 16_384
```

Plus grand côté de canevas accepté par les navigateurs : 16 384 px. Au-delà, l'export PNG est refusé.

`MAX_PNG_AREA` constante

```
const MAX_PNG_AREA = 16_384 * 16_384
```

Plus grande surface de canevas acceptée : 16 384 × 16 384 pixels.

`pngPixelSize` fonction

```
function pngPixelSize(widthMm: number, heightMm: number, resolution: PngResolution):
PixelSize | null
```

Taille en pixels d'un dessin : ppp donnés, ou ppp = pixels × 25,4 / largeur en mm ; largeur et hauteur = mm × ppp / 25,4, arrondies, au moins 1.

Paramètres

`widthMm, heightMm` taille du dessin en millimètres ;

`resolution` la résolution demandée.

Résultat. La `PixelSize`, ou `null` si un côté dépasse `MAX_PNG_SIDE` ou la surface `MAX_PNG_AREA`.

Erreurs. Aucune.

```
pngPixelSize(297, 210, { kind: 'dpi', dpi: 150 });
// { width: 1754, height: 1240, dpi: 150 }
```

`crc32` fonction

```
function crc32(bytes: Uint8Array): number
```

CRC-32 (polynôme `0xEDB88320`, comme zlib) d'octets, par table précalculée ; sert à signer le bloc `pHYs`.

Paramètres

`bytes` les octets (type et données du bloc).

Résultat. Le CRC, entier non signé sur 32 bits.

Erreurs. Aucune.

`pngDimensions` fonction

```
function pngDimensions(bytes: Uint8Array): { width: number; height: number }
```

Largeur et hauteur d'un PNG, lues dans le bloc `IHDR`. Utilisé par les tests.

Paramètres

`bytes` le fichier PNG.

Résultat. `{ width, height }` en pixels.

Erreurs. Lève `Error('Not a PNG file')` si la signature PNG manque.

`pngDpi` fonction

```
function pngDpi(bytes: Uint8Array): number | null
```

Résolution écrite dans le bloc `pHYs` d'un PNG (unité mètre), convertie en points par pouce et arrondie ; la recherche s'arrête au premier bloc `IDAT`. Utilisé par les tests.

Paramètres

`bytes` le fichier PNG.

Résultat. Les ppp horizontaux, ou `null` sans bloc `pHYs` en mètres.

Erreurs. Aucune (un fichier qui n'est pas un PNG donne en général `null`).

`withPngDpi` fonction

```
function withPngDpi(bytes: Uint8Array, dpi: number): Uint8Array
```

Copie d'un PNG avec sa résolution enregistrée : un bloc `pHYs` (pixels par mètre, identiques en X et Y, unité mètre, CRC calculé) inséré juste après `IHDR` ; les blocs `pHYs` existants sont retirés, les autres recopiés.

Paramètres

`bytes` le PNG d'origine ;

`dpi` la résolution en points par pouce.

Résultat. Le nouveau fichier (l'original n'est pas modifié).

Erreurs. Lève `Error('Not a PNG file')` si la signature PNG manque.

`PngBackground` type

```
type PngBackground = 'white' | 'transparent';
```

Fond du PNG : `'white'` (feuille blanche peinte avant le dessin) ou `'transparent'` (ni feuille ni fond).

`rasterizeSvg` fonction

```
async function rasterizeSvg(svg: string, size: PixelSize, background: PngBackground):
Promise<Uint8Array>
```

Dessine un fichier SVG autonome (polices et images intégrées) dans un canevas de la taille voulue, l'encode en PNG et y écrit la résolution (`withPngDpi`). L'URL `blob:` temporaire est libérée dans tous les cas.

Paramètres

`svg` le texte du fichier SVG (`svgFile`) ;

`size` la taille et la résolution ;

`background` le fond.

Résultat. Les octets du PNG.

Erreurs. Rejette si le SVG ne se décode pas, si le contexte 2D est indisponible (`Canvas unavailable`) ou si l'encodage échoue (`PNG encoding failed`).

27.12 `export/spreadsheet`

Fichiers de l'export tableur (EXP-04, EXP-05) : un classeur XLSX (un onglet par liste, ExcelJS chargé à la demande) ou des fichiers CSV (UTF-8 avec BOM, « ; » ou « , »), réunis dans une archive ZIP (fflate) quand il y en a plusieurs. Les contenus viennent de `export/tables`. Utilisé par la fenêtre `ExportSpreadsheetDialog` (commande `export.spreadsheet`) ; `toXlsx` sert aussi au modèle de classeur de l'import (IMP-01) et `toCsv` aux tests de l'import. Réexporte le type `CsvSeparator`.

`toCsv` fonction

```
function toCsv(table: Table, separator: CsvSeparator): string
```

Texte CSV d'une liste : en-tête des libellés de colonnes, puis une ligne par ligne de la liste (cellule absente = vide), au format de `csvCells`.

Paramètres

`table` la liste ;

`separator` le séparateur.

Résultat. Le texte du fichier, BOM compris.

Erreurs. Aucune.

`sheetName` fonction

```
function sheetName(title: string): string
```

Nom d'onglet acceptable par les tableurs : caractères `[ ] : * ? / \` remplacés par des tirets, coupé à 31 caractères.

Paramètres

`title` le titre de la liste.

Résultat. Le nom de l'onglet.

Erreurs. Aucune.

`toXlsx` fonction

```
async function toXlsx(tables: readonly Table[], project: Project): Promise<Uint8Array>
```

Classeur XLSX, un onglet par liste : créateur = nom de l'application, titre = nom du projet ; ligne d'en-tête en gras, figée et munie d'un filtre automatique ; largeur de chaque colonne d'après son libellé et ses valeurs (8 à 50 caractères) ; nombres écrits comme nombres.

Paramètres

`tables` les listes, dans l'ordre des onglets ;

`project` le projet (titre du classeur).

Résultat. Les octets du fichier XLSX.

Erreurs. Rejette si ExcelJS ne peut être chargé ou si deux listes donnent le même nom d'onglet (refus d'ExcelJS ; impossible avec les titres actuels).

`SpreadsheetRequest` interface

```
interface SpreadsheetRequest { format; tables; columns; separator }
```

Options de la fenêtre d'export tableur.

Champs

`format` `'xlsx'` ou `'csv'` ;

`tables` listes à exporter, dans l'ordre ;

`columns` par liste, identifiants des colonnes gardées ; toutes si la liste n'y figure pas ;

`separator` séparateur CSV (ignoré en XLSX).

`buildSpreadsheetExport` fonction

```
async function buildSpreadsheetExport(project: Project, request: SpreadsheetRequest):
Promise<ExportedFile>
```

Construit le fichier de l'export tableur : listes demandées (`buildTable`) réduites aux colonnes choisies (`withColumns`), puis écrit le fichier :

- XLSX : `<projet>_<indice>.xlsx` ;
- CSV d'une seule liste : `<projet>_<indice>_<liste>.csv` ;
- CSV de plusieurs listes : `<projet>_<indice>_csv.zip`, un fichier `<liste>.csv` par liste.

Paramètres

`project` le projet ;

`request` la demande.

Résultat. Un `ExportedFile` (XLSX, `text/csv` ou `application/zip`).

Erreurs. Celles de `toXlsx`. Une demande sans liste n'est pas refusée ici (la fenêtre l'empêche) : en CSV elle donne une archive vide.

```
const file = await buildSpreadsheetExport(project, {
  format: 'csv', separator: ';', tables: ['cabling'],
  columns: { cabling: ['ref', 'aDevice', 'aPort', 'bDevice', 'bPort'] },
});
```

27.13 `export/svgExport`

SVG d'une page ou d'une vue (EXP-02, NF-07) : le dessin de l'écran (`PageContent` de `render/PageView`), rendu hors du document par `renderToStaticMarkup` dans un éditeur créé pour l'occasion, à 100 pixels par millimètre (ports détaillés), sans éléments interactifs ; puis, pour un fichier autonome, polices intégrées. Dépend du rendu, du routage (`routePage`, `loadedAvoid`), de `model/pageGeometry` et de `export/fonts`. Utilisé par `export/imageExport`, `export/pdfExport`, `export/labelsExport` (type), la fenêtre d'export d'image (cadrage) et de nombreux tests de rendu.

`DrawingOptions` interface

```
interface DrawingOptions { parts; hiddenLinkTypes }
```

Ce que dessine un export.

Champs

`parts` parties de la page (`PageParts`) : fond, cadre, cartouche, légende, repères de câbles, grille ;

`hiddenLinkTypes` types de liaisons non dessinés, comme à l'écran (LIA-10).

`SvgDrawing` interface

```
interface SvgDrawing { markup; widthMm; heightMm }
```

Dessin exporté, avant intégration des polices ; entrée commune de `svgFile`, `rasterizeSvg` (via le fichier) et `drawingsToPdf`.

Champs

`markup` l'élément `<svg>` sérialisé, sans polices intégrées ;

`widthMm, heightMm` taille du dessin en millimètres (format de la page, cadrage de la vue ou feuille d'étiquettes).

`VIEW_EXPORT_MARGIN` constante

```
const VIEW_EXPORT_MARGIN = 3
```

Marge ajoutée de chaque côté du cadrage d'une vue exportée seule : 3 mm.

`RenderOptions` interface

```
interface RenderOptions { measure; imageUrl? }
```

Options de rendu communes à `pageSvg` et `viewSvg`.

Champs

`measure` la mesure du texte (police chargée) ;

`imageUrl` facultatif : URL d'une image d'après sa référence ; par défaut une URL `data:` construite une fois par image ; le PDF y pose des repères `urn:portolan-image:`.

`pageSvg` fonction

```
function pageSvg(project: Project, images: ImageMap, page: Page, options: DrawingOptions, {
measure, imageUrl = imageResolver(images) }: RenderOptions): SvgDrawing
```

SVG d'une page entière à sa taille de papier, avec son numéro et le nombre de pages du projet (cartouche).

Paramètres

`project` le projet ;

`images` les images du projet ;

`page` la page ;

`options` parties et types masqués ;

`render` `measure` et `imageUrl` (`RenderOptions`).

Résultat. Le `SvgDrawing` de la page.

Erreurs. Aucune attendue ; une erreur de rendu React se propage.

```
const drawing = pageSvg(project, images, project.pages[0]!,
  { parts: SCREEN_PARTS, hiddenLinkTypes: new Set() }, { measure });
```

`viewOnlyPage` fonction

```
function viewOnlyPage(page: Page, view: View): Page
```

Copie de la page réduite à une vue : seule cette vue, sans les éléments libres du « Schéma » (ils appartiennent à la page, PAG-07). Le routage de cette page recalcule les câbles internes à la vue et transforme les autres en renvois.

Paramètres

`page` la page ;

`view` la vue gardée.

Résultat. Une nouvelle page (l'original n'est pas modifié).

Erreurs. Aucune.

`viewExportBox` fonction

```
function viewExportBox(project: Project, page: Page, view: View, options:
DrawingOptions, measure: MeasureText): Box | undefined
```

Rectangle de la page dessiné par l'export d'une vue : union du rectangle de la vue, de son étiquette (vue extérieure) ou de son titre et de sa colonne d'étiquettes (face d'armoire), des tracés de câbles et de faisceaux de la page réduite, des étiquettes de faisceaux, et, si les repères sont exportés, des repères, renvois et marqueurs ; élargie de `VIEW_EXPORT_MARGIN`, arrondie au centième de millimètre. Libavoid doit être chargé pour un cadrage fidèle (sinon tracés droits).

Paramètres

`project` le projet ;

`page` la page ;

`view` la vue ;

`options` parties et types masqués ;

`measure` la mesure du texte (largeur des repères et renvois).

Résultat. Le rectangle en millimètres de page, ou `undefined` si la vue n'a pas de rectangle (cible disparue).

Erreurs. Aucune.

`viewSvg` fonction

```
function viewSvg(project: Project, images: ImageMap, page: Page, view: View, options:
DrawingOptions, { measure, imageUrl = imageResolver(images) }: RenderOptions): SvgDrawing
| undefined
```

SVG d'une vue seule (face d'armoire ou équipement extérieur), recadré sur `viewExportBox`, sans cadre, cartouche, légende ni grille.

Paramètres

`project` le projet ;

`images` les images ;

`page` la page de la vue ;

`view` la vue ;

`options` parties et types masqués ;

`render` `measure` et `imageUrl`.

Résultat. Le `SvgDrawing` recadré, ou `undefined` si la vue n'a pas de rectangle.

Erreurs. Aucune attendue.

`svgFile` fonction

```
async function svgFile(drawing: SvgDrawing, fonts: FontSources | null): Promise<string>
```

Fichier SVG complet : déclaration XML, attribut `version="1.1"` sur l'élément racine et, si `fonts` est fourni, une feuille de style des polices (`embeddedFontCss`) insérée dans un `<defs>` en tête. Sans polices (`null`), le fichier compte sur une police Inter installée.

Paramètres

`drawing` le dessin ;

`fonts` les sources de polices, ou `null`.

Résultat. Le texte du fichier SVG, terminé par un saut de ligne.

Erreurs. Celles de `embeddedFontCss`.

27.14 `export/tables`

Listes de l'export tableur (EXP-04) : inventaire des équipements, carnet de câblage (avec les extrémités finales à travers les panneaux de brassage, LIA-09), occupation des armoires, ports et besoin en câbles. Pur : les mêmes tableaux alimentent XLSX et CSV, et la fenêtre en lit les colonnes. Dépend de `model/lookup`, `model/links`, `model/placement` et de l'i18n (titres et libellés `tables.*`). Utilisé par `export/spreadsheet`, la fenêtre d'export et l'import (type `Table`, tests).

`TableId` type

```
type TableId = 'inventory' | 'cabling' | 'occupation' | 'ports' | 'cableNeeds';
```

Identifiant d'une liste.

Champs

`'inventory'` inventaire des équipements ;

`'cabling'` carnet de câblage ;

`'occupation'` occupation des armoires, U par U ;

`'ports'` liste des ports ;

`'cableNeeds'` besoin en câbles.

`TABLE_IDS` constante

```
const TABLE_IDS: readonly TableId[]
```

Toutes les listes, dans l'ordre des onglets du classeur et des cases de la fenêtre.

`Cell` type

```
type Cell = string | number | null;
```

Valeur d'une cellule : texte, nombre (écrit comme nombre en XLSX, avec le séparateur décimal du CSV) ou `null` (vide).

`Column` interface

```
interface Column { id; label; numeric? }
```

Colonne d'une liste.

Champs

`id` identifiant stable (clé des lignes, choix des colonnes) ; `field:<clé>` pour un champ libre ;

`label` libellé traduit (`tables.columns.<liste>.<id>`), ou la clé du champ libre ;

`numeric` facultatif : colonne numérique (U, hauteur, longueur, quantité) ; informatif, les écritures se fondent sur le type de la valeur.

`Table` interface

```
interface Table { id; title; columns; rows }
```

Liste prête à écrire.

Champs

`id` son identifiant ;

`title` titre traduit (nom d'onglet et de fichier CSV) ;

`columns` colonnes, dans l'ordre ;

`rows` lignes, chacune indexée par identifiant de colonne (une clé absente vaut une cellule vide).

`orderedDevices` fonction

```
function orderedDevices(project: Project): Device[]
```

Équipements dans l'ordre des listes : par armoire (ordre du projet), d'abord les équipements montés du haut vers le bas (U le plus bas décroissant), puis les 0U par emplacement, puis les équipements extérieurs ; à égalité, par nom (ordre naturel français).

Paramètres

`project` le projet.

Résultat. Une nouvelle liste triée.

Erreurs. Aucune.

`lengthInMetres` fonction

```
function lengthInMetres(link: Link): number | null
```

Longueur d'une liaison en mètres (les centimètres sont divisés par 100).

Paramètres

`link` la liaison.

Résultat. La longueur, ou `null` si elle n'est pas renseignée (ou nulle).

Erreurs. Aucune.

`portCellName` fonction

```
function portCellName(project: Project, ref: PortRef): string
```

Nom d'un port dans les listes : son nom affiché, suivi de sa face (« 1 (avant) ») quand l'autre face de l'équipement porte un port du même nom, comme sur un panneau de brassage ; le carnet peut ainsi être réimporté sans ambiguïté (IMP-03).

Paramètres

`project` le projet ;

`ref` le port (équipement et port).

Résultat. Le nom, ou une chaîne vide si le port est introuvable.

Erreurs. Aucune.

`buildTable` fonction

```
function buildTable(project: Project, id: TableId): Table
```

Construit une liste à partir du projet. Inventaire : colonnes fixes puis une par clé de champ libre ; la colonne Face donne aussi le côté d'un équipement 0U (« Arrière droite (0U) ») ou demi-largeur (« Avant gauche »), relu par l'import tableur. Carnet : une ligne par liaison triée par repère, extrémités directes et finales (orientées comme la liaison), longueur en mètres. Occupation : une ligne par U, du haut vers le bas. Ports : une ligne par port, état Libre/Occupé/Désactivé. Besoin en câbles : quantités par type, catégorie et longueur.

Paramètres

`project` le projet ;

`id` la liste.

Résultat. La `Table`.

Erreurs. Aucune.

```
const cabling = buildTable(project, 'cabling');
cabling.rows[0]?.finalBDevice; // équipement final côté B
```

`withColumns` fonction

```
function withColumns(table: Table, columnIds: readonly string[] | undefined): Table
```

Copie d'une liste réduite aux colonnes choisies (EXP-05), dans l'ordre de la liste ; les lignes gardent toutes leurs valeurs, seules les colonnes écrites changent.

Paramètres

`table` la liste ;

`columnIds` les colonnes gardées, ou `undefined` pour toutes.

Résultat. La liste réduite, ou la liste elle-même sans choix.

Erreurs. Aucune.

27.15 `export/types`

Types communs à tous les exports : le fichier produit et l'erreur de refus. Sans dépendance. Utilisé par tous les modules « build », les fenêtres d'export, `downloadExport` et l'import (modèle de classeur).

`ExportedFile` interface

```
interface ExportedFile { bytes; fileName; mimeType }
```

Fichier produit par un export, prêt à être téléchargé par `downloadExport`.

Champs

`bytes` contenu du fichier ;

`fileName` nom proposé, extension comprise ;

`mimeType` type MIME (`application/pdf`, `image/png`, `text/csv`…).

`ExportError` classe

```
class ExportError extends Error
```

Refus d'un export pour une raison que l'utilisateur peut corriger ; le message, déjà traduit, est affiché tel quel par la fenêtre. Levée pour une page ou une vue disparue, un PNG trop grand, aucune page cochée, un gabarit d'étiquettes refusé ou aucune liaison à étiqueter.

Chapitre 28

Référence : état et commandes

Ce chapitre décrit chaque module de `src/commands/` (registre des commandes, DOC-09) et de `src/store/` (éditeur, historique et actions). La conception d'ensemble, l'algorithme de l'historique et la marche à suivre pour ajouter une action ou une commande sont au chapitre « État, actions et commandes ».

Conventions communes. Une *action* prend l'éditeur en premier paramètre, modifie le projet par `editor.apply` (une entrée d'historique annulable, sauf mention) et renvoie un `ActionResult`. Pour une action, « Erreurs » décrit le refus renvoyé (`ok: false`) ; les fabriques du modèle (`createRack`, `createDevice`…) valident leurs entrées avec Zod et lèvent une `ZodError` sur une entrée invalide, ce qui ne se produit pas avec les entrées construites par l'interface. Pour une *liste de commandes*, chaque entrée donne l'identifiant, le raccourci éventuel, la condition d'activation et l'effet.

28.1 `commands/annotation`

Commandes de l'onglet « Schéma » et de l'arrangement d'une page (PAG-07, PAG-08) : insérer un élément de chaque modèle, le tourner, le passer au-dessus ou au-dessous des vues, aligner et répartir la sélection, afficher la grille. S'appuie sur `store/annotationActions`, `model/annotations`, `model/arrange` et sur le flux d'insertion `ui/annotationFlow`. Ses fonctions de sélection servent aussi à `commands/device` (copier, dupliquer des éléments).

`selectedAnnotations` — fonction

```
function selectedAnnotations(ctx: CommandContext): Annotation[]
```

Renvoie les éléments de la page active (annotations) qui sont dans la sélection de l'interface, dans l'ordre de la page. Utilisée par les commandes de rotation, de calque, de copie et de duplication.

Paramètres

`ctx` contexte de la commande ; sans `ui` ou sans page active, la liste est vide.

Résultat. Les objets `Annotation` sélectionnés (références du projet courant, à ne pas modifier).

Erreurs. Aucune.

`arrangeSelection` fonction

```
function arrangeSelection(ctx: CommandContext): ArrangeSelection
```

Construit la sélection que l'alignement et la répartition déplacent : les vues sélectionnées qui appartiennent à la page active et les éléments sélectionnés.

Paramètres

`ctx` contexte de la commande.

Résultat. Un `ArrangeSelection` (`viewIds`, `annotationIds`), éventuellement vide.

Erreurs. Aucune.

`ALIGN_COMMANDS` constante

```
const ALIGN_COMMANDS = ALIGN.map((a) => a.id)
```

Identifiants des six commandes d'alignement, dans l'ordre d'affichage : `arrange.alignLeft`, `arrange.alignCenter`, `arrange.alignRight`, `arrange.alignTop`, `arrange.alignMiddle`, `arrange.alignBottom`. Le panneau des propriétés des éléments en fait une rangée de boutons ; module : `ui/properties/AnnotationProperties`.

`DISTRIBUTE_COMMANDS` constante

```
const DISTRIBUTE_COMMANDS = DISTRIBUTE.map((d) => d.id)
```

Identifiants des deux commandes de répartition : `arrange.distributeHorizontal` et `arrange.distributeVertical`. Même usage que `ALIGN_COMMANDS`.

`annotationCommands` constante

```
const annotationCommands: Command[]
```

Commandes des éléments de page, ajoutées au registre.

Champs

`annotation.add.<modèle>` une commande par modèle d'élément (`ANNOTATION_PRESETS` : `text`, `rect`, `roundedRect`, `ellipse`, `line`, `arrow`, `doubleArrow`, `elbowArrow`, `image`, `zone`, `bubble`, `note`, `table`) ; zone `library` ; libellé `annotations.presets.<modèle>` ; active avec une interface et une page active ; insère l'élément au centre de la vue par `insertAnnotation` (une image demande d'abord le fichier).

`annotation.rotate` tourne les éléments sélectionnés d'un quart de tour (`rotateAnnotation`), une action « tourner » ; active s'il y en a.

`annotation.toFront` passe les éléments sélectionnés au calque `above` (au-dessus des vues) ; active si l'un d'eux n'y est pas.

`annotation.toBack` passe les éléments sélectionnés au calque `below` ; active si l'un d'eux n'y est pas.

`arrange.align*` six commandes (zone `properties`) : alignent les vues et éléments sélectionnés sur un bord ou un centre de la sélection (`alignItems`) ; actives à partir de deux objets.

`arrange.distribute*` deux commandes : répartissent à intervalles égaux (`distributeItems`) ; actives à partir de trois.

`view.grid` bascule l'affichage de la grille (`showGrid` de l'état de l'interface), coche dans le menu Affichage.

28.2 `commands/context`

Fonctions partagées par les commandes pour lire la page active et la sélection dans un `CommandContext`. Hors de l'application (`ui` absent), elles renvoient une page absente et une sélection vide. Utilisé par tous les fichiers de commandes.

`activePage` fonction

```
function activePage(ctx: CommandContext): Page | undefined
```

Renvoie la page active de l'interface (`activePageId` de l'état de l'interface) dans le projet courant.

Paramètres

`ctx` contexte de la commande.

Résultat. La `Page`, ou `undefined` sans interface ou si la page n'existe plus.

Erreurs. Aucune.

`selection` fonction

```
function selection(ctx: CommandContext)
```

Renvoie la sélection de l'interface : une liste de `SelectionItem` (`kind` parmi `view`, `rack`, `device`, `link`, `port`, `annotation` ; `deviceId` pour un port).

Paramètres

`ctx` contexte de la commande.

Résultat. La sélection, ou un tableau vide sans interface.

Erreurs. Aucune.

`selectedRackId` fonction

```
function selectedRackId(ctx: CommandContext): string | undefined
```

Armoire concernée par la sélection, pour les commandes d'armoire (renommer, dupliquer, créer sa page, ajouter ses vues) et pour le collage : la première armoire sélectionnée, l'armoire d'une vue d'armoire sélectionnée sur la page active, ou l'armoire d'un équipement sélectionné (placé dans une armoire ou en 0U).

Paramètres

`ctx` contexte de la commande.

Résultat. L'identifiant de l'armoire, ou `undefined` (rien de tel dans la sélection).

Erreurs. Aucune.

28.3 `commands/device`

Commandes des équipements (EQP-01, EQP-06, EQP-07, EQP-09, EQT-04) et du presse-papiers : placer, copier, coller, dupliquer, monter ou descendre d'un U, retourner, éditer la machine, propriétés. Le presse-papiers est dans l'état de l'interface (`clipboard` :

équipements, vues extérieures, éléments de page). Utilise `store/deviceActions`, `store/annotationActions` et `placeWithConfirmation`.

`selectedDevices` fonction

```
function selectedDevices(ctx: CommandContext): Device[]
```

Équipements concernés par la sélection : les équipements sélectionnés et les équipements des vues extérieures sélectionnées sur la page active, sans doublon. Utilisée par les commandes d'équipement, par `device.addPort` (`commands/inspect`) et par la copie.

Paramètres

`ctx` contexte de la commande.

Résultat. Les objets `Device` existants (les identifiants disparus sont ignorés).

Erreurs. Aucune.

`deviceCommands` constante

```
const deviceCommands: Command[]
```

Commandes des équipements, ajoutées au registre.

Champs

`device.place` ouvre le dialogue de placement (`placeDevice`), armoire présélectionnée selon `selectedRackId` ; active avec une interface et au moins une armoire.

`edit.copy` Ctrl+C. Copie dans le presse-papiers les équipements sélectionnés (avec la position de leurs vues extérieures) et les éléments de page sélectionnés, puis affiche un message ; active avec une interface et quelque chose à copier.

`edit.paste` Ctrl+V. Colle le presse-papiers : équipements à la place libre la plus proche dans l'armoire de la sélection (sinon la leur), extérieurs sur la page active décalés de 10 mm, éléments décalés de 5 mm ; une seule action ; les copies sont sélectionnées ; un équipement sans place donne un avertissement. Coller plusieurs fois des éléments décale chaque fois un peu plus. Active si le presse-papiers n'est pas vide.

`edit.duplicate` Ctrl+D. Comme copier puis coller, sans toucher au presse-papiers ; active avec des équipements ou éléments sélectionnés.

`device.moveUp` Flèche haut. Monte d'un U les équipements d'armoire sélectionnés, tous ou aucun (`moveDevicesByU`) ; un refus est affiché en avertissement.

`device.moveDown` Flèche bas. Descend d'un U, mêmes règles.

`device.flip` retourne les équipements d'armoire sélectionnés, ou celui du port sélectionné (EQP-09) ; un débordement de profondeur demande confirmation (Forcer).

`device.editMachine` ouvre l'éditeur de machine sur la machine de l'équipement sélectionné seul (EQT-04, EDM-01).

`device.newMachine` ouvre l'éditeur de machine pour une nouvelle machine faite à partir de celle de l'équipement, qui la recevra.

`device.properties` Alt+Entrée. Ouvre le panneau des propriétés et met le focus sur son premier champ ; active avec une sélection.

28.4 `commands/edit`

Commandes du menu Édition qui ne concernent pas le presse-papiers : annuler, rétablir, tout sélectionner, désélectionner, supprimer. La suppression s'appuie sur `commands/selectionOps`.

`editCommands` constante

```
const editCommands: Command[]
```

Commandes du menu Édition, ajoutées au registre.

Champs

`edit.undo` Ctrl+Z. Annule la dernière action (`editor.undo`) ; active si l'historique le permet. Hors d'un champ de saisie seulement (dans un champ, Ctrl+Z reste l'annulation du champ).

`edit.redo` Ctrl+Y. Rétablit (`editor.redo`).

`edit.selectAll` Ctrl+A. Sélectionne toutes les vues et tous les éléments de la page active ; active si la page n'est pas vide.

`edit.deselect` Échap. Ferme le menu contextuel s'il est ouvert, sinon vide la sélection ; toujours active.

`edit.delete` Suppr (ou effacement arrière). Supprime la sélection (`deleteSelection`), avec confirmation quand des liaisons ou des armoires partent avec elle ; active si `deletionPlan` n'est pas vide.

28.5 `commands/export`

Commandes d'export (EXP-01 à EXP-07) : chacune ouvre son dialogue d'options (composants de `src/ui/exports/`), qui fait le travail.

`exportCommands` constante

```
const exportCommands: Command[]
```

Commandes d'export, ajoutées au registre.

Champs

`export.pdf` Ctrl+P. Dialogue d'export PDF ; actif avec au moins une page.

`export.svg` dialogue d'export d'image au format SVG de la page active.

`export.png` même dialogue au format PNG.

`export.spreadsheet` dialogue d'export tableur (nomenclature, liaisons).

`export.labels` dialogue d'export des étiquettes de câbles ; actif avec au moins une liaison.

28.6 `commands/file`

Commandes du menu Fichier (PRJ-01 à PRJ-03, PRJ-10, ERG-08, BIB-13). Elles passent par la session (`Session`), qui gère la demande d'enregistrer un projet modifié, l'ouverture et l'écriture des fichiers ; sans session (tests), elles sont inactives.

`fileCommands` constante

```
const fileCommands: Command[]
```

Commandes de fichier, ajoutées au registre.

Champs

`file.new` nouveau projet (`createStarterProject`).

`file.example` construit le projet d'exemple (`createExampleProject`), le charge comme nouveau projet et quitte l'écran d'accueil.

`file.newFromExisting` dialogue « Nouveau projet à partir d'un existant ».

`file.open` Ctrl+O, aussi dans un champ de saisie. Ouvre un fichier `.rkdg`, après la demande d'enregistrer un projet modifié.

`file.openLibrary` ouvre un `.rkdglib` pour en modifier les machines (BIB-13).

`file.newLibrary` nouveau document bibliothèque (`newLibraryDocument`).

`file.save` Ctrl+S, aussi dans un champ. Enregistre.

`file.saveAs` Ctrl+Maj+S, aussi dans un champ. Enregistre sous.

`file.home` revient à l'écran d'accueil (le projet reste ouvert).

28.7 `commands/help`

Menu Aide (ERG-09, DOC-04, DOC-10, DOC-11, WEB-02, WEB-09) : raccourcis, prise en main rapide, manuel utilisateur, documentation technique, notes de version, archive hors ligne, à propos. Lit `siteLinks` pour savoir si l'application est servie par son site. Les chemins des deux documents de l'utilisateur viennent de `help/documents` (lot 50) ; les trois PDF s'ouvrent dans un nouvel onglet (`window.open(url, '_blank', 'noopener')`, fonction interne `openDocument` pour les deux premiers).

`TECHNICAL_MANUAL_URL` constante

```
const TECHNICAL_MANUAL_URL = 'manual/portolan-technique.pdf'
```

Chemin relatif de cette documentation technique PDF dans l'application déployée (`manual/portolan-technique.pdf`), livrée avec l'application comme le manuel utilisateur et publiée sur le site (DOC-10). Ouvert par `help.technicalManual`.

`helpCommands` constante

```
const helpCommands: Command[]
```

Commandes du menu Aide, ajoutées au registre ; toutes restent actives quand une bibliothèque est ouverte (préfixe `help.`).

Champs

`help.shortcuts` F1. Dialogue des raccourcis, généré depuis le registre.

`help.quickStart` « Prise en main rapide (PDF) » : ouvre `QUICK_START_URL` dans un nouvel onglet ; toujours active.

`help.manual` « Manuel utilisateur (PDF) » : ouvre `MANUAL_URL` dans un nouvel onglet ; toujours active.

`help.technicalManual` ouvre `TECHNICAL_MANUAL_URL` dans un nouvel onglet ; toujours active.

`help.releaseNotes` dialogue des notes de version, embarquées dans l'application (WEB-02).

`help.archive` télécharge l'archive hors ligne (WEB-09) par un lien `download` ; active seulement sur le site, sinon grisée avec la raison `commands.siteOnly`.

`help.about` dialogue « À propos » (version).

28.8 `commands/import`

Commandes de l'import simplifié par tableur (IMP-01 à IMP-07). Utilise `buildImportTemplate` et `downloadBytes`.

`importCommands` constante

```
const importCommands: Command[]
```

Commandes d'import, ajoutées au registre.

Champs

`import.spreadsheet` ouvre le dialogue d'import d'un tableur ; l'application se fait par `importSpreadsheet`.

`import.template` construit le modèle de tableur à remplir, avec les listes du projet courant, et le télécharge ; toujours active.

28.9 `commands/index`

Point d'entrée du dossier : réexporte `commands/registry` et `commands/shortcuts`. L'interface et les outils (`tools/check-doc.ts`) importent `src/commands` ; les listes par domaine ne sont accessibles que par leur fichier ou par `commands`.

28.10 `commands/inspect`

Commandes de recherche et d'inspection (ERG-05, PRT-03, PRT-04) : recherche globale, affichage de l'occupation des ports, ajout d'un port à un équipement placé. Utilise `addInstancePort`.

`inspectCommands` constante

```
const inspectCommands: Command[]
```

Commandes d'inspection, ajoutées au registre.

Champs

`edit.find` Ctrl+F. Ouvre le dialogue de recherche (équipement, port, repère).

`view.portOccupancy` bascule l'affichage de l'état des ports (libre, occupé) sur la page active (PRT-03) ; coche dans le menu.

`device.addPort` ajoute un port à l'équipement sélectionné seul (face avant) ou à l'équipement du port sélectionné (même face), en RJ45 si ce type de connecteur existe, sinon le premier type ; sélectionne le nouveau port, ouvre les propriétés et met le focus sur son nom. Active avec une interface, au moins un type de connecteur et une telle cible.

28.11 `commands/library`

Commandes du panneau Bibliothèque (BIB-01 à BIB-09) et préférences de l'application. La machine visée est celle sélectionnée dans le panneau (`libraryModelId` de l'état de l'interface). Utilise `store/libraryActions`.

`libraryCommands` constante

```
const libraryCommands: Command[]
```

Commandes de la bibliothèque, ajoutées au registre.

Champs

`library.newModel` ouvre l'éditeur de machine pour une nouvelle machine de l'équipement sélectionné dans le panneau (un commutateur par défaut).

`library.newExternalModel` même chose pour un équipement extérieur (écran par défaut).

`library.editModel` ouvre l'éditeur sur la machine sélectionnée.

`library.duplicateModel` duplique la machine sélectionnée (`duplicateModel`) et sélectionne la copie.

`library.removeModel` supprime la machine sélectionnée après confirmation ; si des équipements l'utilisent, la confirmation le dit et ils sont supprimés avec elle. Inactive pour une machine générique (EQT-02).

`library.purge` dialogue « Purger la bibliothèque » (BIB-05).

`library.reimportDefault` dialogue de réimportation des machines par défaut (BIB-07).

`library.connectorTypes` dialogue des types de connecteurs (BIB-09).

`library.linkTypes` dialogue des types de liaisons (BIB-09).

`preferenceCommands` constante

```
const preferenceCommands: Command[]
```

Commandes de l'application (préfixe `app.`, actives aussi en mode bibliothèque), ajoutées au registre.

Champs

`app.preferences` dialogue des préférences (EDM-04).

`app.reload` recharge dans la nouvelle version disponible, après écriture du brouillon (WEB-08) ; zone `global` ; active seulement quand une mise à jour est signalée (`update` de l'état de l'interface).

28.12 `commands/libraryFiles`

Commandes de transfert de bibliothèques (BIB-08, BIB-10, BIB-12, WEB-03) : importer des machines depuis un `.rkdglib` ou un `.rkdg`, exporter une bibliothèque, ouvrir la page des bibliothèques de l'équipe. Lit les fichiers par `io/archive` et ouvre le dialogue d'import des modèles.

`libraryFileCommands` constante

```
const libraryFileCommands: Command[]
```

Commandes des fichiers de bibliothèque, ajoutées au registre.

Champs

`library.importLibrary` choisit un `.rkdglib`, le lit et ouvre le dialogue de choix des modèles à importer ; un fichier illisible est signalé par une alerte (message et cinq premiers détails).

`library.importFromProject` même chose depuis un projet `.rkdg`.

`library.exportLibrary` dialogue d'export d'une bibliothèque ; actif si la bibliothèque du projet contient des machines.

`library.teamLibraries` ouvre la page protégée des bibliothèques de l'équipe dans un nouvel onglet ; seulement sur le site (raison `commands.siteOnly`).

28.13 `commands/link`

Commandes des liaisons (LIA-02, LIA-07, LIA-08, LIA-10), du routage du projet et des vérifications.

`linkCommands` constante

```
const linkCommands: Command[]
```

Commandes des liaisons, ajoutées au registre.

Champs

`link.create` ouvre le dialogue de liaison, avec le port sélectionné comme première extrémité s'il y en a un ; actif avec au moins deux équipements.

`link.edit` ouvre le panneau des propriétés et met le focus sur le repère de la liaison sélectionnée.

`link.table` dialogue du tableau des liaisons.

`link.visibility` dialogue de visibilité des types de liaisons.

`project.routing` dialogue des réglages de routage (goulottes, ROU-02).

`checks.show` ouvre l'onglet « Vérifications » du panneau de gauche.

28.14 `commands/page`

Commandes des onglets de pages (PAG-01). Utilise `store/pageActions`.

`pageCommands` constante

```
const pageCommands: Command[]
```

Commandes des pages, ajoutées au registre (zone `pages`).

Champs

`page.add` ajoute une page après la page active (`addPage`) et l'active ; toujours active.

`page.rename` passe l'onglet de la page active en saisie du titre.

`page.duplicate` duplique la page active et active la copie.

`page.remove` supprime la page active, après confirmation si elle contient des vues ; inactive s'il ne reste qu'une page.

`page.moveLeft` déplace la page active d'un rang vers la gauche ; inactive pour la première.

`page.moveRight` d'un rang vers la droite ; inactive pour la dernière.

28.15 `commands/project`

Commandes des propriétés du projet, du cartouche et des révisions (PRJ-06, PAG-10, PRJ-11).

`projectCommands` constante

```
const projectCommands: Command[]
```

Commandes du projet, ajoutées au registre.

Champs

`project.properties` dialogue des propriétés du projet.

`project.titleBlock` dialogue du cartouche.

`project.revisions` dialogue des révisions.

`project.addRevision` ajoute une révision (`addRevision`, indice suivant, auteur du projet) puis, avec une interface, ouvre le dialogue des révisions sur elle ; toujours active.

28.16 `commands/rack`

Commandes des armoires (ARM-01, PAG-03, PAG-13, EXT-04). L'armoire visée est donnée par `selectedRackId`. Utilise `store/rackActions` et `store/pageActions`.

`rackCommands` constante

```
const rackCommands: Command[]
```

Commandes des armoires et des vues, ajoutées au registre.

Champs

`rack.add` dialogue d'ajout d'une armoire.

`rack.rename` F2. Dialogue de renommage de l'armoire visée.

`rack.duplicate` duplique l'armoire vide (`duplicateRack`), vues ajoutées à la page active, et sélectionne la copie.

`rack.duplicateWithContent` duplique avec ses équipements (sans liaisons).

`view.addRackFront` ajoute à la page active une vue de face avant de l'armoire visée et la sélectionne.

`view.addRackRear` même chose pour la face arrière.

`rack.createPage` crée une page pour l'armoire après la page active (`createRackPage`) et l'active.

`view.addExternal` montre sur la page active l'équipement extérieur sélectionné seul, s'il n'y figure pas (`addExternalView`).

`view.removeFromPage` retire de la page les vues sélectionnées (armoires et équipements conservés) et vide la sélection.

28.17 `commands/registry`

Registre central des commandes (DOC-09) : types d'une commande et de son contexte, liste complète, recherche par identifiant et par touche, activation (avec le filtre du mode bibliothèque, BIB-13) et exécution. Importe toutes les listes par domaine. Utilisé par l'interface (`ui/MenuBar`, `ui/CommandButton`, `ui/hooks`, `ui/useCommandShortcuts`, dialogue des raccourcis) et par `tools/check-doc.ts`, qui vérifie que chaque identifiant a sa section `\commande{id}` dans le manuel.

`CommandZone` type

```
type CommandZone = | 'menu' | 'toolbar' | 'library' | 'explorer' | 'canvas' | 'properties'
| 'pages' | 'dialog' | 'global';
```

Zone de l'interface à laquelle une commande appartient ; groupe le dialogue des raccourcis (titres `zones.<zone>` de `fr.json`).

Champs

`menu` barre de menus ;

`toolbar` barre d'outils (prévue, aucune commande ne l'utilise aujourd'hui) ;

`library` panneau Bibliothèque et onglet Schéma ;

`explorer` explorateur du projet ;

`canvas` zone de dessin et son menu contextuel ;

`properties` panneau des propriétés ;

`pages` onglets de pages ;

`dialog` dialogues ;

`global` application entière (rechargement).

`CommandContext` interface

```
interface CommandContext { editor; session?; ui? }
```

Ce sur quoi une commande agit, passé à `isEnabled`, `isChecked` et `run`. Construit par `useCommandContext` dans l'application, à la main dans les tests.

Champs

`editor` l'éditeur du projet ; toujours présent.

`session` session de fichiers (`Session`) ; absente hors de l'application.

`ui` état et méthodes de l'interface (`Ui`) : sélection, page active, zoom, dialogues, confirmations, messages ; absent hors de l'application.

`Command` interface

```
interface Command { id; labelKey; shortcut?; otherShortcuts?; zone; allowInTextField?;
isEnabled?; isChecked?; disabledHintKey?; run }
```

Déclaration d'une commande du registre.

Champs

`id` identifiant stable (`file.save`) : mots en minuscules séparés par des points, unique ; clé de la section du manuel et attribut `data-command`.

`labelKey` clé i18n du libellé dans `fr.json`.

`shortcut` raccourci principal, facultatif (syntaxe de `parseShortcut`) ; affiché dans les menus et infobulles.

`otherShortcuts` autres raccourcis acceptés, facultatifs, listés dans le dialogue des raccourcis.

`zone` zone de l'interface (`CommandZone`).

`allowInTextField` facultatif ; `true` : déclenchée aussi au clavier pendant la saisie dans un champ ou un dialogue.

`isEnabled` facultatif ; condition d'activation (absente : toujours active). Ne pas l'appeler directement : passer par `isCommandEnabled`.

`isChecked` facultatif ; pour une bascule, `true` affiche une coche dans le menu.

`disabledHintKey` facultatif ; clé i18n de la raison affichée sous une entrée de menu grisée.

`run` exécution ; peut renvoyer une promesse, attendue par `executeCommand`.

`commands` constante

```
const commands: Command[]
```

Liste de toutes les commandes de l'application, concaténation des listes par domaine dans cet ordre : fichier, projet, édition, affichage, armoires, équipements, bibliothèque, fichiers de bibliothèque, liaisons, routes, préférences, pages, exports, import, aide, inspection, éléments de page. C'est la source unique lue par les menus, le dialogue des raccourcis et le contrôle `check:doc`. Les tests vérifient l'unicité des identifiants et des raccourcis et l'existence des libellés.

`findCommand` fonction

```
function findCommand(id: string): Command | undefined
```

Retrouve une commande par son identifiant ; utilisée par les menus pour ignorer les identifiants inconnus et par `executeCommand`.

Paramètres

`id` identifiant de la commande.

Résultat. La `Command`, ou `undefined`.

Erreurs. Aucune.

`LIBRARY_DOCUMENT_COMMANDS` constante

```
const LIBRARY_DOCUMENT_COMMANDS: ReadonlySet<string>
```

Ensemble des identifiants utilisables quand le document ouvert est une bibliothèque (BIB-13) : fichiers (nouveau, ouvrir, enregistrer, accueil…), annuler et rétablir, panneau de gauche, vérifications, commandes des machines, import et export de bibliothèques, bibliothèques de l'équipe, types de connecteurs et de liaisons. Les commandes d'armoires, de pages, de liaisons, d'export, la purge… agissent sur un projet et en sont exclues. Une nouvelle commande y est ajoutée si elle a un sens pour une bibliothèque.

`availableInLibraryDocument` fonction

```
function availableInLibraryDocument(id: string): boolean
```

Indique si une commande peut servir pendant l'édition d'une bibliothèque : elle est dans `LIBRARY_DOCUMENT_COMMANDS` ou son identifiant commence par `help.` ou `app.`.

Paramètres

`id` identifiant de la commande.

Résultat. `true` si elle reste disponible.

Erreurs. Aucune.

`isCommandEnabled` fonction

```
function isCommandEnabled(command: Command, ctx: CommandContext): boolean
```

Seule façon correcte de savoir si une commande est active, utilisée par les menus, les boutons, les raccourcis et `executeCommand`. Quand une bibliothèque est éditée (`project.editedLibrary` défini), une commande non disponible dans ce mode est inactive ; sinon le résultat est celui de `isEnabled` (`true` s'il est absent).

Paramètres

`command` la commande ;

`ctx` le contexte.

Résultat. `true` si la commande peut être exécutée maintenant.

Erreurs. Celles éventuelles de `isEnabled` (aucune pour les commandes actuelles).

`disabledHintKey` fonction

```
function disabledHintKey(command: Command, ctx: CommandContext): string | undefined
```

Raison pour laquelle une commande inactive ne peut pas servir, affichée par les menus sous l'entrée grisée : `commands.libraryDocumentOnly` en mode bibliothèque pour une commande non disponible, sinon le `disabledHintKey` de la commande.

Paramètres

`command` la commande ;

`ctx` le contexte.

Résultat. Une clé i18n, ou `undefined` (pas de raison à afficher). L'appelant ne l'affiche que si la commande est inactive.

Erreurs. Aucune.

`executeCommand` fonction

```
async function executeCommand(id: string, ctx: CommandContext): Promise<boolean>
```

Exécute une commande si elle existe et est active, et attend la fin de son `run`. Point d'entrée de tous les clics de menu et de bouton et des raccourcis ; utilisable dans les tests sans interface.

Paramètres

`id` identifiant de la commande ;

`ctx` le contexte.

Résultat. Promesse de `true` si la commande a été exécutée, `false` si elle est inconnue ou inactive.

Erreurs. Rejette avec l'erreur de `run` si celle-ci échoue.

```
const editor = createEditor(createNewProject());
await executeCommand('page.add', { editor }); // true : une page de plus
await executeCommand('edit.undo', { editor }); // true : annulée
```

`commandShortcuts` fonction

```
function commandShortcuts(command: Command): string[]
```

Rassemble tous les raccourcis d'une commande, le principal en premier puis `otherShortcuts`. Utilisée par la recherche par touche, le dialogue des raccourcis et le test d'unicité.

Paramètres

`command` la commande.

Résultat. Les raccourcis (tableau vide s'il n'y en a pas).

Erreurs. Aucune.

`commandForKey` fonction

```
function commandForKey(event: KeyEventLike, inTextField: boolean): Command | undefined
```

Cherche la commande liée à un événement clavier, pour `useCommandShortcuts`. Ne tient pas compte de l'activation.

Paramètres

`event` l'événement (ou tout objet `KeyEventLike`) ;

`inTextField` `true` si la touche appartient à un champ de saisie ou à un dialogue : seules les commandes `allowInTextField` sont alors candidates.

Résultat. La première commande dont un raccourci correspond, ou `undefined`.

Erreurs. Aucune.

28.18 `commands/route`

Commandes des câbles d'une page : mode repères (ROU-09), faisceaux (ROU-11), câbles masqués d'une vue (PAG-15), retour au tracé automatique (ROU-07). Les câbles concernés par la sélection sont les câbles sélectionnés et ceux des équipements, ports,

vues extérieures et armoires (par leurs vues) sélectionnés. Utilise `store/routeActions`, `store/pageActions` et, pour trouver les faisceaux dessinés, le routage en cache de la page (`routing/page`).

`routeCommands` constante

```
const routeCommands: Command[]
```

Commandes des tracés, ajoutées au registre.

Champs

`view.markers` bascule le mode repères des vues sélectionnées : toutes activées si l'une ne l'est pas, sinon toutes désactivées ; une action.

`view.bundles` même bascule pour les faisceaux des vues d'armoire sélectionnées.

`view.hideCables` même bascule pour les câbles masqués des vues sélectionnées (PAG-15).

`route.ungroup` « Dégrouper le faisceau » : sort de leur faisceau tous les câbles des faisceaux dessinés sur la page qui contiennent un câble sélectionné.

`route.group` « Regrouper en faisceau » : remet dans leur faisceau les câbles dégroupés que la sélection désigne (partenaires de faisceau).

`route.reset` rend le tracé automatique aux câbles sélectionnés qui ont un tracé manuel sur la page active.

`route.resetPage` rend le tracé automatique à tous les câbles de la page active ; active s'il y a au moins un tracé manuel.

28.19 `commands/selectionOps`

Opérations sur la sélection partagées par les commandes (Suppr, menu contextuel) : calcul de ce qu'une suppression retirerait, puis suppression avec confirmation (EQP-06). Utilise les actions de suppression de `store/actions`, `store/pageActions` et `store/annotationActions`.

`deletionPlan` fonction

```
function deletionPlan(ctx: CommandContext)
```

Calcule ce que supprimer la sélection retirerait, sans rien modifier. Une vue d'armoire est retirée de la page (l'armoire reste) ; une vue extérieure désigne son équipement, qui est supprimé ; les équipements des armoires sélectionnées partent avec elles. Utilisée par `edit.delete` (activation) et `deleteSelection`.

Paramètres

`ctx` contexte de la commande.

Résultat. Un objet `{ views, devices, racks, links, annotations, attachedLinks }` : identifiants des vues à retirer de la page active, des équipements, armoires, liaisons et éléments à supprimer, et nombre de liaisons non sélectionnées qui partiraient avec les équipements.

Erreurs. Aucune.

`deleteSelection` fonction

```
async function deleteSelection(ctx: CommandContext): Promise<boolean>
```

Supprime la sélection en une action annulable (« supprimer la sélection ») puis vide la sélection. Si des armoires ou des liaisons attachées partent avec elle et qu'une interface est présente, demande d'abord confirmation.

Paramètres

`ctx` contexte de la commande.

Résultat. Promesse de `true` si quelque chose a été supprimé, `false` si la sélection ne contenait rien à supprimer ou si l'utilisateur a refusé.

Erreurs. Aucune.

28.20 `commands/shortcuts`

Syntaxe, reconnaissance et affichage des raccourcis clavier. Un raccourci s'écrit « `Ctrl+Shift+S` » ; `Ctrl` vaut aussi `Cmd`. Touches nommées : `Plus`, `Minus`, `Delete`, `Escape`, `Space`, `F1`, `F2`, `Enter` et les flèches. Sans dépendance ; utilisé par le registre, les menus et le dialogue des raccourcis.

`KeyEventLike` interface

```
interface KeyEventLike { key; ctrlKey; metaKey; shiftKey; altKey }
```

Partie d'un événement clavier utile à la reconnaissance ; un `KeyboardEvent` du navigateur convient, et les tests passent un objet littéral.

Champs

`key` valeur `key` de l'événement (« s », « ArrowUp », « Delete »…) ;

`ctrlKey` Ctrl enfoncée ;

`metaKey` Cmd (Méta) enfoncée, équivalente à Ctrl ;

`shiftKey` Maj enfoncée ;

`altKey` Alt enfoncée.

`parseShortcut` fonction

```
function parseShortcut(shortcut: string): ParsedShortcut
```

Décompose un raccourci en modificateurs et touche. Les parties sont séparées par « + » ; la dernière est la touche, mise en minuscules ; les modificateurs reconnus (sans tenir compte de la casse) sont `ctrl`, `shift` et `alt`.

Paramètres

`shortcut` le raccourci, par exemple `Ctrl+Shift+S`.

Résultat. Un objet `{ ctrl, shift, alt, key }` (type interne au module).

Erreurs. Aucune ; un texte inattendu donne une touche qui ne correspond à rien.

```
parseShortcut('Ctrl+Shift+S'); // { ctrl: true, shift: true, alt: false, key: 's' }
```

`matchesShortcut` fonction

```
function matchesShortcut(event: KeyEventLike, shortcut: string): boolean
```

Indique si un événement clavier correspond à un raccourci. Ctrl ou Cmd doivent être enfoncées si et seulement si le raccourci contient `Ctrl` ; Alt de même ; Maj de même, sauf pour `Plus` et `Minus`, qui ignorent Maj et reconnaissent aussi « = » et « _ » (position voisine selon la disposition) ; `Delete` reconnaît aussi l'effacement arrière, `Space` la barre d'espace. Les autres touches sont comparées sans tenir compte de la casse.

Paramètres

`event` l'événement ;

`shortcut` le raccourci.

Résultat. `true` si l'événement déclenche ce raccourci.

Erreurs. Aucune.

```
const key = { key: '=', ctrlKey: true, metaKey: false, shiftKey: false, altKey: false };
matchesShortcut(key, 'Ctrl+Plus'); // true
```

`displayShortcut` fonction

```
function displayShortcut(shortcut: string): string
```

Écrit un raccourci comme l'utilisateur le lit, avec le vocabulaire d'un clavier français : `Shift` devient « Maj », `Delete` « Suppr », `Escape` « Échap », `Space` « Espace », `Plus` « + », `Minus` « – », les flèches leurs symboles. Utilisée par les menus, les infobulles et le dialogue des raccourcis.

Paramètres

`shortcut` le raccourci du registre.

Résultat. Le texte affiché, par exemple « Ctrl+Maj+S ».

Erreurs. Aucune.

28.21 `commands/view`

Commandes d'affichage (PAG-12, ERG-01, ERG-10, ARM-05) : zoom, panneaux, occupation des armoires. Agissent seulement sur l'état de l'interface, jamais sur le projet (rien d'annulable). Utilise `fitZoom` et les constantes de zoom de `ui/state`.

`viewCommands` constante

```
const viewCommands: Command[]
```

Commandes d'affichage, ajoutées au registre.

Champs

`view.zoomIn` Ctrl++ (ou Ctrl+=). Zoom avant d'un pas ; actif avec une page active.

`view.zoomOut` Ctrl+– . Zoom arrière d'un pas.

`view.zoomFit` Espace, ou Ctrl+0. Ajuste le zoom pour voir la page entière (ERG-10).

`view.zoomActual` Ctrl+1. Zoom 100 % (96 dpi).

`view.toggleLeftPanel` affiche ou masque le panneau de gauche.

`view.toggleRightPanel` affiche ou masque le panneau des propriétés.

`view.occupation` bascule l'affichage des U libres des armoires, à l'écran seulement (ARM-05) ; coche dans le menu.

28.22 `store/actions`

Socle des actions d'édition : propriétés du projet, armoires, placement et déplacement d'équipements, liaisons, avec les suppressions en cascade. Chaque action vérifie les règles du modèle (`model/placement`, `model/links`) puis modifie le projet par `editor.apply` (une entrée annulable). Définit le type de résultat commun à toutes les actions. Utilisé par les autres modules d'actions, les commandes, les dialogues et panneaux de l'interface, l'import et le projet d'exemple.

`ActionResult` type

```
type ActionResult = { ok: true; id?: Id; warnings: string[] } | { ok: false; messages: string[]; forceable: boolean };
```

Résultat commun des actions.

Champs

`ok: true` action faite ; `id` (facultatif) est l'entité créée ou modifiée ; `warnings` liste des avertissements traduits à afficher (débordement de profondeur accepté, compatibilité forcée…), vide le plus souvent.

`ok: false` action refusée, projet inchangé ; `messages` donne les raisons traduites (parfois vide pour une entité introuvable) ; `forceable` vaut `true` quand l'utilisateur peut « Forcer » : le même appel avec `forced: true` sera accepté (EQP-02, LIA-06).

`dropLinks` fonction

```
function dropLinks(draft: Draft<Project>, linkIds: ReadonlySet<Id>)
```

Retire des liaisons d'un brouillon de projet, avec leurs tracés manuels sur toutes les pages. Fonction de bas niveau, à appeler *dans* une recette d'`apply` ; utilisée par les suppressions de liaisons, d'équipements, d'armoires et de ports ajoutés (`removeInstancePort`).

Paramètres

`draft` brouillon Immer du projet ;

`linkIds` identifiants des liaisons à retirer (ensemble vide : rien).

Résultat. Aucun ; le brouillon est modifié.

Erreurs. Aucune.

`updateProjectMeta` fonction

```
function updateProjectMeta(editor: Editor, patch: Partial<ProjectMeta>): ActionResult
```

Modifie les propriétés du projet (nom, client, auteur, notes…, PRJ-06), depuis le dialogue des propriétés.

Paramètres

`editor` l'éditeur ;

`patch` champs de `ProjectMeta` à remplacer.

Résultat. `ok: true`, sans identifiant. Rien n'est enregistré si rien ne change.

Erreurs. Aucune.

`addRack` fonction

```
function addRack(editor: Editor, input: Partial<Omit<Rack, 'id'>> = {}): ActionResult
```

Ajoute une armoire seule, sans vue (les caractéristiques absentes prennent les valeurs par défaut de `createRack`). L'interface utilise plutôt `addRackWithViews` ; celle-ci sert aux tests et au projet d'exemple.

Paramètres

`editor` l'éditeur ;

`input` caractéristiques de l'armoire (nom, `heightU`, profondeurs en mm, emplacements 0U…), facultatives.

Résultat. `ok: true` avec l'identifiant de la nouvelle armoire.

Erreurs. `ZodError` levée si `input` est invalide.

`updateRack` fonction

```
function updateRack(editor: Editor, rackId: Id, patch: Partial<Omit<Rack, 'id'>>):
ActionResult
```

Modifie une armoire (nom, hauteur, profondeurs, goulottes…, ARM-01 à ARM-04). Une nouvelle hauteur est vérifiée par `checkRackResize` : réduire sous un équipement placé est refusé.

Paramètres

`editor` l'éditeur ;

`rackId` l'armoire ;

`patch` champs à remplacer.

Résultat. `ok: true` avec `rackId`.

Erreurs. Refus sans message si l'armoire n'existe pas ; refus avec le message de la règle si la hauteur ne convient pas.

`removeRack` fonction

```
function removeRack(editor: Editor, rackId: Id): ActionResult
```

Supprime une armoire avec ses équipements (en U et en 0U), leurs liaisons et tracés, et toutes les vues de l'armoire sur toutes les pages. La confirmation est demandée par l'interface (`deleteSelection`).

Paramètres

`editor` l'éditeur ;

`rackId` l'armoire.

Résultat. `ok: true` (aussi si l'armoire n'existait pas : rien n'est fait).

Erreurs. Aucune.

`PlaceOptions` interface

```
interface PlaceOptions { forced?; coalesceKey? }
```

Options d'un placement ou d'un déplacement d'équipement.

Champs

`forced` facultatif ; « Forcer » malgré un débordement de profondeur (EQP-02).

`coalesceKey` facultatif ; clé de fusion d'historique d'un glisser.

`placeDevice` fonction

```
function placeDevice(editor: Editor, modelId: Id, placement: Placement, input:
DeviceInput, options: PlaceOptions = {}): ActionResult
```

Place un nouvel équipement d'une machine (EQP-01, EQP-02, EQP-04, EXT-01), après vérification par `checkPlacement` (place libre, demi-largeur, 0U, profondeur). L'équipement garde la trace d'un débordement forcé (`depthForced`). Pour un placement avec nom automatique, utiliser `placeNewDevice` ; pour un extérieur avec sa vue, `placeExternal`.

Paramètres

`editor` l'éditeur ;

`modelId` la machine de la bibliothèque ;

`placement` le placement (`rack` : armoire, U du bas, face, côté d'une demi-largeur ; `zeroU` : armoire, face, côté, emplacement ; `external`) ;

`input` propriétés de l'équipement (`name` obligatoire, description, mode d'étiquette…) ;

`options` `PlaceOptions`.

Résultat. `ok: true` avec l'identifiant de l'équipement et les avertissements de la vérification.

Erreurs. Refus avec les problèmes de placement ; `forceable` si seul un débordement de profondeur empêche le placement. `ZodError` si `input` est invalide (nom vide).

```
const result = placeDevice(editor, machineId,
  { kind: 'rack', rackId, uBottom: 10, mountFace: 'front', halfSide: null },
  { name: 'SRV01' });
if (!result.ok && result.forceable)
  placeDevice(editor, machineId, placement, { name: 'SRV01' }, { forced: true });
```

`moveDevice` fonction

```
function moveDevice(editor: Editor, deviceId: Id, placement: Placement, options:
PlaceOptions = {}): ActionResult
```

Déplace un équipement vers un autre placement (EQP-06), dans la même armoire ou une autre ; la vérification ignore l'équipement lui-même et considère ses nouveaux voisins pour la profondeur. Appelé au glisser avec une clé de fusion.

Paramètres

`editor` l'éditeur ;

`deviceId` l'équipement ;

`placement` le nouveau placement ;

`options` `PlaceOptions`.

Résultat. `ok: true` avec `deviceId` et les avertissements.

Erreurs. Refus sans message si l'équipement n'existe pas ; refus avec les problèmes de placement (forçable pour la profondeur seule).

`updateDevice` fonction

```
function updateDevice(editor: Editor, deviceId: Id, patch: Partial<Omit<Device, 'id'
| 'machineId' | 'placement'>>): ActionResult
```

Modifie les propriétés d'un équipement (nom, description, champs libres, mode d'étiquette…, EQP-07), depuis le panneau des propriétés. Ne vérifie rien : le placement se change par `moveDevice`, la machine par `setDeviceMachine`.

Paramètres

- `editor` l'éditeur ;
- `deviceId` l'équipement ;
- `patch` champs à remplacer (hors `id`, `machineId` et `placement`, que le type refuse).

Résultat. `ok: true` avec `deviceId` (aussi pour un équipement inconnu : rien n'est fait).

Erreurs. Aucune.

`removeDevices` fonction

```
function removeDevices(editor: Editor, deviceIds: readonly Id[]): ActionResult
```

Supprime des équipements avec les liaisons attachées (et leurs tracés) et les vues des équipements extérieurs (EQP-06). La confirmation est demandée par l'interface.

Paramètres

- `editor` l'éditeur ;
- `deviceIds` les équipements.

Résultat. `ok: true`.

Erreurs. Aucune.

`AddLinkOptions` interface

```
interface AddLinkOptions { linkTypeId?; ref?; forced?; attributes? }
```

Options de création d'une liaison.

Champs

- `linkTypeId` facultatif ; type de liaison, proposé d'après les connecteurs des deux ports s'il est absent (LIA-03).
- `ref` facultatif ; repère, automatique (suivant du type) s'il est absent (LIA-04).
- `forced` facultatif ; garde la liaison malgré des avertissements de compatibilité (LIA-06).
- `attributes` facultatif ; autres attributs de la liaison (longueur, couleur, notes…, LIA-05).

`addLink` fonction

```
function addLink(editor: Editor, a: PortRef, b: PortRef, options: AddLinkOptions = {}):
ActionResult
```

Crée une liaison entre deux ports (LIA-01, LIA-02). Les extrémités sont vérifiées (ports existants et distincts, non désactivés, libres sauf port « multiple ») par `checkLinkEnds`, puis la compatibilité des connecteurs avec le type par `checkCompatibility`. Une liaison forcée est marquée `compatibilityForced`.

Paramètres

- `editor` l'éditeur ;
- `a` première extrémité (`deviceId`, `portId`) ;
- `b` seconde extrémité ;
- `options` `AddLinkOptions`.

Résultat. `ok: true` avec l'identifiant de la liaison et les avertissements de compatibilité acceptés.

Erreurs. Refus non forçable si les extrémités ne conviennent pas ou si aucun type ne peut être proposé ; refus *forçable* avec les avertissements de compatibilité tant que `forced` n'est pas donné.

`updateLink` fonction

```
function updateLink(editor: Editor, linkId: Id, patch: Partial<Omit<Link, 'id' | 'a'
| 'b'>>): ActionResult
```

Modifie une liaison (type, repère, attributs, faisceau…), sans toucher à ses extrémités ; aucune vérification.

Paramètres

- `editor` l'éditeur ;
- `linkId` la liaison ;
- `patch` champs à remplacer.

Résultat. `ok: true` avec `linkId`.

Erreurs. Aucune.

`removeLinks` fonction

```
function removeLinks(editor: Editor, linkIds: readonly Id[]): ActionResult
```

Supprime des liaisons et leurs tracés manuels.

Paramètres

- `editor` l'éditeur ;
- `linkIds` les liaisons.

Résultat. `ok: true`.

Erreurs. Aucune.

28.23 `store/annotationActions`

Actions sur les éléments d'une page (« Schéma », PAG-07, PAG-08) : ajout, modification, remplacement (redimensionnement, rotation), suppression, collage, déplacement groupé avec des vues, alignement et répartition. Une entrée annulable chacune ; un glisser est fusionné par sa clé. S'appuie sur `model/annotations`, `model/arrange` et `model/pageGeometry`. Utilisé par la zone de dessin, le panneau des propriétés des éléments et `commands/annotation`.

`AnnotationPatch` type

```
type AnnotationPatch = { [K in Exclude<AnyKey, 'id'>]?: unknown };
```

Modification d'un élément : n'importe quel champ de l'une des sortes d'éléments (texte, forme, ligne, image, zone, bulle, note, tableau), sauf `id`, avec une valeur non typée. L'appelant est responsable de n'envoyer que des champs de la sorte de l'élément modifié (les panneaux de propriétés le font).

`addAnnotation` fonction

```
function addAnnotation(editor: Editor, pageId: Id, annotation: Annotation): ActionResult
```

Ajoute un élément déjà construit (`ui/annotationFlow`) à une page.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `annotation` l'élément, avec son identifiant.

Résultat. `ok: true` avec l'identifiant de l'élément.

Erreurs. Refus sans message si la page n'existe pas.

`updateAnnotation` fonction

```
function updateAnnotation(editor: Editor, pageId: Id, id: Id, patch:
AnnotationPatch, options: { coalesceKey?: string } = {}): ActionResult
```

Modifie des champs d'un élément (texte, couleurs, épaisseur…), depuis le panneau des propriétés ; avec une clé de fusion pour un réglage continu.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `id` l'élément ;
- `patch` `AnnotationPatch` ;
- `options` `coalesceKey` facultative.

Résultat. `ok: true` avec `id`.

Erreurs. Refus sans message si la page ou l'élément n'existe pas.

`replaceAnnotations` fonction

```
function replaceAnnotations(editor: Editor, pageId: Id, annotations: readonly
Annotation[], options: { coalesceKey?: string; label?: string } = {}): ActionResult
```

Remplace des éléments d'une page par de nouvelles versions de mêmes identifiants : un redimensionnement, une rotation, un changement de calque calculés hors de l'éditeur. Les identifiants inconnus sont ignorés.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `annotations` les nouvelles versions ;
- `options` `coalesceKey` facultative ; `label`, clé d'historique (`history.updateAnnotation` par défaut).

Résultat. `ok: true`.

Erreurs. Aucune.

`removeAnnotations` fonction

```
function removeAnnotations(editor: Editor, pageId: Id, ids: readonly Id[]): ActionResult
```

Supprime des éléments d'une page.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `ids` les éléments.

Résultat. `ok: true`.

Erreurs. Aucune.

`pasteAnnotations` fonction

```
function pasteAnnotations(editor: Editor, pageId: Id, annotations: readonly
Annotation[], dx: number, dy: number): Id[]
```

Ajoute à une page des copies d'éléments (nouveaux identifiants) décalées de (`dx`, `dy`) ; une bulle copiée prend le numéro suivant de la page. Utilisée par coller et dupliquer.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `annotations` les éléments à copier ;
- `dx` décalage horizontal, en mm de papier ;
- `dy` décalage vertical, en mm.

Résultat. Les identifiants des copies (tableau vide s'il n'y a rien à copier ; les identifiants sont renvoyés même si la page n'existe pas, auquel cas rien n'est ajouté).

Erreurs. Aucune.

`MoveBase` interface

```
interface MoveBase { views; annotations }
```

Position de départ des vues et éléments déplacés ensemble au glisser, relevée au début du geste : chaque mouvement repart de cette base, sans cumul d'arrondis.

Champs

- `views` identifiant et position `x`, `y` (mm) de chaque vue ;
- `annotations` copies des éléments au départ.

`moveItems` fonction

```
function moveItems(editor: Editor, pageId: Id, base: MoveBase, dx: number, dy:
number, options: { coalesceKey?: string } = {}): ActionResult
```

Place les vues et éléments de `base` à leur position de départ décalée de (`dx`, `dy`) : glisser d'une sélection mixte (les éléments suivent les vues sélectionnées avec eux, PAG-07). Positions des vues arrondies au dixième de mm.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `base` `MoveBase` ;
- `dx` décalage total depuis le départ, en mm ;
- `dy` idem, vertical ;
- `options` `coalesceKey` du glisser.

Résultat. `ok: true`.

Erreurs. Aucune.

`ArrangeSelection` interface

```
interface ArrangeSelection { viewIds; annotationIds }
```

Sélection d'une page que l'on peut aligner ou répartir.

Champs

- `viewIds` vues sélectionnées ;
- `annotationIds` éléments sélectionnés.

`alignItems` fonction

```
function alignItems(editor: Editor, pageId: Id, selection: ArrangeSelection, mode:
AlignMode): ActionResult
```

Aligne les vues et éléments sélectionnés sur un bord ou un centre du rectangle qui les englobe (PAG-08), d'après leurs rectangles (`viewBounds`, `annotationBounds`) et `alignOffsets`.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `selection` `ArrangeSelection` ;
- `mode` `left`, `centerX`, `right`, `top`, `centerY` ou `bottom`.

Résultat. `ok: true` ; positions arrondies au centième de mm.

Erreurs. Refus sans message s'il y a moins de deux objets.

`distributeItems` fonction

```
function distributeItems(editor: Editor, pageId: Id, selection: ArrangeSelection, axis:
DistributeAxis): ActionResult
```

Répartit les vues et éléments sélectionnés à intervalles égaux le long d'un axe (PAG-08), les deux extrêmes restant en place (`distributeOffsets`).

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `selection` `ArrangeSelection` ;
- `axis` `horizontal` ou `vertical`.

Résultat. `ok: true`.

Erreurs. Refus sans message s'il y a moins de trois objets.

28.24 `store/deviceActions`

Actions d'équipement propres à l'interface (EQP-01, EQP-06, EQP-09, EXT-01) : placement avec nom automatique, équipement extérieur avec sa vue, collage et duplication à la place libre la plus proche, déplacement par U, retournement. Les calculs (noms, places libres, déplacements groupés) sont dans `model/dropTarget`. Utilisé par la zone de dessin (dépôt depuis la bibliothèque), le dialogue de placement et `commands/device`.

`placeNewDevice` fonction

```
function placeNewDevice(editor: Editor, modelId: Id, placement: Placement, options:
PlaceOptions & { name?: string } = {}): ActionResult
```

Place un nouvel équipement d'une machine en le nommant d'après sa sous-catégorie (« SRV01 », « SW02 »… : préfixe de `devicePrefix`, numéro suivant libre), puis délègue à `placeDevice`.

Paramètres

- `editor` l'éditeur ;
- `modelId` la machine ;
- `placement` le placement ;
- `options` `PlaceOptions` et `name` (nom imposé, facultatif).

Résultat. Celui de `placeDevice`.

Erreurs. Refus sans message si la machine n'existe pas ; sinon ceux de `placeDevice`.

`placeExternal` fonction

```
function placeExternal(editor: Editor, modelId: Id, pageId: Id, point: Point, options: {
name?: string; scale?: number } = {}): ActionResult
```

Place un équipement extérieur sur une page (EXT-01) : l'équipement (placement `external`) et sa vue, en une action. L'échelle de la vue est celle des autres vues extérieures de la page, ou 0,1.

Paramètres

- `editor` l'éditeur ;
- `modelId` la machine, qui doit être extérieure ;
- `pageId` la page ;
- `point` coin haut gauche de la vue, en mm de page (arrondi au mm) ;
- `options` `name` et `scale` facultatifs.

Résultat. `ok: true` avec l'identifiant de l'équipement.

Erreurs. Refus sans message si la machine n'existe pas ou n'est pas extérieure.

`PasteOptions` interface

```
interface PasteOptions { rackId?; pageId?; externalViews? }
```

Options d'un collage d'équipements.

Champs

- `rackId` armoire qui reçoit les équipements d'armoire ; leur propre armoire si absent ou `null`.
- `pageId` page qui reçoit les équipements extérieurs ; sans page, ils ne sont pas collés.
- `externalViews` position, échelle, taille et ports masqués des vues extérieures copiées, par identifiant d'équipement d'origine.

`pasteDevices` fonction

```
function pasteDevices(editor: Editor, sources: readonly Device[], options: PasteOptions =
{}): { ok: true; warnings: string[]; ids: Id[] }
```

Colle des copies d'équipements (copier/coller, dupliquer, EQP-06) en une action : chacun à la place libre la plus proche (`nearestFreePlacement`) de l'armoire cible, avec un nom unique ; les extérieurs sur la page, vue décalée de 10 mm. Les équipements du haut sont traités en premier, pour qu'un bloc garde son ordre ; chaque placement tient compte des copies précédentes. Les liaisons ne sont pas copiées.

Paramètres

- `editor` l'éditeur ;
- `sources` les équipements à copier (dont la machine doit exister) ;
- `options` `PasteOptions`.

Résultat. `{ ok: true, warnings, ids }` : identifiants des copies, et un avertissement « pas de place » par équipement qui n'a pas trouvé de place.

Erreurs. Aucune ; les sources inutilisables sont ignorées.

`moveDevicesByU` fonction

```
function moveDevicesByU(editor: Editor, deviceIds: readonly Id[], deltaU: number):
ActionResult
```

Déplace des équipements d'armoire d'un nombre de U (flèches, EQP-06), tous ou aucun (`moveDevicesBy`).

Paramètres

`editor` l'éditeur ;

`deviceIds` les équipements ;

`deltaU` nombre de U, positif vers le haut.

Résultat. `ok: true` (rien d'enregistré si aucun équipement n'est concerné).

Erreurs. Refus avec un message si l'un d'eux ne peut pas bouger (sortie de l'armoire, chevauchement).

`flipRackDevices` fonction

```
function flipRackDevices(editor: Editor, deviceIds: readonly Id[], options: { forced?:
boolean } = {}): ActionResult
```

Retourne des équipements d'armoire (EQP-09) : fixés sur l'autre face, leur face avant tournée vers elle ; tous ou aucun (`flipDevices`).

Paramètres

`editor` l'éditeur ;

`deviceIds` les équipements ;

`options` `forced` : accepter un débordement de profondeur.

Résultat. `ok: true` avec les avertissements (débordements forcés).

Erreurs. Refus avec les messages ; `forceable` si seul un débordement de profondeur l'empêche.

28.25 `store/editor`

L'éditeur : un store Zustand « vanilla » (sans React) qui détient le projet, ses images et l'historique d'annulation fondé sur les patches d'Immer (PRJ-07, PRJ-04 ; décision du 2026-11-04, lot 05). Toute modification du projet passe par `apply`. À chaque `apply`, l'éditeur appelle `followRemovedViews` (ROU-16). Créé au démarrage par `src/main.tsx`, transmis à React par `EditorContext`, à la session et aux commandes ; créé à la volée dans les tests et le projet d'exemple.

`HISTORY_LIMIT` constante

```
const HISTORY_LIMIT = 100
```

Nombre maximal d'entrées d'annulation conservées (100, PRJ-07 : au moins 100). Les plus anciennes sont oubliées au-delà.

`HistoryEntry` interface

```
interface HistoryEntry { id; label; patches; inverse; coalesceKey? }
```

Une étape d'annulation.

Champs

- `id` identifiant unique et croissant (jamais réutilisé, y compris après fusion) ; sert à l'état « modifié ».
- `label` clé i18n décrivant l'action (`history.*`).
- `patches` patches Immer qui refont l'action.
- `inverse` patches qui la défont, dans l'ordre d'application.
- `coalesceKey` facultatif ; clé de fusion : une action suivante de même clé est fusionnée dans cette entrée.

`EditorState` interface

```
interface EditorState { project; images; past; future; savedEntryId }
```

État du store de l'éditeur.

Champs

- `project` le projet courant (immuable).
- `images` octets des images par référence (empreinte + extension), hors historique.
- `past` entrées annulables, la plus récente en dernier.
- `future` entrées rétablissables, la prochaine en premier.
- `savedEntryId` identifiant de l'entrée au sommet de `past` au dernier enregistrement ; 0 pour l'état initial ; `null` pour un brouillon restauré non enregistré (toujours « modifié »).

`ApplyOptions` interface

```
interface ApplyOptions { coalesceKey? }
```

Options de `Editor.apply`.

Champs

- `coalesceKey` facultatif ; fusionne cette modification avec l'entrée précédente de même clé, tant que rien n'a été annulé entre-temps.

`Editor` interface

```
interface Editor { store; getState; apply; transaction; undo; redo; canUndo; canRedo; load;
addImages; markSaved; currentEntryId; isDirty }
```

Interface de l'éditeur, renvoyée par `createEditor`.

Champs

- `store` le store Zustand, pour s'abonner (`useStore`, `subscribe`).
- `getState` état courant (`EditorState`).
- `apply (label, recette, options?)` : exécute la recette sur un brouillon du projet et l'enregistre comme une action annulable ; renvoie `false` si rien n'a changé (aucune entrée créée).

`transaction (label, fn)` : les `apply` faits pendant `fn` forment une seule entrée ; renvoie le résultat de `fn` ; imbriquée, elle fait partie de la transaction englobante.

`undo` annule la dernière entrée ; `false` s'il n'y en a pas.

`redo` rétablit ; `false` s'il n'y a rien.

`canUndo` y a-t-il une entrée à annuler ?

`canRedo` y a-t-il une entrée à rétablir ?

`load (projet, images?, { dirty? })` : remplace le projet (nouveau, ouvert, restauré), vide l'historique ; non modifié sauf `dirty`.

`addImages` ajoute des images (non annulable ; rien si toutes sont déjà présentes).

`markSaved (entryId?)` : marque enregistré l'état courant, ou l'entrée donnée (relevée avant une écriture asynchrone).

`currentEntryId` identifiant de l'entrée au sommet de `past` (0 au départ).

`isDirty` le projet est-il modifié depuis le dernier enregistrement ?

`isDirty` fonction

```
function isDirty(state: EditorState): boolean
```

Indique si le projet a été modifié depuis le dernier enregistrement (PRJ-04) : vrai quand l'identifiant de l'entrée au sommet de `past` (0 s'il n'y en a pas) diffère de `savedEntryId`. Annuler jusqu'à l'état enregistré rend donc `false`. Utilisée par `Editor.isDirty`, `selectIsDirty` et la session.

Paramètres

`state` un état de l'éditeur.

Résultat. `true` si modifié.

Erreurs. Aucune.

`createEditor` fonction

```
function createEditor(project: Project, images: ImageMap = new Map()): Editor
```

Crée un éditeur sur un projet, avec un historique vide, non modifié. Active les patches d'Immer au chargement du module.

Paramètres

`project` le projet initial ;

`images` images du projet (vide par défaut).

Résultat. Un `Editor`.

Erreurs. Aucune. Une exception levée par une recette d'`apply` remonte à l'appelant, sans modification du projet ; dans une transaction, les `apply` déjà faits sont gardés et enregistrés.

```
const editor = createEditor(createNewProject());
editor.apply('history.updateProject', (draft) => {
  draft.meta.name = 'Salle A';
});
editor.isDirty(); // true
editor.undo();
editor.isDirty(); // false
```

28.26 `store/exampleProject`

Construction du projet d'exemple chargé depuis l'écran d'accueil (ERG-08 ; décision du 2026-11-05, lot 43) : une armoire 18U garnie de machines génériques de la bibliothèque par défaut (CG-08), un écran et un portable extérieurs, une étagère qui porte un portable (MAC-03), sept liaisons et une page pour l'armoire. Construit par les actions, comme un utilisateur, pour respecter les règles de placement et de liaison. Utilisé par `file.example` (`commands/file`), l'écran d'accueil et `tools/check-neutral.ts`.

`EXAMPLE_REVISION_DATE` constante

```
const EXAMPLE_REVISION_DATE = '2026-11-05'
```

Date de la révision du projet d'exemple (`2026-11-05`), fixe pour que le document soit identique à chaque création (captures de référence, tests).

`EXAMPLE_LAPTOP_SVG` constante

```
const EXAMPLE_LAPTOP_SVG = …
```

Dessin SVG neutre d'un portable ouvert vu de face (écran, base), 160 × 100, soit un rapport 1,6 ; image portée par la variante d'étagère du projet d'exemple (MAC-03), ajoutée aux images du projet sous son empreinte.

`ExampleProject` interface

```
interface ExampleProject { project; images }
```

Résultat de la construction du projet d'exemple, prêt à être chargé par `session.newProject`.

Champs

`project` le projet ;

`images` ses images (le dessin du portable).

`createExampleProject` fonction

```
async function createExampleProject(now = new Date()): Promise<ExampleProject>
```

Construit le projet d'exemple dans un éditeur temporaire : propriétés et révision, cartouche (logo masqué, deux champs personnalisés), armoire et équipements, variante d'étagère portant le portable (enregistrée par `saveMachine`), liaisons, puis page de l'armoire (`createRackPage`). Textes tirés de la section `example` de `fr.json`.

Paramètres

`now` date de création du projet (maintenant par défaut) ; les tests la fixent.

Résultat. Promesse d'un `ExampleProject`.

Erreurs. Rejette avec une `Error` si une action est refusée (« example project »), si une machine générique ou un port attendu manque : cela signale une règle ou une bibliothèque par défaut qui a changé, à corriger dans la fonction.

28.27 `store/importActions`

Application d'un import de tableur (IMP-05, IMP-07) : les lignes sont vérifiées contre le projet courant par `planImport`, puis tout est ajouté en une action annulable, ou rien. Utilisé par le dialogue d'import.

`importSpreadsheet` fonction

```
function importSpreadsheet(editor: Editor, sheets: readonly Sheet[], options: ImportOptions = {}): ImportResult
```

Vérifie et applique l'import des feuilles lues : équipements, liaisons et vues des équipements extérieurs sur la page choisie.

Paramètres

- `editor` l'éditeur ;
- `sheets` feuilles lues par `import/read` ;
- `options` `pageId` : page qui reçoit les vues des extérieurs (aucune si `null`).

Résultat. Le résultat de `planImport` : `ok: true` avec le plan appliqué et les avertissements, ou `ok: false` avec les erreurs par ligne.

Erreurs. Les erreurs sont renvoyées (`ok: false`, projet inchangé), jamais levées.

28.28 `store/index`

Point d'entrée de `src/store` : réexporte `store/editor`, `store/actions`, `store/selectors`, `store/pageActions`, `store/projectActions`, `store/rackActions`, `store/deviceActions`, `store/libraryActions`, `store/machineActions`, `store/routingActions`, `store/routeActions` et `store/portActions`. Ne réexporte ni les crochets React (`store/react`), ni `store/annotationActions`, `store/importActions` ou `store/exampleProject`, importés par leur chemin.

28.29 `store/libraryActions`

Actions sur la bibliothèque du projet (BIB-03 à BIB-14) : modèles (machines), types de connecteurs et de liaisons, purge, réimportation des machines par défaut, import depuis une bibliothèque ou un projet, description d'une bibliothèque éditée. Une entrée annulable par action. S'appuie sur `library/equipment/index`, `model/purge` et `model/libraryTransfer`. Utilisé par `commands/library`, les dialogues de bibliothèque et la fiche de bibliothèque.

`modelUsage` fonction

```
function modelUsage(project: Pick<Project, 'devices'>): Map<Id, number>
```

Compte les équipements placés de chaque machine (BIB-06), pour l'affichage du panneau, la suppression et la purge.

Paramètres

- `project` le projet (seuls ses équipements sont lus).

Résultat. Une `Map` identifiant de machine → nombre d'équipements (absente si aucun).

Erreurs. Aucune.

`modelImageBytes` fonction

```
function modelImageBytes(model: Machine, imageSize: (ref: ImageRef) => number | undefined): number
```

Taille totale des images d'une machine, en octets (BIB-06).

Paramètres

`model` la machine ;

`imageSize` taille d'une image d'après sa référence (`undefined` si inconnue, comptée 0).

Résultat. Nombre d'octets.

Erreurs. Aucune.

`copyModel` fonction

```
function copyModel(model: Machine, name = model.name): Machine
```

Copie profonde d'une machine avec de nouveaux identifiants (machine et ports ; les paires de ports traversants sont reliées aux nouveaux identifiants), non générique, sans origine. Utilisée par la duplication et l'éditeur de machine.

Paramètres

`model` la machine ;

`name` nom de la copie (celui de la machine par défaut).

Résultat. La nouvelle machine (non ajoutée au projet).

Erreurs. Aucune.

`addModel` fonction

```
function addModel(editor: Editor, model: Machine): ActionResult
```

Ajoute une machine à la bibliothèque après validation par le schéma.

Paramètres

`editor` l'éditeur ;

`model` la machine complète.

Résultat. `ok: true` avec l'identifiant de la machine.

Erreurs. Refus avec les messages de validation du schéma.

`updateModel` fonction

```
function updateModel(editor: Editor, model: Machine): ActionResult
```

Remplace une machine de même identifiant, après validation ; ses équipements la suivent (BIB-03). Ne vérifie pas que les équipements tiennent encore : pour cela, utiliser `saveMachine` en mode `edit`.

Paramètres

`editor` l'éditeur ;

`model` la nouvelle version.

Résultat. `ok: true` avec l'identifiant (rien n'est fait si la machine n'existe pas).

Erreurs. Refus avec les messages de validation du schéma.

`duplicateModel` fonction

```
function duplicateModel(editor: Editor, modelId: Id): ActionResult
```

Duplique une machine (`copyModel`, nom « … (copie) ») et insère la copie juste après elle.

Paramètres

`editor` l'éditeur ;

`modelId` la machine.

Résultat. `ok: true` avec l'identifiant de la copie.

Erreurs. Refus sans message si la machine n'existe pas.

`removeModel` fonction

```
function removeModel(editor: Editor, modelId: Id, options: { withInstances?: boolean } =
{}): ActionResult
```

Supprime une machine (BIB-04). Une machine générique n'est jamais supprimée (EQT-02). Si des équipements l'utilisent, la suppression est refusée, sauf avec `withInstances` : les équipements, leurs liaisons, leurs tracés et les vues des extérieurs partent avec elle.

Paramètres

`editor` l'éditeur ;

`modelId` la machine ;

`options` `withInstances` : supprimer aussi ses équipements.

Résultat. `ok: true`.

Erreurs. Refus sans message si la machine n'existe pas ; refus avec message pour une machine générique, ou utilisée sans `withInstances`.

`purgeLibrary` fonction

```
function purgeLibrary(editor: Editor, imageSize: (ref: ImageRef) => number | undefined = ()
=> 0): ActionResult & { removed: number; savedBytes: number }
```

« Purger la bibliothèque » (BIB-05) : supprime toutes les machines non utilisées que `purgeUnusedMachines` désigne, en une action (aucune si rien n'est retiré).

Paramètres

`editor` l'éditeur ;

`imageSize` taille d'une image, pour estimer le gain (0 par défaut).

Résultat. `ok: true` avec `removed` (nombre de machines retirées) et `savedBytes` (gain estimé en octets).

Erreurs. Aucune.

`missingDefaultModels` fonction

```
function missingDefaultModels(project: Project): Machine[]
```

Machines génériques de la bibliothèque par défaut absentes du projet (BIB-07), proposées par le dialogue de réimportation.

Paramètres

`project` le projet.

Résultat. Les machines manquantes (copies neuves de la bibliothèque par défaut).

Erreurs. Aucune.

`reimportDefaultModels` fonction

```
function reimportDefaultModels(editor: Editor, modelIds: readonly Id[]): ActionResult
```

Réimporte des machines de la bibliothèque par défaut (BIB-07), avec les types de connecteurs qui leur manquent.

Paramètres

`editor` l'éditeur ;

`modelIds` identifiants choisis parmi `missingDefaultModels`.

Résultat. `ok: true` (rien d'enregistré si aucune n'est manquante).

Erreurs. Aucune.

`connectorUsage` fonction

```
function connectorUsage(project: Project, connectorId: Id): Machine[]
```

Machines dont des ports ou des paramètres utilisent un type de connecteur (BIB-09) : empêchent sa suppression.

Paramètres

`project` le projet ;

`connectorId` le type de connecteur.

Résultat. Les machines concernées.

Erreurs. Aucune.

`connectorDeviceUsage` fonction

```
function connectorDeviceUsage(project: Project, connectorId: Id): Device[]
```

Équipements placés dont des ports ajoutés (PRT-04) utilisent un type de connecteur.

Paramètres

`project` le projet ;

`connectorId` le type de connecteur.

Résultat. Les équipements concernés.

Erreurs. Aucune.

`addConnectorType` fonction

```
function addConnectorType(editor: Editor, input: Partial<ConnectorType> = {}): ActionResult
```

Ajoute un type de connecteur (BIB-09), nommé « Nouveau connecteur » de famille `other` par défaut, avec un nouvel identifiant.

Paramètres

`editor` l'éditeur ;

`input` champs du type, facultatifs (`id` ignoré).

Résultat. `ok: true` avec l'identifiant.

Erreurs. `ZodError` levée si `input` est invalide.

`updateConnectorType` fonction

```
function updateConnectorType(editor: Editor, id: Id, patch:
Partial<Omit<ConnectorType, 'id'>>): ActionResult
```

Modifie un type de connecteur (nom, famille, couleur, complément…).

Paramètres

`editor` l'éditeur ;

`id` le type ;

`patch` champs à remplacer.

Résultat. `ok: true` avec `id`.

Erreurs. Refus si le nouveau nom est vide.

`removeConnectorType` fonction

```
function removeConnectorType(editor: Editor, id: Id): ActionResult
```

Supprime un type de connecteur qu'aucune machine ni aucun port ajouté n'utilise (BIB-09) ; les références restantes (complément d'un autre type, connecteurs admis par les types de liaisons) sont nettoyées.

Paramètres

`editor` l'éditeur ;

`id` le type.

Résultat. `ok: true`.

Erreurs. Refus sans message si le type n'existe pas ; refus avec la liste des machines ou des équipements qui l'utilisent.

`addLinkType` fonction

```
function addLinkType(editor: Editor, input: Partial<LinkType> = {}): ActionResult
```

Ajoute un type de liaison (BIB-09), nommé « Nouveau type » et gris foncé par défaut.

Paramètres

`editor` l'éditeur ;

`input` champs du type, facultatifs (`id` ignoré).

Résultat. `ok: true` avec l'identifiant.

Erreurs. `ZodError` levée si `input` est invalide.

`updateLinkType` fonction

```
function updateLinkType(editor: Editor, id: Id, patch:
Partial<Omit<LinkType, 'id'>>, options: { coalesceKey?: string } = {}): ActionResult
```

Modifie un type de liaison (nom, couleur, style, connecteurs admis…) ; avec une clé de fusion pour le sélecteur de couleur.

Paramètres

`editor` l'éditeur ;

`id` le type ;

`patch` champs à remplacer ;

`options` `coalesceKey` facultative.

Résultat. `ok: true` avec `id`.

Erreurs. Refus si le nouveau nom est vide.

`removeLinkType` fonction

```
function removeLinkType(editor: Editor, id: Id): ActionResult
```

Supprime un type de liaison qu'aucune liaison n'utilise (BIB-09) ; le dernier type est toujours gardé.

Paramètres

`editor` l'éditeur ;

`id` le type.

Résultat. `ok: true`.

Erreurs. Refus sans message si le type n'existe pas ; refus avec message s'il est utilisé ou si c'est le dernier.

`applyImportPlan` fonction

```
function applyImportPlan(editor: Editor, plan: ImportPlan): ActionResult
```

Copie dans le projet les modèles, types et images d'un plan d'import (BIB-08, BIB-10, BIB-11, BIB-14), en une action. Le plan est d'abord vérifié par `checkImport` : une machine remplacée (mise à jour) qu'un équipement placé ne pourrait pas suivre est laissée de côté et signalée ; les liaisons sur des ports qui disparaissent sont gardées et signalées. Les images sont ajoutées hors historique.

Paramètres

`editor` l'éditeur ;

`plan` plan produit par `planImport` de `model/libraryTransfer`, non encore vérifié.

Résultat. `ok: true` avec les avertissements (mises à jour refusées, liaisons orphelines) ; rien n'est enregistré si le plan vérifié est vide.

Erreurs. Aucune ; les refus partiels sont des avertissements.

`setLibraryInfo` fonction

```
function setLibraryInfo(editor: Editor, patch: Partial<Omit<LibraryInfo, 'id'
| 'version'>>): ActionResult
```

Modifie la description de la bibliothèque éditée (nom, marque, description…, BIB-13) ; le nom du document suit le nom de la bibliothèque.

Paramètres

`editor` l'éditeur ;

`patch` champs de `LibraryInfo` (hors `id` et `version`).

Résultat. `ok: true`.

Erreurs. Refus si le document n'est pas une bibliothèque, ou si le nom est vide (FMT-09).

28.30 `store/machineActions`

Actions sur les machines et les paramètres d'équipement (EQT-02 à EQT-06, EDM-01) : nouvelle machine générée depuis des paramètres, nouveaux paramètres pour une machine déjà placée (ses équipements la suivent), variantes d'une machine pour certains équipements, machine enregistrée par l'éditeur de machine, autre machine pour un équipement placé, retour d'une machine générique manquante. Les conséquences sont calculées sans rien modifier par `library/equipment/change` (`MachineChange`), affichées par l'interface, puis appliquées en une action. Utilisé par le dialogue des paramètres, l'éditeur de machine, le panneau des propriétés, le dépôt depuis la bibliothèque et le projet d'exemple.

`addMachine` fonction

```
function addMachine(editor: Editor, kind: EquipmentKind, params:
Record<string, unknown>, name?: string): ActionResult
```

Ajoute une nouvelle machine d'un équipement, générée depuis des paramètres (`generateMachine`, BIB-03), avec les types de connecteurs par défaut qui lui manquent.

Paramètres

`editor` l'éditeur ;

`kind` l'équipement (commutateur, serveur, étagère…) ;

`params` paramètres de l'équipement (hauteur en U, groupes de ports…) ;

`name` nom de la machine, facultatif (nom proposé par le générateur sinon).

Résultat. `ok: true` avec l'identifiant de la machine.

Erreurs. Refus avec les problèmes des paramètres (`paramsProblems`).

`MachineParamsOptions` interface

```
interface MachineParamsOptions { name?; deviceIds?; variant? }
```

Options d'un changement de paramètres.

Champs

`name` nom de la machine ; vide : inchangé (ou nom proposé d'une nouvelle machine).

`deviceIds` équipements qui reçoivent la variante (voir `variant`).

`variant` crée une nouvelle machine avec les paramètres, donnée aux équipements `deviceIds` ; la machine source ne change pas. Toujours le cas pour une machine générique, qui garde ses paramètres par défaut (EQT-02).

`variantDevices` fonction

```
function variantDevices(options: { existing: boolean; generic: boolean; deviceId?: Id;
scope: 'all' | 'device'; users: readonly { id: Id }[]; }): Id[]
```

Décide quels équipements reçoivent la nouvelle machine créée par le dialogue des paramètres. Ouvert depuis un équipement : cet équipement seul (« cet équipement seulement »), ou, pour une machine générique, tous ses équipements si l'utilisateur le choisit ; ouvert depuis la bibliothèque (sans équipement) : aucun, la machine est seulement ajoutée.

Paramètres

`options` `existing` (la machine existe déjà), `generic`, `deviceId` (équipement d'où le dialogue est ouvert, facultatif), `scope` (`all` ou `device`), `users` (équipements de la machine).

Résultat. Les identifiants des équipements à basculer sur la variante.

Erreurs. Aucune.

`machineParamsChange` fonction

```
function machineParamsChange(project: Project, machineId: Id, params:
Record<string, unknown>, options: MachineParamsOptions = {}): MachineChange
```

Calcule, sans les appliquer, les conséquences de nouveaux paramètres (EQT-06) : variante (`deriveMachine`) si demandée ou si la machine est générique, sinon régénération de la machine (`changeMachineParams`), avec le nom éventuel. Le dialogue l'appelle à chaque saisie pour afficher les équipements qui ne tiendraient plus et les liaisons orphelines.

Paramètres

`project` le projet ;

`machineId` la machine ;

`params` les nouveaux paramètres ;

`options` `MachineParamsOptions`.

Résultat. Un `MachineChange` : `ok: true` avec la machine, le projet résultant, les liaisons orphelines et les avertissements ; ou `ok: false` avec les problèmes.

Erreurs. Aucune ; les problèmes sont dans le résultat.

`applyMachineParams` fonction

```
function applyMachineParams(editor: Editor, machineId: Id, params:
Record<string, unknown>, options: MachineParamsOptions = {}): ActionResult
```

Applique de nouveaux paramètres à une machine (EQT-06) en une action : ses équipements placés suivent ; refusé si l'un d'eux chevaucherait un voisin ou dépasserait la profondeur. Les liaisons sur des ports qui disparaissent sont gardées et signalées (avertissements, puis Vérifications, LIA-07). Les types de connecteurs par défaut nécessaires reviennent (BIB-09) ; une variante est insérée après la machine source.

Paramètres

`editor` l'éditeur ;

`machineId` la machine ;

`params` les nouveaux paramètres ;

`options` `MachineParamsOptions`.

Résultat. `ok: true` avec l'identifiant de la machine modifiée ou créée et les avertissements.

Erreurs. Refus avec les problèmes de `machineParamsChange`.

`MachineSave` type

```
type MachineSave = { mode: 'new' } | { mode: 'edit' } | { mode: 'variant'; sourceId: Id;
deviceIds: readonly Id[] };
```

Manière dont l'éditeur de machine enregistre (BIB-03, EQT-02, EQT-04).

Champs

`new` nouvelle machine, ajoutée à la bibliothèque.

`edit` la machine remplace celle de même identifiant ; ses équipements la suivent.

`variant` nouvelle machine faite à partir de `sourceId` (inchangée, par exemple une générique), donnée aux équipements `deviceIds`.

`machineSaveChange` fonction

```
function machineSaveChange(project: Project, machine: Machine, save: MachineSave):
MachineChange
```

Calcule, sans les appliquer, les conséquences de l'enregistrement d'une machine de l'éditeur : `changeMachine` en mode `edit`, `addVariantMachine` en mode `variant` ; en mode `new`, vérifie les paramètres et ajoute les types de connecteurs manquants.

Paramètres

`project` le projet ;

`machine` la machine de l'éditeur ;

`save` `MachineSave`.

Résultat. Un `MachineChange`.

Erreurs. Aucune ; les problèmes sont dans le résultat.

`saveMachine` fonction

```
function saveMachine(editor: Editor, machine: Machine, save: MachineSave): ActionResult
```

Enregistre une machine de l'éditeur de machine (EDM-01) en une action, après validation par le schéma. Une machine qui ne tient plus là où ses équipements sont placés est refusée ; les liaisons sur des ports disparus sont gardées et signalées.

Paramètres

`editor` l'éditeur ;

`machine` la machine ;

`save` `MachineSave`.

Résultat. `ok: true` avec l'identifiant de la machine et les avertissements.

Erreurs. Refus avec les messages du schéma ou les problèmes de `machineSaveChange`.

```
const result = saveMachine(editor, variant, {
  mode: 'variant',
  sourceId: genericShelf.id,
  deviceIds: [shelfDeviceId],
});
```

`setDeviceMachine` fonction

```
function setDeviceMachine(editor: Editor, deviceId: Id, machineId: Id): ActionResult
```

Donne une autre machine du même équipement à un équipement placé (EQT-04), par `changeDeviceMachine` : ses liaisons passent aux ports de même nom ; les autres sont gardées et signalées. Une action.

Paramètres

`editor` l'éditeur ;

`deviceId` l'équipement ;

`machineId` la nouvelle machine.

Résultat. `ok: true` avec `deviceId` et les avertissements.

Erreurs. Refus avec les problèmes (machine d'un autre équipement, ne tient pas à sa place).

`ensureGenericMachine` fonction

```
function ensureGenericMachine(editor: Editor, kind: EquipmentKind): Id
```

Renvoie la machine générique d'un équipement, après l'avoir rajoutée à la bibliothèque si elle manque (projet d'une autre source) : déposer un équipement du panneau place toujours sa machine générique (EQT-02). L'ajout éventuel est une action annulable (« réimporter des machines »).

Paramètres

`editor` l'éditeur ;

`kind` l'équipement.

Résultat. L'identifiant de la machine générique.

Erreurs. Aucune.

28.31 `store/pageActions`

Actions sur les pages et les vues (PAG-01, PAG-03, EXT-04) : ajout, modification, duplication, suppression et déplacement de pages ; ajout, modification, déplacement et retrait de vues. Une entrée annulable chacune. Utilisé par `commands/page`, `commands/rack`, `commands/route`, la zone de dessin et les panneaux de propriétés.

`nextPageTitle` fonction

```
function nextPageTitle(pages: readonly Pick<Page, 'title'>[]): string
```

Premier titre de page par défaut libre : « Page *n* », à partir du nombre de pages plus un.

Paramètres

`pages` les pages existantes (seul le titre est lu).

Résultat. Le titre.

Erreurs. Aucune.

addPage

fonction

```
function addPage(editor: Editor, input: Partial<Omit<Page, 'id'>> = {}, afterPageId?: Id): ActionResult
```

Ajoute une page après une page donnée (à la fin par défaut). La nouvelle page reprend le format, la taille personnalisée, l'orientation et les marges de la dernière page.

Paramètres

`editor` l'éditeur ;

`input` champs de la page, facultatifs (titre par défaut : `nextPageTitle`) ;

`afterPageId` page après laquelle insérer, facultative.

Résultat. `ok: true` avec l'identifiant de la page.

Erreurs. `ZodError` levée si `input` est invalide.

updatePage

fonction

```
function updatePage(editor: Editor, pageId: Id, patch: Partial<Omit<Page, 'id' | 'views'>>, options: { coalesceKey?: string } = {}): ActionResult
```

Modifie une page (titre, format, orientation, marges, cartouche…), sans toucher à ses vues.

Paramètres

`editor` l'éditeur ;

`pageId` la page ;

`patch` champs à remplacer ;

`options` `coalesceKey` facultative.

Résultat. `ok: true` avec `pageId`.

Erreurs. Refus sans message si la page n'existe pas.

duplicatePage

fonction

```
function duplicatePage(editor: Editor, pageId: Id): ActionResult
```

Duplique une page juste après elle, avec ses vues, ses éléments et ses tracés manuels (nouveaux identifiants) ; les points des tracés suivent les copies de leurs vues (`remapRouteViews`, ROU-16). Titre « … (copie) ».

Paramètres

`editor` l'éditeur ;

`pageId` la page.

Résultat. `ok: true` avec l'identifiant de la copie.

Erreurs. Refus sans message si la page n'existe pas.

`removePage` fonction

```
function removePage(editor: Editor, pageId: Id): ActionResult
```

Supprime une page ; le projet garde toujours au moins une page. Les armoires et équipements montrés restent.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page.

Résultat. `ok: true`.

Erreurs. Refus avec message s'il ne reste qu'une page.

`movePage` fonction

```
function movePage(editor: Editor, pageId: Id, toIndex: number): ActionResult
```

Déplace une page à un autre rang (à partir de 0, ramené dans les limites).

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `toIndex` rang voulu.

Résultat. `ok: true` avec `pageId` (rien d'enregistré si le rang ne change pas).

Erreurs. Refus sans message si la page n'existe pas.

`addExternalView` fonction

```
function addExternalView(editor: Editor, pageId: Id, deviceId: Id): ActionResult
```

Montre un équipement extérieur sur une page (EXT-04) : une vue de plus du même équipement, placée automatiquement (`externalViewLayout`).

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `deviceId` l'équipement extérieur.

Résultat. `ok: true` avec l'identifiant de la vue.

Erreurs. Refus sans message si la page n'existe pas, si elle montre déjà l'équipement ou si aucune disposition n'est possible.

`addView` fonction

```
function addView(editor: Editor, pageId: Id, input: ViewInput): ActionResult
```

Ajoute une vue construite par l'appelant (position, échelle, plage de U…) à une page.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `input` description de la vue (`ViewInput` : sorte, cible, position en mm, échelle…).

Résultat. `ok: true` avec l'identifiant de la vue.

Erreurs. Refus sans message si la page n'existe pas ; `ZodError` si `input` est invalide.

`updateView` fonction

```
function updateView(editor: Editor, pageId: Id, viewId: Id, patch: Partial<Omit<View, 'id'
| 'kind' | 'targetId'>>, options: { coalesceKey?: string } = {}): ActionResult
```

Modifie une vue (position, échelle, taille, plage de U, repères, faisceaux, câbles masqués…), sans changer sa sorte ni sa cible. Une nouvelle plage de U (PAG-16) ou des câbles masqués (PAG-15) gardent les tracés retouchés de la page.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `viewId` la vue ;
- `patch` champs à remplacer ;
- `options` `coalesceKey` facultative (redimensionnement au glisser).

Résultat. `ok: true` avec `viewId`.

Erreurs. Aucune (rien n'est fait pour une vue inconnue).

`moveViews` fonction

```
function moveViews(editor: Editor, pageId: Id, viewIds: readonly Id[], dx: number, dy:
number, options: { coalesceKey?: string } = {}): ActionResult
```

Déplace des vues d'une page de (`dx`, `dy`) mm, positions arrondies au dixième ; avec une clé de fusion au glisser.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `viewIds` les vues ;
- `dx` décalage horizontal (mm) ;
- `dy` décalage vertical (mm) ;
- `options` `coalesceKey` facultative.

Résultat. `ok: true`.

Erreurs. Aucune.

`removeViews` fonction

```
function removeViews(editor: Editor, pageId: Id, viewIds: readonly Id[]): ActionResult
```

Retire des vues d'une page ; les armoires et équipements restent. Les points des tracés rattachés aux vues retirées passent à la vue la plus proche (par l'éditeur).

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;

`viewIds` les vues.

Résultat. `ok: true`.

Erreurs. Aucune.

28.32 `store/portActions`

Ports ajoutés à un équipement placé seulement (PRT-04) : ajout, modification, suppression. Les règles (place libre sur la face, nom unique) sont dans `model/instancePorts`. Utilisé par `device.addPort` (`commands/inspect`) et le panneau des propriétés.

`addInstancePort` fonction

```
function addInstancePort(editor: Editor, deviceId: Id, face: FaceSide, connectorTypeId:
Id): ActionResult
```

Ajoute un port à un équipement, à une place libre de la face, avec un nom automatique (`newInstancePort`).

Paramètres

`editor` l'éditeur ;

`deviceId` l'équipement ;

`face` `front` ou `rear` ;

`connectorTypeId` type de connecteur du port.

Résultat. `ok: true` avec l'identifiant du port.

Erreurs. Refus sans message si l'équipement n'existe pas ou si la face n'a plus de place.

`updateInstancePort` fonction

```
function updateInstancePort(editor: Editor, deviceId: Id, portId: Id, patch:
Partial<Omit<InstancePort, 'id'>>): ActionResult
```

Modifie un port ajouté (nom, connecteur, position…) ; le nom est débarrassé de ses espaces de bord, et le résultat est vérifié par `instancePortProblems`.

Paramètres

`editor` l'éditeur ;

`deviceId` l'équipement ;

`portId` le port ajouté ;

`patch` champs à remplacer.

Résultat. `ok: true` avec `portId`.

Erreurs. Refus sans message si l'équipement ou le port n'existe pas ; refus avec les problèmes (nom vide ou en double, position…).

`removeInstancePort` fonction

```
function removeInstancePort(editor: Editor, deviceId: Id, portId: Id): ActionResult
```

Supprime un port ajouté, avec les liaisons qui y sont attachées (et leurs tracés) et la surcharge de son nom affiché. La confirmation est demandée par l'interface.

Paramètres

- `editor` l'éditeur ;
- `deviceId` l'équipement ;
- `portId` le port.

Résultat. `ok: true`.

Erreurs. Aucune.

28.33 `store/projectActions`

Révisions du projet (PRJ-11) et champs du cartouche (PAG-10) : une entrée annulable chacune. Utilisé par les dialogues des révisions et du cartouche, `project.addRevision` et le projet d'exemple.

`addRevision` fonction

```
function addRevision(editor: Editor, input: Partial<Omit<Revision, 'id'>> = {}):
ActionResult
```

Ajoute une révision : indice suivant (`nextRevisionIndex`), auteur du projet, date du jour, sauf valeurs données.

Paramètres

- `editor` l'éditeur ;
- `input` champs de la révision, facultatifs (indice, date, auteur, description).

Résultat. `ok: true` avec l'identifiant de la révision.

Erreurs. `ZodError` levée si `input` est invalide.

`updateRevision` fonction

```
function updateRevision(editor: Editor, revisionId: Id, patch:
Partial<Omit<Revision, 'id'>>): ActionResult
```

Modifie une révision.

Paramètres

- `editor` l'éditeur ;
- `revisionId` la révision ;
- `patch` champs à remplacer.

Résultat. `ok: true` avec `revisionId`.

Erreurs. Aucune.

`removeRevision` fonction

```
function removeRevision(editor: Editor, revisionId: Id): ActionResult
```

Supprime une révision.

Paramètres

`editor` l'éditeur ;

`revisionId` la révision.

Résultat. `ok: true`.

Erreurs. Aucune.

`addTitleBlockField` fonction

```
function addTitleBlockField(editor: Editor, input: Partial<Omit<TitleBlockField, 'id'>> &
Pick<TitleBlockField, 'kind'>): ActionResult
```

Ajoute un champ à la fin du cartouche : un champ personnalisé, ou un champ du projet pas encore montré (client…). Une sorte autre que `custom` n'est présente qu'une fois.

Paramètres

`editor` l'éditeur ;

`input` sorte du champ (obligatoire), libellé, valeur, rangée, visibilité…

Résultat. `ok: true` avec l'identifiant du champ.

Erreurs. Refus sans message si un champ de cette sorte existe déjà.

`updateTitleBlockField` fonction

```
function updateTitleBlockField(editor: Editor, fieldId: Id, patch:
Partial<Omit<TitleBlockField, 'id' | 'kind'>>): ActionResult
```

Modifie un champ du cartouche (libellé, valeur, rangée, visibilité, largeur…), sans changer sa sorte.

Paramètres

`editor` l'éditeur ;

`fieldId` le champ ;

`patch` champs à remplacer.

Résultat. `ok: true` avec `fieldId` (rien d'enregistré si rien ne change).

Erreurs. Refus sans message si le champ n'existe pas.

`moveTitleBlockField` fonction

```
function moveTitleBlockField(editor: Editor, fieldId: Id, delta: number): ActionResult
```

Déplace un champ dans la liste du cartouche (son ordre d'affichage), de `delta` places, dans les limites de la liste.

Paramètres

`editor` l'éditeur ;

`fieldId` le champ ;

`delta` nombre de places, négatif vers le début.

Résultat. `ok: true` avec `fieldId`.

Erreurs. Refus sans message si le champ n'existe pas ou ne bouge pas (déjà au bord).

`removeTitleBlockField` fonction

```
function removeTitleBlockField(editor: Editor, fieldId: Id): ActionResult
```

Supprime un champ du cartouche.

Paramètres

`editor` l'éditeur ;

`fieldId` le champ.

Résultat. `ok: true`.

Erreurs. Refus sans message si le champ n'existe pas.

28.34 `store/rackActions`

Actions d'armoire de l'interface (ARM-01, PAG-13) : ajout avec ses vues, vues d'une armoire existante, page d'une armoire, duplication avec ou sans contenu. Les dispositions sont calculées par `model/autoLayout` ; les nouvelles vues sont ajoutées après les vues de la page, qui gardent leur échelle et leur position, même si les nouvelles sortent de la zone de dessin (décision de l'utilisateur du 2026-11-06, PAG-03 ; la réduction de toutes les vues du lot 46 est abandonnée). Utilisé par le dialogue d'ajout d'armoire, `commands/rack` et le projet d'exemple.

`nextRackName` fonction

```
function nextRackName(racks: readonly Pick<Rack, 'name'>[], base = t('defaults.rackName')): string
```

Premier nom d'armoire libre : « Armoire *n* », à partir du nombre d'armoires plus un.

Paramètres

`racks` les armoires existantes (seul le nom est lu) ;

`base` début du nom (`defaults.rackName` par défaut).

Résultat. Le nom.

Erreurs. Aucune.

`AddRackOptions` interface

```
interface AddRackOptions { modelId?; name?; pageId?; faces? }
```

Options de `addRackWithViews`.

Champs

`modelId` modèle d'armoire de la bibliothèque à copier ; aucun : caractéristiques par défaut.

`name` nom ; vide : `nextRackName`.

`pageId` page où ajouter les vues de l'armoire ; aucune : pas de vue.

`faces` faces à montrer (`front` et `rear` par défaut).

`addRackWithViews` fonction

```
function addRackWithViews(editor: Editor, options: AddRackOptions = {}): ActionResult
```

Ajoute une armoire et, sur la page donnée, ses vues avant et arrière placées automatiquement (`rackViewsLayout`), en une action (transaction).

Paramètres

`editor` l'éditeur ;

`options` `AddRackOptions`.

Résultat. `ok: true` avec l'identifiant de l'armoire.

Erreurs. Aucune.

```
const page = editor.getState().project.pages[0]!;
const { id } = addRackWithViews(editor, { name: 'Baie 01', pageId: page.id });
```

`addRackViews` fonction

```
function addRackViews(editor: Editor, pageId: Id, rackId: Id, faces: readonly FaceSide[]):
ActionResult
```

Ajoute à une page des vues d'une armoire existante (avant, arrière ou les deux), placées automatiquement.

Paramètres

`editor` l'éditeur ;

`pageId` la page ;

`rackId` l'armoire ;

`faces` faces à montrer.

Résultat. `ok: true` avec l'identifiant de la première vue ajoutée.

Erreurs. Refus sans message si la page ou l'armoire n'existe pas.

`createRackPage` fonction

```
function createRackPage(editor: Editor, rackId: Id, afterPageId?: Id | null): ActionResult
```

« Créer une page pour cette armoire » (PAG-13) : une nouvelle page après `afterPageId` (à la fin par défaut), titrée du nom de l'armoire, avec les réglages de papier de cette page (ou de la dernière), ses vues avant et arrière et les équipements extérieurs qui lui sont liés, disposés automatiquement (`rackPageLayout`). Une action.

Paramètres

`editor` l'éditeur ;

`rackId` l'armoire ;

`afterPageId` page de référence et d'insertion, facultative.

Résultat. `ok: true` avec l'identifiant de la page.

Erreurs. Refus sans message si l'armoire n'existe pas.

`duplicateRack` fonction

```
function duplicateRack(editor: Editor, rackId: Id, options: { withContent: boolean;
pageId?: Id | null }): ActionResult
```

Duplique une armoire (ARM-01), nom « … (copie) », avec ou sans ses équipements. Les équipements copiés ont de nouveaux identifiants et gardent leurs noms ; les liaisons ne sont pas copiées (un port porte une liaison, la copie est à recâbler). Sur la page donnée, la copie reçoit des vues des mêmes faces que l'original (avant et arrière s'il n'en a pas). Une action.

Paramètres

`editor` l'éditeur ;

`rackId` l'armoire ;

`options` `withContent` : copier les équipements ; `pageId` : page des vues de la copie, facultative.

Résultat. `ok: true` avec l'identifiant de la copie.

Erreurs. Refus sans message si l'armoire n'existe pas.

28.35 `store/react`

Liaison entre l'éditeur et React : contexte et crochets. Seul module de `src/store` qui dépend de React ; non réexporté par `store/index`. Le contexte est fourni par `src/main.tsx` ; les crochets sont utilisés par tous les composants qui lisent le projet.

`EditorContext` constante

```
const EditorContext = createContext<Editor | null>(null)
```

Contexte React qui porte l'éditeur de l'application (`null` par défaut). Fourni à la racine de l'arbre ; les tests de composants le fournissent avec un éditeur créé pour l'occasion.

`useEditor` fonction

```
function useEditor(): Editor
```

Renvoie l'éditeur du contexte, pour appeler des actions depuis un composant.

Résultat. L'`Editor`.

Erreurs. Lève `Error('EditorContext is missing')` hors d'un fournisseur de `EditorContext`.

`useEditorState` fonction

```
function useEditorState<T>(selector: (state: EditorState) => T): T
```

Abonne le composant à une partie de l'état de l'éditeur : le composant se redessine quand la valeur renvoyée par le sélecteur change (comparaison par référence). Utiliser les sélecteurs de `store/selectors` (NF-02b).

Paramètres

`selector` fonction de l'état vers la valeur voulue.

Résultat. La valeur sélectionnée.

Erreurs. Celle de `useEditor` hors contexte.

```
const rack = useEditorState(selectRack(rackId));
const dirty = useEditorState(selectIsDirty);
```

28.36 `store/routeActions`

Tracés manuels des câbles, conservés par page (ROU-06, ROU-07), rattachement de leurs points aux vues (ROU-16, `routing/attach`) et faisceaux (ROU-11). Utilisé par la zone de dessin (retouche d'un tracé), `commands/route` et l'éditeur (`followRemovedViews`).

`setManualRoute` fonction

```
function setManualRoute(editor: Editor, pageId: Id, linkId: Id, points: Point[], options: { coalesceKey?: string } = {}): ActionResult
```

Fixe le tracé manuel d'un câble sur une page (glisser d'un segment ou d'un point : fusionné par sa clé). Les points, donnés en mm de page, sont rattachés aux vues de la page (`attachPoints`) avant d'être enregistrés.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `linkId` la liaison ;
- `points` points de passage, en mm de page ;
- `options` `coalesceKey` du geste.

Résultat. `ok: true`.

Erreurs. Refus sans message si la page n'existe pas ou si les points ne peuvent pas être rattachés (page sans vue).

`resetRoutes` fonction

```
function resetRoutes(editor: Editor, pageId: Id, linkIds?: readonly Id[]): ActionResult
```

Rend le tracé automatique (ROU-07) à certains câbles d'une page, ou à tous les câbles de la page quand `linkIds` est absent.

Paramètres

- `editor` l'éditeur ;
- `pageId` la page ;
- `linkIds` liaisons concernées, facultatives.

Résultat. `ok: true`.

Erreurs. Aucune.

`followRemovedViews` fonction

```
function followRemovedViews(before: Project, draft: Draft<Project>): void
```

Après une modification qui a retiré des vues (vue, armoire, équipement extérieur…), rattache les points des tracés manuels qui leur étaient attachés à la vue restante la plus proche de leur page (`reattachRemovedViews`, ROU-16). Appelée par `Editor.apply` sur chaque modification, dans le brouillon : aucune action n'a à l'appeler.

Paramètres

`before` le projet avant la modification ;

`draft` le brouillon après la recette.

Résultat. Aucun ; le brouillon est modifié (rien si aucune vue d'une page à tracés n'a disparu).

Erreurs. Aucune.

`setBundled` fonction

```
function setBundled(editor: Editor, linkIds: readonly Id[], bundle: boolean): ActionResult
```

Met des câbles dans leur faisceau ou les en sort (ROU-11) : « Regrouper en faisceau » / « Dégrouper le faisceau », une étape d'annulation.

Paramètres

`editor` l'éditeur ;

`linkIds` les liaisons ;

`bundle` `true` pour regrouper, `false` pour dégrouper.

Résultat. `ok: true`.

Erreurs. Aucune.

28.37 `store/routingActions`

Réglages de routage (ROU-02) : goulotte (côté gauche ou droit) de chaque type de liaison, pour tout le projet ou pour une armoire, qui l'emporte. Utilisé par le dialogue de routage et le panneau des propriétés d'une armoire.

`setProjectTray` fonction

```
function setProjectTray(editor: Editor, linkTypeId: Id, side: LateralSide | null):
ActionResult
```

Fixe, ou efface avec `null`, la goulotte d'un type de liaison pour tout le projet.

Paramètres

`editor` l'éditeur ;

`linkTypeId` le type de liaison ;

`side` `left`, `right` ou `null`.

Résultat. `ok: true`.

Erreurs. Aucune.

`setRackTray` fonction

```
function setRackTray(editor: Editor, rackId: Id, linkTypeId: Id, side: LateralSide | null): ActionResult
```

Fixe, ou efface avec `null`, la goulotte d'un type de liaison dans une armoire.

Paramètres

- `editor` l'éditeur ;
- `rackId` l'armoire ;
- `linkTypeId` le type de liaison ;
- `side` `left`, `right` ou `null`.

Résultat. `ok: true` (rien n'est fait si l'armoire n'existe pas).

Erreurs. Aucune.

28.38 `store/selectors`

Sélecteurs de l'état de l'éditeur, à passer à `useEditorState` (NF-02b). Avec le partage structurel d'Immer, une entité garde la même référence tant qu'elle n'est pas modifiée : un composant abonné à une entité ne se redessine que lorsqu'elle change. Les sélecteurs par identifiant passent par l'index en cache du projet (`indexProject`).

`selectProject` constante

```
const selectProject = (state: EditorState) => …
```

Sélectionne le projet entier ; le composant se redessine à chaque modification.

`selectMeta` constante

```
const selectMeta = (state: EditorState) => …
```

Sélectionne les propriétés du projet (nom, client, auteur…), pour l'en-tête et les dialogues.

`selectIsDirty` constante

```
const selectIsDirty = (state: EditorState) => …
```

Sélectionne l'état « modifié » (`isDirty`), pour le titre de la fenêtre et l'indicateur d'enregistrement.

`selectCanUndo` constante

```
const selectCanUndo = (state: EditorState) => …
```

Sélectionne la possibilité d'annuler (`past` non vide).

`selectCanRedo` constante

```
const selectCanRedo = (state: EditorState) => …
```

Sélectionne la possibilité de rétablir (`future` non vide).

`selectRack` constante

```
const selectRack = (id: Id) => …
```

Fabrique un sélecteur de l'armoire d'identifiant donné.

Paramètres

`id` l'armoire.

Résultat. Un sélecteur qui renvoie la `Rack` ou `undefined`.

Erreurs. Aucune.

`selectDevice` constante

```
const selectDevice = (id: Id) => …
```

Fabrique un sélecteur de l'équipement d'identifiant donné.

Paramètres

`id` l'équipement.

Résultat. Un sélecteur qui renvoie le `Device` ou `undefined`.

Erreurs. Aucune.

`selectLink` constante

```
const selectLink = (id: Id) => …
```

Fabrique un sélecteur de la liaison d'identifiant donné.

Paramètres

`id` la liaison.

Résultat. Un sélecteur qui renvoie la `Link` ou `undefined`.

Erreurs. Aucune.

`selectModel` constante

```
const selectModel = (id: Id) => …
```

Fabrique un sélecteur de la machine (modèle de la bibliothèque) d'identifiant donné.

Paramètres

`id` la machine.

Résultat. Un sélecteur qui renvoie la `Machine` ou `undefined`.

Erreurs. Aucune.

`selectPage` constante

```
const selectPage = (id: Id) => …
```

Fabrique un sélecteur de la page d'identifiant donné (recherche dans la liste des pages).

Paramètres

`id` la page.

Résultat. Un sélecteur qui renvoie la `Page` ou `undefined`.

Erreurs. Aucune.

Chapitre 29

Référence : interface utilisateur

Ce chapitre décrit les modules de `src/ui/` (chapitre 13 pour leur organisation). La plupart des déclarations sont des composants React : leur signature (extraite du code) donne leurs propriétés, la description en précise le sens, l'endroit où le composant est monté et son comportement notable. Sauf mention contraire, un composant lit le projet par `useEditorState`, l'état de l'interface par `useUi`, et ne modifie le projet que par les actions annulables de `src/store/`. Les modules sont groupés : fenêtre et commandes, état et crochets, flux d'interaction, zone de dessin, panneaux, propriétés, champs, dialogues, exports, machines et éditeur de faces.

Fenêtre principale et commandes

29.1 `ui/App`

Fenêtre principale de l'application (ERG-01) : en-tête avec la barre de menus, barre d'outils, panneau de gauche, zone de dessin et onglets de pages, panneau des propriétés, puis toutes les couches modales. Monté par `src/main.tsx` à l'intérieur des contextes de l'éditeur, de l'interface et de la session. Dépend de presque tous les autres modules de `src/ui/`, de `app/session` et de `app/pwa`.

App fonction

```
function App({ session, onShown }: { session: Session; onShown?: () => void })
```

Composant racine de l'interface, monté une seule fois par `main.tsx`. Il installe les raccourcis (`useCommandShortcuts`), avertit à la fermeture de l'onglet si le projet est modifié (PRJ-04, sauf pendant un rechargement de mise à jour `updateReload.pending`) et ouvre un fichier `.rkdg` ou `.rkdglib` déposé sur la fenêtre (PRJ-02, BIB-13). En édition de bibliothèque (`project.editedLibrary` défini), la zone centrale montre `LibrarySheet` et le panneau des propriétés est masqué.

Paramètres

`session` la session (fichiers, brouillon, erreurs), passée aussi à l'écran d'accueil, au dialogue du brouillon et au dialogue d'erreur ;

`onShown` facultatif ; appelé après le premier rendu, une fois les raccourcis écoutés : `main.tsx` s'en sert pour marquer le crochet de test `window.__portolan` comme prêt.

29.2 `ui/CommandButton`

Bouton générique qui exécute une commande du registre (`commands/registry`). Utilisé par la barre d'outils, les onglets de pages, les panneaux, les propriétés et certains dialogues.

`CommandButton` fonction

```
function CommandButton({ id, className, children, }: { id: string; className?: string;
children?: React.ReactNode; })
```

Bouton qui lance la commande `id` par `executeCommand`. Libellé, raccourci (affiché dans l'infobulle `title` sous la forme « Libellé (Ctrl+S) ») et état activé viennent du registre, tenus à jour par `useCommand`. Le bouton porte `data-command` ; il n'est pas rendu si la commande n'existe pas.

Paramètres

- `id` identifiant de la commande, par exemple `file.save` ;
- `className` classe CSS (`tool-button` par défaut) ;
- `children` contenu à la place du libellé (le « + » des onglets de pages).

29.3 `ui/ContextMenu`

Menu contextuel (ERG-04), affiché d'après `contextMenu` de l'état de l'interface. Ouvert par la zone de dessin, l'explorateur, la bibliothèque et les onglets de pages. Réutilise `CommandList`.

`ContextMenu` fonction

```
function ContextMenu()
```

Monté une fois dans `App`. Affiche les commandes de `ContextMenuState` à la position demandée, recalée pour rester dans la fenêtre ; se ferme sur un clic en dehors ou après l'exécution d'une commande. Le menu contextuel du navigateur est bloqué sur le menu lui-même. Sans propriétés.

29.4 `ui/DraftDialog`

Proposition de restaurer un brouillon non enregistré au démarrage (PRJ-05), d'après `draftOffer` de la session.

`DraftDialog` fonction

```
function DraftDialog({ session }: { session: Session })
```

Monté dans `App`, au-dessus de l'écran d'accueil (qui attend la réponse). Montre le nom du projet, la date du brouillon et son fichier ; « Restaurer » ferme l'accueil et appelle `session.restoreDraft()`, l'autre bouton `session.dismissDraft()`. Rôle `dialog`, sans fermeture par Échap : il faut choisir.

Paramètres

- `session` la session qui porte la proposition.

29.5 `ui/ErrorDialog`

Message d'erreur d'une ouverture ou d'un enregistrement (ERG-07, REC-03), d'après `error` de la session.

`ErrorDialog` fonction

```
function ErrorDialog({ session }: { session: Session })
```

Monté dans `App`. Rôle `alertdialog` ; affiche le message et la liste des détails (problèmes de validation du fichier, par exemple) ; « Fermer » appelle `session.dismissError`.

Paramètres

`session` la session dont l'erreur est affichée.

29.6 `ui/MenuBar`

Barre de menus (ERG-01) et liste de commandes partagée avec le menu contextuel. Les menus sont définis par `ui/menus` ; libellés, raccourcis, état et explications viennent du registre des commandes.

`menuEntries` fonction

```
function menuEntries(items: readonly string[]): string[]
```

Nettoie une liste d'entrées de menu : retire les identifiants absents du registre, les séparateurs en tête, en fin et en double. Appelée par `CommandList` à chaque rendu.

Paramètres

`items` identifiants de commandes, « - » pour un séparateur.

Résultat. Nouvelle liste, sans séparateur superflu.

Erreurs. Aucune.

```
menuEntries(['-', 'edit.undo', '-', '-', 'inconnue', 'edit.redo', '-']);
// ['edit.undo', '-', 'edit.redo']
```

`CommandList` fonction

```
function CommandList({ items, ctx, onDone, }: { items: readonly string[]; ctx:
CommandContext; onDone: () => void; })
```

Liste `role="menu"` des commandes d'un menu ou du menu contextuel. Chaque entrée est un bouton `menuitem` (ou `menuitemcheckbox` avec `aria-checked` quand la commande a `isChecked`), désactivé selon le registre ; une commande désactivée qui fournit une explication (`disabledHintKey`) l'affiche sous l'entrée (`aria-describedby`). Se met à jour par `useStateVersion`.

Paramètres

`items` entrées du menu ;

`ctx` contexte passé aux commandes ;

`onDone` appelé avant l'exécution d'une commande (fermeture du menu).

MenuBar fonction

```
function MenuBar()
```

Barre de menus de l'en-tête (`nav` nommée « Menus »), un bouton par menu de `MENUS` (`aria-haspopup`, `aria-expanded`, `data-menu`). Un seul menu ouvert à la fois ; le survol passe d'un menu à l'autre quand l'un est ouvert ; Échap ou un clic en dehors le ferme. Sans propriétés.

29.7 `ui/Messages`

Messages communs (ERG-07) : confirmations et alertes, une à la fois au-dessus de tout dialogue, et notifications courtes en bas de la fenêtre. Ils affichent les files `messages` et `toasts` de l'état de l'interface, remplies par `ui.confirm`, `ui.alert` et `ui.notify`.

MessageHost fonction

```
function MessageHost()
```

Monté dans `App`. Affiche le plus ancien `MessageRequest` dans un `Modal` de rôle `alertdialog` : titre (par défaut « Confirmation » ou « Information »), texte, détails en liste, bouton d'annulation pour une confirmation, bouton de validation (rouge si `danger`). Échap ou le fond répondent « non ». Sans propriétés.

Toasts fonction

```
function Toasts()
```

Monté dans `App`. Pile de notifications (`role="status"`, `aria-live="polite"`) ; chacune disparaît après 5 secondes ou par son bouton de fermeture. La classe CSS reprend la sorte (`toast-info`, `toast-warning`, `toast-error`). Sans propriétés.

29.8 `ui/Modal`

Cadre commun des dialogues (ERG-07), utilisé par tous les dialogues de `src/ui/dialogs/`, `exports/`, `machine/` et par `MessageHost`.

Modal fonction

```
function Modal({ title, onClose, children, footer, wide, role = 'dialog', className, }: {
title: string; onClose: () => void; children: React.ReactNode; footer?: React.ReactNode;
wide?: boolean | 'xl'; role?: 'dialog' | 'alertdialog'; className?: string; })
```

Fenêtre modale : fond qui ferme le dialogue au clic, cadre de rôle `dialog` (ou `alertdialog`) avec `aria-modal` et un titre relié par `aria-labelledby`. À l'ouverture, le focus va au premier élément `autofocus` ou au premier champ ou bouton ; il revient à l'élément précédent à la fermeture. Échap ferme le dialogue sans se propager.

Paramètres

`title` titre affiché (nom accessible du dialogue) ;

`onClose` fermeture (Échap, clic sur le fond) ;

`children` corps du dialogue ;

`footer` boutons du pied, facultatif ;

`wide` `true` pour un dialogue large, `'xl'` pour très large ;

`role` `dialog` (défaut) ou `alertdialog` ;

`className` classe CSS supplémentaire.

29.9 `ui/PageTabs`

Onglets de pages en bas de la zone de dessin (PAG-01, ERG-01). Utilise `movePage` et `updatePage` (`store/pageActions`).

`PageTabs` fonction

```
function PageTabs()
```

Monté dans la zone centrale de `App`. Un onglet `role="tab"` par page (`data-page-tab`) : clic ou Entrée pour l'activer, double-clic ou F2 pour le renommer en place (champ validé par Entrée ou en quittant, annulé par Échap), glisser pour réordonner, clic droit pour le menu contextuel des pages (renommer, dupliquer, déplacer, supprimer, ajouter). Un bouton « + » exécute `page.add`. Sans propriétés.

29.10 `ui/Toolbar`

Barre d'outils (ERG-01) : commandes de `TOOLBAR`.

`Toolbar` fonction

```
function Toolbar()
```

Monté sous l'en-tête par `App` (`role="toolbar"`). Rend un `CommandButton` par commande existante, un séparateur pour « - » et, pour l'entrée `zoom`, le niveau de zoom en pourcentage de la taille réelle (`data-testid="zoom-level"`). Sans propriétés.

29.11 `ui/UpdateBanner`

Bandeau « une nouvelle version est disponible » (WEB-08), affiché quand `update` de l'état de l'interface est renseigné par `app/pwa`.

`UpdateBanner` fonction

```
function UpdateBanner()
```

Monté dans `App` (`role="status"`). Propose « Recharger » (commande `app.reload`) ou « Plus tard », qui remet `update` à `null`. Sans propriétés.

29.12 `ui/menus`

Contenu des menus : listes d'identifiants de commandes du registre, « - » marquant un séparateur. Les commandes absentes du registre sont ignorées à l'affichage (`menuEntries`). Aucune dépendance à React ; utilisé par la barre de menus, la barre d'outils, la zone de dessin et l'explorateur.

`MenuDefinition` interface

```
interface MenuDefinition { id; labelKey; items }
```

Définition d'un menu de la barre de menus.

Champs

- `id` identifiant (`file`, `edit`, `view`, `insert`, `library`, `help`), repris dans `data-menu` ;
- `labelKey` clé i18n du titre du menu ;
- `items` identifiants de commandes et séparateurs, dans l'ordre d'affichage.

`MENUS` constante

```
const MENUS: MenuDefinition[]
```

Les six menus de la barre (ERG-01), dans l'ordre : Fichier, Édition, Affichage, Insertion, Bibliothèque, Aide. Ajouter une commande à un menu revient à ajouter son identifiant ici ; le manuel et le contrôle `check:doc` portent sur la commande elle-même.

`contextMenuItems` fonction

```
function contextMenuItems(selection: readonly SelectionItem[], project?: Project): string[]
```

Commandes du menu contextuel selon ce qui a été cliqué du bouton droit (ERG-04) : rien (coller, ajouter une armoire ou une page, tout sélectionner, ajuster), des liaisons, des ports, des équipements, des éléments de schéma, des armoires, ou par défaut des vues (repères, faisceaux, câbles masqués, retirer de la page…). Appelée par la zone de dessin et l'explorateur.

Paramètres

- `selection` la sélection après le clic droit ;
- `project` facultatif (explorateur) : pour un équipement extérieur seul sélectionné, ajoute en tête `view.addExternal` (l'afficher sur la page active, EXT-04).

Résultat. Liste d'identifiants avec séparateurs.

Erreurs. Aucune.

`TOOLBAR` constante

```
const TOOLBAR: string[]
```

Boutons de la barre d'outils (ERG-01) : ouvrir, enregistrer, annuler, rétablir, ajouter une armoire, créer une liaison, ajouter une page, zoom, vérifications. L'entrée spéciale `zoom` affiche le niveau de zoom.

29.13 `ui/useCommandShortcuts`

Raccourcis clavier globaux du registre des commandes (ERG-03, ERG-10), installés par `App`. S'appuie sur `commandForKey` et `isCommandEnabled` (`commands/registry`) et sur `ui/canvas/coords`.

`useCommandShortcuts` fonction

```
function useCommandShortcuts(ctx: CommandContext)
```

Crochet qui écoute `keydown` et `keyup` sur la fenêtre. Quand un dialogue modal est ouvert (élément `.modal-backdrop` présent), quand le focus est dans un champ de texte, ou pour Espace sur une case à cocher ou un bouton radio, seules les commandes `allowInTextField` répondent. Une commande désactivée laisse la touche au navigateur ; sinon la touche est annulée, la commande exécutée et le relâchement de la même touche annulé aussi (un bouton qui a le focus n'est pas activé par Espace).

Paramètres

`ctx` contexte des commandes (`useCommandContext`) ; l'écoute est réinstallée quand il change.

Résultat. Aucun.

Erreurs. Aucune.

État de l'interface et crochets

29.14 `ui/state`

État de l'interface hors du projet et de son historique (décision du 2026-11-04, lot 13) : page active, zoom, sélection, panneaux, dialogue ouvert, menu contextuel, messages, presse-papiers, modes d'affichage. Store Zustand « vanilla », indépendant de React : les commandes le reçoivent dans `ctx.ui`, les composants s'y abonnent par `useUi`. Dépend de `model/lookup` et de l'éditeur (`store/editor`). Testé par `src/ui/state.test.ts`.

`SelectionKind` type

```
type SelectionKind = 'view' | 'rack' | 'device' | 'link' | 'port' | 'annotation';
```

Sorte d'un élément sélectionné : `view` (vue de la page active), `rack`, `device`, `link`, `port` (port d'un équipement) ou `annotation` (élément de l'onglet « Schéma » de la page active, PAG-07). Les sortes `view` et `annotation` n'ont de sens que sur la page active.

`SelectionItem` interface

```
interface SelectionItem { kind; id; deviceId? }
```

Un élément de la sélection.

Champs

`kind` sa sorte (`SelectionKind`) ;

`id` identifiant de l'entité (pour un port, l'identifiant du port) ;

`deviceId` pour un port seulement : l'équipement qui le porte.

`selectionKey` fonction

```
function selectionKey(item: SelectionItem): string
```

Clé texte unique d'un élément de sélection, `sorte:équipement:identifiant` ; deux éléments sont le même quand leurs clés sont égales. Sert à dédoublonner et à tester l'appartenance à la sélection.

Paramètres

`item` l'élément.

Résultat. La clé.

Erreurs. Aucune.

```
selectionKey({ kind: 'port', id: 'p1', deviceId: 'd1' }); // 'port:d1:p1'
selectionKey({ kind: 'rack', id: 'r1' });                 // 'rack::r1'
```

`selectionExists` fonction

```
function selectionExists(project: Project, pageId: Id | null, item: SelectionItem): boolean
```

Indique si un élément sélectionné existe encore dans le projet (après une annulation ou une suppression). Une vue ou un élément de schéma doit appartenir à la page donnée ; un port doit exister sur son équipement (ports de la machine ou ports ajoutés, PRT-04). Appelée par `createUi` à chaque changement de projet.

Paramètres

`project` le projet ;

`pageId` la page active (`null` si aucune) ;

`item` l'élément à vérifier.

Résultat. `true` si l'élément est toujours valide.

Erreurs. Aucune.

`DialogRequest` interface

```
interface DialogRequest { type; props? }
```

Dialogue ouvert (un seul à la fois ; les messages peuvent s'afficher au-dessus).

Champs

`type` nom du dialogue, clé de `DIALOGS` ;

`props` paramètres facultatifs passés au dialogue (machine à éditer, format d'export…).

`MessageRequest` interface

```
interface MessageRequest { id; kind; title?; message; details?; confirmLabel?;
cancelLabel?; danger?; resolve }
```

Message en attente de réponse, affiché par `MessageHost`.

Champs

`id` numéro unique ;

`kind` `confirm` (deux boutons) ou `alert` (un bouton « OK ») ;

`title` titre facultatif ;

`message` texte principal ;

`details` lignes de détail facultatives, affichées en liste ;

`confirmLabel` , `cancelLabel` libellés des boutons, déjà traduits ;

`danger` confirmation destructive (bouton rouge) ;

`resolve` résout la promesse de `ui.confirm` ou `ui.alert`.

`Toast` interface

```
interface Toast { id; message; kind }
```

Notification courte, affichée par `Toasts`.

Champs

`id` numéro unique ;

`message` texte ;

`kind` `info`, `warning` ou `error`.

`ContextMenuState` interface

```
interface ContextMenuState { x; y; items }
```

Menu contextuel ouvert, affiché par `ContextMenu`.

Champs

`x` , `y` position du clic, en pixels de la fenêtre ;

`items` identifiants de commandes, « - » pour un séparateur.

`ZOOM_MIN` constante

```
const ZOOM_MIN = 0.2
```

Zoom minimal, 0,2 pixel écran par millimètre de papier (environ 5 % de la taille réelle). Borne de `clampZoom`.

`ZOOM_MAX` constante

```
const ZOOM_MAX = 16
```

Zoom maximal, 16 pixels par millimètre (environ 430 % de la taille réelle). Borne de `clampZoom`.

`ZOOM_STEP` constante

```
const ZOOM_STEP = 1.25
```

Facteur d'un pas de zoom (1,25), utilisé par les commandes « Zoom avant » et « Zoom arrière ».

`ZOOM_ACTUAL` constante

```
const ZOOM_ACTUAL = 96 / 25.4
```

Zoom de la « taille réelle » sur un écran à 96 dpi : 96/25,4 pixels par millimètre. La barre d'outils exprime le zoom en pourcentage de cette valeur.

`FIT_MARGIN_PX` constante

```
const FIT_MARGIN_PX = 24
```

Marge laissée autour de la page quand on l'ajuste à la zone de dessin, en pixels (24). Utilisée par `fitZoom`.

`clampZoom` fonction

```
function clampZoom(zoom: number): number
```

Borne un zoom entre `ZOOM_MIN` et `ZOOM_MAX`. Appelée par `setZoom`, `zoomBy` et `fitZoom`.

Paramètres

`zoom` zoom demandé, en pixels par millimètre.

Résultat. Le zoom borné.

Erreurs. Aucune.

`fitZoom` fonction

```
function fitZoom(page: { width: number; height: number }, viewport: { width: number;
height: number }): number
```

Zoom qui montre la page entière dans la zone de dessin, avec `FIT_MARGIN_PX` de marge de chaque côté, borné. Appelée par la commande « Ajuster à la page » et par la zone de dessin à l'ouverture d'un projet.

Paramètres

`page` taille de la page, en millimètres ;

`viewport` taille de la zone de dessin, en pixels.

Résultat. Le zoom ; 1 si la zone n'a pas encore de taille.

Erreurs. Aucune.

```
fitZoom({ width: 420, height: 297 }, { width: 1200, height: 800 });
// min((1200 - 48) / 420, (800 - 48) / 297) = 2,53…
```

`UiState` interface

```
interface UiState { activePageId; zoom; zoomAnchor; viewport; selection; panels; leftTab;
dialog; messages; toasts; contextMenu; renamingPageId; home; clipboard; libraryModelId;
libraryKind; prefs; hiddenLinkTypes; showOccupation; update; portOccupancy; reveal;
showGrid }
```

Contenu du store de l'interface ; aucun de ces champs n'est enregistré dans le projet.

Champs

`activePageId` page affichée, `null` si le projet n'en a pas ;

`zoom` zoom en pixels écran par millimètre de papier ;

`zoomAnchor` point (pixels de la fenêtre) gardé fixe par le prochain changement de zoom ; `null` : le centre de la zone ;

`viewport` taille de la zone de dessin en pixels, tenue par la zone de dessin ;

`selection` éléments sélectionnés, sans doublon ;

`panels` visibilité des panneaux de gauche et de droite ;

`leftTab` onglet du panneau de gauche (`LeftTab`) : `library`, `explorer`, `checks` ou `schema` ;

`dialog` dialogue ouvert ou `null` ;

`messages` file des messages, le plus ancien affiché ;

`toasts` notifications affichées ;

`contextMenu` menu contextuel ouvert ou `null` ;

`renamingPageId` page dont l'onglet est en cours de renommage ;

`home` écran d'accueil visible (PRJ-01, PRJ-02) ;

`clipboard` presse-papiers interne (copier, coller) ;

`libraryModelId` machine choisie dans la bibliothèque ;

`libraryKind` équipement choisi dans la bibliothèque (ou celui de la machine) ;

`prefs` préférences de l'utilisateur, gardées dans le navigateur (EDM-04) ;

`hiddenLinkTypes` types de liaisons masqués sur le dessin (LIA-10) ;

`showOccupation` U libres affichées sur les armoires (ARM-05) ;

`update` nouvelle version prête, avec sa fonction de rechargement (WEB-08) ;

`portOccupancy` ports libres et occupés mis en évidence (PRT-03) ;

`reveal` rectangle (mm) d'une page à amener dans la vue quand elle est affichée (résultat de recherche, ERG-05), effacé une fois appliqué ;

`showGrid` grille affichée et aimantation sur elle (PAG-08).

`Preferences` interface

```
interface Preferences { imageLimitMb }
```

Préférences de l'utilisateur, gardées dans le stockage du navigateur par `main.tsx` et modifiées par `PreferencesDialog`.

Champs

`imageLimitMb` seuil d'alerte du poids d'une image importée, en mégaoctets (EDM-04).

`DEFAULT_PREFERENCES` constante

```
const DEFAULT_PREFERENCES: Preferences
```

Préférences par défaut : seuil d'alerte des images de 1 Mo. Valeur initiale de `prefs`, remplacée au démarrage par la valeur gardée dans le navigateur.

`Clipboard` interface

```
interface Clipboard { devices; externalViews; annotations? }
```

Contenu du presse-papiers interne (copier, coller, dupliquer).

Champs

`devices` équipements copiés ;

`externalViews` vues des équipements extérieurs copiés, par identifiant d'équipement, pour les reposer sur la page ;

`annotations` éléments de schéma copiés (PAG-07), facultatif.

`Ui` interface

```
interface Ui { store; getState; setActivePage; select; clearSelection; isSelected; setZoom;
zoomBy; openDialog; closeDialog; confirm; alert; notify; dismissToast; answer;
openContextMenu; closeContextMenu }
```

API de l'état de l'interface, créée par `createUi`, exposée aux composants par `UiContext` et aux commandes par `ctx.ui`.

Champs

`store` , `getState` le store Zustand et sa lecture ;

`setActivePage` change de page en retirant de la sélection vues et éléments de schéma ;

`select` remplace la sélection (dédoublonnée) ; avec `toggle`, ajoute ou retire chaque élément (Maj + clic) ;

`clearSelection` , `isSelected` vider, tester ;

`setZoom` , `zoomBy` zoom absolu ou relatif, borné, avec un point fixe facultatif ;

`openDialog` , `closeDialog` ouvrir un dialogue (ferme le menu contextuel), le fermer ;

`confirm` pose une question ; promesse résolue à `true` si confirmée (ERG-07) ;

`alert` message à un seul bouton ; promesse résolue à la fermeture ;

`notify` , `dismissToast` notification courte, la retirer ;

`answer` répond au message d'identifiant donné et résout sa promesse ;

`openContextMenu` , `closeContextMenu` menu contextuel.

`createUi` fonction

```
function createUi(editor: Editor): Ui
```

Crée l'état de l'interface d'un éditeur : première page active, zoom 1, panneaux visibles, onglet bibliothèque, aucun mode d'affichage. S'abonne à l'éditeur pour garder une page active valide (page de même rang si la page active disparaît, première page d'un autre projet), filtrer la sélection par `selectionExists` et fermer l'écran d'accueil quand un autre projet est chargé. Appelée une fois par `main.tsx`, et par les tests.

Paramètres

`editor` l'éditeur du projet.

Résultat. L'API `Ui`.

Erreurs. Aucune.

```
const editor = createEditor(createStarterProject());
const ui = createUi(editor);
ui.select([{ kind: 'rack', id: rackId }]);
const ok = await ui.confirm({ message: 'Supprimer ?', danger: true });
```

29.15 `ui/hooks`

Contextes React et crochets d'accès à l'état de l'interface, à la session et aux commandes. Les contextes sont fournis par `main.tsx` (et par les tests de composants).

`UiContext` constante

```
const UiContext = createContext<Ui | null>(null)
```

Contexte React qui porte l'objet `Ui` ; valeur `null` par défaut. Fourni par `main.tsx`, lu par `useUiApi`.

`SessionContext` constante

```
const SessionContext = createContext<Session | null>(null)
```

Contexte React qui porte la session (`app/session`) ; `null` par défaut (tests, banc de rendu). Lu par `useSession`.

`useUiApi` fonction

```
function useUiApi(): Ui
```

Renvoie l'API de l'état de l'interface du contexte, pour agir (sélectionner, ouvrir un dialogue, demander une confirmation). Ne provoque pas de nouveau rendu.

Résultat. L'objet `Ui`.

Erreurs. Lève une `Error` « UiContext is missing » hors d'un `UiContext.Provider`.

`useUi` fonction

```
function useUi<T>(selector: (state: UiState) => T): T
```

S'abonne à une tranche de l'état de l'interface : le composant est rendu à nouveau quand la valeur choisie change (comparaison stricte).

Paramètres

`selector` fonction de l'état vers la valeur voulue ; elle doit renvoyer une valeur stable (un champ, une valeur primitive).

Résultat. La valeur choisie.

Erreurs. Celles de `useUiApi`.

```
const zoom = useUi((s) => s.zoom);
const selected = useUi((s) => s.selection.some((x) => x.id === id));
```

`useSession` fonction

```
function useSession(): Session | undefined
```

Renvoie la session du contexte, ou `undefined` hors de l'application (tests). Utilisé par les dialogues qui ouvrent ou créent un projet.

Résultat. La session ou `undefined`.

Erreurs. Aucune.

`useCommandContext` fonction

```
function useCommandContext(): CommandContext
```

Construit le contexte des commandes (`editor`, `ui`, `session`), mémorisé tant que ces trois objets ne changent pas. Utilisé par les boutons, menus et raccourcis qui exécutent des commandes.

Résultat. Un `CommandContext`.

Erreurs. Celles de `useUiApi`.

`useStateVersion` fonction

```
function useStateVersion(): number
```

Numéro qui change chaque fois que l'état de l'éditeur ou celui de l'interface change (`useSyncExternalStore` sur les deux stores, numéro mémorisé par couple d'états). Un composant qui l'appelle est rendu à nouveau à chaque changement : sert à garder à jour l'état activé des commandes, qui dépend des deux.

Résultat. Un entier.

Erreurs. Celles de `useUiApi`.

`useCommand` fonction

```
function useCommand(id: string): { command: Command | undefined; enabled: boolean }
```

Une commande du registre et son état activé, tenu à jour par `useStateVersion`. Utilisé par `CommandButton` et l'onglet « Schéma ».

Paramètres

`id` identifiant de la commande.

Résultat. `{ command, enabled }` ; `command` vaut `undefined` (et `enabled` `false`) pour un identifiant inconnu.

Erreurs. Celles de `useUiApi`.

Flux d'interaction

29.16 `ui/placementFlow`

Placement et déplacement d'équipements depuis l'interface (EQP-02) : un dépassement de profondeur est expliqué et peut être forcé, les autres refus sont expliqués. Sans React ; utilisé par la zone de dessin, le dialogue de placement et les propriétés d'un équipement.

`placeWithConfirmation` fonction

```
async function placeWithConfirmation(ui: Ui | undefined, attempt: (forced: boolean) =>
ActionResult): Promise<ActionResult>
```

Exécute une tentative de placement sans forçage. Si elle est refusée pour la profondeur seulement (`forceable`), demande « Forcer » (confirmation destructive affichant les messages du refus et leur calcul) puis rejoue la tentative forcée ; sinon affiche le refus par `ui.alert`.

Paramètres

`ui` état de l'interface ; absent, le résultat est renvoyé sans dialogue ;

`attempt` fonction qui place (ou déplace) l'équipement, forcé ou non.

Résultat. Promesse du résultat final de l'action (`ok`, `id`, `messages`).

Erreurs. Aucune levée : un refus est renvoyé dans le résultat.

```
const result = await placeWithConfirmation(ui, (forced) =>
  placeNewDevice(editor, machineId, placement, { forced }),
);
if (result.ok && result.id) ui.select([{ kind: 'device', id: result.id }]);
```

29.17 `ui/linkFlow`

Création de liaisons depuis l'interface (LIA-01, LIA-06). Sans React ; utilisé par le geste de la zone de dessin et par `LinkDialog`.

`createLinkWithConfirmation` fonction

```
async function createLinkWithConfirmation(ui: Ui | undefined, editor: Editor, a:
PortRef, b: PortRef, options: AddLinkOptions = {}): Promise<ActionResult>
```

Crée une liaison par `addLink`. Si l'action est refusée par un avertissement de compatibilité forçable, demande s'il faut la créer quand même et la recrée avec `forced: true` ; un autre refus (port déjà relié…) est expliqué par `ui.alert`.

Paramètres

`ui` état de l'interface ; absent, le résultat est renvoyé sans dialogue ;

`editor` l'éditeur ;

`a` , `b` les deux extrémités (équipement et port) ;

`options` options de `addLink` : type, repère, attributs.

Résultat. Promesse du résultat de l'action ; `id` est celui de la liaison créée.

Erreurs. Aucune levée : un refus est renvoyé dans le résultat.

```
const result = await createLinkWithConfirmation(ui, editor,
  { deviceId: switchId, portId: p1 }, { deviceId: panelId, portId: p2 });
```

29.18 `ui/annotationFlow`

Pose des éléments de l'onglet « Schéma » sur une page (PAG-07) : depuis l'onglet (glisser ou clic), avec l'import d'une image (mêmes règles que les photos des machines : jamais déformée, alerte de poids). Sans React ; utilise `model/annotations`, `store/annotationActions` et `ui/imageImport`.

`ANNOTATION_DRAG_TYPE` constante

```
const ANNOTATION_DRAG_TYPE = 'application/x-portolan-annotation'
```

Type de données du glisser d'un élément depuis l'onglet « Schéma » (`application/x-portolan-annotation`) ; la donnée est le nom du préréglage. Posé par `SchemaPanel`, reconnu par la zone de dessin.

`presetTexts` fonction

```
function presetTexts(): PresetTexts
```

Textes par défaut des nouveaux éléments (texte, zone, note, tableau de deux colonnes), dans la langue de l'interface. Passés à `createAnnotation` par `insertAnnotation` et par les commandes.

Résultat. Un objet `PresetTexts`.

Erreurs. Aucune.

`insertAnnotation` fonction

```
async function insertAnnotation(editor: Editor, ui: Ui | undefined, pageId: Id, preset:
AnnotationPreset, center?: Point, { askImage = true }: { askImage?: boolean } = {}):
Promise<Id | null>
```

Ajoute un élément d'un préréglage sur une page, centré sur un point, et le sélectionne. Pour une image, demande d'abord son fichier (et abandonne si l'utilisateur annule), sauf si `askImage` est faux : l'élément déposé reste un cadre à remplir depuis les propriétés. L'image importée donne ses proportions au cadre (rapport borné entre 0,05 et 40) ; une image plus lourde que le seuil des préférences donne une notification.

Paramètres

- `editor` l'éditeur ;
- `ui` état de l'interface, facultatif (sélection, messages) ;
- `pageId` page cible ;
- `preset` préréglage (texte, flèche, rectangle, image, zone…) ;
- `center` centre en millimètres ; par défaut le centre de la zone de dessin de la page ;
- `options` `askImage` (vrai par défaut) : demander le fichier d'une image.

Résultat. Promesse de l'identifiant de l'élément créé, ou `null` (page inconnue, fichier non choisi, action refusée).

Erreurs. Aucune levée : une image illisible est signalée par `ui.alert` et donne `null`.

```
await insertAnnotation(editor, ui, pageId, 'text', { x: 120, y: 80 });
```

`replaceAnnotationImage` fonction

```
async function replaceAnnotationImage(editor: Editor, ui: Ui | undefined, pageId:
Id, annotation: Annotation): Promise<void>
```

Remplace l'image d'un élément image (bouton des propriétés) : demande un fichier, puis le cadre prend les proportions de la nouvelle image, même largeur et même centre vertical (valeurs arrondies au centième de millimètre). Sans effet pour un élément qui n'est pas une image.

Paramètres

- `editor` l'éditeur ;
- `ui` état de l'interface, facultatif ;
- `pageId` page de l'élément ;
- `annotation` l'élément image.

Résultat. Promesse résolue une fois l'élément modifié (ou rien si annulé).

Erreurs. Aucune levée : une image illisible est signalée par `ui.alert`.

29.19 `ui/imageImport`

Import des fichiers image choisis par l'utilisateur (EDM-02, PRJ-06) : logo du projet, photos des machines, images des éléments de schéma. S'appuie sur `prepareImage` (`io/images`), qui nettoie les SVG.

`IMAGE_ACCEPT` constante

```
const IMAGE_ACCEPT =
'.png,.jpg,.jpeg,.svg,.webp,image/png,image/jpeg,image/svg+xml,image/webp'
```

Valeur de l'attribut `accept` des sélecteurs d'images : PNG, JPEG, SVG et WebP, par extension et par type MIME.

`chooseFile` fonction

```
function chooseFile(accept: string): Promise<File | null>
```

Ouvre le sélecteur de fichiers du navigateur par un champ `file` caché, ajouté puis retiré du document. Doit être appelée pendant une action de l'utilisateur (clic).

Paramètres

- `accept` types acceptés (par exemple `IMAGE_ACCEPT`).

Résultat. Promesse du fichier choisi, ou `null` si l'utilisateur annule.

Erreurs. Aucune.

`ImportedImage` interface

```
interface ImportedImage { ref; size; pixels }
```

Image enregistrée dans le projet par `importImageFile`.

Champs

- `ref` référence de l'image (empreinte et extension) dans les images de l'éditeur ;
- `size` taille en octets telle que stockée (EDM-04) ;
- `pixels` taille naturelle en pixels, ou `null` si le navigateur ne peut pas décoder l'image.

`imagePixels` fonction

```
async function imagePixels(bytes: Uint8Array, mime: string): Promise<{ width: number;
height: number } | null>
```

Taille naturelle d'une image, décodée par le navigateur via une URL d'objet temporaire.

Paramètres

- `bytes` contenu du fichier ;
- `mime` type MIME.

Résultat. Promesse de `{ width, height }` en pixels, ou `null` si l'image ne se décode pas ou n'a pas de taille.

Erreurs. Aucune : une erreur de décodage donne `null`.

`importImageFile` fonction

```
async function importImageFile(editor: Editor, file: File): Promise<ImportedImage>
```

Enregistre un fichier image dans l'éditeur (`editor.addImages`, hors historique) : SVG nettoyé, autres formats gardés tels quels, puis mesure de ses pixels.

Paramètres

`editor` l'éditeur ;

`file` le fichier choisi, déposé ou collé.

Résultat. Promesse d'une `ImportedImage`.

Erreurs. Lève une `Error` au message en français (« format non pris en charge ») si l'extension n'est pas reconnue, et les erreurs de `prepareImage` (SVG illisible).

Zone de dessin

29.20 `ui/canvas/Canvas`

Zone de dessin (ERG-01, PAG-12) : la page active au zoom courant, superposée à la couche d'interface, avec tous les gestes (sélection, glisser, placement, liaisons, retouche des tracés, redimensionnement), les infobulles et le dépôt depuis la bibliothèque ou l'onglet « Schéma ». Les calculs sont dans le modèle (`model/dropTarget`, `model/pageGeometry`, `model/arrange`, `routing/manual`) ; ce module traduit les événements du pointeur en actions.

`portAt` fonction

```
function portAt(target: EventTarget | null): PortRef | null
```

Port sous le pointeur : l'élément `data-port-id` le plus proche et son équipement `data-device-id`.

Paramètres

`target` cible d'un événement (ou élément trouvé sous le pointeur).

Résultat. Référence `{ deviceId, portId }` ou `null`.

Erreurs. Aucune.

`itemAt` fonction

```
function itemAt(target: EventTarget | null): SelectionItem | null
```

Élément de sélection sous le pointeur, dans cet ordre : port, liaison, élément de schéma, équipement (seulement dans une vue d'armoire), sinon vue. Utilisée pour le clic, le double-clic et le clic droit.

Paramètres

`target` cible d'un événement.

Résultat. Un `SelectionItem` ou `null` (vide de la page).

Erreurs. Aucune.

`Canvas` fonction

```
function Canvas()
```

Monté dans la zone centrale de `App`. Rend `PageCanvas` et `Overlay` dans une feuille à la taille de la page fois le zoom, publie la taille de la zone (`viewport`), ajuste la page à l'ouverture d'un projet, garde fixe le point sous le pointeur pendant un zoom (Ctrl + molette) et applique les demandes `reveal`. Gestes : défilement au bouton du milieu, retouche d'un tracé, redimensionnement d'un élément de schéma ou d'une vue extérieure, rectangle de sélection, tracé d'une liaison, déplacement d'un équipement, déplacement des vues et éléments sélectionnés avec aimantation (chapitre 13). Double-clic : point ajouté sur un câble, ou propriétés de l'équipement (partie « Machine ») ou du texte d'un élément. Clic droit : supprime un point du tracé sélectionné ou ouvre le menu contextuel. Sans propriétés.

29.21 `ui/canvas/coords`

Conversions entre pixels écran et millimètres de papier, et tests sur les cibles des événements. Sans React ; utilisé par la zone de dessin et les raccourcis.

`Point` interface

```
interface Point { x; y }
```

Point en deux dimensions (pixels de la fenêtre ou millimètres de page selon le contexte).

Champs

`x` , `y` coordonnées.

`clientToPage` fonction

```
function clientToPage(sheet: Element, zoom: number, client: Point): Point
```

Millimètres de page sous un point de la fenêtre, d'après le rectangle affiché de la feuille et le zoom.

Paramètres

`sheet` élément de la feuille (`.sheet`) ;

`zoom` zoom en pixels par millimètre ;

`client` point en pixels de la fenêtre.

Résultat. Le point en millimètres.

Erreurs. Aucune.

```
const mm = clientToPage(sheet, ui.getState().zoom, { x: e.clientX, y: e.clientY });
```

`closestData` fonction

```
function closestData(target: EventTarget | null, attribute: string): Element | null
```

Premier ancêtre (ou l'élément lui-même) qui porte un attribut de données donné (`data-device-id`, `data-view-id`…).

Paramètres

`target` cible d'un événement ;

`attribute` nom de l'attribut.

Résultat. L'élément ou `null` (cible absente ou qui n'est pas un élément).

Erreurs. Aucune.

`isTextField` fonction

```
function isTextField(target: EventTarget | null): boolean
```

Indique si une cible est un champ de saisie de texte : `input` autre que case à cocher, radio, bouton, couleur ou fichier ; `textarea`, `select` ou contenu éditable. Dans un tel champ, seuls les raccourcis `allowInTextField` agissent.

Paramètres

`target` cible de l'événement clavier.

Résultat. `true` pour un champ de texte.

Erreurs. Aucune.

`isToggleInput` fonction

```
function isToggleInput(target: EventTarget | null): boolean
```

Indique si une cible est une case à cocher ou un bouton radio : Espace y garde son rôle natif et le raccourci Espace ne s'applique pas (ERG-10, décision du lot 39).

Paramètres

`target` cible de l'événement clavier.

Résultat. `true` pour une case à cocher ou un bouton radio.

Erreurs. Aucune.

29.22 `ui/canvas/Overlay`

Couche SVG dessinée au-dessus de la page, jamais exportée (`aria-hidden`), dans les mêmes coordonnées en millimètres : grille (PAG-08), occupation des armoires (ARM-05), contours de sélection, poignées, aperçu de dépôt, rectangle de sélection, guides d'alignement. Les épaisseurs y sont en pixels écran (`1 / zoom`).

`SELECTION_COLOR` constante

```
const SELECTION_COLOR = '#1f6feb'
```

Couleur des contours de sélection, des poignées et du rectangle de sélection (`#1f6feb`) ; reprise par `RouteHandles`.

`selectionBoxes` fonction

```
function selectionBoxes(project: Project, page: Page, selection: SelectionItem[]): Box[]
```

Rectangles à entourer pour la sélection sur une page : la vue ou l'élément de schéma, toutes les vues d'une armoire, l'équipement et le port dans chaque vue qui les dessine, les deux ports d'une liaison.

Paramètres

`project` le projet ;

`page` la page affichée ;

`selection` la sélection.

Résultat. Liste de rectangles en millimètres (vide pour ce qui n'est pas sur la page).

Erreurs. Aucune.

`BOX_HANDLES` constante

```
const BOX_HANDLES = ['nw', 'n', 'ne', 'e', 'se', 's', 'sw', 'w'] as const
```

Les huit poignées d'un rectangle, nommées par les bords qu'elles déplacent : `nw`, `n`, `ne`, `e`, `se`, `s`, `sw`, `w`. Rendues autour d'un élément de schéma sélectionné (`data-annotation-handle`).

`BoxHandle` type

```
type BoxHandle = (typeof BOX_HANDLES)[number];
```

Une poignée de `BOX_HANDLES` ; la zone de dessin teste les lettres `n`, `s`, `e`, `w` qu'elle contient pour savoir quels bords bouger.

`GuidesLayer` fonction

```
function GuidesLayer({ guides, zoom }: { guides: readonly Guide[]; zoom: number })
```

Guides d'alignement du glisser en cours (PAG-08) : un trait par guide, sur toute l'étendue indiquée. Enfant de `Overlay` dans la zone de dessin.

Paramètres

`guides` guides calculés par l'aimantation (`model/arrange`) ;

`zoom` zoom, pour l'épaisseur d'un pixel.

`FREE_COLOR` constante

```
const FREE_COLOR = '#2f9e44'
```

Couleur des U libres et du compte de U libres sous chaque vue d'armoire quand l'occupation est affichée (ARM-05), `#2f9e44`.

`DropPreview` interface

```
interface DropPreview { box; status }
```

Aperçu d'un équipement en cours de dépôt ou de déplacement.

Champs

`box` rectangle en millimètres de page ;

`status` `ok` (vert, permis), `forceable` (orange, dépassement de profondeur) ou `refused` (rouge).

`OverlayProps` interface

```
interface OverlayProps { page; zoom; band; preview?; children? }
```

Propriétés de `Overlay`.

Champs

- `page` page affichée ;
- `zoom` zoom en pixels par millimètre ;
- `band` rectangle de sélection en cours, ou `null` ;
- `preview` aperçu de dépôt, facultatif ;
- `children` dessins supplémentaires en millimètres (états de ports, chemin, poignées du tracé, guides, aperçu de liaison).

`Overlay` constante

```
const Overlay = …
```

Composant mémorisé (`memo`) monté par `Canvas` par-dessus la page. Dessine, dans l'ordre : grille si `showGrid`, occupation si `showOccupation`, contours de `selectionBoxes`, poignée de redimensionnement d'une vue extérieure seule sélectionnée (`data-resize-view`, EXT-02), poignées de l'élément de schéma seul sélectionné (huit poignées ou les deux extrémités d'une ligne, `data-line-point`), aperçu de dépôt, enfants, rectangle de sélection. Propriétés : `OverlayProps`.

29.23 `ui/canvas/Highlights`

Mises en évidence à l'écran seulement : chemin de la liaison sélectionnée (LIA-11) et états des ports (PRT-03, décision du lot 40). S'appuie sur `pathElements` (`model/links`), `portStatesOnPage` (`model/portStates`) et le routage en cache de la page.

`PATH_COLOR` constante

```
const PATH_COLOR = '#f59f00'
```

Couleur du chemin de la liaison sélectionnée (`#f59f00`, orange), pour le halo des câbles et le cadre des ports traversés.

`PORT_STATE_COLORS` constante

```
const PORT_STATE_COLORS = { free: '#2f9e44', occupied: '#e03131', disabled: '#868e96' }
```

Couleurs des états de ports (PRT-03) : `free` vert, `occupied` rouge, `disabled` gris. Utilisées par le calque et par la légende.

`PathHighlight` fonction

```
function PathHighlight({ page, zoom }: { page: Page; zoom: number })
```

Quand une seule liaison est sélectionnée, dessine son chemin de bout en bout à travers les panneaux de brassage : halo orange sur le tracé de chaque liaison du chemin présente sur la page, cadre sur chaque port traversé dans chaque vue qui le dessine. Les liaisons d'un type masqué ne sont pas tracées. Enfant de l'`Overlay`.

Paramètres

`page` page affichée ;

`zoom` zoom, pour les épaisseurs.

`PortOccupancy` fonction

```
function PortOccupancy({ page, zoom }: { page: Page; zoom: number })
```

Quand `portOccupancy` est actif, recouvre chaque port dessiné sur la page d'un cadre coloré selon son état (libre, occupé, désactivé), avec `data-port-state`. Enfant de l'`Overlay`.

Paramètres

`page` page affichée ;

`zoom` zoom, pour les épaisseurs.

`PortOccupancyLegend` fonction

```
function PortOccupancyLegend()
```

Légende HTML des états de ports, dans un coin de la zone de dessin, affichée tant que `portOccupancy` est actif. Montée dans la zone centrale de `App`. Sans propriétés.

29.24 `ui/canvas/RouteHandles`

Poignées de retouche du tracé de la liaison sélectionnée (ROU-06, décision du lot 24). Le tracé vient du routage en cache de la page (`routing/page`).

`selectedRoute` fonction

```
function selectedRoute(project: Project, page: Page, linkId: string | undefined, hidden:
ReadonlySet<string>): Point[] | null
```

Polyligne du câble d'une liaison telle qu'elle est dessinée sur la page (tracé automatique ou retouché), base des gestes de retouche.

Paramètres

`project` le projet ;

`page` la page ;

`linkId` liaison sélectionnée, ou `undefined` ;

`hidden` types de liaisons masqués.

Résultat. Les points en millimètres, ou `null` (pas de liaison, ou câble absent de la page).

Erreurs. Aucune.

`RouteHandles` fonction

```
function RouteHandles({ page, zoom }: { page: Page; zoom: number })
```

Quand une seule liaison est sélectionnée : le tracé en pointillés, un rond au milieu de chaque segment (`data-route-segment`, glisser pour le déplacer) et un carré sur chaque point intermédiaire (`data-route-vertex`, glisser pour le déplacer, clic droit pour le supprimer). Pas de poignée près des ports ni sur un segment trop court, pour que le port reste atteignable. Enfant de l'`Overlay`.

Paramètres

`page` page affichée ;

`zoom` zoom, pour la taille des poignées (7 pixels).

29.25 `ui/canvas/PortTooltip`

Infobulle d'un port survolé (PRT-02), affichée par la zone de dessin.

`TooltipState` interface

```
interface TooltipState { port; x; y }
```

Port survolé et position de l'infobulle.

Champs

`port` référence du port ;

`x` , `y` position du pointeur, en pixels de la fenêtre.

`PortTooltip` fonction

```
function PortTooltip({ state }: { state: TooltipState })
```

Infobulle (`role="tooltip"`) : équipement et port, type de connecteur, puis « libre » ou chaque liaison (repère, type, extrémité opposée). Placée à 14 pixels du pointeur ; rien si le port n'existe plus.

Paramètres

`state` port et position.

29.26 `ui/canvas/DepthTooltip`

Infobulle de profondeur d'une U survolée (EQP-05), calculée par `uDepthAt` (`model/depthInfo`).

`DepthTooltipState` interface

```
interface DepthTooltipState { info; x; y }
```

Informations de profondeur de la U survolée et position de l'infobulle.

Champs

`info` armoire, U, équipements avant et arrière, profondeurs occupées et libre ;

`x` , `y` position du pointeur, en pixels de la fenêtre.

`DepthTooltip` fonction

```
function DepthTooltip({ state }: { state: DepthTooltipState })
```

Infobulle (`role="tooltip"`) : armoire et U, équipements fixés à l'avant et à l'arrière avec la profondeur qu'ils utilisent (profondeur et dégagement, en mm), profondeur restante sur la profondeur utile.

Paramètres

`state` informations et position.

Panneau de gauche

29.27 `ui/panels/LeftPanel`

Panneau de gauche à onglets (ERG-01) : bibliothèque, explorateur, vérifications et « Schéma » (PAG-07).

`LeftPanel` fonction

```
function LeftPanel()
```

Monté par `App` quand le champ `panels.left` est vrai, dans un élément `aside`. Quatre onglets de rôle `tab` (identifiants de test `tab-library`, `tab-checks`…) qui changent `leftTab`. Depuis la décision du 06/11/2026 (ERG-11), chaque onglet est une *icône* SVG dessinée depuis `TAB_ICON_PATHS`, dans l'ordre de `LEFT_TABS` ; son nom est donné au survol (`title`) et aux technologies d'assistance (`aria-label`), et les quatre onglets tiennent toujours sur une ligne. L'onglet des vérifications porte une pastille du nombre d'anomalies, jamais coupée et masquée aux lecteurs d'écran, le nombre étant annoncé comme description de l'onglet (`aria-describedby`). Sans propriétés.

29.28 `ui/panels/tabIcons`

Icônes des onglets du panneau de gauche (ERG-11, décision du 06/11/2026) : tracés SVG sur une grille de 20 × 20, dessinés avec la couleur du texte (`currentColor`), sans chargement externe. Données pures, partagées par l'interface (`ui/panels/LeftPanel`) et le manuel utilisateur : `tools/manual` redessine les mêmes tracés en TikZ, ce qui évite toute divergence entre l'application et ses captures.

`LeftTab` type

```
type LeftTab = 'library' | 'explorer' | 'checks' | 'schema';
```

Identifiant d'un onglet du panneau de gauche : `library` (bibliothèque), `explorer` (explorateur), `checks` (vérifications) ou `schema` (onglet « Schéma »). C'est aussi le type du champ `leftTab` de l'état de l'interface (`ui/state`).

`LEFT_TABS` constante

```
const LEFT_TABS: readonly LeftTab[]
```

Ordre des onglets dans le panneau, de gauche à droite : bibliothèque, explorateur, vérifications, schéma. Parcouru par le panneau et par le générateur des icônes du manuel.

`TAB_ICON_SIZE` constante

```
const TAB_ICON_SIZE = 20
```

Côté de la grille des icônes, en pixels (20) : la `viewBox` de chaque icône est `0 0 20 20`.

`TAB_ICON_STROKE` constante

```
const TAB_ICON_STROKE = 1.5
```

Épaisseur du trait des icônes, en unités de la grille (1,5), extrémités et jonctions arrondies.

`TAB_ICON_PATHS` constante

```
const TAB_ICON_PATHS: Record<LeftTab, readonly string[]>
```

Tracés de chaque icône, en syntaxe de chemin SVG limitée à `M`, `L`, `H`, `V`, `A` et `Z` (absolus ou relatifs), seule syntaxe que la bibliothèque TikZ `svg.path` du manuel lit aussi : trois livres pour la bibliothèque, un arbre pour l'explorateur, un presse-papiers coché pour les vérifications, un cadre, un cercle et une flèche pour « Schéma ».

29.29 `ui/panels/LibraryPanel`

Bibliothèque du projet (BIB-01, BIB-02) : arbre catégorie, équipement, machines ; recherche ; aperçu ; glisser vers le dessin. Utilise `libraryTree` (`model/libraryTree`) et le dessin des faces (`DeviceFace`). Ré-exporte aussi `formatBytes` de `i18n/index` pour les dialogues de bibliothèque.

`MODEL_DRAG_TYPE` constante

```
const MODEL_DRAG_TYPE = 'application/x-portolan-model'
```

Type de données d'une machine glissée de la bibliothèque vers le dessin (`application/x-portolan-model`) ; la donnée est l'identifiant de la machine.

`libraryDrag` constante

```
const libraryDrag: { modelId: string | null }
```

Machine en cours de glisser (`modelId`, `null` sinon). Les données d'un glisser ne sont lisibles qu'au dépôt : la zone de dessin lit cet objet pendant le survol pour calculer l'aperçu. Rempli au début du glisser, vidé à sa fin.

`ModelPreview` fonction

```
function ModelPreview({ model, face = 'front', }: { model: Machine; face?: 'front'
| 'rear'; })
```

Aperçu d'une face d'une machine dessinée comme sur le schéma (BIB-01) : largeur 19 pouces ou demi-largeur, hauteur en U ; bande 0U couchée ; équipement extérieur à sa taille d'affichage. Utilisé par la bibliothèque, la feuille de bibliothèque et l'import de modèles.

Paramètres

`model` la machine ;

`face` `front` (défaut) ou `rear`.

`LibraryPanel` fonction

```
function LibraryPanel()
```

Onglet « Bibliothèque ». Boutons de commandes (nouvelle machine, purger, réimporter, importer), recherche plein texte, détails de la machine ou de l'équipement choisi, puis l'arbre (`role="tree"`). Une flèche déplie les machines d'un équipement (machine générique en premier) ; le choix d'une machine (par exemple depuis les vérifications) déplie son équipement. Une machine glissée est posée telle quelle, un équipement glissé pose sa machine générique ; double-clic : éditeur de machine ; clic droit : menu des machines. Chaque machine montre son nombre d'utilisations. Sans propriétés.

29.30 `ui/panels/ExplorerPanel`

Explorateur du projet (ARM-06) : armoires, équipements, ports, puis équipements extérieurs, synchronisé avec la sélection.

`rackDevices` fonction

```
function rackDevices(project: Project, rackId: Id): Device[]
```

Équipements d'une armoire dans l'ordre de l'explorateur : du haut vers le bas (U la plus haute d'abord), puis les équipements 0U ; à égalité, par nom.

Paramètres

`project` le projet ;

`rackId` l'armoire.

Résultat. Nouvelle liste d'équipements.

Erreurs. Aucune.

`ExplorerPanel` fonction

```
function ExplorerPanel()
```

Onglet « Explorateur » : deux arbres (`tree`) pour les armoires et les équipements extérieurs. Chaque ligne (`treeitem`, `data-explorer`) sélectionne son élément (Maj pour ajouter), défile pour rester visible quand il est sélectionné ailleurs, et ouvre au clic droit le menu de `contextMenuItems` avec le projet (pour proposer d'afficher un équipement extérieur sur la page). Sans propriétés.

29.31 `ui/panels/ChecksPanel`

Vérifications (LIA-07) : liste des anomalies du projet calculée par `listAnomalies` (`model/anomalies`).

`ChecksPanel` fonction

```
function ChecksPanel()
```

Onglet « Vérifications » : résumé (erreurs, avertissements) puis une ligne par anomalie (`role="button"`, activable par Entrée). Un clic sélectionne les éléments concernés sur le dessin ; une anomalie de machine affiche la machine dans la bibliothèque. Sans propriétés.

`useAnomalyCount` fonction

```
function useAnomalyCount(): number
```

Crochet : nombre d'anomalies du projet, recalculé quand le projet change. Utilisé par la pastille de l'onglet dans `LeftPanel`.

Résultat. Le nombre d'anomalies.

Erreurs. Aucune.

29.32 `ui/panels/SchemaPanel`

Onglet « Schéma » (PAG-07, décision du lot 48) : éléments à poser sur la page, en quatre groupes de `ANNOTATION_GROUPS` (`model/annotations`).

`SchemaPanel` fonction

```
function SchemaPanel()
```

Onglet « Schéma ». Chaque élément est un bouton de la commande `annotation.add.<élément>` : un clic exécute la commande (élément au centre de la zone de dessin), un glisser pose le type `ANNOTATION_DRAG_TYPE` (élément centré sur le point de dépôt). Un élément dont la commande est désactivée ne se glisse pas. Sans propriétés.

Panneau des propriétés

29.33 `ui/properties/PropertiesPanel`

Panneau de droite : propriétés de la sélection (ERG-01), ou de la page active quand rien n'est sélectionné.

`PropertiesPanel` fonction

```
function PropertiesPanel()
```

Monté par `App` quand `panels.right` est vrai, hors édition de bibliothèque (`aside`, `data-testid="properties"`). Choisit les sections : page active ; sélection multiple (nombre, alignement dès deux vues ou éléments de schéma, propriétés communes si tous sont des équipements ou des vues extérieures) ; élément de schéma ; liaison ; équipement ou port ; vue (avec l'équipement extérieur et ses ports ajoutés, ou l'armoire) ; armoire. Chaque section a pour `key` l'identifiant de l'entité. Sans propriétés.

29.34 `ui/properties/PageProperties`

Propriétés de la page active (PAG-02, PAG-05, PAG-09, PAG-11, PAG-14), modifiées par `updatePage`.

`PageProperties` fonction

```
function PageProperties({ page }: { page: Page })
```

Sections « Page » (titre, format, dimensions personnalisées, orientation), « Marges » et « Affichage » (échelle uniforme des armoires, position de la légende, repères des câbles, tableau des révisions et son nombre de lignes).

Paramètres

`page` la page active.

29.35 `ui/properties/ViewProperties`

Propriétés d'une vue posée sur la page (PAG-03 à PAG-06, PAG-15, PAG-16), modifiées par `updateView`.

`ViewProperties` fonction

```
function ViewProperties({ page, view }: { page: Page; view: View })
```

Pour une vue d'armoire : titre et son affichage, échelle, position, étiquettes à côté, câbles masqués, repères, faisceaux, plage de U affichée (« de … à … » dans la numérotation de l'armoire). Pour une vue extérieure : taille (proportions gardées sauf si décoché), ports masqués.

Paramètres

`page` page de la vue ;

`view` la vue.

29.36 `ui/properties/RackProperties`

Caractéristiques d'une armoire (ARM-02, § 6.3) et ses goulottes (ROU-02).

`RackProperties` fonction

```
function RackProperties({ rack }: { rack: Rack })
```

Nom, hauteur, largeur de montage, largeur, profondeur et profondeur utile, type, roulettes, portes, numérotation, emplacements 0U, couleur ; U libres (ARM-05) ; côté de la goulotte de chaque type de liaison (« comme le projet », gauche, droite). Une modification refusée (hauteur réduite sous un équipement) est expliquée par `ui.alert` et le champ reprend sa valeur.

Paramètres

`rack` l'armoire.

29.37 `ui/properties/DeviceProperties`

Propriétés d'un équipement placé (EQP-07, EQP-08, EQP-01, EQT-04) et propriétés communes de plusieurs équipements (ERG-02).

`DeviceProperties` fonction

```
function DeviceProperties({ device, ports = true, }: { device: Device; /** Show the ports
added to the device (PRT-04) at the end; false when they follow the view. */ ports?:
boolean; })
```

Section « Propriétés » (nom avec alerte de doublon, description, mode d'étiquette, champs libres), partie « Machine » (autre machine du même équipement, avec confirmation si des liaisons seraient perdues ; nouvelle machine à partir de celle-ci ; éditeur de machine), section « Placement » (U du bas, face de fixation, côté d'un demi-largeur, retournement ; face, côté et emplacement d'un 0U), puis les ports ajoutés. Les déplacements passent

par `placeWithConfirmation`. Le champ de la machine porte `data-testid="device-machine"` (focus au double-clic sur l'équipement).

Paramètres

`device` l'équipement ;

`ports` afficher les ports ajoutés à la fin (vrai par défaut) ; faux quand ils suivent les propriétés de la vue extérieure.

`DevicesProperties` fonction

```
function DevicesProperties({ devices }: { devices: Device[] })
```

Propriétés communes de plusieurs équipements sélectionnés : mode d'étiquette (« mixte » quand ils diffèrent), appliqué à tous en une seule action annulable (`editor.transaction`).

Paramètres

`devices` les équipements, au moins un.

29.38 `ui/properties/InstancePorts`

Ports ajoutés à un équipement placé seulement (PRT-04), par `store/portActions`.

`InstancePorts` fonction

```
function InstancePorts({ device }: { device: Device })
```

Section « Ports ajoutés » : nom, connecteur, face et position (centre, en % de la face) de chaque port ; ajout par la commande `device.addPort` ; suppression, avec confirmation destructive quand le port est relié (les liaisons sont supprimées). Un refus est notifié. La machine et ses autres équipements ne changent pas.

Paramètres

`device` l'équipement.

29.39 `ui/properties/LinkProperties`

Propriétés d'une liaison (LIA-04 à LIA-06, LIA-09) et attributs partagés avec le dialogue de création.

`LinkAttributeValues` type

```
type LinkAttributeValues = Partial< Pick<Link, 'length' | 'category' | 'supplierRef'
| 'sheathColor' | 'tray' | 'notes'> >;
```

Attributs facultatifs d'une liaison édités par `LinkAttributes` : `length` (valeur et unité `m` ou `cm`), `category`, `supplierRef`, `sheathColor`, `tray` et `notes`, tels que dans le schéma de la liaison.

`LinkAttributes` fonction

```
function LinkAttributes({ values, onChange, }: { values: LinkAttributeValues; onChange:
(patch: LinkAttributeValues) => void; })
```

Champs des attributs d'une liaison (LIA-05, § 6.9) : longueur et unité, catégorie, référence fournisseur, couleur de gaine, goulotte, notes. Utilisé dans les propriétés d'une liaison et dans `LinkDialog`.

Paramètres

`values` valeurs actuelles ;

`onChange` reçoit chaque modification (un attribut à la fois).

`LinkProperties` fonction

```
function LinkProperties({ link }: { link: Link })
```

Repère (alerte s'il est en double), type, affichage (auto, trait, repères), faisceau, extrémités, extrémités finales à travers les panneaux de brassage (LIA-09) et alerte de boucle, avertissements de compatibilité, puis les attributs.

Paramètres

`link` la liaison.

29.40 `ui/properties/AnnotationProperties`

Propriétés des éléments de l'onglet « Schéma » (PAG-07) et alignement (PAG-08), par `store/annotationActions`.

`AnnotationProperties` fonction

```
function AnnotationProperties({ page, annotation }: { page: Page; annotation: Annotation })
```

Section de l'élément : trait (épaisseur, pointillés), remplissage, police (taille, alignement, gras), couleurs, texte, numéro d'une bulle, titre d'une zone, image (« Choisir une image », `replaceAnnotationImage`), lignes et colonnes d'un tableau et ses cellules, pointes et coude d'une flèche. Section « Placement » : position, taille, rotation d'un quart de tour, plan (sous ou sur les vues). Le texte porte `data-testid="annotation-text"`.

Paramètres

`page` page de l'élément ;

`annotation` l'élément.

`ArrangeSection` fonction

```
function ArrangeSection()
```

Section « Disposer » : boutons des commandes d'alignement (bords, centres) et de répartition à écarts égaux de la sélection de vues et d'éléments. Affichée par `PropertiesPanel` dès deux éléments. Sans propriétés.

Champs de formulaire

29.41 `ui/fields`

Champs des panneaux et des dialogues. Un champ de texte ou de nombre valide sa valeur en quittant le champ ou sur Entrée (une action annulable par modification, pas par frappe) ; Échap rend la valeur. Chaque champ a un `label` relié par `useId`.

`useDraft` fonction

```
function useDraft(value: string): [string, (text: string) => void]
```

Valeur locale d'un champ, remise à la valeur validée quand celle-ci change (autre sélection, annulation…). La remise se fait pendant le rendu, comme React le recommande, sans effet.

Paramètres

`value` valeur validée (celle du projet).

Résultat. Couple `[texte, setTexte]`.

Erreurs. Aucune.

`TextField` fonction

```
function TextField({
label, value, onCommit, multiline, hint, placeholder, testId, autoFocus, }: { label:
string; value: string; onCommit: (value: string) => void; multiline?: boolean; hint?:
string; placeholder?: string; testId?: string; autoFocus?: boolean; })
```

Champ de texte d'une ligne ou de plusieurs (`multiline`). Validé en quittant le champ ou par Entrée (une ligne), seulement si la valeur a changé ; Échap rend la valeur. Après validation, il montre la valeur du projet (la nouvelle, ou l'ancienne si l'action a été refusée).

Paramètres

`label` libellé ;

`value` valeur actuelle ;

`onCommit` reçoit la nouvelle valeur ;

`multiline` , `hint`, `placeholder`, `testId`, `autoFocus` options d'affichage.

`NumberField` fonction

```
function NumberField({
label, value, onCommit, min, max, step, integer, unit, hint, testId, allowEmpty, }: {
label: string; value: number | null; onCommit: (value: number | null) => void; min?:
number; max?: number; step?: number; integer?: boolean; unit?: string; hint?: string;
testId?: string; /** Empty field = null (e.g. unknown depth). */ allowEmpty?: boolean; })
```

Champ numérique (texte avec clavier décimal). La virgule est acceptée ; la valeur est arrondie si `integer`, bornée par `min` et `max` ; une saisie invalide rend la valeur ; vide donne `null` si `allowEmpty`. Les flèches haut et bas ajoutent ou retirent un pas et valident aussitôt.

Paramètres

`label` libellé ;

`value` valeur actuelle, `null` pour vide ;

`onCommit` reçoit la nouvelle valeur ;

`min` , `max`, `step`, `integer` contraintes ;

`unit` unité affichée après le champ (mm, U…) ;

`hint` , `testId` aide, identifiant de test ;

`allowEmpty` un champ vide donne `null` (profondeur inconnue).

`SelectField` fonction

```
function SelectField<T extends string>({ label, value, options, onChange, hint, testId, }:
{ label: string; value: T; options: readonly { value: T; label: string }[]; onChange:
(value: T) => void; hint?: string; testId?: string; })
```

Liste déroulante typée ; chaque choix est transmis aussitôt.

Paramètres

`label` libellé ;

`value` valeur choisie ;

`options` valeurs et libellés ;

`onChange` reçoit la valeur choisie ;

`hint` , `testId` options.

`CheckboxField` fonction

```
function CheckboxField({ label, checked, onChange, testId, }: { label: string; checked:
boolean; onChange: (checked: boolean) => void; testId?: string; })
```

Case à cocher avec son libellé ; chaque changement est transmis aussitôt.

Paramètres

`label` libellé ;

`checked` état ;

`onChange` reçoit le nouvel état ;

`testId` identifiant de test.

`ColorField` fonction

```
function ColorField({ label, value, onChange, testId, }: { label: string; value: string;
onChange: (value: string) => void; testId?: string; })
```

Sélecteur de couleur ; chaque changement est transmis (l'appelant regroupe les changements d'un même geste par une clé de fusion).

Paramètres

`label` libellé ;

`value` couleur `#rrggbb` ;

`onChange` reçoit la couleur ;

`testId` identifiant de test.

`Section` fonction

```
function Section({ title, children }: { title: string; children: React.ReactNode })
```

Groupe de champs avec un titre (`section` et `h3`), utilisé par toutes les sections de propriétés.

Paramètres

- `title` titre ;
- `children` contenu.

Dialogues

29.42 `ui/dialogs/DialogHost`

Registre et hôte des dialogues ouverts par `ui.openDialog(type, props)`. Pour ajouter un dialogue, l'enregistrer dans `DIALOGS`.

`DialogProps` interface

```
interface DialogProps { onClose; props }
```

Propriétés reçues par tout dialogue du registre.

Champs

- `onClose` ferme le dialogue (`ui.closeDialog`) ;
- `props` paramètres passés à `openDialog` (objet vide par défaut) ; chaque dialogue lit ceux qu'il attend.

`DIALOGS` constante

```
const DIALOGS: Record<string, ComponentType<DialogProps>>
```

Dialogues par nom : aide (`shortcuts`, `about`, `releaseNotes`), projet (`projectProperties`, `revisions`, `titleBlock`, `newFromExisting`), armoires (`addRack`, `renameRack`), placement (`placeDevice`), bibliothèque (`purge`, `reimport`, `connectorTypes`, `linkTypes`, `machineEditor`, `importModels`, `exportLibrary`), `preferences`, liaisons (`link`, `linksTable`, `linkVisibility`, `routing`), exports (`exportImage`, `exportPdf`, `exportSpreadsheet`, `exportLabels`), `importSpreadsheet`, `search`.

`DialogHost` fonction

```
function DialogHost()
```

Monté dans `App`. Rend le composant de `DIALOGS` correspondant au dialogue ouvert, avec `onClose` et `props` ; rien si aucun dialogue n'est ouvert ou si le nom est inconnu. Sans propriétés.

29.43 `ui/dialogs/ProjectDialogs`

Dialogues du projet : propriétés, révisions, cartouche, nouveau projet à partir d'un existant, écran d'accueil. Par `store/projectActions` et la session.

`ProjectPropertiesDialog` fonction

```
function ProjectPropertiesDialog({ onClose }: DialogProps)
```

Propriétés du projet (PRJ-06) : nom, client, auteur, notes, date de création, logo du cartouche (choisi par `chooseFile`, importé par `importImageFile` ; une erreur est affichée par `ui.alert`).

Paramètres

`onClose` fermeture.

`RevisionsDialog` fonction

```
function RevisionsDialog({ onClose, props }: DialogProps)
```

Révisions du projet (PRJ-11) : tableau (indice, date, auteur, description), ajout avec l'indice suivant, édition de la révision choisie, suppression après confirmation.

Paramètres

`onClose` fermeture ;

`props` `editId` : révision à éditer à l'ouverture (commande « Ajouter une révision »).

`TitleBlockDialog` fonction

```
function TitleBlockDialog({ onClose }: DialogProps)
```

Cartouche du projet (PAG-10) : champs affichés, ordre, place (ligne du bloc d'informations ou colonne pleine hauteur), libellés, valeurs des champs personnalisés, ajout d'un champ manquant ou personnalisé ; aperçu du cartouche sur la page active.

Paramètres

`onClose` fermeture.

`NewFromExistingDialog` fonction

```
function NewFromExistingDialog({ onClose }: DialogProps)
```

« Nouveau projet à partir d'un projet existant » (PRJ-10) : une case « tout reprendre », puis `session.newFromExisting(keepContent)` qui demande le fichier.

Paramètres

`onClose` fermeture.

`HomeScreen` fonction

```
function HomeScreen({ session }: { session: Session })
```

Écran d'accueil (PRJ-01, PRJ-02, PRJ-05, ERG-08), monté dans `App` hors du registre : visible quand `home` est vrai et qu'aucun brouillon n'attend de réponse. Quatre actions : nouveau projet, ouvrir, à partir d'un projet existant, projet d'exemple (`file.example`) ; en dessous, les liens vers la prise en main rapide et le manuel utilisateur avec leur version (`DocumentLinks`, DOC-11). Il se ferme quand un projet est créé, ouvert ou restauré, ou par Échap.

Paramètres

`session` la session.

29.44 `ui/dialogs/RackDialogs`

Dialogues des armoires (ARM-01), par `store/rackActions`.

`AddRackDialog` fonction

```
function AddRackDialog({ onClose }: DialogProps)
```

« Ajouter une armoire » : modèle d'armoire de la bibliothèque (42U sans roulettes proposé), nom suivant, faces à poser sur la page active. Crée l'armoire et ses vues (`addRackWithViews`) puis la sélectionne.

Paramètres

`onClose` fermeture.

`RenameRackDialog` fonction

```
function RenameRackDialog({ onClose, props }: DialogProps)
```

« Renommer l'armoire » : nouveau nom, appliqué s'il n'est pas vide.

Paramètres

`onClose` fermeture ;

`props` `rackId` : l'armoire à renommer.

29.45 `ui/dialogs/PlaceDeviceDialog`

Placement d'un équipement au clavier (EQP-01).

`PlaceDeviceDialog` fonction

```
function PlaceDeviceDialog({ onClose, props }: DialogProps)
```

« Placer un équipement » : machine (hors extérieures), armoire, U du bas, face (par défaut la face de fixation de la machine, EQP-09), côté pour un 0U ou un demi-largeur, nom (proposé par `nextDeviceName`). Place par `placeWithConfirmation`, ferme et sélectionne l'équipement en cas de succès.

Paramètres

`onClose` fermeture ;

`props` `rackId` : armoire proposée (sinon la première).

29.46 `ui/dialogs/LinkDialog`

Création d'une liaison par formulaire (LIA-02), utile quand les ports sont sur des pages différentes.

LinkDialog fonction

```
function LinkDialog({ onClose, props }: DialogProps)
```

« Nouvelle liaison » : deux extrémités (équipement puis port, les ports occupés signalés), type (proposé selon les connecteurs des deux ports), repère facultatif, attributs (`LinkAttributes`). Crée par `createLinkWithConfirmation`, sélectionne la liaison et se ferme.

Paramètres

`onClose` fermeture ;

`props` `a` : première extrémité déjà choisie (port cliqué).

29.47 `ui/dialogs/LinksTableDialog`

Tableau de toutes les liaisons (LIA-08).

LinksTableDialog fonction

```
function LinksTableDialog({ onClose }: DialogProps)
```

Tableau filtrable (mots, type) et triable par colonne (repère, type, extrémités, extrémités finales, longueur, catégorie, référence, notes ; tri naturel en français, longueur en millimètres). Édition en place du repère, du type, de la longueur, de la catégorie, de la référence et des notes ; sélection d'une liaison sur le dessin ; suppression.

Paramètres

`onClose` fermeture.

29.48 `ui/dialogs/LinkVisibilityDialog`

Types de liaisons affichés ou masqués (LIA-10), réglage d'affichage seulement.

LinkVisibilityDialog fonction

```
function LinkVisibilityDialog({ onClose }: DialogProps)
```

Une case par type de liaison (avec sa couleur et son nombre de liaisons) qui modifie `hiddenLinkTypes` ; « Tout afficher ». Les liaisons masquées restent dans le projet.

Paramètres

`onClose` fermeture.

29.49 `ui/dialogs/RoutingDialog`

Goulottes du projet (ROU-02).

`RoutingDialog` fonction

```
function RoutingDialog({ onClose }: DialogProps)
```

Côté (gauche ou droite) de la goulotte de chaque type de liaison dans les armoires, valeur par défaut `defaultTraySide`, modifiée par `setProjectTray`.

Paramètres

`onClose` fermeture.

29.50 `ui/dialogs/SearchDialog`

Recherche globale (ERG-05, Ctrl+F) d'équipements, de ports et de repères de câbles, par `searchProject` et `pageOfTarget` (`model/search`).

`revealHit` fonction

```
function revealHit(ui: Ui, project: Project, hit: SearchHit, message: string)
```

Montre un résultat de recherche : la page qui le dessine devient active (la page active d'abord si elle le dessine), l'élément est sélectionné et son rectangle est demandé dans `reveal` ; s'il n'est dessiné sur aucune page, une notification affiche `message`.

Paramètres

`ui` état de l'interface ;

`project` le projet ;

`hit` le résultat ;

`message` notification si l'élément n'est sur aucune page.

Résultat. Aucun.

Erreurs. Aucune.

`SearchDialog` fonction

```
function SearchDialog({ onClose }: DialogProps)
```

Champ de recherche, nombre de résultats (limités, avec le total), liste de résultats (sorte, libellé, détail). Flèches pour parcourir, Entrée ou clic pour ouvrir un résultat par `revealHit` après fermeture.

Paramètres

`onClose` fermeture.

29.51 `ui/dialogs/PreferencesDialog`

Préférences de l'utilisateur (EDM-04), gardées dans le navigateur.

`PreferencesDialog` fonction

```
function PreferencesDialog({ onClose }: DialogProps)
```

Seuil d'alerte du poids des images, de 0,1 à 100 Mo, écrit dans `prefs` ; `main.tsx` l'enregistre dans le stockage local.

Paramètres

- `onClose` fermeture.

29.52 `ui/dialogs/HelpDialogs`

Dialogues de l'aide : raccourcis, à propos, notes de version (ERG-03, ERG-09, WEB-02).

`ShortcutsDialog` fonction

```
function ShortcutsDialog({ onClose }: { onClose: () => void })
```

Raccourcis clavier générés depuis le registre des commandes, un tableau par zone de l'interface ; les raccourcis secondaires sont listés après « ou ».

Paramètres

- `onClose` fermeture.

`DocumentLinks` fonction

```
function DocumentLinks({ titleKey }: { titleKey: string })
```

Liens vers les documents de l'utilisateur (`USER_DOCUMENTS` : prise en main rapide puis manuel utilisateur), précédés d'un titre qui donne la version documentée, celle de l'application (DOC-04, DOC-05, DOC-11). Élément `nav` nommé par ce titre ; chaque lien s'ouvre dans un nouvel onglet (`target="_blank"`, `rel="noopener"`) et porte `data-document` (identifiant du document). Utilisé par l'écran d'accueil et par « À propos ».

Paramètres

- `titleKey` clé de traduction du titre, interpolée avec `version` (`home.documents` ou `help.documents`).

`AboutDialog` fonction

```
function AboutDialog({ onClose }: { onClose: () => void })
```

Nom, slogan et version de l'application (`data-testid="about-version"`), mention de confidentialité, liens vers les documents de l'utilisateur (`DocumentLinks`).

Paramètres

- `onClose` fermeture.

`ReleaseNotesDialog` fonction

```
function ReleaseNotesDialog({ onClose }: { onClose: () => void })
```

Notes de version embarquées dans l'application (disponibles hors ligne), lues en Markdown (`parseMarkdown`) et rendues en éléments React (texte seulement, rien n'est injecté en HTML) ; le titre de niveau 1 devient le titre du dialogue.

Paramètres

`onClose` fermeture.

29.53 `ui/dialogs/LibraryDialogs`

Dialogues de la bibliothèque du projet : purge (BIB-05), réimportation (BIB-07), types de connecteurs et de liaisons (BIB-09), par `store/libraryActions`.

`PurgeDialog` fonction

```
function PurgeDialog({ onClose }: DialogProps)
```

« Purger la bibliothèque » : liste des machines inutilisées et gain estimé (images comprises), puis purge et notification du nombre et de la taille libérée.

Paramètres

`onClose` fermeture.

`ReimportDialog` fonction

```
function ReimportDialog({ onClose }: DialogProps)
```

Réimportation de modèles de la bibliothèque par défaut absents du projet : cases à cocher (tout cocher, tout décocher), puis `reimportDefaultModels`.

Paramètres

`onClose` fermeture.

`ConnectorTypesDialog` fonction

```
function ConnectorTypesDialog({ onClose }: DialogProps)
```

Types de connecteurs du projet : nom, famille, complémentaire, utilisation (machines et équipements), ajout, suppression. Un refus (type utilisé) est affiché par `ui.alert`.

Paramètres

`onClose` fermeture.

`LinkTypesDialog` fonction

```
function LinkTypesDialog({ onClose }: DialogProps)
```

Types de liaisons du projet : nom, couleur (changements fusionnés par type), style et épaisseur du trait, préfixe de repère, présence dans la légende, mode repères, connecteurs admis, nombre de liaisons, ajout, suppression (refus expliqué).

Paramètres

`onClose` fermeture.

29.54 `ui/dialogs/ImportModelsDialog`

Import de machines depuis une bibliothèque `.rkdglib` ou un projet `.rkdg` (BIB-08, BIB-10, BIB-11, BIB-14), par `planImport`, `checkImport` et `applyImportPlan`.

`ImportModelsDialog` fonction

```
function ImportModelsDialog({ onClose, props }: DialogProps)
```

Contenu de la source (description, machines par catégorie avec recherche, aperçu de la machine choisie, images servies par des URL d'objet libérées à la fermeture), cases à cocher (nouvelles machines cochées, versions plus récentes proposées en mise à jour), choix pour chaque conflit ou pour tous (garder la machine du projet, la remplacer, ou garder les deux), taille ajoutée. Applique le plan, notifie le nombre de machines copiées et affiche les avertissements.

Paramètres

`onClose` fermeture ;

`props` `source` (contenu lu), `fileName`, `projectName`.

29.55 `ui/dialogs/ExportLibraryDialog`

Export de machines du projet en bibliothèque `.rkdglib` (BIB-12).

`ExportLibraryDialog` fonction

```
function ExportLibraryDialog({ onClose }: DialogProps)
```

Description de la bibliothèque (nom, marque, auteur, version, licence, description, proposés par `proposedDescription`), machines à exporter (celles du projet cochées, les génériques sur demande), taille des images ; écrit l'archive et la télécharge sous `<nom>_v<version>.rkdglib`.

Paramètres

`onClose` fermeture.

29.56 `ui/dialogs/ImportSpreadsheetDialog`

Import tableur simplifié (IMP-01 à IMP-07) : création seulement, tout ou rien, par `import/rules` et `importSpreadsheet`.

`ImportSpreadsheetDialog` fonction

```
function ImportSpreadsheetDialog({ onClose }: DialogProps)
```

Choix d'un fichier `.xlsx` ou `.csv`, lecture, plan d'import sur la page active ; affiche ce qui sera créé ou le tableau des problèmes (feuille, ligne, colonne, motif) avec téléchargement du rapport ; bouton du modèle de fichier (`import.template`). L'import réussi est notifié et ferme le dialogue.

Paramètres

`onClose` fermeture.

Exports

29.57 `ui/exports/download`

Téléchargement direct des fichiers exportés (EXP-01).

`downloadExport` fonction

```
function downloadExport(file: ExportedFile)
```

Télécharge un fichier produit par un export (octets, nom, type MIME) par `downloadBytes` (`io/files`). Appelée par tous les dialogues d'export.

Paramètres

- `file` le fichier exporté.

Résultat. Aucun.

Erreurs. Aucune.

29.58 `ui/exports/ExportPartsFields`

Choix des parties du dessin à exporter (EXP-05), partagé par les exports d'image et de PDF.

`PartOption` type

```
type PartOption = 'titleBlock' | 'legend' | 'cableLabels' | 'grid';
```

Partie facultative d'un export : `titleBlock` (cartouche), `legend`, `cableLabels` (repères des câbles) ou `grid` (grille de repérage).

`ExportPartsFields` fonction

```
function ExportPartsFields({ parts, onChange, only, }: { parts: PageParts; onChange:
(parts: PageParts) => void; /** Options offered (all by default). */ only?: readonly
PartOption[]; })
```

Groupe de cases à cocher, une par partie (`data-testid="export-part-…"`).

Paramètres

- `parts` parties actuelles ;
- `onChange` reçoit les nouvelles parties ;
- `only` parties proposées (toutes par défaut).

29.59 `ui/exports/ExportImageDialog`

Export SVG ou PNG (EXP-02, EXP-03, EXP-05), par `buildImageExport` (`export/imageExport`).

`ExportImageDialog` fonction

```
function ExportImageDialog({ onClose, props }: DialogProps)
```

Format, étendue (page active ou vue sélectionnée), parties, et pour le PNG la résolution (dpi ou largeur en pixels, taille résultante affichée) et le fond. Les types de liaisons masqués le restent. Une erreur d'export est affichée par `ui.alert`.

Paramètres

`onClose` fermeture ;

`props` `format` : `png` pour proposer le PNG (SVG sinon).

29.60 `ui/exports/ExportPdfDialog`

Export PDF (EXP-01, EXP-05), par `buildPdfExport` (`export/pdfExport`).

`ExportPdfDialog` fonction

```
function ExportPdfDialog({ onClose }: DialogProps)
```

Pages à exporter (toutes cochées, avec leur format), parties, qualité JPEG des photos.

Paramètres

`onClose` fermeture.

29.61 `ui/exports/ExportSpreadsheetDialog`

Export des listes du projet en XLSX ou CSV (EXP-04, EXP-05), par `export/tables`.

`ExportSpreadsheetDialog` fonction

```
function ExportSpreadsheetDialog({ onClose }: DialogProps)
```

Format, séparateur CSV, listes à exporter et colonnes de chaque liste ; l'export est impossible si une liste choisie n'a plus de colonne.

Paramètres

`onClose` fermeture.

29.62 `ui/exports/ExportLabelsDialog`

Étiquettes de câbles (EXP-07), par `export/labelsExport`.

`ExportLabelsDialog` fonction

```
function ExportLabelsDialog({ onClose }: DialogProps)
```

Étendue (liaisons sélectionnées ou toutes, triées par repère), gabarit de planche (papier, taille des étiquettes, marges et espacements en mm, nombre de colonnes et de lignes), extrémités finales, contours, aperçu des planches ; export PDF ou CSV. Un gabarit invalide est expliqué et bloque l'export.

Paramètres

`onClose` fermeture.

Bibliothèque en édition, machines et faces

29.63 `ui/library/LibrarySheet`

Centre de la fenêtre pendant l'édition d'une bibliothèque (BIB-13), à la place du dessin.

`LibrarySheet` fonction

```
function LibrarySheet()
```

Description de la bibliothèque (nom, marque, auteur, licence, description ; version d'enregistrement) modifiée par `setLibraryInfo` (refus notifié), commandes (nouvelle machine, nouvelle machine extérieure, import, enregistrer), puis les machines par catégorie en cartes : clic pour choisir, double-clic pour l'éditeur de machine. Sans propriétés.

29.64 `ui/machine/paramFields`

Champs du formulaire des paramètres de chaque équipement intégré (§ 7.4.3) : ce que le formulaire montre et comment. La validation reste aux schémas du catalogue (`library/equipment/params`). Sans React.

`ParamField` type

```
type ParamField = FieldBase & ( | { type: 'number'; integer: boolean } | { type: 'bool' }
| { type: 'text' } | { type: 'color' } /** Choice in a list: labels
`equipment.values.<values>.<option>`, or « n U » for heights. */ | { type: 'enum'; options:
readonly (string | number)[]; values: string | 'heightU' } /** Power supplies: count and
type. */ | { type: '…
```

Un champ du formulaire. Champs communs : `key` (clé du paramètre, libellé `equipment.params.<clé>`) et `when` (condition d'affichage, facultative). Variantes par `type` :

Champs

`number` nombre, entier si `integer` ;

`bool` , `text`, `color` interrupteur, texte, couleur ;

`enum` choix dans `options`, libellés `equipment.values.<values>.…` ou « n U » pour `heightU` ;

`psu` alimentations : nombre et type ;

`portGroups` groupes de ports (connecteur, nombre, face sauf `external`, préfixe) ; connecteurs parmi `connectors` ou types du projet des familles `projectFamilies` ;

`outlets` groupes de prises : nombre et type ;

`patchModules` modules d'un panneau de brassage, emplacement par emplacement ;

`lines` un texte par ligne ;

`enumList` liste de choix (entrées vidéo).

`PARAM_FIELDS` constante

```
const PARAM_FIELDS: Readonly<Record<EquipmentKind, readonly ParamField[]>>
```

Champs du formulaire de chaque équipement, dans l'ordre d'affichage (commutateur, routeur, pare-feu, serveur, stockage, KVM, PDU, onduleur, obturateur, passe-câbles, tablette, panneaux de brassage cuivre et fibre, moniteur, équipements extérieurs, générique). Ajouter un paramètre à un équipement demande de l'ajouter ici et dans le schéma du catalogue.

`ConnectorOption` interface

```
interface ConnectorOption { value; label; source }
```

Connecteur proposé pour un groupe de ports.

Champs

`value` clé d'un type par défaut ou identifiant d'un type du projet ;

`label` nom affiché ;

`source` `default` (type de Portolan) ou `project` (type créé dans la bibliothèque du projet, BIB-09).

`connectorOptions` fonction

```
function connectorOptions(field: { connectors: readonly string[]; projectFamilies: readonly
ConnectorFamily[] | 'all' }, connectorTypes: readonly ConnectorType[], current?: unknown):
ConnectorOption[]
```

Connecteurs proposés pour un groupe de ports : les types par défaut du champ dans leur ordre, puis les types créés dans le projet des familles du champ, par nom. La valeur actuelle est toujours proposée, même si elle ne convient plus (un type supprimé est nommé « type inconnu »).

Paramètres

`field` `connectors` et `projectFamilies` du champ ;

`connectorTypes` types de connecteurs du projet ;

`current` valeur actuelle, facultative.

Résultat. Liste de `ConnectorOption`.

Erreurs. Aucune.

`freshPrefix` fonction

```
function freshPrefix(groups: readonly { prefix?: unknown }[], connector:
string, connectorTypes: readonly ConnectorType[] = []): string
```

Préfixe d'un nouveau groupe de ports : le nom court de son connecteur (suivi d'un tiret s'il finit par un chiffre), ou « G<n>- » s'il est déjà pris, pour que les ports ne prennent jamais les noms d'un autre groupe (les noms de ports sont uniques sur une face).

Paramètres

`groups` groupes existants (leur `prefix`) ;

`connector` connecteur du nouveau groupe ;

`connectorTypes` types du projet, pour nommer un type personnalisé.

Résultat. Le préfixe.

Erreurs. Aucune.

```
freshPrefix([], 'rj45');                      // 'RJ45-'
freshPrefix([{ prefix: 'RJ45-' }], 'rj45');   // 'G2-'
```

29.65 `ui/machine/MachineParamsForm`

Formulaire des paramètres d'un équipement intégré (§ 7.4.3), d'après `PARAM_FIELDS`, et libellé de ses problèmes. Utilisé par l'éditeur de machine.

`paramsIssues` fonction

```
function paramsIssues(kind: EquipmentKind, params: Params, t: (key: string) =>
string, connectorTypes: readonly ConnectorType[])
```

Problèmes de paramètres, libellés avec les noms des champs : erreurs du schéma de l'équipement (« Groupes de ports › 2 › Nombre : … »), sinon connecteurs du projet disparus (BIB-09), sinon noms de ports en double.

Paramètres

`kind` équipement ;

`params` paramètres ;

`t` fonction de traduction ;

`connectorTypes` types de connecteurs du projet.

Résultat. Liste de messages, vide si les paramètres sont valides.

Erreurs. Aucune.

`MachineParamsForm` fonction

```
function MachineParamsForm({ kind, params, set, connectorTypes, }: { kind: EquipmentKind;
params: Params; set: (patch: Params) => void; /** Connector types of the project, proposed
for the groups of ports (BIB-09). */ connectorTypes: readonly ConnectorType[]; })
```

Formulaire : un contrôle par champ visible (`when`) : nombres validés en quittant le champ, interrupteurs, listes (avec groupes d'options), listes éditables de groupes (ajout, suppression, préfixe), modules de panneau de brassage avec remplissage rapide.

Paramètres

`kind` équipement ;

`params` paramètres actuels ;

`set` reçoit un correctif des paramètres ;

`connectorTypes` types de connecteurs du projet proposés pour les ports.

29.66 `ui/machine/MachineEditorDialog`

Éditeur de machine (EDM-01 à EDM-11, EQT-05, MAC-01), par `store/machineActions` et `model/modelEditor`.

`MachineEditorDialog` fonction

```
function MachineEditorDialog({ onClose, props }: DialogProps)
```

Une fenêtre pour une machine d'un équipement intégré. À gauche : nom (proposé tant qu'il suit les paramètres), marque, référence, mots-clés, profondeur, dégagement, face de fixation et `MachineParamsForm`. À droite : `RackPreview`, puis chaque face (`FaceEditor`) avec photo ou dessin par défaut, grille (`PortGridForm`), tableau (`PortsTable`) et copie des positions d'une autre machine, d'une autre face ou d'un fichier (EDM-07). Rien ne change avant « Enregistrer la machine », une seule action : modification, nouvelle machine, ou variante (machine générique, portée limitée à un équipement) donnée aux équipements concernés ; liaisons perdues confirmées, refus expliqués. Fermer avec des modifications demande confirmation.

Paramètres

`onClose` fermeture ;

`props` `machineId` (machine à éditer ; une machine générique est enregistrée comme nouvelle), `fromMachineId` (nouvelle machine à partir de celle-ci, EQT-04), `kind` (équipement d'une nouvelle machine), `deviceId` (équipement d'où l'éditeur est ouvert).

29.67 `ui/modelEditor/RackPreview`

Aperçu de la machine éditée dans une armoire (EDM-09).

`RackPreview` fonction

```
function RackPreview({ model }: { model: Machine })
```

Machine dessinée à l'échelle dans une armoire juste assez haute, face avant et face arrière.

Paramètres

`model` la machine en cours d'édition.

29.68 `ui/modelEditor/FaceEditor`

Face de la machine éditée (EDM-05, MAC-01).

`FaceEditor` fonction

```
function FaceEditor({ model, face, zoom, selected, onSelect, onPorts, onDropFile, }: {
model: Machine; face: FaceSide; zoom: number; selected: ReadonlySet<Id>; onSelect: (ids:
Set<Id>) => void; /** Replaces the ports of the face (one change, or a drag). */ onPorts:
(ports: Port[], dragKey?: string) => void; onDropFile: (file: File) => void; })
```

Photo ou dessin par défaut de la face, avec ses ports (issus des paramètres) déplacés et redimensionnés à la souris (une clé de fusion par glisser), sélection au clic (Maj pour ajouter), zoom ; un fichier image peut y être déposé.

Paramètres

`model` la machine en cours d'édition ;

`face` face affichée ;

`zoom` facteur d'agrandissement ;

`selected` , `onSelect` ports sélectionnés et leur changement ;

`onPorts` remplace les ports de la face (avec la clé du glisser) ;

`onDropFile` reçoit le fichier image déposé.

29.69 `ui/modelEditor/PortGridForm`

« Disposer en grille » (EDM-06), par `arrangePorts` (`model/modelEditor`).

`PortGridForm` fonction

```
function PortGridForm({ ports, selected, onArrange, onClose, }: { ports: Port[]; selected:
ReadonlySet<Id>; onArrange: (ports: Port[], ids: Id[]) => void; onClose: () => void; })
```

Place en une opération les ports choisis (sélection, un groupe ou tous) sur une grille : lignes et colonnes, premier port, espacements, taille (en % de la face), numérotation.

Paramètres

`ports` ports de la face ;

`selected` ports sélectionnés ;

`onArrange` reçoit les ports disposés et les identifiants concernés ;

`onClose` ferme le formulaire.

29.70 `ui/modelEditor/PortsTable`

Tableau des ports d'une face (EDM-08).

`PortsTable` fonction

```
function PortsTable({ ports, connectors, selected, onSelect, onPorts, }: { ports: Port[];
connectors: ConnectorType[]; selected: ReadonlySet<Id>; onSelect: (ids: Set<Id>) => void;
onPorts: (ports: Port[]) => void; })
```

Une ligne par port : nom, connecteur et groupe en lecture (ils viennent des paramètres), position et taille en % de la face et sortie de câble modifiables (valeurs bornées) ; un port positionné à la main est « placé ».

Paramètres

`ports` ports de la face ;

`connectors` types de connecteurs du projet ;

`selected` , `onSelect` sélection ;

`onPorts` reçoit les ports modifiés.

Index

AboutDialog, 516
ActionResult, 440
activePage, 425
addAnnotation, 445
addConnectorType, 458
addExternalView, 465
addInstancePort, 467
addLink, 444
AddLinkOptions, 443
addLinkType, 458
addMachine, 460
addModel, 455
addPage, 464
addRack, 441
AddRackDialog, 513
AddRackOptions, 470
addRackViews, 471
addRackWithViews, 471
addRevision, 468
addTitleBlockField, 469
addVariantMachine, 278
addView, 465
advanceWidth, 336
ALIGN_COMMANDS, 424
alignItems, 447
AlignMode, 217
alignOffsets, 217
alongPlace, 308
anchorManualRoute, 318
Annotation, 180
ANNOTATION_DRAG_TYPE, 492
ANNOTATION_GROUPS, 230
ANNOTATION_PRESETS, 230
annotationBounds, 226
annotationCommands, 424
AnnotationKind, 224
AnnotationPatch, 445
AnnotationPreset, 230
AnnotationProperties, 508
AnnotationSchema, 180
annotationsInRectangle, 229
AnnotationsLayer, 338
Anomaly, 214
AnomalyKind, 214
AnomalyTarget, 214
App, 478
app-version (module), 261
app/pwa (module), 261
app/session (module), 262
app/site (module), 265
APP_FONT_SOURCES, 396
APP_VERSION, 261
applyImportPlan, 459
applyMachineParams, 461
ApplyOptions, 451
ArchiveWriteError, 378
Area, 309
arrangePorts, 244
ArrangeSection, 508
ArrangeSelection, 447
arrangeSelection, 424
arrowHead, 228
arrowHeadLength, 228
arrowHeads, 229
AttachedPoint, 180
AttachedPointSchema, 180
attachPoints, 301
availableInLibraryDocument, 435
AvoidModule, 303

Band, 313
BlankingPanelSchema, 292
Block, 268
Box, 198
BOX_HANDLES, 498
BoxAnnotation, 225
boxContains, 251
boxesIntersect, 250
boxFromPoints, 251
BoxHandle, 498
Bridge, 305
BRIDGE_RADIUS_MM, 304
bridgesByCable, 306
buildImageExport, 402
buildImportReport, 377

buildImportTemplate, 377
buildLabelsCsv, 407
buildLabelsPdf, 407
buildPdfExport, 409
buildSpreadsheetExport, 416
buildTable, 421
builtinId, 161
BUNDLE_FONT_SIZE, 307
BUNDLE_FONT_SIZES, 307
BUNDLE_WIDTH_FACTOR, 307
BundleDrawing, 322
bundlePartners, 310
BundleSplit, 307
byGenericThenName, 240

CABLE_MANAGER_TYPES, 292
CABLE_REF_FONT_SIZE, 307
CableDrawing, 321
CableLabel, 403
cableLabels, 404
CableManagerSchema, 292
cablePathData, 306
CablePaths, 304
CableRefLabel, 322
Canvas, 496
carriedLayout, 356
CASTOR_MM, 197
Catalogue, 271
CATALOGUE_PATH, 271
CatalogueEntry, 271
CatalogueEntrySchema, 270
CatalogueModelSchema, 270
CatalogueSchema, 271
Cell, 420
cellNumber, 375
cellText, 375
centreVisible, 249
changeDeviceMachine, 280
changeMachine, 277
changeMachineParams, 277
CheckboxField, 510
checkCompatibility, 210
checkDevicePlacement, 193
checkImageRatio, 243
checkImport, 238
checkLibraryReferences, 186
checkLinkEnds, 209
checkPlacement, 192
checkRackResize, 194
checkReferences, 186
ChecksPanel, 504
chooseFile, 494
clampPort, 245
clampZoom, 487
clean, 316
clientToPage, 496
Clipboard, 488
clipToWindow, 248
closestData, 496
codePoints, 337
CodeRange, 337
collectImageRefs, 390
collectLibraryImageRefs, 391
ColorField, 510
ColorSchema, 168
Column, 420
COLUMN_SHARE, 259
ColumnUse, 373
Command, 434
CommandButton, 479
CommandContext, 433
commandForKey, 436
CommandList, 480
commands, 434
commands/annotation (module), 423
commands/context (module), 425
commands/device (module), 425
commands/edit (module), 427
commands/export (module), 427
commands/file (module), 427
commands/help (module), 428
commands/import (module), 429
commands/index (module), 429
commands/inspect (module), 429
commands/library (module), 430
commands/libraryFiles (module), 430
commands/link (module), 431
commands/page (module), 431
commands/project (module), 432
commands/rack (module), 432
commands/registry (module), 433
commands/route (module), 436
commands/selectionOps (module), 437
commands/shortcuts (module), 438
commands/view (module), 439
commandShortcuts, 436
CommandZone, 433
CompatibilityProblem, 210
CompatibilityResult, 210
ConflictChoice, 235
connector, 360
CONNECTOR_IDS, 274
CONNECTOR_KEYS, 274
CONNECTOR_SHAPES, 274
connectorDeviceUsage, 457

ConnectorFamily, 170
ConnectorFamilySchema, 170
connectorIds, 236
connectorName, 274
ConnectorNames, 281
connectorNames, 281
ConnectorOption, 522
connectorOptions, 522
connectorProblems, 297
connectorsCompatible, 210
ConnectorType, 170
connectorTypeIdOf, 281
ConnectorTypeSchema, 170
ConnectorTypesDialog, 517
connectorTypeUsers, 276
connectorUsage, 457
connectorValueOf, 288
CONSOLE_TYPES, 289
ContextMenu, 479
contextMenuItems, 483
ContextMenuState, 486
ControlPanelSchema, 295
ConvertedImage, 411
convertImage, 411
copyAnnotations, 231
copyModel, 455
copyPortPositions, 244
countCrossings, 306
coversCodePoint, 337
crc32, 413
createAnnotation, 231
createCanvasMeasurer, 334
createDefaultConnectorTypes, 274
createDefaultLibrary, 298
createDefaultLinkTypes, 275
createDefaultMachines, 273
createDefaultRackModels, 273
createDefaultTitleBlockFields, 160
createDevice, 158
createEditor, 452
createEmptyLibrary, 155
createExampleProject, 453
createGenericMachine, 286
createGenericMachines, 286
createImageUrlResolver, 354
createLink, 158
createLinkWithConfirmation, 492
createMachine, 157
createNewProject, 298
createPage, 158
createPort, 157
createProject, 160
CreateProjectOptions, 160
createRack, 156
createRackFromModel, 156
createRackModel, 156
createRackPage, 471
createRevision, 159
createSession, 264
createStarterProject, 299
createSvgSanitizer, 394
createTitleBlockField, 160
createUi, 489
createView, 159
Crossing, 305
cssFont, 332
CsvCell, 397
csvCells, 397
CsvEncoding, 369
CsvSeparator, 397
csvValue, 397
CUT_MARGIN_MM, 201
CUT_RENVOI_STEP, 326
CUT_RENVOI_STUB, 326
cutFrameBottom, 203
cutLines, 203
cutOutline, 203

Dash, 179
dashArray, 229, 342
DashSchema, 179
dataUrl, 398
decodeCsv, 369
decodeXmlText, 398
decrypt, 272
dedupe, 318
DEFAULT_DEPTHS, 280
DEFAULT_IMAGE_LIMIT_MB, 242
DEFAULT_LABEL_TEMPLATE, 403
DEFAULT_LIBRARY_ID, 285
DEFAULT_LIBRARY_VERSION, 285
DEFAULT_LINK_TYPE_ID, 275
DEFAULT_PREFERENCES, 488
DEFAULT_READ_LIMITS, 380
defaultImportSelection, 238
defaultLabelsBeside, 159
defaultLibraryOrigin, 286
defaultParams, 296
defaultTitleBlockRow, 258
defaultTraySide, 325
defaultUsableDepth, 155
deleteSelection, 438
deletionPlan, 437
DepthOverflow, 189
DepthTerm, 189

DepthTooltip, 501
DepthTooltipState, 501
deriveMachine, 278
DetectedSeparator, 370
detectEncoding, 369
detectSeparator, 370
Device, 178
DEVICE_COLUMN_USE, 373
DEVICE_COLUMNS, 373
deviceBox, 199
deviceColumnLabel, 374
deviceCommands, 426
DeviceFace, 340
DeviceFaceProps, 339
deviceHasPort, 164
DeviceInput, 157
deviceInRackFace, 248
deviceLabel, 254
DeviceMachineChange, 279
DeviceOnPage, 249
deviceOnPage, 249
devicePortEntries, 163
devicePrefix, 205
DeviceProperties, 506
DeviceSchema, 177
devicesInRack, 192
devicesOnPage, 254
DevicesProperties, 507
DeviceView, 347
DialogHost, 511
DialogProps, 511
DialogRequest, 485
DIALOGS, 511
disabledHintKey, 435
displayShortcut, 439
distanceToBox, 300
DISTRIBUTE_COMMANDS, 424
DistributeAxis, 217
distributeItems, 448
distributeOffsets, 218
DocumentLinks, 516
downloadBytes, 387
downloadExport, 519
DraftDialog, 479
DraftRecord, 393
draftRecord, 384
DraftSaver, 383
DraftSaverOptions, 383
drawingArea, 253
DrawingOptions, 416
drawingsToPdf, 410
dropLinks, 440
DropPreview, 498
dropUnderDevice, 311
duplicateModel, 456
duplicatePage, 464
duplicateRack, 472
duplicateRefs, 212

editCommands, 427
Editor, 451
EditorContext, 472
EditorState, 451
effectiveScale, 253
effectiveURange, 202
ELLIPSIS, 333
embeddedFontCss, 400
encrypt, 272
endPlace, 310
EndToEndPath, 212
endToEndPath, 213
ensureGenericMachine, 463
Entry, 314
EQUIPMENT_CATEGORIES, 172
EQUIPMENT_COLORS, 280
EQUIPMENT_KINDS, 172
EquipmentCategory, 172
equipmentCategory, 153
EquipmentKind, 172
EquipmentKindSchema, 172
equipmentParamsProblem, 284
equipmentTree, 153
equivalentPort, 279
ErrorDialog, 480
EXAMPLE_LAPTOP_SVG, 453
EXAMPLE_REVISION_DATE, 453
ExampleProject, 453
executeCommand, 436
ExplorerPanel, 504
export/appFonts (module), 396
export/csv (module), 396
export/encoding (module), 398
export/fileNames (module), 399
export/fonts (module), 400
export/imageExport (module), 401
export/labels (module), 402
export/labelsExport (module), 406
export/pdfExport (module), 408
export/pdfImages (module), 410
export/png (module), 412
export/spreadsheet (module), 414
export/svgExport (module), 416
export/tables (module), 419
export/types (module), 422
exportBaseName, 399

exportCommands, 427
exportDrawing, 401
ExportedFile, 422
ExportError, 422
exportFileName, 399
ExportImageDialog, 520
ExportLabelsDialog, 521
ExportLibraryDialog, 518
ExportPartsFields, 519
ExportPdfDialog, 520
ExportSpreadsheetDialog, 520
extensionFor, 389
EXTERNAL_LABEL_MM, 221
EXTERNAL_LABEL_OVERHANG_MM, 221
EXTERNAL_MAX_SCALE, 221
EXTERNAL_POWER, 294
ExternalDrawing, 341
externalLogoLayout, 356
ExternalPlacementSchema, 177
ExternalPortGroup, 289
ExternalPortGroupSchema, 288
externalViewLayout, 223
externalViewSize, 247

Face, 174
FaceEditor, 524
facePorts, 163
faceScale, 349
FaceSchema, 174
FaceSide, 169
FaceSideSchema, 169
faceTemplateSize, 242
FEET_MM, 197
FIBER_CONNECTORS, 294
FiberPanelSchema, 294
fileCommands, 428
findCommand, 434
findCrossings, 305
findGenericMachine, 287
FirewallSchema, 289
FIT_MARGIN_PX, 487
fitLabel, 406
fitLogo, 355
fitShape, 359
FittedLine, 405
fitZoom, 487
flipDevices, 207
flippedPlacement, 192
flipRackDevices, 450
FlipResult, 206
followRemovedViews, 474
FONT_ASCENT, 331
FONT_DESCENT, 331
FONT_FAMILY, 331
FONT_WEIGHTS, 331
fontBytes, 330
FontSources, 400
FontSpec, 331
FontWeight, 331
FORCED_COLOR, 339
ForeignPortHit, 367
foreignPortHits, 367
FORMAT_VERSION, 168
formatBytes, 270
formatDate, 269
formatDepthCalculation, 191
formatIssue, 185
formatNumber, 269
formatPath, 185
formatURange, 191
FRAME_MM, 197
FREE_COLOR, 498
freeAlongPlace, 309
freeBands, 313
freeOutletPrefix, 291
freePortPosition, 216
freeSpaceBoxes, 252
FreeSpaceSummary, 195
freeSpaceSummary, 196
freeUCount, 195
freshPrefix, 522
FromExistingOptions, 166

generated, 361
GeneratedRackOptions, 364
generatedRackProject, 364
generateMachine, 284
GenerateOptions, 284
genericMachineId, 285
GenericSchema, 293
glyphIndex, 335
grid, 282
GRID_SNAP_STEP, 217
GRID_STEP, 344
gridCell, 244
GridNumbering, 243
Guide, 218
GuidesLayer, 498

HALF_PANEL_MM, 242
hasExternalDrawing, 340
hasText, 225
hasTransparency, 411
headerKey, 375
help/documents (module), 266
help/markdown (module), 267

helpCommands, 428
hiddenEnds, 312
HISTORY_LIMIT, 450
HistoryEntry, 451
HOLE_OFFSETS_MM, 200
HOLE_SIZE_MM, 200
HomeScreen, 512
HORIZONTAL_PDU_MAX_OUTLETS, 291
horizontalClear, 313

i18n, 269
i18n/index (module), 268
Id, 168
IdReuse, 281
IdSchema, 168
IMAGE_ACCEPT, 494
ImageExportRequest, 401
ImageExtension, 388
imageExtension, 389
ImageFormat, 401
ImageMap, 389
imageMimeType, 389
imagePixels, 494
imagePlaceholders, 409
ImageRef, 169
imageRefFor, 390
ImageRefSchema, 169
ImageUrlContext, 354
ImageUrlResolver, 354
import/read (module), 368
import/rules (module), 372
import/template (module), 376
ImportCandidate, 235
importCandidates, 236
ImportCheck, 238
importCommands, 429
ImportedImage, 494
importImageFile, 495
ImportIssue, 372
ImportModelsDialog, 518
ImportOptions, 373
ImportPlan, 237, 372
ImportResult, 372
ImportSheetKind, 372
ImportSource, 235
importSpreadsheet, 454
ImportSpreadsheetDialog, 518
indexProject, 164
INLET_TYPES, 291
insertAnnotation, 493
insertPoint, 317
inspectCommands, 429
INSTANCE_PORT_MM, 215
InstancePort, 174
instancePortName, 215
instancePortProblems, 216
InstancePorts, 507
instancePorts, 163
InstancePortSchema, 173
interMediumTrueType, 363
inWindow, 203
io/archive (module), 377
io/draft (module), 383
io/files (module), 385
io/hash (module), 388
io/images (module), 388
io/index (module), 391
io/migrations (module), 391
io/storage (module), 393
io/svg (module), 394
isAnnotationPreset, 230
isBevel, 312
isCommandEnabled, 435
isCustomConnector, 287
isDefaultConnector, 287
isDirty, 452
isExternal, 154
isExternalKind, 154
isImageTooHeavy, 243
isLibraryDocument, 232
isLight, 339
isLineAnnotation, 225
IsoDateSchema, 182
IsoDateTimeSchema, 182
isPurgeable, 167
isSideways, 226
isTextField, 497
isToggleInput, 497
isZeroU, 154
itemAt, 495

JsonObject, 392

keepOrthogonal, 302
KeyboardMouseSchema, 295
KeyEventLike, 438
KIND_CATEGORY, 173
kindsOfCategory, 153
KvmSchema, 290

LABEL, 343
LABEL_PADDING, 405
LABEL_PAPERS, 403
labelledByParams, 350
LabelLine, 405
labelLines, 405

LabelPaper, 402
labelsCsv, 406
LabelSheet, 341
labelSheetDrawings, 407
labelSheets, 404
LabelsRequest, 406
LabelTemplate, 403
labelZone, 349
LaidOutLine, 333
LaptopSchema, 295
LateralSide, 169
LateralSideSchema, 169
LayoutOptions, 333
layoutText, 334
LEFT_TABS, 502
LeftPanel, 502
LeftTab, 502
Leg, 326
LEGEND_SHARE, 259
LegendBlock, 353
legendLinkTypes, 254
lengthInMetres, 420
libraries/catalogue (module), 270
libraries/crypto (module), 271
libraries/page (module), 273
LIBRARIES_PAGE, 265
Library, 176
library/defaultModels (module), 273
library/defaultTypes (module), 273
library/equipment/change (module), 275
library/equipment/generators (module), 280
library/equipment/index (module), 287
library/equipment/params (module), 287
library/index (module), 298
LIBRARY_DOCUMENT_COMMANDS, 435
libraryCommands, 430
LibraryDescription, 234
LibraryDocument, 184
libraryDocument, 233
LibraryDocumentOptions, 233
LibraryDocumentSchema, 184
libraryDrag, 503
libraryFileCommands, 431
libraryFileName, 234
libraryForExport, 239
LibraryInfo, 184
LibraryInfoSchema, 183
LibraryPanel, 503
LibrarySchema, 176
LibrarySheet, 521
libraryToSave, 234
LibraryTree, 241
libraryTree, 241
LibraryTreeCategory, 241
LibraryTreeEquipment, 240
LineAnnotation, 224
linePath, 228
LineStyleSchema, 170
Link, 178
LINK_COLUMN_USE, 374
LINK_COLUMNS, 373
LINK_TYPE_IDS, 275
LinkAttributes, 508
LinkAttributeValues, 507
linkColumnLabel, 374
linkCommands, 431
LinkDialog, 514
LinkDrawing, 324
linkedExternalDevices, 222
LinkEndMessage, 208
LinkEndProblem, 208
LinkInput, 158
LinkProperties, 508
LinkSchema, 178
LinksLayer, 342
linksOnPort, 209
LinksTableDialog, 514
LinkType, 171
linkType, 361
LinkTypeSchema, 170
LinkTypesDialog, 517
linkTypeShortName, 307
LinkVisibilityDialog, 514
listAnomalies, 215
loadAvoid, 304
loadedAvoid, 304
localBox, 226
LocalStorage, 393
LOGO_MAX_HEIGHT_MM, 355
LOGO_MAX_MM, 259
LOGO_MAX_WIDTH_MM, 355
LOGO_MAX_ZONE_SHARE, 355
LOGO_SHARE, 259
LogoLayout, 355

Machine, 176
machine, 361
machineCategory, 153
MachineChange, 275
machineConnectorTypeIds, 276
MachineEditorDialog, 524
MachineEquipment, 175
MachineEquipmentSchema, 175
machineImageRefs, 152

MachineInput, 157
MachineLogo, 175
MachineLogoSchema, 174
MachineOrigin, 174
MachineOriginSchema, 174
machineParamsChange, 461
MachineParamsForm, 523
MachineParamsOptions, 460
machinePortEntries, 162
MachineSave, 462
machineSaveChange, 462
MachineSchema, 175
main (module), 299
Manifest, 378
ManifestSchema, 378
MANUAL_URL, 267
manualPolyline, 302
ManualRoute, 181
ManualRouteSchema, 181
MarkerLabel, 319
markerLabels, 319
markPlaced, 245
matchesSearch, 240
matchesShortcut, 439
MAX_IMAGE_SIDE, 410
MAX_PNG_AREA, 412
MAX_PNG_SIDE, 412
MAX_TRACK_SPACING_MM, 326
MeasureText, 334
MenuBar, 481
MenuDefinition, 483
menuEntries, 480
MENUS, 483
MessageHost, 481
MessageRequest, 485
migrate, 392
Migration, 392
MigrationResult, 392
MIGRATIONS, 392
MIN_ANNOTATION_SIZE, 227
MIN_FORMAT_VERSION, 392
MIN_TRACK_SPACING_PAPER, 325
mirrored, 282
missingConnectorTypes, 276
missingDefaultModels, 457
missingGenericMachines, 286
Modal, 481
model/annotations (module), 224
model/anomalies (module), 214
model/arrange (module), 217
model/autoLayout (module), 220
model/depthInfo (module), 207
model/dropTarget (module), 204
model/equipment (module), 152
model/factories (module), 155
model/ids (module), 161
model/index (module), 161
model/instancePorts (module), 215
model/libraryEdit (module), 232
model/libraryTransfer (module), 234
model/libraryTree (module), 239
model/links (module), 208
model/lookup (module), 161
model/modelEditor (module), 241
model/pageGeometry (module), 246
model/pageLayout (module), 252
model/placement (module), 188
model/portStates (module), 165
model/projectTemplate (module), 166
model/purge (module), 167
model/rackGeometry (module), 196
model/schema (module), 168
model/search (module), 255
model/titleBlockLayout (module), 257
model/validate (module), 185
MODEL_DRAG_TYPE, 503
modelImageBytes, 455
ModelPreview, 503
modelUsage, 454
MonitorSchema, 294
MoveBase, 447
moveDevice, 442
moveDevicesBy, 206
moveDevicesByU, 450
moveItems, 447
movePage, 465
moveSegment, 316
moveTitleBlockField, 469
moveVertex, 317
moveViews, 466

nameKey, 374
nearestFreePlacement, 205
nearestSegment, 318
nearestView, 300
NETWORK_CONNECTORS, 288
NewFromExistingDialog, 512
newId, 161
newInstancePort, 216
newLibraryDocument, 233
nextBubbleNumber, 229
nextDeviceName, 205
nextLinkRef, 212
nextPageTitle, 463
nextRackName, 470

nextRevisionIndex, 166
normalizeSearch, 256
NotEncryptedError, 272
NUDGING_MM, 320
NumberField, 509
NUMBERING_MM, 197

occupiedUs, 190
offerUpdate, 262
OpenedFile, 386
OPENING_10_MM, 197
OPENING_19_MM, 196
openLocalStorage, 394
orderedDevices, 420
orderTracks, 315
orthogonalize, 321
OUTLET_TYPES, 290
OutletGroup, 291
OutletGroupSchema, 290
outlinedGlyphs, 336
OutsideConnection, 320
OutsideProblem, 320
Overlay, 499
OverlayProps, 499

Page, 182
PageCanvas, 345
pageCommands, 431
PageContent, 345
PageContentProps, 344
PageFormat, 181
pageFormats, 410
PageFormatSchema, 181
pageOfTarget, 257
PageParts, 344
PageProperties, 505
PageRouting, 322
PageSchema, 181
pageSize, 253
pagesToExport, 408
pageSvg, 417
PageTabs, 482
PageView, 344
PageViewProps, 343
pairCrossings, 314
pairedPort, 213
PANEL_10_MM, 196
PANEL_19_MM, 196
PAPER_SIZES, 252
PARAM_FIELDS, 522
PARAM_SCHEMAS, 295
paramConnectors, 296
paramCustomConnectors, 297
ParamField, 521
Params, 295
ParamsInput, 295
paramsIssues, 523
paramsProblems, 283
parseCsv, 370
parseInline, 268
parseLibraryDocument, 187
parseMarkdown, 268
parseParams, 296
parseProject, 186
ParseResult, 185
parseShortcut, 438
parseSiteMarker, 265
parseUnicodeRange, 337
PartOption, 519
pasteAnnotations, 446
pasteDevices, 449
PasteOptions, 449
PATCH_MODULES, 293
PatchChain, 362
patchChain, 363
PatchModule, 293
patchPanelMachine, 362
PatchPanelSchema, 293
PATH_COLOR, 499
pathElements, 213
PathHighlight, 499
PathHop, 212
PBKDF2_ITERATIONS, 271
PdfDependencies, 409
PdfExportRequest, 408
pdfFontStyle, 408
PduSchema, 291
physicalHalfSide, 192
pickFile, 386
PixelSize, 412
placeDevice, 442
PlaceDeviceDialog, 513
PlacedLabel, 404
placeExternal, 449
Placement, 177
placementAt, 204
placementBox, 251
PlacementCandidate, 188
PlacementMessage, 190
PlacementProblem, 189
PlacementResult, 190
PlacementSchema, 177
placeNewDevice, 448
PlaceOptions, 442
placeWithConfirmation, 491

planImport, 237, 376
PLINTH_MM, 197
PLUG_TYPES, 291
PngBackground, 414
pngDimensions, 413
pngDpi, 413
pngPixelSize, 413
PngResolution, 412
Point, 169, 496
pointsBox, 225
PointSchema, 169
Port, 173
PORT_COLORS, 346
PORT_GROUP_PARAMS, 296
PORT_SHAPES, 357
PORT_SIMPLIFY_BELOW_PX_PER_MM, 358
PORT_SIMPLIFY_MIN_VISIBLE_MM2, 358
PORT_STATE_COLORS, 499
portAnchor, 324
portAt, 495
portBox, 250
portCellName, 421
PortDetail, 358
portDetailAt, 359
portDisplayName, 165
PortEntry, 162
PortExitSchema, 173
PortGridForm, 525
PortGridParams, 243
PortGroup, 288
PortGroupSchema, 288
PortInput, 156
portKey, 162
portLabel, 209
PortLook, 357
portLook, 357
portLooks, 358
portNameProblems, 283
PortOccupancy, 500
PortOccupancyLegend, 500
portOf, 363
PortOnPage, 249
portOnPage, 250
PortRect, 366
portRects, 366
PortRef, 178
PortRefSchema, 178
ports, 361
PortSchema, 173
PortShape, 346
PortShapeKey, 357
PortShapeProps, 346
portSimplifyThreshold, 358
PortSpec, 281
PortsTable, 525
PortState, 165
portStatesOnPage, 165
PortTooltip, 501
positionSources, 245
preferenceCommands, 430
Preferences, 488
PreferencesDialog, 516
prepareImage, 390
PRESET_SIZES, 231
PresetTexts, 230
presetTexts, 493
Project, 184
projectCommands, 432
projectFromExisting, 166
ProjectIndex, 162
ProjectMeta, 183
ProjectMetaSchema, 183
ProjectPropertiesDialog, 512
ProjectSchema, 184
PropertiesPanel, 505
proposedDescription, 239
proposeLinkType, 211
proposeLinkTypeForPorts, 211
proposeMachineName, 283
PSU_TYPES, 289
PurgeDialog, 517
purgeLibrary, 456
PurgeResult, 167
purgeUnusedMachines, 167

QuarterTurn, 180
QuarterTurnSchema, 179
QUICK_START_URL, 267

Rack, 172
RACK_MAX_HEIGHT_U, 171
rackCommands, 432
RackContents, 348
RackContentsProps, 348
rackDevices, 504
rackDrawingOrder, 347
rackFitScale, 220
rackFreeSpace, 195
RackGeometry, 198
rackGeometry, 199
rackLogoLayout, 356
RackModel, 171
RackModelSchema, 171
rackOccupation, 194
rackPageLayout, 222

RackPlacement, 177
RackPlacementSchema, 176
RackPreview, 524
RackProperties, 506
RackResizeResult, 193
RackSchema, 171
rackSideLabels, 255
rackToPage, 247
RackView, 348
rackViewAt, 204
rackViewFrame, 247
RackViewProps, 348
RackViewRouting, 327
rackViewsLayout, 221
RackWindow, 202
rackWindow, 202
RangeProject, 365
rangeProject, 365
rasterizeSvg, 414
rasterSize, 411
RATIO_TOLERANCE, 242
RatioCheck, 242
readCsv, 371
ReadError, 381
ReadErrorCode, 381
readFile, 386
readLibraryArchive, 383
ReadLibraryResult, 382
ReadLimits, 380
ReadOptions, 382
readProjectArchive, 382
ReadProjectResult, 381
readSfnt, 335
readSpreadsheet, 371
ReadWorkbook, 369
readXlsx, 371
reattachRemovedViews, 302
RECOMMENDED_WIDTH_PX, 242
Rect, 312
referenceLikeProject, 365
refNumber, 211
regenerateEdited, 279
regenerateMachine, 285
registerSymbolFaces, 330
reimportDefaultModels, 457
ReimportDialog, 517
ReleaseNotesDialog, 516
remapLibraryImages, 391
remapParamConnectors, 297
remapProjectImages, 391
remapRouteViews, 303
removeAnnotations, 446
removeConnectorType, 458
removeDevices, 443
removeInstancePort, 468
removeLinks, 444
removeLinkType, 459
removeModel, 456
removePage, 465
removePoint, 317
removeRack, 441
removeRevision, 469
removeTitleBlockField, 470
removeViews, 466
RenameRackDialog, 513
render/AnnotationsLayer (module), 338
render/DeviceFace (module), 339
render/ExternalDrawing (module), 340
render/images (module), 353
render/LabelSheet (module), 341
render/LinksLayer (module), 341
render/logo (module), 354
render/PageView (module), 343
render/PortShape (module), 345
render/portShapes (module), 357
render/RackContents (module), 346
render/RackView (module), 348
render/StandardDecoration (module), 349
render/TextBlock (module), 350
render/TextMeasureContext (module), 351
render/TitleBlock (module), 352
RenderOptions, 417
Renvoi, 321
RENVOI_FONT, 342
RENVOI_STEP, 323
renvoiText, 323
replaceAnnotationImage, 493
replaceAnnotations, 446
resetRoutes, 473
resizeTable, 232
resolvePoints, 301
resolvePort, 164
restoreDraft, 385
RestoreResult, 384
revealHit, 515
Revision, 183
RevisionSchema, 183
RevisionsDialog, 512
RevisionTable, 353
revisionTableHeight, 222
RevisionTableProps, 353
RKDG_EXTENSION, 378
RKDG_MIME_TYPE, 378
RKDGLIB_EXTENSION, 378

ROOF_MM, 197
rotateAnnotation, 227
rotationTransform, 226
routeCommands, 437
RouteHandles, 500
routeOutside, 320
routePage, 323
routePageCached, 323
routePageRacks, 328
routeRackView, 327
RouterSchema, 289
routing/attach (module), 300
routing/avoid (module), 303
routing/bridges (module), 304
routing/bundles (module), 307
routing/external (module), 310
routing/hidden (module), 311
routing/lanes (module), 312
routing/manual (module), 315
routing/markers (module), 318
routing/outside (module), 320
routing/page (module), 321
routing/straight (module), 324
routing/trays (module), 325
RoutingDialog, 515

safeFileName, 399
SampleProject, 362
sampleProject, 362
sanitizeSvg, 395
SavedFile, 387
saveFile, 387
saveMachine, 462
SaveOptions, 387
SchemaPanel, 505
schemaProblems, 298
SCREEN_PARTS, 344
screenSizeMm, 282
SearchDialog, 515
SearchHit, 256
searchProject, 256
SearchResult, 256
SearchTarget, 256
searchWords, 239
Section, 511
seededRandom, 364
segmentCrossesBox, 311
segmentHitsRect, 366
SegmentRef, 305
selectCanRedo, 475
selectCanUndo, 475
selectDevice, 476
selectedAnnotations, 423
selectedDevices, 426
selectedRackId, 425
selectedRoute, 500
SelectField, 510
selectInRectangle, 251
selection, 425
SELECTION_COLOR, 497
selectionBoxes, 497
selectionExists, 485
SelectionItem, 484
selectionKey, 485
SelectionKind, 484
selectIsDirty, 475
selectLink, 476
selectMeta, 475
selectModel, 476
selectPage, 477
selectProject, 475
selectRack, 476
serializeProject, 379
ServerSchema, 290
Session, 263
SessionContext, 490
SessionMessage, 263
SessionOptions, 263
SessionState, 263
setAnnotationBounds, 227
setAvoidWasmUrl, 303
setBundled, 474
setDeviceMachine, 463
setLibraryInfo, 460
setManualRoute, 473
setProjectTray, 474
setRackTray, 475
Sfnt, 335
sha256Hex, 388
SHAPE_BUFFER_MM, 320
SHAPE_DRAWINGS, 346
SHAPE_SIZES, 359
Sheet, 368
SheetCell, 368
sheetKind, 376
sheetName, 415
ShelfSchema, 293
ShortcutsDialog, 516
SIDE_LABELS, 254
SIDE_LABELS_MM, 221
SideGeometry, 198
SideLabel, 255
sideLabelsWidth, 255
simplify, 327
SITE_MARKER, 265

SiteLinks, 265
siteLinks, 266
siteMarkerTag, 266
snapMove, 219
SnapOptions, 219
SnapResult, 219
snapTargets, 220
snapToGrid, 218
snapValue, 219
sourceOrigin, 235
Span, 198, 267
splitBundle, 308
SpreadsheetRequest, 415
squareUp, 316
StandardDecoration, 350
StandardDecorationProps, 350
startDraftSaver, 384
startOffline, 262
StorageSchema, 290
store/actions (module), 440
store/annotationActions (module), 445
store/deviceActions (module), 448
store/editor (module), 450
store/exampleProject (module), 453
store/importActions (module), 454
store/index (module), 454
store/libraryActions (module), 454
store/machineActions (module), 460
store/pageActions (module), 463
store/portActions (module), 467
store/projectActions (module), 468
store/rackActions (module), 470
store/react (module), 472
store/routeActions (module), 473
store/routingActions (module), 474
store/selectors (module), 475
straightLinks, 324
subsetFont, 336
SubsetOptions, 335
suggestedFileName, 388
supportsFileSystemAccess, 386
SVG_RASTER_WIDTH, 411
svgAspect, 395
SvgDrawing, 417
svgFile, 419
SvgSanitizer, 394
SwitchSchema, 289
SYMBOLS_RANGES, 330
SYMBOLS_SAMPLE, 330
SYMBOLS_UNICODE_RANGE, 330

t, 269
TAB_ICON_PATHS, 503
TAB_ICON_SIZE, 502
TAB_ICON_STROKE, 502
Table, 420
TABLE_IDS, 419
tableColumns, 232
TableId, 419
targetBoxOnPage, 257
TECHNICAL_MANUAL_URL, 428
templateProblem, 404
templateTables, 377
test-setup (module), 299
testing/fixtures (module), 360
testing/fonts (module), 363
testing/generatedRack (module), 364
testing/rangeProject (module), 365
testing/referenceProject (module), 365
testing/routingChecks (module), 366
text/fontFiles (module), 329
text/fonts (module), 331
text/index (module), 332
text/layout (module), 332
text/measure (module), 334
text/sfnt (module), 335
text/unicodeRange (module), 336
TextAlign, 332
textArea, 309
TextBlock, 351
TextBlockProps, 351
TextField, 509
TextLayout, 333
textLength, 309
TextMeasureContext, 351
TextPlace, 308
textUsage, 400
TITLE_BLOCK_HEIGHT, 253
TitleBlock, 182, 353
TitleBlockCell, 258
TitleBlockDialog, 512
TitleBlockField, 182
titleBlockFieldLabel, 352
TitleBlockFieldSchema, 182
TitleBlockLayout, 259
titleBlockLayout, 259
TitleBlockProps, 352
TitleBlockRow, 258
titleBlockRow, 258
TitleBlockSchema, 182
Toast, 486
Toasts, 481
toBase64, 398
toCsv, 415
today, 155

TOOLBAR, 483
Toolbar, 482
tools/build-doc (module), 141
tools/build-example-library (module), 141
tools/build-json-schema (module), 142
tools/build-site (module), 142
tools/build-tech-doc (module), 142
tools/check-doc (module), 143
tools/check-neutral (module), 143
tools/check-site (module), 143
tools/check-tech-doc (module), 144
tools/json-schema (module), 144
tools/libavoid-no-eval (module), 145
tools/manual (module), 145
tools/manual-commands (module), 146
tools/neutral (module), 146
tools/publish-libraries (module), 147
tools/site (module), 147
tools/tech-doc (module), 148
tools/tech-doc-files (module), 149
TooltipState, 501
toXlsx, 415
translateAnnotation, 227
transparentBackground, 350
TRAY_MIN_MM, 198
TrayCable, 314
traySide, 325
TrayUsage, 326
trueTypeUrl, 330
tuckIn, 315
typesOnlyProject, 360

U_MM, 196
uDepthAt, 208
UDepthInfo, 207
UFreeSpace, 195
Ui, 489
ui/annotationFlow (module), 492
ui/App (module), 478
ui/canvas/Canvas (module), 495
ui/canvas/coords (module), 496
ui/canvas/DepthTooltip (module), 501
ui/canvas/Highlights (module), 499
ui/canvas/Overlay (module), 497
ui/canvas/PortTooltip (module), 501
ui/canvas/RouteHandles (module), 500
ui/CommandButton (module), 479
ui/ContextMenu (module), 479
ui/dialogs/DialogHost (module), 511
ui/dialogs/ExportLibraryDialog (module), 518
ui/dialogs/HelpDialogs (module), 516
ui/dialogs/ImportModelsDialog (module), 517
ui/dialogs/ImportSpreadsheetDialog (module), 518
ui/dialogs/LibraryDialogs (module), 517
ui/dialogs/LinkDialog (module), 513
ui/dialogs/LinksTableDialog (module), 514
ui/dialogs/LinkVisibilityDialog (module), 514
ui/dialogs/PlaceDeviceDialog (module), 513
ui/dialogs/PreferencesDialog (module), 515
ui/dialogs/ProjectDialogs (module), 511
ui/dialogs/RackDialogs (module), 513
ui/dialogs/RoutingDialog (module), 514
ui/dialogs/SearchDialog (module), 515
ui/DraftDialog (module), 479
ui/ErrorDialog (module), 480
ui/exports/download (module), 519
ui/exports/ExportImageDialog (module), 519
ui/exports/ExportLabelsDialog (module), 520
ui/exports/ExportPartsFields (module), 519
ui/exports/ExportPdfDialog (module), 520
ui/exports/ExportSpreadsheetDialog (module), 520
ui/fields (module), 509
ui/hooks (module), 489
ui/imageImport (module), 494
ui/library/LibrarySheet (module), 521
ui/linkFlow (module), 492
ui/machine/MachineEditorDialog (module), 524
ui/machine/MachineParamsForm (module), 523
ui/machine/paramFields (module), 521
ui/MenuBar (module), 480
ui/menus (module), 482
ui/Messages (module), 481
ui/Modal (module), 481
ui/modelEditor/FaceEditor (module), 524
ui/modelEditor/PortGridForm (module), 525
ui/modelEditor/PortsTable (module), 525
ui/modelEditor/RackPreview (module), 524
ui/PageTabs (module), 482

ui/panels/ChecksPanel (module), 504
ui/panels/ExplorerPanel (module), 504
ui/panels/LeftPanel (module), 502
ui/panels/LibraryPanel (module), 503
ui/panels/SchemaPanel (module), 505
ui/panels/tabIcons (module), 502
ui/placementFlow (module), 491
ui/properties/AnnotationProperties (module), 508
ui/properties/DeviceProperties (module), 506
ui/properties/InstancePorts (module), 507
ui/properties/LinkProperties (module), 507
ui/properties/PageProperties (module), 505
ui/properties/PropertiesPanel (module), 505
ui/properties/RackProperties (module), 506
ui/properties/ViewProperties (module), 506
ui/state (module), 484
ui/Toolbar (module), 482
ui/UpdateBanner (module), 482
ui/useCommandShortcuts (module), 483
UiContext, 490
UiState, 487
unionBox, 217
uniqueDeviceName, 206
UNKNOWN_PORT_LOOK, 358
unplacePorts, 246
uNumber, 200
UOccupation, 194
updateAnnotation, 445
UpdateBanner, 482
updateConnectorType, 458
updateDevice, 443
updateInstancePort, 467
updateLink, 444
updateLinkType, 459
updateModel, 455
updatePage, 464
updateProjectMeta, 440
updateRack, 441
updateReload, 262
updateRevision, 468
updateTitleBlockField, 469
updateView, 466
upperRowPorts, 340
UpsSchema, 292
URange, 201
USABLE_DEPTH_MARGIN_MM, 155
useAnomalyCount, 504
useAvoid, 342
useCommand, 491
useCommandContext, 491
useCommandShortcuts, 484
useDraft, 509
useEditor, 472
useEditorState, 472
useImageUrl, 354
useMeasureText, 352
usePortLooks, 347
USER_DOCUMENTS, 267
UserDocument, 267
useSession, 490
usesMarkers, 319
useStateVersion, 491
useUi, 490
useUiApi, 490
uSpan, 199

ValidationIssue, 185
variantDevices, 461
verticalClear, 314
VIDEO_INPUTS, 294
View, 179
VIEW_EXPORT_MARGIN, 417
VIEW_GAP_MM, 220
VIEW_TITLE, 343
VIEW_TITLE_GAP, 246
VIEW_TITLE_SIZE, 246
viewBounds, 248
viewCommands, 439
viewExportBox, 418
viewFace, 246
viewFootprint, 222
ViewInput, 159
viewOnlyPage, 418
ViewProperties, 506
ViewSchema, 179
viewSvg, 418

waitForFonts, 332
WEB_FONT_FILES, 329
WebFontFile, 329
withColumns, 421
withComplements, 236
withConnectorTypes, 277
WithKind, 152
withMachine, 276
withPngDpi, 414
withSiteMarker, 266

WorkstationSchema, 295
WritableFileHandle, 385
writeLibraryArchive, 380
WriteLibraryOptions, 380
writeProjectArchive, 379
WriteProjectOptions, 379
WrongPasswordError, 272

ZERO_U_SLOT_MAX_MM, 197
ZERO_U_SLOT_MIN_MM, 197
zeroUDeviceBox, 201
zeroULengthU, 154
ZeroUPlacement, 177
ZeroUPlacementSchema, 176
zeroUSlotBox, 200
zeroUSlotCount, 191
ZIGZAG_DEPTH_MM, 201
ZIGZAG_TOOTH_MM, 201
ZOOM_ACTUAL, 486
ZOOM_MAX, 486
ZOOM_MIN, 486
ZOOM_STEP, 486